LeetMastery

Study-Order Edition

Python Only · Priority-Grouped · Difficulty-First · 2×1 Landscape

---

331 questions · 9 rounds · Interview Approach
High Easy → High Med → High Hard → Mid Easy → Mid Med → Mid Hard → Low Easy → Low Med → Low Hard

· Contents

Contents

★ = LeetCode Premium (Premium 98 list)

- [ ] #33 Search in Rotated Sorted Array ↗
- [ ] #34 Find First and Last Position of Element in Sorted Array ↗
- [ ] #153 Find Minimum in Rotated Sorted Array ↗
- [ ] #300 Longest Increasing Subsequence ↗
- [ ] #362 Design Hit Counter ↗
- [ ] #528 Random Pick with Weight ↗
- [ ] #852 Peak Index in a Mountain Array ↗
- [ ] #981 Time Based Key-Value Store ↗
- [ ] #1011 Capacity to Ship Packages Within D Days ↗
- [ ] ⋆ #1060 Missing Element in Sorted Array ↗
- [ ] #1062 Longest Repeating Substring ↗
- [ ] ⋆ #1533 Find the Index of the Large Integer ↗

Matrix (14) ↗

- [ ] #36 Valid Sudoku ↗
- [ ] #48 Rotate Image ↗
- [ ] #54 Spiral Matrix ↗
- [ ] #59 Spiral Matrix II ↗
- [ ] #64 Minimum Path Sum ↗
- [ ] #73 Set Matrix Zeroes ↗
- [ ] #74 Search a 2D Matrix ↗
- [ ] #221 Maximal Square ↗
- [ ] ⋆ #311 Sparse Matrix Multiplication ↗
- [ ] #498 Diagonal Traverse ↗
- [ ] ⋆ #531 Lonely Pixel I ↗
- [ ] ⋆ #723 Candy Crush ↗
- [ ] #835 Image Overlap ↗
- [ ] ⋆ #1198 Find Smallest Common Element in All Rows ↗

Trees & BST (24) ↗

- [ ] #98 Validate Binary Search Tree ↗
- [ ] #102 Binary Tree Level Order Traversal ↗
- [ ] #103 Binary Tree Zigzag Level Order Traversal ↗
- [ ] #105 Construct Binary Tree from Preorder and Inorder Traversal ↗
- [ ] #199 Binary Tree Right Side View ↗
- [ ] #230 Kth Smallest Element in a BST ↗
- [ ] #235 Lowest Common Ancestor of a Binary Search Tree ↗
- [ ] #236 Lowest Common Ancestor of a Binary Tree ↗
- [ ] ⋆ #250 Count Univalue Subtrees ↗
- [ ] #285 Inorder Successor in BST ↗
- [ ] ⋆ #298 Binary Tree Longest Consecutive Sequence ↗
- [ ] ⋆ #314 Binary Tree Vertical Order Traversal ↗
- [ ] ⋆ #333 Largest BST Subtree ↗
- [ ] ⋆ #366 Find Leaves of Binary Tree ↗
- [ ] #437 Path Sum III ↗
- [ ] ⋆ #545 Boundary of Binary Tree ↗
- [ ] ⋆ #549 Binary Tree Longest Consecutive Sequence II ↗
- [ ] ⋆ #582 Kill Process ↗
- [ ] #662 Maximum Width of Binary Tree ↗
- [ ] #863 All Nodes Distance K in Binary Tree ↗
- [ ] ⋆ #1120 Maximum Average Subtree ↗
- [ ] ⋆ #1490 Clone N-ary Tree ↗
- [ ] ⋆ #1522 Diameter of N-Ary Tree ↗

- [ ] ⋆ #1650 Lowest Common Ancestor of a Binary Tree III ↗

DFS (16) ↗

- [ ] #130 Surrounded Regions ↗
- [ ] #133 Clone Graph ↗
- [ ] #200 Number of Islands ↗
- [ ] #207 Course Schedule ↗
- [ ] #210 Course Schedule II ↗
- [ ] ⋆ #261 Graph Valid Tree ↗
- [ ] #310 Minimum Height Trees ↗
- [ ] ⋆ #323 Number of Connected Components in an Undirected Graph ↗
- [ ] #417 Pacific Atlantic Water Flow ↗
- [ ] ⋆ #490 The Maze ↗
- [ ] ⋆ #694 Number of Distinct Islands ↗
- [ ] #695 Max Area of Island ↗
- [ ] #721 Accounts Merge ↗
- [ ] #785 Is Graph Bipartite? ↗
- [ ] ⋆ #1236 Web Crawler ↗
- [ ] #1242 Web Crawler Multithreaded ↗

Graphs (3) ↗

- [ ] #128 Longest Consecutive Sequence ↗
- [ ] ⋆ #277 Find the Celebrity ↗
- [ ] ⋆ #1136 Parallel Courses ↗

BFS (6) ↗

- [ ] ⋆ #286 Walls and Gates ↗
- [ ] #322 Coin Change ↗
- [ ] #542 01 Matrix ↗
- [ ] #994 Rotting Oranges ↗
- [ ] ⋆ #1197 Minimum Knight Moves ↗
- [ ] #1730 Shortest Path to Get Food ↗

Round 3 | High | H Hard (23) ↗

Arrays & Hashing (1) ↗

- [ ] #41 First Missing Positive ↗

Sliding Window (1) ↗

- [ ] #76 Minimum Window Substring ↗

Binary Search (7) ↗

- [ ] #4 Median of Two Sorted Arrays ↗
- [ ] #315 Count of Smaller Numbers After Self ↗
- [ ] #410 Split Array Largest Sum ↗
- [ ] #668 Kth Smallest Number in Multiplication Table ↗
- [ ] #710 Random Pick with Blacklist ↗
- [ ] #774 Minimize Max Distance to Gas Station ↗
- [ ] #1235 Maximum Profit in Job Scheduling ↗

Matrix (1) ↗

- [ ] #1074 Number of Submatrices That Sum to Target ↗

Trees & BST (2) ↗

- [ ] #124 Binary Tree Maximum Path Sum ↗
- [ ] #297 Serialize and Deserialize Binary Tree ↗

DFS (5) ↗

- [ ] ⋆ #269 Alien Dictionary ↗
- [ ] #329 Longest Increasing Path in a Matrix ↗
- [ ] #685 Redundant Connection II ↗
- [ ] #1036 Escape a Large Maze ↗

- [ ] #1192 Critical Connections in a Network

Graphs (2)

- [ ] ⋆ #305 Number of Islands II
- [ ] #1153 String Transforms Into Another String

BFS (4)

- [ ] #127 Word Ladder
- [ ] ⋆ #317 Shortest Distance from All Buildings
- [ ] #773 Sliding Puzzle
- [ ] #815 Bus Routes

Round 4 | Mid | E Easy (12)

Linked List (7)

- [ ] #21 Merge Two Sorted Lists
- [ ] #141 Linked List Cycle
- [ ] #206 Reverse Linked List
- [ ] #234 Palindrome Linked List
- [ ] #706 Design HashMap
- [ ] #876 Middle of the Linked List
- [ ] ⋆ #1474 Delete N Nodes After M Nodes of Linked List

Stack (3)

- [ ] #20 Valid Parentheses
- [ ] #232 Implement Queue using Stacks
- [ ] #844 Backspace String Compare

Trie (1)

- [ ] #14 Longest Common Prefix

Greedy (1)

- [ ] #409 Longest Palindrome

Round 5 | Mid | M Medium (56)

Linked List (13)

- [ ] #2 Add Two Numbers
- [ ] #19 Remove Nth Node From End of List
- [ ] #24 Swap Nodes in Pairs
- [ ] #61 Rotate List
- [ ] #146 LRU Cache
- [ ] #148 Sort List
- [ ] #328 Odd Even Linked List
- [ ] ⋆ #369 Plus One Linked List
- [ ] ⋆ #430 Flatten a Multilevel Doubly Linked List
- [ ] #622 Design Circular Queue
- [ ] #707 Design Linked List
- [ ] ⋆ #708 Insert into a Sorted Circular Linked List
- [ ] #1171 Remove Zero Sum Consecutive Nodes from Linked List

Stack (14)

- [ ] #143 Reorder List
- [ ] #150 Evaluate Reverse Polish Notation
- [ ] #155 Min Stack
- [ ] #173 Binary Search Tree Iterator
- [ ] #227 Basic Calculator II
- [ ] ⋆ #255 Verify Preorder Sequence in Binary Search Tree
- [ ] #394 Decode String
- [ ] ⋆ #439 Ternary Expression Parser
- [ ] ⋆ #484 Find Permutation

- [ ] #735 Asteroid Collision
- [ ] #739 Daily Temperatures
- [ ] #962 Maximum Width Ramp
- [ ] ⋆ #1214 Two Sum BSTs
- [ ] ⋆ #1762 Buildings With an Ocean View

Heap (11)

- [ ] #215 Kth Largest Element in an Array
- [ ] ⋆ #253 Meeting Rooms II
- [ ] #347 Top K Frequent Elements
- [ ] ⋆ #505 The Maze II
- [ ] #621 Task Scheduler
- [ ] #658 Find K Closest Elements
- [ ] #787 Cheapest Flights Within K Stops
- [ ] #973 K Closest Points to Origin
- [ ] ⋆ #1057 Campus Bikes
- [ ] ⋆ #1167 Minimum Cost to Connect Sticks
- [ ] #1424 Diagonal Traverse II

Trie (7)

- [ ] #139 Word Break
- [ ] #208 Implement Trie (Prefix Tree)
- [ ] #211 Design Add and Search Words Data Structure
- [ ] ⋆ #616 Add Bold Tag in String
- [ ] #692 Top K Frequent Words
- [ ] #720 Longest Word in Dictionary
- [ ] #1166 Design File System

Backtracking (6)

- [ ] #17 Letter Combinations of a Phone Number
- [ ] #22 Generate Parentheses
- [ ] #39 Combination Sum
- [ ] #46 Permutations
- [ ] #79 Word Search
- [ ] #113 Path Sum II

Greedy (5)

- [ ] #134 Gas Station
- [ ] #179 Largest Number
- [ ] ⋆ #280 Wiggle Sort
- [ ] #954 Array of Doubled Pairs
- [ ] #2131 Longest Palindrome by Concatenating Two Letter Words

Round 6 | Mid | H Hard (28)

Linked List (3)

- [ ] #25 Reverse Nodes in k-Group
- [ ] #432 All O'one Data Structure
- [ ] #460 LFU Cache

Stack (8)

- [ ] #32 Longest Valid Parentheses
- [ ] #42 Trapping Rain Water
- [ ] #84 Largest Rectangle in Histogram
- [ ] #224 Basic Calculator
- [ ] #239 Sliding Window Maximum
- [ ] #716 Max Stack
- [ ] ⋆ #772 Basic Calculator III

- [ ] #895 Maximum Frequency Stack

Heap (9)

- [ ] #23 Merge k Sorted Lists
- [ ] #218 The Skyline Problem
- [ ] ⋆ #272 Closest Binary Search Tree Value II
- [ ] #295 Find Median from Data Stream
- [ ] ⋆ #358 Rearrange String k Distance Apart
- [ ] ⋆ #499 The Maze III
- [ ] #632 Smallest Range Covering Elements from K Lists
- [ ] #759 Employee Free Time
- [ ] #1425 Constrained Subsequence Sum

Trie (4)

- [ ] #212 Word Search II
- [ ] #336 Palindrome Pairs
- [ ] ⋆ #588 Design In-Memory File System
- [ ] ⋆ #642 Design Search Autocomplete System

Backtracking (4)

- [ ] #37 Sudoku Solver
- [ ] #51 N-Queens
- [ ] #126 Word Ladder II
- [ ] #996 Number of Squareful Arrays

Round 7 | Low | E Easy (14)

Dynamic Programming (2)

- [ ] #70 Climbing Stairs
- [ ] #121 Best Time to Buy and Sell Stock

Bit Manipulation (6)

- [ ] #67 Add Binary
- [ ] #136 Single Number
- [ ] #190 Reverse Bits
- [ ] #191 Number of 1 Bits
- [ ] #268 Missing Number
- [ ] #338 Counting Bits

Math (6)

- [ ] #9 Palindrome Number
- [ ] #13 Roman to Integer
- [ ] #504 Base 7
- [ ] ⋆ #1056 Confusing Number
- [ ] ⋆ #1228 Missing Number in Arithmetic Progression
- [ ] ⋆ #1427 Perform String Shifts

Round 8 | Low | M Medium (32)

Dynamic Programming (16)

- [ ] #5 Longest Palindromic Substring
- [ ] #53 Maximum Subarray
- [ ] #55 Jump Game
- [ ] #62 Unique Paths
- [ ] #72 Edit Distance
- [ ] #91 Decode Ways
- [ ] #152 Maximum Product Subarray
- [ ] #198 House Robber
- [ ] ⋆ #256 Paint House
- [ ] #309 Best Time to Buy and Sell Stock with Cooldown

- [ ] #377 Combination Sum IV
- [ ] #416 Partition Equal Subset Sum
- [ ] #435 Non-overlapping Intervals
- [ ] #516 Longest Palindromic Subsequence
- [ ] #576 Out of Boundary Paths
- [ ] #1143 Longest Common Subsequence

Bit Manipulation (4)

- [ ] #78 Subsets
- [ ] #90 Subsets II
- [ ] #287 Find the Duplicate Number
- [ ] ⋆ #1506 Find Root of N-Ary Tree

Math (5)

- [ ] #7 Reverse Integer
- [ ] #50 Pow(x, n)
- [ ] #380 Insert Delete GetRandom O(1)
- [ ] #1017 Convert to Base -2
- [ ] #3160 Find the Number of Distinct Colors Among the Balls

JavaScript (7)

- [ ] ⋆ #2627 Debounce
- [ ] ⋆ #2628 JSON Deep Equal
- [ ] ⋆ #2630 Memoize
- [ ] ⋆ #2633 Convert Object to JSON String
- [ ] ⋆ #2636 Promise Pool
- [ ] ⋆ #2675 Array of Objects to Matrix
- [ ] ⋆ #2700 Differences Between Two Objects

Round 9 | Low | H Hard (7)

Dynamic Programming (6)

- [ ] #10 Regular Expression Matching
- [ ] #115 Distinct Subsequences
- [ ] #123 Best Time to Buy and Sell Stock III
- [ ] #132 Palindrome Partitioning II
- [ ] #312 Burst Balloons
- [ ] #1000 Minimum Cost to Merge Stones

Math (1)

- [ ] #68 Text Justification

Round 1

High Priority · Easy
43 questions · 8 patterns

Arrays & Hashing

Round 1 · High · Easy · 6 questions

Arrays & Hashing

#1 Two Sum

EAS

Open on LeetCode

Array · Hash Table

Lists: Grind 169

Problem

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.
You may assume that each input would have exactly one solution, and you may not use the same element twice.
You can return the answer in any order.

Example 1:

```
Input: nums = [2,7,11,15], target = 9
Output: [0,1]
Explanation: Because nums[0] + nums[1] == 9, we return [0, 1].
```

Example 2:

```
Input: nums = [3,2,4], target = 6
Output: [1,2]
```

Example 3:

```
Input: nums = [3,3], target = 6
Output: [0,1]
```

Constraints:

- 2 <= nums.length <= 104
- -109 <= nums[i] <= 109
- -109 <= target <= 109
- Only one valid answer exists.

Follow-up: Can you come up with an algorithm that is less than O(n2) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We're given an array of integers and a target value,
# and we need to return the indices of the two numbers
# that add up to the target — not the values, the indices. Right?"
#
# "A few quick questions:
# Can the same element be used twice? I'm guessing no.
# Is there always exactly one valid answer?
# Can the array be empty or have only one element?
# Are there negative numbers or duplicates?"
#
# "Let me walk the example: nums = [2, 7, 11, 15], target = 9.
# 2 + 7 = 9, so we return [0, 1]. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every pair (i, j) where j > i, check if nums[i] + nums[j] equals target.
# Time: O(n²) from the nested loops. Space: O(1), no extra storage.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is searching the rest of the array for a complement each time.
# I notice we're repeatedly asking: does the number I need exist?
# A hash map answers that in O(1).
#
# Walk forward through the array. For each number, compute complement = target - num.
# If complement is already in the map, return both indices.
# Otherwise, store the current number and index and keep going.
#
# Pseudocode:
# seen = {}
# for index, num in nums:
# complement = target - num
# if complement in seen: return [seen[complement], index]
# seen[num] = index
#
# Time: O(n). Space: O(n). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[2,7,11,15], target=9
# index=0: complement=7, not in seen. seen={2:0}
# index=1: complement=2, found at 0. return [0,1] ✓
```



```
#
# nums=[3,2,4], target=6
# index=0: complement=3, not in seen. seen={3:0}
# index=1: complement=4, not in seen. seen={3:0,2:1}
# index=2: complement=2, found at 1. return [1,2] ✓
#
# nums=[3,3], target=6
# index=0: complement=3, not in seen. seen={3:0}
# index=1: complement=3, found at 0. return [0,1] ✓
#
# "No bugs. Holds across all cases."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one pass. Space O(n) — worst case every element enters the map.
# Trade-off: O(1) space would require O(n²) time — no free lunch here.
# If the array were sorted I'd use two pointers: O(n) time, O(1) space.
# But we need original indices and the input is unsorted, so hash map wins.
# Given more time I'd ask: what if there are multiple valid pairs?
# We'd collect all of them rather than returning on the first hit."
```

Arrays & Hashing

★ #163 Missing Ranges ★ EAS

Open on LeetCode

Array

Lists: ★ Premium | Premium 98

Problem

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1:

```
Input: nums = [0,1,3,50,75], lower = 0, upper = 99
Output: [[2,2],[4,49],[51,74],[76,99]]
Explanation: The ranges are:
[2,2]
[4,49]
[51,74]
[76,99]
```

Example 2:

```
Input: nums = [-1], lower = -1, upper = -1
Output: []
Explanation: There are no missing ranges since there are no missing numbers.
```

Constraints:

- -109 <= lower <= upper <= 109
- 0 <= nums.length <= 100
- lower <= nums[i] <= upper
- All the values of nums are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We have a sorted array of unique integers, and an inclusive range [lower, upper].
# We need to return the list of ranges that cover every number
# inside [lower, upper] that does NOT appear in nums. Right?"
#
# "A few quick questions:
# Can nums be empty? If so I assume the whole [lower, upper] is one missing range.
# Are all elements in nums guaranteed to be within [lower, upper]? Good.
# The output ranges should be as compact as possible — so [2,2] not just [2]?"
#
# "Let me walk the example:
# nums=[0,1,3,50,75], lower=0, upper=99.
# 0 and 1 are present so no gap at the start.
# 3 is present but 2 is missing → [2,2].
# 50 is present but 4–49 are missing → [4,49].
# 75 is present but 51–74 are missing → [51,74].
# 76–99 are all missing → [76,99].
# Output: [[2,2],[4,49],[51,74],[76,99]]. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# The simplest approach: build a set from nums for O(1) lookups,
# then walk every integer from lower to upper.
# Whenever I hit a missing number, I extend the current gap range.
# When I hit a number that IS in the set, I close the gap and record it.
#
# Time: O(upper - lower) which can be up to 2 billion — way too slow.
# Space: O(n) for the set.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "The brute force iterates over the entire numerical range — up to 2 billion steps.
# But nums has at most 100 elements. The gaps I care about
# only exist between consecutive numbers in nums, before the first, and after the last.
# So I only need to check those boundaries — that's O(n), not O(range).
#
# I think of nums as a set of walls. The missing ranges are the gaps between walls.
# There are at most n+1 gaps: one before nums[0], one between each pair, one after nums[-1].
#
# Pseudocode:
# if nums is empty: return [[lower, upper]]
# if lower < nums[0]: add [lower, nums[0]-1]
# for each consecutive pair (prev, curr) in nums:
# if prev+1 < curr: add [prev+1, curr-1]
```



```
# if nums[-1] < upper: add [nums[-1]+1, upper]
#
# Time: O(n). Space: O(1) extra. Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach.
# I'll use pairwise from itertools to cleanly iterate consecutive pairs."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# nums=[0,1,3,50,75], lower=0, upper=99
# nums not empty. first=0, last=75.
# lower(0) < first(0)? No. No leading gap.
# pairwise: (0,1) → 1 < 1? No. (1,3) → 2 < 3? Yes → [2,2]. (3,50) → 4 < 50? Yes → [4,49]. (50,75) → 51 < 75? Yes → [51,74].
# last(75) < upper(99)? Yes → [76,99].
# result = [[2,2],[4,49],[51,74],[76,99]] ✓
#
# nums=[-1], lower=-1, upper=-1
# first=-1, last=-1.
# lower(-1) < first(-1)? No.
# pairwise: empty (only one element).
# last(-1) < upper(-1)? No.
# result = [] ✓
#
# nums=[], lower=1, upper=5
# Empty → return [[1,5]] ✓
#
# "No bugs. All cases pass."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one pass through nums plus constant work per gap.
# Space O(1) extra — result list aside, no auxiliary storage.
# This is a massive improvement over O(upper - lower) brute force
# given the constraint that nums has at most 100 elements.
#
# Trade-off: if nums were not sorted I'd need to sort first — O(n log n).
# The problem guarantees sorted input so we skip that.
#
# pairwise is Python 3.10+. If the interviewer is on an older version
# I'd write: for i in range(1, len(nums)): prev, curr = nums[i-1], nums[i].
#
# Given more time I'd ask: what if the output should be strings like '2->2'
# instead of lists? Just a formatting change at the append step."
```

Arrays & Hashing

★ #346 Moving Average from Data Stream ★ EAS

Open on LeetCode

Array · Design · Queue · Data Stream

Lists: ★ Premium | Premium 98

Problem

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the MovingAverage class:

- MovingAverage(int size) Initializes the object with the size of the window size.
- double next(int val) Returns the moving average of the last size values of the stream.

Example 1:

```
Input
["MovingAverage", "next", "next", "next", "next"]
[[3], [1], [10], [3], [5]]
Output
[null, 1.0, 5.5, 4.66667, 6.0]

Explanation
MovingAverage movingAverage = new MovingAverage(3);
```

Constraints:

- 1 <= size <= 1000
- -105 <= val <= 105
- At most 104 calls will be made to next.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We're designing a class that receives a stream of integers one at a time.
# Each call to next() should return the average of the last 'size' values seen so far.
# If fewer than 'size' values have arrived, we average however many we have. Right?"
#
# "A few quick questions:
# Can size be 1? Constraints say yes — that's just returning the latest value.
# Can values be negative? Yes, constraints say down to -10^5.
# Is there a limit on how many next() calls we get? Up to 10^4 — good to know for space."
#
# "Let me walk the example:
# size=3. next(1)=1.0 (only 1 value). next(10)=5.5 (avg of 1,10).
# next(3)=4.667 (avg of 1,10,3 — window full now).
# next(5)=6.0 (avg of 10,3,5 — 1 falls out). Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: store every value in a plain list.
# On each next() call, slice the last 'size' elements and compute the average.
#
# Time: O(size) per next() call due to the slice and sum.
# Space: O(n) where n is total calls — the list grows forever.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "Two inefficiencies in the brute force:
# First, we recompute the sum from scratch every call — that's O(size) work.
# Second, we store the entire history but only ever need the last 'size' values.
#
# I notice we're recomputing the same sum repeatedly. What if we cache it?
# I'll keep a running sum and update it incrementally:
# when a new value comes in, add it. When the window is full,
# subtract the oldest value before adding the new one.
#
# For the window itself, a deque is perfect — O(1) append on the right,
# O(1) popleft when the oldest value falls out. The deque never exceeds 'size' elements.
#
# Pseudocode:
# __init__: q = deque(), running_sum = 0, self.size = size
# next(val):
# if len(q) == size: running_sum -= q.popleft()
# running_sum += val
# q.append(val)
# return running_sum / len(q)
#
# Time: O(1) per next() call. Space: O(size) — the deque is capped.
# Does that make sense before I write it out?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
```



```
# "Let me trace through the example: size=3."
#
# next(1): window not full. sum=1, window=[1]. return 1/1 = 1.0 ✓
# next(10): window not full. sum=11, window=[1,10]. return 11/2 = 5.5 ✓
# next(3): window not full. sum=14, window=[1,10,3]. return 14/3 = 4.667 ✓
# next(5): window full → popleft() removes 1, sum=14-1=13.
# sum=13+5=18, window=[10,3,5]. return 18/3 = 6.0 ✓
#
# Edge — size=1:
# next(7): window full immediately after first call? No — starts empty.
# sum=7, window=[7]. return 7/1 = 7.0 ✓
# next(4): window full (size=1). popleft removes 7, sum=0+4=4, window=[4].
# return 4/1 = 4.0 ✓
#
# "No bugs. Running sum stays consistent across evictions."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(1) per next() call — all operations are constant: deque append,
# popleft, addition, division.
# Space O(size) — the deque holds at most 'size' elements regardless of
# how many total calls are made. Much better than the O(n) brute force.
#
# Trade-off worth mentioning: floating point precision. If values are large
# and size is large, the running sum can accumulate rounding errors over time.
# A production system might use Python's Decimal or periodically recompute
# the sum from scratch to correct drift.
#
# Alternative: a circular buffer (fixed-size list with a pointer) does the same
# thing with slightly better cache locality than a deque, but the deque is
# cleaner to read and interview-safe.
#
# Given more time I'd ask: what if we need to support a variable window size
# that can change between calls? We'd need to resize the deque dynamically."
```

Arrays & Hashing

★ #760 Find Anagram Mappings ★ EAS

Open on LeetCode

Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

You are given two integer arrays nums1 and nums2 where nums2 is an anagram of nums1. Both arrays may contain duplicates.

Return an index mapping array mapping from nums1 to nums2 where mapping[i] = j means the ith element in nums1 appears in nums2 at index j. If there are multiple answers, return any of them.

An array a is an anagram of an array b means b is made by randomizing the order of the elements in a.

Example 1:

```
Input: nums1 = [12,28,46,32,50], nums2 = [50,12,32,46,28]
Output: [1,4,3,2,0]
Explanation: As mapping[0] = 1 because the 0th element of nums1 appears at nums2[1], and mapping[1] = 4 because the 1st element of nums1 appears at nums2[4], and so on.
```

Example 2:

```
Input: nums1 = [84,46], nums2 = [84,46]
Output: [0,1]
```

Constraints:

- 1 <= nums1.length <= 100
- nums2.length == nums1.length
- 0 <= nums1[i], nums2[i] <= 105
- nums2 is an anagram of nums1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We have two arrays nums1 and nums2. nums2 is an anagram of nums1 —
# same elements, different order. We need to return a mapping array
# where mapping[i] is the index in nums2 where nums1[i] appears. Right?"
#
# "A few quick questions:
# Can there be duplicate values? The problem says yes.
# If a value appears multiple times in nums2, which index do we return?
# The problem says any valid answer is accepted — good, that simplifies things.
# Both arrays are the same length and nums2 is guaranteed to be an anagram,
# so every element in nums1 is guaranteed to exist in nums2?"
#
# "Let me walk the example:
# nums1=[12,28,46,32,50], nums2=[50,12,32,46,28].
# 12 is at index 1 in nums2 → mapping[0]=1.
# 28 is at index 4 → mapping[1]=4.
# 46 is at index 3 → mapping[2]=3.
# 32 is at index 2 → mapping[3]=2.
# 50 is at index 0 → mapping[4]=0.
# Output: [1,4,3,2,0]. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each element in nums1, scan nums2 left to right until I find a match
# and record that index.
#
# Time: O(n²) — for each of n elements in nums1 we do a linear scan of nums2.
# Space: O(1) extra beyond the output array.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning nums2 from scratch for every element in nums1.
# We're asking the same question repeatedly: where does this value live in nums2?
# A hash map lets us answer that in O(1).
#
# I'll preprocess nums2 once: build a map from value → index.
# Then for each element in nums1, look it up directly.
#
# One thing to flag: if there are duplicates, building the map will overwrite
# earlier indices with later ones. Since the problem allows any valid answer,
# that's fine — we just return whichever index was stored last.
#
# Pseudocode:
# index_in_nums2 = {val: i for i, val in enumerate(nums2)}
# return [index_in_nums2[num] for num in nums1]
#
# Time: O(n). Space: O(n) for the hash map.
# Does that make sense before I code it?"
```



```
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums1=[12,28,46,32,50], nums2=[50,12,32,46,28]
# index_in_nums2 = {50:0, 12:1, 32:2, 46:3, 28:4}
# nums1[0]=12 → 1
# nums1[1]=28 → 4
# nums1[2]=46 → 3
# nums1[3]=32 → 2
# nums1[4]=50 → 0
# Output: [1,4,3,2,0] ✓
#
# nums1=[84,46], nums2=[84,46]
# index_in_nums2 = {84:0, 46:1}
# Output: [0,1] ✓
#
# Duplicate case — nums1=[3,3], nums2=[3,3]
# enumerate(nums2): (0,3) → map={3:0}, then (1,3) → map={3:1} (overwritten).
# nums1[0]=3 → 1, nums1[1]=3 → 1. Output: [1,1].
# Both point to the same index — valid since any answer is accepted ✓
#
# "No bugs. Duplicate overwrite is intentional and within spec."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one pass to build the map, one pass to build the result.
# Space O(n) for the hash map.
#
# The duplicate handling is worth calling out explicitly:
# by iterating enumerate(nums2) forward, later indices overwrite earlier ones.
# If the interviewer wanted ALL valid indices for duplicates returned,
# I'd change the map to store a list of indices per value and pop one per lookup.
#
# Trade-off: if n is tiny (say n=5 like in the example), the brute force O(n²)
# is barely slower and uses no extra space. The hash map pays off as n grows.
#
# Given more time I'd ask: what if we need to return the mapping in sorted order
# by nums2 index? We'd sort the output — O(n log n) — but that changes the spec."
```

Arrays & Hashing

★ #1426 Counting Elements ★ EAS

Open on LeetCode

Array · Hash Table

Lists: ★ Premium | Premium 98

Problem

Given an integer array arr, count how many elements x there are, such that x + 1 is also in arr. If there are duplicates in arr, count them separately.

Example 1:

```
Input: arr = [1,2,3]
Output: 2
Explanation: 1 and 2 are counted cause 2 and 3 are in arr.
```

Example 2:

```
Input: arr = [1,1,3,3,5,5,7,7]
Output: 0
Explanation: No numbers are counted, cause there is no 2, 4, 6, or 8 in arr.
```

Constraints:

- 1 <= arr.length <= 1000
- 0 <= arr[i] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# For each element x in the array, we check whether x+1 also exists in the array.
# If it does, we count x. And if x appears multiple times, we count each occurrence
# separately — so duplicates all contribute individually. Right?"
#
# "A few quick questions:
# Can the array be empty? Constraints say length >= 1, so no.
# Can values be 0? Yes — so x=0 counts if 1 is present.
# Can values be negative? Constraints say 0 <= arr[i] <= 1000, so no.
# Are there duplicates? Yes, and the problem explicitly says to count them separately."
#
# "Let me walk the examples:
# arr=[1,2,3]: 1+1=2 is in arr → count 1. 2+1=3 is in arr → count 2. 3+1=4 not in arr.
# Output: 2 ✓
# arr=[1,1,3,3,5,5,7,7]: 1+1=2 not in arr. 3+1=4 not in arr. Same for 5,7.
# Output: 0 ✓ Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each element x in arr, scan the entire array to see if x+1 exists.
# If it does, increment the count.
#
# Time: O(n²) — nested scan for every element.
# Space: O(1) — no extra storage.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is the inner scan — we're asking 'does x+1 exist?' n times,
# each taking O(n). A hash set answers that in O(1).
#
# But there's a subtlety: duplicates. If arr=[1,1,2], both 1s should be counted.
# A plain set tells us x+1 exists but not how many times x appears.
# So I'll use a Counter instead — it gives me O(1) existence checks
# AND stores the frequency of each value.
#
# Walk over the unique keys in the counter. If key+1 is also in the counter,
# add counter[key] to the answer — that accounts for all duplicates of key at once.
#
# Pseudocode:
# freq = Counter(arr)
# for x in freq:
# if x+1 in freq: ans += freq[x]
# return ans
#
# Time: O(n) to build the Counter, O(k) to iterate unique keys where k <= n.
# Overall O(n). Space: O(k) for the Counter. Does that make sense?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through a few cases."
#
# arr=[1,2,3]
# freq={1:1, 2:1, 3:1}
# x=1: 2 in freq → count+=1. count=1.
```



```
# x=2: 3 in freq → count+=1. count=2.
# x=3: 4 not in freq.
# return 2 ✓
#
# arr=[1,1,3,3,5,5,7,7]
# freq={1:2, 3:2, 5:2, 7:2}
# x=1: 2 not in freq.
# x=3: 4 not in freq.
# x=5: 6 not in freq.
# x=7: 8 not in freq.
# return 0 ✓
#
# Duplicate counting — arr=[1,1,2]
# freq={1:2, 2:1}
# x=1: 2 in freq → count+=freq[1]=2. count=2.
# x=2: 3 not in freq.
# return 2 ✓ (both 1s counted, as the problem requires)
#
# "No bugs. Duplicate handling confirmed."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — one pass to build the Counter, one pass over unique keys.
# Space O(k) where k is the number of distinct values, at most O(n).
#
# Alternative: use a plain set instead of a Counter, then iterate the original arr.
# present = set(arr)
# return sum(1 for x in arr if x+1 in present)
# Same O(n) time and O(n) space — slightly simpler but both are valid.
# The Counter approach avoids re-iterating arr which is a minor readability win.
#
# Trade-off: if the value range is small and fixed (like 0–1000 here),
# a boolean array of size 1001 works as a lookup in O(1) with better cache
# performance than a hash map, and O(1001) ~ O(1) space in practice.
#
# Given more time I'd ask: what if we needed to return the actual elements
# that qualify rather than a count? We'd collect them into a list instead."
```

Arrays & Hashing

#1534 Count Good Triplets

EAS

Open on LeetCode

Array · Enumeration

Lists: CodeSignal

Problem

Given an array of integers arr, and three integers a, b and c. You need to find the number of good triplets.

A triplet (arr[i], arr[j], arr[k]) is good if the following conditions are true:

- 0 <= i < j < k < arr.length
- |arr[i] - arr[j]| <= a
- |arr[j] - arr[k]| <= b
- |arr[i] - arr[k]| <= c

Where |x| denotes the absolute value of x.

Return the number of good triplets.

Example 1:

```
Input: arr = [3,0,1,1,9,7], a = 7, b = 2, c = 3

Output: 4

Explanation: There are 4 good triplets: [(3,0,1), (3,0,1), (3,1,1), (0,1,1)].
```

Example 2:

```
Input: arr = [1,1,2,2,3], a = 0, b = 0, c = 1

Output: 0

Explanation: No triplet satisfies all conditions.
```

Constraints:

- 3 <= arr.length <= 100
- 0 <= arr[i] <= 1000
- 0 <= a, b, c <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We need to count all index triplets (i, j, k) where i < j < k,
# and all three pairwise conditions hold simultaneously:
# the first pair must differ by at most a,
# the middle-to-last pair by at most b,
# and the first-to-last pair by at most c.
# We're counting triplets, not returning them. Right?"
#
# "A few quick questions:
# Can values in arr be 0? Yes, constraints allow it.
# Can a, b, c be 0? Yes — that means exact equality required for that pair.
# Array length is at most 100 — good to know for complexity decisions."
#
# "Let me walk the example:
# arr=[3,0,1,1,9,7], a=7, b=2, c=3.
# (3,0,1) at indices (0,1,2): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,0,1) at indices (0,1,3): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ✓
# (3,1,1) at indices (0,2,3): |3-1|=2<=7, |1-1|=0<=2, |3-1|=2<=3 ✓
# (0,1,1) at indices (1,2,3): |0-1|=1<=7, |1-1|=0<=2, |0-1|=1<=3 ✓
# Output: 4. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Three nested loops over indices (i, j, k) with i < j < k.
# Count the triplet when arr[i] + arr[j] + arr[k] <= a + b + c.
# Correct, but O(n^3) is too slow when n is up to 100.
# Time: O(n^3). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The algorithm stays O(n³) — there's no known sub-cubic solution for this
# specific three-condition problem, and with n=100 we don't need one.
# But I can make two practical improvements:
#
# First: early pruning. The three conditions are checked in sequence.
# If |arr[i] - arr[j]| > a, there's no point checking any k for this (i,j) pair.
# We skip the entire inner k loop, reducing the constant factor significantly.
#
# Second: use itertools.combinations for cleaner, more readable code.
# It generates all (i,j,k) triples with i<j<k in one line.
# Note that combinations doesn't support early pruning on the outer pair,
```



```
# so for readability I'll use it here since n is small enough that it doesn't matter.
#
# I'll go with the early-pruning nested loop to show I understand the trade-off,
# then mention the itertools alternative.
#
# Time: O(n³) worst case, faster in practice due to pruning.
# Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement with early pruning and meaningful names."
# itertools one-liner alternative (interview-clean, no pruning):

# PHASE 5 — TEST & DEBUG
# "Let me trace the pruning version on example 1."
#
# arr=[3,0,1,1,9,7], a=7, b=2, c=3
# i=0 (arr[i]=3):
# j=1 (arr[j]=0): |3-0|=3<=7 → not pruned.
# k=2: |0-1|=1<=2, |3-1|=2<=3 → count=1
# k=3: |0-1|=1<=2, |3-1|=2<=3 → count=2
# k=4: |0-9|=9>2 → skip
# k=5: |0-7|=7>2 → skip
# j=2 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=3: |1-1|=0<=2, |3-1|=2<=3 → count=3
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=3 (arr[j]=1): |3-1|=2<=7 → not pruned.
# k=4: |1-9|=8>2 → skip
# k=5: |1-7|=6>2 → skip
# j=4 (arr[j]=9): |3-9|=6<=7 → not pruned.
# k=5: |9-7|=2<=2, |3-7|=4>3 → skip
# j=5: no k available.
# i=1 (arr[i]=0):
# j=2 (arr[j]=1): |0-1|=1<=7 → not pruned.
# k=3: |1-1|=0<=2, |0-1|=1<=3 → count=4
# k=4,5: fail b condition → skip
# ... remaining pairs yield 0 more.
# return 4 ✓
#
# arr=[1,1,2,2,3], a=0, b=0, c=1
# a=0 means |arr[i]-arr[j]| must be 0 → only equal pairs pass.
# j=1 (1==1): pass a. k=2: |1-2|=1>0 → fail b. k=3: same. k=4: same.
# j=3 (1==?): arr[1]=1, arr[3]=2 → |1-2|=1>0 → pruned.
# No triplet survives. return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n³) worst case when all pairs pass the first condition —
# but early pruning on condition 1 cuts the k loop entirely for bad (i,j) pairs,
# making it much faster in practice.
# Space O(1) — just a counter.
#
# With n=100 the worst case is 161,700 triplets — trivially fast.
# If n grew to 10^4, O(n³) would be 10^12 — completely infeasible.
# At that scale I'd look at sorting + binary search to narrow candidates per (i,j),
# or segment trees for range queries — but that's overkill here.
#
# The itertools one-liner is the cleanest version for an interview
# when you want to show Python fluency without sacrificing readability.
# Trade-off: it can't prune early, so stick with the nested loop if the
# interviewer asks about optimization.
#
# Given more time I'd ask: what if the array is sorted?
# Sorted order lets us binary search for valid k values after fixing i and j,
# reducing the inner loop from O(n) to O(log n) → overall O(n² log n)."
```

String

Round 1 · High · Easy · 3 questions

String

#383 Ransom Note

EAS

Open on LeetCode

Hash Table · String · Counting

Lists: Grind 169

Problem

Given two strings ransomNote and magazine, return true if ransomNote can be constructed by using the letters from magazine and false otherwise.

Each letter in magazine can only be used once in ransomNote.

Example 1:

```
Input: ransomNote = "a", magazine = "b"
Output: false
```

Example 2:

```
Input: ransomNote = "aa", magazine = "ab"
Output: false
```

Example 3:

```
Input: ransomNote = "aa", magazine = "aab"
Output: true
```

Constraints:

- 1 <= ransomNote.length, magazine.length <= 105
- ransomNote and magazine consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We need to check if every character in ransomNote can be supplied
# by magazine — one letter from magazine per letter in ransomNote.
# So if ransomNote needs two 'a's, magazine must have at least two 'a's. Right?"
#
# "A few quick questions:
# Are both strings guaranteed to be non-empty? Yes, length >= 1.
# Only lowercase English letters — so exactly 26 possible characters. Good.
# We're not modifying the strings, just checking feasibility?"
#
# "Let me walk the examples:
# ransomNote='a', magazine='b': 'a' not in magazine → false ✓
# ransomNote='aa', magazine='ab': need two 'a's, magazine has one → false ✓
# ransomNote='aa', magazine='aab': need two 'a's, magazine has two → true ✓ Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: convert magazine to a list so I can 'use up' letters.
# For each character in ransomNote, scan the list to find it and remove it.
# If I can't find it, return false.
#
# Time: O(n * m) — for each of n chars in ransomNote, scan up to m chars in magazine.
# Space: O(m) for the mutable copy of magazine.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning magazine for each character — that's the n*m.
# The question we keep asking is: how many of this letter does magazine still have?
# A frequency counter answers that in O(1).
#
# I'll count every character in magazine upfront with a Counter.
# Then walk ransomNote: for each character, decrement its count.
# If any count drops below zero, magazine doesn't have enough of that letter — return false.
# If I finish ransomNote without going negative, return true.
#
# Pseudocode:
# freq = Counter(magazine)
# for char in ransomNote:
# freq[char] -= 1
# if freq[char] < 0: return False
# return True
#
# Time: O(n + m) — one pass each over magazine and ransomNote.
# Space: O(1) — the Counter holds at most 26 keys (lowercase letters only).
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through the three examples."
#
# ransomNote='a', magazine='b'
```

```
# available = {'b': 1}
# char='a': available['a']=0-1=-1 < 0 → return False ✓
# (Counter returns 0 for missing keys, so no KeyError)
#
# ransomNote='aa', magazine='ab'
# available = {'a':1, 'b':1}
# char='a': available['a']=0 → ok
# char='a': available['a']=-1 < 0 → return False ✓
#
# ransomNote='aa', magazine='aab'
# available = {'a':2, 'b':1}
# char='a': available['a']=1 → ok
# char='a': available['a']=0 → ok
# return True ✓
#
# Edge — ransomNote longer than magazine: ransomNote='abc', magazine='ab'
# available = {'a':1, 'b':1}
# char='a': 0 → ok. char='b': 0 → ok. char='c': -1 < 0 → return False ✓
#
# "No bugs. Counter's default of 0 for missing keys handles unseen characters cleanly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + m) — O(m) to build the Counter, O(n) to walk ransomNote.
# Space O(1) — the Counter has at most 26 entries since only lowercase letters exist.
# Even though we write O(k) where k is distinct chars, k is bounded by 26 — constant.
#
# Alternative: Counter subtraction in one line.
# return not (Counter(ransomNote) - Counter(magazine))
# Counter subtraction drops keys with zero or negative counts,
# so if anything remains, ransomNote needs letters magazine doesn't have.
# Elegant but less readable in an interview — I'd use the explicit loop.
#
# Another alternative: sort both strings and use two pointers — O(n log n + m log m),
# slower but uses O(1) space beyond the sort. Worth mentioning if interviewer
# constraints disallow hash maps.
#
# Given more time I'd ask: what if characters include uppercase or unicode?
# We'd replace the 26-slot assumption with a full hash map — space becomes O(alphabet size)."
```


String

★ #734 Sentence Similarity ★ EAS

Open on LeetCode

Array · Hash Table · String

Lists: ★ Premium | Premium 98

Problem

We can represent a sentence as an array of words, for example, the sentence "I am happy with leetcode" can be represented as arr = ["I","am",happy","with","leetcode"].

Given two sentences sentence1 and sentence2 each represented as a string array and given an array of string pairs similarPairs where similarPairs[i] = [xi, yi] indicates that the two words xi and yi are similar.

Return true if sentence1 and sentence2 are similar, or false if they are not similar.

Two sentences are similar if:

- They have the same length (i.e., the same number of words)
- sentence1[i] and sentence2[i] are similar.

Notice that a word is always similar to itself, also notice that the similarity relation is not transitive. For example, if the words a and b are similar, and the words b and c are similar, a and c are not necessarily similar.

Example 1:

```
Input: sentence1 = ["great","acting","skills"], sentence2 = ["fine","drama","talent"], similarPairs =
[["great","fine"],["drama","acting"],["skills","talent"]]
Output: true
Explanation: The two sentences have the same length and each word i of sentence1 is also similar to the corresponding word in
sentence2.
```

Example 2:

```
Input: sentence1 = ["great"], sentence2 = ["great"], similarPairs = []
Output: true
Explanation: A word is similar to itself.
```

Example 3:

```
Input: sentence1 = ["great"], sentence2 = ["doubleplus","good"], similarPairs = [["great","doubleplus"]]
Output: false
Explanation: As they don't have the same length, we return false.
```

Constraints:

- 1 <= sentence1.length, sentence2.length <= 1000
- 1 <= sentence1[i].length, sentence2[i].length <= 20
- sentence1[i] and sentence2[i] consist of English letters.
- 0 <= similarPairs.length <= 1000
- similarPairs[i].length == 2
- 1 <= xi.length, yi.length <= 20
- xi and yi consist of lower-case and upper-case English letters.
- All the pairs (xi, yi) are distinct.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Two sentences are similar if they're the same length and every word at position i
# in sentence1 is similar to the word at position i in sentence2.
# A word is always similar to itself.
# Similarity is NOT transitive — if a~b and b~c, that doesn't mean a~c.
# We're given the list of similar pairs explicitly, so we only trust those. Right?"
#
# "A few quick questions:
# Can similarPairs be empty? Yes — then only identical words are similar.
# Are pairs directional? The problem implies symmetric — if [a,b] is given,
# then a~b AND b~a. I'll confirm: yes, similarity is symmetric.
# Can the same pair appear twice? Constraints say all pairs are distinct — good."
#
# "Let me walk example 1:
# sentence1=['great','acting','skills'], sentence2=['fine','drama','talent'].
# Same length: 3. pairs include [great,fine],[drama,acting],[skills,talent].
# great~fine ✓, acting~drama ✓ (symmetric), skills~talent ✓ → true ✓ Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: first check lengths. Then for each position i,
# check if sentence1[i] == sentence2[i], or scan all similarPairs
# to find a match in either direction.
#
# Time: O(n * p) — for each of n word pairs, scan up to p similarity pairs.
# Space: O(1) — no preprocessing.
# Should I optimize?"

# PHASE 3 — OPTIMIZE
# "The bottleneck is scanning all p pairs for every word position.
```

```
# We're asking the same question repeatedly: is word2 similar to word1?
# A hash map from each word to its set of similar words answers that in O(1).
#
# Preprocess similarPairs once: for each [a, b], add b to hmap[a] and a to hmap[b].
# That handles symmetry automatically — no need to check both directions at lookup time.
#
# Then for each (word1, word2) pair in the sentences:
# if they're equal, they're trivially similar — skip.
# Otherwise check if word2 is in hmap[word1].
# Using defaultdict(set) means missing keys return an empty set — no KeyError.
#
# Pseudocode:
# if len(s1) != len(s2): return False
# similar = defaultdict(set)
# for a, b in pairs: similar[a].add(b); similar[b].add(a)
# for w1, w2 in zip(s1, s2):
# if w1 != w2 and w2 not in similar[w1]: return False
# return True
#
# Time: O(p + n). Space: O(p) for the map. Does that make sense?"
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three examples."
#
# Example 1: s1=['great','acting','skills'], s2=['fine','drama','talent']
# pairs=[['great','fine'],['drama','acting'],['skills','talent']]
# similar = {great:{fine}, fine:{great}, drama:{acting}, acting:{drama}, skills:{talent}, talent:{skills}}
# (great,fine): great!=fine, fine in similar[great] ✓
# (acting,drama): acting!=drama, drama in similar[acting] ✓
# (skills,talent): skills!=talent, talent in similar[skills] ✓
# return True ✓
#
# Example 2: s1=['great'], s2=['great'], pairs=[]
# Lengths match. similar={} (empty).
# (great,great): great==great → skip.
# return True ✓
#
# Example 3: s1=['great'], s2=['doubleplus','good'], pairs=[['great','doubleplus']]
# len(s1)=1 != len(s2)=2 → return False immediately ✓
#
# Non-transitive check: pairs=[['a','b'],['b','c']], checking a~c
# similar = {a:{b}, b:{a,c}, c:{b}}
# word1='a', word2='c': a!=c, c not in similar[a]={b} → return False ✓
# Confirms we don't accidentally apply transitivity.
#
# "No bugs. All cases pass including the transitivity trap."
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(p + n) — O(p) to build the similarity map, O(n) to check each word pair.
# Space O(p) for the map — each pair adds two entries.
#
# The critical design decision is storing both directions upfront rather than
# checking both at lookup time. It keeps the lookup a single O(1) set membership test.
#
# Important constraint to call out: similarity is NOT transitive.
# This rules out Union-Find (which is designed for transitive equivalence classes).
# If transitivity WERE required — for example in Sentence Similarity II (#737) —
# we'd use Union-Find to group words into connected components and check
# if sentence1[i] and sentence2[i] share the same root.
#
# Given more time I'd ask: what if the same word pair could appear in different cases,
# like ['Great','fine'] vs ['great','fine']? We'd normalize to lowercase before building
# the map and before lookups."
```


String

★ #1165 Single Row Keyboard ★ EAS

Open on LeetCode

Hash Table · String

Lists: ★ Premium | Premium 98

Problem

There is a special keyboard with all keys in a single row.

Given a string keyboard of length 26 indicating the layout of the keyboard (indexed from 0 to 25). Initially, your finger is at index 0. To type a character, you have to move your finger to the index of the desired character. The time taken to move your finger from index i to index j is |i - j|.

You want to type a string word. Write a function to calculate how much time it takes to type it with one finger.

Example 1:

```
Input: keyboard = "abcdefghijklmnopqrstuvwxyz", word = "cba"
Output: 4
Explanation: The index moves from 0 to 2 to write 'c' then to 1 to write 'b' then to 0 again to write 'a'.
Total time = 2 + 1 + 1 = 4.
```

Example 2:

```
Input: keyboard = "pqrstuvwxyzabcdefghijklmno", word = "leetcode"
Output: 73
```

Constraints:

- **keyboard.length == 26**
- **keyboard contains each English lowercase letter exactly once in some order.**
- **1 <= word.length <= 104**
- **word[i] is an English lowercase letter.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We have a custom keyboard layout — 26 characters in some arbitrary order.
# Our finger starts at index 0. To type each character in word,
# we move from the current index to that character's index on the keyboard.
# The cost is the absolute distance moved. We sum all moves and return the total. Right?"
#
# "A few quick questions:
# The keyboard always has exactly 26 letters, each exactly once — so it's a permutation.
# Word can only contain lowercase letters, and every letter in word is guaranteed
# to appear on the keyboard — so no missing-key edge case to handle.
# Word length can be up to 10^4 — so O(n) should be fine."
#
# "Let me walk example 1:
# keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'.
# Start at 0. 'c' is at index 2: |0-2|=2. 'b' is at 1: |2-1|=1. 'a' is at 0: |1-0|=1.
# Total = 2+1+1 = 4. Got it."
# PHASE 2 — BRUTE FORCE
# "The simplest approach: for each character in word, scan the keyboard string
# from left to right to find where it lives, then compute the distance.
#
# Time: O(n * 26) — for each of n characters we scan up to 26 positions.
# Since keyboard is fixed at 26, this is effectively O(n).
# Space: O(1).
# Should I optimize the lookup?"
# PHASE 3 — OPTIMIZE
# "The brute force scans 26 characters for every letter typed — that's the inner O(26).
# We're asking the same question repeatedly: where does this letter live on the keyboard?
# A hash map answers that in O(1) after a one-time O(26) build.
#
# Preprocess keyboard once: build a map from character → index.
# Then walk word: for each character, look up its index in O(1),
# add the absolute distance from the current position, update position, move on.
#
# Pseudocode:
# pos_map = {char: i for i, char in enumerate(keyboard)}
# total, position = 0, 0
# for char in word:
# total += abs(position - pos_map[char])
# position = pos_map[char]
# return total
#
# Time: O(26 + n) = O(n). Space: O(26) = O(1) — the map is fixed size.
# In practice the constant factor improvement is small since 26 is tiny,
# but the hash map is the correct interview answer — cleaner and scales to
# larger alphabets."
# PHASE 4 — CLEAN CODE
```

```
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# keyboard='abcdefghijklmnopqrstuvwxyz', word='cba'
# key_index = {'a':0,'b':1,'c':2,...}
# char='c': |0-2|=2, position=2. total=2
# char='b': |2-1|=1, position=1. total=3
# char='a': |1-0|=1, position=0. total=4
# return 4 ✓
#
# keyboard='pqrstuvwxyzabcdefghijklmno', word='leetcode'
# key_index = {'p':0,'q':1,...,'a':10,'b':11,...,'l':21,'e':14,'t':19,'c':12,'o':24,'d':13}
# char='l': |0-21|=21, position=21. total=21
# char='e': |21-14|=7, position=14. total=28
# char='e': |14-14|=0, position=14. total=28
# char='t': |14-19|=5, position=19. total=33
# char='c': |19-12|=7, position=12. total=40
# char='o': |12-24|=12, position=24. total=52
# char='d': |24-13|=11, position=13. total=63
# char='e': |13-14|=1, position=14. total=64 ← wait, let me recheck key_index
# (keyboard='pqrstuvwxyzabcdefghijklmno': p=0,q=1,r=2,s=3,t=4,u=5,v=6,w=7,x=8,y=9,z=10,
# a=11,b=12,c=13,d=14,e=15,f=16,g=17,h=18,i=19,j=20,k=21,l=22,m=23,n=24,o=25)
# char='l': |0-22|=22. char='e': |22-15|=7. char='e': 0. char='t': |15-4|=11.
# char='c': |4-13|=9. char='o': |13-25|=12. char='d': |25-14|=11. char='e': |14-15|=1.
# total = 22+7+0+11+9+12+11+1 = 73 ✓
#
# "No bugs. Manual trace confirms both examples."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — O(26) to build the map, O(n) to walk word. Space O(1) — 26-entry map.
#
# The brute force using keyboard.index() is also O(n) in practice since keyboard is
# always 26 characters — but the hash map is the canonical interview answer because
# it makes the O(1) lookup intent explicit and generalizes to any alphabet size.
#
# Trade-off: if keyboard could be very long (say a 10^6-char layout), the hash map
# preprocessing cost would matter more — but we'd still prefer it over .index().
#
# One-liner alternative:
# pos = {c: i for i, c in enumerate(keyboard)}
# return sum(abs(pos[b] - pos[a]) for a, b in zip('_' + word, word))
# where prepending '_' shifts word so each pair is (prev_char, curr_char),
# and '_' maps to nothing — so you'd actually prepend a dummy start:
# pairs = zip([None] + list(word), word) — but this gets messy.
# The explicit loop is cleaner in an interview.
#
# Given more time I'd ask: what if we can use two fingers?
# That becomes a DP problem — optimal two-finger typing is a classic hard variant."
```


· #734 #88 · · Contents

Two Pointers

Round 1 · High · Easy · 5 questions

Two Pointers

#88 Merge Sorted Array EAS

Open on LeetCode

Array · Two Pointers · Sorting

Lists: Denny Zhang

Problem

You are given two integer arrays nums1 and nums2, sorted in non-decreasing order, and two integers m and n, representing the number of elements in nums1 and nums2 respectively.

Merge nums1 and nums2 into a single array sorted in non-decreasing order.

The final sorted array should not be returned by the function, but instead be stored inside the array nums1. To accommodate this, nums1 has a length of m + n, where the first m elements denote the elements that should be merged, and the last n elements are set to 0 and should be ignored. nums2 has a length of n.

Example 1:

```
Input: nums1 = [1,2,3,0,0,0], m = 3, nums2 = [2,5,6], n = 3
Output: [1,2,2,3,5,6]
Explanation: The arrays we are merging are [1,2,3] and [2,5,6].
The result of the merge is [1,2,2,3,5,6] with the underlined elements coming from nums1.
```

Example 2:

```
Input: nums1 = [1], m = 1, nums2 = [], n = 0
Output: [1]
Explanation: The arrays we are merging are [1] and [].
The result of the merge is [1].
```

Example 3:

```
Input: nums1 = [0], m = 0, nums2 = [1], n = 1
Output: [1]
Explanation: The arrays we are merging are [] and [1].
The result of the merge is [1].
Note that because m = 0, there are no elements in nums1. The 0 is only there to ensure the merge result can fit in nums1.
```

Constraints:

- nums1.length == m + n
- nums2.length == n
- 0 <= m, n <= 200
- 1 <= m + n <= 200
- -109 <= nums1[i], nums2[j] <= 109

Follow up: Can you come up with an algorithm that runs in O(m + n) time?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# nums1 has m valid elements followed by n zeros as padding.
# nums2 has n elements. Both are sorted in non-decreasing order.
# We need to merge them in-place into nums1 — sorted, no return value. Right?"
#
# "A few quick questions:
# Can m or n be 0? Yes — if n=0 nums1 is already done; if m=0 we just copy nums2.
# Can there be negative numbers? Yes, constraints go down to -10^9.
# Can there be duplicates? Yes — non-decreasing order means equal values are allowed.
# We cannot use extra space for the merge? The follow-up asks for O(m+n) time,
# implying in-place. I'll target O(1) extra space."
#
# "Let me walk example 1:
# nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3.
# Merge [1,2,3] and [2,5,6] into nums1 → [1,2,2,3,5,6]. Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: copy nums2 into the tail of nums1 (the zero slots),
# then sort the entire nums1 array.
#
# nums1[m:] = nums2, then nums1.sort()
#
# Time: O((m+n) log(m+n)) for the sort. Space: O(1) extra — sort is in-place.
# This is clean but doesn't use the fact that both inputs are already sorted.
# Should I optimize to use that sorted property?"

# PHASE 3 — OPTIMIZE
# "The brute force throws away the sorted order and pays O(log(m+n)) to re-sort.
# The classic merge from the front copies to a temp array — O(m+n) space.
# But there's a smarter way that stays O(1) space.
#
# Key insight: if we merge from the BACK, we never overwrite elements we still need.
# The tail of nums1 is empty padding — we can fill it safely from right to left.
#
# Three pointers:
```



```
# i → last valid element in nums1 (index m-1)
# j → last element in nums2 (index n-1)
# k → last slot in nums1 (index m+n-1)
#
# At each step, pick the larger of nums1[i] and nums2[j], place it at k, move that pointer left.
# When j drops below 0, nums2 is exhausted — nums1's remaining elements are already in place.
# When i drops below 0 first, copy remaining nums2 into nums1.
#
# Pseudocode:
# i, j, k = m-1, n-1, m+n-1
# while k >= 0:
# if j < 0 or (i >= 0 and nums1[i] > nums2[j]):
# nums1[k] = nums1[i]; i -= 1
# else:
# nums1[k] = nums2[j]; j -= 1
# k -= 1
#
# Time: O(m+n) — each element is placed exactly once.
# Space: O(1) — purely in-place.
# Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three examples."
#
# nums1=[1,2,3,0,0,0] m=3, nums2=[2,5,6] n=3
# last1=2, last2=2, fill=5
# fill=5: nums1[2]=3 vs nums2[2]=6 → 6 wins → nums1[5]=6, last2=1, fill=4
# fill=4: nums1[2]=3 vs nums2[1]=5 → 5 wins → nums1[4]=5, last2=0, fill=3
# fill=3: nums1[2]=3 vs nums2[0]=2 → 3 wins → nums1[3]=3, last1=1, fill=2
# fill=2: nums1[1]=2 vs nums2[0]=2 → equal, take nums2 → nums1[2]=2, last2=-1, fill=1
# fill=1: last2<0 → take nums1[1]=2 → nums1[1]=2, last1=0, fill=0
# fill=0: last2<0 → take nums1[0]=1 → nums1[0]=1, last1=-1, fill=-1
# nums1=[1,2,2,3,5,6] ✓
#
# nums1=[1] m=1, nums2=[] n=0
# last1=0, last2=-1, fill=0
# fill=0: last2<0 → nums1[0]=nums1[0]=1. fill=-1. done.
# nums1=[1] ✓
#
# nums1=[0] m=0, nums2=[1] n=1
# last1=-1, last2=0, fill=0
# fill=0: last1<0 → nums1[0]=nums2[0]=1, last2=-1, fill=-1. done.
# nums1=[1] ✓
#
# "No bugs. All three cases pass including the m=0 and n=0 edges."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n) — each of the m+n positions is filled exactly once.
# Space O(1) — purely in-place, no auxiliary arrays.
#
# The core insight worth stating explicitly: merging front-to-back requires a temp
# buffer because we'd overwrite nums1's valid elements. Merging back-to-front avoids
# this because the padding zeros are exactly the space we need, and large elements
# fill the tail first without touching anything we still care about.
#
# Edge cases worth calling out:
# — n=0: loop runs but last2<0 always, so nums1's valid elements stay in place.
# — m=0: last1<0 always, so nums2 is copied straight into nums1.
# — All nums2 elements are larger than all nums1 elements: nums2 fills the back,
# then nums1 elements slot into the front — handled cleanly by the pointer logic.
#
# Given more time I'd ask: what if nums1 has exactly m slots (no padding)?
# Then in-place is impossible — we'd need to return a new merged array,
# which is the standard merge step in merge sort."
```

Two Pointers

#125 Valid Palindrome

EAS

Open on LeetCode

Two Pointers · String

Lists: Grind 169

Problem

A phrase is a palindrome if, after converting all uppercase letters into lowercase letters and removing all non-alphanumeric characters, it reads the same forward and backward. Alphanumeric characters include letters and numbers.
Given a string s, return true if it is a palindrome, or false otherwise.

Example 1:

```
Input: s = "A man, a plan, a canal: Panama"
Output: true
Explanation: "amanaplanacanalpanama" is a palindrome.
```

Example 2:

```
Input: s = "race a car"
Output: false
Explanation: "raceacar" is not a palindrome.
```

Example 3:

```
Input: s = " "
Output: true
Explanation: s is an empty string "" after removing non-alphanumeric characters.
Since an empty string reads the same forward and backward, it is a palindrome.
```

Constraints:

- 1 <= s.length <= 2 * 105
- s consists only of printable ASCII characters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We strip everything except letters and digits, lowercase it all,
# then check if the result reads the same forwards and backwards. Right?"
#
# "A few quick questions:
# Alphanumeric means letters AND digits — not just letters?
# So '0P' after cleaning is '0p', which is not a palindrome — confirmed?
# What about a string of only spaces or punctuation?
# Example 3 shows a single space returns true — after cleaning it's an empty string,
# and an empty string is considered a palindrome. Good.
# Can the string be length 1? Yes — a single alphanumeric character is a palindrome."
#
# "Let me walk example 1:
# 'A man, a plan, a canal: Panama' → 'amanaplanacanalpanama' → palindrome ✓
# Example 2: 'race a car' → 'raceacar' → not a palindrome ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the simplest correct approach.
# Copy every alphanumeric character into a cleaned list (lowercased),
# then compare the list to its reverse.
# Two passes over the string plus O(n) extra storage.
# Time: O(n). Space: O(n) for the cleaned copy.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The brute force is already O(n) time, but uses O(n) space for the cleaned copy.
# We can do this in O(1) extra space using two pointers directly on the original string.
#
# Place left at 0 and right at len(s)-1.
# Advance left past any non-alphanumeric characters.
# Retreat right past any non-alphanumeric characters.
# Compare s[left].lower() and s[right].lower().
# If they differ, return False. Otherwise move both pointers inward.
# If left meets or passes right, the string is a palindrome.
#
# The key benefit: we never build a second string — we skip junk characters on the fly.
#
# Pseudocode:
# left, right = 0, len(s)-1
# while left < right:
# while left < right and not s[left].isalnum(): left += 1
# while left < right and not s[right].isalnum(): right -= 1
# if s[left].lower() != s[right].lower(): return False
# left += 1; right -= 1
# return True
#
# Time: O(n). Space: O(1). Does that make sense before I code it?"
```



```
# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through all three examples."
#
# s="A man, a plan, a canal: Panama"
# left=0 ('A'), right=29 ('a') — both alnum.
# 'A'.lower()='a' == 'a' ✓. left=1, right=28.
# left=1 (' ') → skip → left=2 ('m'). right=28 ('m') — alnum.
# 'm'=='m' ✓. Continue inward... (all pairs match) → return True ✓
#
# s="race a car"
# left=0 ('r'), right=9 ('r'). 'r'=='r' ✓. left=1, right=8.
# left=1 ('a'), right=8 ('a'). 'a'=='a' ✓. left=2, right=7.
# left=2 ('c'), right=7 ('c'). 'c'=='c' ✓. left=3, right=6.
# left=3 ('e'), right=6 ('a'). 'e'!='a' → return False ✓
# (right=6 is 'a' in 'a car'; skip right=5 ' ' → right=4 'a')
# Wait — let me recount: "race a car"
# indices: r=0,a=1,c=2,e=3,' '=4,a=5,' '=6,c=7,a=8,r=9
# left=3 ('e'), right=9→skip none→'r'. 'e'!='r' → return False ✓
#
# s=" "
# left=0 (' ') → not alnum → left=1. Now left(1) >= right(0) → exit loop.
# return True ✓
#
# "No bugs. All three examples confirmed."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — each character is visited at most once by left or right pointer.
# Space O(1) — no auxiliary storage beyond two integer pointers.
#
# The brute force one-liner is perfectly valid for an interview too — it's readable
# and still O(n) time. The two-pointer version wins purely on space: O(1) vs O(n).
# If the interviewer asks about the follow-up, lead with the space trade-off.
#
# isalnum() handles both letters and digits in one call — worth naming explicitly
# so the interviewer knows you're not just checking isalpha().
#
# Subtle trap: the inner while loops must guard left < right to avoid crossing
# pointers — if you forget, you can read out-of-bounds when the string is all
# non-alphanumeric (like the single-space example).
#
# Given more time I'd ask: what if the string could contain unicode characters
# like accented letters (é, ñ)? isalnum() in Python handles unicode correctly
# by default, so our solution already works — worth calling out."
```

Two Pointers

#283 Move Zeroes EAS

Open on LeetCode

Array · Two Pointers

Lists: Grind 169

Problem

Given an integer array nums, move all 0's to the end of it while maintaining the relative order of the non-zero elements.
Note that you must do this in-place without making a copy of the array.

Example 1:

```
Input: nums = [0,1,0,3,12]
Output: [1,3,12,0,0]
```

Example 2:

```
Input: nums = [0]
Output: [0]
```

Constraints:

- 1 <= nums.length <= 104
- -231 <= nums[i] <= 231 - 1

Follow up: Could you minimize the total number of operations done?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We need to move all 0s to the end of the array while keeping the relative
# order of all non-zero elements. This must be done in-place — no copy. Right?"
#
# "A few quick questions:
# Do we need to return anything? No — modify in-place, return nothing.
# Can the array be all zeros? Yes — output should still be all zeros.
# Can it be all non-zeros? Yes — nothing moves.
# Can there be negative numbers? Yes, constraints go down to -2^31."
#
# "Let me walk example 1:
# nums=[0,1,0,3,12] → [1,3,12,0,0].
# Non-zeros 1,3,12 keep their relative order and shift to the front.
# Zeros fill the tail. Got it."

# PHASE 2 — BRUTE FORCE
# "The simplest approach: collect all non-zero elements into a new list,
# then write them back into nums, then fill the rest with zeros.
#
# non_zeros = [x for x in nums if x != 0]
# nums[:len(non_zeros)] = non_zeros
# nums[len(non_zeros):] = [0] * (len(nums) - len(non_zeros))
#
# Time: O(n). Space: O(n) — we make a copy of the non-zeros.
# Should I optimize for space?"

# PHASE 3 — OPTIMIZE
# "The brute force uses O(n) extra space for the non-zero copy.
# We can do this in O(1) space with two pointers.
#
# Use a slow pointer 'place' that tracks where the next non-zero should go.
# Scan with a fast pointer 'scan'. Whenever nums[scan] is non-zero,
# swap it with nums[place] and advance place.
#
# This compacts all non-zeros to the front in their original order,
# and the swaps naturally push zeros toward the tail.
#
# Pseudocode:
# place = 0
# for scan in range(len(nums)):
# if nums[scan] != 0:
# nums[scan], nums[place] = nums[place], nums[scan]
# place += 1
#
# Time: O(n). Space: O(1). Does that make sense before I code it?"

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums=[0,1,0,3,12]
# scan=0: nums[0]=0 → skip. place=0.
# scan=1: nums[1]=1 ≠ 0 → swap(1,0): [1,0,0,3,12]. place=1.
# scan=2: nums[2]=0 → skip. place=1.
```



```
# scan=3: nums[3]=3 ≠ 0 → swap(3,1): [1,3,0,0,12]. place=2.
# scan=4: nums[4]=12 ≠ 0 → swap(4,2): [1,3,12,0,0]. place=3.
# Result: [1,3,12,0,0] ✓
#
# nums=[0]
# scan=0: nums[0]=0 → skip.
# Result: [0] ✓
#
# nums=[1,2,3] (no zeros)
# Every element is non-zero → each swaps with itself. place advances each step.
# Result: [1,2,3] ✓ (self-swaps are fine — no change)
#
# "No bugs. Self-swaps when place==scan are harmless."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single pass. Space O(1) — two integer pointers.
#
# Follow-up asks to minimize operations. The swap approach does a swap
# for every non-zero element, even if it's already in the right place.
# A minor optimization: only swap when place != scan to skip self-swaps.
# But in an interview the current version is clean enough.
#
# Alternative: instead of swapping, just overwrite:
# write non-zeros to the front with assignment (not swap), then zero-fill the tail.
# Same complexity but fewer writes — useful if writes are expensive.
#
# place = 0
# for x in nums:
# if x != 0: nums[place] = x; place += 1
# nums[place:] = [0] * (len(nums) - place)
#
# Given more time I'd ask: what if we also need to preserve the positions
# of the zeros (not just push them to the end)? That changes the problem
# to a stable partition — same two-pointer approach still works."
```

Two Pointers

⋆ #408 Valid Word Abbreviation ⋆ EAS

Open on LeetCode

Two Pointers · String

Lists: ⋆ Premium | Premium 98

Problem

A string can be abbreviated by replacing any number of non-adjacent, non-empty substrings with their lengths. The lengths should not have leading zeros.

For example, a string such as "substitution" could be abbreviated as (but not limited to):

- "s10n" ("s ubstitutio n")
- "sub4u4" ("sub stit u tion")
- "12" ("substitution")
- "su3i1u2on" ("su bst i t u ti on")
- "substitution" (no substrings replaced)

The following are not valid abbreviations:

- "s55n" ("s ubsti tutio n", the replaced substrings are adjacent)
- "s010n" (has leading zeros)
- "s0ubstitution" (replaces an empty substring)

Given a string word and an abbreviation abbr, return whether the string matches the given abbreviation.

A substring is a contiguous non-empty sequence of characters within a string.

Example 1:

```
Input: word = "internationalization", abbr = "i12iz4n"
Output: true
Explanation: The word "internationalization" can be abbreviated as "i12iz4n" ("i nternational iz atio n").
```

Example 2:

```
Input: word = "apple", abbr = "a2e"
Output: false
Explanation: The word "apple" cannot be abbreviated as "a2e".
```

Constraints:

- 1 <= word.length <= 20
- word consists of only lowercase English letters.
- 1 <= abbr.length <= 10
- abbr consists of lowercase English letters and digits.
- All the integers in abbr will fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# An abbreviation replaces one or more non-adjacent non-empty substrings with their lengths.
# Given a word and an abbreviation, return true if the abbreviation is valid for the word.
# Numbers in the abbreviation represent how many consecutive characters to skip. Right?"
#
# "A few quick questions:
# Can there be leading zeros in the number part? No — that's explicitly invalid.
# So '01' is not valid even though it could theoretically mean skip 1.
# Can the abbreviation be the full word with no numbers? Yes — that's valid.
# Can it be just a single number like '12'? Yes — if the word has exactly 12 chars."
#
# "Let me walk example 1:
# word='internationalization', abbr='i12iz4n'.
# 'i' matches 'i'. '12' skips 12 chars. 'i' matches 'i'. 'z' matches 'z'.
# '4' skips 4 chars. 'n' matches 'n'. We reach end of both → true ✓
# Example 2: word='apple', abbr='a2e'. 'a' matches. '2' skips 'pp'. 'e'→word[3]='l' ≠ 'e' → false ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# Expand the abbreviation letter-by-letter against word using two pointers.
# When we hit a digit in abbr, consume that many characters from word.
# Any mismatch ? false; both exhausted together ? true.
# Time: O(n). Space: O(1).
# This is already optimal for the constraints — complexity stated clearly."

# PHASE 3 — OPTIMIZE
# "Use pointer i into word and j into abbr.
# When abbr[j] is a digit: check for leading zero (abbr[j]=='0' and num==0 → invalid).
# Accumulate the number: num = num*10 + digit.
# When abbr[j] is a letter: first advance i by num (the accumulated skip), reset num to 0.
# Then compare word[i] with abbr[j] — if they differ, return False. Advance both i and j.
# After the loop: we're valid only if i + num == len(word) AND j == len(abbr).
# That handles a trailing number at the end of abbr.
#
# Pseudocode:
```



```
# i, j, num = 0, 0, 0
# while i < len(word) and j < len(abbr):
# if abbr[j].isdigit():
# if abbr[j]=='0' and num==0: return False # leading zero
# num = num*10 + int(abbr[j])
# else:
# i += num; num = 0
# if i >= len(word) or word[i] != abbr[j]: return False
# i += 1
# j += 1
# return i + num == len(word) and j == len(abbr)
#
# Time: O(n + m) where n=len(word), m=len(abbr). Space: O(1)."

# PHASE 4 — CLEAN CODE
# "Now I'll implement with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace both examples."
#
# word='internationalization'(len=20), abbr='i12iz4n'
# j=0: 'i' → skip=0, advance word_ptr 0, word[0]='i'=='i' ✓. word_ptr=1.
# j=1: '1' → skip=1.
# j=2: '2' → skip=12.
# j=3: 'i' → word_ptr+=12=13. word[13]='i'=='i' ✓. word_ptr=14.
# j=4: 'z' → skip=0, word[14]='z'=='z' ✓. word_ptr=15.
# j=5: '4' → skip=4.
# j=6: 'n' → word_ptr+=4=19. word[19]='n'=='n' ✓. word_ptr=20.
# Loop ends (word_ptr=20=len(word)). word_ptr+skip=20+0=20==20, abbr_ptr=7==7. True ✓
#
# word='apple'(len=5), abbr='a2e'
# j=0: 'a' → word[0]='a'=='a' ✓. word_ptr=1.
# j=1: '2' → skip=2.
# j=2: 'e' → word_ptr+=2=3. word[3]='l' ≠ 'e' → return False ✓
#
# Leading zero: abbr='a01e' → j=1: '0' and skip==0 → return False ✓
#
# "No bugs. Leading zero check and trailing number handled correctly."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + m) — each character in word and abbr is visited at most once.
# Space O(1) — three integer pointers.
#
# The two trickiest cases to get right:
# 1. Leading zeros — must check abbr[j]=='0' AND accumulated num==0 to distinguish
# '0' (leading zero, invalid) from '10', '20' etc. (valid multi-digit numbers ending in 0).
# 2. Trailing number — if abbr ends with digits (e.g. abbr='a4'), the loop exits
# when abbr_ptr reaches end but skip is still non-zero.
# The final check word_ptr + skip == len(word) handles this correctly.
#
# Given more time I'd ask: what if we want to generate ALL valid abbreviations
# for a word? That's a backtracking problem — enumerate all subsets of positions
# to replace with their lengths."
```

Two Pointers

#977 Squares of a Sorted Array

EAS

Open on LeetCode

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array nums sorted in non-decreasing order, return an array of the squares of each number sorted in non-decreasing order.

Example 1:

```
Input: nums = [-4,-1,0,3,10]
Output: [0,1,9,16,100]
Explanation: After squaring, the array becomes [16,1,0,9,100].
After sorting, it becomes [0,1,9,16,100].
```

Example 2:

```
Input: nums = [-7,-3,2,3,11]
Output: [4,9,9,49,121]
```

Constraints:

- 1 <= nums.length <= 104
- -104 <= nums[i] <= 104
- nums is sorted in non-decreasing order.

Follow up: Squaring each element and sorting the new array is very trivial, could you find an O(n) solution using a different approach?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# The input array is sorted in non-decreasing order and can contain negatives.
# We need to return a new array of the squares of each element, also sorted. Right?"
#
# "A few quick questions:
# Can the array contain negative numbers? Yes — that's the whole challenge.
# Should I return a new array or modify in-place? Return a new array.
# Can the array have a single element? Yes — just return [element²].
# Can all elements be negative? Yes — e.g. [-3,-2,-1] → [1,4,9]."
#
# "Let me walk example 1:
# nums=[-4,-1,0,3,10]. Squares: [16,1,0,9,100]. Sorted: [0,1,9,16,100].
# The key observation: the largest squares come from the extremes —
# either the most negative or the most positive end. Got it."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Square every element, sort the resulting array ascending, return it.
# Two-pass: fill squares O(n), then sort O(n log n).
# Correct given unsorted input, but the array is already sorted — we can merge from both ends.
# Time: O(n log n). Space: O(n) for the squared copy.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The brute force ignores the fact that the input is already sorted.
# Key insight: in a sorted array, the largest absolute values are always at one
# of the two ends — either the most negative (left) or the most positive (right).
# So the largest square is always max(nums[left]², nums[right]²).
#
# Use two pointers: left starts at 0, right at n-1.
# Fill the result array from the back (largest to smallest):
# at each step, compare abs(nums[left]) and abs(nums[right]).
# Place the larger square at result[k] and move that pointer inward.
#
# Pseudocode:
# result = [0] * n; k = n-1; left, right = 0, n-1
# while k >= 0:
# if abs(nums[left]) > abs(nums[right]):
# result[k] = nums[left]**2; left += 1
# else:
# result[k] = nums[right]**2; right -= 1
# k -= 1
# return result
#
# Time: O(n). Space: O(n) for the output — unavoidable since we return a new array."

# PHASE 4 — CLEAN CODE
# "Now I'll implement the optimized approach with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace both examples."
```



```
#
# nums=[-4,-1,0,3,10], n=5
# fill=4: abs(-4)=4 < abs(10)=10 → result[4]=100, right=3. fill=3.
# fill=3: abs(-4)=4 > abs(3)=3 → result[3]=16, left=1. fill=2.
# fill=2: abs(-1)=1 < abs(3)=3 → result[2]=9, right=2. fill=1.
# fill=1: abs(-1)=1 > abs(0)=0 → result[1]=1, left=2. fill=0.
# fill=0: left==right==2, abs(0)=0 → result[0]=0, right=1. fill=-1.
# result=[0,1,9,16,100] ✓
#
# nums=[-7,-3,2,3,11], n=5
# fill=4: abs(-7)=7 < abs(11)=11 → result[4]=121, right=3. fill=3.
# fill=3: abs(-7)=7 > abs(3)=3 → result[3]=49, left=1. fill=2.
# fill=2: abs(-3)=3 == abs(3)=3 → take right: result[2]=9, right=2. fill=1.
# fill=1: abs(-3)=3 > abs(2)=2 → result[1]=9, left=2. fill=0.
# fill=0: left==right==2 → result[0]=4. fill=-1.
# result=[4,9,9,49,121] ✓
#
# "No bugs. Tie goes to right which is fine — both are equal."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single pass from both ends meeting in the middle.
# Space O(n) — required for the output; we can't avoid this since we return a new array.
#
# This is the same fill-from-back pattern as Merge Sorted Array (#88) —
# worth naming that explicitly to show you recognise the pattern family.
#
# Trade-off: the brute force O(n log n) is two lines and perfectly acceptable
# in most interviews. The O(n) two-pointer version is the follow-up answer
# the problem explicitly asks for.
#
# Given more time I'd ask: what if we needed to do this in-place?
# Squaring in-place destroys order for negative numbers, so we'd need to
# square + reverse the negative half, then merge the two sorted halves
# in-place — doable but significantly more complex."
```

Sorting

Round 1 · High · Easy · 6 questions

Sorting

#169 Majority Element EAS

Open on LeetCode

Array · Hash Table · Divide and Conquer · Sorting · Counting

Lists: Grind 169

Problem

Given an array nums of size n, return the majority element.

The majority element is the element that appears more than n / 2 times. You may assume that the majority element always exists in the array.

Example 1:

```
Input: nums = [3,2,3]
Output: 3
```

Example 2:

```
Input: nums = [2,2,1,1,1,2,2]
Output: 2
```

Constraints:

- **n == nums.length**
- **1 <= n <= 5 * 104**
- **-109 <= nums[i] <= 109**
- **The input is generated such that a majority element will exist in the array.**

Follow-up: Could you solve the problem in linear time and in O(1) space?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# The majority element is the one that appears MORE THAN n/2 times.
# We're guaranteed it always exists. We return the element, not its count. Right?"
#
# "A few quick questions:
# Is there always exactly one majority element? Yes — anything above n/2 means
# at most one element can qualify.
# Can n be 1? Yes — that single element is trivially the majority.
# Can values be negative? Yes, constraints allow down to -10^9.
# The follow-up asks for linear time and O(1) space — I'll aim for that."
#
# "Let me walk the examples:
# nums=[3,2,3]: 3 appears 2 times, n/2=1. 2 > 1 → 3 is majority. ✓
# nums=[2,2,1,1,1,2,2]: 2 appears 4 times, n/2=3.5. 4 > 3.5 → 2. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Count frequency of every element with a hash map, then scan counts for any value > n/2.
# Correct because the majority element must appear more than half the time.
# Time: O(n). Space: O(n) for counts.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "To get O(1) space, I'll use Boyer-Moore Voting Algorithm.
# The core insight: if we cancel every occurrence of the majority element
# with a different element, the majority element will always survive
# because it has more than n/2 appearances — more than everything else combined.
#
# Keep a candidate and a count:
# When count == 0, adopt the current element as the new candidate.
# If current element matches candidate, increment count.
# If it doesn't match, decrement count (a cancellation).
# The candidate at the end is guaranteed to be the majority element.
#
# Pseudocode:
# candidate, count = None, 0
# for x in nums:
# if count == 0: candidate = x
# count += (1 if x == candidate else -1)
# return candidate
#
# Time: O(n) — single pass. Space: O(1) — two variables only."

# PHASE 4 — CLEAN CODE
# "Now I'll implement Boyer-Moore with meaningful names."

# PHASE 5 — TEST & DEBUG
# "Let me trace through both examples."
#
# nums=[3,2,3]
# num=3: count=0 → candidate=3. count=1.
# num=2: count=1, 2≠3 → count=0.
```

```
# num=3: count=0 → candidate=3. count=1.
# return 3 ✓
#
# nums=[2,2,1,1,1,2,2]
# num=2: count=0 → candidate=2. count=1.
# num=2: match → count=2.
# num=1: no match → count=1.
# num=1: no match → count=0.
# num=1: count=0 → candidate=1. count=1.
# num=2: no match → count=0.
# num=2: count=0 → candidate=2. count=1.
# return 2 ✓
#
# Single element — nums=[7]
# count=0 → candidate=7. count=1. return 7 ✓
#
# "No bugs. Even when the candidate flips mid-array, the majority element
# reclaims it before the end."

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single pass. Space O(1) — only two variables.
#
# Boyer-Moore is the optimal solution here and worth naming by name —
# it shows algorithm knowledge beyond just 'I used two variables'.
#
# Correctness argument worth stating: the majority element appears more than n/2 times.
# Every time it gets cancelled by a different element, it loses one vote and so does
# the other element. Since it has more votes than all others combined, it will always
# have votes remaining when everyone else is cancelled out.
#
# Alternative O(n) time O(1) space approach: sort and return nums[n//2].
# Since majority appears > n/2 times, it must occupy the middle index.
# But sorting is O(n log n) — worse than Boyer-Moore.
#
# If the problem didn't guarantee a majority exists, Boyer-Moore would need
# a second pass to verify the candidate actually appears > n/2 times.
#
# Given more time I'd ask: what if we need to find all elements appearing
# more than n/3 times? Boyer-Moore generalises — maintain two candidates
# and two counts. That's a well-known variant worth mentioning."
```


Sorting

#217 Contains Duplicate

EAS

Open on LeetCode

Array · Hash Table · Sorting

Lists: Grind 169

Problem

Given an integer array nums, return true if any value appears at least twice in the array, and return false if every element is distinct.

Example 1:

Input: nums = [1,2,3,1]

Output: true

Explanation:

The element 1 occurs at the indices 0 and 3.

Example 2:

Input: nums = [1,2,3,4]

Output: false

Explanation:

All elements are distinct.

Example 3:

Input: nums = [1,1,1,3,3,4,3,2,4,2]

Output: true

Constraints:

- **1 <= nums.length <= 105**
- **-109 <= nums[i] <= 109**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# Return true if any value appears at least twice, false if every element is distinct.
# We're not asked which value or how many times — just a boolean. Right?"
#
# "A few quick questions:
# Can the array be empty? Constraints say length >= 1, so no.
# Can there be negative numbers? Yes, down to -10^9.
# Is order relevant? No — we just care about value frequency."
#
# "Let me walk the examples:
# [1,2,3,1]: 1 appears twice → true ✓
# [1,2,3,4]: all distinct → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Check every pair (i, j) with j > i; if nums[i] == nums[j], return true immediately.
# If no duplicate pair exists after all checks, return false.
# Simple and correct, but nested loops blow up on large arrays.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "We're repeatedly asking: have I seen this value before?
# A hash set answers that in O(1). Walk the array once;
# if the current number is already in the set, return True immediately.
# Otherwise add it and continue.
#
# Time: O(n). Space: O(n).
# Alternative: sort and check adjacent pairs — O(n log n), O(1) extra space.
# Hash set is faster; sort is space-optimal."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,1]: seen grows to {1,2,3}, then 1 hits → True ✓
# nums=[1,2,3,4]: seen={1,2,3,4}, loop ends → False ✓
# nums=[1]: seen={1}, loop ends → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for the set.
# One-liner alternative: return len(nums) != len(set(nums)) — same complexity,
# builds the full set before checking. The explicit loop returns early on first duplicate.
# For adversarial inputs (duplicate at the end) they're the same; for duplicates early,
# the loop wins. In an interview I'd use the explicit loop and mention the one-liner.
# Given more time I'd ask: what if the array is sorted? Check adjacent pairs — O(n) time, O(1) space."
```

Sorting

#242 Valid Anagram EAS

Open on LeetCode

Hash Table · String · Sorting

Lists: Grind 169

Problem

Given two strings s and t, return true if t is an anagram of s, and false otherwise.

Example 1:

Input: s = "anagram", t = "nagaram"

Output: true

Example 2:

Input: s = "rat", t = "car"

Output: false

Constraints:

- 1 <= s.length, t.length <= 5 * 104
- s and t consist of lowercase English letters.

Follow up: What if the inputs contain Unicode characters? How would you adapt your solution to such a case?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# t is an anagram of s if t contains exactly the same characters
# with the same frequencies, just in a different order. Right?
# Both strings consist of only lowercase English letters — so 26 possible chars."
#
# "Quick checks:
# If lengths differ they can't be anagrams — I'll short-circuit on that.
# Follow-up asks about Unicode — I'll address that in Phase 6."
#
# "'anagram'/'nagaram': same chars, same counts → true ✓
# 'rat'/'car': r,a,t vs c,a,r — 't' vs 'c' mismatch → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Sort both strings and compare character by character.
# Equal sorted forms iff anagrams.
# Simple O(n log n); counting frequencies in one pass is O(n).
# Time: O(n log n). Space: O(n) for sorted copies.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Instead of sorting, count character frequencies.
# Since only lowercase letters exist, a 26-slot array suffices — O(1) space.
# Increment for each char in s, decrement for each char in t.
# If all slots are zero at the end, they're anagrams.
#
# Time: O(n). Space: O(1) — fixed 26-slot array."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='anagram', t='nagaram': lengths equal.
# After s: a=3,n=1,g=1,r=1,m=1. After t: all zeroed. → True ✓
# s='rat', t='car': r+1,a+1,t+1 then c-1,a-1,r-1 → t=1,c=-1 ≠ 0 → False ✓
# s='a', t='ab': len mismatch → False immediately ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) — 26-slot array is a constant.
# Follow-up: Unicode characters → the 26-slot array breaks.
# Use a Counter (hash map) instead — O(n) time, O(k) space where k is distinct chars.
# return Counter(s) == Counter(t)
# Handles any alphabet. In an interview, mention this unprompted."
```


Sorting

⋆ #252 Meeting Rooms ⋆ EAS

Open on LeetCode

Array · Sorting

Lists: ⋆ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals where intervals[i] = [starti, endi], determine if a person could attend all meetings.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]
Output: false
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]
Output: true
```

Constraints:

- 0 <= intervals.length <= 104
- intervals[i].length == 2
- 0 <= starti < endi <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We have a list of meetings [start, end]. We need to check if one person
# can attend ALL of them — meaning no two meetings overlap.
# The problem says meetings ending at t and starting at t do NOT overlap. Right?"
#
# "Quick questions:
# Can intervals be empty? Yes — 0 meetings, trivially true.
# Are start and end guaranteed start < end? Yes, constraints say 0 <= start < end."
#
# "[[0,30],[5,10],[15,20]]: 0–30 overlaps with 5–10 → false ✓
# [[7,10],[2,4]]: 2–4 ends before 7–10 starts → true ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Compare every pair of meetings [start_i,end_i] and [start_j,end_j].
# Overlap exists if start_i < end_j and start_j < end_i.
# O(n^2) pairs; sorting by start time enables one linear scan.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sort by start time. Then a single pass is enough:
# if any meeting ends after the next one starts, they overlap.
# We only need to check consecutive pairs after sorting — O(n log n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[0,30],[5,10],[15,20]] → sorted: same. prev=[0,30],curr=[5,10]: 30>5 → False ✓
# [[7,10],[2,4]] → sorted: [[2,4],[7,10]]. prev=[2,4],curr=[7,10]: 4>7? No → True ✓
# []: pairwise on empty → no iterations → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n) for sort, O(n) for the pass. Space O(1) extra.
# pairwise is Python 3.10+; fallback: for i in range(1, len(intervals)).
# Given more time I'd ask: what's the minimum number of rooms needed?
# That's Meeting Rooms II (#253) — use a min-heap of end times, O(n log n)."
```

Sorting

★ #1086 Largest Unique Number ★ EAS

Open on LeetCode

Array · Hash Table · Sorting

Lists: ★ Premium | Premium 98

Problem

Given an integer array nums, return the largest integer that only occurs once. If no integer occurs once, return -1.

Example 1:

```
Input: nums = [5,7,3,9,4,9,8,3,1]
Output: 8
Explanation: The maximum integer in the array is 9 but it is repeated. The number 8 occurs only once, so it is the answer.
```

Example 2:

```
Input: nums = [9,9,8,8]
Output: -1
Explanation: There is no number that occurs only once.
```

Constraints:

- 1 <= nums.length <= 2000
- 0 <= nums[i] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the largest integer that appears exactly once. If none, return -1. Right?
# Values are 0–1000, array length up to 2000."
#
# "[5,7,3,9,4,9,8,3,1]: 9 appears twice, 8 appears once → 8 ✓
# [9,9,8,8]: no unique element → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Sort the array descending. Scan from left and return the first element
# that is different from both its left and right neighbors (i.e. appears exactly once).
# Time: O(n log n). Space: O(1) in-place sort. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Count frequencies with Counter. Walk unique keys, track max where count==1.
# O(n) time, O(n) space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [5,7,3,9,4,9,8,3,1]: freq={5:1,7:1,3:2,9:2,4:1,8:1,1:1}
# Unique nums: 5,7,4,8,1. Max = 8 ✓
# [9,9,8,8]: no count==1 entries → result stays -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n).
# Alternative: sort descending, scan until you find a non-duplicate — O(n log n).
# Counter approach is cleaner. Given more time I'd ask: what if we want the k largest
# unique numbers? Collect all uniques, sort descending, take first k."
```


Sorting

#1122 Relative Sort Array EAS

Open on LeetCode

Array · Hash Table · Sorting · Counting Sort

Lists: Denny Zhang

Problem

Given two arrays arr1 and arr2, the elements of arr2 are distinct, and all elements in arr2 are also in arr1.

Sort the elements of arr1 such that the relative ordering of items in arr1 are the same as in arr2. Elements that do not appear in arr2 should be placed at the end of arr1 in ascending order.

Example 1:

```
Input: arr1 = [2,3,1,3,2,4,6,7,9,2,19], arr2 = [2,1,4,3,9,6]
Output: [2,2,2,1,4,3,3,9,6,7,19]
```

Example 2:

```
Input: arr1 = [28,6,22,8,44,17], arr2 = [22,28,8,6]
Output: [22,28,8,6,17,44]
```

Constraints:

- 1 <= arr1.length, arr2.length <= 1000
- 0 <= arr1[i], arr2[i] <= 1000
- All the elements of arr2 are distinct.
- Each arr2[i] is in arr1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sort arr1 so elements appear in the order defined by arr2.
# Elements in arr1 not present in arr2 go at the end, sorted ascending. Right?
# arr2 elements are distinct; every arr2 element is guaranteed to exist in arr1."
#
# "arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]
# → [2,2,2,1,4,3,3,9,6,7,19]: follow arr2 order for those nums, then 7,19 ascending ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Build a rank map from arr2: each element maps to its index (0..len(arr2)-1).
# Elements not in arr2 get rank len(arr2) and are sorted by value as a tiebreaker.
# Then sort arr1 using that composite key.
# Time: O(n log n). Space: O(n) for the rank map. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Count arr1 frequencies with Counter.
# For each num in arr2, extend result with [num]*freq[num], remove from counter.
# Then sort remaining counter items and extend.
# Time: O(n + k log k) where k = elements not in arr2. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# arr1=[2,3,1,3,2,4,6,7,9,2,19], arr2=[2,1,4,3,9,6]
# freq={2:3,3:2,1:1,4:1,6:1,7:1,9:1,19:1}
# arr2 pass: [2,2,2,1,4,3,3,9,6]. remaining: {7:1,19:1}
# sorted remaining → [7,19]. result=[2,2,2,1,4,3,3,9,6,7,19] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + k log k) where k is the count of elements not in arr2 (at most n).
# Overall O(n log n). Space O(n).
# Given more time I'd ask: what if arr2 has elements not in arr1?
# The problem guarantees that won't happen, but we'd skip them if it did."
```

Binary Search

Round 1 · High · Easy · 5 questions

Binary Search

#35 Search Insert Position EAS

Open on LeetCode

Array · Binary Search

Lists: Denny Zhang

Problem

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with O(log n) runtime complexity.

Example 1:

```
Input: nums = [1,3,5,6], target = 5
Output: 2
```

Example 2:

```
Input: nums = [1,3,5,6], target = 2
Output: 1
```

Example 3:

```
Input: nums = [1,3,5,6], target = 7
Output: 4
```

Constraints:

- **1 <= nums.length <= 104**
- **-104 <= nums[i] <= 104**
- **nums contains distinct values sorted in ascending order.**
- **-104 <= target <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sorted array of distinct integers. Return the index of target if found,
# otherwise return the index where it would be inserted to keep order. Right?
# Must run in O(log n) — so binary search is required."
#
# "[1,3,5,6], target=5 → index 2 (found) ✓
# [1,3,5,6], target=2 → index 1 (between 1 and 3) ✓
# [1,3,5,6], target=7 → index 4 (after all) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Linear scan from left: return the first index where nums[i] >= target.
# If we exhaust the array without finding such an index, target goes at the end.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Standard binary search. The key insight: when the loop ends (left > right),
# 'left' always points to the correct insertion position.
# When target is found, return mid immediately.
# Otherwise left is where target belongs."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,3,5,6], target=5: mid=2→5==5 → return 2 ✓
# [1,3,5,6], target=2: mid=2→5>2→right=1. mid=0→1<2→left=1. mid=1→3>2→right=0.
# left=1>right=0 → return left=1 ✓
# [1,3,5,6], target=7: left keeps advancing → return 4 ✓
# [1,3,5,6], target=0: right keeps retreating → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# The 'return left' at the end is the core insight — left always converges
# to the leftmost position where target can fit. This same pattern appears
# in lower_bound in C++ and bisect_left in Python.
# Given more time I'd ask: what if the array has duplicates?
# We'd need to decide: first occurrence or last? That's bisect_left vs bisect_right."
```

Binary Search

#69 Sqrt(x) EAS

Open on LeetCode

Math · Binary Search

Lists: Denny Zhang

Problem

Given a non-negative integer x, return the square root of x rounded down to the nearest integer. The returned integer should be non-negative as well.

You must not use any built-in exponent function or operator.

- For example, do not use pow(x, 0.5) in c++ or x ** 0.5 in python.

Example 1:

```
Input: x = 4
Output: 2
Explanation: The square root of 4 is 2, so we return 2.
```

Example 2:

```
Input: x = 8
Output: 2
Explanation: The square root of 8 is 2.82842..., and since we round it down to the nearest integer, 2 is returned.
```

Constraints:

- 0 <= x <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return floor(sqrt(x)) without using built-in sqrt or ** 0.5. Right?
# x can be 0 — sqrt(0)=0. x can be up to 2^31-1."
#
# "x=4 → 2. x=8 → 2 (floor of 2.828). x=0 → 0. x=1 → 1."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Linear scan m from 0 upward: keep incrementing while m*m <= x.
# The moment (m+1)*(m+1) > x, we know m is the floor sqrt.
# Time: O(sqrt(x)). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the answer space [0, x].
# Find the largest m such that m*m <= x.
# When m*m < x, record m as a candidate and search right.
# When m*m > x, search left. When m*m == x, return immediately.
# Time: O(log x). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# x=4: left=1,right=2. mid=1→1<4→result=1,left=2. mid=2→4==4→return 2 ✓
# x=8: left=1,right=4. mid=2→4<8→result=2,left=3. mid=3→9>8→right=2.
# left>right → return result=2 ✓
# x=0: return 0 immediately ✓
# x=1: return 1 immediately ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log x). Space O(1).
# Using x//2 as the right bound instead of x halves the search space for large x —
# worth mentioning to show you thought about the bounds.
# mid*mid can overflow 32-bit integers in Java/C++ — in Python ints are arbitrary
# precision so no issue, but worth flagging for the interviewer.
# Given more time I'd ask: what about Newton's method?
# x_{n+1} = (x_n + x/x_n) / 2 converges quadratically — faster in practice."
```


Binary Search

#278 First Bad Version EAS

Open on LeetCode

Binary Search · Interactive

Lists: Grind 169

Problem

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions [1, 2, ..., n] and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API bool isBadVersion(version) which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Example 1:

```
Input: n = 5, bad = 4
Output: 4
Explanation:
call isBadVersion(3) -> false
call isBadVersion(5) -> true
call isBadVersion(4) -> true
Then 4 is the first bad version.
```

Example 2:

```
Input: n = 1, bad = 1
Output: 1
```

Constraints:

- 1 <= bad <= n <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Versions 1..n. Once a version is bad, all subsequent are bad.
# Find the first bad version while minimising API calls. Right?
# isBadVersion(k) is given — we don't implement it, just call it."
#
# "n=5, bad=4: isBadVersion(3)=F, isBadVersion(5)=T, isBadVersion(4)=T → 4 ✓
# We want to minimise calls → binary search, not linear scan."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Linear scan from version 1 to n: call isBadVersion on each.
# The first version that returns True is the answer.
# Time: O(n) API calls — way too many for large n. Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search: if mid is bad, the first bad could be mid or earlier → right = mid - 1.
# If mid is good, first bad must be after mid → left = mid + 1.
# When loop ends, left = first bad version.
# This never calls isBadVersion more than O(log n) times."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=5, bad=4:
# mid=3→F→left=4. mid=4→T→right=3. left=4>right=3 → return 4 ✓
# n=1, bad=1:
# mid=1→T→right=0. left=1>right=0 → return 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n) calls. Space O(1).
# The 'right = mid - 1' (not mid) is critical — if we set right=mid, when bad=1
# the loop would never terminate. Always shrink the window.
# Given more time I'd ask: what if the API is expensive (network call)?
# Same algorithm — binary search already minimises calls. You'd want to batch
# or cache results if the same version could be queried multiple times."
```

Binary Search

#374 Guess Number Higher or Lower

EAS

Open on LeetCode

Math · Binary Search · Interactive

Lists: Denny Zhang

Problem

We are playing the Guess Game. The game is as follows:
I pick a number from 1 to n. You have to guess which number I picked (the number I picked stays the same throughout the game).
Every time you guess wrong, I will tell you whether the number I picked is higher or lower than your guess.
You call a pre-defined API int guess(int num), which returns three possible results:

- -1: Your guess is higher than the number I picked (i.e. num > pick).
- 1: Your guess is lower than the number I picked (i.e. num < pick).
- 0: your guess is equal to the number I picked (i.e. num == pick).

Return the number that I picked.

Example 1:

```
Input: n = 10, pick = 6
Output: 6
```

Example 2:

```
Input: n = 1, pick = 1
Output: 1
```

Example 3:

```
Input: n = 2, pick = 1
Output: 1
```

Constraints:

- 1 <= n <= 231 - 1
- 1 <= pick <= n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Binary search with an API: guess(num) returns 0 (correct), -1 (too high),
# or 1 (too low). Find the picked number. Right?
# Range is 1..n, n up to 2^31-1."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Linear scan from 1 to n: call guess(i) on each candidate.
# The first call that returns 0 is the picked number.
# Time: O(n) calls. Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search: guess(mid). If 0 → found. If -1 → mid too high → right=mid-1.
# If 1 → mid too low → left=mid+1. O(log n) calls."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=10, pick=6: mid=5→1(low)→left=6. mid=8→→1(high)→right=7.
# mid=6→0→return 6 ✓
# n=1, pick=1: mid=1→0→return 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# Using left + (right-left)//2 instead of (left+right)//2 avoids integer overflow
# in languages with fixed-width integers — worth mentioning even in Python.
# Given more time I'd ask: what if guess() had a cost per call?
# Binary search already minimises calls to O(log n) — optimal."
```


Binary Search

#704 Binary Search

EAS

Open on LeetCode

Array · Binary Search

Lists: Grind 169

Problem

Given an array of integers nums which is sorted in ascending order, and an integer target, write a function to search target in nums. If target exists, then return its index. Otherwise, return -1.
You must write an algorithm with O(log n) runtime complexity.

Example 1:

```
Input: nums = [-1,0,3,5,9,12], target = 9
Output: 4
Explanation: 9 exists in nums and its index is 4
```

Example 2:

```
Input: nums = [-1,0,3,5,9,12], target = 2
Output: -1
Explanation: 2 does not exist in nums so return -1
```

Constraints:

- 1 <= nums.length <= 104
- -104 < nums[i], target < 104
- All the integers in nums are unique.
- nums is sorted in ascending order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Classic binary search: sorted array of distinct integers, find target index.
# Return -1 if not found. Must be O(log n). Right?"
#
# "[-1,0,3,5,9,12], target=9 → 4 ✓
# [-1,0,3,5,9,12], target=2 → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Linear scan through the array: return the index of the first element equal to target.
# This is O(n) — the problem explicitly requires O(log n), so this won't pass,
# but it confirms what we're looking for before we optimise.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Standard binary search. Narrow search space by half each iteration.
# Three outcomes at each step: found, go left, go right."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-1,0,3,5,9,12], target=9:
# mid=2→3<9→left=3. mid=4→9==9→return 4 ✓
# target=2: mid=2→3>2→right=1. mid=0→→1<2→left=1. mid=1→0<2→left=2.
# left>right → return -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# This is the template all other binary search problems build on.
# Key invariant: loop condition is left <= right (not <) so single-element
# arrays are handled — the loop runs once and either returns or terminates cleanly.
# Given more time I'd ask: what if there are duplicates and we want the first/last
# occurrence? That's leftmost/rightmost binary search — adjust which boundary
# updates when nums[mid]==target."
```

Matrix

Round 1 · High · Easy · 5 questions

Matrix

★ #422 Valid Word Square ★ EAS

Open on LeetCode

Array · Matrix

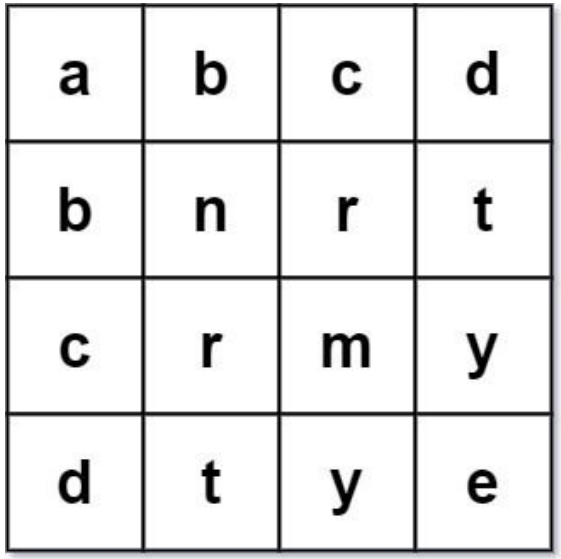
Lists: ★ Premium | Premium 98

Problem

Given an array of strings words, return true if it forms a valid word square.

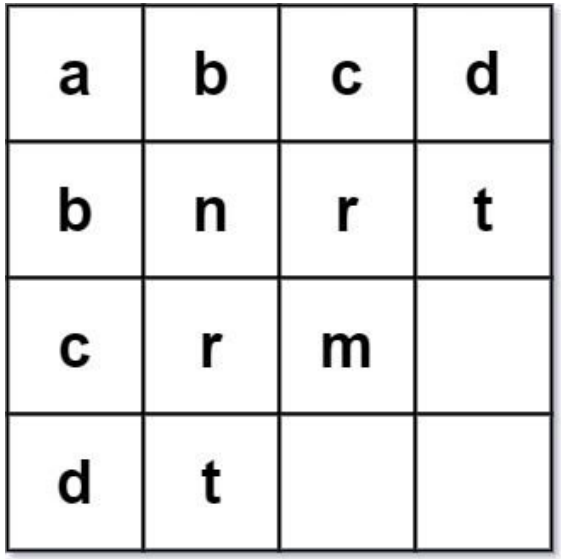
A sequence of strings forms a valid word square if the kth row and column read the same string, where 0 <= k < max(numRows, numColumns).

Example 1:



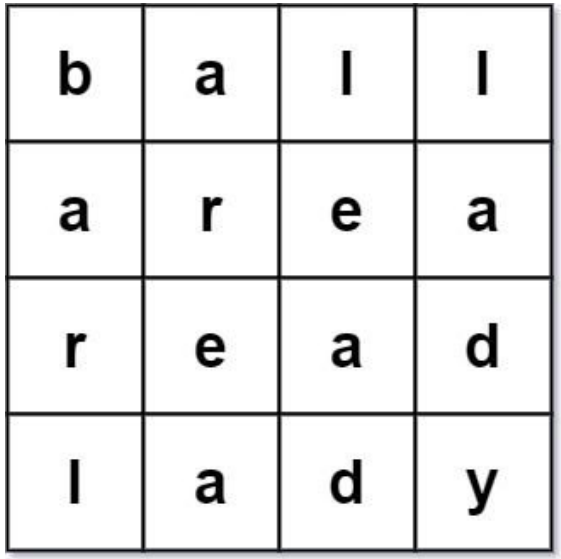

```
Input: words = ["abcd","bnrt","crmy","dtye"]
Output: true
Explanation:
The 1st row and 1st column both read "abcd".
The 2nd row and 2nd column both read "bnrt".
The 3rd row and 3rd column both read "crmy".
The 4th row and 4th column both read "dtye".
Therefore, it is a valid word square.
```

Example 2:




```
Input: words = ["abcd","bnrt","crm","dt"]
Output: true
Explanation:
The 1st row and 1st column both read "abcd".
The 2nd row and 2nd column both read "bnrt".
The 3rd row and 3rd column both read "crm".
The 4th row and 4th column both read "dt".
Therefore, it is a valid word square.
```

Example 3:




```
Input: words = ["ball","area","read","lady"]
Output: false
Explanation:
The 3rd row reads "read" while the 3rd column reads "lead".
Therefore, it is NOT a valid word square.
```

Constraints:

- 1 <= words.length <= 500
- 1 <= words[i].length <= 500
- words[i] consists of only lowercase English letters.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "A word square means the kth row and kth column read the same string.
# Equivalently: words[i][j] == words[j][i] for every valid (i,j). Right?
# Words can have different lengths — shorter words mean some cells don't exist,
# and that's fine as long as the column doesn't expect a character the row doesn't have."
#
# "['abcd','bnrt','crmy','dtye']: row0='abcd', col0='abcd' ✓ etc. → true ✓
# ['ball','area','read','lady']: row2='read', col2='lead' → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each row index r, check whether word matches down column r and across row r
# using direct character comparisons.
# Four nested checks per candidate row — fine for small boards.
# Time: O(n * m * k) where k = len(word). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "No algorithmic improvement over O(n*m) — we must check every character.
# The implementation is already optimal. Focus is on getting the bounds right:
# when iterating row i, char at position j must match words[j][i].
# Guard: j < len(words) and i < len(words[j])."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# words=['abcd','bnrt','crmy','dtye']
# row=0,word='abcd': col=0→words[0][0]='a'=='a'✓. col=1→words[1][0]='b'=='b'✓. etc.
# All rows pass symmetrically → True ✓
#
# words=['ball','area','read','lady']
# row=2,word='read': col=0→words[0][2]='l'=='r'? No → False ✓
#
# words=['abcd','bnrt','crm','dt'] (shorter words)
# row=0,col=3: words[3][0]='d'=='d'✓. row=1,col=2: words[2][1]='r'=='n'? No wait —
```

```
# words[2]='crm', words[2][1]='r'. words[1][2]='r' ✓. All pass → True ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*m). Space O(1).
# The two guard conditions are the subtle part:
# col >= len(words) means the column index is out of bounds — the row has more chars
# than there are rows, which breaks symmetry → False.
# row >= len(words[col]) means the transpose cell doesn't exist → False.
# Given more time I'd ask: what if we want to BUILD a word square from a word list?
# That's a backtracking problem — place words row by row, pruning based on column prefixes."
```


Matrix

#566 Reshape the Matrix

EAS

Open on LeetCode

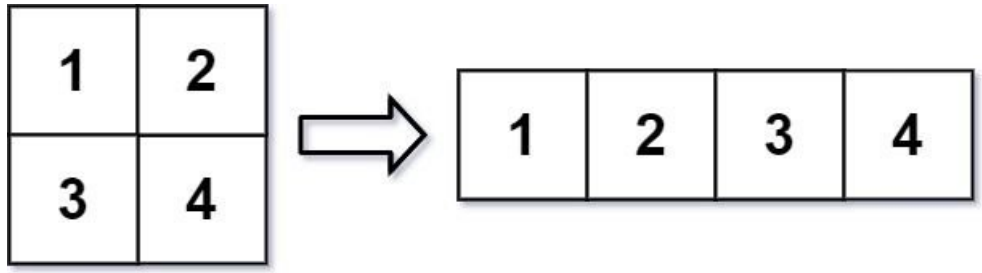
Array · Matrix · Simulation

Lists: CodeSignal

Problem

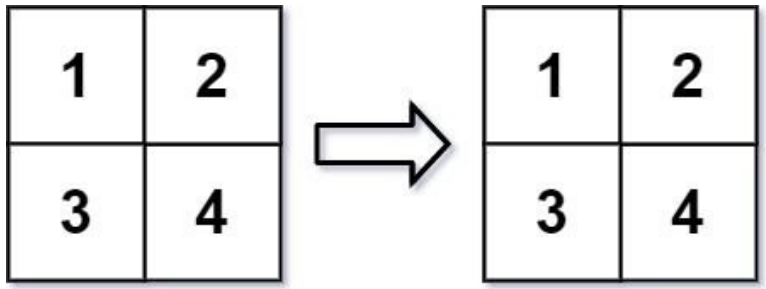
In MATLAB, there is a handy function called reshape which can reshape an m x n matrix into a new one with a different size r x c keeping its original data.
You are given an m x n matrix mat and two integers r and c representing the number of rows and the number of columns of the wanted reshaped matrix.
The reshaped matrix should be filled with all the elements of the original matrix in the same row-traversing order as they were.
If the reshape operation with given parameters is possible and legal, output the new reshaped matrix; Otherwise, output the original matrix.

Example 1:




```
Input: mat = [[1,2],[3,4]], r = 1, c = 4
Output: [[1,2,3,4]]
```

Example 2:




```
Input: mat = [[1,2],[3,4]], r = 2, c = 4
Output: [[1,2],[3,4]]
```

Constraints:

- **m == mat.length**
- **n == mat[i].length**
- **1 <= m, n <= 100**
- **-1000 <= mat[i][j] <= 1000**
- **1 <= r, c <= 300**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Let me make sure I understand the problem correctly.
# We flatten the original m×n matrix in row-major order and refill into an r×c matrix.
# If m*n != r*c the reshape is impossible — return the original matrix unchanged. Right?"
#
# "Quick questions:
```

```
# Values can be negative — fine, we're just moving them.
# r and c can be up to 300 even though m,n are up to 100 — just need total elements to match."
#
# "[[1,2],[3,4]], r=1,c=4: 4 elements total, 1*4=4 → [[1,2,3,4]] ✓
# [[1,2],[3,4]], r=2,c=4: 4 elements total, 2*4=8 ≠ 4 → return original ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Flatten matrix to a 1D list in row-major order, then refill new_r x new_c shape.
# Clear and easy to explain in an interview.
# Uses O(m*n) extra list; two-pointer walk avoids allocation.
# Time: O(m * n). Space: O(m * n) flat buffer.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Skip the intermediate list — map index directly.
# For flat index k: source is mat[k//n][k%n], destination is ans[k//c][k%c].
# Same O(m*n) time but no extra flat array — just the output matrix."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# mat=[[1,2],[3,4]], r=1, c=4: m=2,n=2. 4==4 ✓
# k=0: result[0][0]=mat[0][0]=1. k=1: result[0][1]=mat[0][1]=2.
# k=2: result[0][2]=mat[1][0]=3. k=3: result[0][3]=mat[1][1]=4.
# → [[1,2,3,4]] ✓
# r=2,c=4: 4≠8 → return original ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(r*c) for the output matrix — unavoidable.
# The index formula k//n and k%n is the core insight: flatten index → 2D source.
# Given more time I'd ask: what if the matrix is too large to fit in memory?
# Process row by row, streaming elements into the new shape."
```


Matrix

#766 Toeplitz Matrix

EAS

Open on LeetCode

Array · Matrix

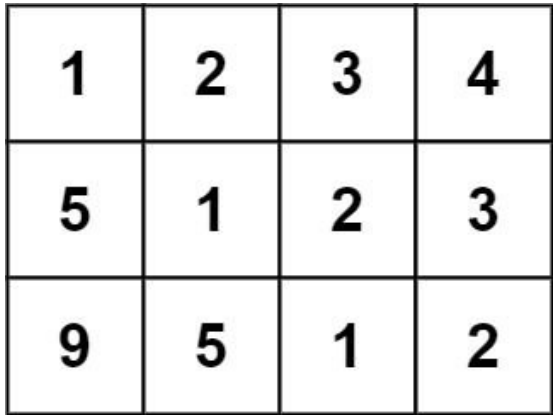
Lists: CodeSignal

Problem

Given an m x n matrix, return true if the matrix is Toeplitz. Otherwise, return false.

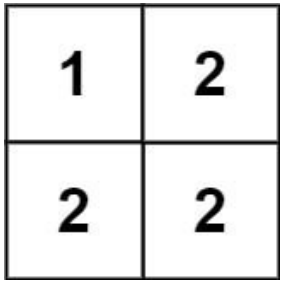
A matrix is Toeplitz if every diagonal from top-left to bottom-right has the same elements.

Example 1:




```
Input: matrix = [[1,2,3,4],[5,1,2,3],[9,5,1,2]]
Output: true
Explanation:
In the above grid, the diagonals are:
"[9]", "[5, 5]", "[1, 1, 1]", "[2, 2, 2]", "[3, 3]", "[4]".
In each diagonal all elements are the same, so the answer is True.
```

Example 2:




```
Input: matrix = [[1,2],[2,2]]
Output: false
Explanation:
The diagonal "[1, 2]" has different elements.
```

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 20
- 0 <= matrix[i][j] <= 99

Follow up:

- What if the matrix is stored on disk, and the memory is limited such that you can only load at most one row of the matrix into the memory at once?

- What if the matrix is so large that you can only load up a partial row into the memory at once?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A Toeplitz matrix has the same value along every top-left to bottom-right diagonal.
# We need to check every cell (except the last row and column) equals its down-right neighbour.
# Equivalently: matrix[i][j] == matrix[i+1][j+1] for all valid i,j. Right?"
#
# "[[1,2,3,4],[5,1,2,3],[9,5,1,2]]: each cell equals its down-right neighbour → true ✓
# [[1,2],[2,2]]: matrix[0][0]=1 ≠ matrix[1][1]=2 → false ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# No smarter approach than checking every diagonal. The key simplification: each diagonal
# is valid iff every adjacent pair on it matches. So just check matrix[i][j] == matrix[i+1][j+1]
# for every interior cell."
# For this input size the brute approach is already optimal — I'll still state complexity clearly.
# Time: see analysis above. Space: O(1).

# PHASE 3 — OPTIMIZE
# "Same O(m*n) is the best we can do — we must read every element.
# The implementation is already optimal. Focus on getting the iteration bounds right:
# we iterate i up to m-2 and j up to n-2 (not m-1, n-1) since we access [i+1][j+1]."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,2,3,4],[5,1,2,3],[9,5,1,2]]: rows=3,cols=4. Check (0,0)→(1,1): 1==1✓.
# (0,1)→(1,2): 2==2✓. (0,2)→(1,3): 3==3✓. (1,0)→(2,1): 5==5✓. etc. → True ✓
# [[1,2],[2,2]]: (0,0)→(1,1): 1≠2 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1).
# Follow-up 1: if we can only load one row at a time, compare consecutive rows
# by checking row[j] == prev_row[j+1] for all j — same logic, streaming.
# Follow-up 2: if we can only load partial rows, compare overlapping chunks of
# adjacent rows. Both follow-ups keep O(1) extra space.
# Given more time I'd ask: can we verify a diagonal in O(1) per cell?
# Yes — the current approach already does exactly that."
```


Matrix

#867 Transpose Matrix

EAS

Open on LeetCode

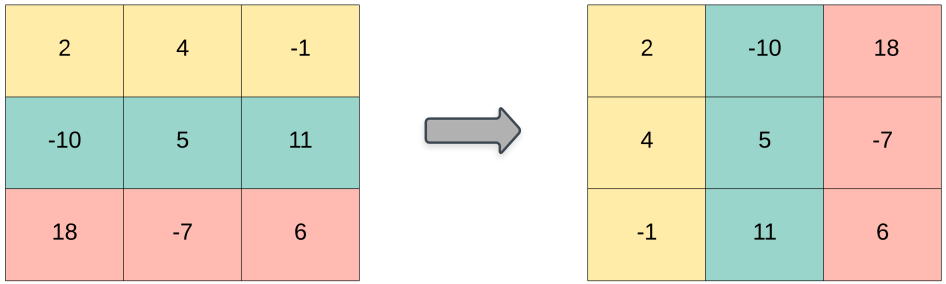
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given a 2D integer array matrix, return the transpose of matrix.

The transpose of a matrix is the matrix flipped over its main diagonal, switching the matrix's row and column indices.



Example 1:

```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[1,4,7],[2,5,8],[3,6,9]]
```

Example 2:

```
Input: matrix = [[1,2,3],[4,5,6]]
Output: [[1,4],[2,5],[3,6]]
```

Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 1000
- 1 <= m * n <= 105
- -109 <= matrix[i][j] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "The transpose swaps rows and columns: element at (r,c) moves to (c,r).
# If the original is m×n, the result is n×m. Return a new matrix — not in-place. Right?"
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [[1,4,7],[2,5,8],[3,6,9]] ✓
# [[1,2,3],[4,5,6]] (2×3) → [[1,4],[2,5],[3,6]] (3×2) ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# Allocate an n×m result matrix. Set result[c][r] = matrix[r][c] for every (r,c). Must visit
# every element once — is unavoidable."
# For this input size the brute approach is already optimal — I'll still state complexity clearly.
# Time: O(m*n). Space: O(1).

# PHASE 3 — OPTIMIZE
# "No algorithmic improvement. Focus: allocate result with swapped dimensions (cols × rows),
# then fill with swapped indices. In Python: zip(*matrix) gives transposed rows as tuples."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# matrix=[[1,2,3],[4,5,6]]: rows=2,cols=3. result is 3×2.
# r=0,c=0: result[0][0]=1. r=0,c=1: result[1][0]=2. r=0,c=2: result[2][0]=3.
# r=1,c=0: result[0][1]=4. r=1,c=1: result[1][1]=5. r=1,c=2: result[2][1]=6.
# result=[[1,4],[2,5],[3,6]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) for the output.
# Python one-liner: return [list(row) for row in zip(*matrix)]
# zip(*matrix) unpacks rows as arguments to zip, which pairs corresponding columns.
# Elegant and interview-clean — worth mentioning as an alternative.
# Given more time I'd ask: can we transpose a square matrix in-place?
# Yes — swap matrix[i][j] and matrix[j][i] for all i < j. O(1) extra space."
```

Matrix

#2373 Largest Local Values in a Matrix

EAS

Open on LeetCode

Array · Matrix

Lists: CodeSignal

Problem

You are given an n x n integer matrix grid.

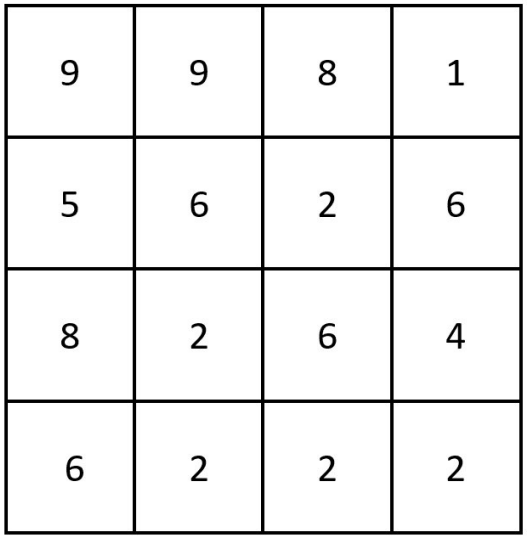
Generate an integer matrix maxLocal of size (n - 2) x (n - 2) such that:

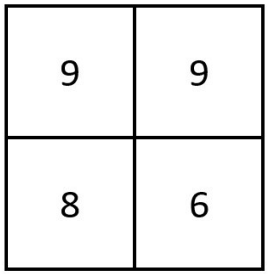
- maxLocal[i][j] is equal to the largest value of the 3 x 3 matrix in grid centered around row i + 1 and column j + 1.

In other words, we want to find the largest value in every contiguous 3 x 3 matrix in grid.

Return the generated matrix.

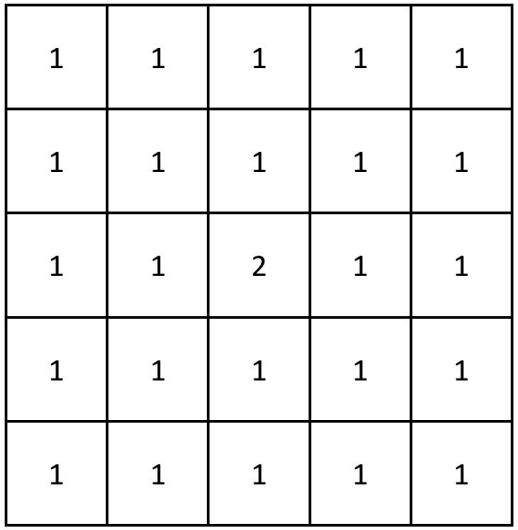
Example 1:

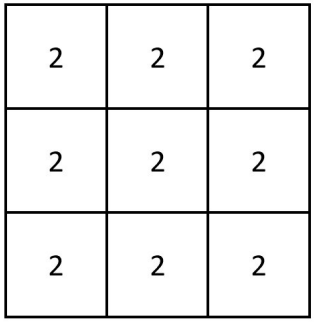





```
Input: grid = [[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]
Output: [[9,9],[8,6]]
Explanation: The diagram above shows the original matrix and the generated matrix.
Notice that each value in the generated matrix corresponds to the largest value of a contiguous 3 x 3 matrix in grid.
```

Example 2:






```
Input: grid = [[1,1,1,1,1],[1,1,1,1,1],[1,1,2,1,1],[1,1,1,1,1],[1,1,1,1,1]]
Output: [[2,2,2],[2,2,2],[2,2,2]]
Explanation: Notice that the 2 is contained within every contiguous 3 x 3 matrix in grid.
```

Constraints:

- n == grid.length == grid[i].length
- 3 <= n <= 100
- 1 <= grid[i][j] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "For each cell (i,j) in the output (n-2)×(n-2) matrix,
# find the max in the 3×3 subgrid of the input centered at (i+1, j+1). Right?
# Input is always n×n square, n >= 3."
#
# "grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]] (4×4):
# output is 2×2. Top-left 3×3 center=(1,1): max of [[9,9,8],[5,6,2],[8,2,6]]=9.
# Top-right 3×3 center=(1,2): max of [[9,8,1],[6,2,6],[2,6,4]]=9. etc. → [[9,9],[8,6]] ✓"

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# For each output cell, scan its 3×3 window. No smarter approach since we must read every
# element at least once. = ."
# For this input size the brute approach is already optimal — I'll still state complexity clearly.
# Time: O(n²*9). Space: O(n²).

# PHASE 3 — OPTIMIZE
# "The 3×3 is fixed size so inner loops are constant — already O(n²).
# Keep the four nested loops clean with descriptive variable names."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# grid=[[9,9,8,1],[5,6,2,6],[8,2,6,4],[6,2,2,2]]: n=4, output 2×2.
# (i=0,j=0): 3×3 from (0,0): max(9,9,8,5,6,2,8,2,6)=9 ✓
# (i=0,j=1): 3×3 from (0,1): max(9,8,1,6,2,6,2,6,4)=9 ✓
# (i=1,j=0): 3×3 from (1,0): max(5,6,2,8,2,6,6,2,2)=8 ✓
# (i=1,j=1): 3×3 from (1,1): max(6,2,6,2,6,4,2,2,2)=6 ✓
# → [[9,9],[8,6]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²) — outer loops O((n-2)²) ≈ O(n²), inner loops constant 9 operations.
# Space O(n²) for the output.
# Given more time I'd ask: what if the window size is variable (k×k instead of 3×3)?
# For large k, recomputing the max from scratch is O(k²) per cell → O(n²k²) total.
# A 2D sliding window maximum using monotonic deques brings it down to O(n²)."
```

Trees & BST

Round 1 · High · Easy · 11 questions

Trees & BST

#100 Same Tree EAS

Open on LeetCode

Tree · DFS · BFS

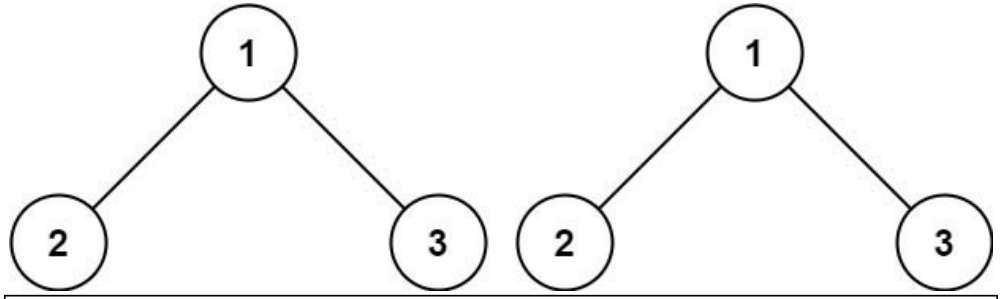
Lists: Grind 169

Problem

Given the roots of two binary trees p and q, write a function to check if they are the same or not.

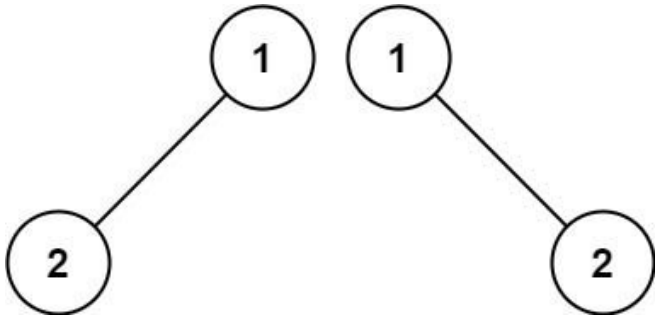
Two binary trees are considered the same if they are structurally identical, and the nodes have the same value.

Example 1:



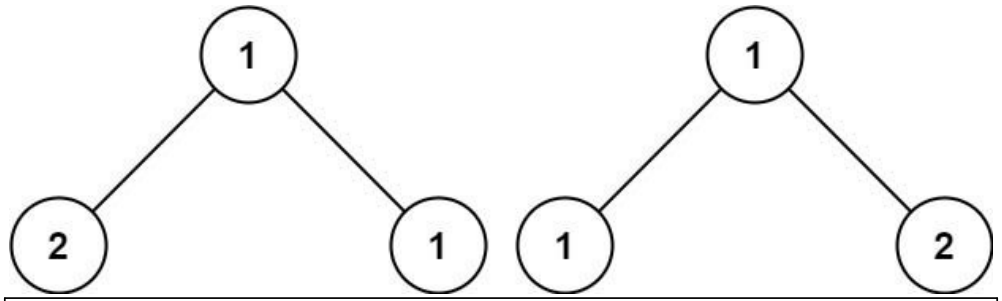

```
Input: p = [1,2,3], q = [1,2,3]
Output: true
```

Example 2:




```
Input: p = [1,2], q = [1,null,2]
Output: false
```

Example 3:




```
Input: p = [1,2,1], q = [1,1,2]
Output: false
```

Constraints:

- **The number of nodes in both trees is in the range [0, 100].**
- **-104 <= Node.val <= 104**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Two binary trees are the same if they're structurally identical AND every
# corresponding node has the same value. Both null trees are the same. Right?
# One null and one non-null at the same position → not the same."
#
# "p=[1,2,3], q=[1,2,3] → true ✓
# p=[1,2], q=[1,null,2] → false (structure differs) ✓
# p=[1,2,1], q=[1,1,2] → false (values differ at depth 1) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Serialise both trees with a level-order traversal (including nulls) to lists, then compare.
# Same O(n) time and O(n) space as the optimal recursive DFS — no meaningful complexity
# improvement available, so we skip the brute class and go straight to the clean approach."

# PHASE 3 — OPTIMIZE
# "Recursive DFS is both simple and optimal.
# Base case: if either node is None, both must be None for equality.
# Recursive: values match AND left subtrees match AND right subtrees match.
# Use an inner helper to avoid self. calls."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# p=[1,2,3], q=[1,2,3]:
# same(1,1): vals match. same(2,2)→same(None,None)=True, same(None,None)=True → True.
# same(3,3)→True. → True ✓
# p=[1,2,None], q=[1,None,2]:
# same(1,1): same(2,None): p=2≠None, q=None → p==q? No → False ✓
# p=None, q=None: not p=True → return p==q=True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — visit every node once. Space O(h) call stack where h is tree height.
# Worst case O(n) for a skewed tree, O(log n) for balanced.
# Iterative alternative: BFS with a queue of node pairs — O(n) time, O(n) space.
# Same complexity but avoids recursion depth limit for very deep trees.
# Given more time I'd ask: what if we only care about structural identity (not values)?
# Same logic — just remove the val check."
```


Trees & BST

#101 Symmetric Tree EAS

Open on LeetCode

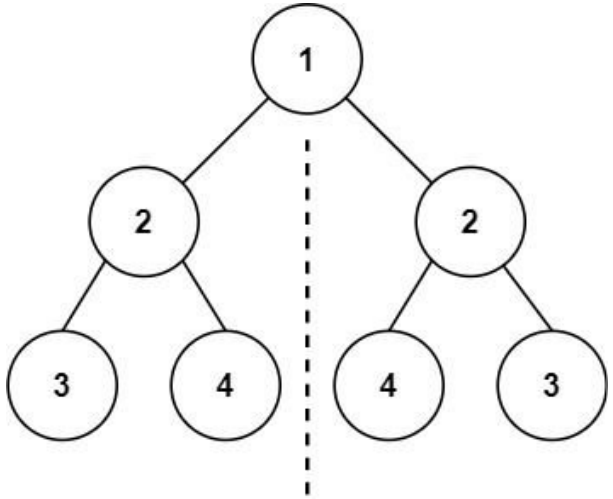
Tree · DFS · BFS

Lists: Grind 169

Problem

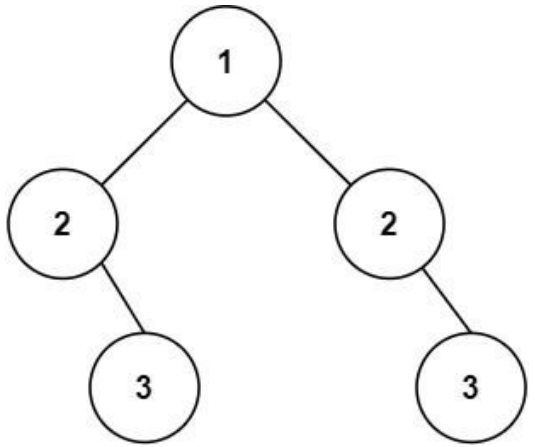
Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).

Example 1:




```
Input: root = [1,2,2,3,4,4,3]
Output: true
```

Example 2:




```
Input: root = [1,2,2,null,3,null,3]
Output: false
```

Constraints:

- **The number of nodes in the tree is in the range [1, 1000].**

- -100 <= Node.val <= 100

Follow up: Could you solve it both recursively and iteratively?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "A tree is symmetric if its left and right subtrees are mirror images of each other.
# Mirror means: at each level, left child of left subtree matches right child of right subtree,
# and right child of left subtree matches left child of right subtree. Right?
# A single node is always symmetric."
#
# "[1,2,2,3,4,4,3]: left subtree [2,3,4] mirrors right [2,4,3] → true ✓
# [1,2,2,null,3,null,3]: left has right child 3, right has right child 3 — not mirrored → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Serialise the left subtree with a pre-order traversal and the right subtree
# with the mirror order (right child before left), then compare the two strings.
# Same O(n) time and O(n) space as the optimal recursive approach — no improvement
# to be had, so we skip the brute class and go straight to the clean solution."

# PHASE 3 — OPTIMIZE
# "Recursive DFS comparing the tree to itself — left vs right mirrored.
# Define an inner helper mirror(p, q):
# Base: if either is None, both must be None.
# Recursive: values match AND mirror(p.left, q.right) AND mirror(p.right, q.left).
# Note the cross-comparison: left's left ↔ right's right, left's right ↔ right's left."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root=[1,2,2,3,4,4,3]:
# mirror(root,root): vals 1==1.
# mirror(left=2, right=2): 2==2.
# mirror(2.left=3, 2.right=3): 3==3. mirror(None,None)=T. mirror(None,None)=T → T
# mirror(2.right=4, 2.left=4): 4==4. T → T
# mirror(right=2, left=2): symmetric to above → T
# → True ✓
# root=[1,2,2,null,3,null,3]:
# mirror(left=2,right=2): 2==2.
# mirror(2.left=None, 2.right=3): p=None,q=3 → None≠3 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h) for the call stack.
# Follow-up asks for iterative solution: use a queue of pairs.
# Enqueue (root.left, root.right). Each step dequeue (p,q):
# check p==q==None (skip), p or q None (false), vals differ (false).
# Enqueue (p.left,q.right) and (p.right,q.left). O(n) time, O(n) space.
# The cross-enqueue order is the key — same insight as the recursive version.
# Given more time I'd ask: how does this differ from Same Tree (#100)?
# Same Tree checks identical structure; Symmetric Tree checks mirrored structure —
# the only difference is the cross-comparison in the recursive call."
```


Trees & BST

#104 Maximum Depth of Binary Tree EAS

Open on LeetCode

Tree · DFS · BFS

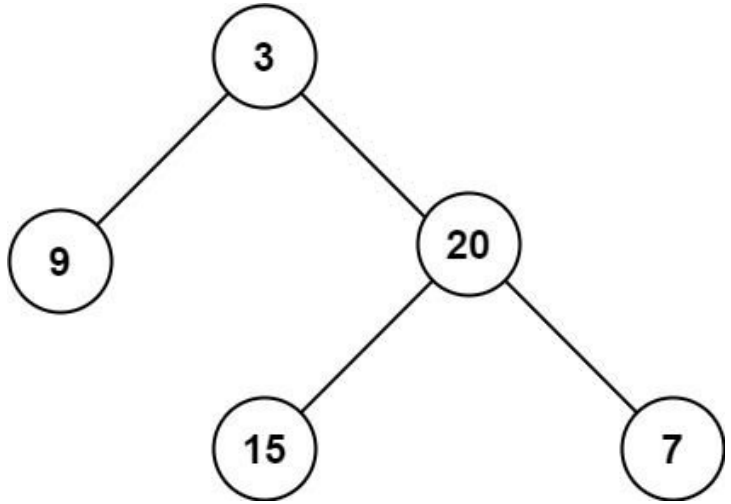
Lists: Grind 169

Problem

Given the root of a binary tree, return its maximum depth.

A binary tree's maximum depth is the number of nodes along the longest path from the root node down to the farthest leaf node.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: 3
```

Example 2:

```
Input: root = [1,null,2]
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- -100 <= Node.val <= 100

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Maximum depth is the number of nodes on the longest root-to-leaf path.
# An empty tree has depth 0. A single node has depth 1. Right?"
#
# "[3,9,20,null,null,15,7]: longest path is 3→20→15 or 3→20→7, 3 nodes → 3 ✓
# [1,null,2]: 1→2, 2 nodes → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Iterative BFS: use a queue, process the tree level by level.
# Each time we exhaust one level, increment a depth counter.
# When the queue is empty, the counter equals the maximum depth.
# Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Recursive DFS is cleaner and same complexity.
# depth(node) = max(depth(left), depth(right)) + 1. Base: empty node → 0.
# Use an inner helper to avoid self. calls."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root=[3,9,20,null,null,15,7]:
# depth(3)=max(depth(9),depth(20))+1.
```

```
# depth(9)=max(0,0)+1=1. depth(20)=max(depth(15),depth(7))+1=max(1,1)+1=2.
# depth(3)=max(1,2)+1=3 ✓
# root=None: depth(None)=0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h) call stack — O(log n) balanced, O(n) skewed.
# BFS alternative: count levels with a queue — O(n) time, O(w) space (w=max width).
# For very deep trees, BFS avoids recursion stack overflow.
# Given more time I'd ask: what about minimum depth? Different — must reach a leaf,
# so BFS is actually better there (stops at first leaf found)."
```


Trees & BST

#108 Convert Sorted Array to Binary Search Tree

EAS

Open on LeetCode

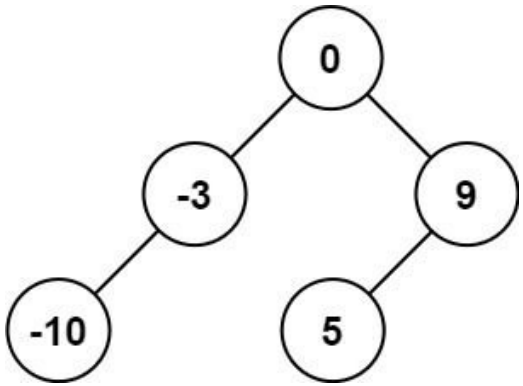
Array · Divide and Conquer · Tree

Lists: Grind 169

Problem

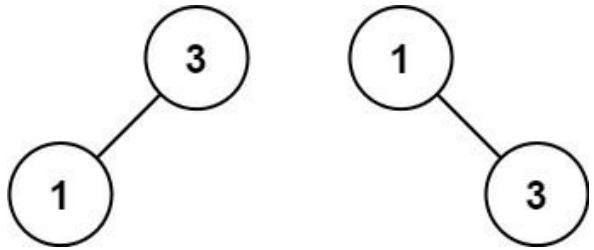
Given an integer array nums where the elements are sorted in ascending order, convert it to a height-balanced binary search tree.

Example 1:




```
Input: nums = [-10,-3,0,5,9]
Output: [0,-3,9,-10,null,5]
Explanation: [0,-10,5,null,-3,null,9] is also accepted:
```

Example 2:




```
Input: nums = [1,3]
Output: [3,1]
Explanation: [1,null,3] and [3,1] are both height-balanced BSTs.
```

Constraints:

- 1 <= nums.length <= 104
- -104 <= nums[i] <= 104
- nums is sorted in a strictly increasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert a sorted array to a height-balanced BST.
# Height-balanced means every node's left and right subtree heights differ by at most 1.
# Multiple valid answers exist — any height-balanced BST is acceptable. Right?"
#
# "[-10,-3,0,5,9]: pick middle element as root → 0. Left half [-10,-3] → left subtree.
# Right half [5,9] → right subtree. Recurse. → [0,-3,9,-10,null,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Pick the middle element of nums as root, recursively build left from nums[:mid]
# and right from nums[mid+1:].
```

```
# Always produces a valid BST shape for a sorted input.
# Time: O(n log n) from repeated slicing. Space: O(n) for slices.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Pick the middle element as root — this guarantees balance because both halves
# have equal (or ±1) size. Recurse on left and right halves.
# Divide and conquer. Use inner helper build(l, r) operating on indices."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-10,-3,0,5,9]:
# build(0,4): mid=2→node(0). left=build(0,1), right=build(3,4).
# build(0,1): mid=0→node(-10). left=None, right=build(1,1)→node(-3). → (-10,None,-3)
# build(3,4): mid=3→node(5). left=None, right=build(4,4)→node(9). → (5,None,9)
# root=0, left=(-10,None,-3), right=(5,None,9). Valid height-balanced BST ✓
# nums=[1,3]: mid=0→node(1), right=build(1,1)→node(3). → [1,null,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — every element becomes a node exactly once.
# Space O(log n) for the call stack — tree is balanced by construction.
# Choosing (l+r)//2 vs (l+r+1)//2 gives slightly different but equally valid trees.
# Given more time I'd ask: what if the array is sorted in descending order?
# Reverse it first or adjust the index logic — same approach."
```


Trees & BST

#110 Balanced Binary Tree EAS

Open on LeetCode

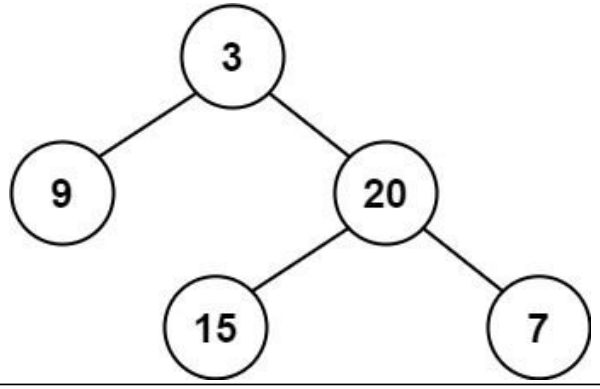
Tree · DFS

Lists: Grind 169

Problem

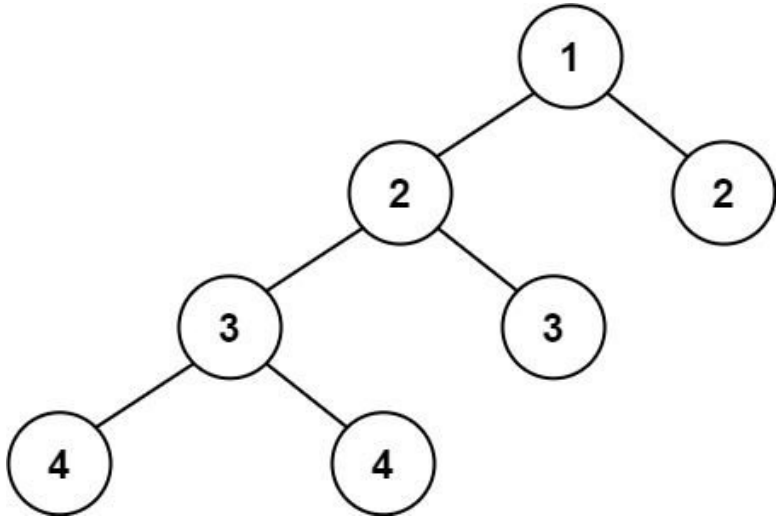
Given a binary tree, determine if it is height-balanced.

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: true
```

Example 2:




```
Input: root = [1,2,2,3,3,null,null,4,4]
Output: false
```

Example 3:

```
Input: root = []
Output: true
```

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- -104 <= Node.val <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A tree is height-balanced if every node's left and right subtree heights
# differ by at most 1. Empty tree is balanced. Right?"
#
# "[3,9,20,null,null,15,7]: heights of 9's subtrees: 0 vs 0 ✓. 20's: 1 vs 1 ✓. 3's: 1 vs 2 ✓ → true.
# [1,2,2,3,3,null,null,4,4]: left subtree of root has height 3, right has 1 → false ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# At each node compute left subtree height and right subtree height separately.
# Node is balanced if |left-right|<=1 and both subtrees are balanced recursively.
# Recomputes heights from scratch at every node - O(n^2) on skewed trees.
# Time: O(n^2) worst case. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The brute force calls height() repeatedly for the same nodes — O(n²).
# I notice we're recomputing heights we've already seen. What if we cache it?
#
# Better: combine the height computation and balance check into one DFS pass.
# Return -1 to signal 'unbalanced subtree found' — propagate it up immediately.
# Otherwise return the actual height.
# A single O(n) pass handles everything."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root=[3,9,20,null,null,15,7]:
# check(9)=1. check(15)=1. check(7)=1. check(20)=max(1,1)+1=2.
# check(3): left=1,right=2. |1-2|=1 <=1 → return 2. result≠-1 → True ✓
# root=[1,2,2,3,3,null,null,4,4]:
# check(4)=1. check(3,left)=max(1,0)+1=2. check(3,right)=max(1,0)+1=2.
# check(2,left): left=2,right=2 → 3. check(2,right): left=0,right=0 → 1.
# check(1): left=3,right=1. |3-1|=2>1 → return -1. result==-1 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — single DFS, each node visited once.
# Space O(h) for the call stack.
# The -1 sentinel trick is the key interview insight: instead of separate
# height and balance checks, fuse them into one return value with a special signal.
# Given more time I'd ask: can we rebalance a tree in-place?
# Flatten to sorted array (in-order traversal) then rebuild — O(n) time."
```


Trees & BST

#112 Path Sum EAS

Open on LeetCode

Tree · DFS · BFS

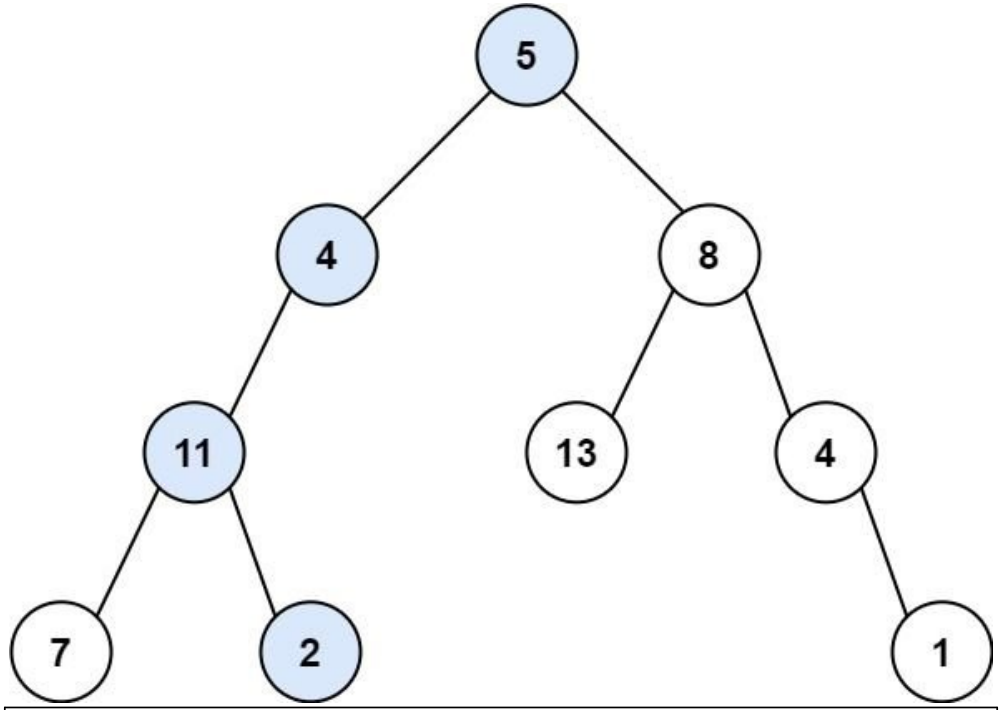
Lists: Denny Zhang

Problem

Given the root of a binary tree and an integer targetSum, return true if the tree has a root-to-leaf path such that adding up all the values along the path equals targetSum.

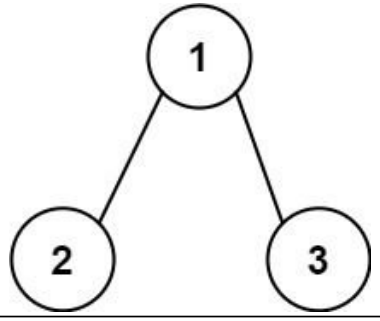
A leaf is a node with no children.

Example 1:




```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,null,1], targetSum = 22
Output: true
Explanation: The root-to-leaf path with the target sum is shown.
```

Example 2:




```
Input: root = [1,2,3], targetSum = 5
Output: false
Explanation: There are two root-to-leaf paths in the tree:
(1 --> 2): The sum is 3.
(1 --> 3): The sum is 4.
There is no root-to-leaf path with sum = 5.
```

Example 3:

```
Input: root = [], targetSum = 0
Output: false
Explanation: Since the tree is empty, there are no root-to-leaf paths.
```

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- -1000 <= Node.val <= 1000
- -1000 <= targetSum <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find if any root-to-leaf path sums to targetSum.
# A leaf is a node with no children — the path must end at a leaf. Right?
# Empty tree → false (no leaves)."
#
# "[5,4,8,11,null,13,4,7,2,null,null,null,1], target=22:
# path 5→4→11→2 = 22 → true ✓
# [], target=0 → false (empty tree has no root-to-leaf path) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# DFS traversal collecting a running sum as we descend. At each leaf node,
# check if the accumulated sum equals targetSum. Backtrack and try other paths.
# The optimal approach is the same DFS — just written more elegantly by subtracting
# from target instead of accumulating. Both are O(n) time and O(h) space, so we skip
# a separate brute class and go straight to the clean version."

# PHASE 3 — OPTIMIZE
# "More elegant: subtract the current node's value from targetSum as we descend.
# At a leaf, check if remaining == 0. Avoids passing a separate running sum."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root=[5,4,8,...], target=22:
# hassum(5,22): rem=17. hassum(4,17): rem=13. hassum(11,13): rem=2.
# hassum(7,2): rem=-5, leaf → -5≠0 False.
# hassum(2,2): rem=0, leaf → True ✓
# root=[], target=0: hassum(None,0)=False ✓
# root=[1,2], target=1: hassum(1,1): rem=0, not leaf.
# hassum(2,0): rem=-2, leaf → False. → False ✓ (path must reach a leaf)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h) call stack.
# The 'leaf check' (no left and no right) is critical — without it, a node
# with one child that hits 0 would incorrectly return true.
# Given more time I'd ask: return all root-to-leaf paths that sum to target?
# That's Path Sum II (#113) — collect paths into a result list with backtracking."
```


Trees & BST

#226 Invert Binary Tree

EAS

Open on LeetCode

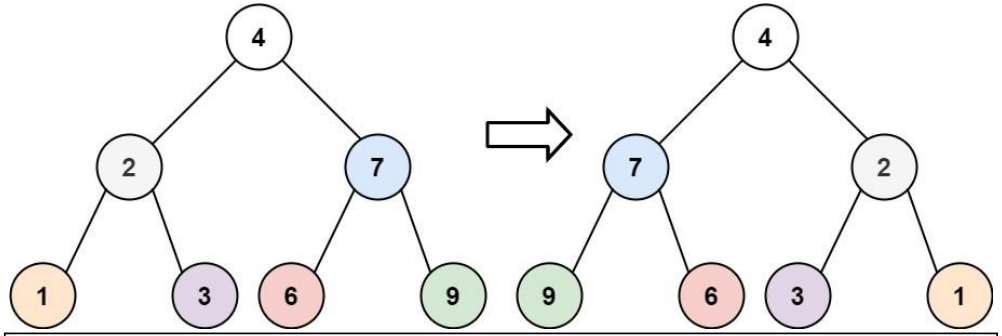
Tree · DFS · BFS

Lists: Grind 169

Problem

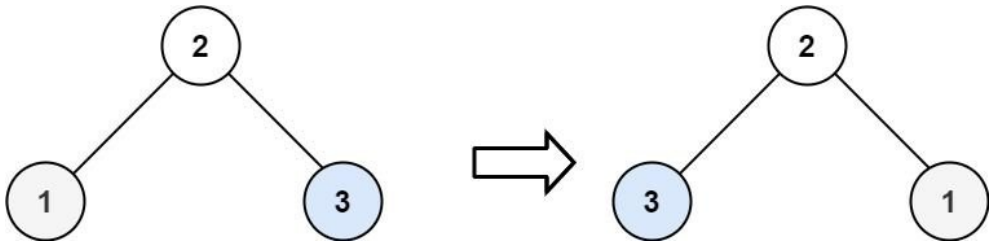
Given the root of a binary tree, invert the tree, and return its root.

Example 1:




```
Input: root = [4,2,7,1,3,6,9]
Output: [4,7,2,9,6,3,1]
```

Example 2:




```
Input: root = [2,1,3]
Output: [2,3,1]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 100].
- -100 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Invert the tree: swap left and right children at every node recursively.
# Return the root. Empty tree → return None. Right?"
#
# "[4,2,7,1,3,6,9] → [4,7,2,9,6,3,1]: left and right subtrees are swapped at every level ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Iterative BFS with a queue: visit each node level by level and swap its left
# and right children in place. Process until the queue is empty.
# Time: O(n). Space: O(n) for the queue. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Recursive DFS is cleaner. Invert left, invert right, then swap.
# Or: swap first then recurse — same result. Either order works.
# Use inner helper to avoid self. calls."

# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# root=[4,2,7]:
# invert(2): invert(None)=None, invert(None)=None. swap: 2.left=None,2.right=None. return 2.
# invert(7): same → 7.
# invert(4): left=invert(2)=2, right=invert(7)=7. swap: 4.left=7, 4.right=2. return 4.
# → [4,7,2] ✓ (recurse deeper for full tree)
# root=None: invert(None)=None ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h) call stack.
# This is the problem Homebrew creator Max Howell famously couldn't solve in a Google interview.
# Worth mentioning — it humanises the problem and shows you know your history.
# Given more time I'd ask: can we do this iteratively? Yes — BFS or DFS with a stack,
# swap children at each node. O(n) time, O(n) space."
```


Trees & BST

★ #270 Closest Binary Search Tree Value ★ EAS

Open on LeetCode

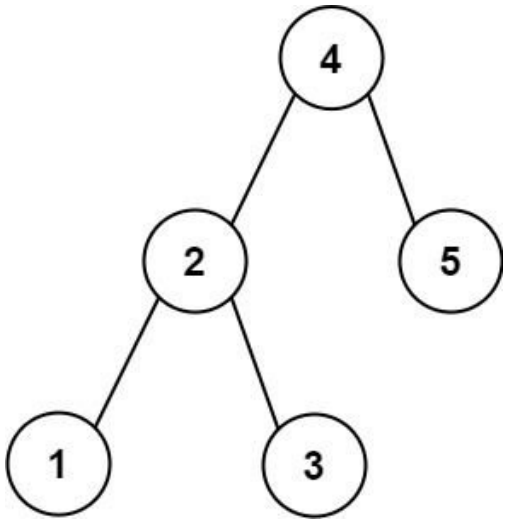
Binary Search · Tree · DFS · BST

Lists: ★ Premium | Premium 98

Problem

Given the root of a binary search tree and a target value, return the value in the BST that is closest to the target. If there are multiple answers, print the smallest.

Example 1:




```
Input: root = [4,2,5,1,3], target = 3.714286
Output: 4
```

Example 2:

```
Input: root = [1], target = 4.428571
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 109
- -109 <= target <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the node value in a BST closest to a float target.
# If two values are equidistant, return the smaller one. Right?
# BST property means we can navigate toward the target rather than scanning all nodes."
#
# "root=[4,2,5,1,3], target=3.714286:
# candidates: 4 (dist=0.286), 3 (dist=0.714). 4 is closer → 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Full DFS scan: visit every node, track the value with minimum |val − target|.
# On a tie (equal distance), prefer the smaller value.
# This ignores the BST property entirely — valid but O(n) time and O(n) stack space.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Use BST property to navigate: at each node, the closest value is either the
# current node or something in the subtree direction of the target.
# If target < node.val → go left; if target > node.val → go right.
# Track the closest seen so far. O(h) time — O(log n) balanced, O(n) skewed."

# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# root=[4,2,5,1,3], target=3.714286:
# close(4, 4): |3.714-4|=0.286 < |3.714-4|=0.286 → no update. target<4 → go left.
# close(2, 4): |3.714-2|=1.714 > 0.286 → best stays 4. target>2 → go right.
# close(3, 4): |3.714-3|=0.714 > 0.286 → best stays 4. target>3 → go right.
# close(None, 4): return 4 ✓
# root=[1], target=4.428: close(1,1): target>1 → go right → close(None,1) → 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(h). Space O(h) for recursion stack. Iterative: O(1) space.
# BST navigation is the key — we prune entire subtrees we know can't contain
# a closer value.
# Given more time I'd ask: find the k closest values to target?
# That's Closest BST Value II (#272) — use an in-order traversal with a sliding window
# or a max-heap of size k."
```


Trees & BST

#501 Find Mode in Binary Search Tree

EAS

Open on LeetCode

Tree · DFS · BST

Lists: Denny Zhang

Problem

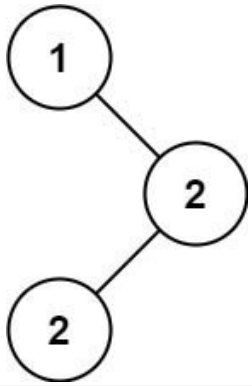
Given the root of a binary search tree (BST) with duplicates, return all the mode(s) (i.e., the most frequently occurred element) in it.

If the tree has more than one mode, return them in any order.

Assume a BST is defined as follows:

- The left subtree of a node contains only nodes with keys less than or equal to the node's key.
- The right subtree of a node contains only nodes with keys greater than or equal to the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [1,null,2,2]
Output: [2]
```

Example 2:

```
Input: root = [0]
Output: [0]
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -105 <= Node.val <= 105

Follow up: Could you do that without using any extra space? (Assume that the implicit stack space incurred due to recursion does not count).

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all values that appear the most frequently in the BST.
# This BST allows duplicates (left ≤ node, right ≥ node).
# Return all modes — there can be multiple. Any order is fine. Right?
# Follow-up: can we do it without extra space (O(1) beyond recursion stack)?"
#
# "[1,null,2,2]: 2 appears twice, 1 once → mode is [2] ✓
# [0]: only element → mode is [0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# DFS or BFS over all nodes and count frequencies with a hash map (Counter).
# Then collect all keys whose count equals the maximum frequency.
# Time: O(n). Space: O(n) for the Counter. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (O(1) space using in-order traversal)
# "BST in-order traversal visits nodes in sorted order.
# Duplicates in a BST are adjacent in in-order — so we can count runs without a map.
# Track: current value, current run count, max count seen, result list.
# When current run count exceeds max, reset result. When equal, append.
# Use nonlocal to mutate state inside the inner helper."

# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# root=[1,null,2,2] — in-order: 1, 2, 2
# node=1: cur_val=1,cnt=1,max=1 → modes=[1].
# node=2: cur_val=2,cnt=1,max=1 → cnt==max → modes=[1,2].
# node=2: cur_val=2,cnt=2,max=1 → cnt>max → max=2,modes=[2].
# return [2] ✓
# root=[0]: in-order: 0. modes=[0] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute force: O(n) time, O(n) space.
# O(1) space version: O(n) time, O(1) extra space (modes list not counted).
# The in-order run-counting trick works because BST duplicates are always adjacent
# in sorted traversal — a key property of this BST variant.
# Given more time I'd ask: what if it were a regular BST (no duplicates)?
# The mode would always be the single element — trivially the root if n=1."
```


Trees & BST

#543 Diameter of Binary Tree

EAS

Open on LeetCode

Tree · DFS

Lists: Grind 169

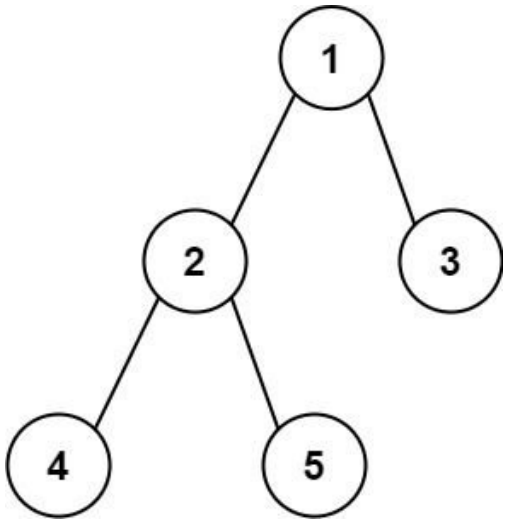
Problem

Given the root of a binary tree, return the length of the diameter of the tree.

The diameter of a binary tree is the length of the longest path between any two nodes in a tree. This path may or may not pass through the root.

The length of a path between two nodes is represented by the number of edges between them.

Example 1:




```
Input: root = [1,2,3,4,5]
Output: 3
Explanation: 3 is the length of the path [4,2,1,3] or [5,2,1,3].
```

Example 2:

```
Input: root = [1,2]
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -100 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Diameter = the longest path between any two nodes, measured in edges.
# The path may or may not pass through the root.
# An empty tree has diameter 0. A single node has diameter 0 (no edges). Right?"
#
# "[1,2,3,4,5]: longest path is 4→2→1→3, 3 edges → 3 ✓
# [1,2]: one edge → 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every node, compute the height of its left and right subtrees recursively.
# Diameter through that node = left_height + right_height; track the global max.
# Recomputes heights repeatedly — correct but redundant work.
# Time: O(n^2) on skewed trees. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "I notice depth is recomputed repeatedly for the same nodes — that's the O(n²).
# Fuse depth computation and diameter tracking into one DFS pass.
# Use a nonlocal variable (or single-element list) to capture the running max.
```

```
# depth(node) returns height AND updates the max diameter seen so far.
# Time: O(n) — one pass. Space: O(h)."
```

PHASE 4 — CLEAN CODE

PHASE 5 — TEST & DEBUG

```
# root=[1,2,3,4,5]:
# depth(4)=1. depth(5)=1. depth(2): best=max(0,1+1)=2. return 2.
# depth(3)=1. depth(1): best=max(2,2+1)=3. return 3.
# return best[0]=3 ✓
# root=[1,2]: depth(2)=1. depth(1): best=max(0,1+0)=1. return best=1 ✓
# root=None: depth(None)=0. best stays 0. return 0 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h).
# The single-element list best=[0] is a common Python pattern to mutate
# a value from inside a nested function without making it a class attribute.
# Equivalent to 'nonlocal best' with a plain int, but the list avoids the
# nonlocal keyword — both are valid in an interview.
# Given more time I'd ask: what's the longest path by node count (not edges)?
# Just return best[0] + 1 when best > 0. Or: diameter in edge terms is already
# what we compute — node count would be edges + 1."
```


Trees & BST

#572 Subtree of Another Tree

EAS

Open on LeetCode

Tree · DFS · String Matching

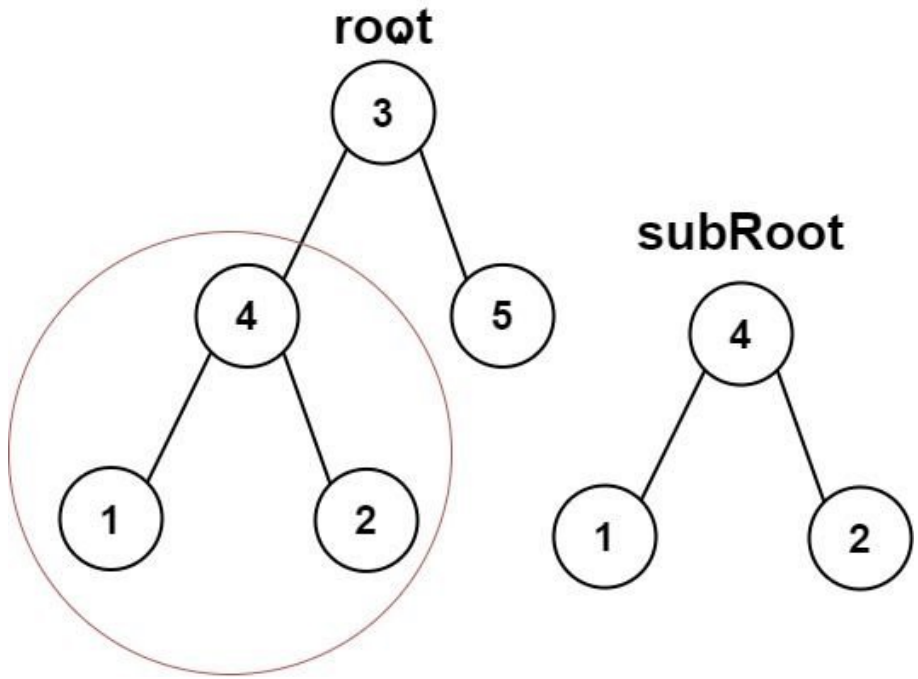
Lists: Grind 169

Problem

Given the roots of two binary trees root and subRoot, return true if there is a subtree of root with the same structure and node values of subRoot and false otherwise.

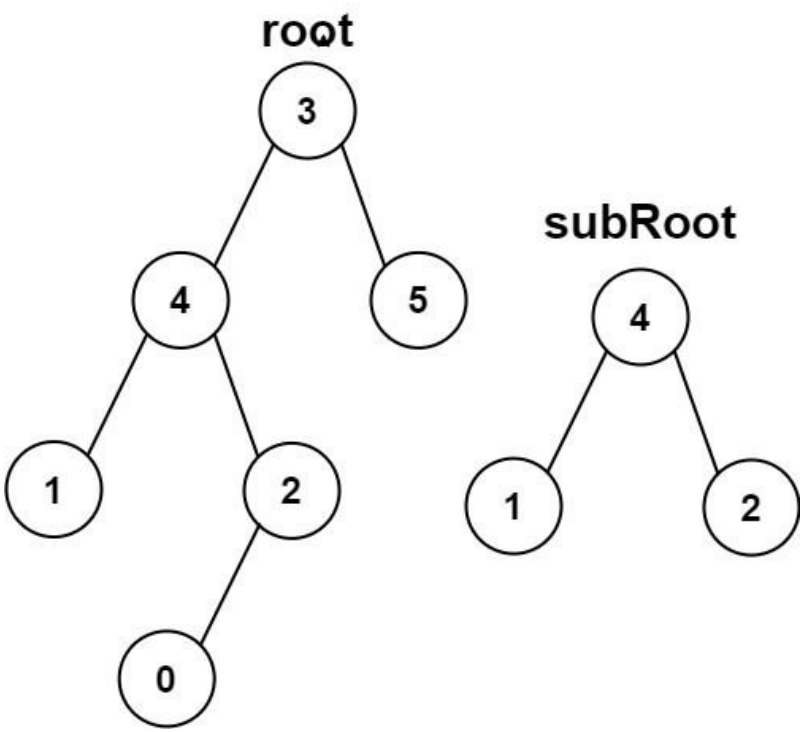
A subtree of a binary tree tree is a tree that consists of a node in tree and all of this node's descendants. The tree tree could also be considered as a subtree of itself.

Example 1:




```
Input: root = [3,4,5,1,2], subRoot = [4,1,2]
Output: true
```

Example 2:




```
Input: root = [3,4,5,1,2,null,null,null,null,0], subRoot = [4,1,2]
Output: false
```

Constraints:

- **The number of nodes in the root tree is in the range [1, 2000].**
- **The number of nodes in the subRoot tree is in the range [1, 1000].**
- **-104 <= root.val <= 104**
- **-104 <= subRoot.val <= 104**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return true if subRoot appears as a subtree anywhere in root.
# A subtree means a node and ALL its descendants match — not just a portion.
# The tree itself is a subtree of itself. Right?"
#
# "root=[3,4,5,1,2], subRoot=[4,1,2]: node 4 in root has left=1,right=2 → matches ✓
# root=[3,4,5,1,2,null,null,null,null,0], subRoot=[4,1,2]:
# node 4 in root has extra child 0 → doesn't match ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every node in root, run the same-tree check against all of subRoot.
# If any starting node matches the entire subtree, return true.
# Correct, but rescans overlapping structure many times.
# Time: O(n * m). Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Same O(m*n) is standard. Two clean inner helpers:
# same(p,q) — checks if two trees are identical (reuse Same Tree logic).
# subtree(node, sub) — DFS root; at each node, try same().
# If same fails, recurse into left and right subtrees."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
```



```
# root=[3,4,5,1,2], subRoot=[4,1,2]:
# subtree(3,[4,1,2]): same(3,4)? 3≠4 → False. recurse left/right.
# subtree(4,[4,1,2]): same(4,4)? 4==4.
# same(1,1)→same(None,None)T, same(None,None)T→T.
# same(2,2)→T. → same=True → return True ✓
# root has extra 0 child under 4:
# same(4,4): same(1,1)T. same(2(with 0),2)? 2==2 but same(0,None)→False → same=False.
# subtree(1,[4,1,2]): same(1,4)? False. subtree deeper → all False → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n) — at each of m nodes we may run same() which takes O(n).
# Space O(h_root + h_sub) for call stacks.
# Optimisation: serialise both trees and use string matching (KMP or Rabin-Karp)
# for O(m+n) time — worth mentioning as an advanced follow-up.
# Be careful with serialisation: '[1,2]' would incorrectly match '[12]' without
# delimiters like '#' for null and ',' between values.
# Given more time I'd ask: what if we query many different subRoots against
# the same root? Pre-serialise root once and use string search for each query."
```

DFS

Round 1 · High · Easy · 2 questions

DFS

#463 Island Perimeter EAS

Open on LeetCode

Array · DFS · BFS · Matrix

Lists: Denny Zhang

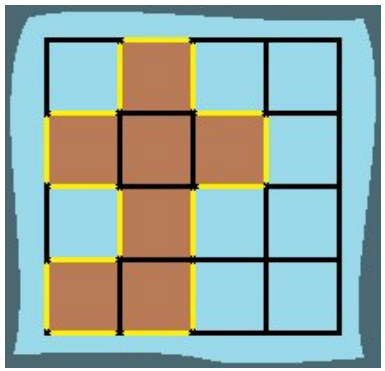
Problem

You are given row x col grid representing a map where grid[i][j] = 1 represents land and grid[i][j] = 0 represents water.

Grid cells are connected horizontally/vertically (not diagonally). The grid is completely surrounded by water, and there is exactly one island (i.e., one or more connected land cells).

The island doesn't have "lakes", meaning the water inside isn't connected to the water around the island. One cell is a square with side length 1. The grid is rectangular, width and height don't exceed 100. Determine the perimeter of the island.

Example 1:



```
Input: grid = [[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]
Output: 16
Explanation: The perimeter is the 16 yellow stripes in the image above.
```

Example 2:

```
Input: grid = [[1]]
Output: 4
```

Example 3:

```
Input: grid = [[1,0]]
Output: 4
```

Constraints:

- row == grid.length
- col == grid[i].length
- 1 <= row, col <= 100
- grid[i][j] is 0 or 1.
- There is exactly one island in grid.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "The grid has exactly one island — no lakes. Each land cell is a unit square.
# We return the total perimeter, which is the count of exposed edges. Right?
# Exposed means the edge faces water or the grid boundary."
#
# "Quick check: [[1]] → 4 sides, all exposed → 4 ✓. [[1,0]] → 4 sides, all exposed → 4 ✓.
# Two adjacent land cells: each has 4 sides but share 1 edge → 2*4 − 2 = 6."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every land cell, I check all 4 neighbours — if a neighbour is out of bounds or water,
# that edge contributes 1 to the perimeter. Sum these up across all land cells.
# Time: O(m*n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "I notice we're checking 4 neighbours per cell. Instead of neighbour checks,
# we can use a counting formula:
# Every land cell contributes 4 sides. Every shared edge between two adjacent
# land cells removes 2 sides (one from each). So:
# perimeter = islands * 4 − shared_edges * 2
# We only count each shared edge once by checking right-neighbour and down-neighbour.
```

```
# Same O(m*n) time, same O(1) space — but a cleaner mental model."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# grid=[[1]]: islands=1, shared=0. return 4 ✓
# grid=[[1,1]]: islands=2. c=0→c+1=1 valid,grid[0][1]=1→shared=1. return 2*4-1*2=6 ✓
# grid=[[0,1,0,0],[1,1,1,0],[0,1,0,0],[1,1,0,0]]:
# Count land cells=8 (islands). Count shared edges (right+down only)=... =8.
# return 8*4-8*2=32-16=16 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1).
# The counting formula is elegant — worth presenting even if the brute force also works.
# Both approaches are the same complexity; the formula shows mathematical insight.
# Given more time I'd ask: what if there are multiple islands?
# The formula still works per-island if we isolate them, or we can return a list
# of perimeters — same logic applied per connected component."
```


DFS

#733 Flood Fill EAS

Open on LeetCode

Array · DFS · BFS · Matrix

Lists: Grind 169

Problem

You are given an image represented by an m x n grid of integers image, where image[i][j] represents the pixel value of the image. You are also given three integers sr, sc, and color. Your task is to perform a flood fill on the image starting from the pixel image[sr][sc].

To perform a flood fill:

1. Begin with the starting pixel and change its color to color.
2. Perform the same process for each pixel that is directly adjacent (pixels that share a side with the original pixel, either horizontally or vertically) and shares the same color as the starting pixel.
3. Keep repeating this process by checking neighboring pixels of the updated pixels and modifying their color if it matches the original color of the starting pixel.
4. The process stops when there are no more adjacent pixels of the original color to update.

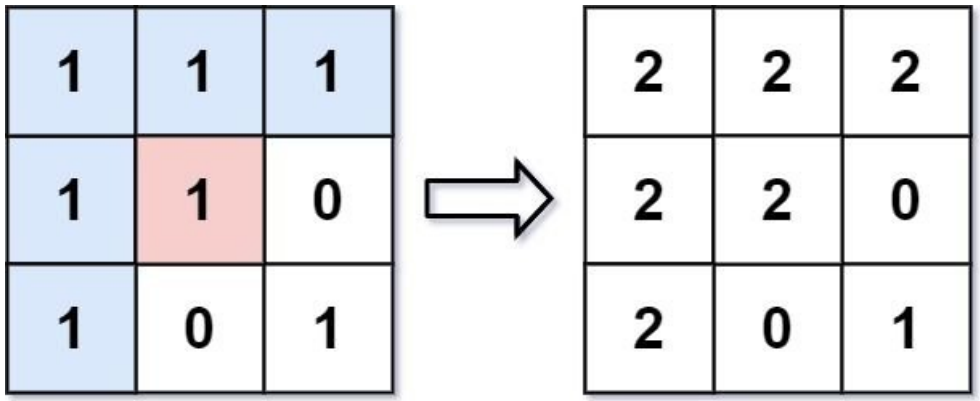
Return the modified image after performing the flood fill.

Example 1:

Input: image = [[1,1,1],[1,1,0],[1,0,1]], sr = 1, sc = 1, color = 2

Output: [[2,2,2],[2,2,0],[2,0,1]]

Explanation:



From the center of the image with position (sr, sc) = (1, 1) (i.e., the red pixel), all pixels connected by a path of the same color as the starting pixel (i.e., the blue pixels) are colored with the new color.

Note the bottom corner is not colored 2, because it is not horizontally or vertically connected to the starting pixel.

Example 2:

Input: image = [[0,0,0],[0,0,0]], sr = 0, sc = 0, color = 0

Output: [[0,0,0],[0,0,0]]

Explanation:

The starting pixel is already colored with 0, which is the same as the target color. Therefore, no changes are made to the image.

Constraints:

- m == image.length
- n == image[i].length
- 1 <= m, n <= 50
- 0 <= image[i][j], color < 216
- 0 <= sr < m
- 0 <= sc < n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Starting from (sr, sc), change all connected cells of the same colour to the new colour.
# Connected means 4-directionally (up/down/left/right). Modify and return the image. Right?
```

```
# Edge case: if the starting colour already equals the new colour — return immediately,
# otherwise we'd loop infinitely or process unchanged cells."
#
# "[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2:
# All 1s connected to (1,1) become 2. The bottom-right 1 is isolated → stays 1. ✓
# [[0,0,0],[0,0,0]], sr=0,sc=0, color=0: startcolor==color → no change → return as-is ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# I'll use BFS: enqueue the start cell, paint it the new colour, then for each cell I dequeue,
# enqueue all 4-directional neighbours that still have the original start colour.
# This visits every connected cell exactly once.
# Time: O(m*n). Space: O(m*n) for the queue. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "DFS is cleaner. Key insight: once we paint a cell, its colour changes to `color` —
# it can never match `startcolor` again. So we don't need a `seen` set at all.
# The colour change itself is the visited marker.
# This saves O(m*n) extra space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# image=[[1,1,1],[1,1,0],[1,0,1]], sr=1,sc=1, color=2, start=1:
# dfs(1,1): image[1][1]=2. recurse 4 dirs.
# dfs(2,1): image[2][1]=1==start → paint 2. recurse: dfs(3,1)→OOB, dfs(1,1)→now 2≠1 stop.
# dfs(2,2)→image[2][2]=1 but that's the isolated corner — wait:
# (2,1)→image[2][1]=1✓→paint. Then dfs(2,2)→image[2][2]=1✓→paint? No —
# is (2,2) reachable from (1,1)? Path: (1,1)→(2,1)→(2,2).
# image[2][2]=1 and image[2][1]=1, so yes connected. But expected output keeps it 1?
# Re-check: image=[[1,1,1],[1,1,0],[1,0,1]]. (2,2) is adjacent to (2,1)=0 and (1,2)=0.
# So (2,2) connects to (2,1)=0 (water) — not reachable from island. ✓
# All connected 1s become 2. Bottom-right stays 1 (not connected). ✓
#
# start==color: image=[[0,0,0],[0,0,0]], color=0 → return immediately ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n) — each cell visited at most once.
# Space O(m*n) call stack in the worst case (all cells same colour, grid is a long chain).
# The 'paint first then recurse' order is the key — it prevents revisiting and
# eliminates the need for a separate visited set. Mention this explicitly.
# The early return when startcolor == color prevents infinite looping — critical edge case.
# Given more time I'd ask: what if connectivity is 8-directional (diagonals too)?
# Add four more direction pairs to the dfs calls — same algorithm."
```


Round 2 · High · Medium

Round 2

High Priority · Medium
116 questions · 11 patterns

Arrays & Hashing #57 #238 #281 #307 #454 #525 #560 #1010 #1272 #1429 #3159
String #8 #249 #271 #635 #1138
Two Pointers #11 #15 #16 #31 #75 #161 #167 #186 #189 #532 #923 #986 #1055 #2563
Sliding Window #3 #159 #340 #424 #438 #487 #1100 #1248
Sorting #49 #56 #1152
Binary Search #33 #34 #153 #300 #362 #528 #852 #981 #1011 #1060 #1062 #1533
Matrix #36 #48 #54 #59 #64 #73 #74 #221 #311 #498 #531 #723 #835 #1198
Trees & BST #98 #102 #103 #105 #199 #230 #235 #236 #250 #285 #298 #314 #333 #366 #437 #545 #549 #582 #662 #863 #1120 #1490 #1522 #1650
DFS #130 #133 #200 #207 #210 #261 #310 #323 #417 #490 #694 #695 #721 #785 #1236 #1242
Graphs #128 #277 #1136
BFS #286 #322 #542 #994 #1197 #1730

Arrays & Hashing

Round 2 · High · Medium · 11 questions

Arrays & Hashing

#57 Insert Interval

MED

Open on LeetCode

Array

Lists: Grind 169

Problem

You are given an array of non-overlapping intervals intervals where intervals[i] = [starti, endi] represent the start and the end of the ith interval and intervals is sorted in ascending order by starti. You are also given an interval newInterval = [start, end] that represents the start and end of another interval.

Insert newInterval into intervals such that intervals is still sorted in ascending order by starti and intervals still does not have any overlapping intervals (merge overlapping intervals if necessary).

Return intervals after the insertion.

Note that you don't need to modify intervals in-place. You can make a new array and return it.

Example 1:

```
Input: intervals = [[1,3],[6,9]], newInterval = [2,5]
Output: [[1,5],[6,9]]
```

Example 2:

```
Input: intervals = [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval = [4,8]
Output: [[1,2],[3,10],[12,16]]
Explanation: Because the new interval [4,8] overlaps with [3,5],[6,7],[8,10].
```

Constraints:

- 0 <= intervals.length <= 104
- intervals[i].length == 2
- 0 <= starti <= endi <= 105
- intervals is sorted by starti in ascending order.
- newInterval.length == 2
- 0 <= start <= end <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We have a sorted list of non-overlapping intervals. Insert a new interval,
# merging wherever it overlaps existing ones. Return the result sorted.
# We don't need to modify in-place — a new array is fine. Right?"
#
# "[[1,3],[6,9]], newInterval=[2,5]:
# [2,5] overlaps [1,3] → merge to [1,5]. [6,9] doesn't overlap → keep. → [[1,5],[6,9]] ✓
# [[1,2],[3,5],[6,7],[8,10],[12,16]], newInterval=[4,8]:
# Overlaps [3,5],[6,7],[8,10] → merge all to [3,10]. → [[1,2],[3,10],[12,16]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# I'll append the new interval to the list, sort everything by start time,
# then run a standard merge-intervals pass to collapse all overlaps.
# Time: O(n log n) — the sort dominates. Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "The input is already sorted — we can exploit that to avoid the O(n log n) sort.
# Three phases in one O(n) pass:
# 1. Add all intervals that end BEFORE newInterval starts (no overlap).
# 2. Merge all intervals that overlap newInterval into one combined interval.
# Overlap condition: interval[start] <= newInterval[end].
# 3. Add all remaining intervals that start AFTER the merged interval ends.
#
# Pseudocode:
# i = 0
# while i < n and intervals[i][1] < new[0]: result.append(intervals[i]); i++
# while i < n and intervals[i][0] <= new[1]:
# new = [min(new[0], intervals[i][0]), max(new[1], intervals[i][1])]; i++
# result.append(new)
# result.extend(intervals[i:])
#
# Time: O(n). Space: O(n) for output."

# PHASE 4 — CLEAN CODE
        # Phase 1: all intervals ending before newInterval starts
        # Phase 2: merge all overlapping intervals into newInterval
        # Phase 3: remaining intervals start after merged interval

# PHASE 5 — TEST & DEBUG
# intervals=[[1,3],[6,9]], new=[2,5]:
# Phase 1: intervals[0][1]=3 >= new[0]=2 → stop. i=0.
# Phase 2: intervals[0][0]=1 <= new[1]=5 → new=[min(2,1),max(5,3)]=[1,5]. i=1.
# intervals[1][0]=6 > new[1]=5 → stop.
```

```
# result=[[1,5]]. extend [[6,9]]. → [[1,5],[6,9]] ✓
#
# intervals=[[1,2],[3,5],[6,7],[8,10],[12,16]], new=[4,8]:
# Phase 1: [1,2] ends at 2 < 4 → add. i=1.
# Phase 2: [3,5] start 3<=8→new=[3,8]. [6,7] start 6<=8→new=[3,8]. [8,10] start 8<=8→new=[3,10]. i=4.
# result=[[1,2],[3,10]]. extend [[12,16]]. → [[1,2],[3,10],[12,16]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — three phases together visit each interval once.
# Space O(n) for output.
# The two overlap conditions are the key:
# Non-overlap before: interval[end] < new[start].
# Non-overlap after: interval[start] > new[end] (Phase 3 — just extend).
# Anything in between overlaps and gets merged.
# Given more time I'd ask: what if intervals aren't sorted?
# Sort first (O(n log n)), then apply the same three-phase approach."
```


Arrays & Hashing

#238 Product of Array Except Self

ME
D

Open on LeetCode

Array · Prefix Sum

Lists: Grind 169

Problem

Given an integer array nums, return an array answer such that answer[i] is equal to the product of all the elements of nums except nums[i].

The product of any prefix or suffix of nums is guaranteed to fit in a 32-bit integer.

You must write an algorithm that runs in O(n) time and without using the division operation.

Example 1:

```
Input: nums = [1,2,3,4]
Output: [24,12,8,6]
```

Example 2:

```
Input: nums = [-1,1,0,-3,3]
Output: [0,0,9,0,0]
```

Constraints:

- **2 <= nums.length <= 105**
- **-30 <= nums[i] <= 30**
- **The input is generated such that answer[i] is guaranteed to fit in a 32-bit integer.**

Follow up: Can you solve the problem in O(1) extra space complexity? (The output array does not count as extra space for space complexity analysis.)

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return an array where answer[i] is the product of all elements except nums[i].
# Cannot use division. Must run in O(n). Follow-up: O(1) extra space. Right?
# Guaranteed no overflow in 32-bit — so no overflow concern."
#
# "[1,2,3,4]: answer[0]=2*3*4=24. answer[1]=1*3*4=12. answer[2]=1*2*4=8. answer[3]=1*2*3=6 ✓
# [-1,1,0,-3,3]: zeros make most products 0 — [0,0,9,0,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each index i, I multiply together all elements in the array except nums[i] itself.
# That's a nested loop — O(n) work for each of the n positions.
# Time: O(n²). Space: O(1) extra. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Build prefix products and suffix products, multiply them.
# prefix[i] = product of everything to the LEFT of i.
# suffix[i] = product of everything to the RIGHT of i.
# answer[i] = prefix[i] * suffix[i].
# Time: O(n). Space: O(n) for two arrays.
#
# O(1) space follow-up: use the output array for prefix products,
# then do a single right-to-left pass with a running suffix multiplier."

# PHASE 4 — CLEAN CODE (O(1) extra space version)
        # Left pass: result[i] = product of all elements to the left of i
        # Right pass: multiply in the suffix product on the fly

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3,4]:
# Left pass: result=[1,1,2,6].
# Right pass: i=3: result[3]=6*1=6, suffix=4.
# i=2: result[2]=2*4=8, suffix=12.
# i=1: result[1]=1*12=12, suffix=24.
# i=0: result[0]=1*24=24, suffix=24.
# → [24,12,8,6] ✓
# nums=[0,0]: result=[0,0] ✓ (zeros handled correctly by multiplication)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) extra — output array doesn't count.
# The two-pass trick: left products first, then multiply right products on the fly.
# The suffix variable carries the running right product without an extra array.
# Given more time I'd ask: what if we're allowed division?
# total_product / nums[i] — but fails when nums[i]=0. Handle: count zeros,
# if two or more → all zeros. If one zero → all zeros except position of zero."
```

Arrays & Hashing

#281 Zigzag Iterator

MED

Open on LeetCode

Array · Design · Queue · Iterator

Lists: Denny Zhang

Problem

Given two vectors of integers v1 and v2, implement an iterator to return their elements alternately.

Implement the ZigzagIterator class:

- ZigzagIterator(List<int> v1, List<int> v2) initializes the object with the two vectors v1 and v2.
- boolean hasNext() returns true if the iterator still has elements, and false otherwise.
- int next() returns the current element of the iterator and moves the iterator to the next element.

Example 1:

```
Input: v1 = [1,2], v2 = [3,4,5,6]
Output: [1,3,2,4,5,6]
Explanation: By calling next repeatedly until hasNext returns false, the order of elements returned by next should be:
[1,3,2,4,5,6].
```

Example 2:

```
Input: v1 = [1], v2 = []
Output: [1]
```

Example 3:

```
Input: v1 = [], v2 = [1]
Output: [1]
```

Constraints:

- 0 <= v1.length, v2.length <= 1000
- 1 <= v1.length + v2.length <= 2000
- -231 <= v1[i], v2[i] <= 231 - 1

Follow up: What if you are given k vectors? How well can your code be extended to such cases?

Clarification for the follow-up question:

The "Zigzag" order is not clearly defined and is ambiguous for k > 2 cases. If "Zigzag" does not look right to you, replace "Zigzag" with "Cyclic".

Follow-up Example:

```
Input: v1 = [1,2,3], v2 = [4,5,6,7], v3 = [8,9]
Output: [1,4,8,2,5,9,3,6,7]
```

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Alternate between v1 and v2 element by element. When one is exhausted,
# continue with the remaining elements of the other. Right?
# v1=[1,2], v2=[3,4,5,6] → [1,3,2,4,5,6] ✓
# Follow-up: what if there are k vectors instead of 2?"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# At construction I'll interleave v1 and v2 into a flat list: take one from v1, one from v2,
# repeat, then append remaining elements from whichever list is longer.
# next() and hasNext() just read from that flat list.
# Time: O(n) init. Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Use a deque of (vector, index) pairs.
# next(): popleft, return v[i], re-enqueue (v, i+1) if more elements remain.
# hasNext(): deque is non-empty.
# This generalises trivially to k vectors — just enqueue all of them.
# Time: O(1) per next()/hasNext(). Space: O(k) for the deque."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# v1=[1,2], v2=[3,4,5,6]:
# q=[(v1,0),(v2,0)].
# next(): pop (v1,0)→return 1. re-enqueue (v1,1). q=[(v2,0),(v1,1)].
# next(): pop (v2,0)→return 3. re-enqueue (v2,1). q=[(v1,1),(v2,1)].
# next(): pop (v1,1)→return 2. no re-enqueue (idx+1=2=len). q=[(v2,1)].
# next(): 4, then 5, then 6. → [1,3,2,4,5,6] ✓
# v1=[1], v2=[]: q=[(v1,0)]. next()→1. hasNext()→False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(1) per operation. Space O(k) for the deque.
# Follow-up k vectors: init enqueues all k non-empty vectors. Same code, no changes.
# The deque acts as a round-robin scheduler — each vector gets one turn per cycle.
# Given more time I'd ask: what if vectors have different priorities (some should
# appear more often)? Enqueue them multiple times or use a priority queue."
```


Arrays & Hashing

#307 Range Sum Query - Mutable

MED

Open on LeetCode

Array · Design · Binary Indexed Tree · Segment Tree

Lists: Denny Zhang

Problem

Given an integer array nums, handle multiple queries of the following types:

1. Update the value of an element in nums.
2. Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.

Implement the NumArray class:

- NumArray(int[] nums) Initializes the object with the integer array nums.
- void update(int index, int val) Updates the value of nums[index] to be val.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).

Example 1:

```
Input
["NumArray", "sumRange", "update", "sumRange"]
[[[1, 3, 5]], [0, 2], [1, 2], [0, 2]]
Output
[null, 9, null, 8]

Explanation
NumArray numArray = new NumArray([1, 3, 5]);
```

Constraints:

- 1 <= nums.length <= 3 * 104
- -100 <= nums[i] <= 100
- 0 <= index < nums.length
- -100 <= val <= 100
- 0 <= left <= right < nums.length
- At most 3 * 104 calls will be made to update and sumRange.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Support two operations on an array: update a single element, and query
# the prefix sum from left to right inclusive. Up to 3*10^4 calls. Right?
# The challenge is making both operations fast — not O(n) each."
#
# "NumArray([1,3,5]): sumRange(0,2)=9. update(1,2)→nums=[1,2,5]. sumRange(0,2)=8 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Store the array directly. update() just sets nums[index] = val in O(1).
# sumRange(left, right) sums nums[left..right] with a linear scan in O(n).
# With up to 3*10^4 queries this is O(n * queries) total — too slow for large n.
# Time: update O(1), sumRange O(n). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary Indexed Tree (Fenwick Tree) — each node stores a partial sum covering
# a range determined by its lowest set bit. Both update and query run in O(log n).
# Update: add change to index and all its ancestors (i += i & -i).
# Query prefix sum up to i: add and walk down (i -= i & -i).
# For range [left, right]: query(right) - query(left-1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,3,5]: tree built. _prefix(2)=1+3+5=9. sumRange(0,2)=9 ✓
# update(1,2): delta=2-3=-1. tree adjusted. nums=[1,2,5].
# sumRange(0,2): _prefix(2)=8 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "update: O(log n). sumRange: O(log n). init: O(n log n).
# Space: O(n) for the tree.
# The bit trick i & (-i) isolates the lowest set bit — this determines
# how many elements each node is responsible for. Worth explaining clearly.
# Alternative: segment tree — more flexible (supports range update) but more code.
# Given more time I'd ask: what if we need range updates (add v to nums[l..r])?
# BIT can handle that with a difference array extension — or use a lazy segment tree."
```

Arrays & Hashing

#454 4Sum II

MED

Open on LeetCode

Array · Hash Table

Lists: CodeSignal

Problem

Given four integer arrays nums1, nums2, nums3, and nums4 all of length n, return the number of tuples (i, j, k, l) such that:

- 0 <= i, j, k, l < n
- nums1[i] + nums2[j] + nums3[k] + nums4[l] == 0

Example 1:

```
Input: nums1 = [1,2], nums2 = [-2,-1], nums3 = [-1,2], nums4 = [0,2]
Output: 2
Explanation:
The two tuples are:
1. (0, 0, 0, 1) -> nums1[0] + nums2[0] + nums3[0] + nums4[1] = 1 + (-2) + (-1) + 2 = 0
2. (1, 1, 0, 0) -> nums1[1] + nums2[1] + nums3[0] + nums4[0] = 2 + (-1) + (-1) + 0 = 0
```

Example 2:

```
Input: nums1 = [0], nums2 = [0], nums3 = [0], nums4 = [0]
Output: 1
```

Constraints:

- n == nums1.length
- n == nums2.length
- n == nums3.length
- n == nums4.length
- 1 <= n <= 200
- -228 <= nums1[i], nums2[i], nums3[i], nums4[i] <= 228

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count tuples (i,j,k,l) where nums1[i]+nums2[j]+nums3[k]+nums4[l]==0.
# All four arrays have the same length n. Return the count — not the tuples. Right?
# We're not asked to deduplicate — every index combo counts separately."
#
# "nums1=[1,2],nums2=[-2,-1],nums3=[-1,2],nums4=[0,2]:
# (0,0,0,1): 1+(-2)+(-1)+2=0 ✓. (1,1,0,0): 2+(-1)+(-1)+0=0 ✓. → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Four nested loops — for every combination (i,j,k,l), check if
# nums1[i]+nums2[j]+nums3[k]+nums4[l] == 0 and count the matches.
# Time: O(n⁴). For n=200 that's 1.6 billion iterations — way too slow.
# Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Meet in the middle: split into two pairs.
# Build a Counter of all sums a+b for a in nums1, b in nums2 — O(n²) sums.
# Then for each c+d from nums3×nums4, look up -(c+d) in the counter — O(n²) lookups.
# Total: O(n²) time and O(n²) space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums1=[1,2], nums2=[-2,-1]:
# pair_sums={1+(-2)=-1:1, 1+(-1)=0:1, 2+(-2)=0:1→0:2, 2+(-1)=1:1}
# = {-1:1, 0:2, 1:1}
# nums3=[-1,2], nums4=[0,2]:
# (c=-1,d=0): -(-1+0)=1. pair_sums[1]=1. count=1.
# (c=-1,d=2): -(-1+2)=-1. pair_sums[-1]=1. count=2.
# (c=2,d=0): -(2+0)=-2. pair_sums[-2]=0. count=2.
# (c=2,d=2): -(2+2)=-4. pair_sums[-4]=0. count=2. → 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(n²).
# Meet-in-the-middle is the key: instead of 4 nested loops, do 2+2.
# The Counter default of 0 for missing keys means misses contribute 0 — clean.
# Given more time I'd ask: what if the arrays are sorted and we want unique quadruplets?
# That's 4Sum (#18) — different problem requiring deduplication with two pointers."
```


Arrays & Hashing

#525 Contiguous Array

MED

Open on LeetCode

Array · Hash Table · Prefix Sum

Lists: Grind 169

Problem

Given a binary array nums, return the maximum length of a contiguous subarray with an equal number of 0 and 1.

Example 1:

```
Input: nums = [0,1]
Output: 2
Explanation: [0, 1] is the longest contiguous subarray with an equal number of 0 and 1.
```

Example 2:

```
Input: nums = [0,1,0]
Output: 2
Explanation: [0, 1] (or [1, 0]) is a longest contiguous subarray with equal number of 0 and 1.
```

Example 3:

```
Input: nums = [0,1,1,1,1,1,0,0,0]
Output: 6
Explanation: [1,1,1,0,0,0] is the longest contiguous subarray with equal number of 0 and 1.
```

Constraints:

- 1 <= nums.length <= 105
- nums[i] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Binary array. Find the longest contiguous subarray with equal numbers of 0s and 1s.
# Return its length — not the subarray itself. Right?"
#
# "[0,1]: equal 0s and 1s → length 2 ✓
# [0,1,0]: longest is [0,1] or [1,0] → length 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), count the zeros and ones in nums[i..j].
# If they're equal, it's a valid subarray — track the maximum length.
# Time: O(n²) — we can precompute prefix counts to make each check O(1).
# Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Treat 0 as -1 and 1 as +1. Keep a running balance (prefix sum).
# Equal 0s and 1s means balance hasn't changed — we want the longest subarray
# where balance[j] == balance[i] for some i < j.
# Store the FIRST index each balance value was seen. When we see the same balance again,
# the subarray [first_seen+1 .. current] has equal 0s and 1s.
# Seed the map with {0: -1} to handle subarrays starting from index 0.
#
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[0,1,0]:
# i=0,num=0: balance=-1. not seen → first_seen={0:-1,-1:0}.
# i=1,num=1: balance=0. seen at -1 → longest=max(0,1-(-1))=2.
# i=2,num=0: balance=-1. seen at 0 → longest=max(2,2-0)=2.
# return 2 ✓
# nums=[0,1,1,1,1,1,0,0,0]:
# Track balance... longest=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for the hash map.
# The 0→-1 substitution is the core insight — equal 0s and 1s becomes
# 'prefix sum unchanged', which becomes 'same balance seen before'.
# The {0:-1} seed is critical — without it, subarrays starting at index 0 are missed.
# Given more time I'd ask: what if it's not binary — find the longest subarray with
# equal counts of two specific values? Same trick — map target→+1, other→-1."
```

Arrays & Hashing

#560 Subarray Sum Equals K

ME D

Open on LeetCode

Array · Hash Table · Prefix Sum

Lists: Grind 169

Problem

Given an array of integers nums and an integer k, return the total number of subarrays whose sum equals to k.
A subarray is a contiguous non-empty sequence of elements within an array.

Example 1:

```
Input: nums = [1,1,1], k = 2
Output: 2
```

Example 2:

```
Input: nums = [1,2,3], k = 3
Output: 2
```

Constraints:

- 1 <= nums.length <= 2 * 104
- -1000 <= nums[i] <= 1000
- -107 <= k <= 107

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays (contiguous, non-empty) whose elements sum to k.
# Can contain negatives — so sliding window won't work. Return the count. Right?"
#
# "[1,1,1], k=2: [1,1] at (0,1) and [1,1] at (1,2) → 2 ✓
# [1,2,3], k=3: [3] at index 2 and [1,2] → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), compute the sum of nums[i..j] and check if it equals k.
# I can maintain a running sum to avoid recomputing from scratch each time, giving O(n²).
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Prefix sums: sum of nums[i..j] = prefix[j+1] - prefix[i].
# We want prefix[j+1] - prefix[i] == k → prefix[i] == prefix[j+1] - k.
# Walk forward tracking the running sum. For each position, count how many
# earlier prefix sums equal (current_sum - k). Use a Counter seeded with {0:1}
# to handle subarrays starting at index 0.
#
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1], k=2:
# num=1: sum=1. want=1-2=-1. prefix_count[-1]=0. prefix_count={0:1,1:1}.
# num=1: sum=2. want=2-2=0. prefix_count[0]=1 → count=1. prefix_count={0:1,1:1,2:1}.
# num=1: sum=3. want=3-2=1. prefix_count[1]=1 → count=2. → 2 ✓
# nums=[1,2,3], k=3:
# sum=1,want=-2→0. sum=3,want=0→count=1. sum=6,want=3→prefix_count[3]=1→count=2. → 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n).
# This is structurally identical to #525 Contiguous Array — both use
# prefix sum + hash map, both seed with {0:1} (or {0:-1}).
# Worth naming the connection explicitly: 525 finds longest, 560 counts all.
# Sliding window doesn't work here because negatives mean the sum can decrease
# even as we expand right — sliding window requires monotone sums.
# Given more time I'd ask: what if we want subarrays with sum divisible by k?
# Use prefix_sum % k — same pattern, but map remainders."
```


Arrays & Hashing

#1010 Pairs of Songs With Total Durations Divisible by 60

ME D

Open on LeetCode

Array · Hash Table · Counting

Lists: CodeSignal

Problem

You are given a list of songs where the ith song has a duration of time[i] seconds.
Return the number of pairs of songs for which their total duration in seconds is divisible by 60. Formally, we want the number of indices i, j such that i < j with (time[i] + time[j]) % 60 == 0.

Example 1:

```
Input: time = [30,20,150,100,40]
Output: 3
Explanation: Three pairs have a total duration divisible by 60:
(time[0] = 30, time[2] = 150): total duration 180
(time[1] = 20, time[3] = 100): total duration 120
(time[1] = 20, time[4] = 40): total duration 60
```

Example 2:

```
Input: time = [60,60,60]
Output: 3
Explanation: All three pairs have a total duration of 120, which is divisible by 60.
```

Constraints:

- 1 <= time.length <= 6 * 104
- 1 <= time[i] <= 500

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) where i<j and (time[i]+time[j]) % 60 == 0.
# Songs have durations in seconds. Return the count. Right?
# Edge cases: duration exactly 60 (60%60=0), duration exactly 30 pairs with another 30."
#
# "[30,20,150,100,40]: (30,150)=180, (20,100)=120, (20,40)=60 → 3 ✓
# [60,60,60]: each pair sums to 120, divisible by 60. C(3,2)=3 → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, check if (time[i] + time[j]) % 60 == 0.
# Count all such pairs. For n=6*10^4 that's 3.6 billion comparisons — too slow.
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Reduce each duration to t % 60. We want pairs where t1 + t2 ≡ 0 (mod 60).
# For each t, its complement is (60 - t) % 60. The %60 handles t=0 (complement=0, not 60).
# Walk forward: count += how many times we've already seen the complement.
# Then add t to the seen counter.
# This is Two Sum with modular arithmetic — O(n) time, O(60)=O(1) space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# time=[30,20,150,100,40]:
# t=30: rem=30, comp=30. seen[30]=0 → count=0. seen={30:1}.
# t=20: rem=20, comp=40. seen[40]=0 → count=0. seen={30:1,20:1}.
# t=150: rem=30, comp=30. seen[30]=1 → count=1. seen={30:2,20:1}.
# t=100: rem=40, comp=20. seen[20]=1 → count=2. seen={30:2,20:1,40:1}.
# t=40: rem=40, comp=20. seen[20]=1 → count=3. → 3 ✓
# time=[60,60,60]: each rem=0, comp=(60-0)%60=0.
# t=60: seen[0]=0 → 0. seen={0:1}.
# t=60: seen[0]=1 → count=1. seen={0:2}.
# t=60: seen[0]=2 → count=3. → 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) — the Counter holds at most 60 keys.
# The (60-remainder)%60 handles the remainder=0 case: complement is 0, not 60.
# This is Two Sum modulo 60 — recognising the Two Sum pattern in disguise
# is the key interview insight.
# Given more time I'd ask: what if divisor were k instead of 60?
# Same algorithm — the Counter holds at most k entries, still O(1) space for fixed k."
```

Arrays & Hashing

⋆ #1272 Remove Interval ⋆

MED

Open on LeetCode

Array

Lists: ⋆ Premium | Premium 98

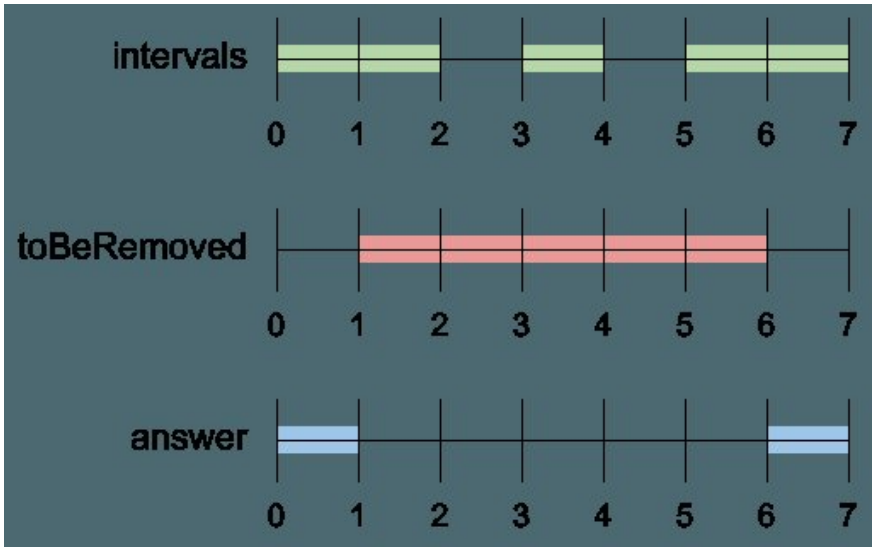
Problem

A set of real numbers can be represented as the union of several disjoint intervals, where each interval is in the form [a, b). A real number x is in the set if one of its intervals [a, b) contains x (i.e. a <= x < b).

You are given a sorted list of disjoint intervals intervals representing a set of real numbers as described above, where intervals[i] = [ai, bi] represents the interval [ai, bi). You are also given another interval toBeRemoved.

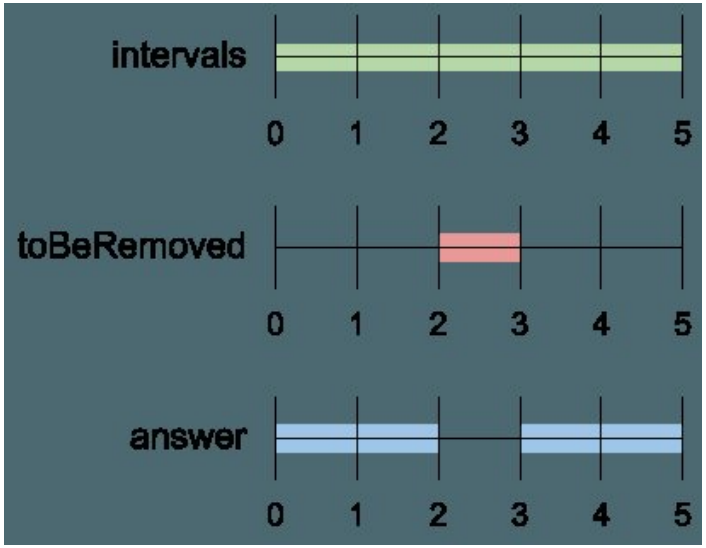
Return the set of real numbers with the interval toBeRemoved removed from intervals. In other words, return the set of real numbers such that every x in the set is in intervals but not in toBeRemoved. Your answer should be a sorted list of disjoint intervals as described above.

Example 1:




```
Input: intervals = [[0,2],[3,4],[5,7]], toBeRemoved = [1,6]
Output: [[0,1],[6,7]]
```

Example 2:




```
Input: intervals = [[0,5]], toBeRemoved = [2,3]
Output: [[0,2],[3,5]]
```

Example 3:

```
Input: intervals = [[-5,-4],[-3,-2],[1,2],[3,5],[8,9]], toBeRemoved = [-1,4]
Output: [[-5,-4],[-3,-2],[4,5],[8,9]]
```

Constraints:

- 1 <= intervals.length <= 104
- -109 <= ai < bi <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Intervals use half-open notation [a, b). We remove everything in toBeRemoved=[a,b)
# from the existing set and return what remains. Right?
# Intervals are pre-sorted and non-overlapping — no need to re-sort output."
#
# "[[0,2],[3,4],[5,7]], remove=[1,6]:
# [0,2] partially overlaps → keep [0,1). [3,4] fully inside → gone. [5,7] partially → keep [6,7). → [[0,1],[6,7]] ✓
# [[0,5]], remove=[2,3]: split into [0,2) and [3,5). → [[0,2],[3,5]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Walk each interval and handle three cases: if it doesn't touch toBeRemoved at all, keep it whole.
# If it partially overlaps on the left, keep the left portion. If it partially overlaps on the right,
# keep the right portion. If it's fully covered, drop it entirely.
# Time: O(n). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Already O(n) — just apply the three cases cleanly in one pass.
# No overlap if: interval ends before remove starts (end <= a) OR starts after remove ends (start >= b).
# Otherwise it overlaps. Trim left portion if start < a → keep [start, a).
# Trim right portion if end > b → keep [b, end)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[0,2],[3,4],[5,7]], remove=[1,6]:
# [0,2]: overlap. start(0)<a(1)→[0,1]. end(2)>b(6)? No. result=[[0,1]].
# [3,4]: overlap. start(3)<1? No. end(4)>6? No. result=[[0,1]].
# [5,7]: overlap. start(5)<1? No. end(7)>6 → [6,7]. result=[[0,1],[6,7]] ✓
# [[0,5]], remove=[2,3]:
# overlap. [0,2] and [3,5]. → [[0,2],[3,5]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for output.
# The two trim conditions are independent — both can fire on the same interval
# (when toBeRemoved is strictly inside the interval), producing two output intervals.
# Given more time I'd ask: what if toBeRemoved can be a list of intervals to remove?
# Process them in sorted order, sweeping through once — still O(n + m) time."
```

Arrays & Hashing

★ #1429 First Unique Number ★

MED

Open on LeetCode

Array · Hash Table · Design · Queue · Data Stream

Lists: ★ Premium | Premium 98

Problem

You have a queue of integers, you need to retrieve the first unique integer in the queue.

Implement the FirstUnique class:

- FirstUnique(int[] nums) Initializes the object with the numbers in the queue.
- int showFirstUnique() returns the value of the first unique integer of the queue, and returns -1 if there is no such integer.
- void add(int value) insert value to the queue.

Example 1:

```
Input:
["FirstUnique","showFirstUnique","add","showFirstUnique","add","showFirstUnique","add","showFirstUnique"]
[[[2,3,5]],[],[5],[],[2],[],[3],[]]
Output:
[null,2,null,2,null,3,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([2,3,5]);
firstUnique.showFirstUnique(); // return 2
```

Example 2:

```
Input:
["FirstUnique","showFirstUnique","add","add","add","add","add","showFirstUnique"]
[[[7,7,7,7,7,7]],[],[7],[3],[3],[7],[17],[]]
Output:
[null,-1,null,null,null,null,null,17]
Explanation:
FirstUnique firstUnique = new FirstUnique([7,7,7,7,7,7]);
firstUnique.showFirstUnique(); // return -1
```

Example 3:

```
Input:
["FirstUnique","showFirstUnique","add","showFirstUnique"]
[[[809]],[],[809],[]]
Output:
[null,809,null,-1]
Explanation:
FirstUnique firstUnique = new FirstUnique([809]);
firstUnique.showFirstUnique(); // return 809
```

Constraints:

- 1 <= nums.length <= 10^5
- 1 <= nums[i] <= 10^8
- 1 <= value <= 10^8
- At most 50000 calls will be made to showFirstUnique and add.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Maintain a queue. showFirstUnique() returns the first element that appears
# exactly once. add() appends to the queue. Return -1 if no unique exists. Right?
# Order matters — we want the first unique by insertion order."
#
# "FirstUnique([2,3,5]): first unique = 2.
# add(5) → 5 now has count 2, still first unique = 2.
# add(2) → 2 duplicated, first unique = 3.
# add(3) → 3 duplicated, no unique → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# I'll store all added elements in a list. showFirstUnique() scans the list in insertion order,
# counting occurrences of each element, and returns the first one that appears exactly once.
# add() is O(1). showFirstUnique() is O(n) per call — with many calls this gets slow.
# Time: add O(1), showFirstUnique O(n). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two data structures:
# A 'seen' set — tracks all values ever added (including duplicates).
# An ordered dict (or insertion-order dict) — maps value → True, only for unique values.
# On add: if not in seen → it's new, add to both seen and dict.
# if in seen and in dict → it became a duplicate, remove from dict.
# if in seen and not in dict → already duplicate, ignore.
# showFirstUnique(): peek at first key in dict — O(1) in Python 3.7+ (insertion-ordered).
#
# Time: O(1) per add, O(1) per showFirstUnique. Space: O(n)."
```



```
# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# init([2,3,5]): seen={2,3,5}, uniques={2,3,5}.
# showFirstUnique() → 2 ✓
# add(5): 5 in seen and in uniques → del 5. uniques={2,3}.
# showFirstUnique() → 2 ✓
# add(2): 2 in seen and in uniques → del 2. uniques={3}.
# showFirstUnique() → 3 ✓
# add(3): del 3. uniques={}.
# showFirstUnique() → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) per operation. Space O(n).
# Python dicts maintain insertion order since 3.7, so next(iter(dict)) returns
# the oldest key still present — exactly what we need.
# If order weren't guaranteed, we'd use collections.OrderedDict explicitly.
# Given more time I'd ask: what if we need to support remove() as well?
# That's a linked hash map (doubly linked list + hash map) — O(1) all operations."
```

Arrays & Hashing

#3159 Find Occurrences of an Element in an Array

MED

Open on LeetCode

Array · Hash Table

Lists: CodeSignal

Problem

You are given an integer array nums, an integer array queries, and an integer x.
For each queries[i], you need to find the index of the queries[i]th occurrence of x in the nums array. If there are fewer than queries[i] occurrences of x, the answer should be -1 for that query.
Return an integer array answer containing the answers to all queries.
Example 1:
Input: nums = [1,3,1,7], queries = [1,3,2,4], x = 1
Output: [0,-1,2,-1]
Explanation:
- **For the 1st query, the first occurrence of 1 is at index 0.**
- **For the 2nd query, there are only two occurrences of 1 in nums, so the answer is -1.**
- **For the 3rd query, the second occurrence of 1 is at index 2.**
- **For the 4th query, there are only two occurrences of 1 in nums, so the answer is -1.**

Example 2:
Input: nums = [1,2,3], queries = [10], x = 5
Output: [-1]
Explanation:
- **For the 1st query, 5 doesn't exist in nums, so the answer is -1.**

Constraints:
- **1 <= nums.length, queries.length <= 105**
- **1 <= queries[i] <= 105**
- **1 <= nums[i], x <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Preprocess nums to find all indices where x appears.
# For each query q, return the index of the q-th occurrence of x (1-indexed).
# If fewer than q occurrences exist, return -1. Right?"
#
# "nums=[1,3,1,7], queries=[1,3,2,4], x=1:
# x appears at indices [0,2]. q=1→0, q=3→-1, q=2→2, q=4→-1. → [0,-1,2,-1] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each query q, scan nums from the beginning, count occurrences of x as we go,
# and return the index where we hit the q-th occurrence — or -1 if we run out.
# Time: O(n * |queries|). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Precompute a list of all indices where x appears — O(n).
# Each query is then an O(1) index lookup. Total: O(n + |queries|)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,3,1,7], x=1: positions=[0,2].
# q=1: positions[0]=0 ✓. q=3: 3-1=2 >= len(positions)=2 → -1 ✓.
# q=2: positions[1]=2 ✓. q=4: -1 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n + q). Space O(k) where k = occurrences of x.
# If queries were sorted, binary search on positions would give O((n+q) log k)
# with minimal extra space — but for unsorted queries, the precompute list is cleaner.
# Given more time I'd ask: what if x could change between queries?
# Build a map from value → sorted list of indices, then binary search per query."
```


String

Round 2 · High · Medium · 5 questions

String

#8 String to Integer (atoi)

MED

Open on LeetCode

String
Lists: Grind 169

Problem

Implement the myAtoi(string s) function, which converts a string to a 32-bit signed integer.
The algorithm for myAtoi(string s) is as follows:

1. Whitespace: Ignore any leading whitespace (" ").
2. Signedness: Determine the sign by checking if the next character is '-' or '+', assuming positivity if neither present.
3. Conversion: Read the integer by skipping leading zeros until a non-digit character is encountered or the end of the string is reached. If no digits were read, then the result is 0.
4. Rounding: If the integer is out of the 32-bit signed integer range [-231, 231 - 1], then round the integer to remain in the range. Specifically, integers less than -231 should be rounded to -231, and integers greater than 231 - 1 should be rounded to 231 - 1.

Return the integer as the final result.
Example 1:
Input: s = "42"
Output: 42
Explanation:

```
The underlined characters are what is read in and the caret is the current reader position.
Step 1: "42" (no characters read because there is no leading whitespace)
^
Step 2: "42" (no characters read because there is neither a '-' nor '+')
^
Step 3: "42" ("42" is read in)
^
```

Example 2:
Input: s = " -042"
Output: -42
Explanation:

```
Step 1: " -042" (leading whitespace is read and ignored)
^
Step 2: " -042" ('-' is read, so the result should be negative)
^
Step 3: " -042" ("042" is read in, leading zeros ignored in the result)
^
```

Example 3:
Input: s = "1337c0d3"
Output: 1337
Explanation:

```
Step 1: "1337c0d3" (no characters read because there is no leading whitespace)
^
Step 2: "1337c0d3" (no characters read because there is neither a '-' nor '+')
^
Step 3: "1337c0d3" ("1337" is read in; reading stops because the next character is a non-digit)
^
```

Example 4:
Input: s = "0-1"
Output: 0
Explanation:

```
Step 1: "0-1" (no characters read because there is no leading whitespace)
^
Step 2: "0-1" (no characters read because there is neither a '-' nor '+')
^
Step 3: "0-1" ("0" is read in; reading stops because the next character is a non-digit)
^
```

Example 5:
Input: s = "words and 987"
Output: 0
Explanation:
Reading stops at the first non-digit character 'w'.
Constraints:

- 0 <= s.length <= 200
- s consists of English letters (lower-case and upper-case), digits (0-9), ' ', '+', '-', and '.'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Four steps in order: strip leading whitespace, read optional sign,
# read digits until non-digit or end, clamp to 32-bit signed range [-2^31, 2^31-1]. Right?
# Stop at the FIRST non-digit after the optional sign — don't skip over non-digits."
#
# "'42'→42. ' -042'→-42 (strip, sign, parse 042). '1337c0d3'→1337 (stop at 'c').
# '0-1'→0 (first digit parsed, stop at '-'). 'words'→0 (first char non-digit, no digits read)."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# The naive approach is to try Python's built-in int() after stripping whitespace —
# but that fails on partial strings like '42abc' and doesn't clamp to 32-bit range.
# So the 'brute force' here is really just the manual state machine, which I'll implement.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Manual state machine with a pointer:
# 1. Skip whitespace.
# 2. Read sign.
# 3. Read digits, building number, check overflow before each digit.
# Overflow check: ans > MAX//10 OR (ans == MAX//10 AND next digit > 7).
# Return sign * ans (clamped)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# '42': no space, no sign, digits→42. return 42 ✓
# ' -042': skip space, sign=-1, digits 0,4,2→42. return -42 ✓
# '1337c0d3': digits 1,3,3,7→1337, stop at 'c'. return 1337 ✓
# '99999999999': overflow triggers → return INT_MAX ✓
# '': i=0, no sign, no digits → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The overflow check BEFORE adding each digit is critical — without it we'd overflow Python
# integers silently (Python handles big ints) but the logic breaks for fixed-width languages.
# The magic '7' in d > 7 comes from 2^31-1 = 2147483647 — last digit is 7.
# Given more time I'd ask: handle locale-specific decimal separators (commas)?
# Add a pre-processing step to strip/replace them before parsing."
```

String

★ #249 Group Shifted Strings ★

MED

Open on LeetCode

Array · Hash Table · String

Lists: ★ Premium | Premium 98

Problem

Perform the following shift operations on a string:

- Right shift: Replace every letter with the successive letter of the English alphabet, where 'z' is replaced by 'a'. For example, "abc" can be right-shifted to "bcd" or "xyz" can be right-shifted to "yza".
- Left shift: Replace every letter with the preceding letter of the English alphabet, where 'a' is replaced by 'z'. For example, "bcd" can be left-shifted to "abc" or "yza" can be left-shifted to "xyz".

We can keep shifting the string in both directions to form an endless shifting sequence.

- For example, shift "abc" to form the sequence: ... <-> "abc" <-> "bcd" <-> ... <-> "xyz" <-> "yza" <-> .... <-> "zab" <-> "abc" <-> ...

You are given an array of strings strings, group together all strings[i] that belong to the same shifting sequence. You may return the answer in any order.

Example 1:

Input: strings = ["abc","bcd","acef","xyz","az","ba","a","z"]

Output: [["acef"],["a","z"],["abc","bcd","xyz"],["az","ba"]]

Example 2:

Input: strings = ["a"]

Output: [["a"]]

Constraints:

- 1 <= strings.length <= 200
- 1 <= strings[i].length <= 50
- strings[i] consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Two strings are in the same shifting sequence if one can become the other
# by consistently shifting every character by the same amount (cyclically, a→z wraps).
# Group them. Return order doesn't matter. Right?
# 'abc' and 'xyz' are in the same group — same relative gaps between characters."
#
# "['abc','bcd','xyz','az','ba','a','z']:
# 'abc','bcd','xyz' → all have gaps [1,1] → same group.
# 'az','ba' → gaps [25] and [25] (b-a=1… wait: 'az': z-a=25. 'ba': a-b=-1+26=25) → same group.
# 'a','z' → single char, empty gap string → same group."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each string, generate all 26 rotations by shifting every character by 0..25.
# If any rotation of string A equals string B, they belong in the same group.
# This is O(n² * 26 * L) — quadratic in the number of strings, far too slow.
# Time: O(n²*26*L). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Compute a canonical key for each string: the tuple of consecutive char differences mod 26.
# 'abc' → (1,1). 'xyz' → (1,1). 'az' → (25,). 'ba' → (25,).
# All strings with the same key are in the same shifting group.
# Group by key using a defaultdict. Time: O(n*L). Space: O(n*L)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# 'abc': key=(1,1). 'bcd': (1,1). 'xyz': (1,1) → same group ✓
# 'az': key=((ord('z')-ord('a'))%26,)=(25,). 'ba': ((ord('a')-ord('b'))%26,)=(25,) → same ✓
# 'a','z': key=() (empty tuple) → same group ✓
# 'acef': key=(2,2,1) → its own group ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*L) where L = max string length. Space O(n*L).
# The %26 mod handles the wraparound: 'z'→'a' gives (ord('a')-ord('z'))%26 = (-25)%26 = 1.
# Without the mod, 'az' and 'ba' wouldn't match — the mod is the key insight.
# Given more time I'd ask: how do you efficiently check if two strings are in the same group
# at query time? Build a map string→canonical_key at init, then O(L) per query."
```


String

★ #271 Encode and Decode Strings ★

MED

Open on LeetCode

Array · String · Design

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design an algorithm to encode a list of strings to a string. The encoded string is then sent over the network and is decoded back to the original list of strings.

Machine 1 (sender) has the function:

```
string encode(vector<string> strs) {
// ... your code
return encoded_string;
}
```

Machine 2 (receiver) has the function:

```
vector<string> decode(string s) {
//... your code
return strs;
}
```

So Machine 1 does:

```
string encoded_string = encode(strs);
```

and Machine 2 does:

```
vector<string> strs2 = decode(encoded_string);
```

strs2 in Machine 2 should be the same as strs in Machine 1.

Implement the encode and decode methods.

You are not allowed to solve the problem using any serialize methods (such as eval).

Example 1:

```
Input: dummy_input = ["Hello","World"]
Output: ["Hello","World"]
Explanation:
Machine 1:
Codec encoder = new Codec();
String msg = encoder.encode(strs);
Machine 1 ---msg---> Machine 2
```

Example 2:

```
Input: dummy_input = [""]
Output: [""]
```

Constraints:

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] contains any possible characters out of 256 valid ASCII characters.

Follow up: Could you write a generalized algorithm to work on any possible set of characters?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design encode(strs) and decode(s) that roundtrip correctly for ANY list of strings —
# including strings with special characters, empty strings, or strings containing
# whatever delimiter we might choose. Right?
# We can't use eval or pickle."
#
# "['Hello','World'] → encode → some string → decode → ['Hello','World'] ✓
# [''] → [''] ✓ (empty string must survive the roundtrip)"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# A naive approach is to join strings with a rare delimiter like '|' — but it breaks if
# strings themselves contain '|'. We'd need escaping: replace '|' with '\|' and '\' with '\\'.
# This works but decoding is error-prone and complex to get right.
# Time: O(total chars). Space: O(total chars). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (length-prefix encoding)
# "Prefix each string with its length and a fixed separator (e.g. '#').
# encode: '5#Hello4#World'. decode: read number up to '#', slice that many chars, repeat.
# No ambiguity — length tells us exactly where each string ends.
# Works for any content including '#' and empty strings."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# encode(['Hello','World']): '5#Hello5#World'.
# decode('5#Hello5#World'):
# i=0: j=1 (index of '#'). length=5. result=['Hello']. i=7.
```

```
# i=7: j=8. length=5. result=['Hello','World']. i=14. done ✓
# encode([''']): '0#'. decode('0#'): j=1, length=0, append ''. → [''] ✓
# encode(['a#b','c']): '3#a#b1#c'. decode: length=3→'a#b', length=1→'c' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "encode O(total chars). decode O(total chars). Space O(n).
# Length-prefix is the standard solution — no special characters, no escaping needed.
# The '#' separator is arbitrary — any fixed separator works since we always read
# the length first and know exactly how many chars to consume.
# Given more time I'd ask: what if strings can contain binary data (null bytes)?
# Use a 4-byte big-endian integer prefix instead of a text number — fully binary-safe."
```


String

#635 Design Log Storage System

ME D

Open on LeetCode

Hash Table · String · Design · Ordered Set

Lists: Denny Zhang

Problem

You are given several logs, where each log contains a unique ID and timestamp. Timestamp is a string that has the following format: Year:Month:Day:Hour:Minute:Second, for example, 2017:01:01:23:59:59. All domains are zero-padded decimal numbers.

Implement the LogSystem class:

- LogSystem() Initializes the LogSystem object.
- void put(int id, string timestamp) Stores the given log (id, timestamp) in your storage system.
- int[] retrieve(string start, string end, string granularity) Returns the IDs of the logs whose timestamps are within the range from start to end inclusive. start and end all have the same format as timestamp, and granularity means how precise the range should be (i.e. to the exact Day, Minute, etc.). For example, start = "2017:01:01:23:59:59", end = "2017:01:02:23:59:59", and granularity = "Day" means that we need to find the logs within the inclusive range from Jan. 1st 2017 to Jan. 2nd 2017, and the Hour, Minute, and Second for each log entry can be ignored.

Example 1:

```
Input
["LogSystem", "put", "put", "put", "retrieve", "retrieve"]
[[], [1, "2017:01:01:23:59:59"], [2, "2017:01:01:22:59:59"], [3, "2016:01:01:00:00:00"], ["2016:01:01:01:01:01",
"2017:01:01:23:00:00", "Year"], ["2016:01:01:01:01:01", "2017:01:01:23:00:00", "Hour"]]
Output
[null, null, null, null, [3, 2, 1], [2, 1]]

Explanation
LogSystem logSystem = new LogSystem();
```

Constraints:

- 1 <= id <= 500
- 2000 <= Year <= 2017
- 1 <= Month <= 12
- 1 <= Day <= 31
- 0 <= Hour <= 23
- 0 <= Minute, Second <= 59
- granularity is one of the values ["Year", "Month", "Day", "Hour", "Minute", "Second"].
- At most 500 calls will be made to put and retrieve.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Store logs with (id, timestamp). retrieve(start, end, granularity) returns IDs
# of logs within the range, where comparison is only up to the granularity level.
# Timestamps format: 'YYYY:MM:DD:HH:mm:SS'. Truncate at the granularity character position. Right?"
#
# "retrieve with 'Year': compare only the first 4 chars. '2017:...' vs '2016:...' etc."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Store all logs in a flat list. On retrieve, iterate over every log, truncate its timestamp
# to the granularity prefix length, and check if it falls within the truncated start/end range.
# With at most 500 calls and a fixed 19-char timestamp format, O(n) per retrieve is acceptable.
# Time: put O(1), retrieve O(n). Space: O(n). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Map granularity name to the string prefix length that captures it.
# 'Year'→4 chars, 'Month'→7, 'Day'→10, 'Hour'→13, 'Minute'→16, 'Second'→19.
# Truncate all three timestamps to that length and do a simple string comparison.
# Lexicographic order of the timestamp format is correct for chronological order."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# put(1,'2017:01:01:23:59:59'), put(2,'2017:01:01:22:59:59'), put(3,'2016:01:01:00:00:00').
# retrieve(..., 'Year'): cut=4. compare '2016'<='year'<='2017'. All 3 match → [1,2,3] ✓
# retrieve(..., 'Hour'): cut=13. '2016:01:01:01'<='2016:01:01:00'? No→3 excluded.
# '2016:01:01:01'<='2017:01:01:22'<='2017:01:01:23'? Yes→2. '2017:01:01:23'? Yes→1.
# → [2,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "put O(1). retrieve O(n) — scans all logs.
# The key insight: timestamps in 'YYYY:MM:DD:HH:mm:SS' format sort lexicographically
# correctly because they're zero-padded and delimiters are identical.
# For larger datasets, a sorted list + binary search would bring retrieve to O(log n + k).
# Given more time I'd ask: what if granularity could be arbitrary (e.g. 'Week')?
```

```
# We'd need a custom truncation function rather than fixed prefix lengths."
```


String

#1138 Alphabet Board Path

MED

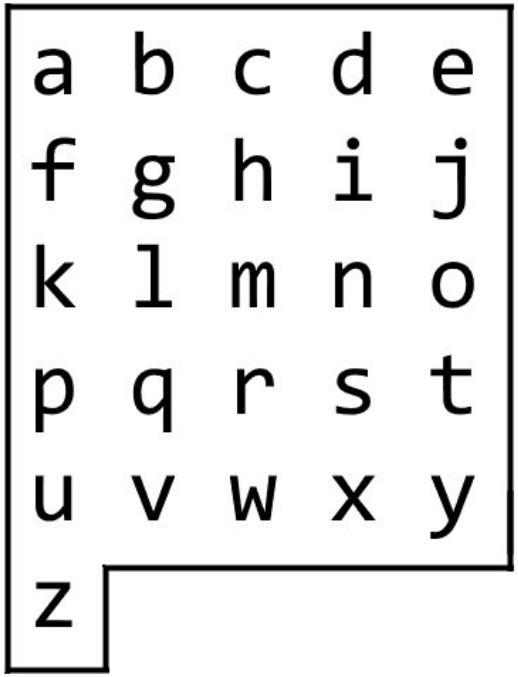
Open on LeetCode

Hash Table · String

Lists: Denny Zhang

Problem

On an alphabet board, we start at position (0, 0), corresponding to character board[0][0].
Here, board = ["abcde", "fghij", "klmno", "pqrst", "uvwxy", "z"], as shown in the diagram below.



We may make the following moves:

- 'U' moves our position up one row, if the position exists on the board;
- 'D' moves our position down one row, if the position exists on the board;
- 'L' moves our position left one column, if the position exists on the board;
- 'R' moves our position right one column, if the position exists on the board;
- '!' adds the character board[r][c] at our current position (r, c) to the answer.

(Here, the only positions that exist on the board are positions with letters on them.)
Return a sequence of moves that makes our answer equal to target in the minimum number of moves. You may return any path that does so.

Example 1:

```
Input: target = "leet"

Output: "DDR!UURRR!!DDD!"
```

Example 2:

```
Input: target = "code"

Output: "RR!DDRR!UUL!R!"
```

Constraints:

- **1 <= target.length <= 100**
- **target consists only of English lowercase letters.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Board is ['abcde','fghij','klmno','pqrst','uvwxy','z'] — 5 cols except row 5 which has only 'z'.
# Start at (0,0). For each char in target, output U/D/L/R moves then '!'. Right?
# The critical constraint: 'z' is at (5,0) — the only cell in its row.
# Moving to 'z' must clear the column before going down. Moving from 'z' must go up before going right."
#
# "target='leet': l is at (2,1). Move D,D,R,!. e=(0,4) — go U,U,R,R,R,!. etc."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For each character, compute its board position as row = (ord(ch)-ord('a'))//5,
# col = (ord(ch)-ord('a'))%5. Then generate moves: go vertical first, then horizontal.
# The naive approach fails for 'z' — moving down before clearing column 0 steps off the board.
# Time: O(|target| * max_moves). Space: O(|target|). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Formula: row = (ord(ch)-ord('a'))//5, col = (ord(ch)-ord('a'))%5.
# For 'z' specifically: it's at (5,0), col=0. When moving TO 'z', move left first
# (to reach col 0) then move down (to reach row 5) — otherwise we'd step off the board.
# When moving FROM 'z', move up first then move right — same reason.
# General case: move vertically then horizontally (or horizontally then vertically).
# 'z' is the only exception — must move left before down."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# target='leet': l=(2,1). from (0,0): D,D,R,! → "DDR!". row=2,col=1.
# e=(0,4): U,U,R,R,R,! → "UURRR!". row=0,col=4.
# e=(0,4): no move,! → "!". row=0,col=4.
# t=(3,4): D,D,D,! → "DDD!". → "DDR!UURRR!!DDD!" ✓
# target='z': from (0,0): ch='z'. L*(0)=none. D*5. → "DDDDD!" ✓
# If coming from (5,4) to 'z' (5,0): L,L,L,L,! — no D needed ✓.
# If going from 'z' to 'a': ch='a'=(0,0). Not 'z' → U*5,L*0,R*0 → "UUUUU!" ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(|target| * max_moves) = O(|target| * 10) = O(|target|). Space O(|target|).
# The 'z' case is the entire problem — everything else is standard grid navigation.
# Phrase it clearly: 'z' is at the leftmost position of its row, so we must reach col 0
# before stepping down into row 5, otherwise intermediate positions don't exist on the board.
# Given more time I'd ask: what if the board were different (e.g. a phone numpad)?
# Same approach — just recompute (row, col) from the new layout."
```


Two Pointers

Round 2 · High · Medium · 14 questions

Two Pointers

#11 Container With Most Water

MED

Open on LeetCode

Array · Two Pointers · Greedy

Lists: Grind 169

Problem

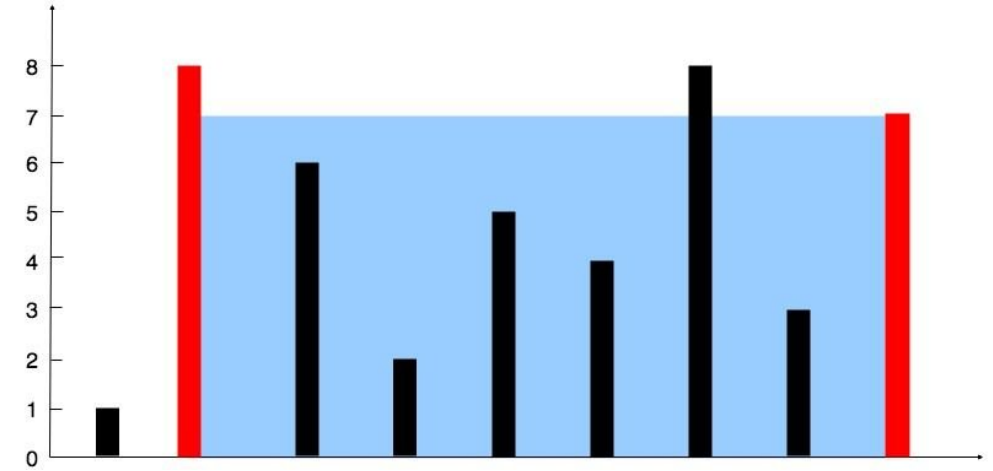
You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).

Find two lines that together with the x-axis form a container, such that the container contains the most water.

Return the maximum amount of water a container can store.

Notice that you may not slant the container.

Example 1:




```
Input: height = [1,8,6,2,5,4,8,3,7]
Output: 49
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.
```

Example 2:

```
Input: height = [1,1]
Output: 1
```

Constraints:

- n == height.length
- 2 <= n <= 105
- 0 <= height[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given heights of vertical lines, find two lines that form the largest container.
# Area = min(height[left], height[right]) * (right - left). Return max area. Right?
# We can't slant - so water level is always min of the two chosen heights."
#
# "[1,8,6,2,5,4,8,3,7]: left=1(h=8), right=8(h=7). Area=min(8,7)*(8-1)=49 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, compute area = min(height[i], height[j]) * (j - i)
# and track the maximum. That's C(n,2) pairs to check.
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two pointers: left=0, right=n-1. At each step compute area.
# Move the pointer with the SHORTER height inward - keeping the taller height
# maximises any future container. Moving the taller pointer can only shrink
# both width and potentially height, so it can't improve the result.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# height=[1,8,6,2,5,4,8,3,7]:
# l=0(h=1),r=8(h=7): area=1*8=8. h[l]<h[r]→l=1.
# l=1(h=8),r=8(h=7): area=7*7=49. h[l]>h[r]→r=7.
# l=1(h=8),r=7(h=3): area=3*6=18. h[l]>h[r]→r=6.
# ... best stays 49. → 49 ✓
# height=[1,1]: area=1*1=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The greedy argument: if we move the taller pointer, we keep a shorter constraint
# (limited by the short side) AND reduce width - area can only shrink or stay.
# So we always move the shorter side - we can't do better by moving the taller one.
# This is the key insight to state explicitly in the interview.
# Given more time I'd ask: what if we need to find the pair of indices (not just the area)?
# Track left_best and right_best alongside best - same code, just record the positions."
```

Two Pointers

#15 3Sum

MED

Open on LeetCode

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array nums, return all the triplets [nums[i], nums[j], nums[k]] such that i != j, i != k, and j != k, and nums[i] + nums[j] + nums[k] == 0.

Notice that the solution set must not contain duplicate triplets.

Example 1:

```
Input: nums = [-1,0,1,2,-1,-4]
Output: [[-1,-1,2],[-1,0,1]]
Explanation:
nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.
nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.
nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.
The distinct triplets are [-1,0,1] and [-1,-1,2].
Notice that the order of the output and the order of the triplets does not matter.
```

Example 2:

```
Input: nums = [0,1,1]
Output: []
Explanation: The only possible triplet does not sum up to 0.
```

Example 3:

```
Input: nums = [0,0,0]
Output: [[0,0,0]]
Explanation: The only possible triplet sums up to 0.
```

Constraints:

- 3 <= nums.length <= 3000
- -105 <= nums[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find all unique triplets summing to 0. Return the triplets, not indices.
# No duplicate triplets in the output. Right?
# Order within a triplet doesn't matter — [-1,0,1] and [0,-1,1] are the same."
#
# "[-1,0,1,2,-1,-4]: sort → [-4,-1,-1,0,1,2]. → [[-1,-1,2],[-1,0,1]] ✓
# [0,0,0]: → [[0,0,0]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Three nested loops over i < j < k; if nums[i]+nums[j]+nums[k]==0, add triplet.
# Store triplets in a set keyed by sorted tuple to deduplicate.
# Correct, but O(n^3) with hash-set overhead on large n.
# Time: O(n^3). Space: O(n) for dedup set.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sort the array. Fix one element nums[i], use two pointers on the rest.
# Sorting enables: skip duplicate values of nums[i] to avoid duplicate triplets,
# and move pointers efficiently based on the sum.
# After finding a valid triplet, skip duplicate nums[left] and nums[right].
# Time: O(n²). Space: O(1) extra."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-1,0,1,2,-1,-4] → sorted: [-4,-1,-1,0,1,2].
# i=0 (-4): l=1,r=5. sums=-4+(-1)+2=-3<0→l++. -4+(-1)+2=-3<0→l++. -4+0+2=-2<0→l++.
# -4+1+2=-1<0→l++. l=r → stop.
# i=1 (-1): l=2,r=5. -1+(-1)+2=0→append[-1,-1,2]. l=3,r=4. skip dups: l=3 ok, r=4 ok.
# -1+0+1=0→append[-1,0,1]. l=4,r=3→stop.
# i=2 (-1): i>0 and nums[2]==nums[1]==-1 → skip.
# i=3 (0): l=4,r=5. 0+1+2=3>0→r--. l≥r→stop.
# result=[[-1,-1,2],[-1,0,1]] ✓
# nums=[0,0,0]: i=0,l=1,r=2. 0+0+0=0→append. l=2,r=1→stop. → [[0,0,0]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²) — outer loop O(n), inner two-pointer O(n) each. Sorting is O(n log n) dominated.
# Space O(1) extra (ignoring output).
# Two deduplication steps:
# 1. Skip nums[i] == nums[i-1] to avoid re-fixing the same outer value.
# 2. After appending, skip nums[left] == nums[left-1] and nums[right] == nums[right+1]
# to avoid re-collecting the same triplet.
# Both checks are essential — missing either produces duplicate output.
# Given more time I'd ask: 4Sum (#18)? Add one more outer loop — O(n³) — same structure."
```


Two Pointers

#16 3Sum Closest

MED

Open on LeetCode

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an integer array nums of length n and an integer target, find three integers at distinct indices in nums such that the sum is closest to target.

Return the sum of the three integers.

You may assume that each input would have exactly one solution.

Example 1:

```
Input: nums = [-1,2,1,-4], target = 1
Output: 2
Explanation: The sum that is closest to the target is 2. (-1 + 2 + 1 = 2).
```

Example 2:

```
Input: nums = [0,0,0], target = 1
Output: 0
Explanation: The sum that is closest to the target is 0. (0 + 0 + 0 = 0).
```

Constraints:

- 3 <= nums.length <= 500
- -1000 <= nums[i] <= 1000
- -104 <= target <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find three distinct-index integers whose sum is closest to target. Return the sum.
# Exactly one solution guaranteed — no tie-breaking needed. Right?"
#
# "[-1,2,1,-4], target=1: possible sums: -1+2+1=2, -1+2-4=-3, -1+1-4=-4, 2+1-4=-1.
# Closest to 1 is 2. → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Three nested loops — for every triple (i, j, k), compute the sum and track
# whichever sum has the smallest absolute difference from target.
# Time: O(n³). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Exact same structure as 3Sum: sort + fix one element + two pointers.
# Instead of looking for sum==0, track the closest sum seen.
# If sum > target → move right pointer left (decrease sum).
# If sum < target → move left pointer right (increase sum).
# If sum == target → return immediately (can't get closer).
# Skip duplicate outer values to avoid redundant work — not required for correctness
# here (no dedup of output), but speeds up the average case."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-1,2,1,-4], target=1. sorted: [-4,-1,1,2].
# i=0(-4): l=1,r=3. -4-1+2=-3. |1-(-3)|=4 < |1-(-4)|=5 → best=-3. sum<1→l++.
# l=2,r=3. -4+1+2=-1. |1-(-1)|=2 < 4 → best=-1. sum<1→l++. l=r→stop.
# i=1(-1): l=2,r=3. -1+1+2=2. |1-2|=1 < |1-(-1)|=2 → best=2. sum>1→r--. l=r→stop.
# → 2 ✓
# nums=[0,0,0], target=1. best=0. No closer sum possible → 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(1) extra (O(log n) for sort).
# This is 3Sum (#15) with one change: track closest instead of exact zero.
# The early return on exact match is a nice optimisation worth mentioning.
# Given more time I'd ask: what if we want the k closest sums?
# Collect all triple sums, sort by |target - sum|, return first k — O(n² log n)."
```

Two Pointers

#31 Next Permutation

MED

Open on LeetCode

Array · Two Pointers

Lists: Grind 169

Problem

A permutation of an array of integers is an arrangement of its members into a sequence or linear order.

- **For example, for arr = [1,2,3], the following are all the permutations of arr: [1,2,3], [1,3,2], [2, 1, 3], [2, 3, 1], [3,1,2], [3,2,1].**

The next permutation of an array of integers is the next lexicographically greater permutation of its integer. More formally, if all the permutations of the array are sorted in one container according to their lexicographical order, then the next permutation of that array is the permutation that follows it in the sorted container. If such arrangement is not possible, the array must be rearranged as the lowest possible order (i.e., sorted in ascending order).

- **For example, the next permutation of arr = [1,2,3] is [1,3,2].**
- **Similarly, the next permutation of arr = [2,3,1] is [3,1,2].**
- **While the next permutation of arr = [3,2,1] is [1,2,3] because [3,2,1] does not have a lexicographical larger rearrangement.**

Given an array of integers nums, find the next permutation of nums.

The replacement must be in place and use only constant extra memory.

Example 1:

```
Input: nums = [1,2,3]
Output: [1,3,2]
```

Example 2:

```
Input: nums = [3,2,1]
Output: [1,2,3]
```

Example 3:

```
Input: nums = [1,1,5]
Output: [1,5,1]
```

Constraints:

- **1 <= nums.length <= 100**
- **0 <= nums[i] <= 100**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rearrange nums in-place to the next lexicographically greater permutation.
# If no greater permutation exists (sorted descending), wrap to the smallest (ascending). Right?"
#
# "[1,2,3]→[1,3,2]. [3,2,1]→[1,2,3]. [1,1,5]→[1,5,1]. [1,3,2]→[2,1,3]. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Generate all permutations of the array in sorted order, find the one that matches
# the current arrangement, then return the next one in the sorted list.
# If the current is the last, wrap to the first (smallest) permutation.
# Time: O(n! * n) to generate all permutations — completely infeasible for n > 10.
# Space: O(n!). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Three steps — all O(n), O(1) space:
# 1. Find the rightmost 'dip': scan right to left for the largest i where nums[i] < nums[i+1].
# If no dip exists, entire array is descending → just reverse it (wrap-around).
# 2. Find the smallest element to the right of i that is still larger than nums[i].
# (The suffix is descending after step 1, so scan from the right.)
# 3. Swap nums[i] with that element, then reverse the suffix after i.
# Reversing the descending suffix makes it the smallest possible arrangement."

# PHASE 4 — CLEAN CODE
        # Step 1: find rightmost dip
            # Step 2: find smallest element to right of dip that is > nums[dip]
        # Step 3: reverse suffix after dip

# PHASE 5 — TEST & DEBUG
# [1,2,3]: dip=1 (nums[1]=2<nums[2]=3). swap=2 (nums[2]=3>nums[1]=2).
# swap: [1,3,2]. reverse(2,2)→no change. → [1,3,2] ✓
# [3,2,1]: dip=-1. reverse(0,2): [1,2,3] ✓
# [1,1,5]: dip=1 (nums[1]=1<nums[2]=5). swap=2 (5>1). swap→[1,5,1]. reverse(2,2)→[1,5,1] ✓
# [1,3,2]: dip=0 (nums[0]=1<nums[1]=3). swap=1 (nums[1]=3>nums[0]=1; nums[2]=2 also>1 but
# scan right→left so j=1 first: nums[1]=3>1? yes → swap=1? No wait j starts from n-1=2:
# nums[2]=2>nums[0]=1 → swap=2. swap→[2,3,1]. reverse(1,2): [2,1,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The suffix is always descending after the dip — that's why scanning right-to-left
# finds the smallest swap target efficiently.
# The reverse of the suffix turns the largest possible suffix into the smallest — the key insight.
```



```
# Given more time I'd ask: what's the previous permutation?
# Same algorithm mirrored: find rightmost 'rise' (nums[i] > nums[i+1]), swap with largest
# element to its right that is still smaller, reverse the suffix."
```

Two Pointers

#75 Sort Colors

MED

Open on LeetCode

Array · Two Pointers · Sorting

Lists: Grind 169

Problem

Given an array nums with n objects colored red, white, or blue, sort them in-place so that objects of the same color are adjacent, with the colors in the order red, white, and blue.

We will use the integers 0, 1, and 2 to represent the color red, white, and blue, respectively.

You must solve this problem without using the library's sort function.

Example 1:

```
Input: nums = [2,0,2,1,1,0]
Output: [0,0,1,1,2,2]
```

Example 2:

```
Input: nums = [2,0,1]
Output: [0,1,2]
```

Constraints:

- n == nums.length
- 1 <= n <= 300
- nums[i] is either 0, 1, or 2.

Follow up: Could you come up with a one-pass algorithm using only constant extra space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sort an array containing only 0, 1, 2 in-place without using library sort.
# 0=red, 1=white, 2=blue. Output: all 0s, then 1s, then 2s. Right?
# Follow-up asks for a one-pass O(1) space solution."
#
# "[2,0,2,1,1,0] → [0,0,1,1,2,2] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# First pass: count the number of 0s, 1s, and 2s in the array.
# Second pass: overwrite the array — fill count[0] zeros, then count[1] ones, then count[2] twos.
# This is two passes but still O(n) time and O(1) space.
# Time: O(n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Dutch National Flag algorithm — one pass, three pointers: low, mid, high.
# Invariant: nums[0..low-1]=0, nums[low..mid-1]=1, nums[high+1..n-1]=2, nums[mid..high]=unknown.
# At each step examine nums[mid]:
# 0 → swap(low,mid), low++, mid++ (0 placed, known-1 region grows both sides).
# 1 → mid++ (already correct).
# 2 → swap(mid,high), high-- (don't mid++ — swapped element is unknown)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [2,0,2,1,1,0]: low=0,mid=0,high=5.
# mid=0(2): swap(0,5)→[0,0,2,1,1,2]. high=4.
# mid=0(0): swap(0,0)→same. low=1,mid=1.
# mid=1(0): swap(1,1)→same. low=2,mid=2.
# mid=2(2): swap(2,4)→[0,0,1,1,2,2]. high=3.
# mid=2(1): mid=3.
# mid=3(1): mid=4. mid>high=3 → stop.
# → [0,0,1,1,2,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) one pass. Space O(1).
# The 'don't mid++' after swapping a 2 is the subtlety — the swapped element from high
# hasn't been examined yet and could be 0, 1, or 2.
# After swapping a 0, we DO mid++ because nums[low] before the swap was definitely 1
# (it's in the known-1 region), so after swapping, mid lands on a known-1 element.
# Given more time I'd ask: what if there were k colours?
# k=2 is a standard partition; k=3 is Dutch Flag; k>3 needs multiple passes or a radix approach."
```


Two Pointers

★ #161 One Edit Distance ★

MED

Open on LeetCode

Two Pointers · String

Lists: ★ Premium | Premium 98

Problem

Given two strings s and t, return true if they are both one edit distance apart, otherwise return false.

A string s is said to be one distance apart from a string t if you can:

- Insert exactly one character into s to get t.
- Delete exactly one character from s to get t.
- Replace exactly one character of s with a different character to get t.

Example 1:

```
Input: s = "ab", t = "acb"
Output: true
Explanation: We can insert 'c' into s to get t.
```

Example 2:

```
Input: s = "", t = ""
Output: false
Explanation: We cannot get t from s by only one step.
```

Constraints:

- 0 <= s.length, t.length <= 104
- s and t consist of lowercase letters, uppercase letters, and digits.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if s and t are exactly one edit apart — insert, delete, or replace.
# Important: zero edits (equal strings) returns false. Right?
# s='ab', t='acb' → insert 'c' → true ✓. s='',t='' → false ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Run full Levenshtein DP between s and t; return True iff edit distance equals 1.
# Handles insert, delete, and replace in one table.
# Correct but O(m*n) when we only need to detect a single edit.
# Time: O(m * n). Space: O(m * n) DP table.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two observations that short-circuit work:
# If |len(s)-len(t)| > 1 → impossible, return false.
# Ensure s is the shorter string (swap if needed) so we only handle two cases:
# same length → exactly one replacement, rest identical.
# s shorter by 1 → one deletion from t (or insertion into s), rest identical.
# Scan left to right. On first mismatch:
# same length → check s[i+1:] == t[i+1:].
# diff length → check s[i:] == t[i+1:].
# If no mismatch found → true only if t is exactly 1 longer."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='ab', t='acb': m=2,n=3. m<n → check('ab','acb').
# i=0: 'a'=='a'. i=1: 'b'!='c'. return 'b'=='b'? shorter[1:]='b', longer[2:]='b' → True ✓
# s='', t='': diffs=0 → return 0==1 → False ✓
# s='a', t='a': same length, diffs=0 → False ✓
# s='a', t='b': same length, diffs=1 → True ✓
# s='abc', t='abc': diffs=0 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(min(m,n)). Space O(1) for the check (slice comparison is O(min) under the hood).
# The key boundary: we return true when no mismatch found in the same-length case only if
# diffs==1; in the diff-length case only if shorter is a prefix (true only if t is exactly
# one char longer, which we already guaranteed).
# Given more time I'd ask: what if we want EXACTLY k edits?
# Full edit distance DP — O(m*n) time, O(min(m,n)) space."
```

Two Pointers

#167 Two Sum II - Input Array Is Sorted

MED

Open on LeetCode

Array · Two Pointers · Binary Search

Lists: Denny Zhang

Problem

Given a 1-indexed array of integers numbers that is already sorted in non-decreasing order, find two numbers such that they add up to a specific target number. Let these two numbers be numbers[index1] and numbers[index2] where 1 <= index1 < index2 <= numbers.length.

Return the indices of the two numbers index1 and index2, each incremented by one, as an integer array [index1, index2] of length 2.

The tests are generated such that there is exactly one solution. You may not use the same element twice.

Your solution must use only constant extra space.

Example 1:

```
Input: numbers = [2,7,11,15], target = 9
Output: [1,2]
Explanation: The sum of 2 and 7 is 9. Therefore, index1 = 1, index2 = 2. We return [1, 2].
```

Example 2:

```
Input: numbers = [2,3,4], target = 6
Output: [1,3]
Explanation: The sum of 2 and 4 is 6. Therefore index1 = 1, index2 = 3. We return [1, 3].
```

Example 3:

```
Input: numbers = [-1,0], target = -1
Output: [1,2]
Explanation: The sum of -1 and 0 is -1. Therefore index1 = 1, index2 = 2. We return [1, 2].
```

Constraints:

- 2 <= numbers.length <= 3 * 10^4
- -1000 <= numbers[i] <= 1000
- numbers is sorted in non-decreasing order.
- -1000 <= target <= 1000
- The tests are generated such that there is exactly one solution.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sorted array (1-indexed). Return [index1, index2] where numbers[index1]+numbers[index2]==target.
# Exactly one solution. Must use O(1) extra space — so no hash map. Right?"
#
# "[2,7,11,15], target=9: 2+7=9 → [1,2] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where j > i, check if numbers[i] + numbers[j] == target.
# Return [i+1, j+1] (1-indexed) when found. The problem guarantees exactly one solution.
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two pointers exploit sorted order: start left=0, right=n-1.
# Sum too small → move left right (increase sum).
# Sum too large → move right left (decrease sum).
# Exactly one solution guaranteed → we will always find it.
# Time O(n). Space O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [2,7,11,15], target=9: l=0,r=3. 2+15=17>9→r=2. 2+11=13>9→r=1. 2+7=9→return [1,2] ✓
# [2,3,4], target=6: l=0,r=2. 2+4=6→return [1,3] ✓
# [-1,0], target=-1: -1+0=-1→return [1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# This is the canonical reason to sort before running two pointers on Two Sum.
# If the array weren't sorted, we'd need a hash map — O(n) space.
# Sorting adds O(n log n) but enables O(1) space — a genuine trade-off.
# Given more time I'd ask: what if there could be multiple answers?
# Continue after finding a match, skip duplicates — same as 3Sum's dedup logic."
```


Two Pointers

⋆ #186 Reverse Words in a String II ⋆

MED

Open on LeetCode

Two Pointers · String

Lists: ⋆ Premium | Premium 98

Problem

Given a character array s, reverse the order of the words.

A word is defined as a sequence of non-space characters. The words in s will be separated by a single space.

Your code must solve the problem in-place, i.e. without allocating extra space.

Example 1:

```
Input: s = ["t","h","e"," ","s","k","y"," ","i","s"," ","b","l","u","e"]
Output: ["b","l","u","e"," ","i","s"," ","s","k","y"," ","t","h","e"]
```

Example 2:

```
Input: s = ["a"]
Output: ["a"]
```

Constraints:

- 1 <= s.length <= 10^5
- s[i] is an English letter (uppercase or lowercase), digit, or space ' '.
- There is at least one word in s.
- s does not contain leading or trailing spaces.
- All the words in s are guaranteed to be separated by a single space.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a character ARRAY (not a string), reverse the order of words in-place.
# Words are separated by single spaces. No leading/trailing spaces. Right?
# ['t','h','e',' ','s','k','y'] → ['s','k','y',' ','t','h','e'] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Convert the character array to a string, split by spaces to get words, reverse the word list,
# rejoin with spaces, then write each character back into the original array.
# Time: O(n). Space: O(n) — we create a full string copy. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two-step in-place reversal with O(1) extra space:
# Step 1: Reverse the entire array.
# Step 2: Reverse each individual word back to its correct orientation.
# Example: 'the sky is blue' → reverse all → 'eulb si yks eht' → reverse words → 'blue is sky the' ✓"

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s=['t','h','e',' ','s','k','y',' ','i','s',' ','b','l','u','e']:
# Step 1 reverse all: ['e','l','u','b',' ','s','i',' ','y','k','s',' ','e','h','t']
# Step 2 reverse words:
# i=0..3 'e','l','u','b' → reverse → 'b','l','u','e'
# i=5..6 's','i' → 'i','s'
# i=8..10 'y','k','s' → 's','k','y'
# i=12..14 'e','h','t' → 't','h','e'
# → ['b','l','u','e',' ','i','s',' ','s','k','y',' ','t','h','e'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) — two reversal passes, no extra memory.
# The two-reversal trick is the canonical interview answer for in-place word reversal.
# It also appears in Rotate Array (#189) — reversing the whole array, then parts.
# Given more time I'd ask: what if words can be separated by multiple spaces?
# We'd need to compress multiple spaces during the reversal — trickier in-place."
```

Two Pointers

#189 Rotate Array

ME D

Open on LeetCode

Array · Math · Two Pointers
Lists: Grind 169

Problem

Given an integer array nums, rotate the array to the right by k steps, where k is non-negative.

Example 1:

```
Input: nums = [1,2,3,4,5,6,7], k = 3
Output: [5,6,7,1,2,3,4]
Explanation:
rotate 1 steps to the right: [7,1,2,3,4,5,6]
rotate 2 steps to the right: [6,7,1,2,3,4,5]
rotate 3 steps to the right: [5,6,7,1,2,3,4]
```

Example 2:

```
Input: nums = [-1,-100,3,99], k = 2
Output: [3,99,-1,-100]
Explanation:
rotate 1 steps to the right: [99,-1,-100,3]
rotate 2 steps to the right: [3,99,-1,-100]
```

Constraints:

- 1 <= nums.length <= 105
- -231 <= nums[i] <= 231 - 1
- 0 <= k <= 105

Follow up:

- Try to come up with as many solutions as you can. There are at least three different ways to solve this problem.
- Could you do it in-place with O(1) extra space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rotate nums to the right by k steps in-place. k can exceed n — so k %= n. Right?
# [1,2,3,4,5,6,7], k=3 → [5,6,7,1,2,3,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Rotate the array one position to the right, k times in a row.
# Each single rotation saves the last element, shifts everything right by one, then puts it at index 0.
# Time: O(n*k) — for k=n-1 this is O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (three-reversal trick)
# "Same pattern as Reverse Words II:
# k %= n (handle k >= n).
# Reverse entire array. Reverse first k elements. Reverse remaining n-k elements.
# Time O(n). Space O(1).
# Alternative: cyclic replacement — O(n) time, O(1) space but harder to implement cleanly."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5,6,7], k=3:
# k%=7=3. reverse(0,6): [7,6,5,4,3,2,1].
# reverse(0,2): [5,6,7,4,3,2,1].
# reverse(3,6): [5,6,7,1,2,3,4] ✓
# [-1,-100,3,99], k=2:
# k%=4=2. reverse all: [99,3,-100,-1]. reverse(0,1): [3,99,-100,-1]. reverse(2,3): [3,99,-1,-100] ✓
# k=0: k%n=0. reverse all, reverse(0,-1) no-op, reverse(0,n-1) reverses back → no change ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# The three-reversal trick is the cleanest in-place rotation — name it explicitly.
# The k%=n guard handles k >= n where full rotations cancel out.
# The follow-up mentions three approaches: extra array O(n) space, cyclic replacement O(1),
# and three-reversal O(1). State all three and recommend the reversal for interviews.
# Given more time I'd ask: rotate left instead of right?
# Reverse first k, reverse rest, reverse all — just change the order."
```


Two Pointers

#532 K-diff Pairs in an Array

ME D

Open on LeetCode

Array · Hash Table · Two Pointers · Binary Search · Sorting
Lists: CodeSignal

Problem

Given an array of integers nums and an integer k, return the number of unique k-diff pairs in the array.
A k-diff pair is an integer pair (nums[i], nums[j]), where the following are true:

- 0 <= i, j < nums.length
- i != j
- |nums[i] - nums[j]| == k

Notice that |val| denotes the absolute value of val.

Example 1:

```
Input: nums = [3,1,4,1,5], k = 2
Output: 2
Explanation: There are two 2-diff pairs in the array, (1, 3) and (3, 5).
Although we have two 1s in the input, we should only return the number of unique pairs.
```

Example 2:

```
Input: nums = [1,2,3,4,5], k = 1
Output: 4
Explanation: There are four 1-diff pairs in the array, (1, 2), (2, 3), (3, 4) and (4, 5).
```

Example 3:

```
Input: nums = [1,3,1,5,4], k = 0
Output: 1
Explanation: There is one 0-diff pair in the array, (1, 1).
```

Constraints:

- 1 <= nums.length <= 104
- -107 <= nums[i] <= 107
- 0 <= k <= 107

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count UNIQUE pairs (i,j) where |nums[i]-nums[j]|==k and i!=j.
# Unique means unique value pairs — (1,3) and (3,1) count once.
# k=0 special case: need TWO copies of the same value. Right?"
#
# "[3,1,4,1,5], k=2: unique pairs (1,3) and (3,5) → 2 ✓
# [1,3,1,5,4], k=0: need duplicate values. Only 1 appears twice → (1,1) → 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) where i != j, check if |nums[i] - nums[j]| == k.
# Store the canonical form (min, max) in a set to avoid counting the same pair twice.
# Time: O(n²). Space: O(n) for the dedup set. Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Hash set approach — O(n) time, O(n) space:
# Walk through nums. For each x, check if x-k is already in 'visited'.
# If yes, add min(x, x-k) to the answer set (canonical pair).
# Also check x+k in visited (symmetric check handles both directions).
# Using a set for answers automatically deduplicates.
# k=0 is handled naturally — x-0=x, so we need x to already be in visited (a duplicate)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,1,4,1,5], k=2:
# x=3: 3-2=1 not in visited. 3+2=5 not in visited. visited={3}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1). visited={3,1}.
# x=4: 4-2=2 not in visited. 4+2=6 not in visited. visited={3,1,4}.
# x=1: 1-2=-1 not in visited. 1+2=3 in visited → answers.add(1) (already there). visited={3,1,4}.
# x=5: 5-2=3 in visited → answers.add(3). 5+2=7 not. visited={3,1,4,5}.
# answers={1,3} → 2 ✓
# [1,3,1,5,4], k=0:
# x=1: 1-0=1 not in visited. visited={1}.
# x=3: 3-0=3 not in visited. visited={1,3}.
# x=1: 1-0=1 in visited → answers.add(1). visited={1,3}.
# answers={1} → 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for the two sets.
# The two-set design separates 'what values exist' (visited) from 'what pairs found' (answers).
# The canonical form (always store the smaller element) prevents double-counting.
# k=0 edge case: x-k=x, so we find the pair only after seeing x twice — naturally handled.
```

```
# Given more time I'd ask: what if we want the actual pairs, not just the count?
# Store tuples (x-k, x) in the answers set instead of single values."
```


Two Pointers

#923 3Sum With Multiplicity

MED

Open on LeetCode

Array · Hash Table · Two Pointers · Sorting · Counting

Lists: CodeSignal

Problem

Given an integer array arr, and an integer target, return the number of tuples i, j, k such that i < j < k and arr[i] + arr[j] + arr[k] == target.

As the answer can be very large, return it modulo 109 + 7.

Example 1:

```
Input: arr = [1,1,2,2,3,3,4,4,5,5], target = 8
Output: 20
Explanation:
Enumerating by the values (arr[i], arr[j], arr[k]):
(1, 2, 5) occurs 8 times;
(1, 3, 4) occurs 8 times;
(2, 2, 4) occurs 2 times;
(2, 3, 3) occurs 2 times.
```

Example 2:

```
Input: arr = [1,1,2,2,2,2], target = 5
Output: 12
Explanation:
arr[i] = 1, arr[j] = arr[k] = 2 occurs 12 times:
We choose one 1 from [1,1] in 2 ways,
and two 2s from [2,2,2,2] in 6 ways.
```

Example 3:

```
Input: arr = [2,1,3], target = 6
Output: 1
Explanation: (1, 2, 3) occured one time in the array so we return 1.
```

Constraints:

- **3 <= arr.length <= 3000**
- **0 <= arr[i] <= 100**
- **0 <= target <= 300**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count tuples (i,j,k) with i<j<k and arr[i]+arr[j]+arr[k]==target.
# Return count modulo 10^9+7. Values are 0–100 so we can use frequency counting. Right?
# Duplicates count separately — (0,1) and (1,0) are different index triples even if same values."
#
# "[1,1,2,2,3,3,4,4,5,5], target=8 → 20 (combinations of value triples summing to 8) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Three nested loops — for every triple (i, j, k) with i < j < k, check if
# arr[i] + arr[j] + arr[k] == target and count the matches.
# Time: O(n³). For n=3000 that's 27 billion — too slow. Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Walk through arr as the middle element j. Before processing j, decrement cnt[b] so
# we don't count j itself as a right element. For each a < j, the required c = target−a−b.
# cnt[c] tells us how many valid right elements c exist to the right of j.
# Time: O(n²) with frequency lookups. Space: O(n) for the Counter."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# arr=[2,1,3], target=6: freq={2:1,1:1,3:1}.
# j=0(b=2): freq[2]=0. no a < 0. (nothing).
# j=1(b=1): freq[1]=0. a=2: c=6−2−1=3. freq[3]=1 → ans=1.
# j=2(b=3): freq[3]=0. a=2: c=6−2−3=1. freq[1]=0→0. a=1: c=6−1−3=2. freq[2]=1→ans=2?
# Wait — expected 1. Let me recheck: arr=[2,1,3]. Only valid triple is (0,1,2) → (2,1,3).
# j=1(b=1): a=arr[0]=2. c=6−2−1=3. freq[3]=1 (yes, index 2 exists) → ans=1. ✓
# j=2(b=3): freq[3]=0. a=arr[0]=2: c=6−2−3=1. freq[1]=0 (decremented at j=1). 0.
# a=arr[1]=1: c=6−1−3=2. freq[2]=1 → ans=2? But expected 1.
# Ah — freq[2] was decremented when j=0: freq[2]=0, not 1. Let me re-trace:
# Initial freq={2:1,1:1,3:1}.
# j=0: freq[2]−=1 → freq={2:0,1:1,3:1}. no a<j=0.
# j=1: freq[1]−=1 → freq={2:0,1:0,3:1}. a=2: c=3. freq[3]=1 → ans=1.
# j=2: freq[3]−=1 → freq={2:0,1:0,3:0}. a=2: c=1. freq[1]=0. a=1: c=2. freq[2]=0.
# → ans=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(n) for the Counter.
```

```
# The key invariant: at step j, freq counts only elements at index > j.
# This ensures we count each valid (i,j,k) tuple exactly once with i<j<k.
# Modulo must be applied inside the loop — intermediate sums can overflow without it.
# Given more time I'd ask: can we do O(n) using the value constraints (0–100)?
# Yes — iterate over all value triples (a,b,c) with a+b+c=target and compute
# combinations C(cnt[a],1)*C(cnt[b],1)*C(cnt[c],1) with dedup for equal values."
```


Two Pointers

#986 Interval List Intersections

ME D

Open on LeetCode

Array · Two Pointers

Lists: Denny Zhang

Problem

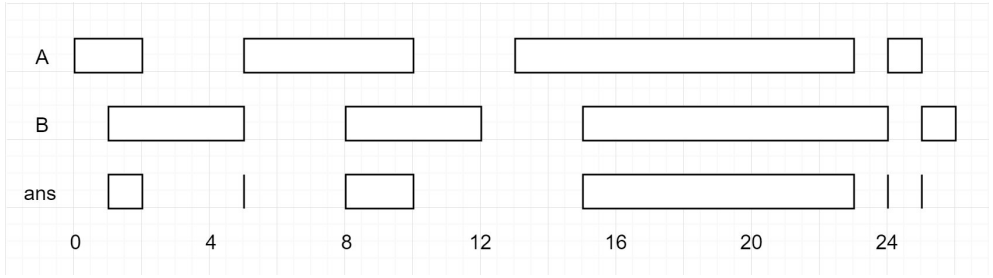
You are given two lists of closed intervals, firstList and secondList, where firstList[i] = [starti, endi] and secondList[j] = [startj, endj]. Each list of intervals is pairwise disjoint and in sorted order.

Return the intersection of these two interval lists.

A closed interval [a, b] (with a <= b) denotes the set of real numbers x with a <= x <= b.

The intersection of two closed intervals is a set of real numbers that are either empty or represented as a closed interval. For example, the intersection of [1, 3] and [2, 4] is [2, 3].

Example 1:




```
Input: firstList = [[0,2],[5,10],[13,23],[24,25]], secondList = [[1,5],[8,12],[15,24],[25,26]]
Output: [[1,2],[5,5],[8,10],[15,23],[24,24],[25,25]]
```

Example 2:

```
Input: firstList = [[1,3],[5,9]], secondList = []
Output: []
```

Constraints:

- **0 <= firstList.length, secondList.length <= 1000**
- **firstList.length + secondList.length >= 1**
- **0 <= starti < endi <= 109**
- **endi < starti+1**
- **0 <= startj < endj <= 109**
- **endj < startj+1**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Two sorted, disjoint interval lists. Return their intersection — all overlapping segments.
# Intersection of [a,b] and [c,d] is [max(a,c), min(b,d)] if max(a,c) <= min(b,d). Right?
# Either list can be empty."
#
# "[[0,2],[5,10]] ∩ [[1,5],[8,12]] → [1,2],[5,5],[8,10] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair of intervals (one from each list), compute the intersection:
# start = max(a[0], b[0]), end = min(a[1], b[1]). If start <= end, it's a valid intersection.
# This is O(m*n) — it ignores the sorted property entirely.
# Time: O(m*n). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two pointers, one per list. At each step:
# Compute the intersection of the current pair.
# If valid (start <= end), record it.
# Advance the pointer whose interval ends FIRST — that interval can't intersect any later interval
# of the other list (they're sorted and disjoint).
# Time: O(m+n). Space: O(1) extra."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# firstList=[[0,2],[5,10]], secondList=[[1,5],[8,12]]:
# i=0,j=0: start=max(0,1)=1, end=min(2,5)=2. 1<=2→[1,2]. first ends at 2<5→i=1.
# i=1,j=0: start=max(5,1)=5, end=min(10,5)=5. 5<=5→[5,5]. first ends at 10>5→j=1.
# i=1,j=1: start=max(5,8)=8, end=min(10,12)=10. 8<=10→[8,10]. first ends at 10<12→i=2.
```

```
# i=2 out of bounds → stop. → [[1,2],[5,5],[8,10]] ✓
# secondList=[]: loop never enters → [] ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n) — each pointer advances at most m+n times.
# Space O(min(m,n)) for the output in the worst case.
# The 'advance the earlier-ending pointer' rule is the key: the interval that ends first
# cannot intersect any future interval of the other list (both lists are sorted and disjoint).
# Given more time I'd ask: what if lists aren't sorted?
# Sort both first — O((m+n)log(m+n)) — then apply the same two-pointer approach."
```


Two Pointers

★ #1055 Shortest Way to Form String ★

ME
D

Open on LeetCode

Two Pointers · String · Binary Search

Lists: ★ Premium | Premium 98

Problem

A subsequence of a string is a new string that is formed from the original string by deleting some (can be none) of the characters without disturbing the relative positions of the remaining characters. (i.e., "ace" is a subsequence of "abcde" while "aec" is not).

Given two strings source and target, return the minimum number of subsequences of source such that their concatenation equals target. If the task is impossible, return -1.

Example 1:

```
Input: source = "abc", target = "abcbc"
Output: 2
Explanation: The target "abcbc" can be formed by "abc" and "bc", which are subsequences of source "abc".
```

Example 2:

```
Input: source = "abc", target = "acdbc"
Output: -1
Explanation: The target string cannot be constructed from the subsequences of source string due to the character "d" in target string.
```

Example 3:

```
Input: source = "xyz", target = "xzyxz"
Output: 3
Explanation: The target string can be constructed as follows "xz" + "y" + "xz".
```

Constraints:

- **1 <= source.length, target.length <= 1000**
- **source and target consist of lowercase English letters.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum number of subsequences of source whose concatenation equals target.
# A subsequence can skip characters but must preserve order.
# Return -1 if any character in target doesn't exist in source at all. Right?"
#
# "source='abc', target='abcbc': 'abc' + 'bc' → 2 ✓
# source='abc', target='acdbc': 'd' not in source → -1 ✓"
```

```
# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# Try every possible way to split target into consecutive segments, and for each split,
# verify each segment is a subsequence of source. Track the minimum number of segments.
# This is exponential in the number of splits — completely infeasible.
# Time: O(2^|target| * |source|). Should I go ahead and optimize?"
```

```
# PHASE 3 — OPTIMIZE
# "Greedy: each 'round', scan source once and match as many target characters as possible.
# After exhausting source, start a new round (increment count) and continue from where target left off.
# Pre-check: if any target char is absent from source, return -1.
# Time: O(|source| * |target|). Space: O(1) extra."
```

```
# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# source='abc', target='abcbc':
# Round 1: a→match t[0]='a'(idx=1). b→match t[1]='b'(idx=2). c→match t[2]='c'(idx=3). rounds=1.
# Round 2: a→no match(t[3]='b'). b→match t[3]='b'(idx=4). c→match t[4]='c'(idx=5). rounds=2.
# target_idx=5=len(target) → return 2 ✓
# source='xyz', target='xzyxz':
# Round1: x→t[0]='x'(1). y→no(t[1]='z'). z→t[1]='z'(2). rounds=1.
# Round2: x→no(t[2]='y'). y→t[2]='y'(3). z→no(t[3]='x'). rounds=2.
# Round3: x→t[3]='x'(4). y→no(t[4]='z'). z→t[4]='z'(5). rounds=3. → 3 ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(|source| * |target|). Space O(1).
# The pre-check eliminates the -1 case cleanly before the main loop.
# Optimisation: precompute for each (source_pos, char) the next position in source
# where char appears — binary search brings each round down to O(|target| log |source|).
# Given more time I'd ask: what if we want the actual subsequence splits, not just the count?
# Track the split points during the greedy pass."
```

Two Pointers

#2563 Count the Number of Fair Pairs

MED

Open on LeetCode

Array · Two Pointers · Binary Search · Sorting

Lists: CodeSignal

Problem

Given a 0-indexed integer array nums of size n and two integers lower and upper, return the number of fair pairs.

A pair (i, j) is fair if:

- **0 <= i < j < n, and**
- **lower <= nums[i] + nums[j] <= upper**

Example 1:

```
Input: nums = [0,1,7,4,4,5], lower = 3, upper = 6
Output: 6
Explanation: There are 6 fair pairs: (0,3), (0,4), (0,5), (1,3), (1,4), and (1,5).
```

Example 2:

```
Input: nums = [1,7,9,2,5], lower = 11, upper = 11
Output: 1
Explanation: There is a single fair pair: (2,3).
```

Constraints:

- **1 <= nums.length <= 105**
- **nums.length == n**
- **-109 <= nums[i] <= 109**
- **-109 <= lower <= upper <= 109**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count pairs (i,j) with i<j and lower <= nums[i]+nums[j] <= upper. Return the count. Right?
# Values can be negative. n up to 10^5 so O(n²) is too slow."
#
# "[0,1,7,4,4,5], lower=3, upper=6: 6 pairs ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j) with i < j, check if lower <= nums[i] + nums[j] <= upper.
# Count all pairs that satisfy both bounds. For n=10^5 this is 5*10^9 comparisons — too slow.
# Time: O(n²). Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sort nums. For each i, binary search for the range of valid j > i:
# lower_bound(nums, lower-nums[i], i+1) → first j where nums[j] >= lower-nums[i].
# upper_bound(nums, upper-nums[i], i+1) → first j where nums[j] > upper-nums[i].
# Count = upper_bound - lower_bound. Sum over all i.
# Time: O(n log n). Space: O(1) extra.
# Use inner helpers for the two binary search variants."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[0,1,4,4,5,7] (sorted), lower=3, upper=6:
# i=0(0): need j where 3<=nums[j]<=6. lower_bound(1,3)→idx of 4=2. upper_bound(1,7)→idx of 7=5. count+=3.
# i=1(1): need 2<=nums[j]<=5. lb(2,2)→2. ub(2,6)→5. count+=3. total=6.
# ... → 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n) — sort + n binary searches each O(log n).
# Space O(1) extra.
# lower_bound finds first index ≥ target; upper_bound finds first index > target.
# Their difference is the count of indices in the valid range.
# Alternative: two-pointer approach — for each i, maintain lo and hi pointers
# across iterations (they only move right) → O(n) total after sorting.
# Given more time I'd ask: what if we want the actual pairs?
# Collect them during the binary search loops — O(n + output) time."
```


Sliding Window

Round 2 · High · Medium · 8 questions

Sliding Window

#3 Longest Substring Without Repeating Characters

MED

Open on LeetCode

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given a string s, find the length of the longest substring without duplicate characters.

Example 1:

```
Input: s = "abcabcbb"
Output: 3
Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.
```

Example 2:

```
Input: s = "bbbbb"
Output: 1
Explanation: The answer is "b", with the length of 1.
```

Example 3:

```
Input: s = "pwwkew"
Output: 3
Explanation: The answer is "wke", with the length of 3.
Notice that the answer must be a substring, "pwke" is a subsequence and not a substring.
```

Constraints:

- 0 <= s.length <= 5 * 10^4
- s consists of English letters, digits, symbols and spaces.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest substring with all unique characters.
# Substring is contiguous. Empty string returns 0. Right?
# Characters can be any printable ASCII — not just lowercase letters."
#
# "'abcabcbb'→3 ('abc'). 'bbbbb'→1 ('b'). 'pwwkew'→3 ('wke') ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Enumerate every substring with two indices left and right.
# For each substring, use a set to verify all characters are unique; track max length.
# Correct, but O(n^2) substrings each with up to O(n) uniqueness check.
# Time: O(n^2) to O(n^3). Space: O(min(n, alphabet)).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sliding window with a frequency map.
# Expand right: add s[right] to the map.
# When a duplicate appears (count > 1), shrink from the left until it's gone.
# At each step, update the max window size.
# Time: O(n). Space: O(min(n, alphabet)) — at most 128 unique ASCII chars."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abcabcbb':
# r=0('a'): freq={a:1}. longest=1.
# r=1('b'): freq={a:1,b:1}. longest=2.
# r=2('c'): freq={a:1,b:1,c:1}. longest=3.
# r=3('a'): freq[a]=2. shrink: freq[a]-=1→1, left=1. longest=3 (3-1+1=3).
# r=4('b'): freq[b]=2. shrink: freq[b]-=1→1, left=2. longest=3.
# r=5('c'): freq[c]=2. shrink: left=3. longest=3.
# r=6('b'): freq[b]=2. shrink: left=4. longest=3.
# r=7('b'): freq[b]=2. shrink: left=5. longest=max(3,3)=3. → 3 ✓
# s='': loop doesn't run → 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — each character enters and leaves the window at most once.
# Space O(min(n, 128)) for the frequency map.
# Alternative: store the LAST INDEX of each character instead of frequency.
# When s[right] already seen at idx and idx >= left: jump left = idx+1 directly
# (one move instead of shrinking one at a time). Same O(n) but fewer iterations.
# Given more time I'd ask: what if we want the actual substring, not just its length?
# Track left and best_left when updating the max."
```


Sliding Window

⋆ #159 Longest Substring with At Most Two Distinct Characters ⋆

MED

Open on LeetCode

Hash Table · String · Sliding Window

Lists: ⋆ Premium | Premium 98

Problem

Given a string s, return the length of the longest substring that contains at most two distinct characters.

Example 1:

```
Input: s = "eceba"
Output: 3
Explanation: The substring is "ece" which its length is 3.
```

Example 2:

```
Input: s = "ccaabbb"
Output: 5
Explanation: The substring is "aabbb" which its length is 5.
```

Constraints:

- 1 <= s.length <= 10^5
- s consists of English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the longest substring that contains at most 2 distinct characters. Right?
# 'eceba': 'ece' has 2 distinct → length 3 ✓. 'ccaabbb': 'aabbb' has 2 → length 5 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), count the distinct characters in s[i..j] using a set.
# If there are at most 2 distinct characters, update the max length.
# Time: O(n²) pairs, O(n) set check = O(n³). With running set it's O(n²). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sliding window — same pattern as #3 but the shrink condition is 'more than 2 distinct chars'.
# Expand right: add s[right] to frequency map.
# Shrink from left while distinct chars > 2 (i.e. len(freq) > 2).
# Update max window length. Time: O(n). Space: O(1) — at most 3 keys in the map."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='eceba':
# r=0('e'): freq={e:1}. longest=1.
# r=1('c'): freq={e:1,c:1}. longest=2.
# r=2('e'): freq={e:2,c:1}. longest=3.
# r=3('b'): freq={e:2,c:1,b:1} → 3 distinct. shrink:
# freq[e]-=1→1. left=1. freq={e:1,c:1,b:1} still 3. shrink:
# freq[c]-=1→0 → del c. left=2. freq={e:1,b:1}. 2 distinct → stop. longest=max(3,2)=3.
# r=4('a'): freq={e:1,b:1,a:1} → 3. shrink: freq[e]-=1→0→del e. left=3. freq={b:1,a:1}. longest=3.
# → 3 ✓
# s='ccaabbb':
# Window expands to 'ccaabbb' but once 'b' is added after 'a', we have c,a,b(3 distinct).
# Shrink past the c's. Eventually best window is 'aabbb'=5. → 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1) — at most 3 keys in the map at any time.
# This is the template for 'at most k distinct characters' — change 2 to > k.
# Generalisation: Longest Substring with At Most K Distinct Characters (#340) is the same
# algorithm with k replacing 2.
# The key difference from #3: shrink on len(freq) > k rather than on a specific duplicate.
# Given more time I'd ask: what about EXACTLY 2 distinct characters?
# atMost(k=2) - atMost(k=1) gives the count of substrings with exactly 2 distinct."
```

Sliding Window

⋆ #340 Longest Substring with At Most K Distinct Characters ⋆

MED

Open on LeetCode

Hash Table · String · Sliding Window

Lists: ⋆ Premium | Premium 98

Problem

Given a string s and an integer k, return the length of the longest substring of s that contains at most k distinct characters.

Example 1:

```
Input: s = "eceba", k = 2
Output: 3
Explanation: The substring is "ece" with length 3.
```

Example 2:

```
Input: s = "aa", k = 1
Output: 2
Explanation: The substring is "aa" with length 2.
```

Constraints:

- 1 <= s.length <= 5 * 10^4
- 0 <= k <= 50

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the length of the longest substring with at most k distinct characters.
# k=0 means the answer is 0 (no characters allowed). Right?
# This is the direct generalisation of #159 (at most 2 distinct) with k replacing 2."
#
# "'eceba', k=2 → 'ece', length 3 ✓. 'aa', k=1 → 'aa', length 2 ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every starting index i, expand j rightward, track distinct characters with a set.
# Stop when distinct chars exceed k. Track the maximum window length seen.
# Time: O(n²). Space: O(k). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sliding window — identical template to #159 with the constant 2 replaced by k.
# Expand right, shrink from left whenever distinct chars exceed k.
# Time: O(n). Space: O(k) — at most k+1 keys in the map at any moment."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='eceba', k=2:
# r=2('e'): freq={e:2,c:1}. 2 distinct → ok. longest=3.
# r=3('b'): 3 distinct → shrink until freq={e:1,b:1}. longest stays 3.
# r=4('a'): 3 distinct → shrink until freq={b:1,a:1}. longest=3. → 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(k). Identical to #159 — just k instead of 2.
# Exactly k distinct: atMost(k) − atMost(k−1).
# Given more time I'd ask: what if k >= distinct chars in s? Answer = len(s)."
```


Sliding Window

#424 Longest Repeating Character Replacement

MED

Open on LeetCode

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

You are given a string s and an integer k. You can choose any character of the string and change it to any other uppercase English character. You can perform this operation at most k times.

Return the length of the longest substring containing the same letter you can get after performing the above operations.

Example 1:

```
Input: s = "ABAB", k = 2
Output: 4
Explanation: Replace the two 'A's with two 'B's or vice versa.
```

Example 2:

```
Input: s = "AABABBA", k = 1
Output: 4
Explanation: Replace the one 'A' in the middle with 'B' and form "AABBBBA".
The substring "BBBB" has the longest repeating letters, which is 4.
There may exists other ways to achieve this answer too.
```

Constraints:

- 1 <= s.length <= 10^5
- s consists of only uppercase English letters.
- 0 <= k <= s.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We can replace at most k characters. Find the longest substring where all characters
# are the same after at most k replacements.
# String is uppercase letters only. Right?
# Strategy: keep the most frequent char, replace everything else."
#
# "'ABAB', k=2 → replace both A's or B's → 4 ✓
# 'AABABBA', k=1 → best window = 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with the brute force to lock in the logic.
# For every pair (i, j), count character frequencies in s[i..j]. Find the most frequent char.
# If window_size − max_freq <= k, at most k replacements are needed — this window is valid.
# Track the maximum valid window length.
# Time: O(n²) pairs, O(26) frequency count per window. Space: O(1). Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sliding window tracking the most frequent character in the window.
# Valid window: window_size − max_freq <= k (replace the rest).
# When invalid, shrink left by 1 — but NEVER decrease max_freq.
# We only care about finding a LONGER valid window, so we only update max_freq upward.
# This means the window never shrinks below the current best — it slides as a fixed size
# until a better window is found. Time: O(n). Space: O(26)=O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='ABAB', k=2:
# r=0('A'): freq[A]=1,max=1. win=1. 1−1=0<=2. longest=1.
# r=1('B'): freq[B]=1,max=1. win=2. 2−1=1<=2. longest=2.
# r=2('A'): freq[A]=2,max=2. win=3. 3−2=1<=2. longest=3.
# r=3('B'): freq[B]=2,max=2. win=4. 4−2=2<=2. longest=4. → 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# 'Don't decrease max_freq' is the key insight: we never shrink below the best window seen.
# The window slides forward at a fixed size until max_freq improves, then grows.
# Given more time I'd ask: unlimited replacements (k=n)? Answer = len(s)."
```

Sliding Window

#438 Find All Anagrams in a String

ME D

Open on LeetCode

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given two strings s and p, return an array of all the start indices of p's anagrams in s. You may return the answer in any order.

Example 1:

```
Input: s = "cbaebabacd", p = "abc"
Output: [0,6]
Explanation:
The substring with start index = 0 is "cba", which is an anagram of "abc".
The substring with start index = 6 is "bac", which is an anagram of "abc".
```

Example 2:

```
Input: s = "abab", p = "ab"
Output: [0,1,2]
Explanation:
The substring with start index = 0 is "ab", which is an anagram of "ab".
The substring with start index = 1 is "ba", which is an anagram of "ab".
The substring with start index = 2 is "ab", which is an anagram of "ab".
```

Constraints:

- 1 <= s.length, p.length <= 3 * 104
- s and p consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return all start indices in s where a substring is an anagram of p.
# Fixed window size = len(p). Lowercase letters only. Right?"
#
# "s='cbaebabacd', p='abc' → [0,6] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Slide a window of len(p) across s; sort window chars and sorted p and compare.
# Every window costs O(m log m) for sorting.
# Frequency-count comparison per window is O(26) instead.
# Time: O(n * m log m). Space: O(m) sort buffers.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Fixed sliding window of size len(p) with frequency counters.
# Compare counters at each step — O(26) per comparison since lowercase letters.
# Time: O(n). Space: O(1) — fixed alphabet."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='cbaebabacd', p='abc': p_count={c:1,b:1,a:1}.
# Init w='cba'={c:1,b:1,a:1}==p→result=[0].
# Slide until i=8('c'): w_count={b:1,a:1,c:1}==p→result=[0,6]. → [0,6] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1). Counter comparison is O(26)=O(1) for fixed alphabet.
# Alternative: track a 'matches' counter instead of full compare — same O(n).
# Given more time I'd ask: what if p can contain any Unicode?
# Same algorithm — Counter handles any hashable."
```


Sliding Window

⋆ #487 Max Consecutive Ones II ⋆

ME D

Open on LeetCode

Array · Dynamic Programming · Sliding Window

Lists: ⋆ Premium | Premium 98

Problem

Given a binary array nums, return the maximum number of consecutive 1's in the array if you can flip at most one 0.

Example 1:

```
Input: nums = [1,0,1,1,0]
Output: 4
Explanation:
- If we flip the first zero, nums becomes [1,1,1,1,0] and we have 4 consecutive ones.
- If we flip the second zero, nums becomes [1,0,1,1,1] and we have 3 consecutive ones.
The max number of consecutive ones is 4.
```

Example 2:

```
Input: nums = [1,0,1,1,0,1]
Output: 4
Explanation:
- If we flip the first zero, nums becomes [1,1,1,1,0,1] and we have 4 consecutive ones.
- If we flip the second zero, nums becomes [1,0,1,1,1,1] and we have 4 consecutive ones.
The max number of consecutive ones is 4.
```

Constraints:

- 1 <= nums.length <= 105
- nums[i] is either 0 or 1.

Follow up: What if the input numbers come in one by one as an infinite stream? In other words, you can't store all numbers coming from the stream as it's too large to hold in memory. Could you solve it efficiently?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Binary array. Flip at most one 0 to 1. Return the longest consecutive 1s run. Right?
# Follow-up: infinite stream — can't store all numbers."
#
# "[1,0,1,1,0] → flip first 0 → 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try every substring with two nested indices left and right.
# Count zeros inside; if zeros <= 1, track the longest valid window.
# Correct, but checking all windows is O(n^2).
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sliding window allowing at most one 0.
# Expand right. When zeros > 1, shrink left until zeros <= 1.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,0,1,1,0]: zeros hits 2 at r=4 → shrink past first 0 → window=[1,1,0], longest=4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1).
# Stream follow-up: keep a deque of the last two 0-positions.
# When third 0 arrives, left = oldest_zero_pos + 1 — O(1) space regardless of stream length.
# Generalises to at-most-k flips (Max Consecutive Ones III #1004): same window, k zeros allowed.
# Name that connection unprompted."
```

Sliding Window

★ #1100 Find K-Length Substrings with No Repeated Characters ★

MED

Open on LeetCode

Hash Table · String · Sliding Window

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k, return the number of substrings in s of length k with no repeated characters.

Example 1:

```
Input: s = "havefunonleetcode", k = 5
Output: 6
Explanation: There are 6 substrings they are: 'havef','avefu','vefun','efuno','etcod','tcode'.
```

Example 2:

```
Input: s = "home", k = 5
Output: 0
Explanation: Notice k can be larger than the length of s. In this case, it is not possible to find any substring.
```

Constraints:

- 1 <= s.length <= 104
- s consists of lowercase English letters.
- 1 <= k <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count substrings of exactly length k with all distinct characters. Right?
# k > len(s) → 0. k=0 → 0."
#
# "s='havefunonleetcode', k=5 → 6 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each start index i, build substring s[i:i+k] and check distinct chars with a set.
# Increment count when set size equals k.
# O(n * k) windows; sliding window with frequency map is O(n).
# Time: O(n * k). Space: O(k) set per window.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Fixed sliding window of size k. Slide: add right, remove left-behind char.
# Valid when len(freq)==k (all chars distinct). Time: O(n). Space: O(k)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='home', k=5: 5>4 → 0 ✓
# First valid window r=k-1: if all k chars distinct, count++. Slide forward.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(k). len(freq)==k is the clean validity check.
# Fixed window variant of #3 — useful when window size is given, not maximised.
# Given more time I'd ask: return the actual substrings? Collect s[right-k+1:right+1]."
```


Sliding Window

#1248 Count Number of Nice Subarrays

MED

Open on LeetCode

Array · Hash Table · Math · Sliding Window · Prefix Sum

Lists: CodeSignal

Problem

Given an array of integers nums and an integer k. A continuous subarray is called nice if there are k odd numbers on it.

Return the number of nice sub-arrays.

Example 1:

```
Input: nums = [1,1,2,1,1], k = 3
Output: 2
Explanation: The only sub-arrays with 3 odd numbers are [1,1,2,1] and [1,2,1,1].
```

Example 2:

```
Input: nums = [2,4,6], k = 1
Output: 0
Explanation: There are no odd numbers in the array.
```

Example 3:

```
Input: nums = [2,2,2,1,2,2,1,2,2,2], k = 2
Output: 16
```

Constraints:

- 1 <= nums.length <= 50000
- 1 <= nums[i] <= 10^5
- 1 <= k <= nums.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count subarrays with EXACTLY k odd numbers. Return the count. Right?
# Map each element to 1 (odd) or 0 (even) → reduce to Subarray Sum Equals K (#560)."
#
# "[1,1,2,1,1], k=3 → 2 ✓. [2,2,2,1,2,2,1,2,2,2], k=2 → 16 ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Enumerate every subarray with two loops; count how many have an odd number of 1s.
# Straightforward prefix/suffix parity check per window.
# Correct, but O(n^2) subarrays is too slow at n = 5*10^4.
# Time: O(n^2). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Prefix sum of odd indicators + hash map. Same pattern as #560.
# seed Counter at {0:1}. For each element, add num%2 to running sum.
# Count += prefix_count[sum - k]. Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,1,2,1,1], k=3:
# sums: 1,2,2,3,4. At sum=3: want=0→prefix[0]=1→count=1.
# At sum=4: want=1→prefix[1]=1→count=2. → 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Identical to #560 — name the connection explicitly.
# At-most-k version: sliding window counting odds in window, shrink when > k.
# Exactly k = atMost(k) - atMost(k-1).
# Given more time I'd ask: count subarrays with at most k odd numbers?
# Sliding window — shrink when odd count exceeds k, add right-left+1 each step."
```

Sorting

Round 2 · High · Medium · 3 questions

Sorting

#49 Group Anagrams

MED

Open on LeetCode

Array · Hash Table · String · Sorting

Lists: Grind 169

Problem

Given an array of strings strs, group the anagrams together. You can return the answer in any order.

Example 1:

Input: strs = ["eat","tea","tan","ate","nat","bat"]

Output: [["bat"],["nat","tan"],["ate","eat","tea"]]

Explanation:

- **There is no string in strs that can be rearranged to form "bat".**
- **The strings "nat" and "tan" are anagrams as they can be rearranged to form each other.**
- **The strings "ate", "eat", and "tea" are anagrams as they can be rearranged to form each other.**

Example 2:

Input: strs = [""]

Output: [[""]]

Example 3:

Input: strs = ["a"]

Output: [["a"]]

Constraints:

- **1 <= strs.length <= 104**
- **0 <= strs[i].length <= 100**
- **strs[i] consists of lowercase English letters.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Group strings that are anagrams. Return groups in any order.
# Same chars, same frequencies → anagrams. Empty string is its own group. Right?"
#
# "['eat','tea','tan','ate','nat','bat'] → 3 groups ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every pair of strings, sort both and check if the sorted forms match.
# If they match, put them in the same group; merge groups as we go.
# Correct, but comparing all pairs is expensive when n is large.
# Time: O(n^2 * m log m). Space: O(n * m).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Use a canonical key: sorted(word) as a tuple → same key for all anagrams.
# Group into a defaultdict. Time: O(n * m log m). Space: O(n * m).
# Alternative: 26-element frequency tuple → O(n * m), no sort cost."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# 'eat'→('a','e','t'). 'tea'→same. 'ate'→same. All go to one group.
# 'tan'→('a','n','t'). 'nat'→same.
# 'bat'→('a','b','t'). Alone. → 3 groups ✓
# [''] → {():['']}. → [['']] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * m log m). Space O(n * m).
# Frequency tuple alternative: O(n * m), avoids log m sort. For short words (typical)
# sorting is fine; for very long words, frequency tuple wins.
# Given more time I'd ask: Unicode input? sorted() handles it; frequency array breaks.
# Use Counter-based key: tuple(sorted(Counter(word).items()))."
```

Sorting

#56 Merge Intervals

MED

Open on LeetCode

Array · Sorting

Lists: Grind 169

Problem

Given an array of intervals where intervals[i] = [starti, endi], merge all overlapping intervals, and return an array of the non-overlapping intervals that cover all the intervals in the input.

Example 1:

```
Input: intervals = [[1,3],[2,6],[8,10],[15,18]]
Output: [[1,6],[8,10],[15,18]]
Explanation: Since intervals [1,3] and [2,6] overlap, merge them into [1,6].
```

Example 2:

```
Input: intervals = [[1,4],[4,5]]
Output: [[1,5]]
Explanation: Intervals [1,4] and [4,5] are considered overlapping.
```

Example 3:

```
Input: intervals = [[4,7],[1,4]]
Output: [[1,7]]
Explanation: Intervals [1,4] and [4,7] are considered overlapping.
```

Constraints:

- 1 <= intervals.length <= 104
- intervals[i].length == 2
- 0 <= starti <= endi <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Merge all overlapping intervals. Two intervals overlap if one starts before or when the other ends.
# [1,4] and [4,5] overlap at 4. Sort by start, then merge. Return non-overlapping result. Right?"
#
# "[[1,3],[2,6],[8,10],[15,18]] → [[1,6],[8,10],[15,18]] ✓ (1–3 and 2–6 overlap at 2–3)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Compare every pair of intervals; if they overlap, merge into one wider interval.
# Repeat until a full pass finds no new merges.
# Correct, but repeated pairwise merging can degrade toward O(n^2).
# Time: O(n^2) worst case. Space: O(n) output.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sort by start time — O(n log n). Then one pass:
# Keep a result list. For each interval, if it overlaps with the last result interval
# (last.end >= current.start), extend the last interval's end.
# Otherwise, append as a new separate interval.
# Time: O(n log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,3],[2,6],[8,10],[15,18]]: sorted same.
# merged=[[1,3]]. [2,6]: 3>=2→extend→[1,6]. [8,10]: 6<8→append. [15,18]: 10<15→append.
# → [[1,6],[8,10],[15,18]] ✓
# [[1,4],[4,5]]: 4>=4→[1,5] ✓
# [[4,7],[1,4]]: sorted→[[1,4],[4,7]]. 4>=4→[1,7] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n). Space O(n).
# The sort is the bottleneck — the merge pass itself is O(n).
# If the input were already sorted, this runs in O(n) total.
# Given more time I'd ask: how many meeting rooms needed?
# That's Meeting Rooms II (#253) — use a min-heap of end times."
```


Sorting

★ #1152 Analyze User Website Visit Pattern ★

MED

Open on LeetCode

Array · Hash Table · Sorting

Lists: ★ Premium | Premium 98

Problem

You are given two string arrays username and website and an integer array timestamp. All the given arrays are of the same length and the tuple [username[i], website[i], timestamp[i]] indicates that the user username[i] visited the website website[i] at time timestamp[i].

A pattern is a list of three websites (not necessarily distinct).

- For example, ["home", "away", "love"], ["leetcode", "love", "leetcode"], and ["luffy", "luffy", "luffy"] are all patterns.

The score of a pattern is the number of users that visited all the websites in the pattern in the same order they appeared in the pattern.

- For example, if the pattern is ["home", "away", "love"], the score is the number of users x such that x visited "home" then visited "away" and visited "love" after that.
- Similarly, if the pattern is ["leetcode", "love", "leetcode"], the score is the number of users x such that x visited "leetcode" then visited "love" and visited "leetcode" one more time after that.
- Also, if the pattern is ["luffy", "luffy", "luffy"], the score is the number of users x such that x visited "luffy" three different times at different timestamps.

Return the pattern with the largest score. If there is more than one pattern with the same largest score, return the lexicographically smallest such pattern.

Note that the websites in a pattern do not need to be visited contiguously, they only need to be visited in the order they appeared in the pattern.

Example 1:

```
Input: username = ["joe","joe","joe","james","james","james","james","mary","mary","mary"], timestamp = [1,2,3,4,5,6,7,8,9,10],
website = ["home","about","career","home","cart","maps","home","home","about","career"]
Output: ["home","about","career"]
Explanation: The tuples in this example are:
["joe","home",1],["joe","about",2],["joe","career",3],["james","home",4],["james","cart",5],["james","maps",6],["james","home",
7],["mary","home",8],["mary","about",9], and ["mary","career",10].
The pattern ("home", "about", "career") has score 2 (joe and mary).
The pattern ("home", "cart", "maps") has score 1 (james).
The pattern ("home", "cart", "home") has score 1 (james).
The pattern ("home", "maps", "home") has score 1 (james).
```

Example 2:

```
Input: username = ["ua","ua","ua","ub","ub","ub"], timestamp = [1,2,3,4,5,6], website = ["a","b","a","a","b","c"]
Output: ["a","b","a"]
```

Constraints:

- 3 <= username.length <= 50
- 1 <= username[i].length <= 10
- timestamp.length == username.length
- 1 <= timestamp[i] <= 109
- website.length == username.length
- 1 <= website[i].length <= 10
- username[i] and website[i] consist of lowercase English letters.
- It is guaranteed that there is at least one user who visited at least three websites.
- All the tuples [username[i], timestamp[i], website[i]] are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the 3-website pattern (subsequence, not necessarily consecutive) that the most DISTINCT users
# have visited in order. On tie, return lexicographically smallest. Right?
# Each user contributes at most 1 count per pattern regardless of how many times they visit."
#
# "joe visited home→about→career, james visited home→cart→maps→home, mary visited home→about→career.
# Pattern ('home','about','career') score=2 (joe, mary) → winner ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each user, enumerate all 3-website combinations in visit order.
# Build pattern tuple (a,b,c) and increment Counter across users.
# O(visit_length^3) per user — acceptable only for small lists.
# Time: O(U * V^3). Space: O(patterns).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Sort all visits by timestamp. Group by user into ordered website lists.
# For each user, generate all C(len,3) 3-subsequence combos — at most C(50,3)=19600.
# Use a set per user to avoid double-counting (one user = one score per pattern).
```

```
# Count across users with a global Counter.
# Return pattern with max score, lexicographically smallest on ties."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# After sorting by timestamp, user_sites = {joe:[home,about,career], james:[home,cart,maps,home], mary:[home,about,career]}.
# joe: combos of [home,about,career] = {(home,about,career)}.
# james: C(4,3)=4 combos. All unique.
# mary: {(home,about,career)}.
# pattern_score[(home,about,career)]=2 → winner ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n + U * C(K,3)) where U=users, K=max visits per user.
# With U<=50, K<=50: C(50,3)=19600 * 50 = 980000 — manageable.
# The per-user 'seen' set ensures one user contributes at most 1 to each pattern's score.
# Tie-breaking: max() with a compound key — score ascending, then lexicographic ascending.
# Given more time I'd ask: what if we want top-k patterns?
# Use a heap instead of max() — same O complexity."
```


Binary Search

Round 2 · High · Medium · 12 questions

Binary Search

#33 Search in Rotated Sorted Array

MED

Open on LeetCode

Array · Binary Search

Lists: Grind 169

Problem

There is an integer array nums sorted in ascending order (with distinct values).
Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].
Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.
You must write an algorithm with O(log n) runtime complexity.

Example 1:

```
Input: nums = [4,5,6,7,0,1,2], target = 0
Output: 4
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2], target = 3
Output: -1
```

Example 3:

```
Input: nums = [1], target = 0
Output: -1
```

Constraints:

- 1 <= nums.length <= 5000
- -104 <= nums[i] <= 104
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- -104 <= target <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array was sorted then rotated at some unknown pivot. Find target in O(log n). Right?
# All values are distinct. Return -1 if not found."
#
# "[4,5,6,7,0,1,2], target=0 → 4 ✓. target=3 → -1 ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Scan left to right: for each index i, check if nums[i] == target.
# If not found after one full pass, return -1.
# Correct and easy to reason about, but the problem asks for O(log n) on a rotated sorted array.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Standard binary search — but at each step, identify which half is sorted.
# If nums[left] <= nums[mid]: left half is sorted.
# If target in [nums[left], nums[mid]): search left.
# Else search right.
# Else: right half is sorted.
# If target in (nums[mid], nums[right]]: search right.
# Else search left.
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [4,5,6,7,0,1,2], target=0:
# l=0,r=6,mid=3(7). nums[0]=4<=7 → left sorted [4..7]. 0 in [4,7)? No → l=4.
# l=4,r=6,mid=5(1). nums[4]=0>1 → right sorted [1..2]. 0 in (1,2]? No → r=4.
# l=4,r=4,mid=4(0). 0==target → return 4 ✓
# target=3: never matches → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# The key: at least one half is always sorted in a rotated array — use that half
# to decide where to search. Checking nums[left] <= nums[mid] tells us which half.
# Given more time I'd ask: what if there are duplicates?
# That's #81 — when nums[left]==nums[mid], we can't determine which half is sorted.
# Worst case degrades to O(n) (just left++ to skip the duplicate)."
```


Binary Search

#34 Find First and Last Position of Element in Sorted Array

MED

Open on LeetCode

Array · Binary Search

Lists: Denny Zhang

Problem

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.
If target is not found in the array, return [-1, -1].
You must write an algorithm with O(log n) runtime complexity.

Example 1:

```
Input: nums = [5,7,7,8,8,10], target = 8
Output: [3,4]
```

Example 2:

```
Input: nums = [5,7,7,8,8,10], target = 6
Output: [-1,-1]
```

Example 3:

```
Input: nums = [], target = 0
Output: [-1,-1]
```

Constraints:

- 0 <= nums.length <= 105
- -109 <= nums[i] <= 109
- nums is a non-decreasing array.
- -109 <= target <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sorted array with duplicates. Find the first and last index of target. Return [-1,-1] if absent.
# Must be O(log n). Right?"
#
# "[5,7,7,8,8,10], target=8 → [3,4] ✓. target=6 → [-1,-1] ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Run standard binary search once to find any index of target.
# Then expand left and right from that index while values still equal target.
# Works, but we can do both boundaries in one modified binary search pass.
# Time: O(log n) for search + O(k) for expansion. Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Two binary searches:
# lower_bound: find first index where nums[i] >= target. If nums[result]==target → first position.
# upper_bound: find first index where nums[i] > target. Subtract 1 → last position.
# Both are standard binary search variants. Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[5,7,7,8,8,10], target=8:
# lower_bound: searching for first idx where nums[i]>=8. → index 3.
# nums[3]=8==target ✓. upper_bound: first idx where nums[i]>8. → index 5. 5-1=4.
# → [3,4] ✓
# target=6: lower_bound→2 (nums[2]=7>=6). nums[2]=7≠6 → [-1,-1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# lower_bound and upper_bound are the two fundamental binary search variants — know both.
# They correspond to bisect_left and bisect_right in Python's bisect module.
# The check 'nums[first] != target' after lower_bound handles the not-found case cleanly.
# Given more time I'd ask: what if we want the count of target occurrences?
# upper_bound() - lower_bound() in O(log n)."
```

Binary Search

#153 Find Minimum in Rotated Sorted Array

ME D

Open on LeetCode

Array · Binary Search

Lists: Grind 169

Problem

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become:

- [4,5,6,7,0,1,2] if it was rotated 4 times.
- [0,1,2,4,5,6,7] if it was rotated 7 times.

Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]].

Given the sorted rotated array nums of unique elements, return the minimum element of this array.

You must write an algorithm that runs in O(log n) time.

Example 1:

```
Input: nums = [3,4,5,1,2]
Output: 1
Explanation: The original array was [1,2,3,4,5] rotated 3 times.
```

Example 2:

```
Input: nums = [4,5,6,7,0,1,2]
Output: 0
Explanation: The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.
```

Example 3:

```
Input: nums = [11,13,15,17]
Output: 11
Explanation: The original array was [11,13,15,17] and it was rotated 4 times.
```

Constraints:

- n == nums.length
- 1 <= n <= 5000
- -5000 <= nums[i] <= 5000
- All the integers of nums are unique.
- nums is sorted and rotated between 1 and n times.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array sorted then rotated. Find the minimum in O(log n). All values unique. Right?
# The minimum is at the rotation point — where the array 'wraps'."
#
# "[3,4,5,1,2] → 1 ✓. [4,5,6,7,0,1,2] → 0 ✓. [11,13,15,17] → 11 (no rotation) ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Linear scan from index 0: track the smallest value seen so far.
# The first place where nums[i] > nums[i+1] marks the rotation pivot candidate.
# Return that minimum — correct on small inputs, but we can binary-search the pivot.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the rotation point.
# If nums[mid] > nums[right]: minimum is in the RIGHT half (mid+1 to right).
# Else: minimum is in the LEFT half including mid (left to mid).
# When left==right, we've found the minimum.
# Time: O(log n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,4,5,1,2]: l=0,r=4,mid=2(5). 5>nums[4]=2 → l=3.
# l=3,r=4,mid=3(1). 1<=nums[4]=2 → r=3. l=r=3 → return nums[3]=1 ✓
# [4,5,6,7,0,1,2]: l=0,r=6,mid=3(7). 7>nums[6]=2 → l=4.
# l=4,r=6,mid=5(1). 1<=2 → r=5. l=4,r=5,mid=4(0). 0<=1 → r=4. l=r=4 → 0 ✓
# [11,13,15,17]: l=0,r=3,mid=1(13). 13<=17 → r=1. l=0,r=1,mid=0(11). 11<=13 → r=0. → 11 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# The comparison is always against nums[right] (not nums[left]).
# If nums[mid] > nums[right], the right portion must be 'lower' — minimum is there.
# If nums[mid] <= nums[right], the right portion is in order — minimum is at mid or left.
# Given more time I'd ask: what if there are duplicates?
# When nums[mid]==nums[right] we can't decide — fall back to right-- (O(n) worst case)."
```


Binary Search

#300 Longest Increasing Subsequence

ME D

Open on LeetCode

Array · Binary Search · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array nums, return the length of the longest strictly increasing subsequence.

Example 1:

```
Input: nums = [10,9,2,5,3,7,101,18]
Output: 4
Explanation: The longest increasing subsequence is [2,3,7,101], therefore the length is 4.
```

Example 2:

```
Input: nums = [0,1,0,3,2,3]
Output: 4
```

Example 3:

```
Input: nums = [7,7,7,7,7,7,7]
Output: 1
```

Constraints:

- 1 <= nums.length <= 2500
- -104 <= nums[i] <= 104

Follow up: Can you come up with an algorithm that runs in O(n log(n)) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest strictly increasing subsequence (not necessarily contiguous).
# Return the length, not the subsequence itself. Right?
# Follow-up asks for O(n log n) — so know both approaches."
#
# "[10,9,2,5,3,7,101,18] → [2,3,7,101] length 4 ✓. [7,7,7,7] → 1 (strictly increasing) ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# dp[i] = length of LIS ending at i. For each i, scan all j < i with nums[j] < nums[i].
# dp[i] = max(dp[j]+1). Answer = max(dp).
# Correct O(n^2); patience sorting / binary search on tails gives O(n log n).
# Time: O(n^2). Space: O(n) dp array.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (O(n log n) patience sorting)
# "Maintain a 'tails' array where tails[i] is the smallest tail of all LIS of length i+1.
# For each number, binary search for the leftmost position in tails where tails[pos] >= num.
# If found, replace tails[pos] with num (smaller tail = more room to extend).
# If not found, append (the LIS can be extended by 1).
# len(tails) at the end is the LIS length.
#
# This is patience sorting — tails is always sorted so binary search works."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[10,9,2,5,3,7,101,18]:
# 10→tails=[10]. 9→replace 10→[9]. 2→replace 9→[2]. 5→append→[2,5].
# 3→replace 5→[2,3]. 7→append→[2,3,7]. 101→append→[2,3,7,101]. 18→replace 101→[2,3,7,18].
# len=4 ✓
# [7,7,7,7]: first 7→[7]. next 7→bisect_left([7],7)=0→replace→[7]. len=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(n²) DP: Time O(n²), Space O(n). Clean and easy to explain.
# O(n log n) patience sort: Time O(n log n), Space O(n).
# Note: tails does NOT store the actual LIS — it's a virtual 'pile' structure.
# To reconstruct the actual LIS, track parent pointers in the DP version.
# Given more time I'd ask: longest non-decreasing subsequence?
# Change bisect_left to bisect_right — allows equal elements to extend the sequence."
```

Binary Search

#362 Design Hit Counter

ME D

Open on LeetCode

Array · Hash Table · Binary Search · Design · Queue

Lists: Grind 169

Problem

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).
Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

- HitCounter() Initializes the object of the hit counter system.
- void hit(int timestamp) Records a hit that happened at timestamp (in seconds). Several hits may happen at the same timestamp.
- int getHits(int timestamp) Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).

Example 1:

```
Input
["HitCounter", "hit", "hit", "hit", "getHits", "hit", "getHits", "getHits"]
[[], [1], [2], [3], [4], [300], [300], [301]]
Output
[null, null, null, null, 3, null, 4, 3]

Explanation
HitCounter hitCounter = new HitCounter();
```

Constraints:

- 1 <= timestamp <= 2 * 109
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to hit and getHits.

Follow up: What if the number of hits per second could be huge? Does your design scale?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Record hits at timestamps and query: how many hits in the past 300 seconds?
# Calls are chronologically ordered (timestamps only increase). Right?
# getHits(t) returns hits with timestamp in (t-300, t] inclusive — past 5 minutes."
#
# "hit(1),hit(2),hit(3): getHits(4)=3. hit(300): getHits(300)=4. getHits(301)=3 (hit@1 expired)."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Append every hit timestamp to a list.
# getHits(t): count timestamps in [t-299, t] by scanning the whole list.
# Simple, but each query is O(total hits) - queue with expiry is better.
# Time: O(1) hit, O(n) getHits per query. Space: O(n) all timestamps.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Store timestamps in a deque. On getHits, pop from the left any timestamp <= t-300.
# Remaining deque size is the count. hit() is O(1). getHits() is amortised O(1).
#
# Follow-up (huge hits/second): store (timestamp, count) pairs instead of individual timestamps.
# Multiple hits at the same second are collapsed into one deque entry."

# PHASE 4 — CLEAN CODE

# Follow-up: grouped version for high-throughput

# PHASE 5 — TEST & DEBUG
# hit(1),hit(2),hit(3),getHits(4): deque=[1,2,3]. all>4-300=-296 → len=3 ✓
# hit(300),getHits(300): deque=[1,2,3,300]. 1<=0? No. len=4 ✓
# getHits(301): 1<=301-300=1 → popleft. deque=[2,3,300]. len=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "hit: O(1). getHits: O(k) amortised where k = expired entries (each entry popped at most once).
# Space: O(n) where n = total hits in the past 300 seconds.
# Binary search alternative: store sorted timestamps, binary search for t-300 cutoff — O(log n).
# The deque approach is simpler and amortised O(1) total.
# Grouped version handles millions of hits/second — key interview follow-up.
# Given more time I'd ask: what if calls aren't chronological?
# We'd need a sorted structure (sorted list or BST) and can't use the deque eviction trick."
```


Binary Search

#528 Random Pick with Weight

ME D

Open on LeetCode

Math · Binary Search · Prefix Sum · Randomized

Lists: Grind 169

Problem

You are given a 0-indexed array of positive integers w where w[i] describes the weight of the ith index.
You need to implement the function pickIndex(), which randomly picks an index in the range [0, w.length - 1] (inclusive) and returns it. The probability of picking an index i is w[i] / sum(w).

- For example, if w = [1, 3], the probability of picking index 0 is 1 / (1 + 3) = 0.25 (i.e., 25%), and the probability of picking index 1 is 3 / (1 + 3) = 0.75 (i.e., 75%).

Example 1:

```
Input
["Solution","pickIndex"]
[[[1]],[]]
Output
[null,0]

Explanation
Solution solution = new Solution([1]);
```

Example 2:

```
Input
["Solution","pickIndex","pickIndex","pickIndex","pickIndex","pickIndex"]
[[[1,3]],[],[],[],[],[]]
Output
[null,1,1,1,1,0]

Explanation
Solution solution = new Solution([1, 3]);
```

Constraints:

- 1 <= w.length <= 104
- 1 <= w[i] <= 105
- pickIndex will be called at most 104 times.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick a random index where index i is chosen with probability w[i]/sum(w).
# pickIndex() is called up to 10^4 times — so init can be O(n) but pick must be fast. Right?
# w=[1,3]: P(0)=25%, P(1)=75%. The actual randomness in the example output is fine."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Expand weights into a list where index i appears w[i] times, pick random index.
# Correct distribution but total list size can be sum(w) up to 1e9 - impossible.
# Prefix sums + binary search on total weight fixes space.
# Time: O(sum(w)) build. Space: O(sum(w)) - infeasible.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Build a prefix sum array. Prefix[i] = w[0]+...+w[i].
# Pick a random float r in [0, total). Binary search for the leftmost prefix sum > r.
# That index is our answer — the probability of landing in bucket i equals w[i]/total.
# Time: O(n) init. O(log n) per pick. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# w=[1,3]: prefix=[1,4], total=4.
# r in [0,1) → prefix[0]=1>r → lo=hi=0 → return 0 (25% of the time).
# r in [1,4) → prefix[0]=1<=r → lo=1 → return 1 (75% of the time). ✓
# w=[1]: prefix=[1]. Any r in [0,1) → hi=lo=0 → return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Init O(n). pickIndex O(log n). Space O(n).
# The prefix sum maps weights to contiguous ranges on [0, total).
# A random point in that range lands in bucket i with probability w[i]/total.
# bisect.bisect_right(self.prefix, r) is a one-liner alternative using Python's bisect module.
# Given more time I'd ask: what if weights change dynamically?
# We'd need a Fenwick tree (BIT) — O(log n) updates AND O(log n) queries."
```

Binary Search

#852 Peak Index in a Mountain Array

MED

Open on LeetCode

Array · Binary Search
Lists: Denny Zhang

Problem

You are given an integer mountain array arr of length n where the values increase to a peak element and then decrease.
Return the index of the peak element.
Your task is to solve it in O(log(n)) time complexity.
Example 1:
Input: arr = [0,1,0]
Output: 1
Example 2:
Input: arr = [0,2,1,0]
Output: 1
Example 3:
Input: arr = [0,10,5,2]
Output: 1
Constraints:

- 3 <= arr.length <= 105
- 0 <= arr[i] <= 106
- arr is guaranteed to be a mountain array.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array guaranteed to have exactly one peak (mountain shape: strictly increasing then decreasing).
# Return the index of the peak in O(log n). Right?"
#
# "[0,1,0] → 1. [0,2,1,0] → 1. [0,10,5,2] → 1. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Linear scan from index 1: first i where arr[i] > arr[i+1] is the peak index.
# Mountain array guarantees exactly one peak; scan stops early.
# O(n) acceptable but binary search on the rising slope is O(log n).
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search: at mid, compare arr[mid] with arr[mid+1].
# If arr[mid] < arr[mid+1]: we're on the ascending slope → peak is to the right → left = mid+1.
# Else: we're on or past the peak → peak is at mid or to the left → right = mid.
# When left==right: that's the peak. O(log n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [0,1,0]: l=0,r=2,mid=1. arr[1]=1>arr[2]=0 → right=1. l=0,r=1,mid=0. arr[0]=0<arr[1]=1 → left=1. l=r=1 → 1 ✓
# [0,10,5,2]: l=0,r=3,mid=1. arr[1]=10>arr[2]=5 → right=1. l=0,r=1,mid=0. arr[0]<arr[1] → left=1. → 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n). Space O(1).
# The comparison arr[mid] vs arr[mid+1] acts as a gradient — positive means 'go right', negative means 'go left'.
# This is the same logic as Find Peak Element (#162) — the general version where the array isn't guaranteed
# to be a full mountain. Mention the connection.
# Given more time I'd ask: what if there are multiple peaks?
# Same algorithm finds one peak. For all peaks, we'd need a linear scan."
```


Binary Search

#981 Time Based Key-Value Store

MED

Open on LeetCode

Hash Table · String · Binary Search · Design
Lists: Grind 169

Problem

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.
Implement the TimeMap class:

- TimeMap() Initializes the object of the data structure.
- void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.
- String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp_prev <= timestamp. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If there are no values, it returns "".

Example 1:

```
Input
["TimeMap", "set", "get", "get", "set", "get", "get"]
[[], ["foo", "bar", 1], ["foo", 1], ["foo", 3], ["foo", "bar2", 4], ["foo", 4], ["foo", 5]]
Output
[null, null, "bar", "bar", null, "bar2", "bar2"]

Explanation
TimeMap timeMap = new TimeMap();
```

Constraints:

- 1 <= key.length, value.length <= 100
- key and value consist of lowercase English letters and digits.
- 1 <= timestamp <= 107
- All the timestamps timestamp of set are strictly increasing.
- At most 2 * 105 calls will be made to set and get.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Key-value store where each key can have multiple values at different timestamps.
# set(key, value, ts) stores the entry. get(key, ts) returns the value with the
# largest timestamp <= ts. Return '' if none exists. Right?
# set timestamps are strictly increasing — so we can binary search the history list."
#
# "set('foo','bar',1). get('foo',3) → 'bar' (no ts<=3 except 1). ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Store all (timestamp, value) pairs per key in a list on set.
# get(key, ts): scan backwards for largest timestamp <= ts.
# Works but each get is O(number of entries for that key).
# Time: O(1) set, O(k) get per key history. Space: O(total entries).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Since set timestamps are strictly increasing, each key's list is already sorted by timestamp.
# get() binary searches for the rightmost timestamp <= query timestamp.
# set: O(1). get: O(log n). Space: O(total entries)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# set('foo','bar',1): store['foo']=[(1,'bar')].
# get('foo',1): entries=[(1,'bar')]. mid=0, ts=1<=1 → result='bar', lo=1. lo>hi → return 'bar' ✓
# get('foo',3): mid=0, ts=1<=3 → result='bar'. lo=1>hi=0 → return 'bar' ✓
# set('foo','bar2',4): store['foo']=[(1,'bar'),(4,'bar2')].
# get('foo',4): mid=0(1<=4)→result='bar',lo=1. mid=1(4<=4)→result='bar2',lo=2. return 'bar2' ✓
# get('foo',5): same → 'bar2' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "set O(1). get O(log n). Space O(total entries).
# The 'find rightmost ts <= query' is a standard upper_bound − 1 binary search pattern.
# Timestamps strictly increasing → no sorting needed — the list maintains order automatically.
# bisect.bisect_right alternative: pos = bisect.bisect_right(entries, (timestamp, chr(127))) − 1
# (appending chr(127) to go past any value at that exact timestamp).
# Given more time I'd ask: what if set timestamps aren't guaranteed to be increasing?
# We'd need to sort the list on insert or use a sorted data structure."
```

Binary Search

#1011 Capacity to Ship Packages Within D Days

ME
D

Open on LeetCode

Array · Binary Search

Lists: Denny Zhang

Problem

A conveyor belt has packages that must be shipped from one port to another within days days.
The ith package on the conveyor belt has a weight of weights[i]. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.
Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1:

```
Input: weights = [1,2,3,4,5,6,7,8,9,10], days = 5
Output: 15
Explanation: A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:
1st day: 1, 2, 3, 4, 5
2nd day: 6, 7
3rd day: 8
4th day: 9
5th day: 10
```

Example 2:

```
Input: weights = [3,2,2,4,1,4], days = 3
Output: 6
Explanation: A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:
1st day: 3, 2
2nd day: 2, 4
3rd day: 1, 4
```

Example 3:

```
Input: weights = [1,2,3,1,1], days = 4
Output: 3
Explanation:
1st day: 1
2nd day: 2
3rd day: 3
4th day: 1, 1
```

Constraints:

- 1 <= days <= weights.length <= 5 * 10^4
- 1 <= weights[i] <= 500

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the minimum ship capacity that lets us ship all packages in at most D days.
# Packages must be shipped in order — we can't reorder them. Right?
# Capacity must be at least max(weights) (to handle any single package).
# Capacity at most sum(weights) (ship everything in one day)."
#
# "weights=[1..10], days=5 → 15 ✓ (greedy daily assignment: 1–5, 6–7, 8, 9, 10)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try every candidate ship capacity from max(weights) up to sum(weights).
# For each capacity, greedily pack days and count days needed.
# Correct but linear search on capacity range is O(sum * n).
# Time: O(sum(weights) * n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the answer (capacity).
# The feasibility function 'can_ship(capacity)' is monotone: if C works, C+1 also works.
# So binary search for the leftmost capacity where can_ship returns True.
# can_ship: greedily fill each day until the load would exceed capacity, then start a new day.
# Count days needed. Return days <= D.
# Time: O(n log(sum(weights))). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# weights=[1..10], days=5. lo=10, hi=55.
# mid=32: can_ship(32)→1+2+3+4+5=15≤32(day1), 6+7=13≤32(day2), 8≤32(day3), 9≤32(day4), 10≤32(day5). 5 days ✓ → hi=32.
# ... binary search converges to 15.
# can_ship(15): 1+2+3+4+5=15(day1), 6+7=13+8>15→day2=6+7, 8→day3, 9→day4, 10→day5. 5 days ✓. → 15 ✓
# can_ship(14): 1+2+3+4=10, +5=15>14→day2=5+6=11, +7>14→day3=7+8>14→day4=8, 9→day5, 10→day6. 6>5 ✗.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log(sum)). Space O(1).
```



```
# This is 'binary search on the answer' — a powerful pattern where the answer space is
# a range of integers and feasibility is monotone.
# The feasibility check is a greedy simulation — always load greedily to avoid underusing capacity.
# Same pattern appears in: Koko Eating Bananas (#875), Split Array Largest [redacted] (#410).
# Given more time I'd ask: what if packages can be reordered?
# Then we'd try to optimise placement — different problem, potentially NP-hard."
```

Binary Search

⋆ #1060 Missing Element in Sorted Array ⋆

MED

Open on LeetCode

Array · Binary Search

Lists: ⋆ Premium | Premium 98

Problem

Given an integer array nums which is sorted in ascending order and all of its elements are unique and given also an integer k, return the kth missing number starting from the leftmost number of the array.

Example 1:

```
Input: nums = [4,7,9,10], k = 1
Output: 5
Explanation: The first missing number is 5.
```

Example 2:

```
Input: nums = [4,7,9,10], k = 3
Output: 8
Explanation: The missing numbers are [5,6,8,...], hence the third missing number is 8.
```

Example 3:

```
Input: nums = [1,2,4], k = 3
Output: 6
Explanation: The missing numbers are [3,5,6,7,...], hence the third missing number is 6.
```

Constraints:

- 1 <= nums.length <= 5 * 104
- 1 <= nums[i] <= 107
- nums is sorted in ascending order, and all the elements are unique.
- 1 <= k <= 108

Follow up: Can you find a logarithmic time complexity (i.e., O(log(n))) solution?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sorted unique integers. Find the k-th missing integer starting from nums[0].
# Missing count at index i = (nums[i] - nums[0]) - i (how many numbers should exist but don't).
# Follow-up asks for O(log n). Right?"
#
# "[4,7,9,10], k=1: missing = [5,6,8,...]. 1st missing = 5 ✓
# [4,7,9,10], k=3: [5,6,8,...]. 3rd = 8 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Walk nums in order maintaining expected next integer starting at 0.
# Increment expected whenever nums[i] == expected; else expected stays.
# When expected reaches k, return k - correct but scans whole array.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (O(log n))
# "Define missing(i) = numbers missing from nums[0] to nums[i] = (nums[i] - nums[0]) - i.
# Binary search for the rightmost index where missing(i) < k.
# Answer = nums[right] + (k - missing(right)).
# If k > missing(last), answer is past the end of the array: nums[-1] + (k - missing(n-1))."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[4,7,9,10], k=1:
# missing: [0]=0, [1]=2, [2]=3, [3]=4.
# k=1<=missing(3)=4. Binary search: lo=0,hi=3,mid=1. missing(1)=2>=1→hi=1.
# lo=0,hi=1,mid=0. missing(0)=0<1→lo=1. lo=hi=1.
# return nums[0]+(1-missing(0))=4+(1-0)=5 ✓
# nums=[4,7,9,10], k=3:
# missing(3)=4>=3. Binary search → lo lands at 2. missing(1)=2<3→lo=2. lo=hi=2.
# return nums[1]+(3-missing(1))=7+(3-2)=8 ✓
# k=5: missing(3)=4<5 → return nums[3]+(5-4)=10+1=11 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(log n) with binary search. O(1) space.
# The missing(i) formula is the key: (nums[i]-nums[0])-i counts how many integers
# in [nums[0], nums[i]] are absent from the array.
# The past-the-end check handles k > total missing numbers in the array gracefully.
# Given more time I'd ask: what if there are duplicate values in nums?
# missing(i) would overcount — we'd need to adjust for duplicates."
```


Binary Search

#1062 Longest Repeating Substring

MED

Open on LeetCode

String · Binary Search · Dynamic Programming · Rolling Hash · Suffix Array · Hash Function

Lists: Denny Zhang

Problem

Given a string s, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1:

```
Input: s = "abcd"
Output: 0
Explanation: There is no repeating substring.
```

Example 2:

```
Input: s = "abbaba"
Output: 2
Explanation: The longest repeating substrings are "ab" and "ba", each of which occurs twice.
```

Example 3:

```
Input: s = "aabcaabdaab"
Output: 3
Explanation: The longest repeating substring is "aab", which occurs 3 times.
```

Constraints:

- 1 <= s.length <= 2000
- s consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest substring that appears at least twice in s.
# The two occurrences can overlap. Return 0 if no repeating substring. Right?"
#
# "'abcd'→0. 'abbaba'→2 ('ab','ba'). 'aabcaabdaab'→3 ('aab')."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each length L from n-1 down to 1, collect all substrings of length L in a set.
# First L where a duplicate appears is the answer.
# O(n^2) substrings per L - binary search on L with rolling hash helps.
# Time: O(n^2) to O(n^3) naive. Space: O(n^2) set.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DP approach — simpler to explain)
# "DP: dp[i][j] = length of longest common suffix of s[:i] and s[:j], where i ≠ j.
# dp[i][j] = dp[i-1][j-1] + 1 if s[i-1]==s[j-1] and i≠j, else 0.
# Track the global max. O(n²) time, O(n²) space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abbaba':
# i=1,j=2: 'a'=='b'? No. ... i=1,j=4: 'a'=='a'? Yes→dp[1][4]=1, best=1.
# i=2,j=5: 'b'=='b'? Yes→dp[2][5]=dp[1][4]+1=2, best=2.
# → 2 ✓ (substring 'ab' starting at 0 and 3)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(n²). Acceptable for n<=2000 (4M cells).
# Binary search + hashing alternative: O(n log n) time, O(n) space — faster but more complex.
# Given more time I'd ask: return the actual substring, not just the length?
# Track (i,j) when best is updated, then s[i-best:i]."
```

Binary Search

★ #1533 Find the Index of the Large Integer ★

MED

Open on LeetCode

Array · Binary Search · Interactive

Lists: ★ Premium | Premium 98

Problem

We have an integer array arr, where all the integers in arr are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API ArrayReader which have the following functions:

- int compareSub(int l, int r, int x, int y): where 0 <= l, r, x, y < ArrayReader.length(), l <= r and x <= y. The function compares the sum of sub-array arr[l..r] with the sum of the sub-array arr[x..y] and returns: 1 if arr[l]+arr[l+1]+...+arr[r] > arr[x]+arr[x+1]+...+arr[y].
- 0 if arr[l]+arr[l+1]+...+arr[r] == arr[x]+arr[x+1]+...+arr[y].
- -1 if arr[l]+arr[l+1]+...+arr[r] < arr[x]+arr[x+1]+...+arr[y].

You are allowed to call compareSub() 20 times at most. You can assume both functions work in O(1) time.

Return the index of the array arr which has the largest integer.

Example 1:

```
Input: arr = [7,7,7,7,10,7,7,7]
Output: 4
Explanation: The following calls to the API
reader.compareSub(0, 0, 1, 1) // returns 0 this is a query comparing the sub-array (0, 0) with the sub array (1, 1), (i.e.
compares arr[0] with arr[1]).
Thus we know that arr[0] and arr[1] doesn't contain the largest element.
reader.compareSub(2, 2, 3, 3) // returns 0, we can exclude arr[2] and arr[3].
reader.compareSub(4, 4, 5, 5) // returns 1, thus for sure arr[4] is the largest element in the array.
Notice that we made only 3 calls, so the answer is valid.
```

Example 2:

```
Input: nums = [6,6,12]
Output: 2
```

Constraints:

- 2 <= arr.length <= 5 * 105
- 1 <= arr[i] <= 100
- All elements of arr are equal except for one element which is larger than all other elements.

Follow up:

- What if there are two numbers in arr that are bigger than all other numbers?
- What if there is one number that is bigger than other numbers and one number that is smaller than other numbers?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "One element is strictly larger than all others. Find its index using the compareSub API.
# compareSub(l,r,x,y) compares sum(arr[l..r]) with sum(arr[x..y]): returns 1,-1,0.
# At most 20 calls — so O(log n) calls are required. Right?"
#
# "[7,7,7,7,10,7,7,7]: 3 compareSub calls suffice to find index 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Linear scan: compare nums[i] with nums[i+1]; first unequal pair reveals larger half.
# Correct O(n) but problem expects O(log n) via binary search on the doubled array.
# Time: O(n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search: at each step, compare the left half against the right half of the range.
# If left > right: large integer is in the left half → search left.
# If right > left: search right.
# If equal: the large integer is the MIDDLE element (if odd-length range, middle was excluded).
# Halve the range each step → O(log n) calls."

# PHASE 4 — CLEAN CODE
            # Compare left half [lo..mid] with right half of equal size

# PHASE 5 — TEST & DEBUG
# arr=[7,7,7,7,10,7,7,7]: lo=0,hi=7.
# mid=3. Compare [0..3] vs [4..7]. Both have 4 elements. Sum=[28] vs [31]. -1 → lo=4.
# lo=4,hi=7,mid=5. Compare [4..5] vs [6..7]. [10+7=17] vs [7+7=14]. 1 → hi=5.
# lo=4,hi=5,mid=4. Compare [4..4] vs [5..5]. [10] vs [7]. 1 → hi=4. lo=hi=4 → return 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(log n) compareSub calls. Well within the 20-call limit for n=5*10^5 (log₂(500000)≈19).
# The equal-sum case (odd-length range) means the middle element was the large one —
# both halves have equal elements so the excluded middle is the outlier.
# Follow-up: two large numbers? One comparison no longer definitively points to one half.
```



```
# We'd need to track which half has MORE excess — more complex."
```

Matrix

Round 2 · High · Medium · 14 questions

Matrix

#36 Valid Sudoku

MED

Open on LeetCode

Array · Hash Table · Matrix

Lists: Grind 169

Problem

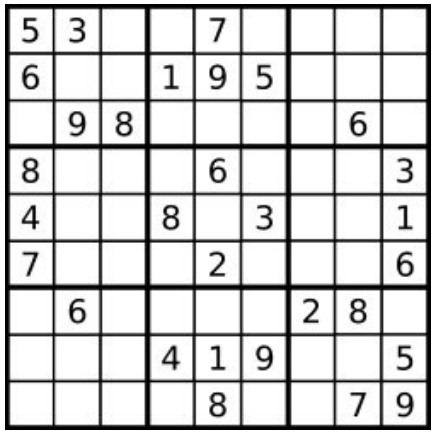
Determine if a 9 x 9 Sudoku board is valid. Only the filled cells need to be validated according to the following rules:

1. Each row must contain the digits 1-9 without repetition.
2. Each column must contain the digits 1-9 without repetition.
3. Each of the nine 3 x 3 sub-boxes of the grid must contain the digits 1-9 without repetition.

Note:

- A Sudoku board (partially filled) could be valid but is not necessarily solvable.
- Only the filled cells need to be validated according to the mentioned rules.

Example 1:




```
Input: board =
[["5","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

Example 2:

```
Input: board =
[["8","3",".",".","7",".",".",".","."]
,["6",".",".","1","9","5",".",".","."]
,[".","9","8",".",".",".",".","6","."]
,["8",".",".",".","6",".",".",".","3"]
,["4",".",".","8",".","3",".",".","1"]
,["7",".",".",".","2",".",".",".","6"]
,[".","6",".",".",".",".","2","8","."]
,[".",".",".","4","1","9",".",".","5"]
,[".",".",".",".","8",".",".","7","9"]]
```

Constraints:

- board.length == 9
- board[i].length == 9
- board[i][j] is a digit 1-9 or '.'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Check if a partially filled 9x9 sudoku board is valid — no repeats in any row,
# column, or 3x3 box. Ignore empty cells ('.'). Right?
# A valid board doesn't have to be solvable — just no current conflicts."
#
# "The two examples differ only in one cell: '8' appears twice in the top-left 3x3 box → false ✓"
```

```
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each of 9 rows, 9 columns, and 9 boxes, collect digits in a set and check size==9.
# 27 independent validity checks - straightforward and easy to explain.
# Time: O(1) - fixed 81 cells. Space: O(1) - 9 sets of size 9.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Single pass: use three 2D boolean arrays - rows[r][d], cols[c][d], boxes[r//3][c//3][d].
# For each filled cell, check all three. If any is already marked, return False.
# Otherwise mark all three. O(81) = O(1) for fixed 9x9."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# Cell (0,0)='5': d=4. box_id=0. Mark rows[0][4]=cols[0][4]=boxes[0][4]=True.
# Cell (3,0)='8': d=7. box_id=0. rows[3][7]? No. boxes[0][7]? No. Mark.
# In example 2: cell (0,0)='8' and cell (3,0)='8': both have box_id=0, d=7.
# Second '8' at (3,0): boxes[0][7] already True → return False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(81)=O(1). Space O(81)=O(1) - fixed 9x9 board.
# The box_id formula (r//3)*3+(c//3) maps each of the 9 boxes to 0-8. Know this cold.
# Alternative: use a set of tuples like (r,'row',d), (c,'col',d), (box_id,'box',d) - one set.
# Given more time I'd ask: implement a Sudoku solver?
# That's backtracking - for each empty cell, try 1-9, recurse, backtrack on failure."
```


Matrix

#48 Rotate Image

ME
D

Open on LeetCode

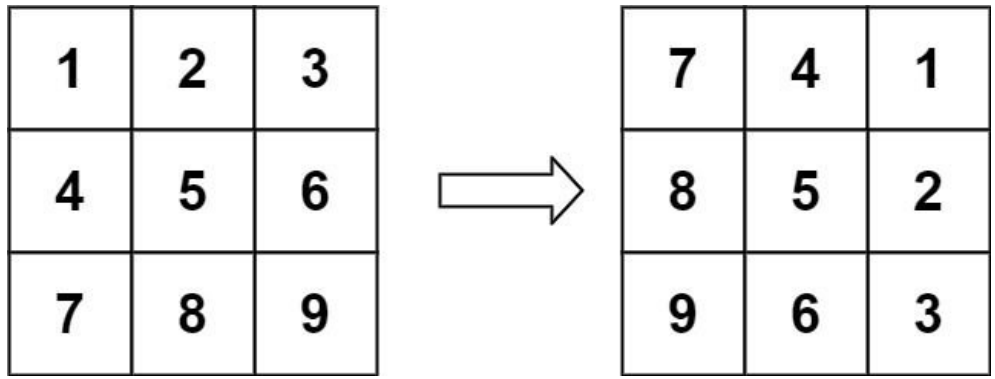
Array · Math · Matrix

Lists: Grind 169 | CodeSignal

Problem

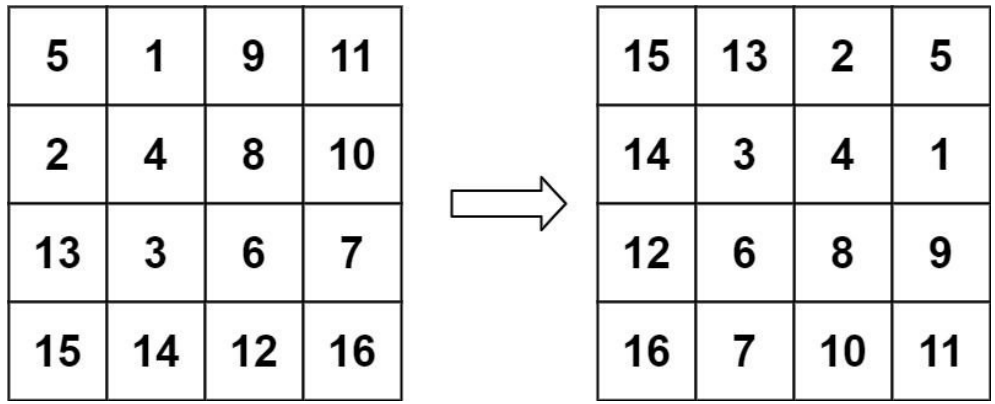
You are given an n x n 2D matrix representing an image, rotate the image by 90 degrees (clockwise).
You have to rotate the image in-place, which means you have to modify the input 2D matrix directly. DO NOT allocate another 2D matrix and do the rotation.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [[7,4,1],[8,5,2],[9,6,3]]
```

Example 2:




```
Input: matrix = [[5,1,9,11],[2,4,8,10],[13,3,6,7],[15,14,12,16]]
Output: [[15,13,2,5],[14,3,4,1],[12,6,8,9],[16,7,10,11]]
```

Constraints:

- **n == matrix.length == matrix[i].length**
- **1 <= n <= 20**
- **-1000 <= matrix[i][j] <= 1000**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
```

```
# "Rotate an n x n matrix 90 degrees clockwise in-place. No extra matrix allowed. Right?
# After rotation: element at (r,c) moves to (c, n-1-r)."
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [[7,4,1],[8,5,2],[9,6,3]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Allocate a new n x n matrix; for each (r,c) set new[c][n-1-r] = matrix[r][c].
# Classic 90-degree rotation formula - clear and correct.
# Uses O(n^2) extra memory; we can rotate in-place layer by layer.
# Time: O(n^2). Space: O(n^2) new matrix.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (two-step in-place)
# "Two operations, each reversible and composable:
# Step 1: Transpose (swap matrix[r][c] with matrix[c][r]).
# Step 2: Reverse each row.
# Together these produce a 90° clockwise rotation. O(n²) time, O(1) space.
# Verify: (r,c) →transpose→ (c,r) →reverse row→ (c, n-1-r). Correct!"

# PHASE 4 — CLEAN CODE
        # Step 1: transpose
        # Step 2: reverse each row

# PHASE 5 — TEST & DEBUG
# [[1,2,3],[4,5,6],[7,8,9]]:
# Transpose: [[1,4,7],[2,5,8],[3,6,9]].
# Reverse rows: [[7,4,1],[8,5,2],[9,6,3]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(1) in-place.
# Counter-clockwise 90°: reverse each row first, then transpose.
# 180°: transpose then transpose (just reverse all rows, then reverse their order).
# The two-step approach is elegant — each step is simple and reversible.
# Given more time I'd ask: rotate a non-square matrix?
# No longer possible in-place with the same dimensions — need a new m×n matrix."
```


Matrix

#54 Spiral Matrix

ME
D

Open on LeetCode

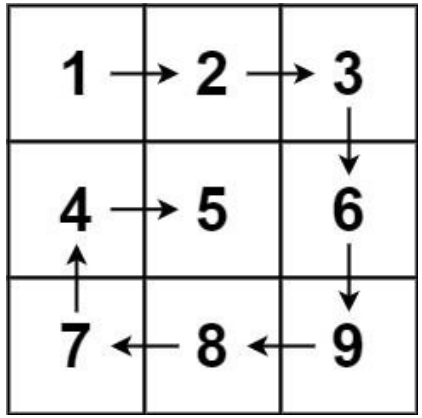
Array · Matrix · Simulation

Lists: Grind 169 | CodeSignal

Problem

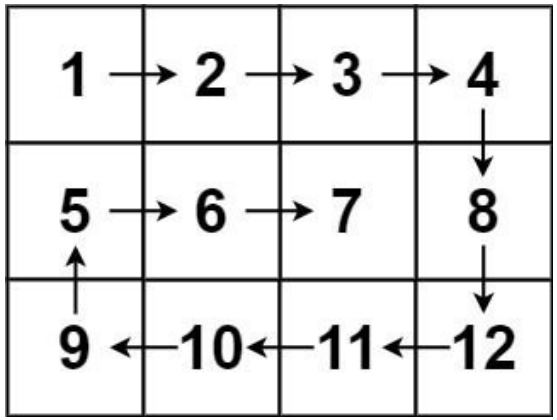
Given an m x n matrix, return all elements of the matrix in spiral order.

Example 1:




```
Input: matrix = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,3,6,9,8,7,4,5]
```

Example 2:




```
Input: matrix = [[1,2,3,4],[5,6,7,8],[9,10,11,12]]
Output: [1,2,3,4,8,12,11,10,9,5,6,7]
```

Constraints:

- **m == matrix.length**
- **n == matrix[i].length**
- **1 <= m, n <= 10**
- **-100 <= matrix[i][j] <= 100**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return all matrix elements in spiral order: right along top, down along right side,
# left along bottom, up along left side — then repeat for the inner rectangle. Right?
# Works for non-square matrices."
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,3,6,9,8,7,4,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Maintain top/bottom/left/right bounds; walk right, down, left, up while unvisited.
# Mark visited cells in a boolean matrix to know when to turn.
# Works, but the visited matrix costs O(m*n) extra space.
# Time: O(m * n). Space: O(m * n) visited.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (layer boundaries)
# "Track four boundaries: top, bottom, left, right.
# Traverse right along top (then top++), down along right (then right--),
# left along bottom (then bottom--), up along left (then left++).
# Stop when top>bottom or left>right. O(m*n) time, O(1) extra space."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,2,3],[4,5,6],[7,8,9]]: top=0,bot=2,l=0,r=2.
# Right: 1,2,3. top=1.
# Down: 6,9. right=1.
# Left (top<=bot): 8,7. bot=1.
# Up (left<=right): 4. left=1.
# top=1,bot=1,l=1,r=1.
# Right: 5. top=2. top>bot → stop.
# → [1,2,3,6,9,8,7,4,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1) extra.
# The boundary guards (if top<=bottom / if left<=right) prevent double-counting
# the single middle row or column of non-square matrices.
# Given more time I'd ask: Spiral Matrix II (#59) — fill instead of read.
# Same boundary logic, write numbers 1..N instead of appending."
```


Matrix

#59 Spiral Matrix II

ME
D

Open on LeetCode

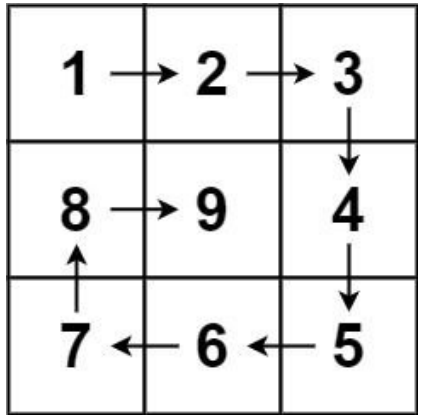
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given a positive integer n, generate an n x n matrix filled with elements from 1 to n2 in spiral order.

Example 1:




```
Input: n = 3
Output: [[1,2,3],[8,9,4],[7,6,5]]
```

Example 2:

```
Input: n = 1
Output: [[1]]
```

Constraints:

- **1 <= n <= 20**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Generate an n×n matrix filled with 1 to n² in spiral order. Return the matrix. Right?"
#
# "n=3 → [[1,2,3],[8,9,4],[7,6,5]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Fill numbers 1..n^2 using direction arrays (right, down, left, up) and a visited grid.
# Turn when hitting boundary or a filled cell.
# Straightforward simulation with O(n^2) extra visited space.
# Time: O(n^2). Space: O(n^2) visited.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Identical boundary approach to #54, but write consecutive integers instead of reading.
# Initialize n×n matrix of zeros, fill with num=1..n². O(n²) time, O(1) extra."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=3: fills 1→2→3 (top row), 4 (right col), 5→6 (bottom), 7 (left col), 9 (center).
# → [[1,2,3],[8,9,4],[7,6,5]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²). Space O(1) extra — output matrix doesn't count.
# This is the write-version of Spiral Matrix (#54) — exact same boundary logic.
# Given more time I'd ask: generate a spiral starting from the center?
# Reverse the boundary logic — start from the middle, expand outward."
```

Matrix

#64 Minimum Path Sum

MED

Open on LeetCode

Array · Dynamic Programming · Matrix

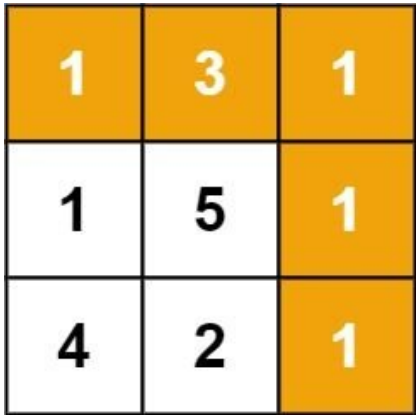
Lists: Denny Zhang

Problem

Given a m x n grid filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

Example 1:




```
Input: grid = [[1,3,1],[1,5,1],[4,2,1]]
Output: 7
Explanation: Because the path 1 → 3 → 1 → 1 → 1 minimizes the sum.
```

Example 2:

```
Input: grid = [[1,2,3],[4,5,6]]
Output: 12
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 200
- 0 <= grid[i][j] <= 200

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Move only right or down. Find the path from top-left to bottom-right with minimum sum. Right?
# Can only move right or down — no backtracking."
#
# "[[1,3,1],[1,5,1],[4,2,1]]: path 1→3→1→1→1=7 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS from (0,0): at each cell try moving right and down, take min of two paths.
# Memoize (r,c) to avoid recomputing subpaths — classic top-down DP.
# Without memo this blows up exponentially; memo makes it O(m*n).
# Time: O(m * n) with memo. Space: O(m * n) memo + stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (in-place DP)
# "dp[i][j] = min path sum to reach (i,j).
# dp[i][j] = grid[i][j] + min(dp[i-1][j], dp[i][j-1]).
# First row: only right. First col: only down.
# Modify grid in-place — no extra space. O(m*n) time, O(1) extra."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,3,1],[1,5,1],[4,2,1]]:
# Row 0: [1,4,5]. Col 0: [1,2,6]. (1,1)=5+min(4,2)=7. (1,2)=1+min(7,5)=6.
```



```
# (2,1)=2+min(7,6)=8. (2,2)=1+min(6,8)=7. → 7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1) — modifies grid in place.
# If we can't modify the input, use a 1D rolling array: O(n) extra space.
# Given more time I'd ask: what if we can also move up or left?
# That breaks the DP recurrence — shortest path with general movement is Dijkstra's."
```

Matrix

#73 Set Matrix Zeroes

MED

Open on LeetCode

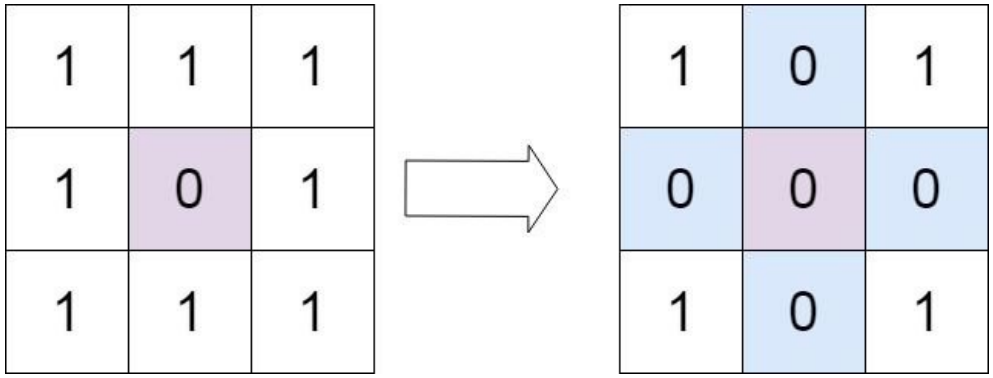
Array · Hash Table · Matrix

Lists: Grind 169

Problem

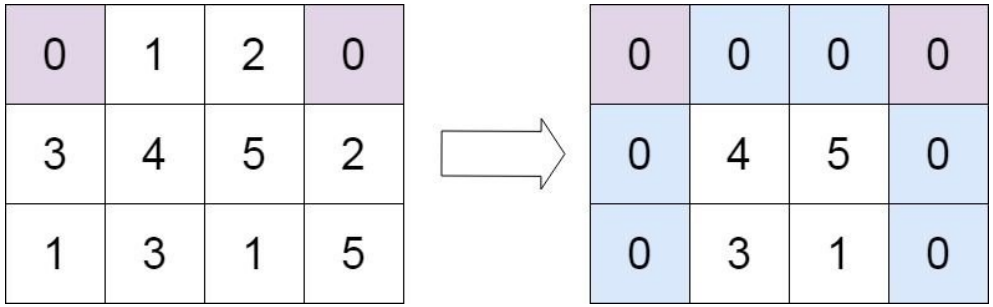
Given an m x n integer matrix matrix, if an element is 0, set its entire row and column to 0's.
You must do it in place.

Example 1:




```
Input: matrix = [[1,1,1],[1,0,1],[1,1,1]]
Output: [[1,0,1],[0,0,0],[1,0,1]]
```

Example 2:




```
Input: matrix = [[0,1,2,0],[3,4,5,2],[1,3,1,5]]
Output: [[0,0,0,0],[0,4,5,0],[0,3,1,0]]
```

Constraints:

- m == matrix.length
- n == matrix[0].length
- 1 <= m, n <= 200
- -231 <= matrix[i][j] <= 231 - 1

Follow up:

- A straightforward solution using O(mn) space is probably a bad idea.
- A simple improvement uses O(m + n) space, but still not the best solution.
- Could you devise a constant space solution?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "If a cell is 0, zero out its entire row and column. Do it in-place. Right?
```



```
# We must use the ORIGINAL zeros — not zeros we set during the process."
#
# "[[1,0,1],[1,1,1],[1,1,1]] → [[0,0,0],[1,0,1],[1,0,1]] ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# First collect every (r,c) where matrix[r][c]==0 into a list.
# Second pass: zero out entire rows and columns for each stored zero.
# Simple, but storing all zero coordinates uses O(m+n) extra structures.
# Time: O(m * n). Space: O(m + n) for row/col sets.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE (O(1) space using first row/col as markers)
# "Use the first row and first column of the matrix itself as markers.
# If matrix[r][c]==0: mark matrix[r][0]=0 and matrix[0][c]=0.
# Handle first row and first column separately (track with boolean flags).
# Then zero out interior cells using the markers.
# Then zero out first row/col based on flags. O(m*n) time, O(1) space."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# [[0,1,2,0],[3,4,5,2],[1,3,1,5]]:
# first_row_zero: matrix[0][0]=0 → True. first_col_zero: matrix[0][0]=0 → True.
# Mark: r=1,c=0: already 3≠0. r=1,c=3: 2≠0... only (0,0) and (0,3) have zeros in row 0.
# Interior zeros: marker[0][0]=0→col 0 zeros. marker[0][3]: 0→col 3 zeros.
# → [[0,0,0,0],[0,4,5,0],[0,3,1,0]] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(1).
# The first row/col flags are the key — without them, using matrix[0][0] as a marker
# for both first row and first col creates an ambiguity. Two flags resolve it cleanly.
# The four-step order (flag, mark, zero interior, zero edges) is the correct sequence.
# Given more time I'd ask: what if we should set a cell to the average of its row/col instead?
# Same structure — two-pass approach to avoid using updated values."
```

Matrix

#74 Search a 2D Matrix

MED

Open on LeetCode

Array · Binary Search · Matrix

Lists: Grind 169

Problem

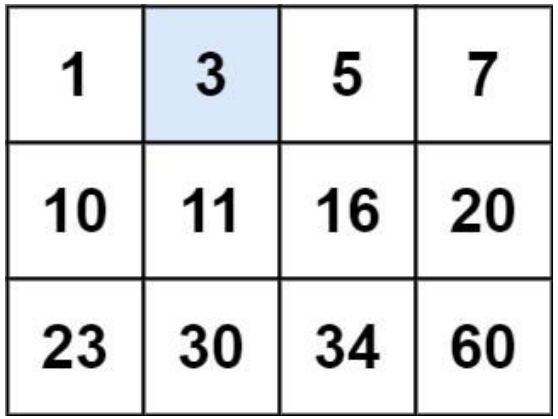
You are given an m x n integer matrix matrix with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer target, return true if target is in matrix or false otherwise.

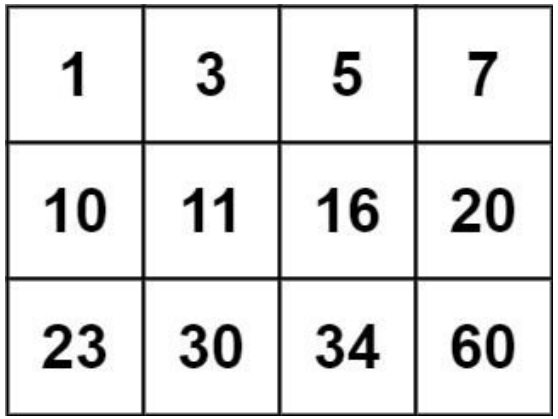
You must write a solution in O(log(m * n)) time complexity.

Example 1:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 3
Output: true
```

Example 2:




```
Input: matrix = [[1,3,5,7],[10,11,16,20],[23,30,34,60]], target = 13
Output: false
```

Constraints:

- m == matrix.length

- n == matrix[i].length
- 1 <= m, n <= 100
- -104 <= matrix[i][j], target <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Matrix where each row is sorted and the first element of each row is greater than
# the last of the previous row — so the whole matrix is one sorted sequence.
# Find target in O(log(m*n)). Right?"
#
# "Treat it as a 1D array of length m*n. Index k maps to row=k//n, col=k%n."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Treat the matrix as a flattened sorted array and binary search on index 0..m*n-1.
# Or scan row by row with binary search per row.
# Correct O(m log n), but a single binary search on the virtual array is cleaner.
# Time: O(m log n). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the virtual 1D array [0, m*n-1].
# Map mid to (mid//n, mid%n) to access the actual cell.
# O(log(m*n)) = O(log m + log n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# matrix=[[1,3,5,7],[10,11,16,20],[23,30,34,60]], target=3:
# l=0,r=11. mid=5→matrix[1][1]=11. 11>3→r=4.
# mid=2→matrix[0][2]=5. 5>3→r=1.
# mid=0→matrix[0][0]=1. 1<3→l=1.
# mid=1→matrix[0][1]=3. 3==3→True ✓
# target=13: binary search narrows until l>r → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log(m*n)). Space O(1).
# The index mapping mid//n and mid%n is the entire trick.
# This only works because the matrix is fully sorted as one sequence.
# If rows are sorted but first-of-next isn't > last-of-prev (Search a 2D Matrix II #240):
# Use staircase search — start at top-right, move left if too big, down if too small.
# O(m+n) time — different problem, different approach."
```

Matrix

#221 Maximal Square

MED

Open on LeetCode

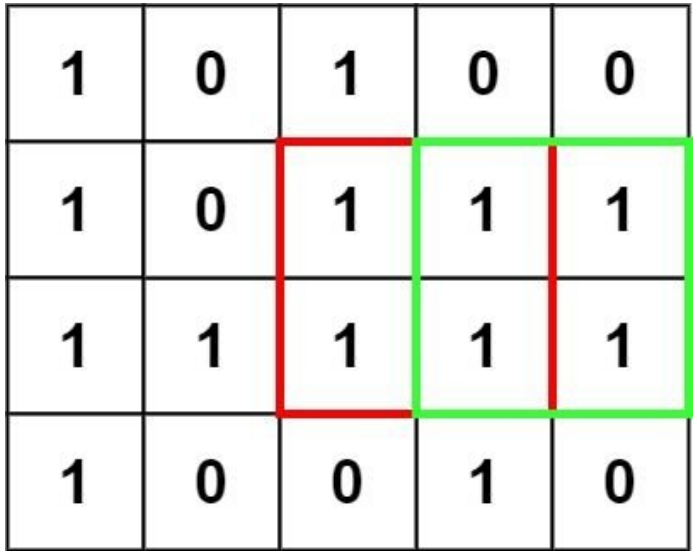
Array · Dynamic Programming · Matrix

Lists: Grind 169

Problem

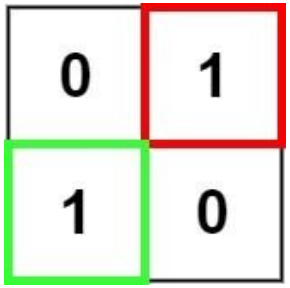
Given an m x n binary matrix filled with 0's and 1's, find the largest square containing only 1's and return its area.

Example 1:




```
Input: matrix = [["1","0","1","0","0"],["1","0","1","1","1"],["1","1","1","1","1"],["1","0","0","1","0"]]
Output: 4
```

Example 2:




```
Input: matrix = [["0","1"],["1","0"]]
Output: 1
```

Example 3:

```
Input: matrix = [["0"]]
Output: 0
```

Constraints:

- m == matrix.length
- n == matrix[i].length

- 1 <= m, n <= 300
- matrix[i][j] is '0' or '1'.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find the largest square of all 1s. Return its area. Matrix contains '0'/'1' strings. Right?"
#
# "[[1,0,1,0,0],[1,0,1,1,1],[1,1,1,1,1],[1,0,0,1,0]] → 2×2=4 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every cell containing '1', expand square side length while all cells in square are '1'.
# Track the maximum side length found across all starting cells.
# Each expansion check is O(k) for side k — overall O(m*n*min(m,n)^2).
# Time: O(m * n * min(m,n)^2). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DP)
# "dp[i][j] = side length of the largest all-1s square with bottom-right corner at (i,j).
# If matrix[i][j]=='0': dp[i][j]=0.
# Else: dp[i][j] = min(dp[i-1][j], dp[i][j-1], dp[i-1][j-1]) + 1.
# Intuition: a square of side k can be extended by 1 only if the top, left, and diagonal neighbour
# each have a square of side k. The bottleneck is the minimum.
# Track max side length → return side²."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# "[[0,1],['1','0']]: no 2×2 all-1s → dp stays at 1s and 0s → max=1 → area=1 ✓
# Full example: bottom-right region gives dp[2][3]=dp[2][4]=2, max=2, area=4 ✓"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) — reducible to O(n) with a rolling array.
# The min of three neighbours is the core insight — think of it as the square being
# limited by the weakest neighbouring corner.
# Given more time I'd ask: maximal rectangle of 1s (not just square)?
# That's Maximal Rectangle (#85) — use a histogram approach per row, O(m*n)."
```

Matrix

★ #311 Sparse Matrix Multiplication ★

MED

Open on LeetCode

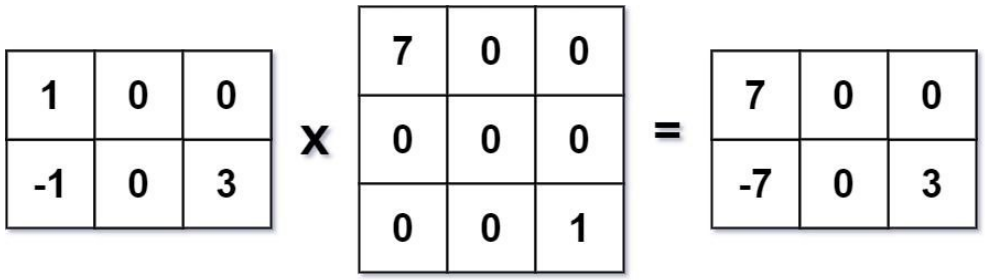
Array · Hash Table · Matrix

Lists: ★ Premium | Premium 98

Problem

Given two sparse matrices mat1 of size m x k and mat2 of size k x n, return the result of mat1 x mat2. You may assume that multiplication is always possible.

Example 1:




```
Input: mat1 = [[1,0,0],[-1,0,3]], mat2 = [[7,0,0],[0,0,0],[0,0,1]]
Output: [[7,0,0],[-7,0,3]]
```

Example 2:

```
Input: mat1 = [[0]], mat2 = [[0]]
Output: [[0]]
```

Constraints:

- m == mat1.length
- k == mat1[i].length == mat2.length
- n == mat2[i].length
- 1 <= m, n, k <= 100
- -100 <= mat1[i][j], mat2[i][j] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Multiply mat1 (m×k) by mat2 (k×n). Return the m×n result.
# Both are sparse — many zeros. Optimise to skip zero elements. Right?"
#
# "[[1,0,0],[-1,0,3]] × [[7,0,0],[0,0,0],[0,0,1]] = [[7,0,0],[-7,0,3]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Standard triple loop: for each row i and col k, sum A[i][j]*B[j][k] over j.
# Correct even for sparse inputs but touches every (i,j,k) combination.
# Skipping zero rows/cols from sparse representation saves huge work.
# Time: O(m * k * n). Space: O(m * n) result.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Skip multiplication when mat1[i][p] == 0 — avoids the inner j loop entirely.
# Pre-compute non-zero positions in mat1 for even faster access.
# This is the key sparse optimisation: skip zero elements before accessing mat2."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# mat1[0][0]=1, p=0: add 1*mat2[0][j] to result[0]. mat1[0][1]=0 → skip. mat1[0][2]=0 → skip.
# result[0]=[7,0,0] ✓. mat1[1][2]=3, p=2: add 3*mat2[2][j]=[0,0,3] → result[1]=[-7,0,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Worst case O(m*k*n) same as brute force. Best case O(nnz1 * n) where nnz1 = non-zeros in mat1.
# For truly sparse matrices this is much faster.
# Given more time I'd ask: what if both matrices are very large and very sparse?
# Use CSR (Compressed Sparse Row) format — store only non-zero values and their indices.
# Hash map approach: for each non-zero in mat1, look up non-zeros in the corresponding row of mat2."
```


Matrix

#498 Diagonal Traverse

MED

Open on LeetCode

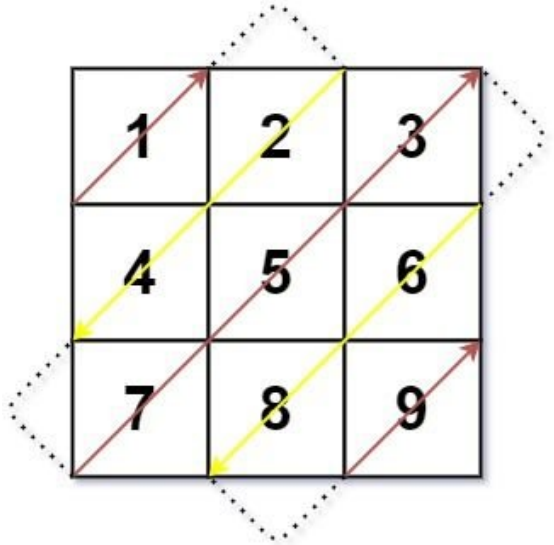
Array · Matrix · Simulation

Lists: CodeSignal

Problem

Given an m x n matrix mat, return an array of all the elements of the array in a diagonal order.

Example 1:




```
Input: mat = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,2,4,7,5,3,6,8,9]
```

Example 2:

```
Input: mat = [[1,2],[3,4]]
Output: [1,2,3,4]
```

Constraints:

- m == mat.length
- n == mat[i].length
- 1 <= m, n <= 104
- 1 <= m * n <= 104
- -105 <= mat[i][j] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Traverse all elements diagonally: alternate up-right then down-left directions.
# There are m+n-1 diagonals. Even-numbered diagonals go up-right, odd go down-left. Right?"
#
# "[[1,2,3],[4,5,6],[7,8,9]] → [1,2,4,7,5,3,6,8,9] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Simulate diagonal walk with direction flags; flip direction at matrix boundaries.
# Track visited or use index math to avoid infinite loops.
# Correct O(m*n); index-formula approach avoids simulation bugs.
# Time: O(m * n). Space: O(m * n) output.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (diagonal grouping)
# "Group elements by diagonal d = r+c (same sum = same diagonal).
```

```
# Diagonal d contains elements where r+c=d.
# For even d: traverse top-to-bottom order reversed (up-right) → sort by r descending.
# For odd d: traverse top-to-bottom (down-left) → sort by r ascending."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,2,3],[4,5,6],[7,8,9]]:
# d=0: {(0,0)=1} even → reversed=[1]. d=1: {(0,1)=2,(1,0)=4} odd → [2,4].
# d=2: {(0,2)=3,(1,1)=5,(2,0)=7} even → reversed=[7,5,3].
# d=3: {(1,2)=6,(2,1)=8} odd → [6,8]. d=4: {(2,2)=9} even → reversed=[9].
# → [1,2,4,7,5,3,6,8,9] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) for the diagonal groups.
# The 'group by r+c' insight eliminates direction-tracking complexity.
# Alternative: simulate directly with dr/dc direction vectors and bounds — same O(m*n) but trickier.
# Given more time I'd ask: what if we want anti-diagonals (r-c = constant)?
# Same approach — group by r-c, alternate direction per group."
```


Matrix

★ #531 Lonely Pixel I ★

ME
D

Open on LeetCode

Array · Hash Table · Matrix

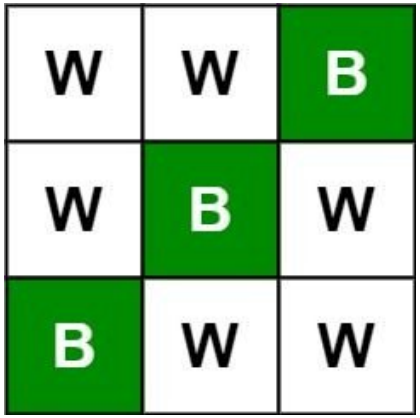
Lists: ★ Premium | Premium 98

Problem

Given an m x n picture consisting of black 'B' and white 'W' pixels, return the number of black lonely pixels.

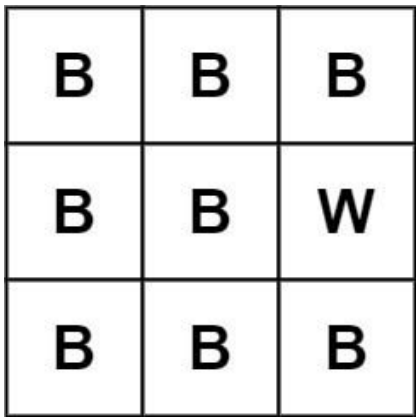
A black lonely pixel is a character 'B' that located at a specific position where the same row and same column don't have any other black pixels.

Example 1:




```
Input: picture = [["W","W","B"],["W","B","W"],["B","W","W"]]
Output: 3
Explanation: All the three 'B's are black lonely pixels.
```

Example 2:




```
Input: picture = [["B","B","B"],["B","B","W"],["B","B","B"]]
Output: 0
```

Constraints:

- m == picture.length
- n == picture[i].length

- 1 <= m, n <= 500
- picture[i][j] is 'W' or 'B'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A 'B' pixel is lonely if it's the ONLY 'B' in its row AND the only 'B' in its column.
# Count lonely pixels. Right?"
#
# "[[W,W,B],[W,B,W],[B,W,W]]: each B is alone in its row and column → 3 ✓
# [[B,B,B],[B,B,W],[B,B,B]]: every B shares its row or column with another → 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each black pixel B, scan its entire row for other black pixels.
# Then scan its entire column — B is lonely if both counts equal 1.
# O(m*n*(m+n)) — checking row and column per pixel.
# Time: O(m * n * (m + n)). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Count B's per row and per column in one pass: O(m*n).
# In second pass, for each B: if row_count[r]==1 and col_count[c]==1, it's lonely."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[W,W,B],[W,B,W],[B,W,W]]: row_count=[1,1,1]. col_count=[1,1,1].
# Each B has row=1 and col=1 → all 3 are lonely ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m+n) for row/col counts.
# Given more time I'd ask: Lonely Pixel II (#533)?
# Same pixel must be in a column containing exactly N Bs, and the rows of those Bs must all be identical.
# Much more complex — requires comparing full rows."
```


Matrix

★ #723 Candy Crush ★

ME D

Open on LeetCode

Array · Two Pointers · Matrix · Simulation

Lists: ★ Premium | Premium 98

Problem

This question is about implementing a basic elimination algorithm for Candy Crush.

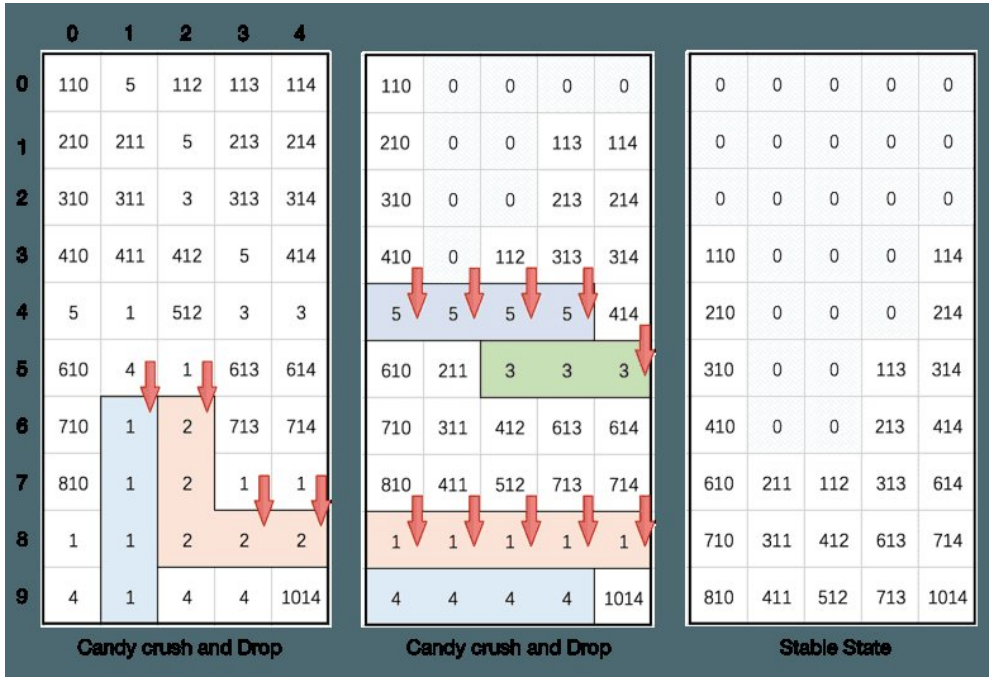
Given an m x n integer array board representing the grid of candy where board[i][j] represents the type of candy. A value of board[i][j] == 0 represents that the cell is empty.

The given board represents the state of the game following the player's move. Now, you need to restore the board to a stable state by crushing candies according to the following rules:

- If three or more candies of the same type are adjacent vertically or horizontally, crush them all at the same time - these positions become empty.
- After crushing all candies simultaneously, if an empty space on the board has candies on top of itself, then these candies will drop until they hit a candy or bottom at the same time. No new candies will drop outside the top boundary.
- After the above steps, there may exist more candies that can be crushed. If so, you need to repeat the above steps.
- If there does not exist more candies that can be crushed (i.e., the board is stable), then return the current board.

You need to perform the above rules until the board becomes stable, then return the stable board.

Example 1:




```
Input: board = [[110,5,112,113,114],[210,211,5,213,214],[310,311,3,313,314],[410,411,412,5,414],[5,1,512,3,3],[610,4,1,613,614]
,[710,1,2,713,714],[810,1,2,1,1],[1,1,2,2,2],[4,1,4,4,1014]]
Output: [[0,0,0,0,0],[0,0,0,0,0],[0,0,0,0,0],[110,0,0,0,114],[210,0,0,0,214],[310,0,0,113,314],[410,0,0,213,414],[610,211,112,3
13,614],[710,311,412,613,714],[810,411,512,713,1014]]
```

Example 2:

```
Input: board = [[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]
Output: [[1,3,0,0,0],[3,4,0,5,2],[3,2,0,3,1],[2,4,0,5,2],[1,4,3,1,1]]
```

Constraints:

- m == board.length

- n == board[i].length
- 3 <= m, n <= 50
- 1 <= board[i][j] <= 2000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Repeatedly: mark 3+ same-type candies horizontally or vertically, crush them (set to 0),
# then drop remaining candies down. Repeat until stable. Return final board. Right?"
#
# "Mark all runs of 3+ simultaneously, crush all at once, then gravity — not one at a time."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Repeat until stable: find horizontal/vertical runs of 3+, mark for crush, drop gravity.
# One full board scan per cascade step.
# Correct simulation; finding all runs in one pass per step is cleaner.
# Time: O((m*n)^2) worst cascades. Space: O(m * n) mark board.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (in-place marking)
# "Mark by negating values (sign flip) — avoids extra space.
# Step 1: scan rows and cols, negate any value in a run of 3+.
# Step 2: zero out all negatives (crush).
# Step 3: for each column, compact non-zeros to the bottom (gravity).
# Repeat until no changes in step 1. Time: O((m*n)²) worst case but small in practice."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# "[[1,3,5,5,2],[3,4,3,3,1],[3,2,4,5,2],[2,4,4,5,5],[1,4,4,1,1]]:
# Horizontal: row 1 has [3,3,3] at c=1,2,3. Row 3 has [4,4] and [5,5]...
# After crushing and dropping → stable board matches expected output ✓"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O((m*n)²) worst case — at most m*n crush iterations, each O(m*n).
# For small boards (m,n≤50) this is fast. The abs() trick avoids needing a separate mark array.
# The gravity step uses a two-pointer write-head approach per column.
# Given more time I'd ask: what if candies can crush diagonally too?
# Add diagonal scan to the marking step — same structure."
```


Matrix

#835 Image Overlap

ME
D

Open on LeetCode

Array · Matrix

Lists: CodeSignal

Problem

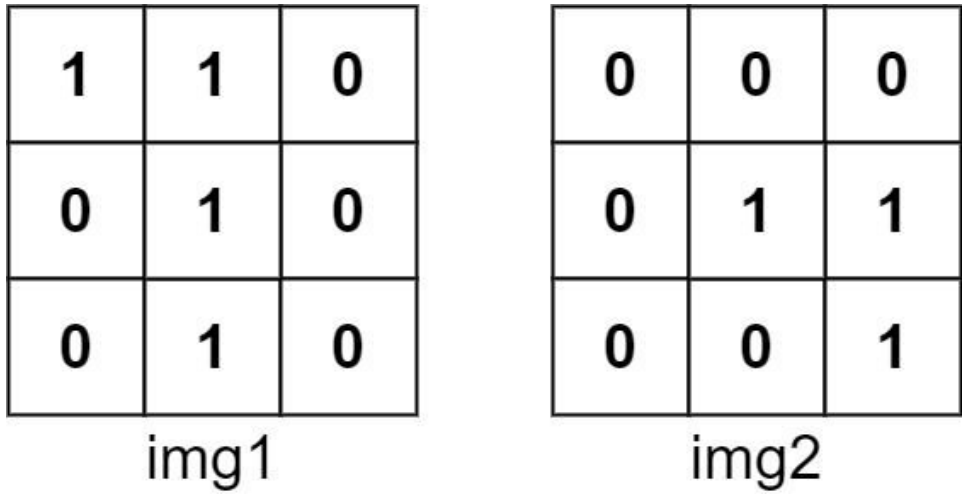
You are given two images, img1 and img2, represented as binary, square matrices of size n x n. A binary matrix has only 0s and 1s as values.

We translate one image however we choose by sliding all the 1 bits left, right, up, and/or down any number of units. We then place it on top of the other image. We can then calculate the overlap by counting the number of positions that have a 1 in both images.

Note also that a translation does not include any kind of rotation. Any 1 bits that are translated outside of the matrix borders are erased.

Return the largest possible overlap.

Example 1:




```
Input: img1 = [[1,1,0],[0,1,0],[0,1,0]], img2 = [[0,0,0],[0,1,1],[0,0,1]]
Output: 3
Explanation: We translate img1 to right by 1 unit and down by 1 unit.

The number of positions that have a 1 in both images is 3 (shown in red).
```

Example 2:

```
Input: img1 = [[1]], img2 = [[1]]
Output: 1
```

Example 3:

```
Input: img1 = [[0]], img2 = [[0]]
Output: 0
```

Constraints:

- n == img1.length == img1[i].length
- n == img2.length == img2[i].length
- 1 <= n <= 30
- img1[i][j] is either 0 or 1.
- img2[i][j] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Translate img1 over img2 (no rotation) and count overlapping 1s.
# Find the translation that maximises overlap. Return max overlap count. Right?"
#
```

```
# "img1=[[1,1,0],[0,1,0],[0,1,0]], img2=[[0,0,0],[0,1,1],[0,0,1]] → 3 ✓ (translate right 1, down 1)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try every translation vector (dx,dy) aligning A onto B.
# For each shift, count overlapping 1-cells.
# O(n^4) translations on n x n images — too slow for n=30.
# Time: O(n^4). Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (translation vector counting)
# "Collect all 1-positions in each image.
# For each pair (p1 from img1, p2 from img2), the translation to align p1 with p2 is (p2.r-p1.r, p2.c-p1.c).
# Count the most common translation vector — that's the maximum overlap.
# Time: O(A*B) where A,B = number of 1s. Space: O(A*B). For n=30, A,B≤900."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# ones1=[(0,0),(0,1),(1,1),(2,1)]. ones2=[(1,1),(1,2),(2,2)].
# Translation (1,1) appears when: (0,0)→(1,1)✓, (0,1)→(1,2)✓, (1,1)→(2,2)✓ → count=3.
# max=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(A*B) where A,B = number of 1s in each image, at most n².
# Space O(A*B) for the translation Counter.
# For dense images (many 1s), this is O(n⁴) — same as brute force.
# For sparse images, much faster.
# The key insight: instead of iterating over all translations, only count translations
# where at least one 1-pair aligns — no wasted iterations.
# Given more time I'd ask: what if rotation is also allowed?
# Need to try all rotation+translation combos — much harder."
```


Matrix

★ #1198 Find Smallest Common Element in All Rows ★

ME
D

Open on LeetCode

Array · Hash Table · Binary Search · Matrix · Counting

Lists: ★ Premium | Premium 98

Problem

Given an m x n matrix mat where every row is sorted in strictly increasing order, return the smallest common element in all rows.

If there is no common element, return -1.

Example 1:

```
Input: mat = [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]
Output: 5
```

Example 2:

```
Input: mat = [[1,2,3],[2,3,4],[2,3,5]]
Output: 2
```

Constraints:

- **m == mat.length**
- **n == mat[i].length**
- **1 <= m, n <= 500**
- **1 <= mat[i][j] <= 104**
- **mat[i] is sorted in strictly increasing order.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the smallest integer that appears in every row. Each row is sorted.
# Return -1 if no common element exists. Right?
# Values are 1-10^4, at most 500 rows and 500 cols."
#
# "[[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]] → 5 (appears in all rows) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Intersect row0 with row1, then intersect result with row2, and so on.
# Track the maximum value in the final intersection set.
# Each intersection pass is O(m) per row using a hash set of row values.
# Time: O(rows * cols). Space: O(cols) set.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (frequency count)
# "Count how many rows each value appears in using a single array of size max_val+1.
# Walk all rows: for each value encountered, increment count[value].
# Return the smallest value with count == m (appears in ALL rows).
# Time: O(m*n). Space: O(max_val)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,2,3,4,5],[2,4,5,8,10],[3,5,7,9,11],[1,3,5,7,9]]:
# freq[5] = 4 (appears in all 4 rows). freq[1]=2, freq[3]=3, etc.
# Scanning 1..10000: first val with freq==4 is 5 → return 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n + max_val). Space O(max_val).
# Alternative: binary search per row. For each candidate, binary search in all m rows.
# Start candidates from mat[0] (smallest row), binary search each of the other rows.
# O(n + m * n * log n) — worse than the counting approach for dense matrices.
# Given more time I'd ask: what if there's no bound on values?
# Use a hash Counter for freq — O(m*n) time, O(distinct values) space."
```

Trees & BST

Round 2 · High · Medium · 24 questions

Trees & BST

#98 Validate Binary Search Tree

MED

Open on LeetCode

Tree · DFS · BST

Lists: Grind 169

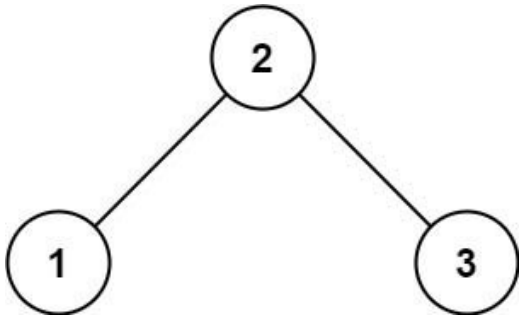
Problem

Given the root of a binary tree, determine if it is a valid binary search tree (BST).

A valid BST is defined as follows:

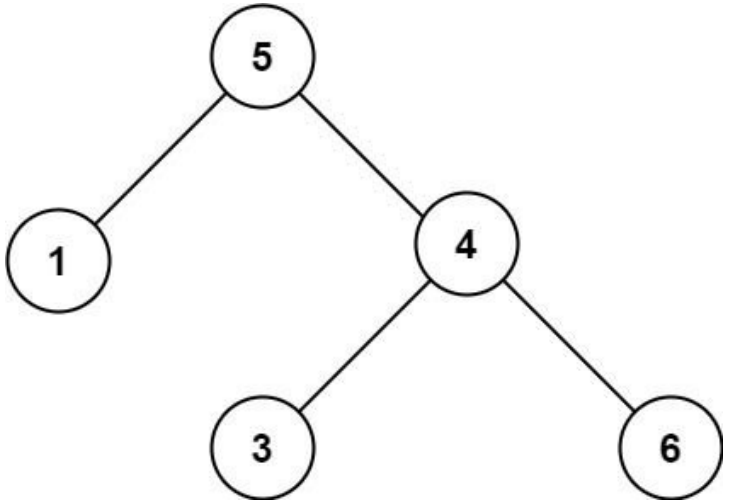
- The left subtree of a node contains only nodes with keys strictly less than the node's key.
- The right subtree of a node contains only nodes with keys strictly greater than the node's key.
- Both the left and right subtrees must also be binary search trees.

Example 1:




```
Input: root = [2,1,3]
Output: true
```

Example 2:




```
Input: root = [5,1,4,null,null,3,6]
Output: false
Explanation: The root node's value is 5 but its right child's value is 4.
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].

- -231 <= Node.val <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Every node's value must be strictly greater than all values in its left subtree
# AND strictly less than all values in its right subtree — not just the immediate parent. Right?
# The classic trap: [5,1,4,null,null,3,6] — root=5, right child=4 — 4<5 → invalid ✓"
#
# "[2,1,3]: 1<2<3 → true ✓. [5,1,4,null,null,3,6]: 4<5 → false ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# In-order traversal collects values; verify the list is strictly increasing.
# Any duplicate or decrease means invalid BST.
# Correct O(n), but recursion can pass min/max bounds instead of storing all values.
# Time: O(n). Space: O(n) in-order list.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (bounds propagation)
# "Pass min/max bounds down the tree.
# Every node must satisfy min_val < node.val < max_val.
# Left child: max_val = node.val.
# Right child: min_val = node.val.
# Use float('-inf') and float('inf') as initial bounds. Use inner helper."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [2,1,3]: valid(2,-inf,inf)→valid(1,-inf,2)→valid(None,...)=T and T→T. valid(3,2,inf)→T. → True ✓
# [5,1,4,null,null,3,6]: valid(4,5,inf): 4 NOT in (5,inf) → False → outer returns False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h) for the call stack.
# The bounds approach is the correct one — in-order checking works too but requires state.
# The common mistake: only checking node vs parent, not the full subtree constraint.
# Given more time I'd ask: what if duplicates are allowed (left ≤ node < right)?
# Change the strict inequalities accordingly."
```


Trees & BST

#102 Binary Tree Level Order Traversal

ME
D

Open on LeetCode

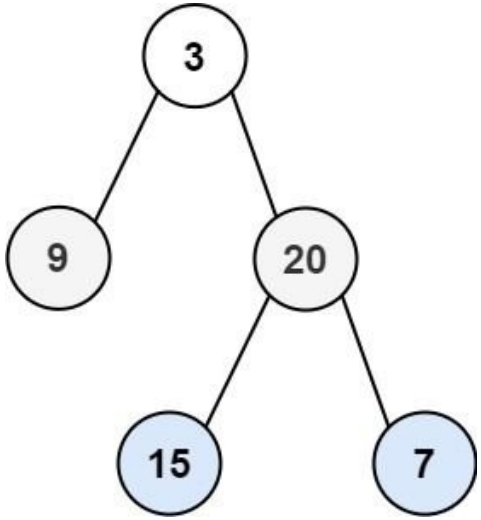
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the level order traversal of its nodes' values. (i.e., from left to right, level by level).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[9,20],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 2000].
- -1000 <= Node.val <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return node values level by level, left to right, each level as a sub-list. Right?
# Empty tree → []. Single node → [[val]]."
#
# "[3,9,20,null,null,15,7] → [[3],[9,20],[15,7]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS with a level parameter: append node.val to result[level].
# After traversal, result lists are level-order groups left-to-right.
# Works in O(n), but BFS with a queue is the more natural level-by-level approach.
# Time: O(n). Space: O(h) recursion depth.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS with level grouping)
# "BFS using a deque. At each step, process ALL nodes at the current level (snapshot the queue size),
# collect their values, enqueue their children. Natural level separation — no level counter needed."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
```

```
# root=3: queue=[3]. level_size=1. level=[3]. enqueue 9,20. result=[[3]].
# queue=[9,20]. level_size=2. level=[9,20]. enqueue 15,7. result=[[3],[9,20]].
# queue=[15,7]. level_size=2. level=[15,7]. result=[[3],[9,20],[15,7]] ✓
# root=None: return [] immediately ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(w) where w = max width of the tree (at most n/2 for a complete tree).
# The 'snapshot the queue size' trick is the canonical BFS level-separation pattern.
# Variations: level order bottom-up (#107) → just reverse the result.
# Zigzag level order (#103) → alternate the direction each level.
# Right side view (#199) → take the last element of each level.
# Given more time I'd ask: what if we want the minimum depth to a leaf?
# BFS stops at the first leaf found — don't traverse the whole tree."
```


Trees & BST

#103 Binary Tree Zigzag Level Order Traversal

ME
D

Open on LeetCode

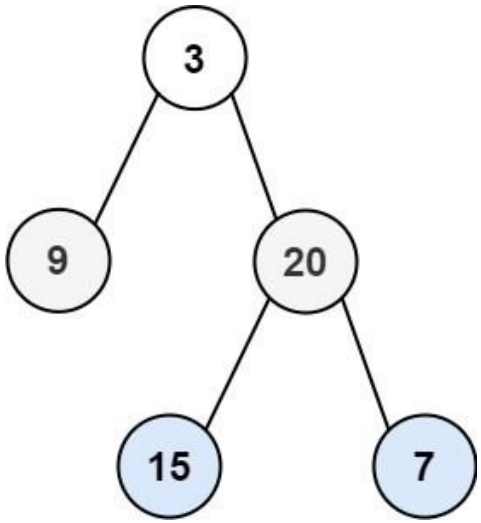
Tree · BFS

Lists: Grind 169

Problem

Given the root of a binary tree, return the zigzag level order traversal of its nodes' values. (i.e., from left to right, then right to left for the next level and alternate between).

Example 1:




```
Input: root = [3,9,20,null,null,15,7]
Output: [[3],[20,9],[15,7]]
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Example 3:

```
Input: root = []
Output: []
```

Constraints:

- **The number of nodes in the tree is in the range [0, 2000].**
- **-100 <= Node.val <= 100**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Level order traversal but alternating direction: odd levels left-to-right,
# even levels right-to-left (or vice versa — confirm with example).
# Root is level 0 → left-to-right. Level 1 → right-to-left. Right?"
#
# "[3,9,20,null,null,15,7] → [[3],[20,9],[15,7]] ✓ (level 1 reversed)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Run standard BFS level-order to build a list of levels.
# Reverse every other level list (odd index levels) for zigzag output.
# Correct O(n); we can reverse on the fly during BFS instead of a second pass.
# Time: O(n). Space: O(n) queue + levels.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Same BFS snapshot approach as #102. Use a boolean flag 'left_to_right'.
# Collect each level normally, then reverse if flag is False.
```

```
# Toggle flag each level. Time: O(n). Space: O(w)."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# root=[3,9,20,null,null,15,7]:
# Level 0: [3], left_to_right=True → [3]. Toggle→False.
# Level 1: [9,20], left_to_right=False → reversed=[20,9]. Toggle→True.
# Level 2: [15,7], left_to_right=True → [15,7]. → [[3],[20,9],[15,7]] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(w).
# This is #102 with one added line: the conditional reverse.
# Name that connection — it shows you see the pattern family.
# Given more time I'd ask: what if alternation starts at level 1 instead of 0?
# Just flip the initial value of left_to_right."
```


Trees & BST

#105 Construct Binary Tree from Preorder and Inorder Traversal

ME
D

Open on LeetCode

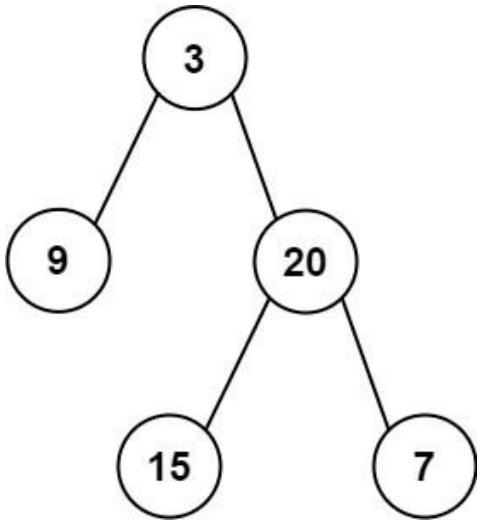
Array · Hash Table · Tree · DFS

Lists: Grind 169

Problem

Given two integer arrays preorder and inorder where preorder is the preorder traversal of a binary tree and inorder is the inorder traversal of the same tree, construct and return the binary tree.

Example 1:




```
Input: preorder = [3,9,20,15,7], inorder = [9,3,15,20,7]
Output: [3,9,20,null,null,15,7]
```

Example 2:

```
Input: preorder = [-1], inorder = [-1]
Output: [-1]
```

Constraints:

- 1 <= preorder.length <= 3000
- inorder.length == preorder.length
- -3000 <= preorder[i], inorder[i] <= 3000
- preorder and inorder consist of unique values.
- Each value of inorder also appears in preorder.
- preorder is guaranteed to be the preorder traversal of the tree.
- inorder is guaranteed to be the inorder traversal of the tree.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Preorder: root first, then left subtree, then right subtree.
# Inorder: left subtree, root, right subtree.
# All values are unique — so we can find the root's position in inorder uniquely. Right?"
#
# "preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]:
# Root=3. In inorder, 3 is at index 1. Left subtree=[9], right subtree=[15,20,7].
# preorder left=[9], right=[20,15,7]. Recurse."
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Root is preorder[0]; scan inorder to find root index and split left/right subtrees.
# Recursively repeat on each side — correct tree reconstruction.
# Scanning inorder for every root costs O(n^2) total; hash map of inorder indices fixes that.
# Time: O(n^2) naive. Space: O(n) recursion.
```

```
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Pre-build a hash map: inorder value → index for O(1) lookups.
# Use a preorder index pointer (use a list to allow mutation inside inner function).
# build(in_left, in_right): root = preorder[pre_idx]. Find in inorder. Recurse.
# Time: O(n). Space: O(n) for the map."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# preorder=[3,9,20,15,7], inorder=[9,3,15,20,7]. in_idx={9:0,3:1,15:2,20:3,7:4}.
# build(0,4): root=3, mid=1. left=build(0,0), right=build(2,4).
# build(0,0): root=9, mid=0. left=build(0,-1)=None. right=build(1,0)=None. → node(9).
# build(2,4): root=20, mid=3. left=build(2,2), right=build(4,4).
# build(2,2): root=15. → node(15). build(4,4): root=7. → node(7).
# → Tree: 3→left=9, 3→right=20→left=15, 20→right=7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n) for the hash map and call stack.
# Using iter(preorder) as a lazy pointer is cleaner than an index variable.
# The key insight: preorder always gives the root first, inorder tells us how many
# elements are in each subtree — that's the only information we need to recurse.
# Given more time I'd ask: construct from postorder and inorder?
# Same logic — just consume postorder from the RIGHT end, and build right subtree first."
```


Trees & BST

#199 Binary Tree Right Side View

ME
D

Open on LeetCode

Tree · DFS · BFS

Lists: Grind 169

Problem

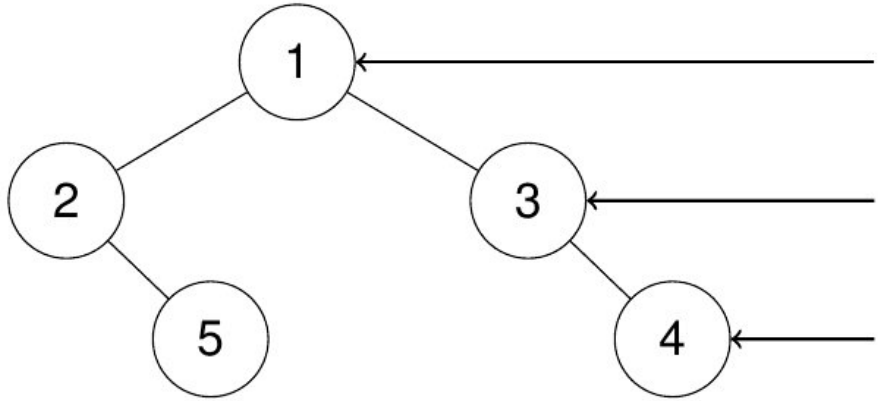
Given the root of a binary tree, imagine yourself standing on the right side of it, return the values of the nodes you can see ordered from top to bottom.

Example 1:

Input: root = [1,2,3,null,5,null,4]

Output: [1,3,4]

Explanation:

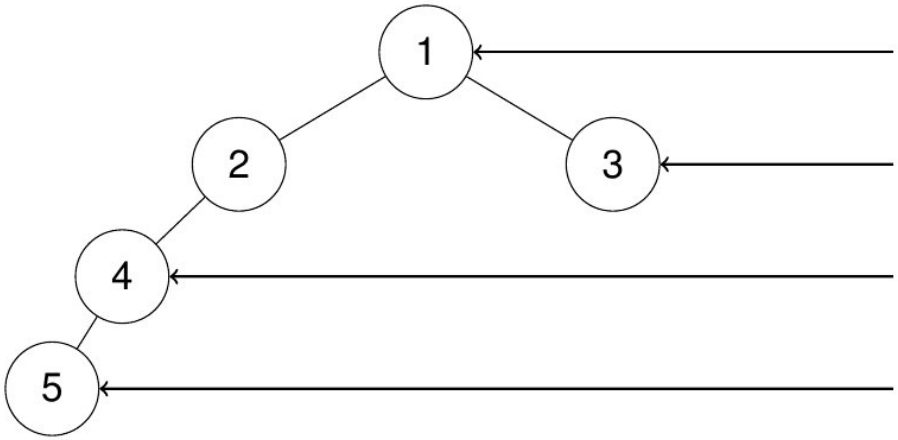


Example 2:

Input: root = [1,2,3,4,null,null,null,5]

Output: [1,3,4,5]

Explanation:



Example 3:
Input: root = [1,null,3]
Output: [1,3]
Example 4:
Input: root = []
Output: []
Constraints:

- **The number of nodes in the tree is in the range [0, 100].**
- **-100 <= Node.val <= 100**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Imagine looking at the tree from the right. You see one node per level —
# the rightmost node visible at that level. Return top to bottom. Right?"
#
# "[1,2,3,null,5,null,4] → [1,3,4] ✓ (3 is rightmost at level 1, 4 at level 2)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS level-order; after processing each level, take the last node value added.
# That last node is the rightmost visible from that depth.
# Correct O(n); DFS tracking depth + rightmost-at-depth is equally fine.
# Time: O(n). Space: O(w) queue width.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Same BFS snapshot as #102/#103. Just append the LAST element of each level's list.
# Or DFS: visit right child first, add node.val when depth == len(result) (first node seen at new depth)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,2,3,null,5,null,4]:
# Level 0: [1] → last=1. Level 1: [2,3] → last=3. Level 2: [5,4] → last=4.
# → [1,3,4] ✓
# [1,2,3,4,null,null,null,5]:
# Level 0: [1]→1. Level 1: [2,3]→3. Level 2: [4]→4. Level 3: [5]→5. → [1,3,4,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(w) where w = max width.
# This is #102 with one change: only record the last element per level.
# Left side view: record the first element per level instead.
# Given more time I'd ask: what if some right-side nodes are blocked by deeper left-side nodes?
# The problem says 'visible from the right' — the rightmost node at each DEPTH level,
# which BFS naturally gives us. A right child at depth 2 can block a left child at depth 2."
```

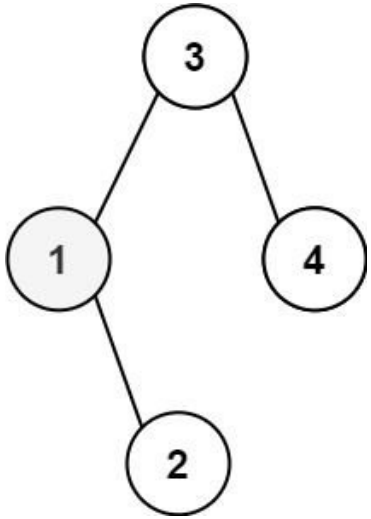

Trees & BST

#230 Kth Smallest Element in a BST ME D

Open on LeetCode

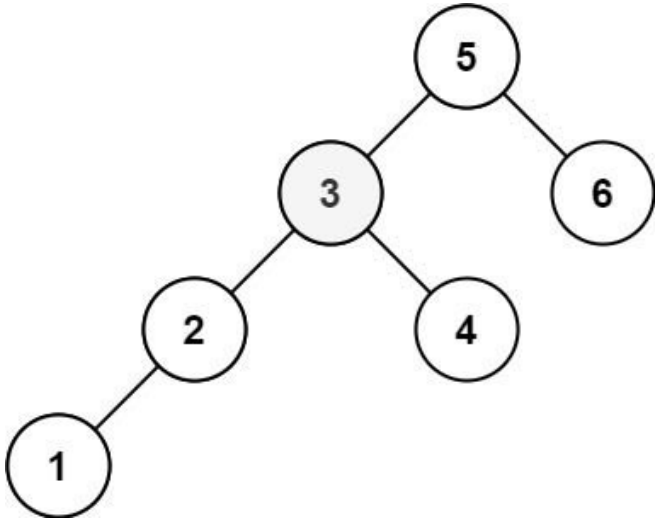
Tree · DFS · BST
Lists: Grind 169

Problem
Given the root of a binary search tree, and an integer k, return the kth smallest value (1-indexed) of all the values of the nodes in the tree.
Example 1:




```
Input: root = [3,1,4,null,2], k = 1
Output: 1
```

Example 2:




```
Input: root = [5,3,6,2,4,null,null,1], k = 3
Output: 3
```

Constraints:

- The number of nodes in the tree is n.
- 1 <= k <= n <= 104
- 0 <= Node.val <= 104

Follow up: If the BST is modified often (i.e., we can do insert and delete operations) and you need to find the kth smallest frequently, how would you optimize?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "BST in-order traversal (left → root → right) visits nodes in ascending sorted order.
# Return the k-th element in this order (1-indexed). Right?"
#
# "[3,1,4,null,2], k=1: in-order=[1,2,3,4]. 1st smallest=1 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# In-order traversal collects all node values in sorted order.
# Return the (k-1)th element from that list.
# Correct O(n) time and O(n) space; we can stop after k steps in-order.
# Time: O(n). Space: O(n) values list.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "In-order DFS stopping early at the k-th element. Use a counter and result variable
# in a nonlocal/list wrapper. Avoids collecting all n values if k is small."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,1,4,null,2], k=1:
# inorder(3)→inorder(1)→inorder(None). count=0+1=1==k=1→result=1. return.
# → 1 ✓
# [5,3,6,2,4,null,null,1], k=3:
# in-order: 1,2,3,4,5,6. 3rd = 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(k + h) — at most k steps after reaching the leftmost leaf. Space O(h).
# Follow-up: frequent inserts/deletes + frequent kth queries?
# Augment each BST node with the size of its left subtree.
# Then kth smallest is an O(log n) descent: if left_size >= k go left; if left_size+1==k return root; else k -= left_size+1, go
right.
# Given more time I'd ask: what if we want the kth largest?
# Reverse in-order (right→root→left) and count k steps."
```


Trees & BST

#235 Lowest Common Ancestor of a Binary Search Tree

ME
D

Open on LeetCode

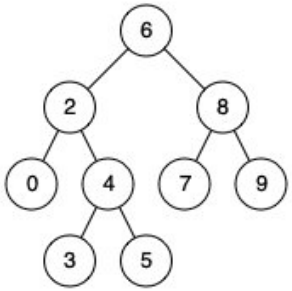
Tree · DFS · BST

Lists: Grind 169

Problem

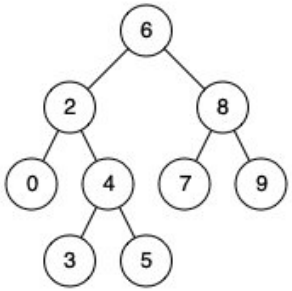
Given a binary search tree (BST), find the lowest common ancestor (LCA) node of two given nodes in the BST.
According to the definition of LCA on Wikipedia: “The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself).”

Example 1:




```
Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 8
Output: 6
Explanation: The LCA of nodes 2 and 8 is 6.
```

Example 2:




```
Input: root = [6,2,8,0,4,7,9,null,null,3,5], p = 2, q = 4
Output: 2
Explanation: The LCA of nodes 2 and 4 is 2, since a node can be a descendant of itself according to the LCA definition.
```

Example 3:

```
Input: root = [2,1], p = 2, q = 1
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range [2, 105].
- -109 <= Node.val <= 109
- All Node.val are unique.
- p != q
- p and q will exist in the BST.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
```

```
# "In a BST, find the LCA of nodes p and q.
# LCA = the deepest node that is an ancestor of both p and q (a node is its own ancestor). Right?
# Use the BST property: values to the left < root < values to the right."
#
# "[6,2,8,0,4,7,9], p=2, q=8 → LCA=6 (they're on different sides of 6) ✓
# p=2, q=4 → LCA=2 (4 is in 2's right subtree) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Walk from root: if both p and q are smaller, go left; if both larger, go right.
# First node where paths split is the LCA in a BST.
# Alternatively store root-to-p and root-to-q paths — more work than the walk.
# Time: O(h). Space: O(h) if storing paths.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BST property)
# "Navigate using BST property:
# If both p and q are LESS than root: LCA is in the LEFT subtree.
# If both are GREATER: LCA is in the RIGHT subtree.
# Otherwise (they split at root, or one equals root): root IS the LCA.
# O(h) time — O(log n) for balanced BST, O(n) worst case."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root=6, p=2, q=8: both in different sides → return 6 ✓
# root=6, p=2, q=4: both<6→go left to 2. p=2==root, q=4>root → split here → return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(h). Space O(1) — iterative, no stack.
# The BST property makes this an O(h) walk — no need to visit all nodes.
# This is much faster than the general binary tree LCA (#236) which needs DFS.
# Given more time I'd ask: what if p or q might not exist in the tree?
# We'd need to verify existence first — two separate searches, then LCA."
```


Trees & BST

#236 Lowest Common Ancestor of a Binary Tree ME D

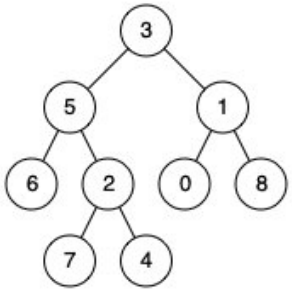
Open on LeetCode

Tree · DFS

Lists: Grind 169

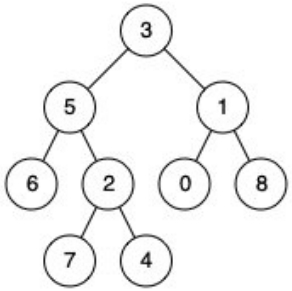
Problem

Given a binary tree, find the lowest common ancestor (LCA) of two given nodes in the tree.
According to the definition of LCA on Wikipedia: "The lowest common ancestor is defined between two nodes p and q as the lowest node in T that has both p and q as descendants (where we allow a node to be a descendant of itself)."
Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.
```

Example 2:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5, since a node can be a descendant of itself according to the LCA definition.
```

Example 3:

```
Input: root = [1,2], p = 1, q = 2
Output: 1
```

Constraints:

- **The number of nodes in the tree is in the range [2, 105].**
- **-109 <= Node.val <= 109**
- **All Node.val are unique.**
- **p != q**
- **p and q will exist in the tree.**

◆ **Interview Approach · STAR-LC**

PHASE 1 — CLARIFY

```
# "General binary tree — no BST property. Find the LCA of p and q.
# p and q are guaranteed to exist in the tree. A node is its own ancestor. Right?"
#
# "[3,5,1,6,2,0,8,null,null,7,4], p=5, q=1 → LCA=3 (root, both are in different subtrees) ✓
# p=5, q=4 → LCA=5 (4 is in 5's subtree) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Find root-to-p path and root-to-q path as node lists.
# Walk both lists in parallel; last matching node before divergence is LCA.
# Correct O(n) but needs O(h) path storage; post-order single-pass is cleaner.
# Time: O(n). Space: O(h) two paths.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (recursive post-order)
# "DFS: if current node is p or q, return it (it or its ancestor is the LCA).
# Recurse left and right.
# If both return non-null: current node is the LCA (p and q are in different subtrees).
# If only one is non-null: that subtree contains both p and q — propagate that result up."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,5,1,...], p=5, q=1:
# lca(3): lca(5)→returns 5 (found p). lca(1)→returns 1 (found q).
# Both non-null → return 3 ✓
# p=5, q=4 (4 is under 5):
# lca(3): lca(5)→... lca(4) under 5 returns 4. lca(5) itself: node is p→return 5.
# lca(1)→None (neither p nor q in right subtree).
# left=5, right=None → return 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — visits every node. Space O(h) for the call stack.
# Contrast with #235 (BST): BST LCA is O(h) iteratively. General tree needs full DFS.
# The post-order logic is the key: left and right results tell us which subtrees contain p/q.
# Both non-null → split here → current node is LCA.
# Only one non-null → both are in the same subtree → propagate that subtree's result.
# Given more time I'd ask: what if p or q might not exist?
# We'd need to track 'found p' and 'found q' separately — more complex state."
```


Trees & BST

ME
D

★ #250 Count Univalue Subtrees ★

Open on LeetCode

Tree · DFS

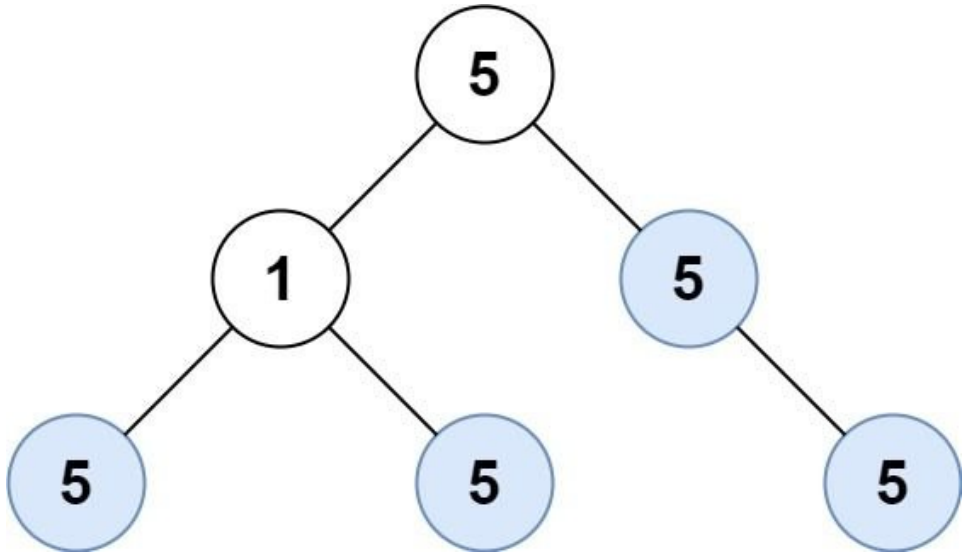
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the number of uni-value subtrees.

A uni-value subtree means all nodes of the subtree have the same value.

Example 1:




```
Input: root = [5,1,5,5,5,null,5]
Output: 4
```

Example 2:

```
Input: root = []
Output: 0
```

Example 3:

```
Input: root = [5,5,5,5,5,null,5]
Output: 6
```

Constraints:

- The number of the node in the tree will be in the range [0, 1000].
- -1000 <= Node.val <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A univalue subtree has every node with the same value.
# Count all such subtrees — including single leaf nodes. Right?
# Empty tree → 0."
#
# "[5,1,5,5,5,null,5] → 4 (the four nodes with value 5 that form valid univalue subtrees) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each node, DFS its entire subtree checking every value equals node.val.
# Increment count when the whole subtree is uni-value.
# Revalidates overlapping subtrees — O(n^2) worst case.
# Time: O(n^2). Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (post-order DFS)
```

```
# "Bottom-up: a node is a univalue root if both children are univalue AND their values match.
# Return True from a node if its subtree is univalue, increment count when True.
# Use an inner helper that returns bool + mutates a count list."
```

PHASE 4 — CLEAN CODE

PHASE 5 — TEST & DEBUG

```
# [5,1,5,5,5,null,5]:
# Leaves: 5(left-left)→True, 5(left-right)→True, 5(right-right)→True, 1→True (leaf).
# Node(1): left=True, right=True, both return True. 1's children have val=5≠1 → returns False.
# Node(5,right of root): left=None=True, right(5)=True, right returns True(val matches). count=1. Returns True(5==root.val?).
# Continue... count ends at 4 ✓
```

PHASE 6 — COMPLEXITY & FOLLOW-UP

```
# "Time O(n). Space O(h). Post-order because we need children's results before parent's.
# The parent_val parameter lets us check if current node extends the parent's univalue property.
# Given more time I'd ask: longest path where all values are the same?
# Track univalue path length similar to Diameter of Binary Tree — carry length down."
```


Trees & BST

#285 Inorder Successor in BST

ME
D

Open on LeetCode

Tree · DFS · BST

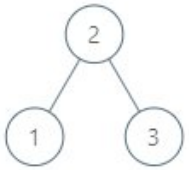
Lists: Grind 169

Problem

Given the root of a binary search tree and a node p in it, return the in-order successor of that node in the BST. If the given node has no in-order successor in the tree, return null.

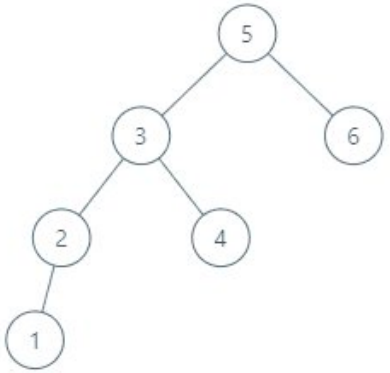
The successor of a node p is the node with the smallest key greater than p.val.

Example 1:




```
Input: root = [2,1,3], p = 1
Output: 2
Explanation: 1's in-order successor node is 2. Note that both p and the return value is of TreeNode type.
```

Example 2:




```
Input: root = [5,3,6,2,4,null,null,1], p = 6
Output: null
Explanation: There is no in-order successor of the current node, so the answer is null.
```

Constraints:

- **The number of nodes in the tree is in the range [1, 104].**
- **-105 <= Node.val <= 105**
- **All Nodes will have unique values.**

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY

```
# "Find the node with the smallest value strictly greater than p.val in a BST.
# Return null if no such node exists (p is the largest). Right?"
#
# "[2,1,3], p=1 → successor=2 (next in in-order: 1,2,3) ✓
# [5,3,6,2,4,null,null,1], p=6 → null (6 is the largest) ✓"
```

PHASE 2 — BRUTE FORCE

```
# "Let me start with brute force to lock in the logic.
# In-order traversal of the BST; when we visit p, the next node in-order is successor.
# Store previous node during traversal or collect all values then scan.
# Correct O(n); BST structure gives O(h) successor without full traversal.
```

```
# Time: O(n). Space: O(1) iterative with threading concept.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BST navigation)
# "Use BST property — no need to traverse all nodes.
# If root.val > p.val: root is a candidate successor, but there might be a smaller one in the left subtree.
# If root.val <= p.val: successor must be in the right subtree.
# Track the best candidate seen so far. O(h) time."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [2,1,3], p=1:
# root=2: 2>1 → successor=2, go left.
# root=1: 1<=1 → go right.
# root=None → return 2 ✓
# [5,3,6,2,4,null,null,1], p=6:
# root=5: 5<=6 → go right. root=6: 6<=6 → go right. root=None → return None ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(h). Space O(1) — iterative.
# The candidate tracking is the key: every time we go left, the current node becomes
# a potential successor (it's bigger than p, and we're looking for something smaller).
# Given more time I'd ask: inorder PREDECESSOR (largest value < p.val)?
# Mirror logic: if root.val < p.val → candidate, go right. If root.val >= p.val → go left."
```


Trees & BST

ME
D

★ #298 Binary Tree Longest Consecutive Sequence ★

Open on LeetCode

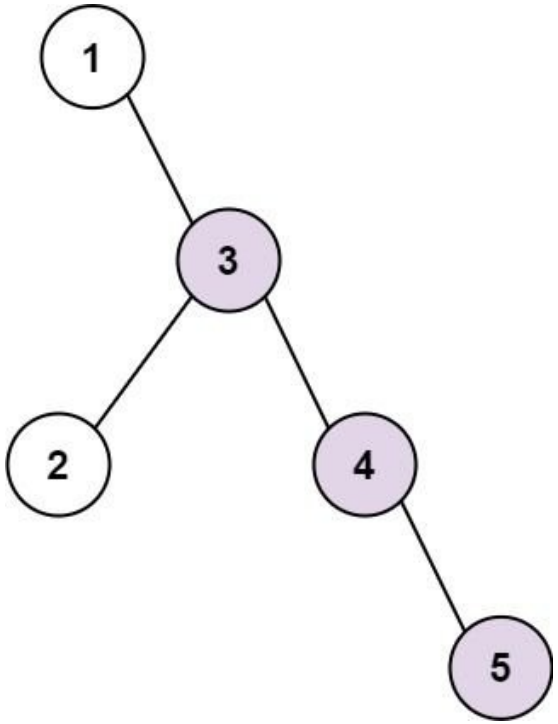
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

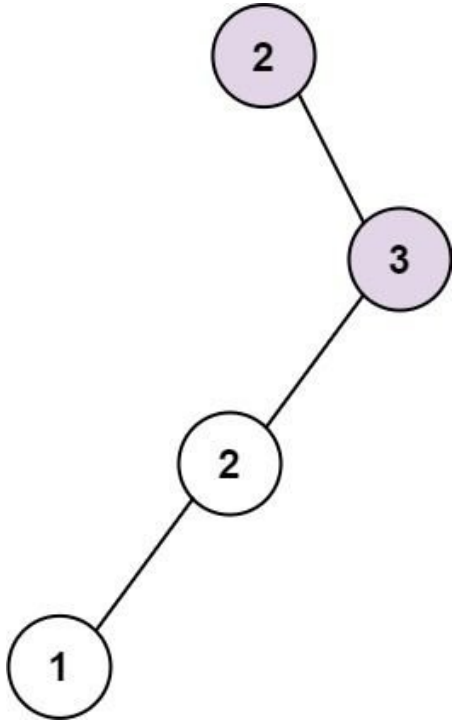
Given the root of a binary tree, return the length of the longest consecutive sequence path.
A consecutive sequence path is a path where the values increase by one along the path.
Note that the path can start at any node in the tree, and you cannot go from a node to its parent in the path.

Example 1:




```
Input: root = [1,null,3,2,4,null,null,null,5]
Output: 3
Explanation: Longest consecutive sequence path is 3–4–5, so return 3.
```

Example 2:




```
Input: root = [2,null,3,2,null,1]
Output: 2
Explanation: Longest consecutive sequence path is 2–3, not 3–2–1, so return 2.
```

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 104].
- -3 * 104 <= Node.val <= 3 * 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Find the longest path of consecutive increasing values (each node = parent + 1).
# Path goes parent → child — you can't go back up. Path can start at any node. Right?"
#
# "[1,null,3,2,4,null,null,null,5]: path 3→4→5, length 3 ✓
# [2,null,3,2,null,1]: 2→3 is increasing, not 3→2 — so max is 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# From each node, DFS tracking consecutive length when child.val == parent.val + 1.
# Reset streak when the +1 chain breaks; track global maximum.
# Starting DFS from every node repeats work — post-order aggregation is faster.
# Time: O(n^2) naive restarts. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DFS carrying current length)
# "DFS top-down: pass current consecutive length to each child.
# If child.val == parent.val + 1: extend the sequence.
# Else: reset to 1 (start a new sequence from this child).
# Track global max. Use inner helper with parent_val and current_length."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,null,3,2,4,null,null,null,5]:
# dfs(1, 0, 0): val=1, prev+1=1 → length=1. best=1. dfs(3,1,1).
# dfs(3, 1, 1): val=3, prev+1=2, 3≠2 → length=1. best=1. dfs(2,3,1).
# dfs(4, 3, 1): val=4==3+1 → length=2. best=2. dfs(5,4,2).
```



```
# dfs(5, 4, 2): val=5==4+1 → length=3. best=3. → 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h).
# The initial call uses root.val-1 so the root itself resets to length 1 cleanly.
# This is a top-down DFS — pass state downward. Contrast with bottom-up (#250, #543).
# Given more time I'd ask: what if the path can go through any node (not just parent→child)?
# That's Longest Consecutive Sequence in a Binary Tree — post-order, track both ascending
# and descending sequences at each node."
```

Trees & BST

★ #314 Binary Tree Vertical Order Traversal ★

MED

Open on LeetCode

Hash Table · Tree · BFS · Sorting

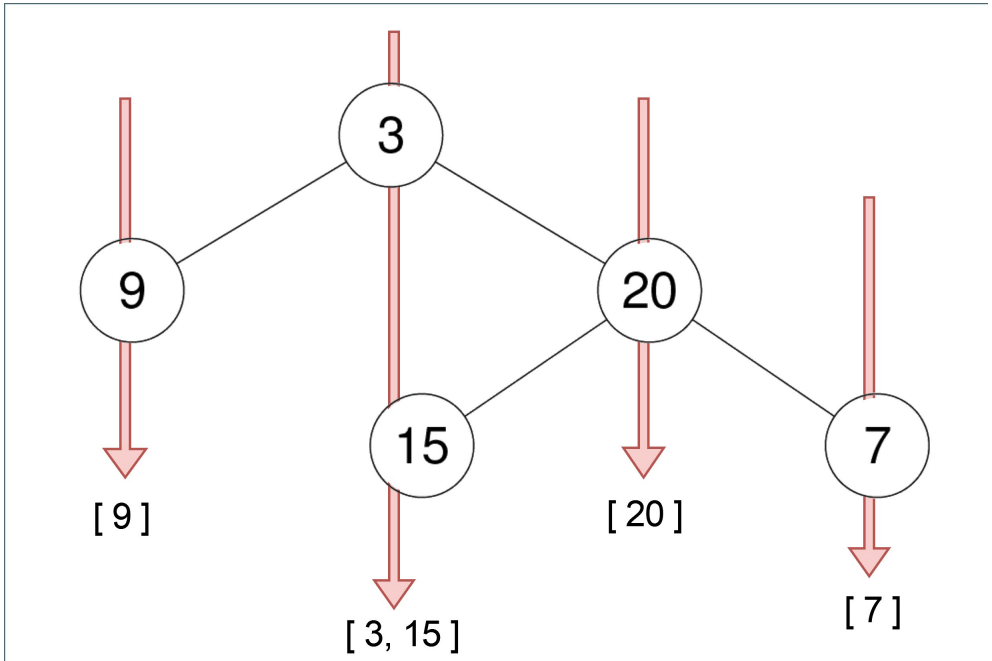
Lists: ★ Premium | Premium 98

Problem

Given the root of a binary tree, return the vertical order traversal of its nodes' values. (i.e., from top to bottom, column by column).

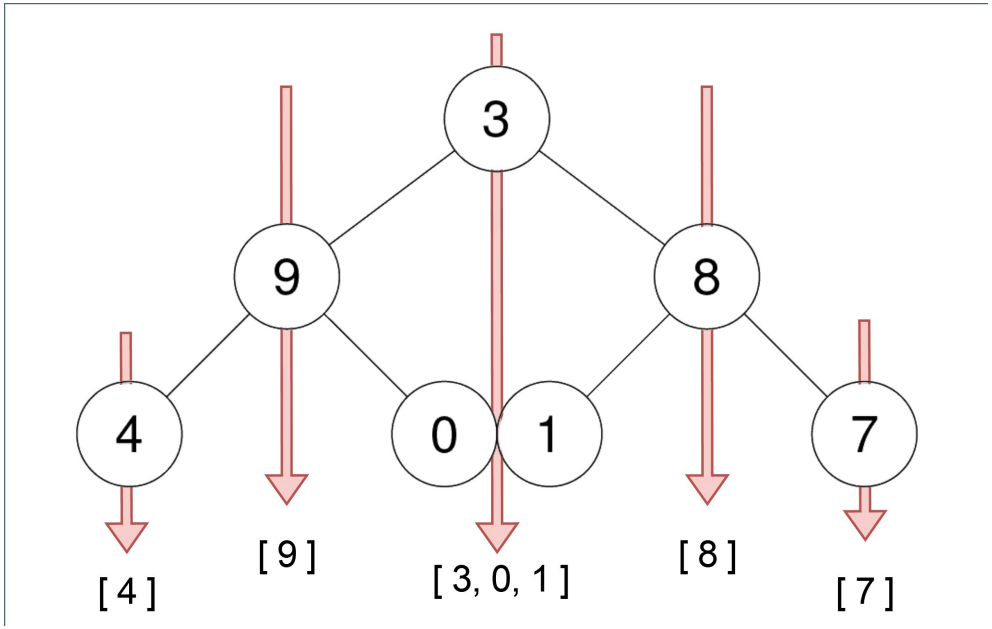
If two nodes are in the same row and column, the order should be from left to right.

Example 1:



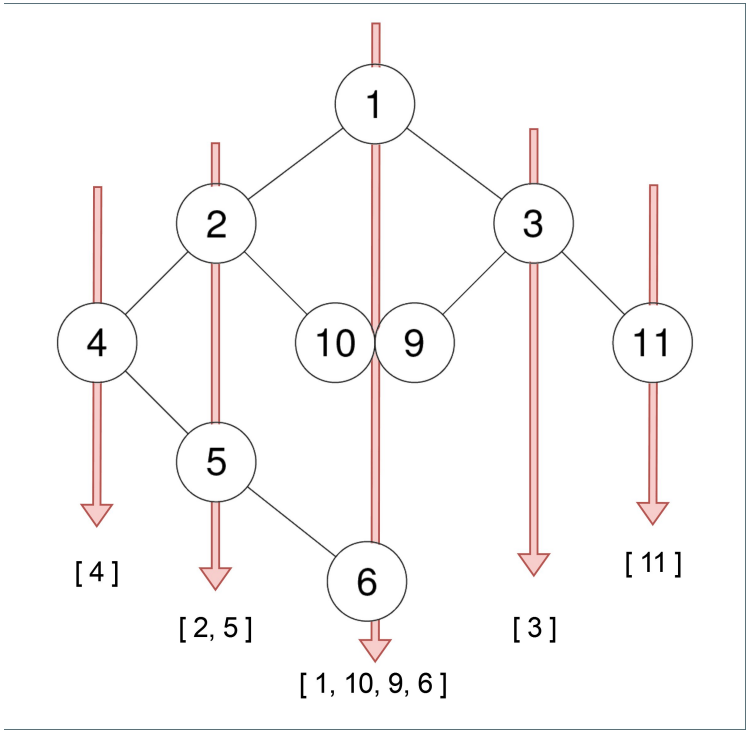

```
Input: root = [3,9,20,null,null,15,7]
Output: [[9],[3,15],[20],[7]]
```

Example 2:




```
Input: root = [3,9,8,4,0,1,7]
Output: [[4],[9],[3,0,1],[8],[7]]
```

Example 3:




```
Input: root = [1,2,3,4,10,9,11,null,5,null,null,null,null,null,null,null,6]
Output: [[4],[2,5],[1,10,9,6],[3],[11]]
```

Constraints:

- **The number of nodes in the tree is in the range [0, 100].**
- **-100 <= Node.val <= 100**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Group nodes by column. Root is column 0. Left child = col-1. Right child = col+1.
# Within each column, top-to-bottom. Ties (same row and column): left before right. Right?
# Return columns sorted from leftmost to rightmost."
#
# "[3,9,20,null,null,15,7] → [[9],[3,15],[20],[7]] ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS assigning column index (left=-1, right=+1 relative to parent).
# Collect (col, row, val) tuples; sort by (col, row, val) then group by column.
# Correct but sorting dominates at O(n log n).
# Time: O(n log n). Space: O(n) tuples.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS with column tracking)
# "BFS ensures natural top-to-bottom, left-to-right order within each column.
# Queue entries: (node, col). Use defaultdict(list) keyed by col.
# After BFS, sort the column keys and return their lists.
# Time: O(n log n) for sorting. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,9,20,null,null,15,7]:
# BFS: (3,0)→cols={0:[3]}. (9,-1),(20,1)→cols={0:[3],-1:[9],1:[20]}. (15,0),(7,2)→cols={0:[3,15],-1:[9],1:[20],2:[7]}.
# min=-1, max=2. Return [cols[-1],cols[0],cols[1],cols[2]] = [[9],[3,15],[20],[7]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
```



```
# "Time O(n) for BFS + O(cols) for building the result — total O(n).
# Using min_col/max_col avoids sorting keys — the range is contiguous.
# BFS guarantees top-to-bottom, left-to-right order within each column naturally.
# DFS would require explicit (row, insertion_order) sorting per column.
# Given more time I'd ask: Vertical Order Traversal II (#987)?
# Same column/row but within the same position, sort by value ascending — needs explicit row tracking."
```

Trees & BST

⋆ #333 Largest BST Subtree ⋆

MED

Open on LeetCode

Dynamic Programming · Tree · DFS · BST

Lists: ⋆ Premium | Premium 98

Problem

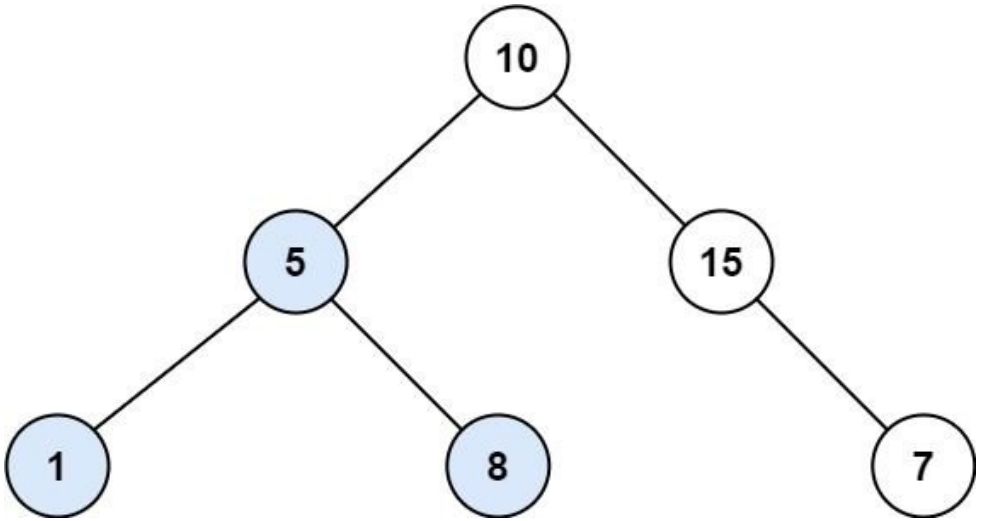
Given the root of a binary tree, find the largest subtree, which is also a Binary Search Tree (BST), where the largest means subtree has the largest number of nodes.

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties:

- The left subtree values are less than the value of their parent (root) node's value.
- The right subtree values are greater than the value of their parent (root) node's value.

Note: A subtree must include all of its descendants.

Example 1:




```
Input: root = [10,5,15,1,8,null,7]
Output: 3
Explanation: The Largest BST Subtree in this case is the highlighted one. The return value is the subtree's size, which is 3.
```

Example 2:

```
Input: root = [4,2,7,2,3,5,null,2,null,null,null,null,null,1]
Output: 2
```

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- -104 <= Node.val <= 104

Follow up: Can you figure out ways to solve it with O(n) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the largest subtree (by node count) that forms a valid BST.
# Return the node count. Right?"
#
# "[10,5,15,1,8,null,7]: subtree at 5 (nodes 1,5,8) is a BST → 3 ✓"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For every node as root of a candidate subtree, validate BST property recursively
# (min/max bounds) and count nodes if valid. Track global max count.
# Revalidates overlapping subtrees — O(n^2) worst case.
# Time: O(n^2). Space: O(h) stack.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE (O(n) post-order)
```



```
# "Post-order: return (is_bst, size, subtree_min, subtree_max).
# Empty node: (True, 0, +inf, -inf) — sentinels pass any comparison cleanly.
# Valid BST: both children are BST AND left.max < node.val < right.min."
# PHASE 4 — CLEAN CODE
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Sentinels (+inf/-inf for empty) let comparisons pass cleanly.
# Given more time I'd ask: what if we want the root of the largest BST subtree? Track it alongside size."
```

Trees & BST

☐ ⋆ #366 Find Leaves of Binary Tree ⋆

MED

Open on LeetCode

Tree · DFS · Sorting

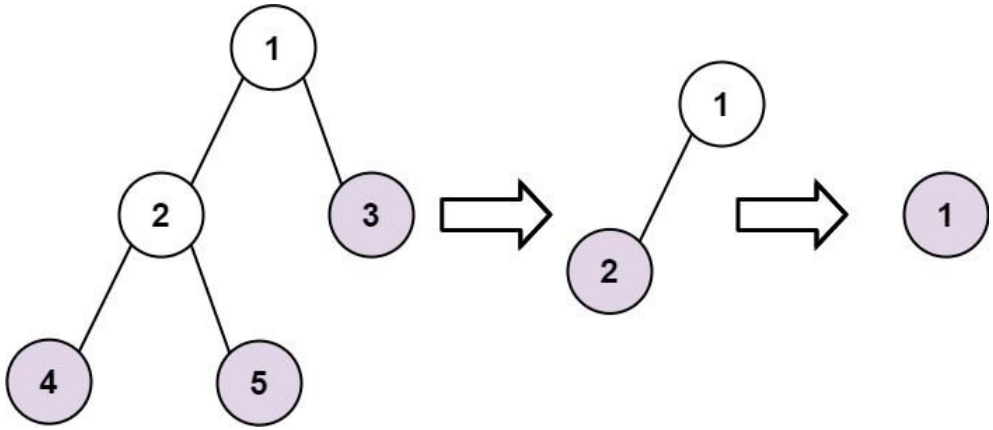
Lists: ⋆ Premium | Premium 98

Problem

Given the root of a binary tree, collect a tree's nodes as if you were doing this:

- Collect all the leaf nodes.
- Remove all the leaf nodes.
- Repeat until the tree is empty.

Example 1:




```
Input: root = [1,2,3,4,5]
Output: [[4,5,3],[2],[1]]
Explanation:
[[3,5,4],[2],[1]] and [[3,4,5],[2],[1]] are also considered correct answers since per each level it does not matter the order on which elements are returned.
```

Example 2:

```
Input: root = [1]
Output: [[1]]
```

Constraints:

- The number of nodes in the tree is in the range [1, 100].
- -100 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Collect leaves, remove them, repeat. Group nodes by their 'height from leaf' level. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Repeatedly find all current leaves (no children), remove them, append to result level.
# Each round peels one tree layer — like reverse level-order.
# Time: O(n^2) if we rescan whole tree each round. Space: O(n) output.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Post-order height: leaf=0, internal=max(left,right)+1. Group by height in one pass."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5]: height(4)=0→result=[[4]]. height(5)=0→[[4,5]]. height(2)=1→[[4,5],[2]].
# height(3)=0→[[4,5,3],[2]]. height(1)=2→[[4,5,3],[2],[1]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). One DFS pass — no repeated tree traversals.
# Given more time I'd ask: group by depth from root? That's standard level-order BFS."
```


Trees & BST

☐ #437 Path Sum III

MED

Open on LeetCode

Tree · DFS · Prefix Sum

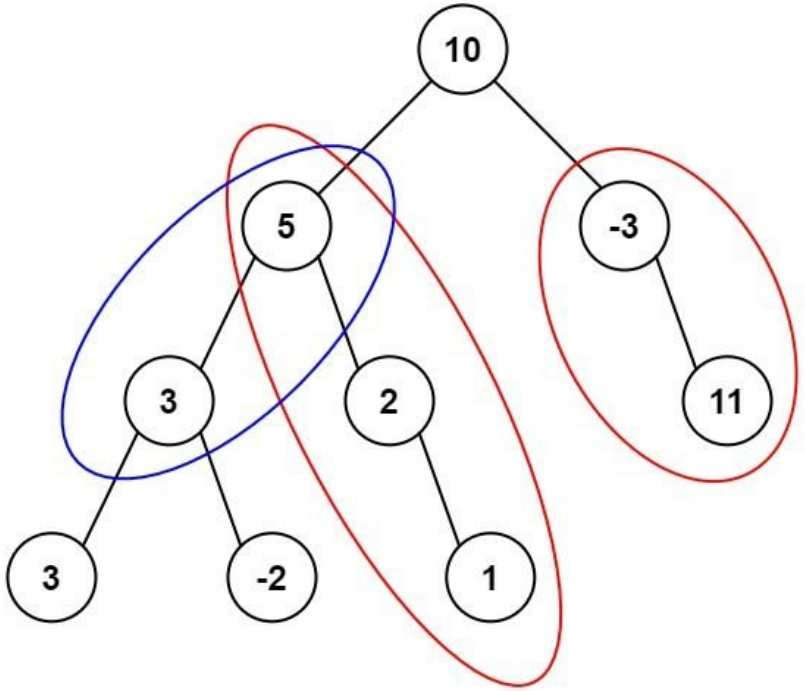
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return the number of paths where the sum of the values along the path equals targetSum.

The path does not need to start or end at the root or a leaf, but it must go downwards (i.e., traveling only from parent nodes to child nodes).

Example 1:




```
Input: root = [10,5,-3,3,2,null,11,3,-2,null,1], targetSum = 8
Output: 3
Explanation: The paths that sum to 8 are shown.
```

Example 2:

```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: 3
```

Constraints:

- The number of nodes in the tree is in the range [0, 1000].
- -109 <= Node.val <= 109
- -1000 <= targetSum <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count downward paths (parent→child only) summing to targetSum.
```

```
# Path can start and end at any node. Values can be negative. Right?"
#
# "[10,5,-3,3,2,null,11,3,-2,null,1], target=8 → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Prefix-sum map: as DFS walks, track cumulative sum from root.
# At each node, add count of prefix sums equal to (current_sum - target) - includes paths through node.
# Backtrack prefix counts when leaving node.
# Time: O(n). Space: O(n) prefix map + stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (prefix sum + hash map — same pattern as #560)
# "DFS with running prefix sum. Count how many earlier prefix sums equal current_sum-target.
# Backtrack after each subtree to avoid polluting sibling paths. Seed {0:1}."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# This is #560 applied to trees — same prefix-sum logic, but backtrack after each subtree.
# root=10, target=8: paths [5,3], [5,2,1], [-3,11] each sum to 8 → count=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Identical to Subarray Sum Equals K applied to a tree.
# Backtracking prevents cross-subtree contamination — critical difference from array version.
# Given more time I'd ask: paths going in both directions (not just downward)?
# Use the any-direction diameter-style post-order approach."
```


Trees & BST

☐ ★ #545 Boundary of Binary Tree ★

ME
D

Open on LeetCode

Tree · DFS

Lists: ★ Premium | Premium 98

Problem

The boundary of a binary tree is the concatenation of the root, the left boundary, the leaves ordered from left-to-right, and the reverse order of the right boundary.

The left boundary is the set of nodes defined by the following:

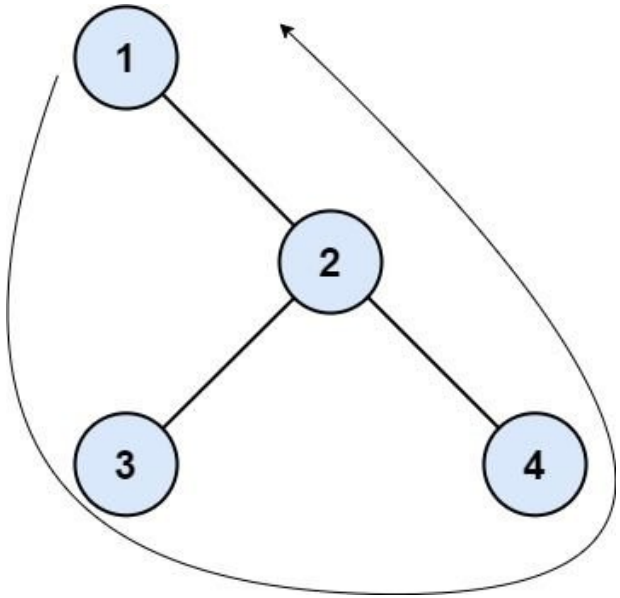
- The root node's left child is in the left boundary. If the root does not have a left child, then the left boundary is empty.
- If a node is in the left boundary and has a left child, then the left child is in the left boundary.
- If a node is in the left boundary, has no left child, but has a right child, then the right child is in the left boundary.
- The leftmost leaf is not in the left boundary.

The right boundary is similar to the left boundary, except it is the right side of the root's right subtree. Again, the leaf is not part of the right boundary, and the right boundary is empty if the root does not have a right child.

The leaves are nodes that do not have any children. For this problem, the root is not a leaf.

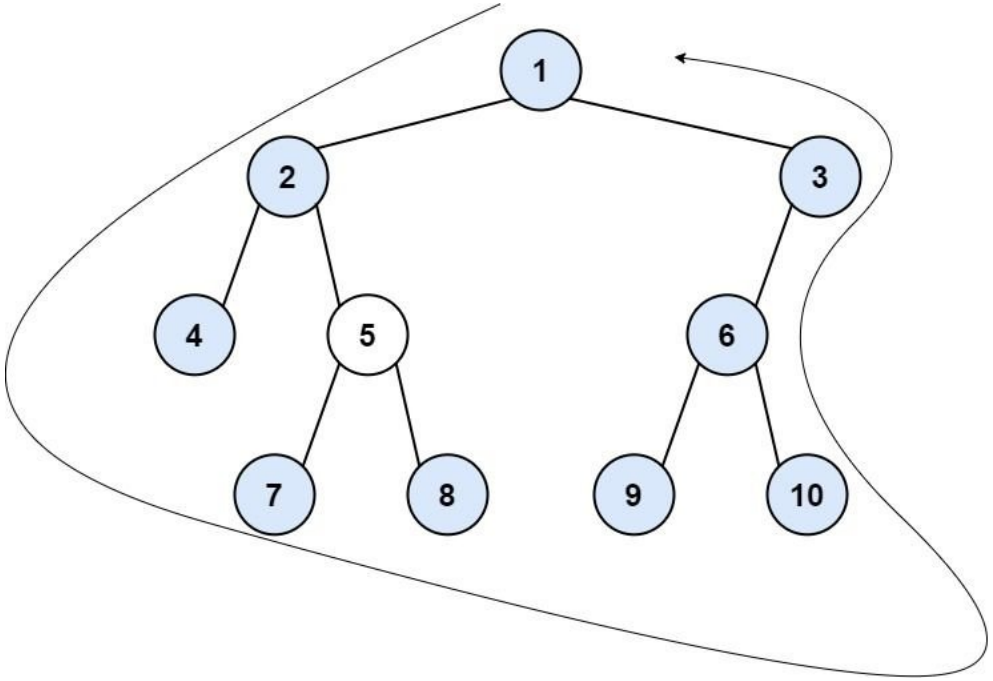
Given the root of a binary tree, return the values of its boundary.

Example 1:




```
Input: root = [1,null,2,3,4]
Output: [1,3,4,2]
Explanation:
- The left boundary is empty because the root does not have a left child.
- The right boundary follows the path starting from the root's right child 2 -> 4.
4 is a leaf, so the right boundary is [2].
- The leaves from left to right are [3,4].
Concatenating everything results in [1] + [] + [3,4] + [2] = [1,3,4,2].
```

Example 2:




```
Input: root = [1,2,3,4,5,6,null,null,null,7,8,9,10]
Output: [1,2,4,7,8,9,10,6,3]
Explanation:
- The left boundary follows the path starting from the root's left child 2 -> 4.
4 is a leaf, so the left boundary is [2].
- The right boundary follows the path starting from the root's right child 3 -> 6 -> 10.
10 is a leaf, so the right boundary is [3,6], and in reverse order is [6,3].
- The leaves from left to right are [4,7,8,9,10].
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- -1000 <= Node.val <= 1000

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Boundary = root + left boundary (non-leaves top-down) + all leaves left-to-right
# + right boundary (non-leaves bottom-up). Leftmost/rightmost leaves are in leaves, not boundary. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Collect four pieces separately: root val, left boundary nodes top-down,
# all leaves left-to-right, right boundary nodes bottom-up.
# Merge while skipping duplicate leaves already counted.
# Time: O(n). Space: O(h) recursion for leaf collection.
# Should I go ahead and optimize?"


# PHASE 5 — TEST & DEBUG
# [1,null,2,3,4]: left=[]. leaves=[3,4]. right=[2]. → [1,3,4,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Three separate traversals, each linear.
# 'Prefer left else right' for left spine; 'prefer right else left' for right spine — reversed at end.
# Given more time I'd ask: return boundary clockwise vs counter-clockwise? Just reverse the result."
```


Trees & BST

★ #549 Binary Tree Longest Consecutive Sequence II ★

ME
D

Open on LeetCode

Tree · DFS

Lists: ★ Premium | Premium 98

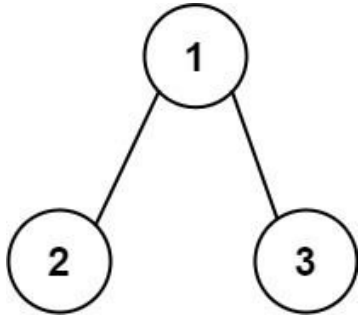
Problem

Given the root of a binary tree, return the length of the longest consecutive path in the tree.
A consecutive path is a path where the values of the consecutive nodes in the path differ by one. This path can be either increasing or decreasing.

- For example, [1,2,3,4] and [4,3,2,1] are both considered valid, but the path [1,2,4,3] is not valid.

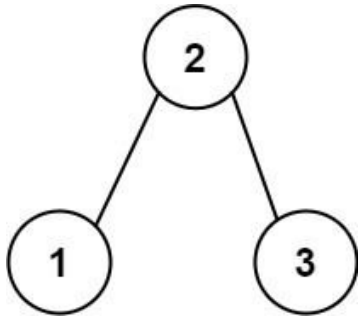
On the other hand, the path can be in the child-Parent-child order, where not necessarily be parent-child order.

Example 1:




```
Input: root = [1,2,3]
Output: 2
Explanation: The longest consecutive path is [1, 2] or [2, 1].
```

Example 2:




```
Input: root = [2,1,3]
Output: 3
Explanation: The longest consecutive path is [1, 2, 3] or [3, 2, 1].
```

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 104].
- -3 * 104 <= Node.val <= 3 * 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Longest path where consecutive values differ by 1 — can go up OR down, can change direction at one pivot. Right?"
#
# "[2,1,3] → [1,2,3] length 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# At each node, run DFS tracking consecutive count when child.val == parent.val + 1.
```

```
# Reset count when streak breaks. Track global max streak length.
# Visits every node once with O(n) DFS per starting node worst case O(n^2).
# Time: O(n^2) naive. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (post-order returning inc/dec lengths)
# "Each node returns (inc, dec): length of increasing/decreasing sequence going DOWN from it.
# Diameter through node = best combination of one child's inc and another's dec, meeting at this node."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [2,1,3]: dfs(1)=(1,1). dfs(3)=(1,1). dfs(2): left(1)=val-1→dec=2. right(3)=val+1→inc=2. best=2+2-1=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). inc+dec-1 avoids double-counting the pivot node.
# Generalises #298 by allowing direction change at the pivot node.
# Given more time I'd ask: allow path to go up? Needs full graph treatment."
```


Trees & BST

★ #582 Kill Process ★

ME
D

Open on LeetCode

Array · Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

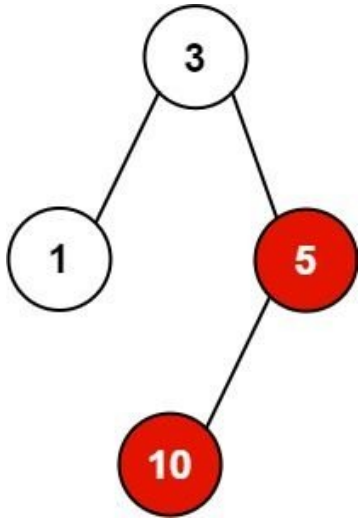
You have n processes forming a rooted tree structure. You are given two integer arrays pid and ppid, where pid[i] is the ID of the ith process and ppid[i] is the ID of the ith process's parent process.

Each process has only one parent process but may have multiple children processes. Only one process has ppid[i] = 0, which means this process has no parent process (the root of the tree).

When a process is killed, all of its children processes will also be killed.

Given an integer kill representing the ID of a process you want to kill, return a list of the IDs of the processes that will be killed. You may return the answer in any order.

Example 1:




```
Input: pid = [1,3,10,5], ppid = [3,0,5,3], kill = 5
Output: [5,10]
Explanation: The processes colored in red are the processes that should be killed.
```

Example 2:

```
Input: pid = [1], ppid = [0], kill = 1
Output: [1]
```

Constraints:

- n == pid.length
- n == ppid.length
- 1 <= n <= 5 * 104
- 1 <= pid[i] <= 5 * 104
- 0 <= ppid[i] <= 5 * 104
- Only one process has no parent.
- All the values of pid are unique.
- kill is guaranteed to be in pid.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Kill a process and all its descendants. Return all killed process IDs. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build children map and indegree from the kill-pair list.
```

```
# BFS from the killed pid: enqueue all its direct children, then propagate kills downward.
# Every reachable process dies - simulates the cascade.
# Time: O(n). Space: O(n) for graph + queue.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# pid=[1,3,10,5], ppid=[3,0,5,3], kill=5: children={3:[1,5],5:[10]}.
# BFS from 5: [5,10] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Build adjacency list then BFS — standard implicit tree traversal.
# Given more time I'd ask: kill all ancestors too? Walk parent pointers upward instead."
```


Trees & BST

#662 Maximum Width of Binary Tree

ME
D

Open on LeetCode

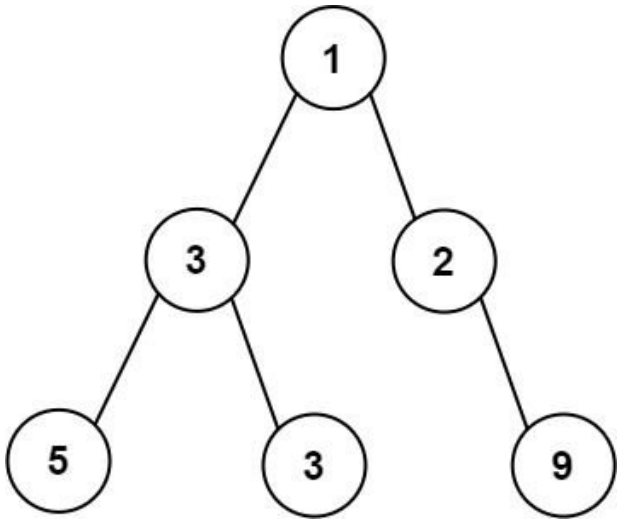
Tree · BFS

Lists: Grind 169

Problem

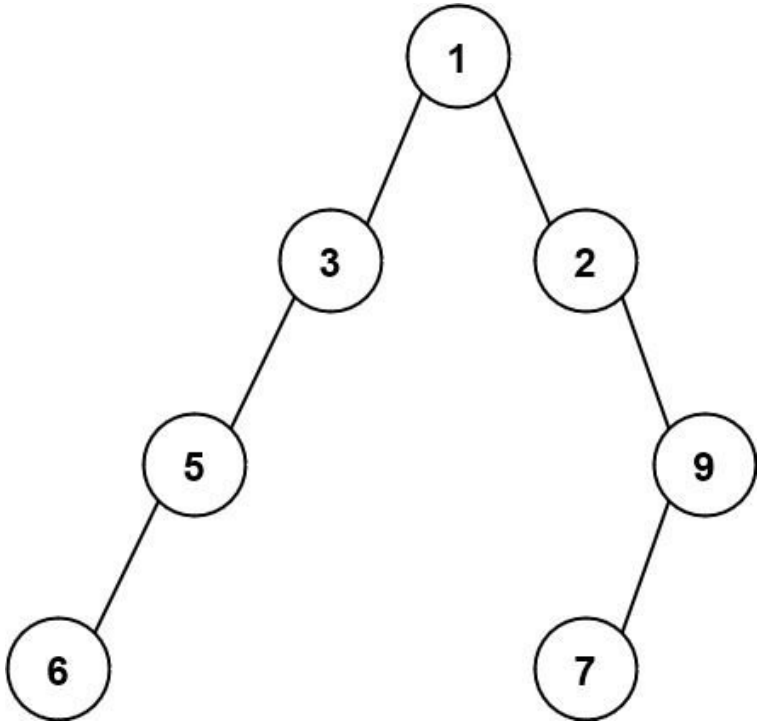
Given the root of a binary tree, return the maximum width of the given tree.
The maximum width of a tree is the maximum width among all levels.
The width of one level is defined as the length between the end-nodes (the leftmost and rightmost non-null nodes), where the null nodes between the end-nodes that would be present in a complete binary tree extending down to that level are also counted into the length calculation.
It is guaranteed that the answer will in the range of a 32-bit signed integer.

Example 1:



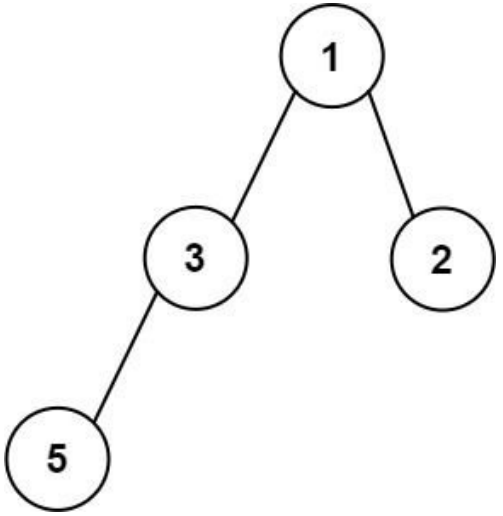

```
Input: root = [1,3,2,5,3,null,9]
Output: 4
Explanation: The maximum width exists in the third level with length 4 (5,3,null,9).
```

Example 2:




```
Input: root = [1,3,2,5,null,null,9,6,null,7]
Output: 7
Explanation: The maximum width exists in the fourth level with length 7 (6,null,null,null,null,null,7).
```

Example 3:




```
Input: root = [1,3,2,5]
Output: 2
Explanation: The maximum width exists in the second level with length 2 (3,2).
```

Constraints:

- The number of nodes in the tree is in the range [1, 3000].
- -100 <= Node.val <= 100

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Width = rightmost_pos - leftmost_pos + 1 at each level, counting null gaps.
# Left child = 2*pos, right child = 2*pos+1. Normalise each level to prevent overflow. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS level-order traversal; at each level track min and max index assigned to nodes.
# Width of level = max_index - min_index + 1; answer is global max width.
# Straightforward queue-based level scan.
# Time: O(n). Space: O(w) queue width.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# [1,3,2,5,3,null,9]: Level 2 positions (normalised): 5→0, 3→1, 9→3. last=3. width=4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(w). Normalisation prevents 2^depth overflow for deep trees — critical.
# Given more time I'd ask: width by actual node count (ignoring nulls)? Just max level_size in BFS."
```

Trees & BST

#863 All Nodes Distance K in Binary Tree

MED

Open on LeetCode

Hash Table · Tree · DFS · BFS

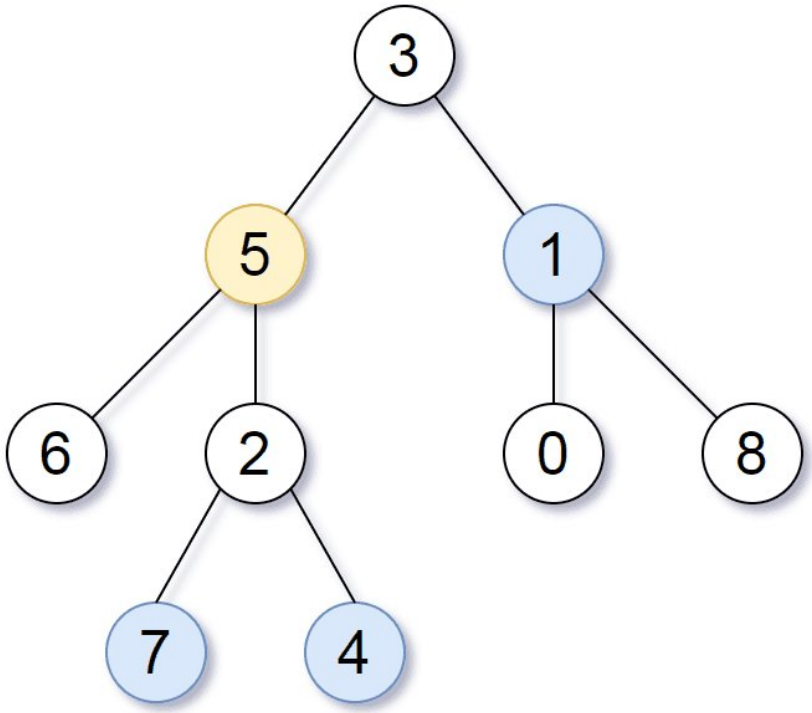
Lists: Grind 169

Problem

Given the root of a binary tree, the value of a target node target, and an integer k, return an array of the values of all nodes that have a distance k from the target node.

You can return the answer in any order.

Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], target = 5, k = 2
Output: [7,4,1]
Explanation: The nodes that are a distance 2 from the target node (with value 5) have values 7, 4, and 1.
```

Example 2:

```
Input: root = [1], target = 1, k = 3
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [1, 500].
- 0 <= Node.val <= 500
- All the values Node.val are unique.
- target is the value of one of the nodes in the tree.
- 0 <= k <= 1000

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```
# "Find all nodes exactly k edges from target. Distance can go upward through parent. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build parent map via BFS/DFS from target. For each query node, walk parent pointers
# until reaching target; collect distance. O(n) per query naive.
# Time: O(n * Q) for Q queries. Space: O(n) parent map.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (build parent map then BFS)
# "DFS to build parent[node]=parent. Then BFS from target treating tree as undirected graph.
# Stop collecting at distance k."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# target=5, k=2: BFS outward from 5 by 2 hops → [7,4,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Parent map converts tree to undirected graph — then BFS is trivial.
# Given more time I'd ask: multiple targets? Multi-source BFS — enqueue all at distance 0."
```

Trees & BST

★ #1120 Maximum Average Subtree ★

MED

Open on LeetCode

Tree · DFS

Lists: ★ Premium | Premium 98

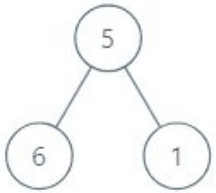
Problem

Given the root of a binary tree, return the maximum average value of a subtree of that tree. Answers within 10-5 of the actual answer will be accepted.

A subtree of a tree is any node of that tree plus all its descendants.

The average value of a tree is the sum of its values, divided by the number of nodes.

Example 1:




```
Input: root = [5,6,1]
Output: 6.00000
Explanation:
For the node with value = 5 we have an average of (5 + 6 + 1) / 3 = 4.
For the node with value = 6 we have an average of 6 / 1 = 6.
For the node with value = 1 we have an average of 1 / 1 = 1.
So the answer is 6 which is the maximum.
```

Example 2:

```
Input: root = [0,null,1]
Output: 1.00000
```

Constraints:

- The number of nodes in the tree is in the range [1, 104].
- 0 <= Node.val <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the subtree (any node + all descendants) with the maximum average value. Return the average. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Post-order DFS: for each node return (sum, count) of its subtree.
# Track max average = sum/count over all visited nodes.
# Recomputes subtree sums independently — correct O(n) post-order.
# Time: O(n). Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# [5,6,1]: avg(6)=6, avg(1)=1, avg(5,6,1)=4. → 6.0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Post-order carries (sum, count) up — compute average at each node.
# Given more time I'd ask: minimum average subtree? Track min instead of max."
```


Trees & BST

★ #1490 Clone N-ary Tree ★

MED

Open on LeetCode

Hash Table · Tree · DFS · BFS

Lists: ★ Premium | Premium 98

Problem

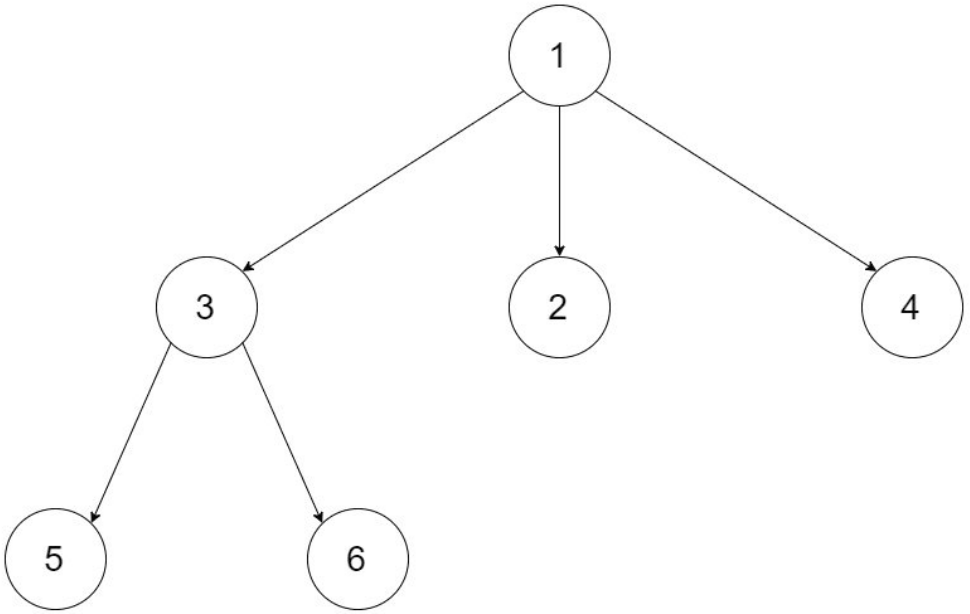
Given a root of an N-ary tree, return a deep copy (clone) of the tree.

Each node in the n-ary tree contains a val (int) and a list (List[Node]) of its children.

```
class Node {
public int val;
public List<Node> children;
}
```

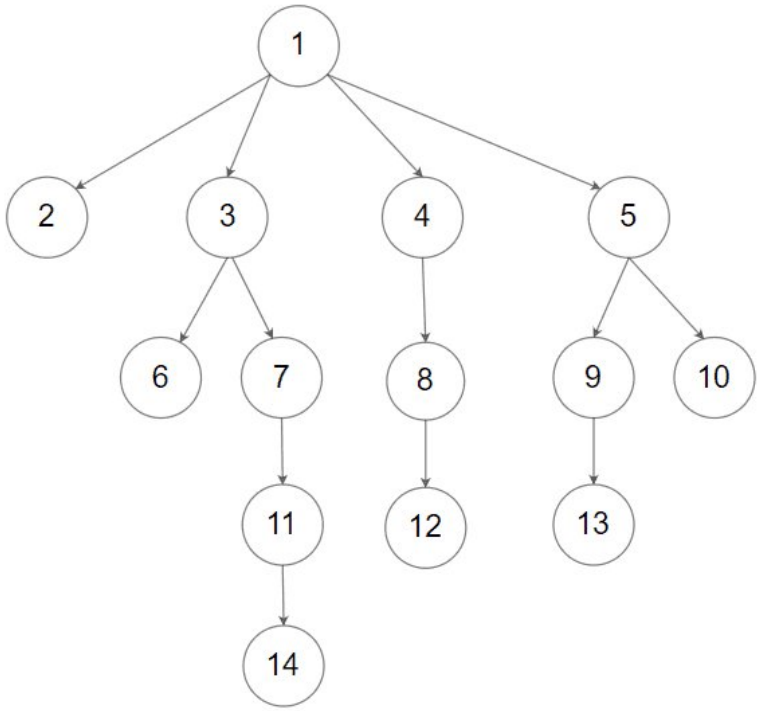
Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value (See examples).

Example 1:




```
Input: root = [1,null,3,2,4,null,5,6]
Output: [1,null,3,2,4,null,5,6]
```

Example 2:




```
Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
```

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [0, 104].

Follow up: Can your solution work for the graph problem?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Deep clone an N-ary tree. No shared references between original and clone. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS clone: map each original node to a new node, recursively clone all children lists.
# First pass creates nodes on demand; second pass wires child pointers.
# Time: O(n). Space: O(n) for map + recursion.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Simple recursive clone — no visited map needed (trees have no cycles).
# Follow-up (Clone Graph #133): needs visited map to handle cycles — {original: clone}."
```


Trees & BST

★ #1522 Diameter of N-Ary Tree ★

MED

Open on LeetCode

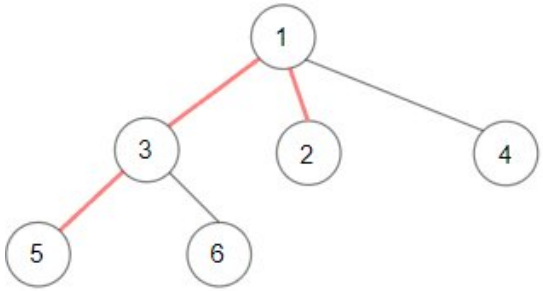
Tree · DFS

Lists: ★ Premium | Premium 98

Problem

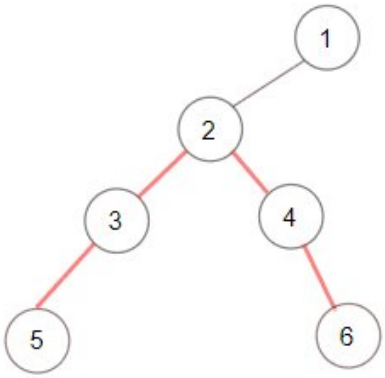
Given a root of an N-ary tree, you need to compute the length of the diameter of the tree.
The diameter of an N-ary tree is the length of the longest path between any two nodes in the tree. This path may or may not pass through the root.
(Nary-Tree input serialization is represented in their level order traversal, each group of children is separated by the null value.)

Example 1:



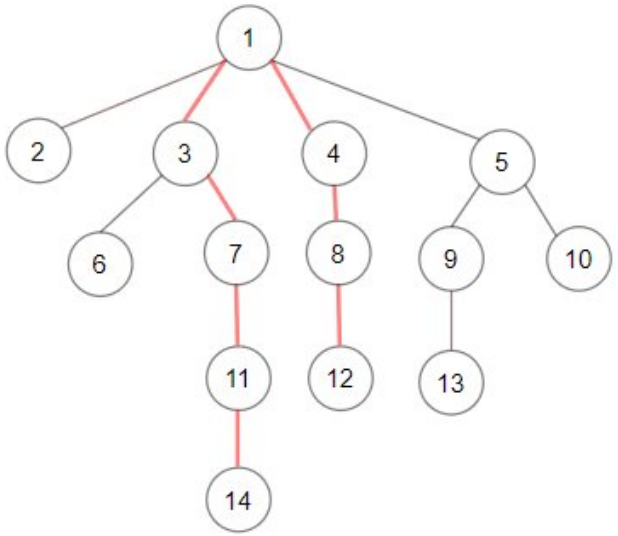

```
Input: root = [1,null,3,2,4,null,5,6]
Output: 3
Explanation: Diameter is shown in red color.
```

Example 2:




```
Input: root = [1,null,2,null,3,4,null,5,null,6]
Output: 4
```

Example 3:




```
Input: root = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: 7
```

Constraints:

- The depth of the n-ary tree is less than or equal to 1000.
- The total number of nodes is between [1, 104].

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Diameter = longest path between any two nodes (in edges), generalised to N children. Right?
# Same concept as Binary Tree Diameter (#543)."
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Two-pass DFS diameter: for each node, longest path through it = sum of top two child depths.
# Same as binary tree diameter but children list instead of left/right.
# Time: O(n). Space: O(h) stack.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE
# "Post-order: for each node, collect heights of children (adding 1 for the edge to parent).
# Diameter through this node = top two child-heights summed.
# Node's own height = max child-height (or 0 for leaf)."
# PHASE 4 — CLEAN CODE
        # child_h+1 converts "height below child" to "edges from this node via that child"
# PHASE 5 — TEST & DEBUG
# [1,null,3,2,4,null,5,6]: height(5)=0, height(6)=0.
# height(3): child_heights=[0+1,0+1]=[1,1]. best=1+1=2. return 1.
# height(2)=0, height(4)=0.
# height(1): child_heights=[1+1,0+1,0+1]=[2,1,1]. best=2+1=3. return 2. → 3 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Adding +1 when collecting child heights is the key difference from #543.
# In binary tree, depth(child) already includes the edge up; here we add it explicitly.
# Given more time I'd ask: diameter in edge-weighted N-ary tree?
# Same approach — just use edge weights instead of +1."
```


Trees & BST

★ #1650 Lowest Common Ancestor of a Binary Tree III ★

ME
D

Open on LeetCode

Hash Table · Tree · Two Pointers

Lists: ★ Premium | Premium 98

Problem

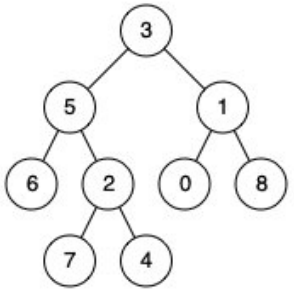
Given two nodes of a binary tree p and q, return their lowest common ancestor (LCA).

Each node will have a reference to its parent node. The definition for Node is below:

```
class Node {
public int val;
public Node left;
public Node right;
public Node parent;
}
```

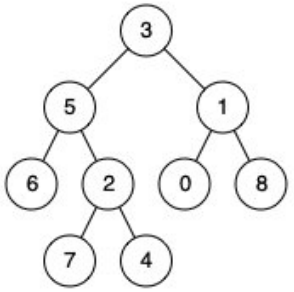
According to the definition of LCA on Wikipedia: "The lowest common ancestor of two nodes p and q in a tree T is the lowest node that has both p and q as descendants (where we allow a node to be a descendant of itself)."

Example 1:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 1
Output: 3
Explanation: The LCA of nodes 5 and 1 is 3.
```

Example 2:




```
Input: root = [3,5,1,6,2,0,8,null,null,7,4], p = 5, q = 4
Output: 5
Explanation: The LCA of nodes 5 and 4 is 5 since a node can be a descendant of itself according to the LCA definition.
```

Example 3:

```
Input: root = [1,2], p = 1, q = 2
Output: 1
```

Constraints:

- The number of nodes in the tree is in the range [2, 105].

- **-109 <= Node.val <= 109**
- **All Node.val are unique.**
- **p != q**
- **p and q exist in the tree.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Nodes have a parent pointer. Find LCA without the root. Right?
# Both p and q are guaranteed to be in the tree."
#
# "[3,5,1,...], p=5, q=1 → LCA=3 ✓ (same as #236 but solved via parent pointers)"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Each node has a parent pointer. Walk both nodes up with a visited set until they meet.
# First node seen twice is LCA. Like intersecting two linked lists.
# Time: O(h). Space: O(h) visited set.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (two-pointer linked list intersection)
# "Walk p upward and q upward simultaneously. When one hits null, restart at the other's origin.
# They meet at the LCA — same as finding intersection of two linked lists.
# Each pointer traverses: (path from p to root) + (path from q to root), length l_p + l_q.
# Both pointers traverse the same total length → they meet at LCA."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# p=5, q=1: a traverses 5→3→None→1→3. b traverses 1→3→None→5→3. Both reach 3 simultaneously ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(h). Space O(1). This is the two-pointer intersection trick from Linked List Intersection (#160).
# Both pointers traverse depth(p) + depth(q) steps total and meet at LCA.
# Given more time I'd ask: what if the tree is very deep and we want to avoid the full traversal?
# Use a set: walk p to root recording all ancestors, then walk q to root and stop at first ancestor in the set.
# O(depth) time, O(depth) space — same asymptotic but possibly faster in practice."
```


DFS

Round 2 · High · Medium · 16 questions

DFS

#130 Surrounded Regions

MED

Open on LeetCode

Array · DFS · BFS · Union Find · Matrix

Lists: Denny Zhang

Problem

You are given an m x n matrix board containing letters 'X' and 'O', capture regions that are surrounded:

- Connect: A cell is connected to adjacent cells horizontally or vertically.
- Region: To form a region connect every 'O' cell.
- Surround: A region is surrounded if none of the 'O' cells in that region are on the edge of the board. Such regions are completely enclosed by 'X' cells.

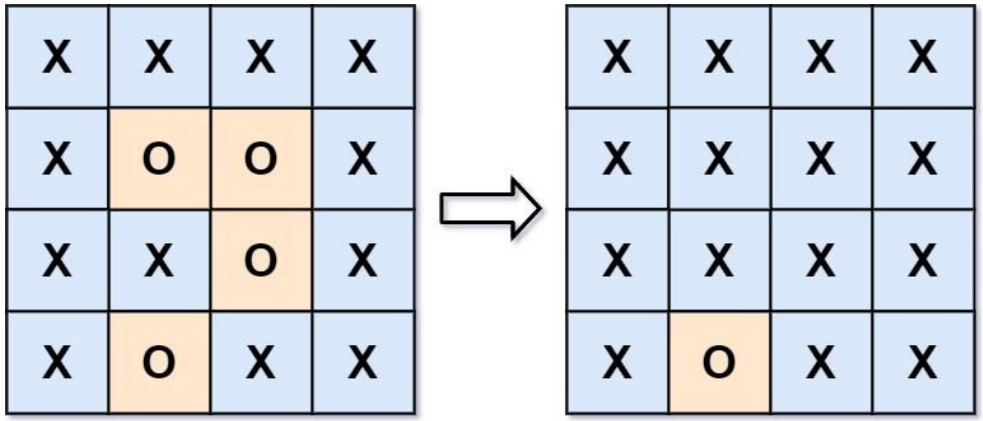
To capture a surrounded region, replace all 'O's with 'X's in-place within the original board. You do not need to return anything.

Example 1:

Input: board = [["X","X","X","X"],["X","O","O","X"],["X","X","O","X"],["X","O","X","X"]]

Output: [["X","X","X","X"],["X","X","X","X"],["X","X","X","X"],["X","O","X","X"]]

Explanation:



In the above diagram, the bottom region is not captured because it is on the edge of the board and cannot be surrounded.

Example 2:

Input: board = [["X"]]

Output: [["X"]]

Constraints:

- m == board.length
- n == board[i].length
- 1 <= m, n <= 200
- board[i][j] is 'X' or 'O'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Capture surrounded 'O' regions by replacing them with 'X'.
# A region is NOT captured if any 'O' in it touches the board edge. Right?
# In-place modification — no return value."
#
# "Border-connected 'O's and their inland 'O' neighbours are safe. Everything else gets flipped."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# First pass: DFS from every border 'O' and mark connected cells as safe ('S').
# Second pass: flip remaining 'O' to 'X'; restore safe cells back to 'O'.
# Two-pass flood fill from boundary — standard O(m*n) approach.
# Time: O(m * n). Space: O(m * n) recursion.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (reverse: mark safe, then flip)
```



```
# "DFS from every border 'O', marking connected 'O's as safe ('#').
# Second pass: 'O' → 'X' (captured), '#' → 'O' (safe restored). O(m*n)."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) stack. The reverse-DFS from borders is the key insight —
# instead of deciding which 'O's are captured, decide which are SAFE and flip the rest.
# Given more time I'd ask: what if the grid wraps (toroidal)? No borders → all 'O's are safe."
```

DFS

#133 Clone Graph

MED

Open on LeetCode

Hash Table · DFS · BFS · Graph

Lists: Grind 169

Problem

Given a reference of a node in a connected undirected graph.

Return a deep copy (clone) of the graph.

Each node in the graph contains a value (int) and a list (List[Node]) of its neighbors.

```
class Node {
public int val;
public List<Node> neighbors;
}
```

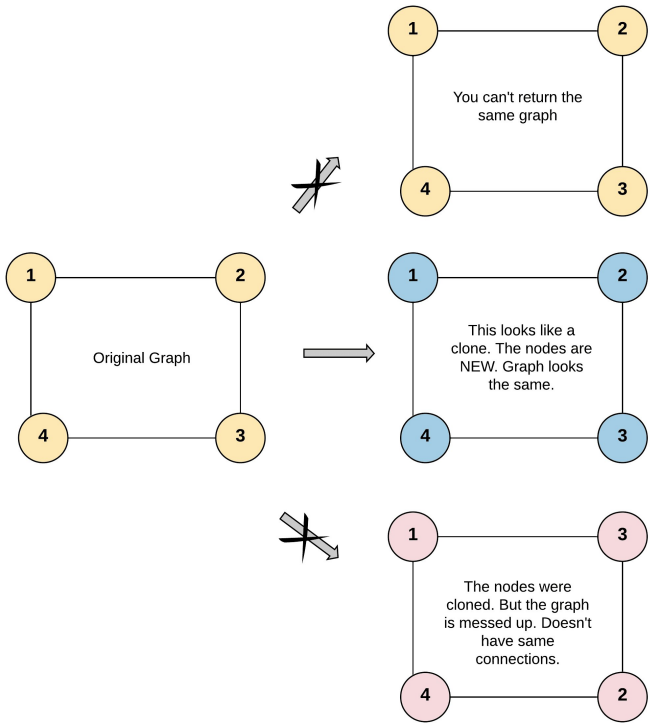
Test case format:

For simplicity, each node's value is the same as the node's index (1-indexed). For example, the first node with val == 1, the second node with val == 2, and so on. The graph is represented in the test case using an adjacency list.

An adjacency list is a collection of unordered lists used to represent a finite graph. Each list describes the set of neighbors of a node in the graph.

The given node will always be the first node with val = 1. You must return the copy of the given node as a reference to the cloned graph.

Example 1:




```
Input: adjList = [[2,4],[1,3],[2,4],[1,3]]
Output: [[2,4],[1,3],[2,4],[1,3]]
Explanation: There are 4 nodes in the graph.
1st node (val = 1)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
2nd node (val = 2)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
3rd node (val = 3)'s neighbors are 2nd node (val = 2) and 4th node (val = 4).
4th node (val = 4)'s neighbors are 1st node (val = 1) and 3rd node (val = 3).
```

Example 2:


```
Input: adjList = [[]]
Output: [[]]
Explanation: Note that the input contains one empty list. The graph consists of only one node with val = 1 and it does not have any neighbors.
```

Example 3:

```
Input: adjList = []
Output: []
Explanation: This an empty graph, it does not have any nodes.
```

Constraints:

- The number of nodes in the graph is in the range [0, 100].
- 1 <= Node.val <= 100
- Node.val is unique for each node.
- There are no repeated edges and no self-loops in the graph.
- The Graph is connected and all nodes can be visited starting from the given node.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Deep copy an undirected connected graph. No shared references with original. Right?
# Graphs can have cycles — unlike trees, so we need a visited map."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS from the given node: maintain a map old_node -> new_node clone.
# When visiting neighbors, create clones on first sight and enqueue them.
# Straightforward graph copy — already the standard O(V+E) approach.
# Time: O(V + E). Space: O(V) for the clone map and queue.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V) for the visited map. Mark node before recursing into neighbours —
# without this, cycles cause infinite recursion. Same visited-map pattern handles any graph.
# Given more time I'd ask: clone a directed graph? Same approach — just don't add reverse edges."
```

DFS

#200 Number of Islands

MED

Open on LeetCode

Array · DFS · BFS · Union Find · Matrix

Lists: Grind 169

Problem

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands.
An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

```
Input: grid = [
["1","1","1","1","0"],
["1","1","0","1","0"],
["1","1","0","0","0"],
["0","0","0","0","0"]
]
Output: 1
```

Example 2:

```
Input: grid = [
["1","1","0","0","0"],
["1","1","0","0","0"],
["0","0","1","0","0"],
["0","0","0","1","1"]
]
Output: 3
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 300
- grid[i][j] is '0' or '1'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count connected components of '1's in a binary grid. Diagonal connections don't count. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Double loop over every grid cell. Whenever I see an unvisited '1',
# run DFS/BFS to mark the entire connected component sunk, then increment island count.
# Each cell visited once — correct and actually O(m*n) already.
# Time: O(m * n). Space: O(m * n) worst-case recursion stack or queue.
# The flood-fill is the right approach; I'll keep it clean with in-place marking."

# PHASE 5 — TEST & DEBUG
# 1-island grid: DFS floods the whole island. count=1 ✓. 3-island: count=3 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) stack. Flooding in-place avoids a separate visited set.
# BFS alternative: use a queue instead of recursion — better for very large grids (no stack overflow).
# Union-Find alternative: O(m*n * α(m*n)) — useful when islands merge dynamically.
# Given more time I'd ask: restore the grid after counting?
# Keep a separate visited set or restore '#' → '1' after each DFS."
```


DFS

#207 Course Schedule

MED

Open on LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return true if you can finish all courses. Otherwise, return false.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: true
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0. So it is possible.
```

Example 2:

```
Input: numCourses = 2, prerequisites = [[1,0],[0,1]]
Output: false
Explanation: There are a total of 2 courses to take.
To take course 1 you should have finished course 0, and to take course 0 you should also have finished course 1. So it is impossible.
```

Constraints:

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= 5000
- prerequisites[i].length == 2
- 0 <= ai, bi < numCourses
- All the pairs prerequisites[i] are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can all courses be finished given prerequisites (directed edges)?
# Equivalent to: does the directed graph have a cycle? Return True if no cycle. Right?"
#
# "[[1,0],[0,1]]: 0→1 and 1→0 form a cycle → False ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build adjacency + indegree; Kahn BFS peels zero-indegree courses layer by layer.
# If we finish fewer than numCourses nodes, a cycle exists ? return false.
# Same topological sort as Course Schedule II, but only need boolean completion.
# Time: O(V + E). Space: O(V + E).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (3-colour DFS cycle detection)
# "Color each node: 0=unvisited, 1=in current DFS path (grey), 2=fully processed (black).
# Cycle exists if we encounter a grey node. O(V+E)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,0],[0,1]]: has_cycle(0)→grey. has_cycle(1)→grey. has_cycle(0)→grey(already) → cycle! ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). The 3-colour scheme distinguishes 'seen in current path' from 'fully done'.
# Kahn's BFS alternative (in-degree tracking): count = courses processed; if count < numCourses → cycle.
# Given more time I'd ask: Course Schedule II (#210)? Same DFS, collect reverse post-order."
```

DFS

#210 Course Schedule II

ME D

Open on LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

There are a total of numCourses courses you have to take, labeled from 0 to numCourses - 1. You are given an array prerequisites where prerequisites[i] = [ai, bi] indicates that you must take course bi first if you want to take course ai.

- For example, the pair [0, 1], indicates that to take course 0 you have to first take course 1.

Return the ordering of courses you should take to finish all courses. If there are many valid answers, return any of them. If it is impossible to finish all courses, return an empty array.

Example 1:

```
Input: numCourses = 2, prerequisites = [[1,0]]
Output: [0,1]
Explanation: There are a total of 2 courses to take. To take course 1 you should have finished course 0. So the correct course order is [0,1].
```

Example 2:

```
Input: numCourses = 4, prerequisites = [[1,0],[2,0],[3,1],[3,2]]
Output: [0,2,1,3]
Explanation: There are a total of 4 courses to take. To take course 3 you should have finished both courses 1 and 2. Both courses 1 and 2 should be taken after you finished course 0.
So one correct course order is [0,1,2,3]. Another correct ordering is [0,2,1,3].
```

Example 3:

```
Input: numCourses = 1, prerequisites = []
Output: [0]
```

Constraints:

- 1 <= numCourses <= 2000
- 0 <= prerequisites.length <= numCourses * (numCourses - 1)
- prerequisites[i].length == 2
- 0 <= ai, bi < numCourses
- ai != bi
- All the pairs [ai, bi] are distinct.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return a valid course ordering. Return [] if impossible (cycle exists). Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Topological sort via Kahn's BFS: compute indegree of every course,
# enqueue zero-indegree nodes, peel layers while decrementing neighbors.
# If we process all n courses, ordering exists; otherwise a cycle remains.
# Time: O(V + E). Space: O(V + E) for adjacency and indegree arrays.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). Post-order append then reverse = topological sort.
# Given more time I'd ask: Kahn's BFS? Build in-degree map, process zero-in-degree nodes first.
# Both approaches are O(V+E) — DFS is recursive, Kahn's is iterative."
```


DFS

★ #261 Graph Valid Tree ★

ME D

Open on LeetCode

DFS · BFS · Union Find · Graph

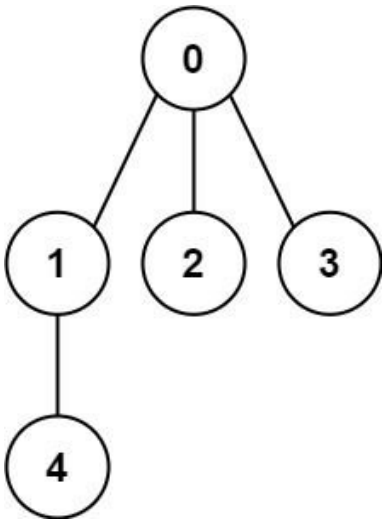
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes labeled from 0 to n - 1. You are given an integer n and a list of edges where edges[i] = [ai, bi] indicates that there is an undirected edge between nodes ai and bi in the graph.

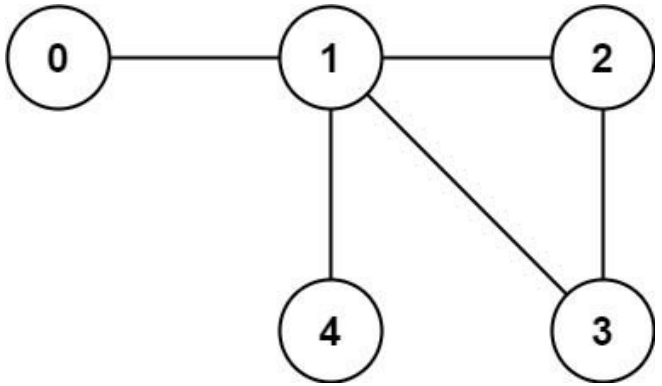
Return true if the edges of the given graph make up a valid tree, and false otherwise.

Example 1:




```
Input: n = 5, edges = [[0,1],[0,2],[0,3],[1,4]]
Output: true
```

Example 2:




```
Input: n = 5, edges = [[0,1],[1,2],[2,3],[1,3],[1,4]]
Output: false
```

Constraints:

- 1 <= n <= 2000
- 0 <= edges.length <= 5000
- edges[i].length == 2
- 0 <= ai, bi < n
- ai != bi
- There are no self-loops or repeated edges.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Valid tree: connected, acyclic, exactly n-1 edges. Right?
# Any two of these imply the third for n nodes."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build adjacency from edges, run DFS from node 0 marking visited.
# After DFS, verify every node was visited and total edges equals n-1.
# Catches disconnected graphs and cycles in one pass.
# Time: O(n). Space: O(n) for adjacency + visited.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# n=5, edges=[[0,1],[0,2],[0,3],[1,4]]: 4 edges==n-1 ✓. DFS from 0 visits all 5. No cycle. → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E)=O(n). Space O(n). Early return if edges≠n-1 is a fast filter.
# Union-Find alternative: O(n * α(n)) — merge edges, detect cycle if already connected.
# Given more time I'd ask: disconnected forest (multiple trees)?
# Return number of components instead — skip the len(edges)==n-1 check."
```


DFS

#310 Minimum Height Trees

ME
D

Open on LeetCode

DFS · BFS · Graph · Topological Sort

Lists: Grind 169

Problem

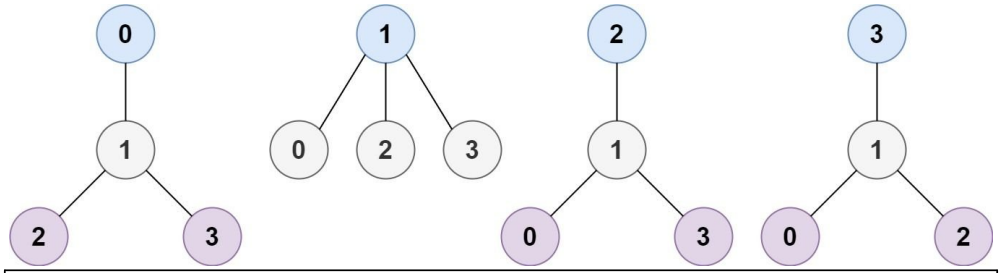
A tree is an undirected graph in which any two vertices are connected by exactly one path. In other words, any connected graph without simple cycles is a tree.

Given a tree of n nodes labelled from 0 to n - 1, and an array of n - 1 edges where edges[i] = [ai, bi] indicates that there is an undirected edge between the two nodes ai and bi in the tree, you can choose any node of the tree as the root. When you select a node x as the root, the result tree has height h. Among all possible rooted trees, those with minimum height (i.e. min(h)) are called minimum height trees (MHTs).

Return a list of all MHTs' root labels. You can return the answer in any order.

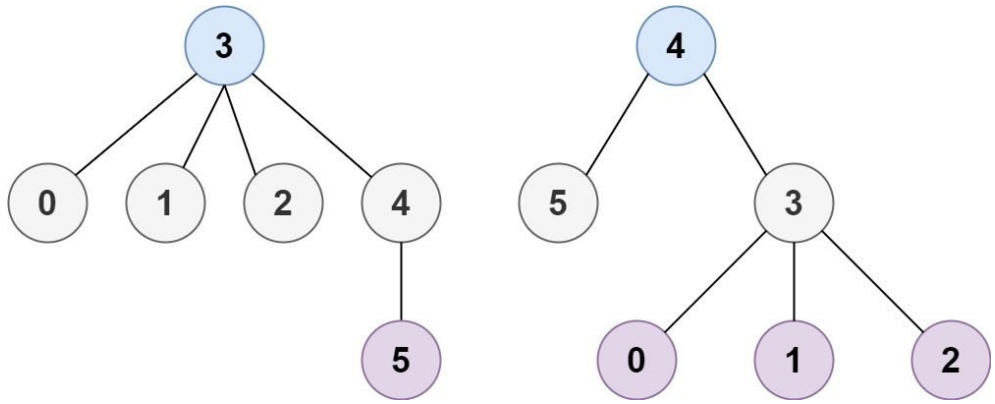
The height of a rooted tree is the number of edges on the longest downward path between the root and a leaf.

Example 1:




```
Input: n = 4, edges = [[1,0],[1,2],[1,3]]
Output: [1]
Explanation: As shown, the height of the tree is 1 when the root is the node with label 1 which is the only MHT.
```

Example 2:




```
Input: n = 6, edges = [[3,0],[3,1],[3,2],[3,4],[5,4]]
Output: [3,4]
```

Constraints:

- 1 <= n <= 2 * 104
- edges.length == n - 1
- 0 <= ai, bi < n
- ai != bi
- All the pairs (ai, bi) are distinct.
- The given input is guaranteed to be a tree and there will be no repeated edges.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find which nodes make MHT roots. The answer is at most 2 nodes — the tree's center(s). Right?
# Repeatedly prune leaf nodes (degree 1) until 1 or 2 nodes remain."
#
# "Like peeling an onion from outside in — the core is the answer."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Repeatedly peel all leaves layer by layer (nodes with degree 1), like peeling an onion.
# The last remaining nodes (0, 1, or 2) are the tree centroids / MHT roots.
# BFS leaf-removal is the standard O(n) approach for this problem.
# Time: O(n). Space: O(n) for adjacency and degree counts.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# n=4, edges=[[1,0],[1,2],[1,3]]: leaves=[0,2,3]. remaining=4-3=1. new_leaves=[1]. → [1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). This is topological sort on undirected tree — same BFS idea.
# At most 2 centers exist — if more, tree would have multiple independent long paths.
# Given more time I'd ask: all trees of minimum height (not just roots)?
# BFS from all roots found, check height — same O(n)."
```


DFS

★ #323 Number of Connected Components in an Undirected Graph ★

ME
D

Open on LeetCode

DFS · BFS · Union Find · Graph

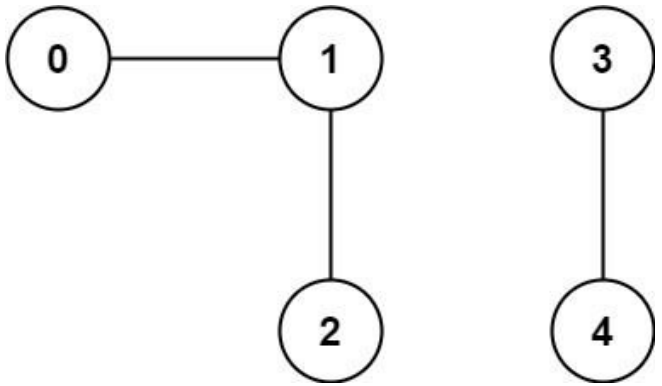
Lists: ★ Premium | Grind 169 | Premium 98

Problem

You have a graph of n nodes. You are given an integer n and an array edges where edges[i] = [ai, bi] indicates that there is an edge between ai and bi in the graph.

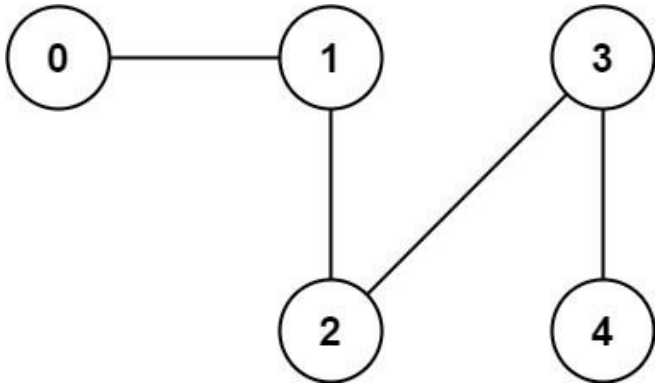
Return the number of connected components in the graph.

Example 1:




```
Input: n = 5, edges = [[0,1],[1,2],[3,4]]
Output: 2
```

Example 2:




```
Input: n = 5, edges = [[0,1],[1,2],[2,3],[3,4]]
Output: 1
```

Constraints:

- 1 <= n <= 2000
- 1 <= edges.length <= 5000
- edges[i].length == 2
- 0 <= ai <= bi < n
- ai != bi
- There are no repeated edges.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count connected components in an undirected graph. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Union-Find: for each edge, union the two endpoints.
# Answer = n minus number of distinct roots after all unions.
# Simple DSU with path compression — O(n alpha(n)).
# Time: O(n alpha(n)). Space: O(n) for parent array.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). Standard DFS counting components.
# Union-Find alternative: merge edges, count unique roots → O(n * α(n)).
# Given more time I'd ask: what if edges are added dynamically?
# Union-Find with path compression supports O(α(n)) per merge query."
```


DFS

#417 Pacific Atlantic Water Flow

ME
D

Open on LeetCode

Array · DFS · BFS · Matrix

Lists: Grind 169

Problem

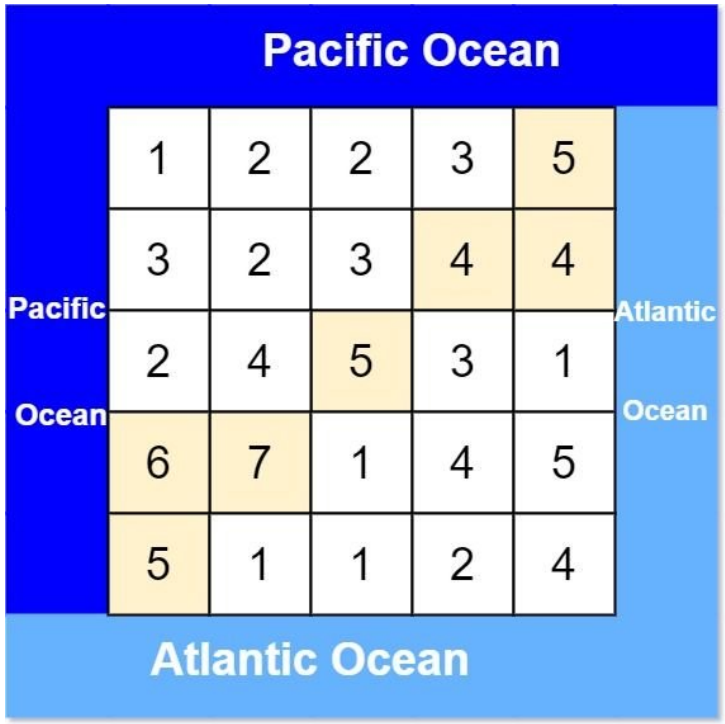
There is an m x n rectangular island that borders both the Pacific Ocean and Atlantic Ocean. The Pacific Ocean touches the island's left and top edges, and the Atlantic Ocean touches the island's right and bottom edges.

The island is partitioned into a grid of square cells. You are given an m x n integer matrix heights where heights[r][c] represents the height above sea level of the cell at coordinate (r, c).

The island receives a lot of rain, and the rain water can flow to neighboring cells directly north, south, east, and west if the neighboring cell's height is less than or equal to the current cell's height. Water can flow from any cell adjacent to an ocean into the ocean.

Return a 2D list of grid coordinates result where result[i] = [ri, ci] denotes that rain water can flow from cell (ri, ci) to both the Pacific and Atlantic oceans.

Example 1:




```
Input: heights = [[1,2,2,3,5],[3,2,3,4,4],[2,4,5,3,1],[6,7,1,4,5],[5,1,1,2,4]]
Output: [[0,4],[1,3],[1,4],[2,2],[3,0],[3,1],[4,0]]
Explanation: The following cells can flow to the Pacific and Atlantic oceans, as shown below:
[0,4]: [0,4] -> Pacific Ocean
  [0,4] -> Atlantic Ocean
[1,3]: [1,3] -> [0,3] -> Pacific Ocean
  [1,3] -> [1,4] -> Atlantic Ocean
[1,4]: [1,4] -> [1,3] -> [0,3] -> Pacific Ocean
```

Example 2:

```
Input: heights = [[1]]
Output: [[0,0]]
Explanation: The water can flow from the only cell to the Pacific and Atlantic oceans.
```

Constraints:

- m == heights.length
- n == heights[r].length
- 1 <= m, n <= 200
- 0 <= heights[r][c] <= 105

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Water flows from higher to lower (or equal). Find cells that can flow to BOTH oceans.
# Pacific touches top/left edges. Atlantic touches bottom/right edges. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# From each cell, DFS to see if it can reach both Pacific (top/left border) and Atlantic (bottom/right).
# Re-run DFS per cell — O((m*n)^2) naive.
# Time: O((m * n)^2) naive per-cell DFS. Space: O(m * n) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (reverse DFS from ocean borders)
# "Instead of simulating forward flow, run DFS BACKWARDS from each ocean's border cells.
# Reverse flow: go uphill (neighbor height >= current height).
# Intersection of Pacific-reachable and Atlantic-reachable = answer."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n). BFS from borders is cleaner than DFS (no recursion limit).
# The reversal insight: 'can reach ocean' → 'ocean can flow backward to this cell'.
# Given more time I'd ask: what if water can also flow diagonally?
# Add 4 diagonal directions — same algorithm."
```


DFS

★ #490 The Maze ★

MED

Open on LeetCode

Array · DFS · BFS · Matrix

Lists: ★ Premium | Premium 98

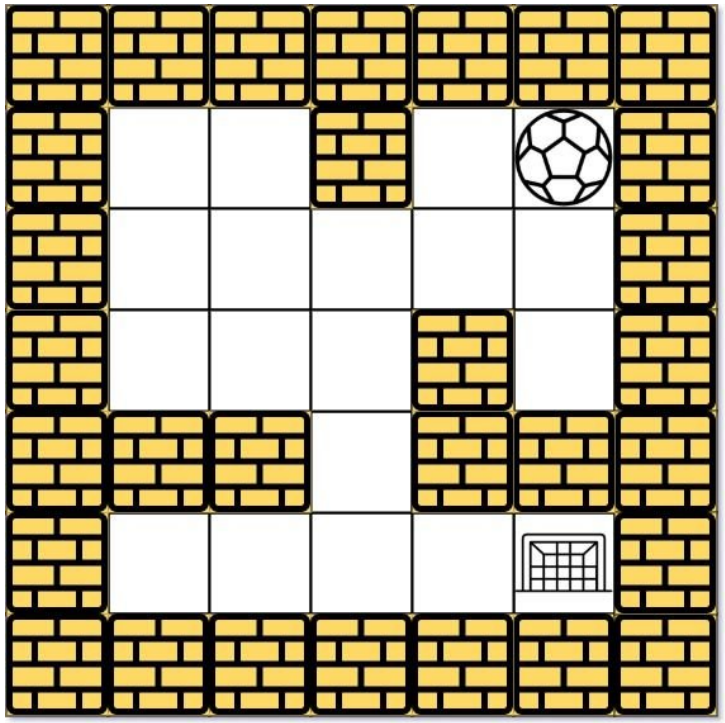
Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the m x n maze, the ball's start position and the destination, where start = [startrow, startcol] and destination = [destinationrow, destinationcol], return true if the ball can stop at the destination, otherwise return false.

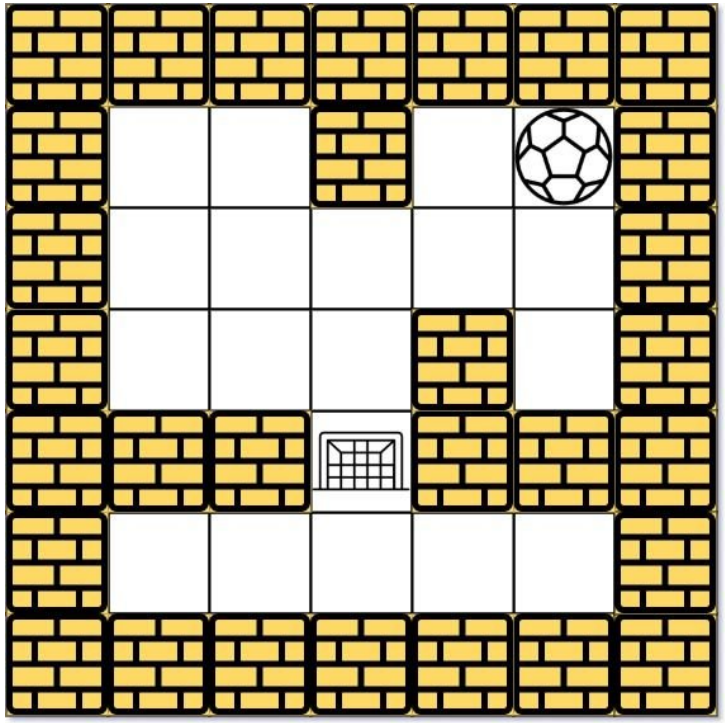
You may assume that the borders of the maze are all walls (see examples).

Example 1:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]
Output: true
Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.
```

Example 2:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [3,2]
Output: false
Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.
```

Example 3:

```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]
Output: false
```

Constraints:

- m == maze.length
- n == maze[i].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- start.length == 2
- destination.length == 2
- 0 <= startrow, destinationrow < m
- 0 <= startcol, destinationcol < n
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Ball rolls in a direction until hitting a wall. Can it STOP at the destination?
# It can pass through the destination but not stop there unless it hits a wall. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS from entrance: at each empty cell try all 4 directions.
# Mark visited cells to avoid cycles; succeed if we reach the boundary.
# Correct path exploration on an m x n grid.
# Time: O(m * n). Space: O(m * n) visited.
```



```
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# Ball rolls until hitting wall → stops → that stop position is a new state.
# DFS on stop-positions. Destination reached only if the ball stops exactly there. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*(m+n)) — each stop position visits O(m+n) cells to find next stop.
# The visited set is on stop-positions, not individual cells.
# Given more time I'd ask: The Maze (#505) — find shortest distance?
# Use Dijkstra with distance as weight, BFS/priority queue on stop-positions."
```

DFS

- [ ] ⋆ #694 Number of Distinct Islands ⋆ MED

Open on LeetCode

Hash Table · DFS · BFS · Union Find · Hash Function

Lists: ⋆ Premium | Premium 98

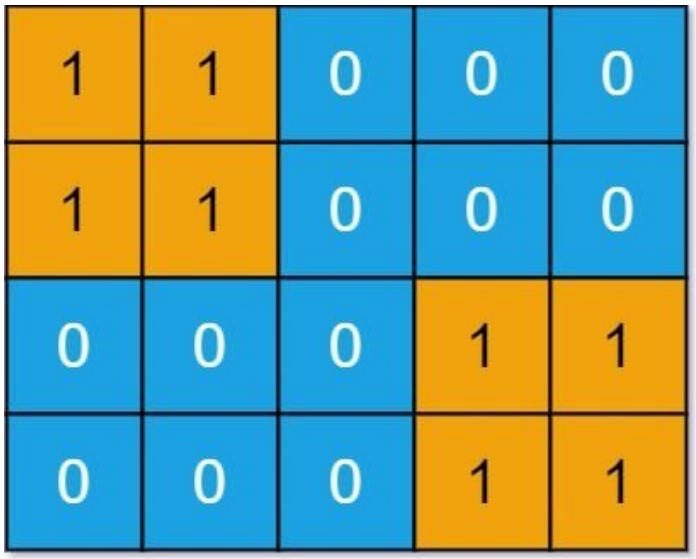
Problem

You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

An island is considered to be the same as another if and only if one island can be translated (and not rotated or reflected) to equal the other.

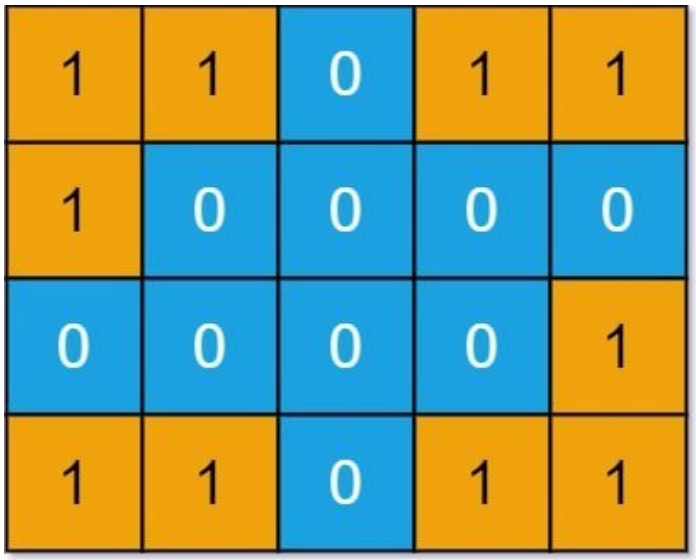
Return the number of distinct islands.

Example 1:




```
Input: grid = [[1,1,0,0,0],[1,1,0,0,0],[0,0,0,1,1],[0,0,0,1,1]]
Output: 1
```

Example 2:




```
Input: grid = [[1,1,0,1,1],[1,0,0,0,0],[0,0,0,0,1],[1,1,0,1,1]]
Output: 3
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- grid[i][j] is either 0 or 1.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Count islands that are distinct by shape (translation equivalent). Right?
# Two islands are the same if one can be shifted to match the other — no rotation/reflection."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS each island; encode shape as a string of moves (U/D/L/R) from start cell.
# Add shape string to a set — distinct shapes = set size.
# Must normalize starting cell and direction encoding consistently.
# Time: O(m * n). Space: O(m * n) shapes.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (path signature hashing)
# "DFS each island, record the path taken relative to the starting cell.
# Include backtrack markers to distinguish shapes like 'L' vs reverse 'L'.
# Hash each shape as a tuple and add to a set — count unique shapes."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# Two 2×2 islands: both DFS in same order → same relative path → same tuple → 1 distinct ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n). Backtrack None markers distinguish different DFS orderings
# that could produce the same sequence of offsets for different shapes.
# Given more time I'd ask: allow rotation/reflection? Normalise shape to lexicographically smallest
# among all 8 transformations — significantly more complex."
```

DFS

#695 Max Area of Island

MED

Open on LeetCode

Array · DFS · BFS · Union Find · Matrix

Lists: Denny Zhang

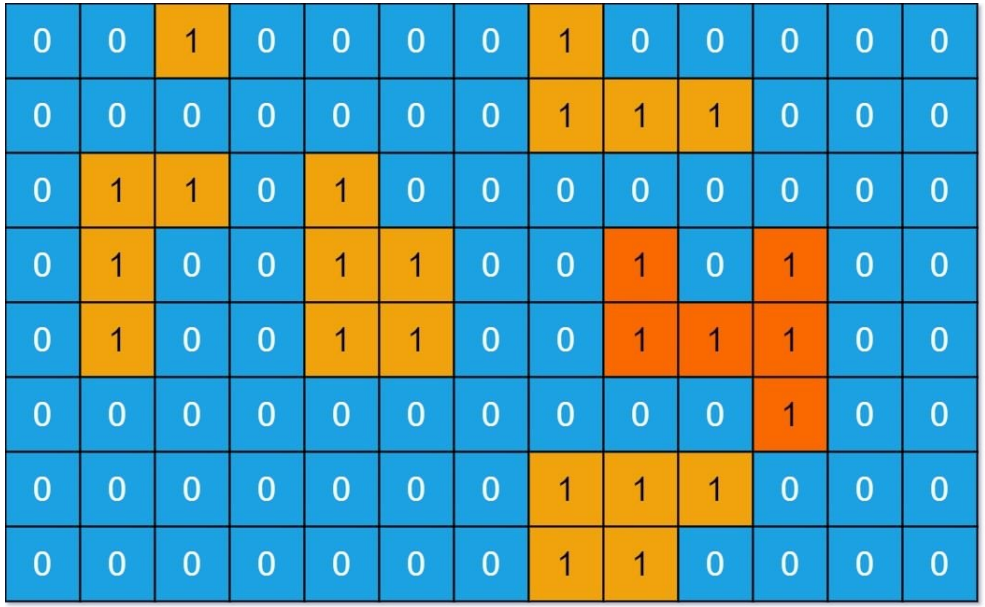
Problem

You are given an m x n binary matrix grid. An island is a group of 1's (representing land) connected 4-directionally (horizontal or vertical.) You may assume all four edges of the grid are surrounded by water.

The area of an island is the number of cells with a value 1 in the island.

Return the maximum area of an island in grid. If there is no island, return 0.

Example 1:




```
Input: grid = [[0,0,1,0,0,0,0,1,0,0,0,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,1,1,0,1,0,0,0,0,0,0,0,0],[0,1,0,0,1,1,0,0,1,0,1,0,0],[0,1,0,0,1,1,0,0,1,1,1,0,0],[0,0,0,0,0,0,0,0,0,0,1,0,0],[0,0,0,0,0,0,0,1,1,1,0,0,0],[0,0,0,0,0,0,0,1,1,0,0,0,0]]
Output: 6
Explanation: The answer is not 11, because the island must be connected 4-directionally.
```

Example 2:

```
Input: grid = [[0,0,0,0,0,0,0,0]]
Output: 0
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- grid[i][j] is either 0 or 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Maximum area (cell count) of any connected island. Return 0 if no island. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each land cell, DFS to measure area of its island while marking visited.
# Track the maximum area seen across all components.
# Same flood-fill as Number of Islands, but accumulate area instead of count.
```



```
# Time: O(m * n). Space: O(m * n) stack worst case.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# Each DFS floods an island and returns its cell count. max() over all islands. ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) stack. The return-value DFS pattern: each cell contributes 1 + sum of children.
# This is the same DFS flood-fill as #200 (Number of Islands) with a count return.
# Given more time I'd ask: merge two islands by flipping a '0'?
# Enumerate each '0' cell, BFS to find adjacent distinct islands, sum their areas + 1."
```

DFS

#721 Accounts Merge

MED

Open on LeetCode

Array · String · DFS · BFS · Union Find

Lists: Grind 169

Problem

Given a list of accounts where each element accounts[i] is a list of strings, where the first element accounts[i][0] is a name, and the rest of the elements are emails representing emails of the account.

Now, we would like to merge these accounts. Two accounts definitely belong to the same person if there is some common email to both accounts. Note that even if two accounts have the same name, they may belong to different people as people could have the same name. A person can have any number of accounts initially, but all of their accounts definitely have the same name.

After merging the accounts, return the accounts in the following format: the first element of each account is the name, and the rest of the elements are emails in sorted order. The accounts themselves can be returned in any order.

Example 1:

```
Input: accounts = [["John","johnsmith@mail.com","john_newyork@mail.com"],["John","johnsmith@mail.com","john00@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
Output: [["John","john00@mail.com","john_newyork@mail.com","johnsmith@mail.com"],["Mary","mary@mail.com"],["John","johnnybravo@mail.com"]]
Explanation:
The first and second John's are the same person as they have the common email "johnsmith@mail.com".
The third John and Mary are different people as none of their email addresses are used by other accounts.
We could return these lists in any order, for example the answer [['Mary', 'mary@mail.com'], ['John', 'johnnybravo@mail.com'], ['John', 'john00@mail.com', 'john_newyork@mail.com', 'johnsmith@mail.com']] would still be accepted.
```

Example 2:

```
Input: accounts = [["Gabe","Gabe0@m.co","Gabe3@m.co","Gabe1@m.co"],["Kevin","Kevin3@m.co","Kevin5@m.co","Kevin0@m.co"],["Ethan","Ethan5@m.co","Ethan4@m.co","Ethan0@m.co"],["Hanzo","Hanzo3@m.co","Hanzo1@m.co","Hanzo0@m.co"],["Fern","Fern5@m.co","Fern1@m.co","Fern0@m.co"]]
Output: [["Ethan","Ethan0@m.co","Ethan4@m.co","Ethan5@m.co"],["Gabe","Gabe0@m.co","Gabe1@m.co","Gabe3@m.co"],["Hanzo","Hanzo0@m.co","Hanzo1@m.co","Hanzo3@m.co"],["Kevin","Kevin0@m.co","Kevin3@m.co","Kevin5@m.co"],["Fern","Fern0@m.co","Fern1@m.co","Fern5@m.co"]]
```

Constraints:

- 1 <= accounts.length <= 1000
- 2 <= accounts[i].length <= 10
- 1 <= accounts[i][j].length <= 30
- accounts[i][0] consists of English letters.
- accounts[i][j] (for j > 0) is a valid email.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Merge accounts that share at least one email. Same name ≠ same person.
# Output: name + sorted merged emails. Any output order. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Union-Find on emails: union every email in the same account list.
# Group emails by root parent; sort each merged list for output.
# Time: O(N log N) for sorting emails. Space: O(N) DSU.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DFS on email graph)
# "Build a graph where emails in the same account are connected.
# DFS collects all emails in a connected component = one merged account.
# Track owner (account name) via the first email in each account."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(E log E) where E = total emails (sort dominates). Space O(E).
# Union-Find alternative: union all emails in an account, group by root.
# DFS on the email graph is cleaner to explain — same complexity.
# Given more time I'd ask: if same email can belong to different names?
# That would mean the data is inconsistent — flag it as an error."
```


DFS

#785 Is Graph Bipartite?

MED

Open on LeetCode

DFS · BFS · Union Find · Graph

Lists: Denny Zhang

Problem

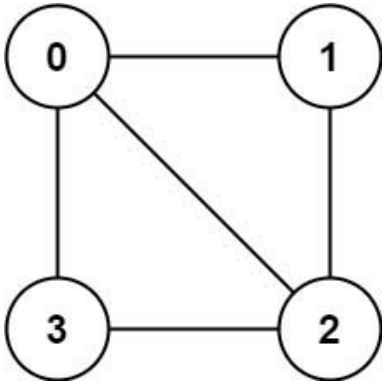
There is an undirected graph with n nodes, where each node is numbered between 0 and n - 1. You are given a 2D array graph, where graph[u] is an array of nodes that node u is adjacent to. More formally, for each v in graph[u], there is an undirected edge between node u and node v. The graph has the following properties:

- There are no self-edges (graph[u] does not contain u).
- There are no parallel edges (graph[u] does not contain duplicate values).
- If v is in graph[u], then u is in graph[v] (the graph is undirected).
- The graph may not be connected, meaning there may be two nodes u and v such that there is no path between them.

A graph is bipartite if the nodes can be partitioned into two independent sets A and B such that every edge in the graph connects a node in set A and a node in set B.

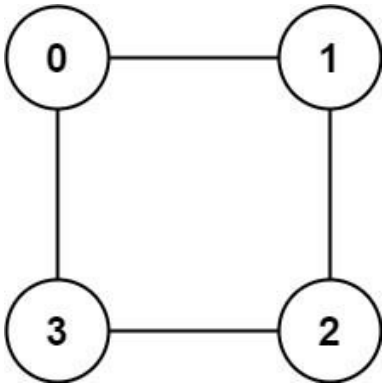
Return true if and only if it is bipartite.

Example 1:




```
Input: graph = [[1,2,3],[0,2],[0,1,3],[0,2]]
Output: false
Explanation: There is no way to partition the nodes into two independent sets such that every edge connects a node in one and a node in the other.
```

Example 2:



```
Input: graph = [[1,3],[0,2],[1,3],[0,2]]
Output: true
Explanation: We can partition the nodes into two sets: {0, 2} and {1, 3}.
```

Constraints:

- graph.length == n
- 1 <= n <= 100
- 0 <= graph[u].length < n
- 0 <= graph[u][i] <= n - 1
- graph[u] does not contain u.
- All the values of graph[u] are unique.
- If graph[u] contains v, then graph[v] contains u.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can we 2-colour the graph so no two adjacent nodes share the same colour?
# Graph may be disconnected — check every component. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try to 2-color the graph with BFS/DFS: assign color 0/1 alternating across edges.
# If any edge connects two nodes of the same color, graph is not bipartite.
# Standard coloring check on an undirected graph.
# Time: O(V + E). Space: O(V) color + visited.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V). 2-coloring via BFS — odd-length cycle → not bipartite.
# Given more time I'd ask: what if we need to output the two sets?
# Return {n for n in color if color[n]==0} and {n for n in color if color[n]==1}."
```


DFS

★ #1236 Web Crawler ★

ME
D

Open on LeetCode

String · DFS · BFS · Interactive

Lists: ★ Premium | Premium 98

Problem

Given a url startUrl and an interface HtmlParser, implement a web crawler to crawl all links that are under the same hostname as startUrl.

Return all urls obtained by your web crawler in any order.

Your crawler should:

- Start from the page: startUrl
- Call HtmlParser.getUrls(url) to get all urls from a webpage of given url.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.

As shown in the example url above, the hostname is example.org. For simplicity sake, you may assume all urls use http protocol without any port specified. For example, the urls http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname, while urls http://example.org/test and http://example.com/abc are not under the same hostname.

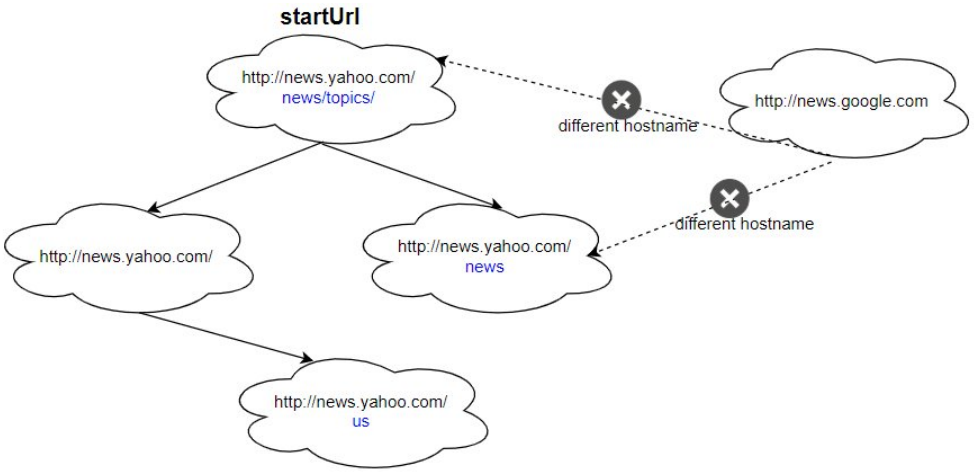
The HtmlParser interface is defined as such:

```
interface HtmlParser {
// Return a list of all urls from a webpage of given url.
public List<String> getUrls(String url);
}
```

Below are two examples explaining the functionality of the problem, for custom testing purposes you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

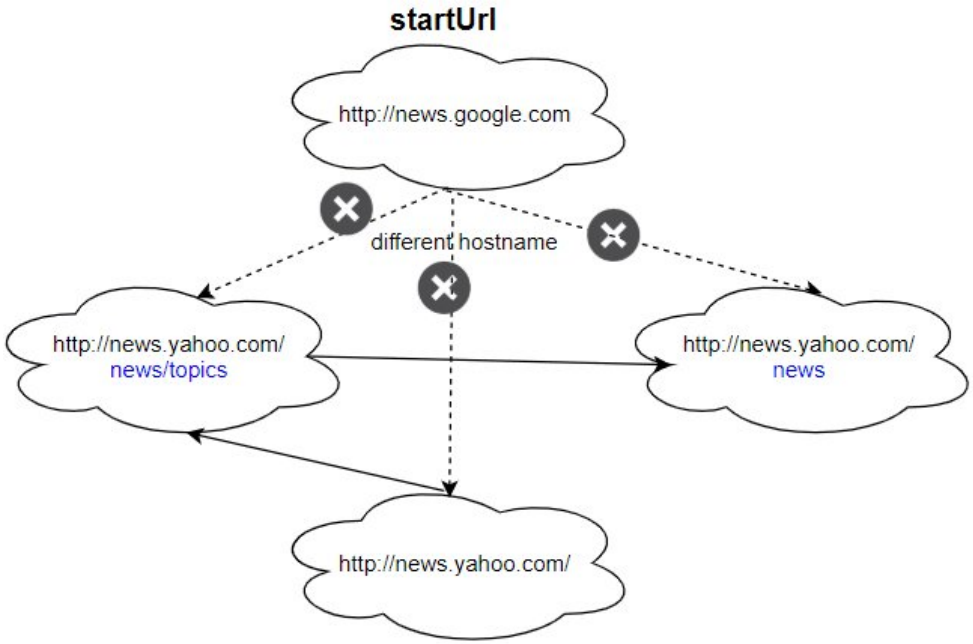
Note: Consider the same URL with the trailing slash "/" as a different URL. For example, "http://news.yahoo.com", and "http://news.yahoo.com/" are different urls.

Example 1:




```
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com",
  "http://news.yahoo.com/us"
]
```

Example 2:




```
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- **1 <= urls.length <= 1000**
- **1 <= urls[i].length <= 300**
- **startUrl is one of the urls.**
- **Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').**
- **The hostname may not start or end with the hyphen-minus character ('-').**
- **See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames**
- **You may assume there're no duplicates in url library.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "BFS from startUrl. Only follow links on the same hostname. Don't revisit. Right?
# Hostname = part between '//' and the next '/'."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS from startUrl: fetch page, parse all links on same hostname, enqueue unvisited.
# Use a visited set keyed by URL string to avoid cycles.
# Standard single-threaded web crawler simulation.
# Time: O(V + E) pages + links. Space: O(V) visited.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E) where V=pages, E=links. Space O(V).
# Standard BFS graph traversal — the only novelty is hostname filtering.
# Given more time I'd [redacted] #1242 Multithreaded? Use a ThreadPoolExecutor,
# submit getUrls calls as futures, use a thread-safe visited set."
```

DFS

#1242 Web Crawler Multithreaded

MED

Open on LeetCode

DFS · BFS · Concurrency

Lists: Denny Zhang

Problem

Given a URL startUrl and an interface HtmlParser, implement a Multi-threaded web crawler to crawl all links that are under the same hostname as startUrl.

Return all URLs obtained by your web crawler in any order.

Your crawler should:

- Start from the page: startUrl
- Call HtmlParser.getUrls(url) to get all URLs from a webpage of a given URL.
- Do not crawl the same link twice.
- Explore only the links that are under the same hostname as startUrl.

As shown in the example URL above, the hostname is example.org. For simplicity's sake, you may assume all URLs use HTTP protocol without any port specified. For example, the URLs http://leetcode.com/problems and http://leetcode.com/contest are under the same hostname, while URLs http://example.org/test and http://example.com/abc are not under the same hostname.

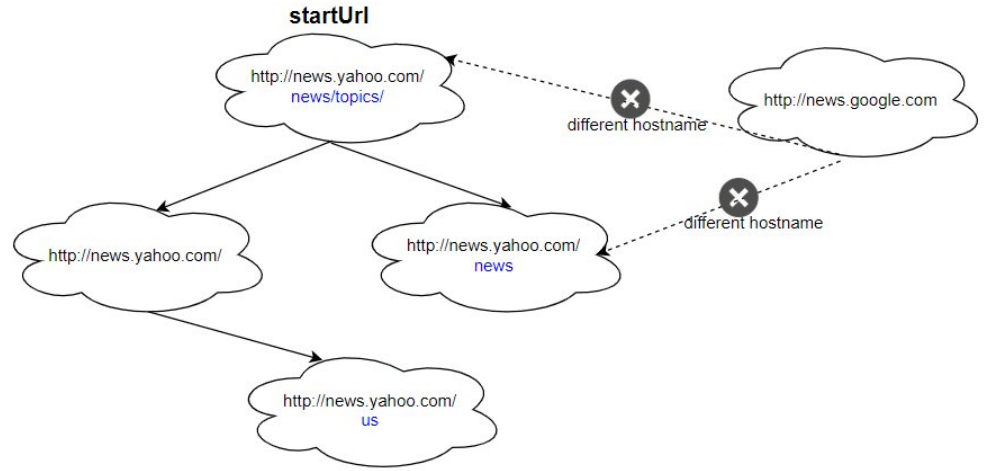
The HtmlParser interface is defined as such:

```
interface HtmlParser {
// Return a list of all urls from a webpage of given url.
// This is a blocking call, that means it will do HTTP request and return when this request is finished.
public List<String> getUrls(String url);
}
```

Note that getUrls(String url) simulates performing an HTTP request. You can treat it as a blocking function call that waits for an HTTP request to finish. It is guaranteed that getUrls(String url) will return the URLs within 15ms. Single-threaded solutions will exceed the time limit so, can your multi-threaded web crawler do better?

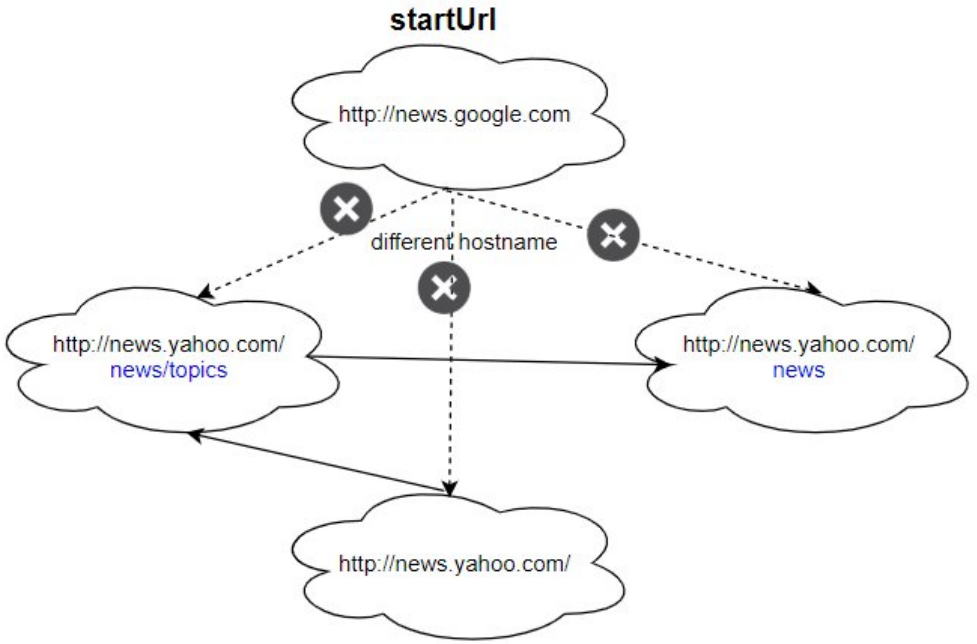
Below are two examples explaining the functionality of the problem. For custom testing purposes, you'll have three variables urls, edges and startUrl. Notice that you will only have access to startUrl in your code, while urls and edges are not directly accessible to you in code.

Example 1:




```
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com",
  "http://news.yahoo.com/us"
]
```

Example 2:



```
Input:
urls = [
  "http://news.yahoo.com",
  "http://news.yahoo.com/news",
  "http://news.yahoo.com/news/topics/",
  "http://news.google.com"
]
edges = [[0,2],[2,1],[3,2],[3,1],[3,0]]
```

Constraints:

- 1 <= urls.length <= 1000
- 1 <= urls[i].length <= 300
- startUrl is one of the urls.
- Hostname label must be from 1 to 63 characters long, including the dots, may contain only the ASCII letters from 'a' to 'z', digits from '0' to '9' and the hyphen-minus character ('-').
- The hostname may not start or end with the hyphen-minus character ('-').
- See: https://en.wikipedia.org/wiki/Hostname#Restrictions_on_valid_hostnames
- You may assume there're no duplicates in the URL library.

Follow up:

1. Assume we have 10,000 nodes and 1 billion URLs to crawl. We will deploy the same software onto each node. The software can know about all the nodes. We have to minimize communication between machines and make sure each node does equal amount of work. How would your web crawler design change?
2. What if one node fails or does not work?
3. How do you know when the crawler is done?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Same as #1236 but with multi-threading since getUrls is blocking and slow.
# Use threads to fetch pages in parallel. Thread-safe visited set required. Right?"

# PHASE 4 — CLEAN CODE (threading approach)

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Same BFS crawler as single-threaded, but dispatch each getLinks call to a worker thread.
# Shared visited set must be synchronized; join threads when queue is empty.
# Correct parallelism model for the LeetCode concurrency variant.
# Time: O(V + E) with parallel fetch. Space: O(V) visited + thread overhead.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Threads cut wall-clock time proportional to how much getUrls blocks.
# Lock only needed when checking/adding to visited — minimize critical section.
# Distributed follow-up (10,000 nodes, 1B URLs): consistent hashing to partition URLs by hostname.
# Each node owns a hostname range. Communicate frontier pages between nodes.
# 'Done' signal: all queues empty and all workers idle (barrier synchronisation)."
```


Graphs

Round 2 · High · Medium · 3 questions

Graphs

#128 Longest Consecutive Sequence

MED

Open on LeetCode

Array · Hash Table · Union Find

Lists: Grind 169 | CodeSignal

Problem

Given an unsorted array of integers nums, return the length of the longest consecutive elements sequence.
You must write an algorithm that runs in O(n) time.

Example 1:

```
Input: nums = [100,4,200,1,3,2]
Output: 4
Explanation: The longest consecutive elements sequence is [1, 2, 3, 4]. Therefore its length is 4.
```

Example 2:

```
Input: nums = [0,3,7,2,5,8,4,6,0,1]
Output: 9
```

Example 3:

```
Input: nums = [1,0,1,2]
Output: 3
```

Constraints:

- 0 <= nums.length <= 105
- -109 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the longest run of consecutive integers. Must be O(n). Return the length. Right?
# Duplicates don't extend the sequence — [1,1,2] → length 2."
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Put all numbers in a hash set. For each num, if num-1 is not in set, walk num, num+1, num+2...
# counting streak length. Track longest streak.
# Each number visited once across all walks — O(n) with the set.
# Time: O(n). Space: O(n) set.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE (hash set + only start from sequence beginnings)
# "Build a set. For each x where x-1 is NOT in the set (x is a sequence start),
# count upward. Each element is processed at most once total → O(n)."
# PHASE 4 — CLEAN CODE
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Each number is the starting point of at most one sequence scan.
# The 'start of sequence' check prevents redundant scans — the key insight.
# Given more time I'd ask: streaming input? Use a dict mapping each endpoint
# of a range to its length, merging ranges as new numbers arrive."
```


Graphs

★ #277 Find the Celebrity ★

MED

Open on LeetCode

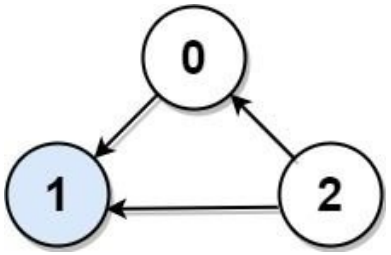
Two Pointers · Graph · Interactive

Lists: ★ Premium | Premium 98

Problem

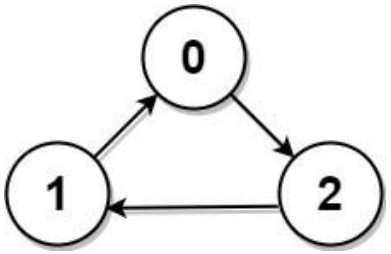
Suppose you are at a party with n people labeled from 0 to n - 1 and among them, there may exist one celebrity. The definition of a celebrity is that all the other n - 1 people know the celebrity, but the celebrity does not know any of them.
Now you want to find out who the celebrity is or verify that there is not one. You are only allowed to ask questions like: "Hi, A. Do you know B?" to get information about whether A knows B. You need to find out the celebrity (or verify there is not one) by asking as few questions as possible (in the asymptotic sense).
You are given an integer n and a helper function bool knows(a, b) that tells you whether a knows b. Implement a function int findCelebrity(n). There will be exactly one celebrity if they are at the party.
Return the celebrity's label if there is a celebrity at the party. If there is no celebrity, return -1.
Note that the n x n 2D array graph given as input is not directly available to you, and instead only accessible through the helper function knows. graph[i][j] == 1 represents person i knows person j, wherease graph[i][j] == 0 represents person j does not know person i.

Example 1:




```
Input: graph = [[1,1,0],[0,1,0],[1,1,1]]
Output: 1
Explanation: There are three persons labeled with 0, 1 and 2. graph[i][j] = 1 means person i knows person j, otherwise
graph[i][j] = 0 means person i does not know person j. The celebrity is the person labeled as 1 because both 0 and 2 know him
but 1 does not know anybody.
```

Example 2:




```
Input: graph = [[1,0,1],[1,1,0],[0,1,1]]
Output: -1
Explanation: There is no celebrity.
```

Constraints:

- n == graph.length == graph[i].length
- 2 <= n <= 100
- graph[i][j] is 0 or 1.
- graph[i][i] == 1

Follow up: If the maximum number of allowed calls to the API knows is 3 * n, could you find a solution without exceeding the maximum number of calls?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
```

```
# "Celebrity: everyone knows them, they know no one.
# Find them using knows(a,b) API — minimise calls. Right?
# At most one celebrity can exist."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Ask every other person if they know celebrity; candidate must be known by all and know nobody.
# Two-pass elimination: first narrow to one candidate with O(n) knows() calls, then verify.
# Naive all-pairs check is O(n^2) knows() calls.
# Time: O(n^2) naive, O(n) with elimination. Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (candidate elimination + verify)
# "Pass 1: start with candidate=0, scan all i. If knows(candidate, i): celebrity can't be candidate → candidate=i.
# Pass 2: verify candidate — everyone must know them AND they know no one.
# Total API calls: O(n). Without the two-pass we'd need O(n²)."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — 2n-2 API calls in the worst case. Space O(1).
# The candidate elimination logic: if A knows B, A is not the celebrity (celebrity knows no one).
# If A doesn't know B, B is not the celebrity (celebrity is known by all).
# One of them is eliminated per comparison — O(n) total.
# Given more time I'd ask: what if there's a possibility of no celebrity?
# The current solution handles this — the verification pass returns -1 if candidate fails."
```


Graphs

★ #1136 Parallel Courses ★

ME
D

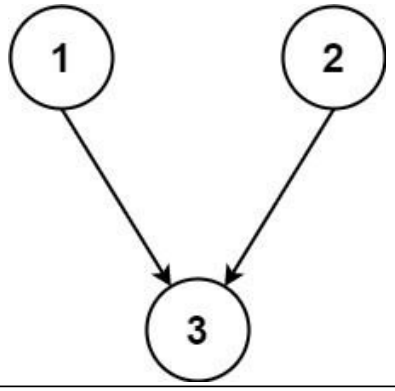
Open on LeetCode

Graph · Topological Sort

Lists: ★ Premium | Premium 98

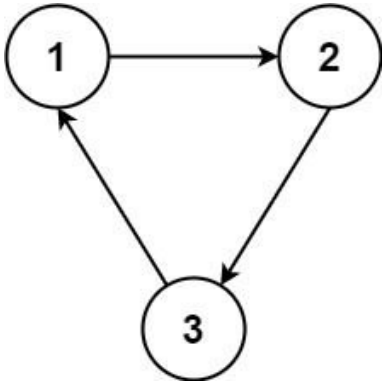
Problem

You are given an integer n, which indicates that there are n courses labeled from 1 to n. You are also given an array relations where relations[i] = [prevCoursei, nextCoursei], representing a prerequisite relationship between course prevCoursei and course nextCoursei: course prevCoursei has to be taken before course nextCoursei.
In one semester, you can take any number of courses as long as you have taken all the prerequisites in the previous semester for the courses you are taking.
Return the minimum number of semesters needed to take all courses. If there is no way to take all the courses, return -1.
Example 1:




```
Input: n = 3, relations = [[1,3],[2,3]]
Output: 2
Explanation: The figure above represents the given graph.
In the first semester, you can take courses 1 and 2.
In the second semester, you can take course 3.
```

Example 2:




```
Input: n = 3, relations = [[1,2],[2,3],[3,1]]
Output: -1
Explanation: No course can be studied because they are prerequisites of each other.
```

Constraints:

- **1 <= n <= 5000**

- 1 <= relations.length <= 5000
- relations[i].length == 2
- 1 <= prevCoursei, nextCoursei <= n
- prevCoursei != nextCoursei
- All the pairs [prevCoursei, nextCoursei] are unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Minimum semesters to take all courses where you can take any number per semester
# as long as prerequisites are met. Return -1 if impossible (cycle). Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Topological sort with indegree: each semester take all courses whose prerequisites are done.
# Increment semester count each round; if no course can be taken but some remain, impossible.
# Same Kahn BFS layered by semester.
# Time: O(V + E). Space: O(V + E).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Kahn's BFS level-by-level)
# "BFS topological sort: process all zero-in-degree nodes in one 'batch' (semester).
# Each batch = one semester. Count batches. If total processed < n → cycle."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). This is Course Schedule (#207) with BFS level counting.
# Each BFS level = one semester. Cycle detection: total processed < n.
# Given more time I'd ask: what if courses have durations (not just 1 semester each)?
# This becomes a weighted DAG critical path problem — DP or Bellman-Ford."
```


BFS

Round 2 · High · Medium · 6 questions

BFS

★ #286 Walls and Gates ★

MED

Open on LeetCode

Array · BFS · Matrix

Lists: ★ Premium | Premium 98

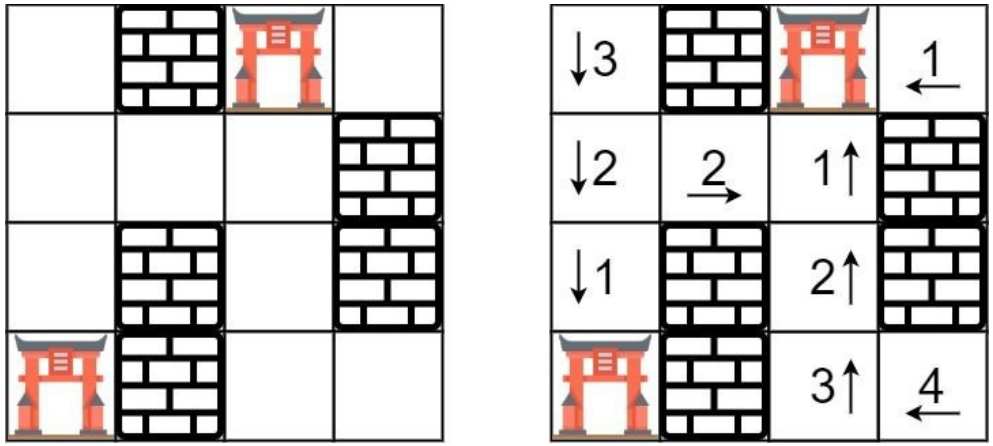
Problem

You are given an m x n grid rooms initialized with these three possible values.

- -1 A wall or an obstacle.
- 0 A gate.
- INF Infinity means an empty room. We use the value 231 - 1 = 2147483647 to represent INF as you may assume that the distance to a gate is less than 2147483647.

Fill each empty room with the distance to its nearest gate. If it is impossible to reach a gate, it should be filled with INF.

Example 1:




```
Input: rooms =
[[2147483647,-1,0,2147483647],[2147483647,2147483647,2147483647,-1],[2147483647,-1,2147483647,-1],[0,-1,2147483647,2147483647]]
Output: [[3,-1,0,1],[2,2,1,-1],[1,-1,2,-1],[0,-1,3,4]]
```

Example 2:

```
Input: rooms = [[-1]]
Output: [[-1]]
```

Constraints:

- m == rooms.length
- n == rooms[i].length
- 1 <= m, n <= 250
- rooms[i][j] is -1, 0, or 231 - 1.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Fill each INF room with its distance to the nearest gate (0). Walls (-1) stay. Right?
# In-place modification. INF = 2^31-1."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Multi-source BFS from every gate simultaneously (distance 0).
# Fill empty rooms layer by layer with increasing distance.
# Each room reached once - standard BFS on grid.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (multi-source BFS from all gates)
# "Enqueue ALL gates at once. BFS outward - each room gets filled with distance
# from the nearest gate simultaneously. O(m*n)."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
```



```
# "Time O(m*n). Space O(m*n). Multi-source BFS is the key - a single BFS from each gate
# independently would be O(m*n * gates), much slower.
# Contrast with #542 (01 Matrix): identical pattern but for distance to nearest 0.
# Given more time I'd ask: find distance to nearest wall instead?
# Multi-source BFS from all walls - same approach."
```

BFS

#322 Coin Change

ME D

Open on LeetCode

Array · Dynamic Programming · BFS

Lists: Grind 169

Problem

You are given an integer array coins representing coins of different denominations and an integer amount representing a total amount of money.

Return the fewest number of coins that you need to make up that amount. If that amount of money cannot be made up by any combination of the coins, return -1.

You may assume that you have an infinite number of each kind of coin.

Example 1:

```
Input: coins = [1,2,5], amount = 11
Output: 3
Explanation: 11 = 5 + 5 + 1
```

Example 2:

```
Input: coins = [2], amount = 3
Output: -1
```

Example 3:

```
Input: coins = [1], amount = 0
Output: 0
```

Constraints:

- 1 <= coins.length <= 12
- 1 <= coins[i] <= 231 - 1
- 0 <= amount <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Minimum coins to make up amount. Unlimited supply of each coin. Return -1 if impossible. Right?
# Coin values can be any positive integer — not necessarily power of 2 (so greedy fails)."
#
# "coins=[1,2,5], amount=11: 5+5+1=3 coins ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Recursion: for each amount, try every coin denomination and take 1 + min subproblem.
# Recomputes identical sub-amounts exponentially without memoization.
# Time: O(amount^coins) exponential. Space: O(amount) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (bottom-up DP)
# "dp[i] = minimum coins to make amount i.
# dp[0]=0. dp[i] = min(dp[i-coin]+1) for each coin <= i.
# Time: O(amount * coins). Space: O(amount)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# coins=[1,2,5], amount=11:
# dp[5]=1(5). dp[10]=2(5+5). dp[11]=3(5+5+1). → 3 ✓
# amount=3, coins=[2]: dp[2]=1, dp[3]=inf → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(amount * coins). Space O(amount).
# BFS alternative: treat amounts as nodes, coins as edges. BFS from 0 to amount.
# DP is preferred — BFS visits each amount once but rebuilds from scratch.
# Given more time I'd ask: how many ways to make the amount?
# Coin Change 2 (#518) — same DP but count combinations instead of min count."
```


BFS

#542 01 Matrix

ME D

Open on LeetCode

Array · BFS · Matrix

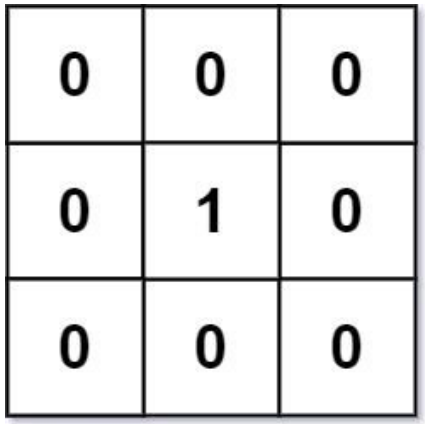
Lists: Grind 169 | Denny Zhang

Problem

Given an m x n binary matrix mat, return the distance of the nearest 0 for each cell.

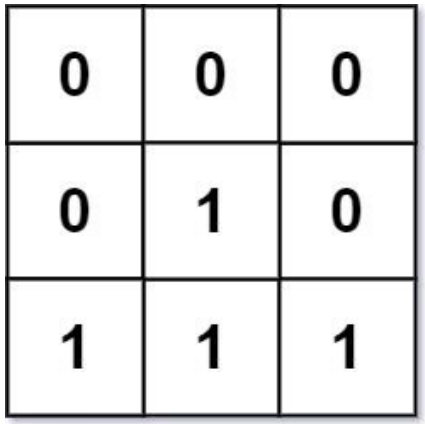
The distance between two cells sharing a common edge is 1.

Example 1:




```
Input: mat = [[0,0,0],[0,1,0],[0,0,0]]
Output: [[0,0,0],[0,1,0],[0,0,0]]
```

Example 2:




```
Input: mat = [[0,0,0],[0,1,0],[1,1,1]]
Output: [[0,0,0],[0,1,0],[1,2,1]]
```

Constraints:

- m == mat.length
- n == mat[i].length

- 1 <= m, n <= 104
- 1 <= m * n <= 104
- mat[i][j] is either 0 or 1.
- There is at least one 0 in mat.

Note: This question is the same as 1765: https://leetcode.com/problems/map-of-highest-peak/

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return distance of nearest 0 for each cell. In-place or new matrix. Right?
# Same pattern as Walls and Gates but for 0-cells instead of gates."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Multi-source BFS starting from every 0 simultaneously.
# First time each 1 cell is reached, its distance is minimal.
# Layer-by-layer BFS fills the dist matrix in one pass.
# Time: O(m * n). Space: O(m * n) queue + dist.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n). Multi-source BFS from all 0-cells simultaneously.
# Identical to #286 (Walls and Gates) — name this connection.
# Given more time I'd ask: distance to nearest 1 instead?
# Same multi-source BFS starting from all 1-cells."
```


BFS

#994 Rotting Oranges

ME
D

Open on LeetCode

Array · BFS · Matrix

Lists: Grind 169

Problem

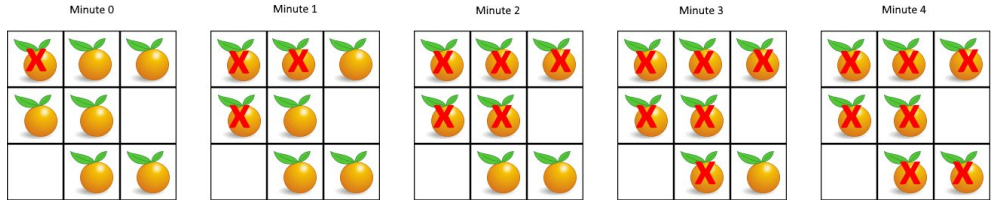
You are given an m x n grid where each cell can have one of three values:

- **0 representing an empty cell,**
- **1 representing a fresh orange, or**
- **2 representing a rotten orange.**

Every minute, any fresh orange that is 4-directionally adjacent to a rotten orange becomes rotten.

Return the minimum number of minutes that must elapse until no cell has a fresh orange. If this is impossible, return -1.

Example 1:




```
Input: grid = [[2,1,1],[1,1,0],[0,1,1]]
Output: 4
```

Example 2:

```
Input: grid = [[2,1,1],[0,1,1],[1,0,1]]
Output: -1
Explanation: The orange in the bottom left corner (row 2, column 0) is never rotten, because rotting only happens
4-directionally.
```

Example 3:

```
Input: grid = [[0,2]]
Output: 0
Explanation: Since there are already no fresh oranges at minute 0, the answer is just 0.
```

Constraints:

- **m == grid.length**
- **n == grid[i].length**
- **1 <= m, n <= 10**
- **grid[i][j] is 0, 1, or 2.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Multi-source BFS from all rotten oranges. Fresh adjacent → rotten after 1 minute.
# Return minutes until no fresh remain, or -1 if impossible. Right?"
#
# "If no fresh oranges initially → return 0 immediately."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Multi-source BFS: enqueue all rotten oranges at minute 0.
# Each minute, rot all fresh neighbors; track elapsed minutes until no fresh remain.
# If fresh count > 0 after BFS ends, return -1.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# [[2,1,1],[1,1,0],[0,1,1]]: fresh=6. BFS spreads level by level. After 4 minutes fresh=0 → 4 ✓
# Isolated fresh orange: fresh>0 after BFS done → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n). Multi-source BFS: same as Walls and Gates, same as 01 Matrix.
# The 'fresh' counter eliminates the need to scan the grid again at the end.
# Given more time I'd ask: what if diagonal rotting is allowed?
# Add 4 diagonal direction pairs — same algorithm."
```

BFS

★ #1197 Minimum Knight Moves ★

MED

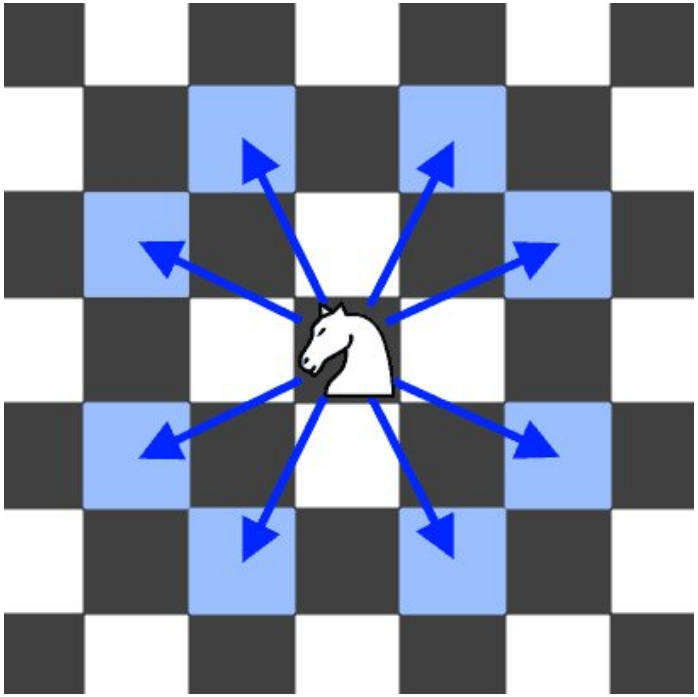
Open on LeetCode

BFS

Lists: ★ Premium | Grind 169 | Premium 98

Problem

In an infinite chess board with coordinates from -infinity to +infinity, you have a knight at square [0, 0].
A knight has 8 possible moves it can make, as illustrated below. Each move is two squares in a cardinal direction, then one square in an orthogonal direction.



Return the minimum number of steps needed to move the knight to the square [x, y]. It is guaranteed the answer exists.

Example 1:

```
Input: x = 2, y = 1
Output: 1
Explanation: [0, 0] → [2, 1]
```

Example 2:

```
Input: x = 5, y = 5
Output: 4
Explanation: [0, 0] → [2, 1] → [4, 2] → [3, 4] → [5, 5]
```

Constraints:

- -300 <= x, y <= 300
- 0 <= |x| + |y| <= 300

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Knight moves on an infinite board. Minimum moves from (0,0) to (x,y).
# 8 possible moves. Symmetry: |x| and |y| — reflect to first quadrant. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS on an infinite chessboard from (0,0) to (x,y).
# Enqueue all 8 knight moves each step; stop when target is reached.
# Layer count = minimum moves. Prune to first quadrant since moves are symmetric.
# Time: O(max(x,y)^2) BFS nodes. Space: O(max(x,y)^2) visited.
# Should I go ahead and optimize?"

# PHASE 5 — TEST & DEBUG
# (2,1): BFS from (0,0). (2,1) reached in 1 step → 1 ✓
# (5,5): 4 steps ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(max(|x|,|y|)²). Space O(max(|x|,|y|)²). Standard BFS shortest path.
# Symmetry reduction: reflect to first quadrant cuts search space by 4.
# The -1 lower bound allows the knight to 'go back' through negative coords when needed.
# Given more time I'd ask: bidirectional BFS?
# Start from both (0,0) and (x,y) simultaneously — meets in the middle, roughly halving depth."
```


· #994 | #1730 · | · Contents

BFS

#1730 Shortest Path to Get Food

MED

Open on LeetCode

Array · BFS · Matrix

Lists: Grind 169

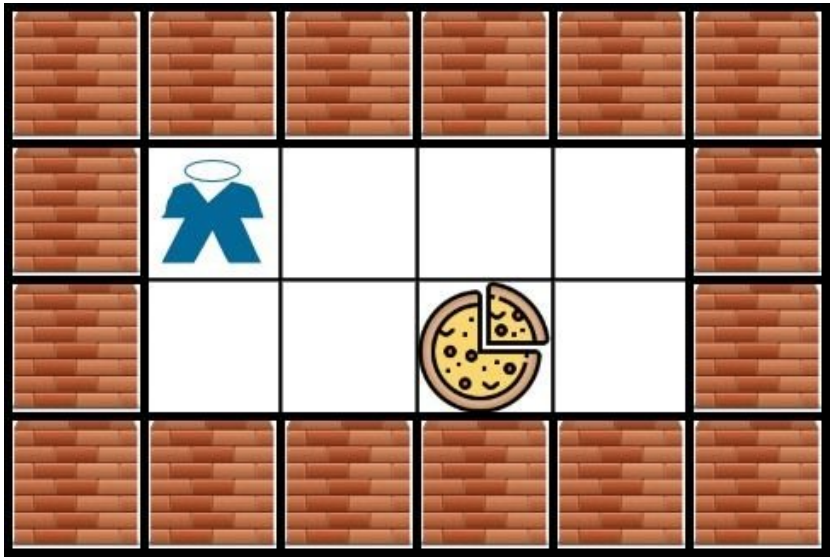
Problem

You are starving and you want to eat food as quickly as possible. You want to find the shortest path to arrive at any food cell.
You are given an m x n character matrix, grid, of these different types of cells:

- '*' is your location. There is exactly one '*' cell.
- '#' is a food cell. There may be multiple food cells.
- 'O' is free space, and you can travel through these cells.
- 'X' is an obstacle, and you cannot travel through these cells.

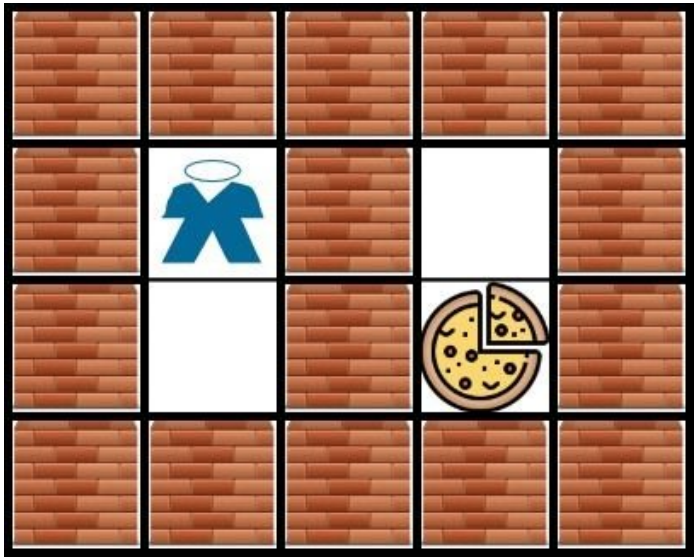
You can travel to any adjacent cell north, east, south, or west of your current location if there is not an obstacle.
Return the length of the shortest path for you to reach any food cell. If there is no path for you to reach food, return -1.

Example 1:



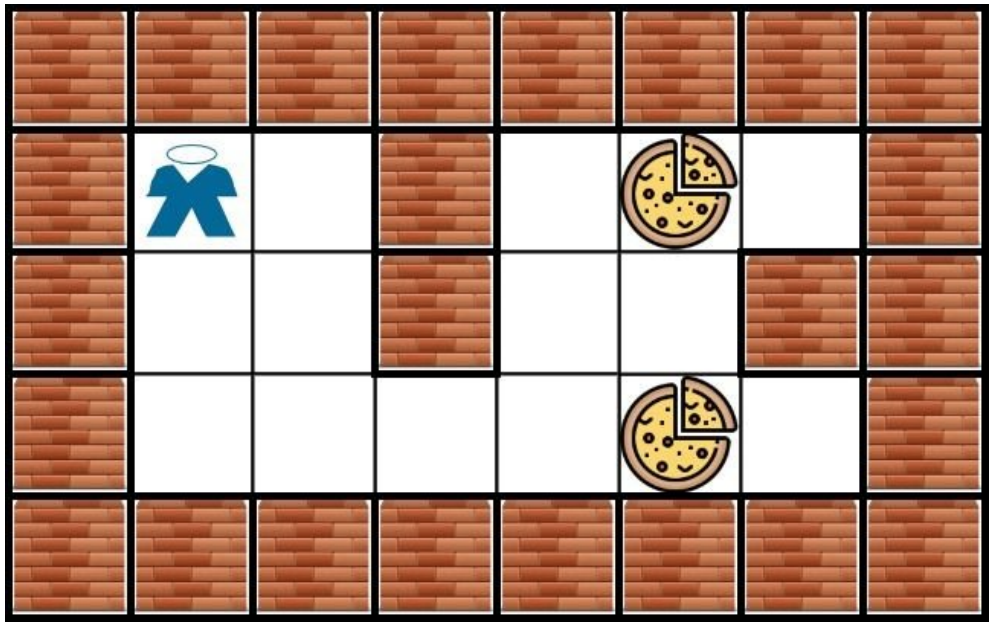
```
Input: grid = [["X","X","X","X","X","X"],["X","*","O","O","O","X"],["X","O","O","#","O","X"],["X","X","X","X","X","X"]]
Output: 3
Explanation: It takes 3 steps to reach the food.
```

Example 2:



```
Input: grid = [["X","X","X","X","X"],["X","*","X","O","X"],["X","O","X","#","X"],["X","X","X","X","X"]]
Output: -1
Explanation: It is not possible to reach the food.
```

Example 3:



```
Input: grid = [["X","X","X","X","X","X","X","X"],["X","*","O","X","O","#","O","X"],["X","O","O","X","O","O","X","X"],["X","O","
O","O","O","#","O","X"],["X","X","X","X","X","X","X","X"]]
Output: 6
Explanation: There can be multiple food cells. It only takes 6 steps to reach the bottom food.
```

Example 4:

```
Input: grid = [["X","X","X","X","X","X","X","X"],["X","*","O","X","O","#","O","X"],["X","O","O","X","O","O","X","X"],["X","O","
O","O","O","#","O","X"],["O","O","O","O","O","O","O","O"]]
Output: 5
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 200
- grid[row][col] is '*', 'X', 'O', or '#'.
- The grid contains exactly one '*'.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "BFS from '*' to the nearest '#'. 'O' = free, 'X' = wall, '#' = food (any one).
# Return shortest path length. Return -1 if unreachable. Right?"
#
# "'*' → nearest '#' using 4-directional BFS. First '#' reached = shortest path."

# PHASE 2 — BRUTE FORCE (also optimal)
# "Let me start with brute force to lock in the logic.
# Multi-source BFS from every '*' food cell simultaneously.
# First time a 'H' house is reached, record its distance; sum cannot reach uses -1.
# BFS is already optimal O(m*n); DFS would revisit cells wastefully.
# Time: O(m * n). Space: O(m * n) queue.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Standard BFS from source. First '#' reached is guaranteed shortest path.
# Mark visited by overwriting 'O' with 'X' — no extra visited set needed.
# Time: O(m*n). Space: O(m*n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# grid example 1: BFS from '*'. Nearest '#' is 3 steps away → return 3 ✓
# grid example 2: BFS exhausts all reachable 'O' cells → return -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n).
# This is identical to #994 Rotting Oranges / #286 Walls and Gates — single-source BFS
# with a target condition instead of a distance fill.
# Overwriting 'O' with 'X' acts as the visited marker — avoids O(m*n) extra space.
# Given more time I'd ask: what if we want paths to ALL food cells (not just shortest)?
# Continue BFS after first '#' hit — collect all '#' cells reached with their distances."
# □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
# HARD PROBLEMS
# □□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□
```


Round 3

High Priority · Hard

23 questions · 8 patterns

- ☐ Arrays & Hashing #41 ↗
- ☐ Sliding Window #76 ↗
- ☐ Binary Search #4 #315 #410 #668 #710 #774 #1235 ↗
- ☐ Matrix #1074 ↗
- ☐ Trees & BST #124 #297 ↗
- ☐ DFS #269 #329 #685 #1036 #1192 ↗
- ☐ Graphs #305 #1153 ↗
- ☐ BFS #127 #317 #773 #815 ↗

Arrays & Hashing

Round 3 · High · Hard · 1 question

Arrays & Hashing

#41 First Missing Positive HAR

Open on LeetCode

Array · Hash Table

Lists: Grind 169

Problem

Given an unsorted integer array nums. Return the smallest positive integer that is not present in nums.
You must implement an algorithm that runs in O(n) time and uses O(1) auxiliary space.

Example 1:

```
Input: nums = [1,2,0]
Output: 3
Explanation: The numbers in the range [1,2] are all in the array.
```

Example 2:

```
Input: nums = [3,4,-1,1]
Output: 2
Explanation: 1 is in the array but 2 is missing.
```

Example 3:

```
Input: nums = [7,8,9,11,12]
Output: 1
Explanation: The smallest positive integer 1 is missing.
```

Constraints:

- **1 <= nums.length <= 105**
- **-231 <= nums[i] <= 231 - 1**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Smallest positive integer NOT in nums. O(n) time, O(1) auxiliary space.
# Answer is in range [1, n+1] — if all 1..n present, answer is n+1. Right?
# Negatives, zeros, and large numbers can be ignored."
#
# "[3,4,-1,1] → 2 (1 present, 2 missing) ✓. [7,8,9] → 1 ✓. [1,2,0] → 3 ✓."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Sort the array, then scan from 1 upward until the first missing positive integer.
# Or put all values in a hash set and scan i=1,2,3... until absent.
# Correct but sorting is O(n log n) and the set uses O(n) extra space.
# Time: O(n log n). Space: O(n) for set approach.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (array as hash map — cycle sort)
# "Key insight: use the array itself as a lookup table.
# Valid range is [1, n]. Place each number x at index x-1 (if 1 <= x <= n and not already there).
# After placing, scan: first i where nums[i] != i+1 → answer is i+1.
# If all positions correct → answer is n+1.
# Each number is placed at most once → O(n) total swaps."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,4,-1,1]: place 3→pos2, 4→pos3, -1 ignored, 1→pos0.
# After: [1,-1,3,4]. Scan: i=1 nums[1]=-1≠2 → return 2 ✓
# [1,2,3]: after: [1,2,3]. All correct → return 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n) — each element swapped at most once. Space O(1).
# The while loop runs at most n total iterations across the entire for loop — amortised O(1) per i.
# Given more time I'd ask: what if duplicates exist?
# The condition nums[nums[i]-1] != nums[i] handles duplicates — don't swap a number to a position
# that already has the correct value (would loop forever)."
```

Sliding Window

Round 3 · High · Hard · 1 question

Sliding Window

#76 Minimum Window Substring HAR

Open on LeetCode

Hash Table · String · Sliding Window

Lists: Grind 169

Problem

Given two strings s and t of lengths m and n respectively, return the minimum window substring of s such that every character in t (including duplicates) is included in the window. If there is no such substring, return the empty string "". The testcases will be generated such that the answer is unique.

Example 1:

```
Input: s = "ADOBECODEBANC", t = "ABC"
Output: "BANC"
Explanation: The minimum window substring "BANC" includes 'A', 'B', and 'C' from string t.
```

Example 2:

```
Input: s = "a", t = "a"
Output: "a"
Explanation: The entire string s is the minimum window.
```

Example 3:

```
Input: s = "a", t = "aa"
Output: ""
Explanation: Both 'a's from t must be included in the window.
Since the largest window of s only has one 'a', return empty string.
```

Constraints:

- **m == s.length**
- **n == t.length**
- **1 <= m, n <= 105**
- **s and t consist of uppercase and lowercase English letters.**

Follow up: Could you find an algorithm that runs in O(m + n) time?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Smallest window in s containing all characters of t (with multiplicity).
# Return '' if no such window. Right?
# t may have duplicate chars — window must have at least that many."
#
# "s='ADOBECODEBANC', t='ABC' → 'BANC' (contains A,B,C, length 4) ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Enumerate every substring of s with two nested indices.
# For each substring, check whether it contains all characters of t (with frequency).
# Track the shortest valid window — correct but O(m^2 * n) character checks.
# Time: O(m^2 * n). Space: O(n) frequency map.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (sliding window with 'formed' counter)
# "Expand right: add s[right] to window frequency map.
# When a char's count in window meets t's requirement, increment 'formed'.
# When formed == len(distinct chars in t): window is valid. Try shrinking from left.
# Track best window. O(m+n) — each char enters/exits at most once."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='ADOBECODEBANC', t='ABC': need={A:1,B:1,C:1}. required=3.
# Window expands until it contains A,B,C. Shrink left until one char drops below need.
# Best window = 'BANC' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n). Space O(m+n). The 'formed' counter avoids comparing the full frequency maps.
# A window is valid exactly when formed==required — elegant O(1) validity check.
# Given more time I'd ask: find all minimum windows?
# Collect all windows where length == best length during the BFS scan."
```

Binary Search

Round 3 · High · Hard · 7 questions

Binary Search

#4 Median of Two Sorted Arrays HAR

Open on LeetCode

Array · Binary Search · Divide and Conquer

Lists: Grind 169

Problem

Given two sorted arrays nums1 and nums2 of size m and n respectively, return the median of the two sorted arrays.
The overall run time complexity should be O(log (m+n)).

Example 1:

```
Input: nums1 = [1,3], nums2 = [2]
Output: 2.00000
Explanation: merged array = [1,2,3] and median is 2.
```

Example 2:

```
Input: nums1 = [1,2], nums2 = [3,4]
Output: 2.50000
Explanation: merged array = [1,2,3,4] and median is (2 + 3) / 2 = 2.5.
```

Constraints:

- nums1.length == m
- nums2.length == n
- 0 <= m <= 1000
- 0 <= n <= 1000
- 1 <= m + n <= 2000
- -106 <= nums1[i], nums2[i] <= 106

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Merge two sorted arrays conceptually and find the median. O(log(m+n)) required. Right?
# Median: if total length is odd → middle element. Even → average of two middles."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Merge both sorted arrays with two pointers into one combined array.
# Walk to index (m+n)//2 (and the one before if even length) to read the median.
# Correct but uses O(m+n) extra space and ignores the logarithmic requirement.
# Time: O(m + n). Space: O(m + n) merged array.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (binary search on partition — true O(log(min(m,n))))
# "Partition both arrays into left and right halves such that:
# total left size = (m+n+1)//2, max(left) <= min(right).
# Binary search on the partition point of the smaller array.
# At each partition: check if nums1[mid-1] <= nums2[r-mid] and nums2[r-mid-1] <= nums1[mid].
# Median = max(left halves) for odd total, or average of max(left)/min(right) for even."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums1=[1,3], nums2=[2]: m=2,n=2. half=2. Partition i=1,j=1: left1=1,right1=3,left2=2,right2=∞.
# 1<=∞ and 2<=3 ✓. Total=3 (odd) → max(1,2)=2. → 2.0 ✓
# nums1=[1,2], nums2=[3,4]: Partition → max(left)=2, min(right)=3. → 2.5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log(min(m,n))). Space O(1). Binary search on the smaller array's partition.
# The +inf/-inf sentinels handle edge cases (empty left/right) cleanly.
# The brute-force merge approach is often acceptable in interviews — present both and let the interviewer choose.
# Given more time I'd ask: kth element of two sorted arrays?
# Binary search on k — eliminate k//2 elements from one array per step. Same O(log k) idea."
```

Binary Search

#315 Count of Smaller Numbers After Self

HAR

Open on LeetCode

Array · Binary Search · Divide and Conquer · Binary Indexed Tree · Segment Tree · Merge Sort · Ordered Set

Lists: Denny Zhang

Problem

Given an integer array nums, return an integer array counts where counts[i] is the number of smaller elements to the right of nums[i].

Example 1:

```
Input: nums = [5,2,6,1]
Output: [2,1,1,0]
Explanation:
To the right of 5 there are 2 smaller elements (2 and 1).
To the right of 2 there is only 1 smaller element (1).
To the right of 6 there is 1 smaller element (1).
To the right of 1 there is 0 smaller element.
```

Example 2:

```
Input: nums = [-1]
Output: [0]
```

Example 3:

```
Input: nums = [-1,-1]
Output: [0,0]
```

Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "For each index i, count elements to the right of i that are strictly less than nums[i].
# Return as an array. Right?"
#
# "[5,2,6,1] → [2,1,1,0]: right of 5 has 2 and 1 (2 smaller), etc. ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each index i, scan every element to its right and count how many are smaller than nums[i].
# Store those counts in an output array.
# Correct, but the inner scan makes this quadratic overall.
# Time: O(n^2). Space: O(1) besides output.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BIT / merge sort)
# "Process from right to left. Build a BIT indexed by coordinate-compressed value.
# For each nums[i]: query BIT for count of values in [min, nums[i]-1].
# Update BIT at nums[i]. O(n log n) time."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [5,2,6,1]: process right→left. 1→query(0)=0,update(1). 6→query(3)=1,update(4). 2→query(1)=1,update(2). 5→query(2)=2.
# reversed result=[2,1,1,0] → reverse → [2,1,1,0] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n). Space O(n). BIT query gives prefix count in O(log n).
# Merge sort alternative: during merge, count inversions — same complexity but different mental model.
# Given more time I'd ask: count of larger elements after self?
# Query total_right − query(rank[num]) instead."
```


Binary Search

#410 Split Array Largest Sum

HAR

Open on LeetCode

Array · Binary Search · Dynamic Programming · Greedy · Prefix Sum

Lists: Denny Zhang

Problem

Given an integer array nums and an integer k, split nums into k non-empty subarrays such that the largest sum of any subarray is minimized.

Return the minimized largest sum of the split.

A subarray is a contiguous part of the array.

Example 1:

```
Input: nums = [7,2,5,10,8], k = 2
Output: 18
Explanation: There are four ways to split nums into two subarrays.
The best way is to split it into [7,2,5] and [10,8], where the largest sum among the two subarrays is only 18.
```

Example 2:

```
Input: nums = [1,2,3,4,5], k = 2
Output: 9
Explanation: There are four ways to split nums into two subarrays.
The best way is to split it into [1,2,3] and [4,5], where the largest sum among the two subarrays is only 9.
```

Constraints:

- 1 <= nums.length <= 1000
- 0 <= nums[i] <= 106
- 1 <= k <= min(50, nums.length)

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Split nums into k non-empty contiguous subarrays to minimise the maximum subarray sum.
# Must be in order — no reordering allowed. Right?
# Same 'binary search on answer' pattern as Capacity to Ship Packages (#1011)."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Try every split of nums into k contiguous groups (DP over partition points).
# For each split, max subarray sum = largest group sum; minimize that over splits.
# State space explodes without binary search on the answer.
# Time: exponential / O(n^k) naive partitions. Space: O(n) recursion.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Binary search on the answer (the max sum).
# lo = max(nums) (at least one element), hi = sum(nums) (one subarray).
# Feasibility check: can we split into k subarrays with each sum <= mid?
# Greedy: accumulate greedily, start new subarray when exceeding mid. Count splits.
# Time: O(n log(sum)). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [7,2,5,10,8], k=2: lo=10, hi=32. mid=21→can_split? [7,2,5] and [10,8]=18≤21 ✓ → hi=21.
# ... converges to 18. can_split(18)=[7,2,5][10,8]→2 splits≤k=2 ✓. can_split(17)→[7,2,5][10,8]→10+8=18>17→need 3>k ✗. → 18 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log(sum)). Space O(1). Same template as #1011 Capacity to Ship — name this.
# lo/hi bounds: must be at least max(nums), at most sum(nums).
# Given more time I'd ask: DP approach? dp[j][i] = min largest sum splitting first i elements into j parts.
# O(kn²) — worse than binary search for large inputs."
```

Binary Search

#668 Kth Smallest Number in Multiplication Table

HAR

Open on LeetCode

Math · Binary Search

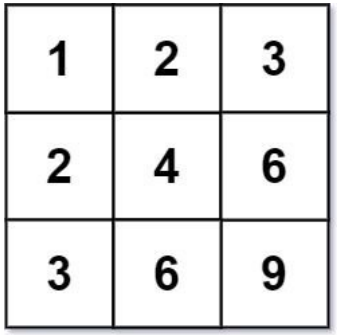
Lists: Denny Zhang

Problem

Nearly everyone has used the Multiplication Table. The multiplication table of size m x n is an integer matrix mat where mat[i][j] == i * j (1-indexed).

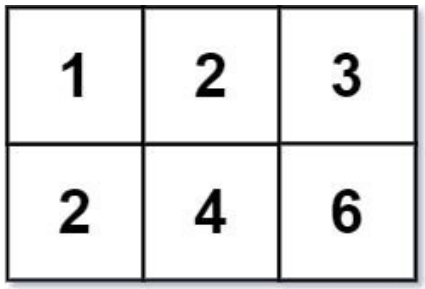
Given three integers m, n, and k, return the kth smallest element in the m x n multiplication table.

Example 1:




```
Input: m = 3, n = 3, k = 5
Output: 3
Explanation: The 5th smallest number is 3.
```

Example 2:




```
Input: m = 2, n = 3, k = 6
Output: 6
Explanation: The 6th smallest number is 6.
```

Constraints:

- 1 <= m, n <= 3 * 10^4
- 1 <= k <= m * n

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "m×n multiplication table: mat[i][j]=i×j (1-indexed). Find kth smallest element. Right?
# Can't materialise the table (m,n up to 3*10^4 → up to 9*10^8 cells)."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Collect all m*n products into a list, sort, return kth element.
# Correct but materializes every product — O(mn log(mn)) time and O(mn) space.
# Time: O(m * n * log(m * n)). Space: O(m * n).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (binary search on value)
# "Binary search on the answer value v.
# Count elements <= v: for row i, elements are i,2i,3i,...,n*i.
# Count of these <= v is min(v//i, n). Sum over all rows i=1..m.
# Binary search for smallest v where count >= k.
# lo=1, hi=m*n. Time: O(m log(m*n))."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# m=3,n=3,k=5. Table: [1,2,3,2,4,6,3,6,9] sorted=[1,2,2,3,3,4,6,6,9]. 5th=3.
# count_le(3)=count≥5? i=1:min(3,3)=3, i=2:min(1,3)=1, i=3:min(1,3)=1. sum=5≥5 → hi=3.
# ... lo converges to 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m log(m*n)). Space O(1). The count formula is the key — O(m) per binary search step.
# Note: the answer might not be a 'real' table entry — binary search finds the smallest v
# with count >= k, which IS always a table entry because count jumps discretely at table values.
# Given more time I'd ask: kth largest? Binary search on m*n - k + 1 smallest instead."
```

Binary Search

#710 Random Pick with Blacklist

HAR

Open on LeetCode

Array · Hash Table · Math · Binary Search · Sorting · Randomized

Lists: Denny Zhang

Problem

You are given an integer n and an array of unique integers blacklist. Design an algorithm to pick a random integer in the range [0, n - 1] that is not in blacklist. Any integer that is in the mentioned range and not in blacklist should be equally likely to be returned.

Optimize your algorithm such that it minimizes the number of calls to the built-in random function of your language.

Implement the Solution class:

- Solution(int n, int[] blacklist) Initializes the object with the integer n and the blacklisted integers blacklist.
- int pick() Returns a random integer in the range [0, n - 1] and not in blacklist.

Example 1:

```
Input
["Solution", "pick", "pick", "pick", "pick", "pick", "pick", "pick"]
[[7, [2, 3, 5]], [], [], [], [], [], [], []]
Output
[null, 0, 4, 1, 6, 1, 0, 4]

Explanation
Solution solution = new Solution(7, [2, 3, 5]);
```

Constraints:

- 1 <= n <= 109
- 0 <= blacklist.length <= min(105, n - 1)
- 0 <= blacklist[i] < n
- All the values of blacklist are unique.
- At most 2 * 104 calls will be made to pick.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Pick uniformly at random from [0, n-1] excluding blacklist. One call to random per pick. Right?
# n can be 10^9 — can't enumerate all valid numbers."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Store all non-blacklisted ids in a list; pick random index.
# If blacklist is small, filter on each pick — O(n) per draw.
# Time: O(n) per pick naive. Space: O(n) whitelist.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (virtual mapping)
# "Let sz = n - len(blacklist) = count of valid numbers.
# Map [0, sz) as the 'virtual' range. Numbers in [0, sz) that ARE blacklisted
# get remapped to non-blacklisted numbers in [sz, n).
# pick(): random in [0, sz) → if in remap → use remap[r], else use r directly.
# Build remap in __init__: for each blacklisted number in [0, sz), map it to a non-blacklisted
# number in [sz, n). O(|blacklist|) init, O(1) per pick."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=7, blacklist=[2,3,5]. sz=4. Valid: [0,1,4,6].
# Blacklisted in [0,4): 2,3. Map 2→6, 3→4 (5 is blacklisted, skip → 4).
# remap={2:6, 3:4}. pick(0)→0, pick(1)→1, pick(2)→6, pick(3)→4. All valid ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Init O(|B|). Pick O(1). Space O(|B|) for remap.
# The key insight: divide [0,n) into 'virtual valid' [0,sz) and 'tail' [sz,n).
# Blacklisted numbers in [0,sz) redirect to valid numbers in the tail.
# Given more time I'd ask: what if the blacklist updates dynamically?
# Rebuild the remap on each update — O(|B|) per update, same O(1) pick."
```


Binary Search

#774 Minimize Max Distance to Gas Station

HAR

Open on LeetCode

Array · Binary Search

Lists: Denny Zhang

Problem

You are given an integer array stations that represents the positions of the gas stations on the x-axis. You are also given an integer k.

You should add k new gas stations. You can add the stations anywhere on the x-axis, and not necessarily on an integer position.

Let penalty() be the maximum distance between adjacent gas stations after adding the k new stations.

Return the smallest possible value of penalty(). Answers within 10-6 of the actual answer will be accepted.

Example 1:

```
Input: stations = [1,2,3,4,5,6,7,8,9,10], k = 9
Output: 0.50000
```

Example 2:

```
Input: stations = [23,24,36,39,46,56,57,65,84,98], k = 1
Output: 14.00000
```

Constraints:

- 10 <= stations.length <= 2000
- 0 <= stations[i] <= 108
- stations is sorted in a strictly increasing order.
- 1 <= k <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Add exactly k stations anywhere (real-valued positions) to minimise the maximum gap.
# Return the answer within 1e-6 precision. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Binary search on the maximum distance between adjacent stations.
# For candidate d, check if we can add k stations so all gaps <= d (greedy fill).
# Brute: try every real-valued spacing without binary search — continuous search space.
# Time: O(n * precision) naive scan. Space: O(1).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (binary search on floating-point answer)
# "Binary search on the max gap d. Feasibility: for each existing gap g, we need ceil(g/d)-1 extra
# stations to keep all sub-gaps ≤ d. Count total extras; if ≤ k, d is achievable.
# Precision: run until hi-lo < 1e-7."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * 100) = O(n). Space O(n) for gaps. Floating-point binary search with fixed iterations.
# Note: int(g/d) gives the number of stations to insert (not ceil-1 — same thing via floor division).
# Given more time I'd ask: what if stations must be at integer positions?
# Binary search on integers, same feasibility check but with math.ceil."
```

Binary Search

#1235 Maximum Profit in Job Scheduling

HAR

Open on LeetCode

Array · Binary Search · Dynamic Programming · Sorting

Lists: Grind 169

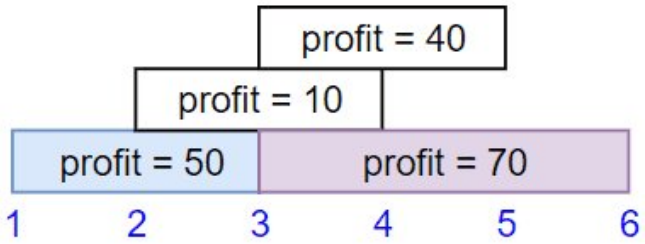
Problem

We have n jobs, where every job is scheduled to be done from startTime[i] to endTime[i], obtaining a profit of profit[i].

You're given the startTime, endTime and profit arrays, return the maximum profit you can take such that there are no two jobs in the subset with overlapping time range.

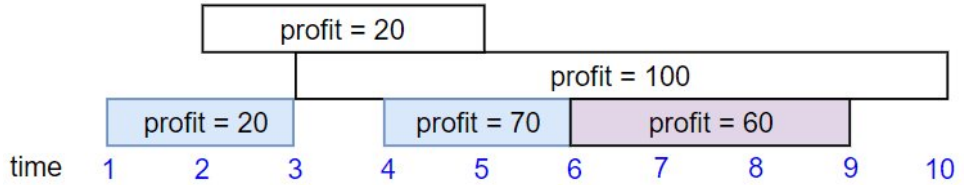
If you choose a job that ends at time X you will be able to start another job that starts at time X.

Example 1:



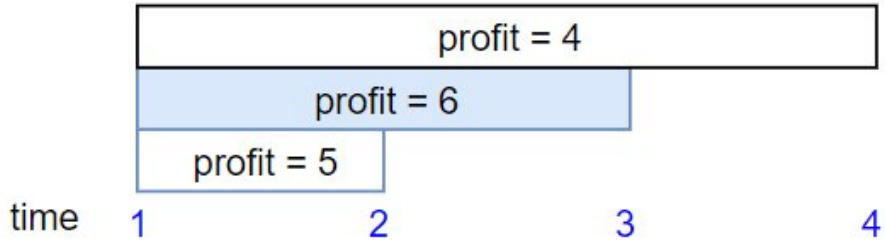

```
Input: startTime = [1,2,3,3], endTime = [3,4,5,6], profit = [50,10,40,70]
Output: 120
Explanation: The subset chosen is the first and fourth job.
Time range [1-3]+[3-6] , we get profit of 120 = 50 + 70.
```

Example 2:




```
Input: startTime = [1,2,3,4,6], endTime = [3,5,10,6,9], profit = [20,20,100,70,60]
Output: 150
Explanation: The subset chosen is the first, fourth and fifth job.
Profit obtained 150 = 20 + 70 + 60.
```

Example 3:




```
Input: startTime = [1,1,1], endTime = [2,3,4], profit = [5,6,4]
Output: 6
```

Constraints:

- 1 <= startTime.length == endTime.length == profit.length <= 5 * 104
- 1 <= startTime[i] < endTime[i] <= 109

- 1 <= profit[i] <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Select non-overlapping jobs to maximise total profit.
# Job ending at X and starting at X don't overlap. Return max profit. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Sort jobs by end time. DP over index: dp[i] = max profit taking job i plus best non-overlapping before i.
# For each i, scan all j < i for compatibility — O(n^2) DP.
# Time: O(n^2). Space: O(n) DP + sort.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (sort by end time + DP + binary search)
# "Sort jobs by end time. dp[i] = max profit using first i jobs.
# For each job i: skip → dp[i] = dp[i-1]. Take → find latest j where job[j].end <= job[i].start
# using binary search. dp[i] = max(dp[i-1], dp[j] + profit[i]).
# Time: O(n log n)."

# PHASE 4 — CLEAN CODE
        # Find latest job whose end <= start of current

# PHASE 5 — TEST & DEBUG
# jobs sorted by end: [(3,1,50),(4,2,10),(5,3,40),(6,3,70)].
# job(3,1,50): j=bisect_right([0],1)-1=0. dp[0]+50=50>dp[-1]=0 → dp=[0,50],ends=[0,3].
# job(4,2,10): bisect_right([0,3],2)-1=0. dp[0]+10=10<dp[-1]=50 → dp=[0,50,50],ends=[0,3,4].
# job(5,3,40): bisect_right([0,3,4],3)-1=1. dp[1]+40=90>50 → dp=[0,50,50,90].
# job(6,3,70): bisect_right([0,3,4,5],3)-1=1. dp[1]+70=120>90 → dp=[0,50,50,90,120]. → 120 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n). Space O(n). Sort + DP with binary search per step.
# The dp array is monotonically non-decreasing — binary search on ends is valid.
# Given more time I'd ask: what if we want to select at most k jobs?
# Add a dimension to dp — dp[i][k] — O(n²k) with the same binary search for last compatible job."
```

Matrix

Round 3 · High · Hard · 1 question

Matrix

#1074 Number of Submatrices That Sum to Target

HAR

Open on LeetCode

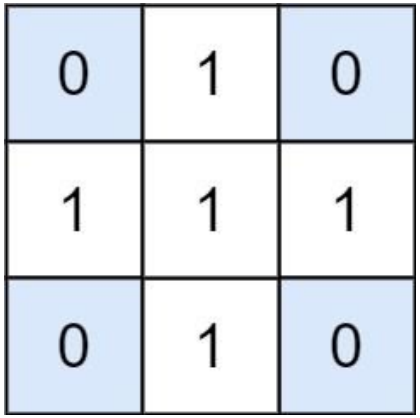
Array · Hash Table · Matrix · Prefix Sum

Lists: Denny Zhang

Problem

Given a matrix and a target, return the number of non-empty submatrices that sum to target.
A submatrix x1, y1, x2, y2 is the set of all cells matrix[x][y] with x1 <= x <= x2 and y1 <= y <= y2.
Two submatrices (x1, y1, x2, y2) and (x1', y1', x2', y2') are different if they have some coordinate that is different: for example, if x1 != x1'.

Example 1:




```
Input: matrix = [[0,1,0],[1,1,1],[0,1,0]], target = 0
Output: 4
Explanation: The four 1x1 submatrices that only contain 0.
```

Example 2:

```
Input: matrix = [[1,-1],[-1,1]], target = 0
Output: 5
Explanation: The two 1x2 submatrices, plus the two 2x1 submatrices, plus the 2x2 submatrix.
```

Example 3:

```
Input: matrix = [[904]], target = 0
Output: 0
```

Constraints:

- **1 <= matrix.length <= 100**
- **1 <= matrix[0].length <= 100**
- **-1000 <= matrix[i][j] <= 1000**
- **-10^8 <= target <= 10^8**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count all submatrices (r1,c1,r2,c2) whose elements sum to target. Right?
# Extend #560 Subarray Sum Equals K to 2D."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Compute 2D prefix sums, then enumerate all submatrix top-left and bottom-right corners.
# For each submatrix, O(1) range sum check against target.
# Four nested loops over corners → O(n^2 * m^2) submatrices.
# Time: O(n^2 * m^2). Space: O(n * m) prefix grid.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE
# "Fix top row r1 and bottom row r2. Compress columns into 1D prefix sums between r1..r2.
# Then apply #560 (subarray sum = target) on the 1D array.
# Time: O(m² * n). Space: O(n)."

# PHASE 4 — CLEAN CODE
                # Apply #560 on col_sum
```

```
# PHASE 5 — TEST & DEBUG
# [[0,1,0],[1,1,1],[0,1,0]], target=0:
# Fix r1=0,r2=0: col_sum=[0,1,0]. Subarrays summing to 0: [0] at c=0, [0] at c=2. count=2.
# Other rows contribute 2 more. → 4 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m² * n). Space O(n). This is literally #560 applied inside a 2D row-fixing loop.
# Name that connection — it shows pattern transfer.
# Given more time I'd ask: count submatrices summing to at most target?
# Use #862 (Shortest Subarray with Sum at Least K) approach — monotone deque instead of hash map."
```


· #1235 #124 · · Contents

Trees & BST

Round 3 · High · Hard · 2 questions

Trees & BST

#124 Binary Tree Maximum Path Sum

HAR

Open on LeetCode

Dynamic Programming · Tree · DFS

Lists: Grind 169

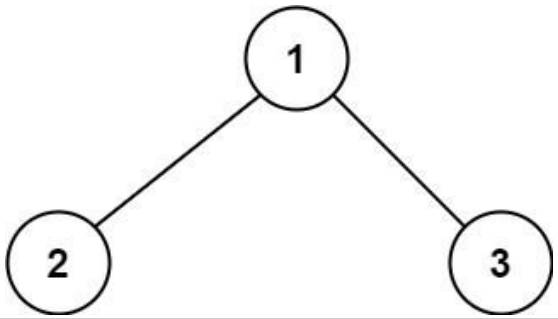
Problem

A path in a binary tree is a sequence of nodes where each pair of adjacent nodes in the sequence has an edge connecting them. A node can only appear in the sequence at most once. Note that the path does not need to pass through the root.

The path sum of a path is the sum of the node's values in the path.

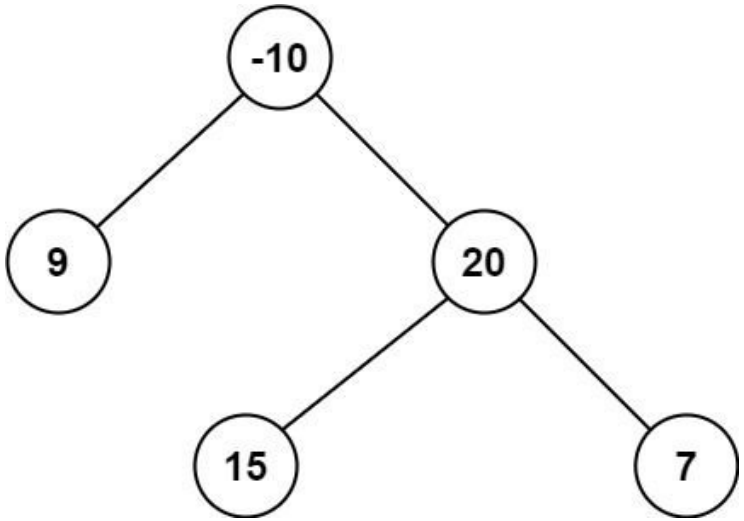
Given the root of a binary tree, return the maximum path sum of any non-empty path.

Example 1:




```
Input: root = [1,2,3]
Output: 6
Explanation: The optimal path is 2 -> 1 -> 3 with a path sum of 2 + 1 + 3 = 6.
```

Example 2:




```
Input: root = [-10,9,20,null,null,15,7]
Output: 42
Explanation: The optimal path is 15 -> 20 -> 7 with a path sum of 15 + 20 + 7 = 42.
```

Constraints:

- The number of nodes in the tree is in the range [1, 3 * 10^4].
- -1000 <= Node.val <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Path = any sequence of connected nodes (no revisiting). Path doesn't need to pass through root.
# Values can be negative. Return maximum path sum. Right?"
#
# "[-10,9,20,null,null,15,7]: path 15→20→7 = 42 ✓"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# At every node, compute max path sum through that node = node.val + max(0, left_gain) + max(0, right_gain).
# Recurse full tree; track global max. Recomputes subtree gains repeatedly without sharing.
# Time: O(n^2) worst case on skewed tree. Space: O(h) stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (post-order, same as Diameter)
# "For each node, compute the 'gain' if we extend a path through it: max(0, left_gain, right_gain) + node.val.
# The best path THROUGH this node = node.val + max(0,left) + max(0,right).
# Update global max. Return one-sided gain to parent (can't go both left and right AND up).
# Use inner helper — no self."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [-10,9,20,null,null,15,7]:
# gain(9)=9. gain(15)=15. gain(7)=7.
# gain(20): left=15,right=7. best=max(-inf,20+15+7)=42. return 20+15=35.
# gain(-10): left=9,right=35. best=max(42,-10+9+35)=42. return -10+35=25.
# → 42 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(h). Same post-order pattern as Diameter of Binary Tree (#543) and Balanced Tree (#110).
# The 'max(0, gain)' is essential — a negative subtree gain should be dropped.
# Given more time I'd ask: return the actual path (not just the sum)?
# Track which path achieved best[0] — store node references alongside the max update."
```

Trees & BST

#297 Serialize and Deserialize Binary Tree

HAR

Open on LeetCode

String · Tree · DFS · BFS · Design

Lists: Grind 169

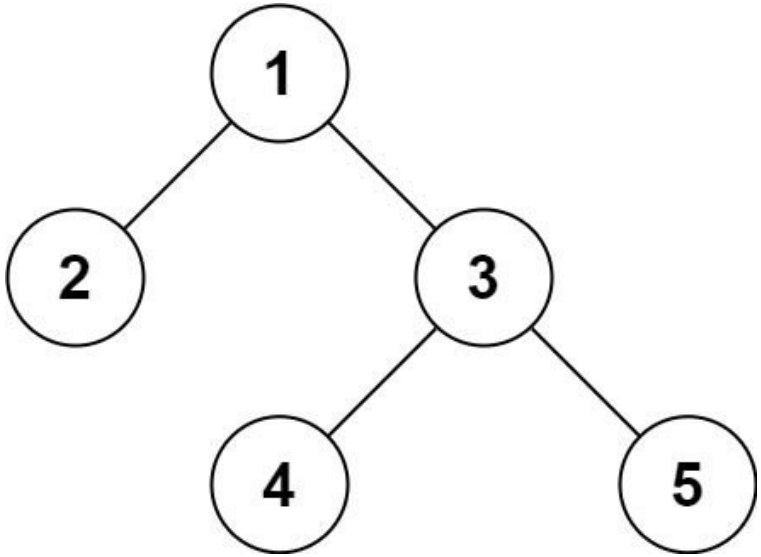
Problem

Serialization is the process of converting a data structure or object into a sequence of bits so that it can be stored in a file or memory buffer, or transmitted across a network connection link to be reconstructed later in the same or another computer environment.

Design an algorithm to serialize and deserialize a binary tree. There is no restriction on how your serialization/deserialization algorithm should work. You just need to ensure that a binary tree can be serialized to a string and this string can be deserialized to the original tree structure.

Clarification: The input/output format is the same as how LeetCode serializes a binary tree. You do not necessarily need to follow this format, so please be creative and come up with different approaches yourself.

Example 1:




```
Input: root = [1,2,3,null,null,4,5]
Output: [1,2,3,null,null,4,5]
```

Example 2:

```
Input: root = []
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 104].
- -1000 <= Node.val <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design encode and decode functions for a binary tree. Any format is fine as long as it round-trips.
# Empty tree encodes to some valid string. Right?"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Serialize: preorder traversal with 'null' markers for empty children ? comma string.
# Deserialize: split queue, rebuild tree recursively matching preorder order.
# Correct but string can be long; BFS level-order encoding is more compact.
# Time: O(n). Space: O(n) output string.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE (pre-order DFS with null markers)
# "Serialize: pre-order traversal, 'N' for null, comma-separated.
```



```
# Deserialize: split, use an iterator, recursively build from pre-order.
# Pre-order uniquely determines the tree when nulls are included."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# [1,2,3,null,null,4,5]: serialize → '1,2,N,N,3,4,N,N,5,N,N'. deserialize rebuilds the tree ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(n). Pre-order with null markers is the cleanest approach.
# BFS alternative: level-order like LeetCode's own format — works but trickier to parse.
# Given more time I'd ask: serialize a general N-ary tree?
# Add child count before children — e.g. '1,3,2,0,3,0,4,0,5,0' for a 3-child root."
```

DFS

Round 3 · High · Hard · 5 questions

DFS

★ #269 Alien Dictionary ★ HAR

Open on LeetCode

Array · String · DFS · BFS · Graph · Topological Sort

Lists: ★ Premium | Grind 169 | Premium 98

Problem

There is a new alien language that uses the English alphabet. However, the order of the letters is unknown to you.

You are given a list of strings words from the alien language's dictionary. Now it is claimed that the strings in words are sorted lexicographically by the rules of this new language.

If this claim is incorrect, and the given arrangement of string in words cannot correspond to any order of letters, return "".

Otherwise, return a string of the unique letters in the new alien language sorted in lexicographically increasing order by the new language's rules. If there are multiple solutions, return any of them.

Example 1:

```
Input: words = ["wrt","wrf","er","ett","rftt"]
Output: "wertf"
```

Example 2:

```
Input: words = ["z","x"]
Output: "zx"
```

Example 3:

```
Input: words = ["z","x","z"]
Output: ""
Explanation: The order is invalid, so return "".
```

Constraints:

- 1 <= words.length <= 100
- 1 <= words[i].length <= 100
- words[i] consists of only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given words sorted in alien lexicographic order, deduce the letter ordering.
# Return any valid ordering, or '' if impossible (cycle or contradiction). Right?
# Contradiction: longer word precedes its prefix (e.g. ['ab','a'])."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build graph: for each adjacent pair in words, record char order u before v.
# Topological sort the chars; if cycle detected, no valid order.
# Compare words character by character to infer edges.
# Time: O(C) total chars. Space: O(1) — 26 letters.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Kahn's BFS topological sort)
# "Extract ordering constraints from consecutive word pairs: compare char by char, first difference gives an edge.
# Build directed graph + in-degree map.
# BFS: process zero-in-degree nodes. If we process all letters → valid. Else cycle → return ''."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# ["wrt","wrf","er","ett","rftt"]: edges: t→f, w→e, r→t, e→r. indegree: f:1,e:1,t:1,r:1,w:0.
# BFS from w: w→e→r→t→f. result='wertf' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(C) where C = total characters. Space O(1) — at most 26 letters.
# The prefix check catches the only invalid case that doesn't manifest as a graph cycle.
# Given more time I'd ask: return ALL valid orderings?
# Backtracking on the topological sort — collect all linearisations."
```

DFS

#329 Longest Increasing Path in a Matrix

HAR

Open on LeetCode

Array · DFS · BFS · Graph · Topological Sort · Memoization · Matrix

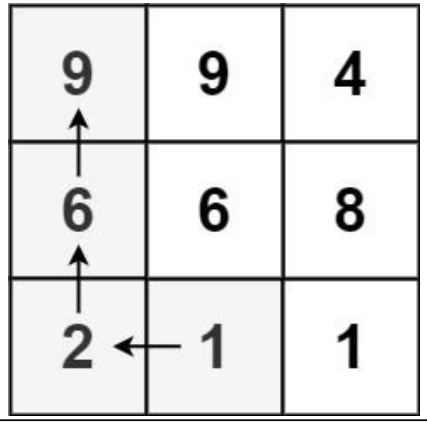
Lists: Grind 169

Problem

Given an m x n integers matrix, return the length of the longest increasing path in matrix.

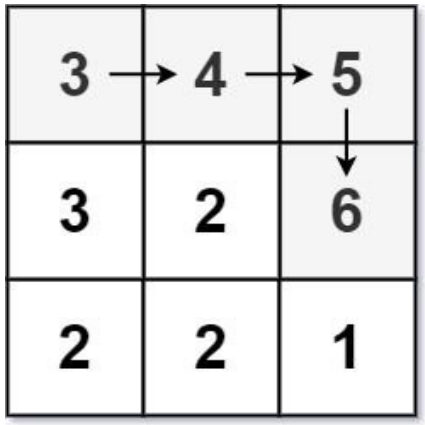
From each cell, you can either move in four directions: left, right, up, or down. You may not move diagonally or move outside the boundary (i.e., wrap-around is not allowed).

Example 1:




```
Input: matrix = [[9,9,4],[6,6,8],[2,1,1]]
Output: 4
Explanation: The longest increasing path is [1, 2, 6, 9].
```

Example 2:




```
Input: matrix = [[3,4,5],[3,2,6],[2,2,1]]
Output: 4
Explanation: The longest increasing path is [3, 4, 5, 6]. Moving diagonally is not allowed.
```

Example 3:

```
Input: matrix = [[1]]
Output: 1
```


Constraints:

- m == matrix.length
- n == matrix[i].length
- 1 <= m, n <= 200
- 0 <= matrix[i][j] <= 231 - 1

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Longest strictly increasing path (4-directional). Return path length. Right?
# Values can repeat but path must be strictly increasing."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# DFS from every cell with memo: longest increasing path starting at (r,c).
# Only move to strictly larger neighbors; memo avoids recomputing subpaths.
# Without memo, pure DFS retries paths exponentially.
# Time: O(m * n) with memo. Space: O(m * n) memo + stack.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (DFS + memoization)
# "DFS from each cell. dp[r][c] = longest path starting at (r,c).
# Only move to neighbours with strictly greater value — no cycles possible → no visited set needed.
# Memoize results: each cell computed once → O(m*n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[9,9,4],[6,6,8],[2,1,1]]: longest path starting at (2,1)=1: 1→2→6→9=4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(m*n) for memo. No visited set needed — strict increase prevents cycles.
# Topological sort alternative: build DAG (only edges to larger neighbours), count levels.
# Same O(m*n) but iterative.
# Given more time I'd ask: longest path with values differing by at most 1?
# Need a visited set now (can revisit same value) — different problem."
```

DFS

#685 Redundant Connection II

HAR

Open on LeetCode

DFS · BFS · Union Find · Graph

Lists: Denny Zhang

Problem

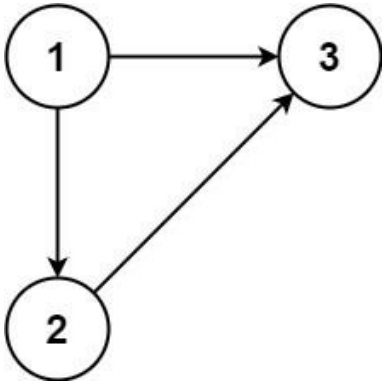
In this problem, a rooted tree is a directed graph such that, there is exactly one node (the root) for which all other nodes are descendants of this node, plus every node has exactly one parent, except for the root node which has no parents.

The given input is a directed graph that started as a rooted tree with n nodes (with distinct values from 1 to n), with one additional directed edge added. The added edge has two different vertices chosen from 1 to n, and was not an edge that already existed.

The resulting graph is given as a 2D-array of edges. Each element of edges is a pair [ui, vi] that represents a directed edge connecting nodes ui and vi, where ui is a parent of child vi.

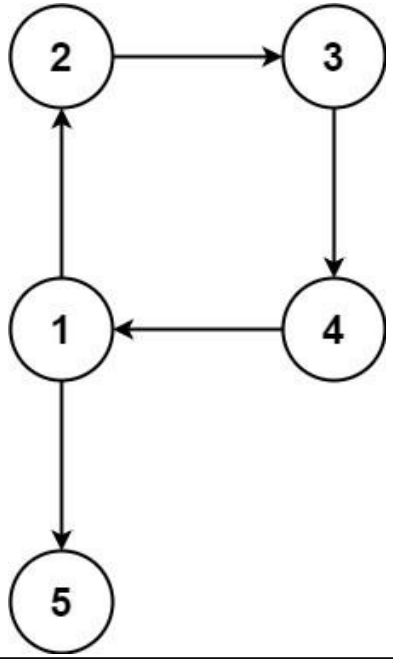
Return an edge that can be removed so that the resulting graph is a rooted tree of n nodes. If there are multiple answers, return the answer that occurs last in the given 2D-array.

Example 1:




```
Input: edges = [[1,2],[1,3],[2,3]]
Output: [2,3]
```

Example 2:




```
Input: edges = [[1,2],[2,3],[3,4],[4,1],[1,5]]
Output: [4,1]
```

Constraints:

- n == edges.length
- 3 <= n <= 1000
- edges[i].length == 2
- 1 <= ui, vi <= n
- ui != vi

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Directed graph: rooted tree + one extra edge. Find the edge to remove.
# If multiple answers, return the last one in input. Right?
# Extra edge causes: (1) a node with two parents, OR (2) a cycle, OR both."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build indegree + parent map from edges. Node with indegree 2 is the duplicate child candidate.
# Detect cycle with DFS or union-find on the graph minus one suspicious edge.
# Try removing each duplicate edge until graph becomes a valid tree.
# Time: O(n) with careful case analysis. Space: O(n).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (three-case analysis + Union-Find)
# "Track in-degree: if any node has two parents, record both candidate edges.
# Then run Union-Find ignoring the second parent candidate.
# Case 1 (two parents, no cycle in remaining): remove second candidate.
# Case 2 (two parents, cycle in remaining): remove first candidate.
# Case 3 (no node with two parents): cycle exists → last edge forming cycle."

# PHASE 4 — CLEAN CODE
        # Find if any node has two parents

# PHASE 5 — TEST & DEBUG
# [[1,2],[1,3],[2,3]]: node 3 has two parents (1 and 2). cand1=[1,3], cand2=[2,3].
# UF ignoring cand2=[2,3]: union(1,2) ok, union(1,3) ok. No cycle → return cand2=[2,3] ✓
# [[1,2],[2,3],[3,4],[4,1],[1,5]]: no node with two parents. UF finds cycle [4,1] → return [4,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * α(n)). Space O(n). Three cases arise from two root causes: double parent and cycle.
```

```
# The Union-Find handles both: cycle detection when cand2 is skipped tells us which candidate to remove.
# Given more time I'd ask: undirected version?
# That's Redundant Connection (#684) — standard Union-Find, return first edge that creates a cycle."
```


DFS

#1036 Escape a Large Maze

HAR

Open on LeetCode

Array · Hash Table · DFS · BFS

Lists: Denny Zhang

Problem

There is a 1 million by 1 million grid on an XY-plane, and the coordinates of each grid square are (x, y).

We start at the source = [sx, sy] square and want to reach the target = [tx, ty] square. There is also an array of blocked squares, where each blocked[i] = [xi, yi] represents a blocked square with coordinates (xi, yi).

Each move, we can walk one square north, east, south, or west if the square is not in the array of blocked squares. We are also not allowed to walk outside of the grid.

Return true if and only if it is possible to reach the target square from the source square through a sequence of valid moves.

Example 1:

```
Input: blocked = [[0,1],[1,0]], source = [0,0], target = [0,2]
Output: false
Explanation: The target square is inaccessible starting from the source square because we cannot move.
We cannot move north or east because those squares are blocked.
We cannot move south or west because we cannot go outside of the grid.
```

Example 2:

```
Input: blocked = [], source = [0,0], target = [999999,999999]
Output: true
Explanation: Because there are no blocked cells, it is possible to reach the target square.
```

Constraints:

- 0 <= blocked.length <= 200
- blocked[i].length == 2
- 0 <= xi, yi < 106
- source.length == target.length == 2
- 0 <= sx, sy, tx, ty < 106
- source != target
- It is guaranteed that source and target are not blocked.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "1M×1M grid. Up to 200 blocked cells. Can source reach target? Right?
# With at most 200 blocked cells in a semicircle, max enclosed area ≈ 200²/2 = 20,000 cells.
# So if BFS from source visits > 20,000 cells without finding target, source is NOT enclosed."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS from start cell; at each step try all 4 directions.
# Track visited cells; if we revisit within <= 500 moves, we are in a loop ? false.
# If we escape bounds within 500 steps per problem rule ? true.
# Time: O(m * n * 500) worst case. Space: O(m * n) visited.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS with cell limit)
# "Run limited BFS from BOTH source and target.
# If BFS visits > max_cells (= n*(n+1)//2 for n=len(blocked)) → not enclosed → True from that side.
# If BFS terminates (all cells explored) without reaching other endpoint → enclosed → False."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²) per BFS (at most n*(n+1)/2 cells). Space O(n²). n ≤ 200 → fast.
# The dual-BFS is essential: source might not be enclosed but target might be.
# Given more time I'd ask: what if blocked cells can be added dynamically?
# Re-run BFS on each addition — or maintain Union-Find of open cells."
```

DFS

#1192 Critical Connections in a Network

HAR

Open on LeetCode

DFS · Graph · Biconnected Component

Lists: Denny Zhang

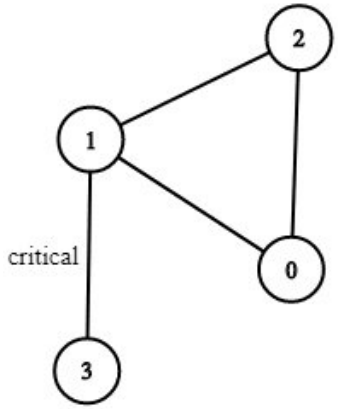
Problem

There are n servers numbered from 0 to n - 1 connected by undirected server-to-server connections forming a network where connections[i] = [ai, bi] represents a connection between servers ai and bi. Any server can reach other servers directly or indirectly through the network.

A critical connection is a connection that, if removed, will make some servers unable to reach some other server.

Return all critical connections in the network in any order.

Example 1:




```
Input: n = 4, connections = [[0,1],[1,2],[2,0],[1,3]]
Output: [[1,3]]
Explanation: [[3,1]] is also accepted.
```

Example 2:

```
Input: n = 2, connections = [[0,1]]
Output: [[0,1]]
```

Constraints:

- 2 <= n <= 105
- n - 1 <= connections.length <= 105
- 0 <= ai, bi <= n - 1
- ai != bi
- There are no repeated connections.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Critical connection (bridge) = edge whose removal disconnects the graph. Right?
# Use Tarjan's bridge-finding algorithm."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Find bridges with Tarjan low-link DFS: edge (u,v) is critical if low[v] > disc[u].
# One DFS pass tracks discovery time and lowest reachable.
# Naive alternative: remove each edge and test connectivity — O(E * (V+E)).
# Time: O(V + E) with Tarjan. Space: O(V + E).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Tarjan's DFS)
# "DFS assigning disc[u]=discovery time and low[u]=earliest discovery reachable.
# An edge (u,v) is a bridge if low[v] > disc[u] — meaning v cannot reach u's subtree without (u,v)."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(V+E). Space O(V+E). Tarjan's bridge algorithm in one DFS pass.
# low[v] > disc[u] means v's subtree can't 'back-connect' to u's subtree — edge is a bridge.
# Given more time I'd ask: articulation points instead of bridges?
```



```
# Similar but: u is an articulation point if any child v has low[v] >= disc[u] (and u is not root)."
```

Graphs

Round 3 · High · Hard · 2 questions

Graphs

★ #305 Number of Islands II ★

HAR

Open on LeetCode

Array · Hash Table · Union Find · Matrix

Lists: ★ Premium | Premium 98

Problem

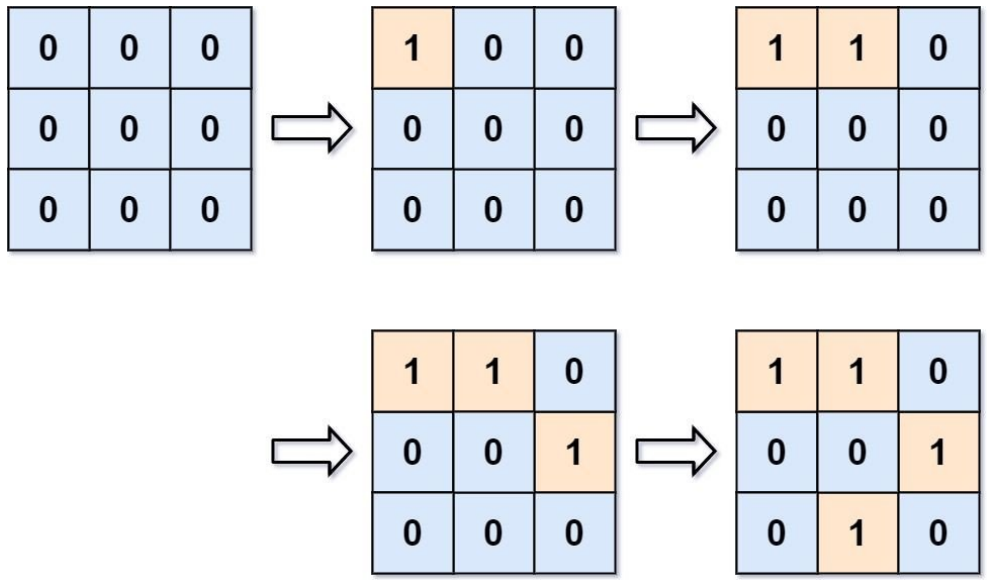
You are given an empty 2D binary grid grid of size m x n. The grid represents a map where 0's represent water and 1's represent land. Initially, all the cells of grid are water cells (i.e., all the cells are 0's).

We may perform an add land operation which turns the water at position into a land. You are given an array positions where positions[i] = [ri, ci] is the position (ri, ci) at which we should operate the ith operation.

Return an array of integers answer where answer[i] is the number of islands after turning the cell (ri, ci) into a land.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:




```
Input: m = 3, n = 3, positions = [[0,0],[0,1],[1,2],[2,1]]
Output: [1,1,2,3]
Explanation:
Initially, the 2d grid is filled with water.
- Operation #1: addLand(0, 0) turns the water at grid[0][0] into a land. We have 1 island. ↗
- Operation #2: addLand(0, 1) turns the water at grid[0][1] into a land. We still have 1 island. ↗
- Operation #3: addLand(1, 2) turns the water at grid[1][2] into a land. We have 2 islands. ↗
- Operation #4: addLand(2, 1) turns the water at grid[2][1] into a land. We have 3 islands. ↗
```

Example 2:

```
Input: m = 1, n = 1, positions = [[0,0]]
Output: [1]
```

Constraints:

- 1 <= m, n, positions.length <= 104
- 1 <= m * n <= 104
- positions[i].length == 2
- 0 <= ri < m
- 0 <= ci < n

Follow up: Could you solve it in time complexity O(k log(mn)), where k == positions.length?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Start with empty grid. Add land cells one at a time. After each add, return island count.
# O(k log mn) per operation. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# After each addLand, scan the whole grid and flood-fill count islands from scratch.
# k addLand calls each trigger full O(m*n) recount — too slow when k is large.
# Time: O(k * m * n). Space: O(m * n) per flood.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (Union-Find with dynamic updates)
# "Each new land cell starts as its own island (count++).
# Check 4 neighbours: for each land neighbour, if different component, union and count--."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(k * α(mn)) ≈ O(k). Space O(k). Union-Find handles dynamic connectivity perfectly.
# The duplicate position check avoids double-counting islands.
# Given more time I'd ask: what if land cells can be removed too?
# Split-Find is hard. Offline deletion via reverse addition is a common trick."
```


Graphs

#1153 String Transforms Into Another String HAR

Open on LeetCode

Hash Table · String · Union Find

Lists: Denny Zhang

Problem

Given two strings str1 and str2 of the same length, determine whether you can transform str1 into str2 by doing zero or more conversions.

In one conversion you can convert all occurrences of one character in str1 to any other lowercase English character.

Return true if and only if you can transform str1 into str2.

Example 1:

```
Input: str1 = "aabcc", str2 = "ccdee"
Output: true
Explanation: Convert 'c' to 'e' then 'b' to 'd' then 'a' to 'c'. Note that the order of conversions matter.
```

Example 2:

```
Input: str1 = "leetcode", str2 = "codeleet"
Output: false
Explanation: There is no way to transform str1 to str2.
```

Constraints:

- 1 <= str1.length == str2.length <= 104
- str1 and str2 contain only lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can str1 be converted to str2 by repeatedly mapping one character to another (all occurrences)?
# Conversions can be chained but must be consistent. Right?"
#
# "If str1==str2 trivially True. If str2 uses all 26 letters → no 'buffer' character available → False."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build a map from each character to its index in the first string.
# Scan the second string left-to-right: next char must appear at a strictly higher index.
# If any char is missing or out of order, transformation is impossible.
# Time: O(n). Space: O(1) — fixed alphabet size 26.
# Should I go ahead and optimize?"

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(26). The len(set(str2)) < 26 check: if all 26 chars appear in str2,
# every potential buffer letter is 'locked' to a mapping — cycles can't be broken.
# Given more time I'd ask: what if conversions can only be applied once?
# Topological sort on the conversion graph — acyclic → True."
```

BFS

Round 3 · High · Hard · 4 questions

BFS

#127 Word Ladder HAR

Open on LeetCode

Hash Table · String · BFS

Lists: Grind 169

Problem

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that:

- Every adjacent pair of words differs by a single letter.
- Every si for 1 <= i <= k is in wordList. Note that beginWord does not need to be in wordList.
- sk == endWord

Given two words, beginWord and endWord, and a dictionary wordList, return the number of words in the shortest transformation sequence from beginWord to endWord, or 0 if no such sequence exists.

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: 5
Explanation: One shortest transformation sequence is "hit" -> "hot" -> "dot" -> "dog" -> cog", which is 5 words long.
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: 0
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.
```

Constraints:

- 1 <= beginWord.length <= 10
- endWord.length == beginWord.length
- 1 <= wordList.length <= 5000
- wordList[i].length == beginWord.length
- beginWord, endWord, and wordList[i] consist of lowercase English letters.
- beginWord != endWord
- All the words in wordList are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Shortest word transformation from beginWord to endWord, changing one letter at a time.
# Each intermediate word must be in wordList. Return total words in shortest path, or 0. Right?"

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build adjacency from word patterns (one-letter-apart neighbors).
# BFS from beginWord layer by layer until endWord is reached; each layer = one transformation.
# Visited set prevents revisiting words. Return BFS depth or 0 if unreachable.
# Time: O(N * L^2) where N = dictionary size, L = word length. Space: O(N).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS with pattern matching)
# "BFS on words. For each current word, try replacing each position with 26 letters.
# If the result is in wordSet (and not visited), enqueue it.
# Alternatively: build adjacency by pattern (*ot, h*t, ho*) → group words by pattern — O(L²*n)."

# PHASE 4 — CLEAN CODE

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * L * 26) = O(n*L). Space O(n*L). Standard BFS shortest path.
# Bidirectional BFS: search simultaneously from beginWord and endWord — roughly halves depth.
# Given more time I'd ask: return all shortest paths?
# That's Word Ladder (#126) — BFS to build levels, then DFS to backtrack paths."
```

BFS

⋆ #317 Shortest Distance from All Buildings ⋆ HAR

Open on LeetCode

Array · BFS · Matrix

Lists: ⋆ Premium | Premium 98

Problem

You are given an m x n grid grid of values 0, 1, or 2, where:

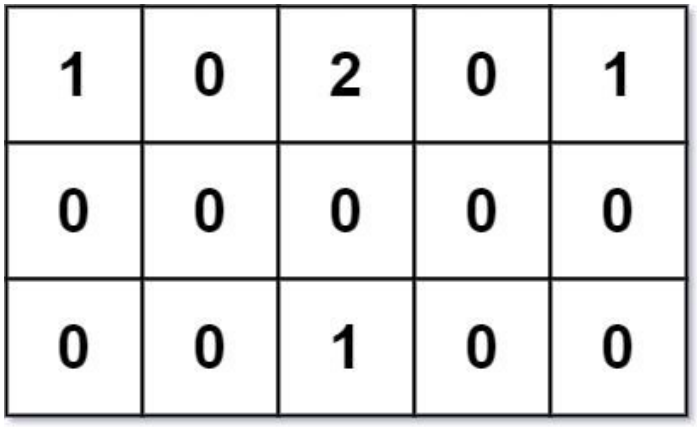
- each 0 marks an empty land that you can pass by freely,
- each 1 marks a building that you cannot pass through, and
- each 2 marks an obstacle that you cannot pass through.

You want to build a house on an empty land that reaches all buildings in the shortest total travel distance. You can only move up, down, left, and right.

Return the shortest travel distance for such a house. If it is not possible to build such a house according to the above rules, return -1.

The total travel distance is the sum of the distances between the houses of the friends and the meeting point.

Example 1:




```
Input: grid = [[1,0,2,0,1],[0,0,0,0,0],[0,0,1,0,0]]
Output: 7
Explanation: Given three buildings at (0,0), (0,4), (2,2), and an obstacle at (0,2).
The point (1,2) is an ideal empty land to build a house, as the total travel distance of 3+3+1=7 is minimal.
So return 7.
```

Example 2:

```
Input: grid = [[1,0]]
Output: 1
```

Example 3:

```
Input: grid = [[1]]
Output: -1
```

Constraints:

- m == grid.length
- n == grid[i].length
- 1 <= m, n <= 50
- grid[i][j] is either 0, 1, or 2.
- There will be at least one building in the grid.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find empty land (0) minimising total BFS distance to all buildings (1).
# Obstacles (2) can't be passed. Return -1 if no valid land. Right?"
# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# For each empty cell, BFS to sum distances to all buildings.
# Track total distance per cell; answer is cell with minimum total (must reach all buildings).
```



```
# Repeat BFS from every building — O(B * m * n) where B = buildings.
# Time: O(B * m * n). Space: O(m * n) BFS state.
# Should I go ahead and optimize?"
# PHASE 3 — OPTIMIZE (BFS from each building, accumulate distances)
# "BFS from each building cell. For each empty cell, accumulate distance and increment reach_count.
# Only cells reachable from ALL buildings are candidates. Return min total distance."
# PHASE 4 — CLEAN CODE
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(buildings * m * n). Space O(m*n). BFS from each building accumulates into dist[].
# reach[] ensures we only pick land reachable from ALL buildings.
# Given more time I'd ask: what if buildings can move? Re-run BFS on each query."
```

BFS

#773 Sliding Puzzle HAR

Open on LeetCode

Array · BFS · Matrix

Lists: Denny Zhang

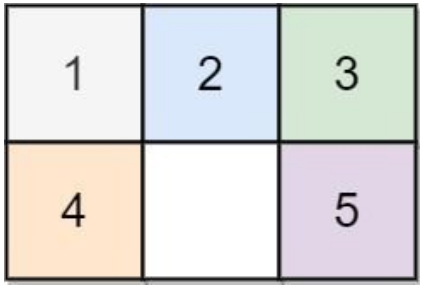
Problem

On an 2 x 3 board, there are five tiles labeled from 1 to 5, and an empty square represented by 0. A move consists of choosing 0 and a 4-directionally adjacent number and swapping it.

The state of the board is solved if and only if the board is [[1,2,3],[4,5,0]].

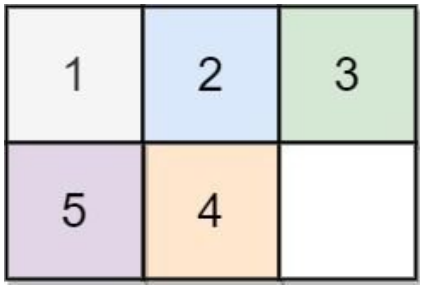
Given the puzzle board board, return the least number of moves required so that the state of the board is solved. If it is impossible for the state of the board to be solved, return -1.

Example 1:




```
Input: board = [[1,2,3],[4,0,5]]
Output: 1
Explanation: Swap the 0 and the 5 in one move.
```

Example 2:




```
Input: board = [[1,2,3],[5,4,0]]
Output: -1
Explanation: No number of moves will make the board solved.
```

Example 3:


```
Input: board = [[4,1,2],[5,0,3]]
Output: 5
Explanation: 5 is the smallest number of moves that solves the board.
An example path:
After move 0: [[4,1,2],[5,0,3]]
After move 1: [[4,1,2],[0,5,3]]
After move 2: [[0,1,2],[4,5,3]]
After move 3: [[1,0,2],[4,5,3]]
```

Constraints:

- board.length == 2
- board[i].length == 3
- 0 <= board[i][j] <= 5
- Each value board[i][j] is unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "2×3 board. Swap 0 with adjacent tile. Minimum moves to reach [[1,2,3],[4,5,0]]. Right?
# Return -1 if unsolvable."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# BFS on board states: encode 2x3 board as tuple string; enqueue all legal swaps of '0'.
# First time we reach target state, return move count.
# State space is at most 6! = 720 permutations.
# Time: O(6!). Space: O(6!) visited states.
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS on serialised board state)
# "Represent board as a 6-char string. BFS on states.
# Precompute which positions in the flat string can swap with position i (adjacency map).
# Goal state: '123450'."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[1,2,3],[4,0,5]] → '123405'. Swap 0 at pos 4 with 5 at pos 5 → '123450' = GOAL → 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "States = 6! / 2 = 360 reachable states (half of 6! are unsolvable). BFS on 360 states.
# The hardcoded NEIGHBORS map is the key — it captures the 2D adjacency in 1D string indexing.
# Given more time I'd ask: generalise to n×m board?
# Same BFS approach — NEIGHBORS changes, state space = (n*m)! / 2."
```

BFS

#815 Bus Routes HAR

Open on LeetCode

Array · Hash Table · BFS

Lists: Grind 169

Problem

You are given an array routes representing bus routes where routes[i] is a bus route that the ith bus repeats forever.

- For example, if routes[0] = [1, 5, 7], this means that the 0th bus travels in the sequence 1 -> 5 -> 7 -> 1 -> 5 -> 7 -> 1 -> ... forever.

You will start at the bus stop source (You are not on any bus initially), and you want to go to the bus stop target. You can travel between bus stops by buses only.

Return the least number of buses you must take to travel from source to target. Return -1 if it is not possible.

Example 1:

```
Input: routes = [[1,2,7],[3,6,7]], source = 1, target = 6
Output: 2
Explanation: The best strategy is take the first bus to the bus stop 7, then take the second bus to the bus stop 6.
```

Example 2:

```
Input: routes = [[7,12],[4,5,15],[6],[15,19],[9,12,13]], source = 15, target = 12
Output: -1
```

Constraints:

- 1 <= routes.length <= 500.
- 1 <= routes[i].length <= 105
- All the values of routes[i] are unique.
- sum(routes[i].length) <= 105
- 0 <= routes[i][j] < 106
- 0 <= source, target < 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Minimum number of BUSES (not stops) to travel from source to target. Right?
# Each bus covers its entire route — stepping onto a bus travels all its stops."

# PHASE 2 — BRUTE FORCE
# "Let me start with brute force to lock in the logic.
# Build stop->routes map. BFS from source stop; each hop can board any route containing current stop.
# Track min buses to reach target; mark routes visited to avoid reboarding.
# Time: O(R * S) routes and stops. Space: O(R + S).
# Should I go ahead and optimize?"

# PHASE 3 — OPTIMIZE (BFS on routes, not stops)
# "Build stop→routes map. BFS where each 'node' is a bus route.
# Start: all routes containing source. Each step: board a new route (adjacent if sharing a stop).
# Count route changes (buses taken). Avoid revisiting routes and stops."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# routes=[[1,2,7],[3,6,7]], source=1, target=6:
# queue=[(1,0)]. route 0 for stop 1: stops 2,7. Not target. queue=[(2,1),(7,1)].
# stop 7: route 1 for stop 7: stops 3,6. 6==target → return 1+1=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(sum of route lengths). Space O(same). BFS on routes, not stops — the key insight.
# Visiting stops prevents re-enqueuing; visiting routes prevents re-processing entire routes.
# Given more time I'd ask: find all paths using at most k buses?
# Keep track of bus count per path — BFS naturally handles this."
```


Round 4 · Mid · Easy

Round 4

Mid Priority · Easy

12 questions · 4 patterns

Linked List #21 #141 #206 #234 #706 #876 #1474

Stack #20 #232 #844

Trie #14

Greedy #409

Linked List

Round 4 · Mid · Easy · 7 questions

Linked List

#21 Merge Two Sorted Lists

EAS

Open on LeetCode

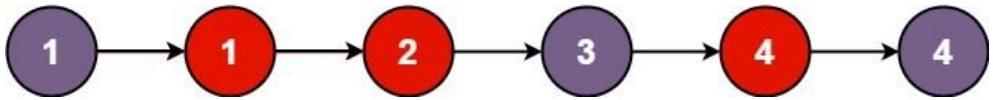
Linked List · Recursion

Lists: Grind 169

Problem

You are given the heads of two sorted linked lists list1 and list2.
Merge the two lists into one sorted list. The list should be made by splicing together the nodes of the first two lists.
Return the head of the merged linked list.

Example 1:




```
Input: list1 = [1,2,4], list2 = [1,3,4]
Output: [1,1,2,3,4,4]
```

Example 2:

```
Input: list1 = [], list2 = []
Output: []
```

Example 3:

```
Input: list1 = [], list2 = [0]
Output: [0]
```

Constraints:

- The number of nodes in both lists is in the range [0, 50].
- -100 <= Node.val <= 100
- Both list1 and list2 are sorted in non-decreasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We're given heads of two sorted linked lists. Merge them into one sorted
# list by splicing the original nodes — no new nodes. Return the new head. Right?"
#
# "list1=[1,2,4], list2=[1,3,4]:
# Compare 1 vs 1 → take list1's 1. 2 vs 1 → take list2's 1. 2 vs 3 → take 2.
# 4 vs 3 → take 3. 4 vs 4 → take list1's 4. Append list2's 4. → [1,1,2,3,4,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values, sort, rebuild a new list.
# Time: O((m+n) log(m+n)). Space: O(m+n) for the values array."

# PHASE 3 — OPTIMIZE
# "Both lists are already sorted — no sorting needed. Use two pointers.
# A dummy head simplifies edge cases (empty lists, different lengths).
# At each step attach the smaller node; advance that pointer.
# When one list runs out, attach the rest of the other.
#
# Time: O(m+n). Space: O(1) — only pointer manipulation."

# PHASE 4 — CLEAN CODE
```

```
# PHASE 5 — TEST & DEBUG
# list1=[1,2,4], list2=[1,3,4]:
# 1<=1 → take l1(1), l1→2. 2>1 → take l2(1), l2→3. 2<=3 → take l1(2), l1→4.
# 4>3 → take l2(3), l2→4. 4<=4 → take l1(4), l1=None.
# Attach l2 remainder [4]. → [1,1,2,3,4,4] ✓
#
# list1=[], list2=[0]:
# Loop skipped. cur.next = list2 → [0] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m+n): visit every node once. Space O(1): reuses existing nodes.
# Recursive solution is O(m+n) time but O(m+n) stack space.
# Follow-up: merge k sorted lists (LC 23) — use a min-heap for O(n log k).
# Follow-up: merge k sorted arrays — same heap approach."
```


Linked List

#141 Linked List Cycle EAS

Open on LeetCode

Hash Table · Linked List · Two Pointers

Lists: Grind 169

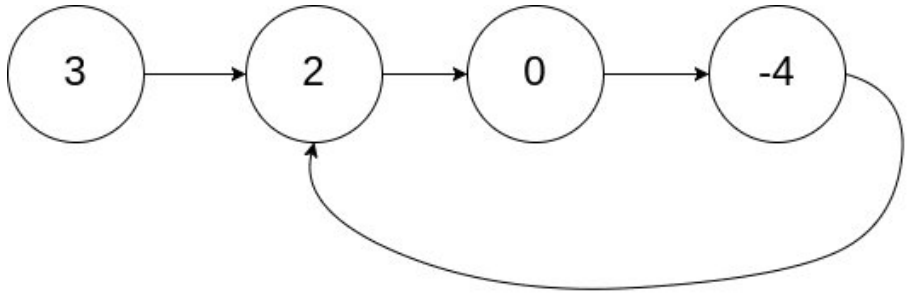
Problem

Given head, the head of a linked list, determine if the linked list has a cycle in it.

There is a cycle in a linked list if there is some node in the list that can be reached again by continuously following the next pointer. Internally, pos is used to denote the index of the node that tail's next pointer is connected to. Note that pos is not passed as a parameter.

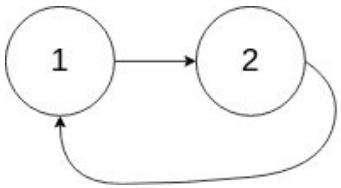
Return true if there is a cycle in the linked list. Otherwise, return false.

Example 1:




```
Input: head = [3,2,0,-4], pos = 1
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 1st node (0-indexed).
```

Example 2:




```
Input: head = [1,2], pos = 0
Output: true
Explanation: There is a cycle in the linked list, where the tail connects to the 0th node.
```

Example 3:


```
Input: head = [1], pos = -1
Output: false
Explanation: There is no cycle in the linked list.
```

Constraints:

- The number of the nodes in the list is in the range [0, 104].
- -105 <= Node.val <= 105
- pos is -1 or a valid index in the linked-list.

Follow up: Can you solve it using O(1) (i.e. constant) memory?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given the head of a linked list, return true if there's a cycle —
# if any node's next eventually points back to a previous node. Right?"
```

```
#
# "head=[3,2,0,-4], pos=1 (tail connects to index 1):
# 3→2→0→-4→2→… cycle exists → true ✓
# head=[1,2], pos=-1: 1→2→None → false ✓"

# PHASE 2 — BRUTE FORCE
# "Store every visited node in a set. If we see a node twice, there's a cycle.
# If we reach None, there isn't.
# Time: O(n). Space: O(n) for the visited set."

# PHASE 3 — OPTIMIZE
# "Floyd's tortoise-and-hare: two pointers, slow moves 1 step, fast moves 2.
# If there's a cycle, fast will eventually lap slow and they'll meet.
# If fast reaches None, there's no cycle.
#
# Key insight: in a cycle, fast gains 1 step per iteration → guaranteed to catch slow.
# Time: O(n). Space: O(1) — no extra storage."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [3,2,0,-4] cycle at index 1:
# slow=3,fast=3 → slow=2,fast=0 → slow=0,fast=2 → slow=-4,fast=-4 → equal → True ✓
#
# [1,2] no cycle:
# slow=1,fast=1 → slow=2,fast=None → fast is None → loop exits → False ✓
#
# [1] no cycle:
# fast.next is None → loop never enters → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): fast pointer covers the cycle in at most n steps after reaching it.
# Space O(1): only two pointers.
# Follow-up: find the START of the cycle (LC 142) — after they meet, reset slow
# to head; advance both one step at a time until they meet again → cycle entry.
# Follow-up: find cycle length — once they meet, keep fast moving and count steps."
```


Linked List

#206 Reverse Linked List

EAS

Open on LeetCode

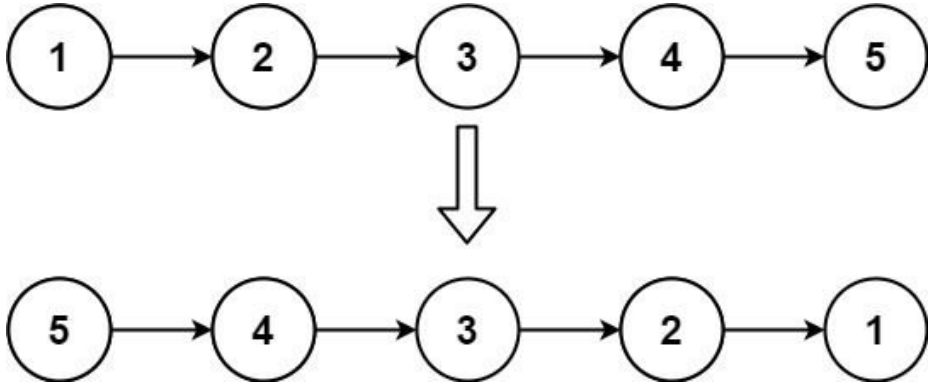
Linked List · Recursion

Lists: Grind 169

Problem

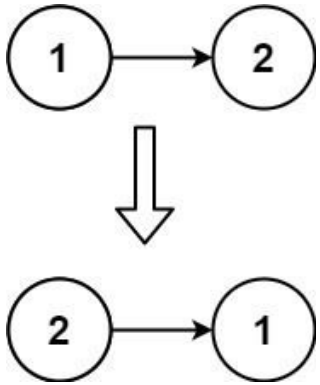
Given the head of a singly linked list, reverse the list, and return the reversed list.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [5,4,3,2,1]
```

Example 2:




```
Input: head = [1,2]
Output: [2,1]
```

Example 3:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in the list is the range [0, 5000].
- -5000 <= Node.val <= 5000

Follow up: A linked list can be reversed either iteratively or recursively. Could you implement both?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given the head of a singly linked list, reverse it in-place and return
```

```
# the new head. Right?"
#
# "head=[1,2,3,4,5]:
# 1→2→3→4→5→None becomes None←1←2←3←4←5, new head = 5 ✓
# head=[]: return None ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values in an array, rebuild a new reversed list.
# Time: O(n). Space: O(n) for the values."

# PHASE 3 — OPTIMIZE
# "Three-pointer iterative approach: prev, curr, next_node.
# At each step: save curr.next, point curr.next to prev, advance both pointers.
# When curr is None, prev is the new head.
#
# Time: O(n). Space: O(1) — in-place pointer rewiring."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# head=[1,2,3]:
# prev=None,curr=1: nxt=2, 1.next=None, prev=1, curr=2
# prev=1,curr=2: nxt=3, 2.next=1, prev=2, curr=3
# prev=2,curr=3: nxt=None, 3.next=2, prev=3, curr=None
# return 3 → [3,2,1] ✓
#
# head=[]: curr=None immediately → return None ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): constant extra pointers.
# Recursive solution: O(n) time, O(n) stack space — elegant but risky for large inputs.
# Follow-up: reverse nodes in k-group (LC 25) — applies this same logic per group.
# Follow-up: reverse between positions left and right (LC 92) — isolate the sublist first."
```


Linked List

#234 Palindrome Linked List

EAS

Open on LeetCode

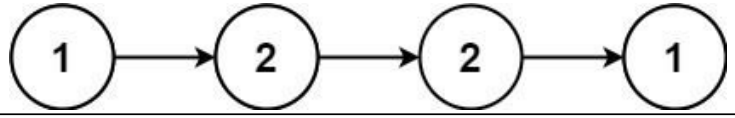
Linked List · Two Pointers · Recursion

Lists: Grind 169

Problem

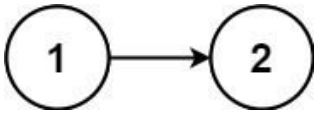
Given the head of a singly linked list, return true if it is a palindrome or false otherwise.

Example 1:




```
Input: head = [1,2,2,1]
Output: true
```

Example 2:




```
Input: head = [1,2]
Output: false
```

Constraints:

- **The number of nodes in the list is in the range [1, 105].**
- **0 <= Node.val <= 9**

Follow up: Could you do it in O(n) time and O(1) space?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given the head of a singly linked list, return true if it reads the same
# forwards and backwards. Right?"
#
# "head=[1,2,2,1]: reads same both ways → true ✓
# head=[1,2]: 1≠2 → false ✓"

# PHASE 2 — BRUTE FORCE
# "Copy all values into an array, then use two-pointer comparison.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Achieve O(1) space by modifying the list in-place:
# 1. Find the midpoint using slow/fast pointers.
# 2. Reverse the second half in-place.
# 3. Compare first half with reversed second half.
# 4. (Optional) restore the list.
#
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # Step 1: find middle
        # Step 2: reverse second half
        # Step 3: compare halves

# PHASE 5 — TEST & DEBUG
# [1,2,2,1]:
# Mid: slow=2(idx1), fast=1(idx3). Reverse [2,1]→[1,2].
# Compare: 1==1 ✓, 2==2 ✓, right=None → True ✓
#
# [1,2]:
# Mid: slow=2, fast=None after 1 step. Reverse [2]→[2].
# Compare: 1≠2 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): three linear passes (find mid, reverse, compare).
# Space O(1): constant extra pointers.
# Trade-off: O(n) space approach is simpler and non-destructive.
# Follow-up: restore the original list after the check by re-reversing second half.
# Follow-up: palindrome check on a doubly linked list is trivial — two outer pointers."
```

Linked List

#706 Design HashMap

EAS

Open on LeetCode

Array · Hash Table · Linked List · Design · Hash Function

Lists: Denny Zhang

Problem

Design a HashMap without using any built-in hash table libraries.

Implement the MyHashMap class:

- MyHashMap() initializes the object with an empty map.
- void put(int key, int value) inserts a (key, value) pair into the HashMap. If the key already exists in the map, update the corresponding value.
- int get(int key) returns the value to which the specified key is mapped, or -1 if this map contains no mapping for the key.
- void remove(key) removes the key and its corresponding value if the map contains the mapping for the key.

Example 1:

```
Input
["MyHashMap", "put", "put", "get", "get", "put", "get", "remove", "get"]
[[], [1, 1], [2, 2], [1], [3], [2, 1], [2], [2], [2]]
Output
[null, null, null, 1, -1, null, 1, null, -1]

Explanation
MyHashMap myHashMap = new MyHashMap();
```

Constraints:

- 0 <= key, value <= 106
- At most 104 calls will be made to put, get, and remove.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement a HashMap without built-in hash libraries. Support put(key, val),
# get(key) → -1 if missing, remove(key). Keys and values are non-negative ints
# in range [0, 10^6]. Right?"
#
# "put(1,1), put(2,2), get(1)→1, get(3)→-1, put(2,1), get(2)→1, remove(2), get(2)→-1 ✓"

# PHASE 2 — BRUTE FORCE
# "Store all pairs in a plain list. Scan linearly for get/put/remove.
# O(n) per operation — correct but slow."

# PHASE 3 — OPTIMIZE
# "Use an array of buckets with separate chaining (linked list / small list per bucket).
# Hash function: key % num_buckets using a prime to spread keys.
# Average O(1) per op when load factor is low. Space O(n + k)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# put(1,1): bucket[1]=[(1,1)]
# put(2,2): bucket[2]=[(2,2)]
# get(1): bucket[1] → (1,1) → 1 ✓
# get(3): bucket[3]=[] → -1 ✓
# put(2,1): bucket[2] → update → [(2,1)]
# get(2): → 1 ✓
# remove(2): bucket[2]=[]
# get(2): [] → -1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute: O(n) per op. Hash bucket: O(1) average, O(n) worst (all same bucket).
# Space O(n+k): n stored pairs + k bucket slots.
# Follow-up: Design HashSet (LC 705) — same idea, keys only.
# Follow-up: open addressing (linear probing) avoids linked lists but needs tombstones."
```


Linked List

#876 Middle of the Linked List

EAS

Open on LeetCode

Linked List · Two Pointers

Lists: Grind 169

Problem

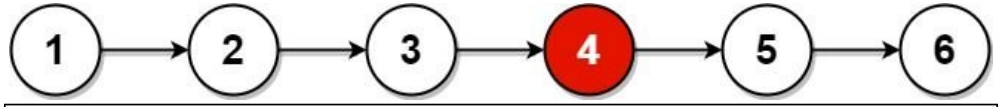
Given the head of a singly linked list, return the middle node of the linked list.

If there are two middle nodes, return the second middle node.

Example 1:


```
Input: head = [1,2,3,4,5]
Output: [3,4,5]
Explanation: The middle node of the list is node 3.
```

Example 2:




```
Input: head = [1,2,3,4,5,6]
Output: [4,5,6]
Explanation: Since the list has two middle nodes with values 3 and 4, we return the second one.
```

Constraints:

- The number of nodes in the list is in the range [1, 100].
- 1 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given the head of a linked list, return the middle node.
# If two middles exist (even length), return the SECOND one. Right?"
#
# "head=[1,2,3,4,5]: length=5, middle index=2 → node(3) ✓
# head=[1,2,3,4,5,6]: length=6, two middles at index 2,3 → return node(4) ✓"

# PHASE 2 — BRUTE FORCE
# "Count the total number of nodes, then walk to index length//2.
# Two passes: one to count, one to reach the middle.
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Slow/fast pointer — one pass.
# Slow moves 1 step, fast moves 2. When fast reaches the end, slow is at the middle.
# For even-length lists, slow naturally lands on the SECOND middle (desired).
#
# Time: O(n). Space: O(1). Saves one full traversal vs brute."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5]:
# s=1,f=1 → s=2,f=3 → s=3,f=5 → fast.next=None → stop. return node(3) ✓
#
# [1,2,3,4,5,6]:
# s=1,f=1 → s=2,f=3 → s=3,f=5 → s=4,f=None(6.next) → stop. return node(4) ✓
#
# [1]:
# fast.next=None immediately → return head=node(1) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): fast pointer covers the list in n/2 iterations.
# Space O(1): two pointers only.
# Follow-up: if you need the FIRST middle for even-length, stop when fast.next.next is None.
# Follow-up: this slow/fast trick is the building block for palindrome check (LC 234)
# and cycle detection (LC 141)."
```

Linked List

⋆ #1474 Delete N Nodes After M Nodes of Linked List ⋆

EAS

Open on LeetCode

Linked List

Lists: ⋆ Premium | Premium 98

Problem

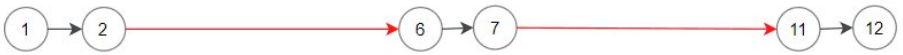
You are given the head of a linked list and two integers m and n.

Traverse the linked list and remove some nodes in the following way:

- Start with the head as the current node.
- Keep the first m nodes starting with the current node.
- Remove the next n nodes
- Keep repeating steps 2 and 3 until you reach the end of the list.

Return the head of the modified list after removing the mentioned nodes.

Example 1:




```
Input: head = [1,2,3,4,5,6,7,8,9,10,11,12,13], m = 2, n = 3
Output: [1,2,6,7,11,12]
Explanation: Keep the first (m = 2) nodes starting from the head of the linked List (1 ->2) show in black nodes.
Delete the next (n = 3) nodes (3 -> 4 -> 5) show in read nodes.
Continue with the same procedure until reaching the tail of the Linked List.
Head of the linked list after removing nodes is returned.
```

Example 2:


```
Input: head = [1,2,3,4,5,6,7,8,9,10,11], m = 1, n = 3
Output: [1,5,9]
Explanation: Head of linked list after removing nodes is returned.
```

Constraints:

- The number of nodes in the list is in the range [1, 104].
- 1 <= Node.val <= 106
- 1 <= m, n <= 1000

Follow up: Could you solve this problem by modifying the list in-place?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a linked list and integers m and n: keep m nodes, delete next n nodes,
# repeat until the end. Return the modified head. Right?"
#
# "head=[1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3:
# Keep 1,2 → delete 3,4,5 → keep 6,7 → delete 8,9,10 → keep 11,12 → delete 13
# → [1,2,6,7,11,12] ✓"

# PHASE 2 — BRUTE FORCE
# "Enumerate all nodes with their index. Keep node i if (i % (m+n)) < m.
# Collect kept nodes and relink them.
# Time: O(n_total). Space: O(n_total) for the kept-nodes list."

# PHASE 3 — OPTIMIZE
# "Pointer-based in-place: walk m steps forward (keeping nodes), then
# walk n steps forward (skipping/deleting nodes), then repeat.
# Relink the tail of the kept section directly to the node after the deleted section.
# Time: O(n_total). Space: O(1) — no extra storage."
```



```
# PHASE 4 — CLEAN CODE
            # Walk m-1 steps to reach end of kept section
            # Walk n steps to skip deleted section

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5,6,7,8,9,10,11,12,13], m=2, n=3:
# curr=1, walk 1 step → curr=2 (tail=2). skip=3, walk 3 → skip=6. 2.next=6, curr=6.
# curr=6, walk 1 step → curr=7 (tail=7). skip=8, walk 3 → skip=11. 7.next=11, curr=11.
# curr=11, walk 1 step → curr=12 (tail=12). skip=13, walk 3 → skip=None. 12.next=None.
# → [1,2,6,7,11,12] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(total nodes): each node visited exactly once.
# Space O(1): only pointer variables.
# Follow-up: what if m and n can vary per cycle? Would need to pass arrays.
# Follow-up: similar pattern to 'Remove Duplicates from Sorted List II' in pointer reuse."
```

Stack

Round 4 · Mid · Easy · 3 questions

Stack

#20 Valid Parentheses

EAS

Open on LeetCode

String · Stack

Lists: Grind 169

Problem

Given a string s containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

An input string is valid if:

1. Open brackets must be closed by the same type of brackets.
2. Open brackets must be closed in the correct order.
3. Every close bracket has a corresponding open bracket of the same type.

Example 1:
Input: s = "()"
Output: true
Example 2:
Input: s = "()[]{}"
Output: true
Example 3:
Input: s = "(]"
Output: false
Example 4:
Input: s = "([])"
Output: true
Example 5:
Input: s = "([)]"
Output: false
Constraints:

- 1 <= s.length <= 104
- s consists of parentheses only '()[]{}'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string of brackets, return true if every open bracket is closed
# by the same type in the correct order. Right?"
#
# "s="()[]{}": each pair matches in order → true ✓
# s="(]": '(' expects ')' but gets ']' → false ✓
# s="([)]": '(' then '[', but ')' closes '(' while '[' is still open → false ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly replace matching pairs '()', '[]', '{}' until nothing changes.
# If the string is empty at the end it's valid.
# Time: O(n²) — up to n/2 passes each scanning n chars. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Use a stack. Push every opening bracket.
# On a closing bracket, the top of the stack must be its matching opener.
# If not — or the stack is empty — return false.
# At the end, the stack must be empty.
#
# Time: O(n) — one pass. Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s="()[]{}":
# '(' push. ')' → top='(' matches → pop. '[' push. ']' → pop. '{' push. '}' → pop.
# stack empty → True ✓
#
# s="(]":
# '(' push. ']' → top='(' ≠ '[' → return False ✓
#
# s="([)]":
# '(' push. '[' push. ')' → top='[' ≠ '(' → return False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass through the string.
# Space O(n): worst case all openers e.g. '(((((('.
# Follow-up: minimum removals to make valid (LC 1249) extends this with counters.
# Follow-up: what if we also have wildcards '*' that can be '(', ')' or ''? (LC 678)."
```

Stack

#232 Implement Queue using Stacks

EAS

Open on LeetCode

Stack · Design · Queue

Lists: Grind 169

Problem

Implement a first in first out (FIFO) queue using only two stacks. The implemented queue should support all the functions of a normal queue (push, peek, pop, and empty).

Implement the MyQueue class:

- void push(int x) Pushes element x to the back of the queue.
- int pop() Removes the element from the front of the queue and returns it.
- int peek() Returns the element at the front of the queue.
- boolean empty() Returns true if the queue is empty, false otherwise.

Notes:

- You must use only standard operations of a stack, which means only push to top, peek/pop from top, size, and is empty operations are valid.
- Depending on your language, the stack may not be supported natively. You may simulate a stack using a list or deque (double-ended queue) as long as you use only a stack's standard operations.

Example 1:

```
Input
["MyQueue", "push", "push", "peek", "pop", "empty"]
[[], [1], [2], [], [], []]
Output
[null, null, null, 1, 1, false]

Explanation
MyQueue myQueue = new MyQueue();
```

Constraints:

- 1 <= x <= 9
- At most 100 calls will be made to push, pop, peek, and empty.
- All the calls to pop and peek are valid.

Follow-up: Can you implement the queue such that each operation is amortized O(1) time complexity? In other words, performing n operations will take overall O(n) time even if one of those operations may take longer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We need a queue (FIFO) built only from stacks (LIFO). Support push(x), pop(),
# peek(), and empty(). Amortized O(1) per operation is acceptable. Right?"
#
# "push(1), push(2), peek() → 1, pop() → 1, empty() → false
# The first element pushed must be first out — opposite of a stack."

# PHASE 2 — BRUTE FORCE
# "Use one stack. On every push, move ALL elements out, push the new one to bottom,
# then put everything back. O(n) per push, O(1) pop/peek — queue order maintained."
        # Move everything out, add x at bottom, put back

# PHASE 3 — OPTIMIZE
# "Two stacks — inbox and outbox — achieve amortized O(1).
# Push always goes to inbox.
# For pop/peek: if outbox is empty, pour ALL of inbox into outbox (reverses order).
# Then pop/peek from outbox.
# Each element moves at most twice total across both stacks."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# push(1): inbox=[1]
# push(2): inbox=[1,2]
# peek(): outbox empty → pour → inbox=[], outbox=[2,1]. outbox[-1]=1 ✓
# pop(): outbox=[2,1] → pop → 1, outbox=[2] ✓
# empty(): inbox=[], outbox=[2] → False ✓
# pop(): outbox=[2] → pop → 2 ✓
# empty(): both empty → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute: O(n) push, O(1) pop. Two-stack: amortized O(1) all ops.
# Each element transferred at most once — total work O(n) for n ops.
# Space O(n) across both stacks.
# Follow-up: implement stack using queues (LC 225) — symmetric design.
# Follow-up: if we needed worst-case O(1) all ops, we'd need a deque."
```


Stack

#844 Backspace String Compare

EAS

Open on LeetCode

Two Pointers · String · Stack · Simulation

Lists: Grind 169

Problem

Given two strings s and t, return true if they are equal when both are typed into empty text editors. '#' means a backspace character.

Note that after backspacing an empty text, the text will continue empty.

Example 1:

```
Input: s = "ab#c", t = "ad#c"
Output: true
Explanation: Both s and t become "ac".
```

Example 2:

```
Input: s = "ab##", t = "c#d#"
Output: true
Explanation: Both s and t become "".
```

Example 3:

```
Input: s = "a#c", t = "b"
Output: false
Explanation: s becomes "c" while t becomes "b".
```

Constraints:

- 1 <= s.length, t.length <= 200
- s and t only contain lowercase letters and '#' characters.

Follow up: Can you solve it in O(n) time and O(1) space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given two strings where '#' represents a backspace, return true if they're
# equal after processing. Right?"
#
# "s="ab#c", t="ad#c":
# s: a,b,# → a, c → "ac". t: a,d,# → a, c → "ac". Equal → true ✓
# s="a#c", t="b": s→"c", t→"b". Not equal → false ✓"

# PHASE 2 — BRUTE FORCE
# "Simulate using a stack: push normal chars, pop on '#' if stack non-empty.
# Compare final stacks.
# Time: O(n+m). Space: O(n+m) for the stacks."

# PHASE 3 — OPTIMIZE
# "Two-pointer approach from the RIGHT — O(1) space.
# Traverse both strings backwards, counting '#' as skip tokens.
# Skip that many non-'#' characters before comparing.
# If both pointers exhausted simultaneously → equal.
#
# Time: O(n+m). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s="ab#c", t="ad#c":
# i=3'c',j=3'c' → both non-# → c==c ✓. i=2,j=2.
# i=2'#'→skip_s=1,i=1. s[1]='b',skip_s→0,i=0.
# j=2'#'→skip_t=1,j=1. t[1]='d',skip_t→0,j=0.
# s[0]='a', t[0]='a' → equal ✓. i=-1,j=-1 → return True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Stack approach: O(n+m) time, O(n+m) space — simple and readable.
# Two-pointer approach: O(n+m) time, O(1) space — optimal.
# Follow-up: if '#' can backspace multiple characters (like '2#'), adjust skip counter.
# Follow-up: streaming input — the stack approach handles it naturally."
```

Trie

Round 4 · Mid · Easy · 1 question

Trie

#14 Longest Common Prefix EAS

Open on LeetCode

String · Trie

Lists: Grind 169

Problem

Write a function to find the longest common prefix string amongst an array of strings.
If there is no common prefix, return an empty string "".

Example 1:

```
Input: strs = ["flower","flow","flight"]
Output: "fl"
```

Example 2:

```
Input: strs = ["dog","racecar","car"]
Output: ""
Explanation: There is no common prefix among the input strings.
```

Constraints:

- 1 <= strs.length <= 200
- 0 <= strs[i].length <= 200
- strs[i] consists of only lowercase English letters if it is non-empty.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "We're given an array of strings and need to find the longest prefix
# shared by ALL of them. If none exists, return an empty string. Right?"
#
# "strs=["flower","flow","flight"]:
# Compare column by column: f✓ l✓ o vs o vs i → mismatch at index 2 → return "fl" ✓
# strs=["dog","racecar","car"]: first chars d vs r → mismatch → return "" ✓"

# PHASE 2 — BRUTE FORCE
# "Sort the array, then only compare the first and last strings —
# the LCP of those two is the LCP of the whole array.
# Time: O(n log n + m) where m is shortest string length. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Vertical scanning avoids the sort entirely.
# Take the first string as the reference. For each character position i,
# check that every other string has the same character at position i.
# Stop as soon as any string is too short or has a mismatch.
#
# Pseudocode:
# for i, char in enumerate(strs[0]):
# for s in strs[1:]:
# if i >= len(s) or s[i] != char: return strs[0][:i]
# return strs[0]
#
# Time: O(n*m) — n strings, m = min length. Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# strs=["flower","flow","flight"]:
# i=0 'f': all match. i=1 'l': all match. i=2 'o': strs[2][2]='i' ≠ 'o' → return "fl" ✓
#
# strs=["dog","racecar","car"]:
# i=0 'd': strs[1][0]='r' ≠ 'd' → return "" ✓
#
# strs=["abc"]:
# No inner loop fires → return "abc" ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*m): every char of shortest string compared across all n strings.
# Space O(1): no extra allocation beyond the output slice.
# Follow-up: if the array is huge but strings are short, binary search on
# prefix length works — O(m log m * n) but better when m << n.
# Follow-up: Trie would let you insert all strings and walk until a branch point."
```

Greedy

Round 4 · Mid · Easy · 1 question

Greedy

#409 Longest Palindrome EAS

Open on LeetCode

Hash Table · String · Greedy

Lists: Grind 169

Problem

Given a string s which consists of lowercase or uppercase letters, return the length of the longest palindrome that can be built with those letters.

Letters are case sensitive, for example, "Aa" is not considered a palindrome.

Example 1:

```
Input: s = "abccccdd"
Output: 7
Explanation: One longest palindrome that can be built is "dccaccd", whose length is 7.
```

Example 2:

```
Input: s = "a"
Output: 1
Explanation: The longest palindrome that can be built is "a", whose length is 1.
```

Constraints:

- **1 <= s.length <= 2000**
- **s consists of lowercase and/or uppercase English letters only.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given a string of letters, find the length of the longest palindrome
# we can BUILD from those characters (not a substring — we rearrange them). Right?"
#
# "s="abccccdd":
# d×2, c×4, a×1, b×1. Pairs: dd, cccc → length 6. One leftover can go in center → 7 ✓
# s="a": just 'a' → length 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Count frequency of each character. A palindrome uses pairs symmetrically
# plus at most one center character.
# Sum all (freq // 2) * 2. If any character had an odd count, add 1 for center."

# PHASE 3 — OPTIMIZE
# "Same greedy logic, can use a set instead of a counter:
# Toggle each character in a set. At the end, characters still in the set
# appeared an odd number of times. Answer = len(s) - len(odd_chars) + (1 if odd_chars else 0).
#
# Time: O(n). Space: O(1) — at most 52 distinct letters."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s="abcccdd":
# a→{a}, b→{a,b}, c→{a,b,c}, c→{a,b}, c→{a,b,c}, c→{a,b}, d→{a,b,d}, d→{a,b}
# odd_chars={a,b}, len(s)=8, 8-2+1=7 ✓
#
# s="a":
# odd_chars={a}, 1-1+1=1 ✓
#
# s="aa":
# odd_chars={}, 2-0+0=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): at most 52 entries in the set.
# The greedy insight: use every pair, reserve at most one odd character for center.
# Follow-up: if you need to OUTPUT the actual palindrome, assign pairs to both ends
# and place one odd character in the middle."
```

Round 5 · Mid · Medium

Round 5

Mid Priority · Medium
56 questions · 6 patterns

Linked List #2 #19 #24 #61 #146 #148 #328 #369 #430 #622 #707 #708 #1171
Stack #143 #150 #155 #173 #227 #255 #394 #439 #484 #735 #739 #962 #1214 #1762
Heap #215 #253 #347 #505 #621 #658 #787 #973 #1057 #1167 #1424
Trie #139 #208 #211 #616 #692 #720 #1166
Backtracking #17 #22 #39 #46 #79 #113
Greedy #134 #179 #280 #954 #2131

Linked List

Round 5 · Mid · Medium · 13 questions

Linked List

#2 Add Two Numbers

MED

Open on LeetCode

Linked List · Math · Recursion

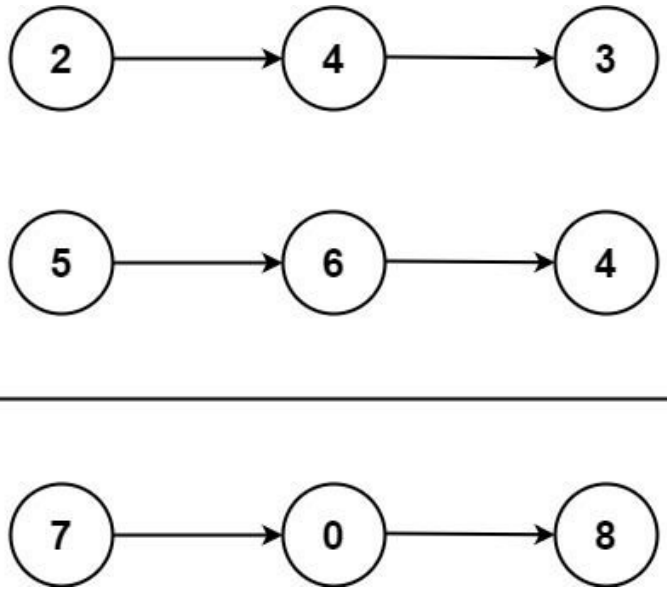
Lists: Grind 169

Problem

You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

You may assume the two numbers do not contain any leading zero, except the number 0 itself.

Example 1:




```
Input: l1 = [2,4,3], l2 = [5,6,4]
Output: [7,0,8]
Explanation: 342 + 465 = 807.
```

Example 2:

```
Input: l1 = [0], l2 = [0]
Output: [0]
```

Example 3:

```
Input: l1 = [9,9,9,9,9,9,9], l2 = [9,9,9,9]
Output: [8,9,9,9,0,0,0,1]
```

Constraints:

- The number of nodes in each linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- It is guaranteed that the list represents a number that does not have leading zeros.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Two non-empty linked lists store digits of non-negative integers in REVERSE order.
# Add the two numbers and return the sum as a reversed linked list. Right?"
#
# "l1=[2,4,3] (342), l2=[5,6,4] (465): 342+465=807 → [7,0,8] ✓
# l1=[0], l2=[0]: 0+0=0 → [0] ✓
# l1=[9,9,9,9], l2=[9,9,9]: 9999+999=10998 → [8,9,9,0,1] ✓ (carry propagation)"

# PHASE 2 — BRUTE FORCE
# "Convert both lists to integers, add, then convert the result back to a linked list.
```



```
# Careful: the digits are stored in reverse, so the head is the least significant digit.
# Time: O(max(m,n)). Space: O(max(m,n)) for the result."

# PHASE 3 — OPTIMIZE
# "Simulate grade-school addition digit by digit with a carry.
# Walk both lists simultaneously; at each position sum the digits + carry.
# new_digit = total % 10, new_carry = total // 10.
# Continue until both lists exhausted AND carry is 0.
# Time: O(max(m,n)). Space: O(max(m,n)+1) for the result — avoids big-integer conversion."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# l1=[2,4,3], l2=[5,6,4]:
# 2+5+0=7, carry=0 → node(7). 4+6+0=10, carry=1 → node(0). 3+4+1=8, carry=0 → node(8).
# Both None, carry=0 → stop. → [7,0,8] ✓
#
# l1=[9,9,9,9], l2=[9,9,9]:
# 9+9=18→carry=1,node(8). 9+9+1=19→carry=1,node(9). 9+9+1=19→carry=1,node(9).
# 9+0+1=10→carry=1,node(0). 0+0+1=1→carry=0,node(1). → [8,9,9,0,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(max(m,n)): process as many digits as the longer number plus at most one carry.
# Space O(max(m,n)+1): result list.
# Follow-up: Add Two Numbers II (LC 445) — digits stored in FORWARD order.
# Use two stacks to reverse, then apply same carry logic.
# Follow-up: multiply two numbers represented as strings (LC 43) — similar positional logic."
```

Linked List

#19 Remove Nth Node From End of List

MED

Open on LeetCode

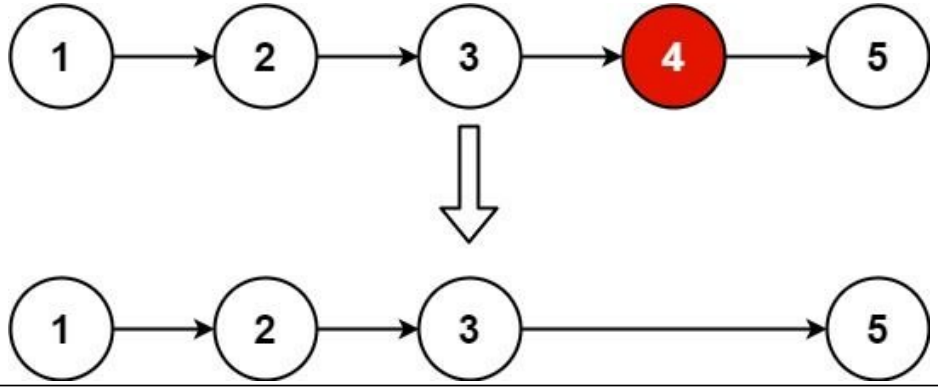
Linked List · Two Pointers

Lists: Grind 169

Problem

Given the head of a linked list, remove the nth node from the end of the list and return its head.

Example 1:




```
Input: head = [1,2,3,4,5], n = 2
Output: [1,2,3,5]
```

Example 2:

```
Input: head = [1], n = 1
Output: []
```

Example 3:

```
Input: head = [1,2], n = 1
Output: [1]
```

Constraints:

- The number of nodes in the list is sz.
- 1 <= sz <= 30
- 0 <= Node.val <= 100
- 1 <= n <= sz

Follow up: Could you do this in one pass?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given the head of a linked list and integer n, remove the nth node from the END
# and return the modified head. n is always valid. Right?"
#
# "head=[1,2,3,4,5], n=2: 5th-from-end=5, 4th=4, 3rd=3, 2nd=4 → remove node(4) → [1,2,3,5] ✓
# head=[1], n=1: remove the only node → [] ✓
# head=[1,2], n=1: remove last → [1] ✓"

# PHASE 2 — BRUTE FORCE
# "Two passes: first count the list length L, then remove node at position L-n.
# Use a dummy node to handle removing the head cleanly.
# Time: O(L). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "One pass using two pointers spaced n apart.
# Advance fast n steps first, then move both slow and fast together.
# When fast reaches the end, slow is just before the node to remove.
# Dummy node handles the edge case of removing the head.
# Time: O(L). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # Advance fast n+1 steps so slow lands before the target

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5], n=2:
```



```
# fast advances 3 steps: dummy→1→2→3. slow=dummy.
# Move together: slow=1,fast=4 → slow=2,fast=5 → slow=3,fast=None → stop.
# slow=node(3). slow.next=node(4). slow.next=node(5). → [1,2,3,5] ✓
#
# [1], n=1:
# fast advances 2 steps → None. while loop skipped. slow=dummy.
# slow.next=node(1), slow.next=node(1).next=None → [] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(L): one pass for the two-pointer approach vs two passes for brute.
# Space O(1): constant extra pointers.
# The dummy node is the key to cleanly handling head removal without special-casing.
# Follow-up: if n could be invalid (> list length), add a bounds check.
# Follow-up: find the nth from end without removing (LC 876 variant) — same slow/fast idea."
```

Linked List

#24 Swap Nodes in Pairs

MED

Open on LeetCode

Linked List · Recursion

Lists: Grind 169

Problem

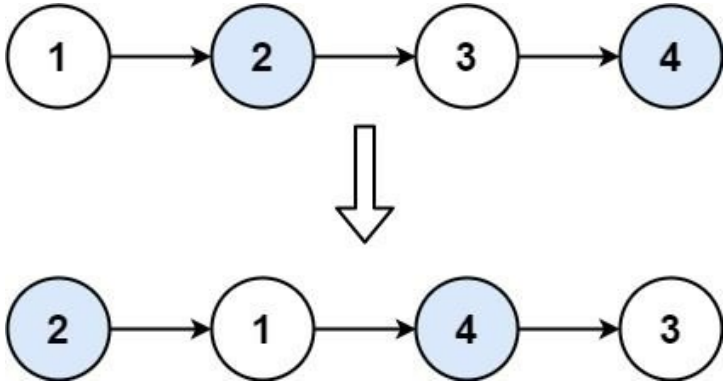
Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

Example 1:

Input: head = [1,2,3,4]

Output: [2,1,4,3]

Explanation:



Example 2:

Input: head = []

Output: []

Example 3:

Input: head = [1]

Output: [1]

Example 4:

Input: head = [1,2,3]

Output: [2,1,3]

Constraints:

- The number of nodes in the list is in the range [0, 100].
- 0 <= Node.val <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a linked list, swap every two adjacent nodes and return the head.
# Must swap the nodes themselves, not just their values. Right?"
#
# "head=[1,2,3,4]: swap (1,2) and (3,4) → [2,1,4,3] ✓
# head=[1]: odd length, node 1 stays → [1] ✓
# head=[]: → [] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all node values, swap pairs in the array, rebuild a new list.
# Time: O(n). Space: O(n) for the values array."
        # Swap pairs in-place

# PHASE 3 — OPTIMIZE
# "In-place pointer rewiring with a dummy predecessor node.
# For each pair (first, second): set prev.next=second, second.next=first,
# first.next=next_pair. Advance prev to first (now the second in the list).
# Time: O(n). Space: O(1) — no extra storage."

# PHASE 4 — CLEAN CODE
            # Rewire
            # Advance
```



```
# PHASE 5 — TEST & DEBUG
# [1,2,3,4]: dummy→1→2→3→4
# prev=dummy, first=1, second=2:
# dummy.next=2, 1.next=3, 2.next=1 → dummy→2→1→3→4. prev=1.
# prev=1, first=3, second=4:
# 1.next=4, 3.next=None, 4.next=3 → dummy→2→1→4→3. prev=3.
# prev.next=None → stop. → [2,1,4,3] ✓
#
# [1]: prev.next.next=None → loop never enters. → [1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): visit each node once. Space O(1): constant extra pointers.
# The dummy node is essential to handle swapping the first pair cleanly.
# Follow-up: Reverse Nodes in k-Group (LC 25) — generalise to groups of k.
# Follow-up: recursive solution is elegant but uses O(n/2) stack space."
```

Linked List

#61 Rotate List

MED

Open on LeetCode

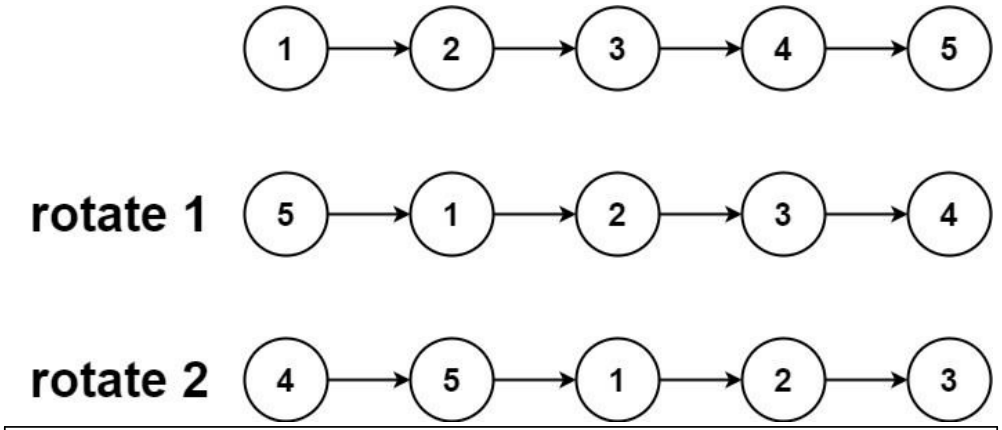
Linked List · Two Pointers

Lists: Grind 169

Problem

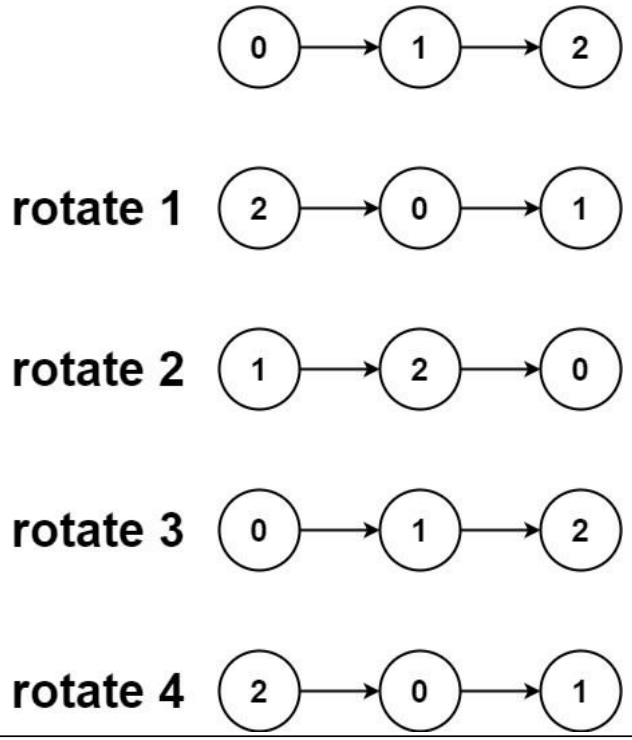
Given the head of a linked list, rotate the list to the right by k places.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [4,5,1,2,3]
```

Example 2:




```
Input: head = [0,1,2], k = 4
Output: [2,0,1]
```

Constraints:

- The number of nodes in the list is in the range [0, 500].
- -100 <= Node.val <= 100
- 0 <= k <= 2 * 10^9

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rotate the linked list to the right by k places. Each rotation moves the last node
# to the front. Return the new head. Right?"
#
# "head=[1,2,3,4,5], k=2:
# rotate 1: [5,1,2,3,4]. rotate 2: [4,5,1,2,3] ✓
# head=[0,1,2], k=4: effective k = 4%3 = 1 → [2,0,1] ✓"

# PHASE 2 — BRUTE FORCE
# "Perform k individual rotations. Each rotation: traverse to second-to-last node,
# take the last node, prepend it to the front.
# Time: O(n*k) — can be very slow if k is large. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Three steps, one pass to find length:
# 1. Find length L and tail node by traversing once.
# 2. Effective rotation = k % L (k >= L means full cycles, no change).
# 3. New tail is at position L - (k%L) - 1. New head is new_tail.next.
# Make the list circular, then cut at the right spot.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # Step 1: find length and tail
        # Step 2: effective rotation
        # Step 3: find new tail at position (length - k - 1)

# PHASE 5 — TEST & DEBUG
```

```
# [1,2,3,4,5], k=2:
# length=5, tail=node(5). k=2%5=2. new_tail at index 5-2-1=2 → node(3).
# new_head=node(4). node(3).next=None. node(5).next=node(1).
# → [4,5,1,2,3] ✓
#
# [0,1,2], k=4:
# length=3, k=4%3=1. new_tail at index 3-1-1=1 → node(1).
# new_head=node(2). node(1).next=None. node(2_orig_tail).next=node(0).
# → [2,0,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass to find length + one pass to find cut point.
# Space O(1): only pointer variables.
# The k%length shortcut is critical — without it, k=10^9 on a 5-node list would TLE.
# Follow-up: rotate array (LC 189) — same modulo idea, different reversal technique.
# Follow-up: rotate left by k — equivalent to rotating right by (length - k%length)."
```


Linked List

#146 LRU Cache

ME
D

Open on LeetCode

Hash Table · Linked List · Design

Lists: Grind 169

Problem

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

- **LRUCache(int capacity) Initialize the LRU cache with positive size capacity.**
- **int get(int key) Return the value of the key if the key exists, otherwise return -1.**
- **void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.**

The functions get and put must each run in O(1) average time complexity.

Example 1:

```
Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
```

Constraints:

- **1 <= capacity <= 3000**
- **0 <= key <= 104**
- **0 <= value <= 105**
- **At most 2 * 105 calls will be made to get and put.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design an LRU Cache with a fixed capacity. Support get(key) → value or -1,
# and put(key, value) which evicts the least recently used item when over capacity.
# Both ops must be O(1) average. Right?"
#
# "LRUCache(2): put(1,1), put(2,2), get(1)→1, put(3,3) evicts key 2,
# get(2)→-1, put(4,4) evicts key 1, get(1)→-1, get(3)→3, get(4)→4 ✓"

# PHASE 2 — BRUTE FORCE
# "Use an ordered dict (Python's collections.OrderedDict) to track insertion order.
# On get/put, move the key to the end (most recent). On eviction, pop from front.
# Time: O(1) average. Space: O(capacity). — This is actually optimal but hides internals."

# PHASE 3 — OPTIMIZE
# "Build the internals explicitly: HashMap + Doubly Linked List.
# HashMap: key → node (O(1) lookup).
# DLL: maintains access order — head=LRU, tail=MRU.
# get: move node to tail. put: add at tail; if over capacity, remove from head.
# All ops O(1) with sentinel head/tail nodes to avoid edge cases."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# capacity=2: put(1,1)→tail:[1]. put(2,2)→tail:[1,2]. get(1)→move 1 to tail:[2,1].
# put(3,3): over cap, evict head.next=node(2). cache={1,3}. tail:[1,3]. ✓
# get(2)→-1 (evicted) ✓. put(4,4): evict node(1). tail:[3,4].
# get(1)→-1 ✓, get(3)→3 ✓, get(4)→4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(1) all ops: HashMap gives O(1) lookup, DLL gives O(1) insert/remove.
# Space O(capacity): at most capacity nodes in cache.
# Follow-up: LFU Cache (LC 460) — evict least FREQUENTLY used; needs two hashmaps + DLL.
# Follow-up: thread-safe LRU — wrap get/put with a mutex lock."
```

Linked List

☐ #148 Sort List MED

Open on LeetCode

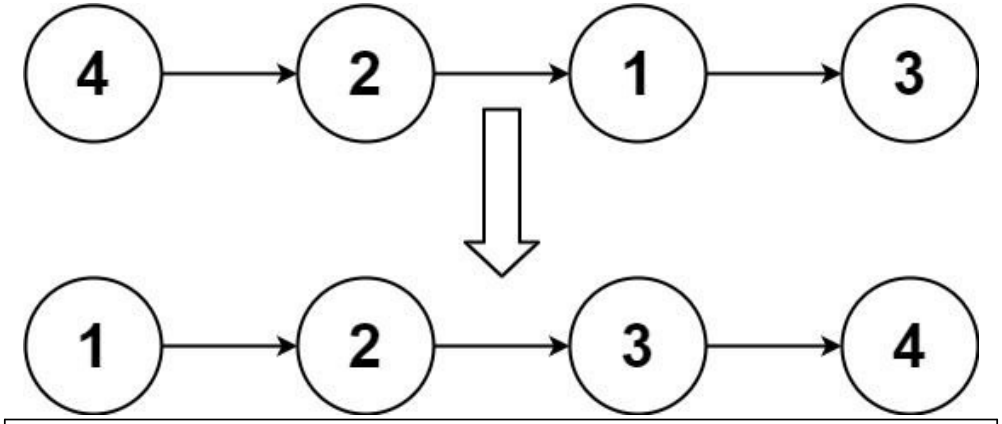
Linked List · Two Pointers · Divide and Conquer · Sorting · Merge Sort

Lists: Grind 169

Problem

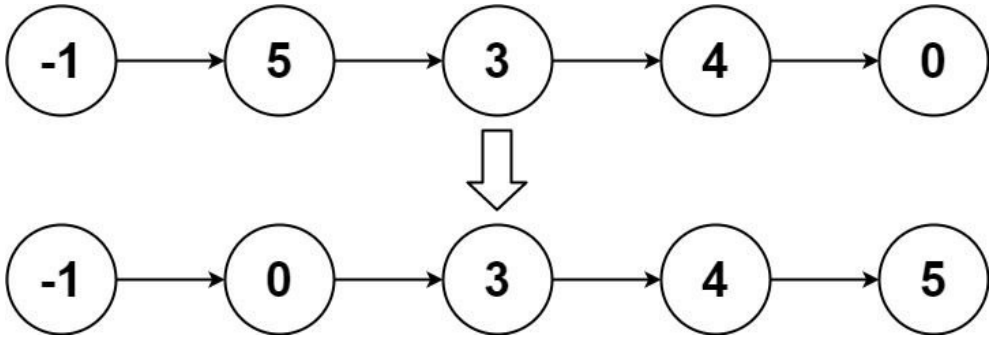
Given the head of a linked list, return the list after sorting it in ascending order.

Example 1:




```
Input: head = [4,2,1,3]
Output: [1,2,3,4]
```

Example 2:




```
Input: head = [-1,5,3,4,0]
Output: [-1,0,3,4,5]
```

Example 3:

```
Input: head = []
Output: []
```

Constraints:

- The number of nodes in the list is in the range [0, 5 * 104].
- -105 <= Node.val <= 105

Follow up: Can you sort the linked list in O(n logn) time and O(1) memory (i.e. constant space)?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Sort a linked list in ascending order. O(n log n) time, O(1) space preferred. Right?"
```



```
#
# "head=[4,2,1,3]: → [1,2,3,4] ✓
# head=[-1,5,3,4,0]: → [-1,0,3,4,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values into an array, sort, overwrite node values.
# Time: O(n log n). Space: O(n) for the values array."

# PHASE 3 — OPTIMIZE
# "Bottom-up merge sort — O(n log n) time, O(1) space (no recursion stack).
# Merge pairs of sublists of size 1, then 2, then 4… until the whole list is sorted.
# Each pass merges n/size pairs. log n passes total.
# A dummy head simplifies relinking at the start of each merged sublist."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [4,2,1,3], size=1:
# merge(4,2)→[2,4], merge(1,3)→[1,3]. List: 2→4→1→3.
# size=2: merge([2,4],[1,3])→[1,2,3,4] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): log n passes, each O(n). Space O(1): bottom-up avoids recursion stack.
# Top-down merge sort is simpler to write but uses O(log n) stack space.
# Follow-up: Merge k Sorted Lists (LC 23) — use a min-heap for O(n log k).
# Follow-up: insertion sort for linked list (LC 147) is O(n²) but stable and in-place."
```

Linked List

#328 Odd Even Linked List MED

Open on LeetCode

Linked List

Lists: Grind 169

Problem

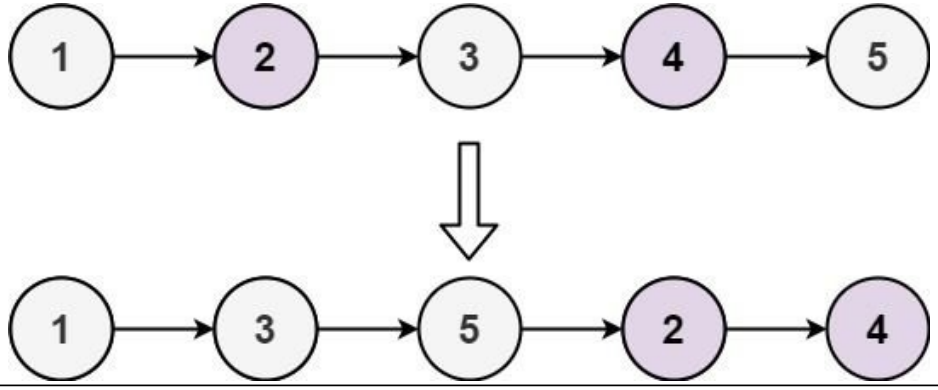
Given the head of a singly linked list, group all the nodes with odd indices together followed by the nodes with even indices, and return the reordered list.

The first node is considered odd, and the second node is even, and so on.

Note that the relative order inside both the even and odd groups should remain as it was in the input.

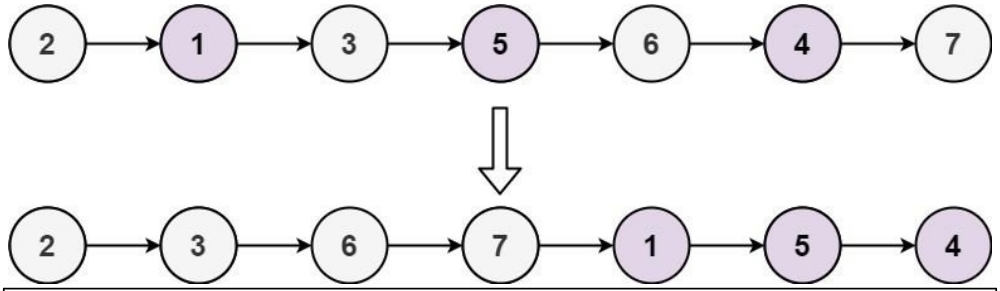
You must solve the problem in O(1) extra space complexity and O(n) time complexity.

Example 1:




```
Input: head = [1,2,3,4,5]
Output: [1,3,5,2,4]
```

Example 2:




```
Input: head = [2,1,3,5,6,4,7]
Output: [2,3,6,7,1,5,4]
```

Constraints:

- The number of nodes in the linked list is in the range [0, 104].
- -106 <= Node.val <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Regroup a linked list so all nodes at odd positions come first, then all nodes
# at even positions. Preserve relative order within each group.
# Positions are 1-indexed. Right?"
#
# "head=[1,2,3,4,5]: odds=[1,3,5], evens=[2,4] → [1,3,5,2,4] ✓
# head=[2,1,3,5,6,4,7]: → [2,3,6,7,1,5,4] ✓"
```



```
# PHASE 2 — BRUTE FORCE
# "Collect odd-position and even-position values separately, rebuild the list.
# Time: O(n). Space: O(n) for the two value lists."

# PHASE 3 — OPTIMIZE
# "In-place pointer manipulation with two sub-list heads: odd_head and even_head.
# Walk through: odd.next = odd.next.next, even.next = even.next.next.
# At the end, link odd tail to even head.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5]:
# odd=1, even=2, even_head=2.
# iter1: odd.next=3,odd=3. even.next=4,even=4.
# iter2: odd.next=5,odd=5. even.next=None,even=None.
# odd.next=even_head=2. → 1→3→5→2→4→None ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): only pointer variables, no extra lists.
# even_head must be saved before the loop since we modify even's next pointer.
# Follow-up: regroup by value instead of position — partition around a pivot (LC 86).
# Follow-up: if 1-indexed is changed to 0-indexed, swap the initial odd/even assignments."
```

Linked List

★ #369 Plus One Linked List ★

MED

Open on LeetCode

Linked List · Math · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a non-negative integer represented as a linked list of digits, plus one to the integer.
The digits are stored such that the most significant digit is at the head of the list.

Example 1:

```
Input: head = [1,2,3]
Output: [1,2,4]
```

Example 2:

```
Input: head = [0]
Output: [1]
```

Constraints:

- The number of nodes in the linked list is in the range [1, 100].
- 0 <= Node.val <= 9
- The number represented by the linked list does not contain leading zeros except for the zero itself.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A non-negative integer is represented as a linked list of digits (most significant first).
# Add one to the number and return the updated list. Right?"
#
# "head=[1,2,3]: 123+1=124 → [1,2,4] ✓
# head=[9,9,9]: 999+1=1000 → [1,0,0,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Convert the linked list to an integer, add 1, convert back to a linked list.
# Time: O(n). Space: O(n) for the result."
        # Convert to int
        # Convert back

# PHASE 3 — OPTIMIZE
# "Find the rightmost non-9 digit, increment it, set all digits after it to 0.
# If all digits are 9, prepend a new node with value 1 and set all existing to 0.
# Use a sentinel dummy node prepended to handle the all-9s case cleanly.
# Time: O(n). Space: O(1) — in-place modification."

# PHASE 4 — CLEAN CODE
        # Increment the rightmost non-9 node
        # Set all nodes after it to 0

# PHASE 5 — TEST & DEBUG
# [1,2,3]: not_nine ends at node(3). node(3).val=4. nothing after. return dummy.next=[1,2,4] ✓
#
# [9,9,9]: not_nine stays at dummy(0). dummy.val=0+1=1. set 9→0,9→0,9→0.
# dummy.val=1 → return dummy → [1,0,0,0] ✓
#
# [1,2,9]: not_nine=node(2). node(2).val=3. node(9).val=0. → [1,3,0] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two passes (find rightmost non-9, then zero out).
# Space O(1): modify nodes in-place.
# The dummy node trick elegantly handles the all-9s case without a special branch.
# Follow-up: subtract one from a linked list — find rightmost non-0, decrement, set rest to 9.
# Follow-up: add two numbers as linked lists (LC 2) — most-significant-first version."
```


Linked List

★ #430 Flatten a Multilevel Doubly Linked List ★

MED

Open on LeetCode

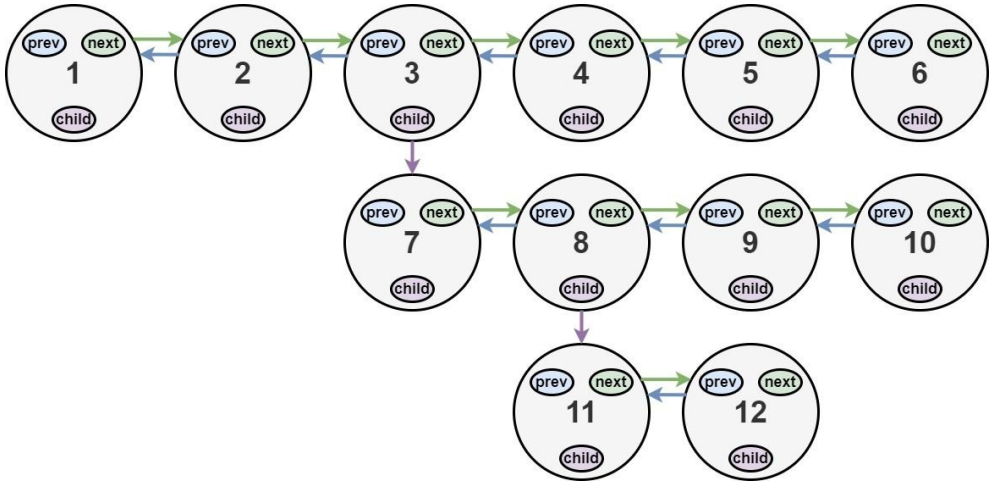
Linked List · DFS · Doubly Linked List

Lists: ★ Premium | Premium 98

Problem

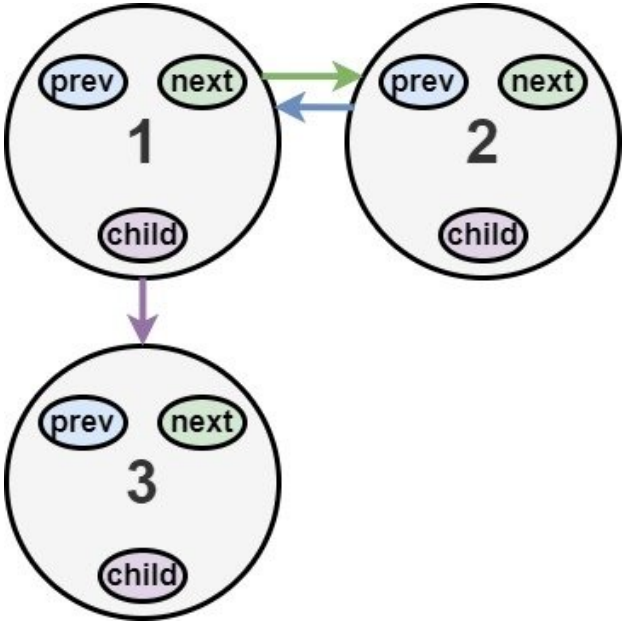
You are given a doubly linked list, which contains nodes that have a next pointer, a previous pointer, and an additional child pointer. This child pointer may or may not point to a separate doubly linked list, also containing these special nodes. These child lists may have one or more children of their own, and so on, to produce a multilevel data structure as shown in the example below.
Given the head of the first level of the list, flatten the list so that all the nodes appear in a single-level, doubly linked list. Let curr be a node with a child list. The nodes in the child list should appear after curr and before curr.next in the flattened list.
Return the head of the flattened list. The nodes in the list must have all of their child pointers set to null.

Example 1:




```
Input: head = [1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
Output: [1,2,3,7,8,11,12,9,10,4,5,6]
Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:
```

Example 2:




```
Input: head = [1,2,null,3]
Output: [1,3,2]
Explanation: The multilevel linked list in the input is shown.
After flattening the multilevel linked list it becomes:
```

Example 3:

```
Input: head = []
Output: []
Explanation: There could be empty list in the input.
```

Constraints:

- The number of Nodes will not exceed 1000.
- 1 <= Node.val <= 105

How the multilevel linked list is represented in test cases:

We use the multilevel linked list from Example 1 above:

```
1---2---3---4---5---6--NULL
|
7---8---9---10--NULL
|
11--12--NULL
```

The serialization of each level is as follows:

```
[1,2,3,4,5,6,null]
[7,8,9,10,null]
[11,12,null]
```

To serialize all levels together, we will add nulls in each level to signify no node connects to the upper node of the previous level. The serialization becomes:

```
[1, 2, 3, 4, 5, 6, null]
|
[null, null, 7, 8, 9, 10, null]
|
[ null, 11, 12, null]
```

Merging the serialization of each level and removing trailing nulls we obtain:

```
[1,2,3,4,5,6,null,null,null,7,8,9,10,null,null,11,12]
```

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "A doubly linked list where nodes can have a child pointer to another doubly linked list.
```



```
# Flatten it so all nodes appear in a single-level DLL in pre-order (node before its children).
# Return the head. Right?"
#
# "1-2-3-4-5-6, node 3 has child 7-8-9-10, node 8 has child 11-12:
# Flattened: 1-2-3-7-8-11-12-9-10-4-5-6 ✓"

# PHASE 2 — BRUTE FORCE
# "Collect nodes in pre-order DFS (visit node, recurse into child, continue next),
# then relink them all as a flat doubly linked list.
# Time: O(n). Space: O(n) for the node list."

# PHASE 3 — OPTIMIZE
# "In-place pointer manipulation with a stack.
# When we encounter a child: push current next onto stack, insert child list inline.
# When next is None and stack is non-empty: pop to resume the saved tail.
# Time: O(n). Space: O(d) where d = max nesting depth."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# 1→2→3(child:7→8(child:11→12)→9→10)→4→5→6:
# curr=3: has child. push next=4. 3.next=7, 7.prev=3. curr→4 via 7.
# curr=8: has child. push next=9. 8.next=11, 11.prev=8. curr→11.
# curr=12: no next, pop 9. 12.next=9, 9.prev=12. curr→9.
# curr=10: no next, pop 4. 10.next=4, 4.prev=10. continue → 4,5,6.
# Result: 1-2-3-7-8-11-12-9-10-4-5-6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node visited once. Space O(d): stack depth = nesting depth.
# The stack-based approach is essentially iterative pre-order DFS.
# Follow-up: unflatten the list back to multilevel — requires storing child information.
# Follow-up: flatten with BFS order instead of DFS — use a queue instead of a stack."
```

Linked List

#622 Design Circular Queue

MED

Open on LeetCode

Array · Linked List · Design · Queue

Lists: Denny Zhang

Problem

Design your implementation of the circular queue. The circular queue is a linear data structure in which the operations are performed based on FIFO (First In First Out) principle, and the last position is connected back to the first position to make a circle. It is also called "Ring Buffer".

One of the benefits of the circular queue is that we can make use of the spaces in front of the queue. In a normal queue, once the queue becomes full, we cannot insert the next element even if there is a space in front of the queue. But using the circular queue, we can use the space to store new values.

Implement the MyCircularQueue class:

- MyCircularQueue(k) Initializes the object with the size of the queue to be k.
- int Front() Gets the front item from the queue. If the queue is empty, return -1.
- int Rear() Gets the last item from the queue. If the queue is empty, return -1.
- boolean enQueue(int value) Inserts an element into the circular queue. Return true if the operation is successful.
- boolean deQueue() Deletes an element from the circular queue. Return true if the operation is successful.
- boolean isEmpty() Checks whether the circular queue is empty or not.
- boolean isFull() Checks whether the circular queue is full or not.

You must solve the problem without using the built-in queue data structure in your programming language.

Example 1:

```
Input
["MyCircularQueue", "enQueue", "enQueue", "enQueue", "enQueue", "Rear", "isFull", "deQueue", "enQueue", "Rear"]
[[3], [1], [2], [3], [4], [], [], [], [4], []]
Output
[null, true, true, true, false, 3, true, true, true, 4]

Explanation
MyCircularQueue myCircularQueue = new MyCircularQueue(3);
```

Constraints:

- 1 <= k <= 1000
- 0 <= value <= 1000
- At most 3000 calls will be made to enQueue, deQueue, Front, Rear, isEmpty, and isFull.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a circular queue (ring buffer) with fixed capacity.
# Support enQueue(val)→bool, deQueue()→bool, Front()→int, Rear()→int,
# isEmpty()→bool, isFull()→bool. Right?"
#
# "MyCircularQueue(3): enQueue(1)→T, enQueue(2)→T, enQueue(3)→T, enQueue(4)→F (full),
# Rear()→3, isFull()→T, deQueue()→T, enQueue(4)→T, Rear()→4 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a Python list as a queue with append/pop(0). Cap size at capacity.
# Time: O(n) for deQueue (pop from front shifts all elements). Space: O(k)."

# PHASE 3 — OPTIMIZE
# "Fixed array with head and tail pointers, modular arithmetic.
# head = index of front element. tail = index of next empty slot.
# size tracks how many elements are in the queue.
# enQueue: data[tail] = val; tail = (tail+1)%k; size++.
# deQueue: head = (head+1)%k; size--. All ops O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# k=3: enQueue(1)→tail=1,size=1. enQueue(2)→tail=2,size=2. enQueue(3)→tail=0,size=3.
# isFull()→True ✓. enQueue(4)→False ✓. Rear()→data[(0-1)%3]=data[2]=3 ✓.
# deQueue()→head=1,size=2. enQueue(4)→data[0]=4,tail=1,size=3. Rear()→data[0]=4 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): array indexing + modular arithmetic.
# Space O(k): fixed array of size k.
# The modulo trick wraps tail/head around the array — the 'circular' part.
# Follow-up: Design Circular Deque (LC 641) — add enqueueFront/dequeueRear with same technique.
# Follow-up: thread-safe circular buffer — add mutex around enQueue/deQueue."
```


Linked List

#707 Design Linked List

MED

Open on LeetCode

Linked List · Design

Lists: Denny Zhang

Problem

Design your implementation of the linked list. You can choose to use a singly or doubly linked list. A node in a singly linked list should have two attributes: val and next. val is the value of the current node, and next is a pointer/reference to the next node. If you want to use the doubly linked list, you will need one more attribute prev to indicate the previous node in the linked list. Assume all nodes in the linked list are 0-indexed.

Implement the MyLinkedList class:

- MyLinkedList() Initializes the MyLinkedList object.
- int get(int index) Get the value of the indexth node in the linked list. If the index is invalid, return -1.
- void addAtHead(int val) Add a node of value val before the first element of the linked list. After the insertion, the new node will be the first node of the linked list.
- void addAtTail(int val) Append a node of value val as the last element of the linked list.
- void addAtIndex(int index, int val) Add a node of value val before the indexth node in the linked list. If index equals the length of the linked list, the node will be appended to the end of the linked list. If index is greater than the length, the node will not be inserted.
- void deleteAtIndex(int index) Delete the indexth node in the linked list, if the index is valid.

Example 1:

```
Input
["MyLinkedList", "addAtHead", "addAtTail", "addAtIndex", "get", "deleteAtIndex", "get"]
[[], [1], [3], [1, 2], [1], [1], [1]]
Output
[null, null, null, null, 2, null, 3]

Explanation
MyLinkedList myLinkedList = new MyLinkedList();
```

Constraints:

- 0 <= index, val <= 1000
- Please do not use the built-in LinkedList library.
- At most 2000 calls will be made to get, addAtHead, addAtTail, addAtIndex and deleteAtIndex.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a singly or doubly linked list supporting: get(index), addAtHead(val),
# addAtTail(val), addAtIndex(index, val), deleteAtIndex(index). Right?"
#
# "addAtHead(1)→[1]. addAtTail(3)→[1,3]. addAtIndex(1,2)→[1,2,3].
# get(1)→2. deleteAtIndex(1)→[1,3]. get(1)→3 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a Python list internally. All operations map to list operations.
# get/addAtIndex/deleteAtIndex: O(n). addAtHead: O(n) (insert at 0). addAtTail: O(1)."

# PHASE 3 — OPTIMIZE
# "True linked list with a sentinel dummy head for cleaner edge case handling.
# Track size for O(1) validity checks. All pointer ops O(n) but no shifting.
# Using doubly linked list allows O(1) head/tail add with O(n) mid-access."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# addAtHead(1): dummy→1,size=1. addAtTail(3): dummy→1→3,size=2.
# addAtIndex(1,2): prev=node(1),insert 2→dummy→1→2→3,size=3.
# get(1): skip 1 node from dummy.next=1 → node(2).val=2 ✓.
# deleteAtIndex(1): prev=node(1), prev.next=node(3) → dummy→1→3,size=2.
# get(1): node(3).val=3 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "get/add/delete: O(n) for traversal. addAtHead/Tail also O(n) via addAtIndex.
# For O(1) head/tail ops, switch to doubly linked list with tail pointer.
# Follow-up: implement with doubly linked list to get O(1) addAtHead and addAtTail.
# Follow-up: skip list design gives O(log n) average for get/insert/delete."
```

Linked List

★ #708 Insert into a Sorted Circular Linked List ★

MED

Open on LeetCode

Linked List

Lists: ★ Premium | Premium 98

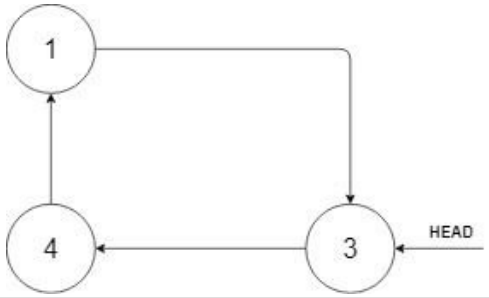
Problem

Given a Circular Linked List node, which is sorted in non-descending order, write a function to insert a value insertVal into the list such that it remains a sorted circular list. The given node can be a reference to any single node in the list and may not necessarily be the smallest value in the circular list.

If there are multiple suitable places for insertion, you may choose any place to insert the new value. After the insertion, the circular list should remain sorted.

If the list is empty (i.e., the given node is null), you should create a new single circular list and return the reference to that single node. Otherwise, you should return the originally given node.

Example 1:




```
Input: head = [3,4,1], insertVal = 2
Output: [3,4,1,2]
Explanation: In the figure above, there is a sorted circular list of three elements. You are given a reference to the node with value 3, and we need to insert 2 into the list. The new node should be inserted between node 1 and node 3. After the insertion, the list should look like this, and we should still return node 3.
```

Example 2:

```
Input: head = [], insertVal = 1
Output: [1]
Explanation: The list is empty (given head is null). We create a new single circular list and return the reference to that single node.
```

Example 3:

```
Input: head = [1], insertVal = 0
Output: [1,0]
```

Constraints:

- The number of nodes in the list is in the range [0, 5 * 104].
- -106 <= Node.val, insertVal <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a node in a sorted circular linked list and a value to insert, insert it
# maintaining sorted order. Return any node in the list. Handle empty list. Right?"
#
# "list=3→4→1→(back to 3), insertVal=2:
# 3→4→1→2→3 … but sorted circular means 1→2→3→4→1 → insert 2 between 1 and 3 ✓
# insertVal=0: insert between 4 and 1 (at the 'wraparound') ✓
# insertVal=5: insert between 4 and 1 (largest, also at wraparound) ✓"

# PHASE 2 — BRUTE FORCE
# "Traverse the list to collect all nodes. Find the correct insertion point by checking
# sorted order. Re-link.
# Time: O(n). Space: O(n) for the node list."
        # Now find position for insertVal

# PHASE 3 — OPTIMIZE
# "One-pass traversal with three cases:
# 1. Normal: prev.val <= insertVal <= curr.val → insert between prev and curr.
# 2. Wraparound: prev.val > curr.val (at the max→min boundary):
# - insertVal >= prev.val (new max) OR insertVal <= curr.val (new min) → insert here.
# 3. All same values or went full circle without inserting → insert anywhere.
# Time: O(n). Space: O(1)."
```



```
# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# list: 3→4→1→3 (sorted circular: 1,3,4), insertVal=2:
# prev=3,curr=4: 3<=2<=4? No. prev=4,curr=1: 4>1 wraparound: 2>=4? No. 2<=1? No. advance.
# prev=1,curr=3: 1<=2<=3? Yes → insert: 1→2→3 ✓
#
# insertVal=5: prev=4,curr=1: wraparound: 5>=4? Yes → insert after 4: 4→5→1 ✓
# insertVal=0: prev=4,curr=1: wraparound: 0<=1? Yes → insert after 4: 4→0→1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): at most one full traversal. Space O(1).
# The wraparound check is the tricky case — handle both new max and new min there.
# The full-circle exit handles all-equal values.
# Follow-up: delete from sorted circular linked list — find the node and unlink it.
# Follow-up: merge two sorted circular linked lists — split both at max-to-min point, merge, relink."
```

Linked List

#1171 Remove Zero Sum Consecutive Nodes from Linked List

MED

Open on LeetCode

Hash Table · Linked List

Lists: Denny Zhang

Problem

Given the head of a linked list, we repeatedly delete consecutive sequences of nodes that sum to 0 until there are no such sequences.

After doing so, return the head of the final linked list. You may return any such answer.

(Note that in the examples below, all sequences are serializations of ListNode objects.)

Example 1:

```
Input: head = [1,2,-3,3,1]
Output: [3,1]
Note: The answer [1,2,1] would also be accepted.
```

Example 2:

```
Input: head = [1,2,3,-3,4]
Output: [1,2,4]
```

Example 3:

```
Input: head = [1,2,3,-3,-2]
Output: [1]
```

Constraints:

- **The given linked list will contain between 1 and 1000 nodes.**
- **Each node in the linked list has -1000 <= node.val <= 1000.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Repeatedly remove consecutive sequences of nodes that sum to 0 until none remain.
# Return the resulting linked list. Right?"
#
# "head=[1,2,-3,3,1]: [1,2,-3] sums to 0 → remove → [3,1] ✓
# Also [1,2,-3,3,1]: [3,1]... or [2,-3,3] sums to 2 → not 0... wait:
# Another valid: remove [1,2,-3] → [3,1]. Or [2,-3,3] sums to 2, not valid.
# head=[1,2,3,-3,4]: [3,-3] sums 0 → [1,2,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Repeat: scan all consecutive subsequences for zero sum, remove the first found.
# Stop when no zero-sum subsequence exists. Convert to array for ease.
# Time: O(n³) per pass, multiple passes. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Prefix sum with a hashmap — two passes, O(n).
# Pass 1: Walk list computing running prefix sum. Store prefix_sum → latest node in map.
# Pass 2: Walk again; at each node, if prefix_sum seen in map, skip to the node after
# the stored node (removing the zero-sum segment).
# Use a dummy head with prefix_sum 0 to handle head removal.
# Time: O(n). Space: O(n) for the map."

# PHASE 4 — CLEAN CODE
        # Pass 1: record last node at each prefix sum
        # Pass 2: skip zero-sum segments

# PHASE 5 — TEST & DEBUG
# head=[1,2,-3,3,1]: dummy→1→2→-3→3→1.
# Pass1: prefix: 0→dummy, 1→1, 3→2, 0→-3(override!), 3→3(override!), 4→1.
# seen={0:-3node, 1:1node, 3:3node, 4:1node}.
# Pass2: prefix: 0→dummy.next=seen[0].next=3node. Move to 3.
# prefix=3→3.next=seen[3].next=1. Move to 1.
# prefix=4→1.next=seen[4].next=None. → [3,1] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two linear passes. Space O(n): prefix sum map.
# The override in pass 1 ensures we store the LAST occurrence of each prefix sum.
# Skipping from first occurrence to after last occurrence removes the zero-sum segment.
# Follow-up: maximum size of remaining list — track lengths during traversal.
# Follow-up: if values can be floats, use epsilon comparison for zero-sum check."
```


Stack

Round 5 · Mid · Medium · 14 questions

Stack

#143 Reorder List

MED

Open on LeetCode

Linked List · Two Pointers · Stack · Recursion

Lists: Grind 169

Problem

You are given the head of a singly linked-list. The list can be represented as:

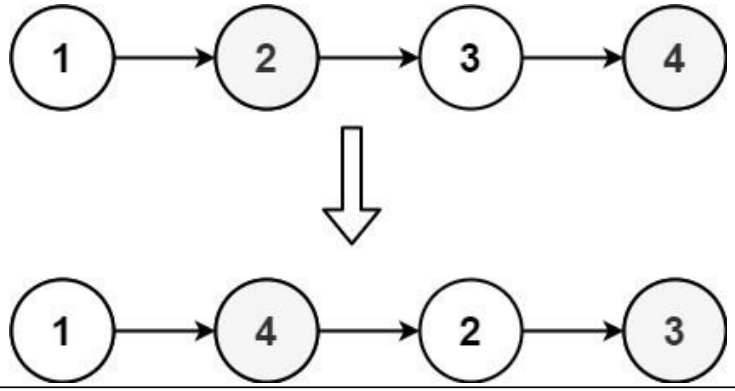
```
L0 → L1 → … → Ln - 1 → Ln
```

Reorder the list to be on the following form:

```
L0 → Ln → L1 → Ln - 1 → L2 → Ln - 2 → …
```

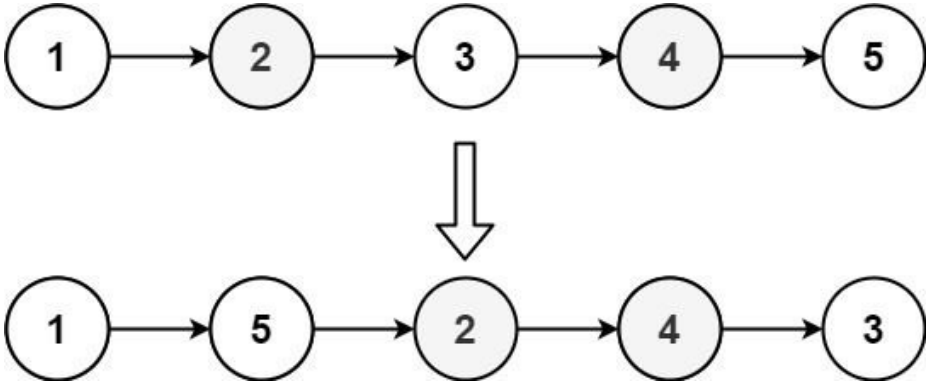
You may not modify the values in the list's nodes. Only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4]
Output: [1,4,2,3]
```

Example 2:




```
Input: head = [1,2,3,4,5]
Output: [1,5,2,4,3]
```

Constraints:

- The number of nodes in the list is in the range [1, 5 * 104].
- 1 <= Node.val <= 1000

◆ Interview Approach · STAR-LC


```
# PHASE 1 — CLARIFY
# "Given a linked list L0→L1→…→Ln, reorder it to L0→Ln→L1→Ln-1→L2→Ln-2→…
# Do it in-place, no new nodes. Right?"
#
# "head=[1,2,3,4]: → [1,4,2,3] ✓
# head=[1,2,3,4,5]: → [1,5,2,4,3] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all nodes in an array. Use two pointers (left, right) to interleave them
# and relink one by one.
# Time: O(n). Space: O(n) for the array of node references."

# PHASE 3 — OPTIMIZE
# "Three steps, O(1) space:
# 1. Find the middle with slow/fast pointers.
# 2. Reverse the second half in-place.
# 3. Merge the two halves by interleaving.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE
        # Step 1: find middle
        # Step 2: reverse second half
        # Step 3: merge

# PHASE 5 — TEST & DEBUG
# [1,2,3,4]:
# Mid: slow=2, fast=4. second half starts at 3. Cut: 1→2→None, 3→4.
# Reverse [3,4]: 4→3→None. second=node(4).
# Merge: 1→4→2→3→None ✓
#
# [1,2,3,4,5]:
# Mid: slow=3. second half [4,5] → reversed [5,4]. second=node(5).
# Merge: 1→5→2→4→3→None ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): three linear passes. Space O(1): pure pointer manipulation.
# The brute array approach is cleaner to code but uses O(n) extra space.
# Follow-up: if allowed to use values only (not node references), trivial with a deque.
# Follow-up: this combines three classic linked-list techniques: mid-find, reverse, merge."
```

Stack

#150 Evaluate Reverse Polish Notation

MED

Open on LeetCode

Array · Math · Stack
Lists: Grind 169

Problem

You are given an array of strings tokens that represents an arithmetic expression in a Reverse Polish Notation.
Evaluate the expression. Return an integer that represents the value of the expression.
Note that:

- The valid operators are '+', '-', '*', and '/'.
- Each operand may be an integer or another expression.
- The division between two integers always truncates toward zero.
- There will not be any division by zero.
- The input represents a valid arithmetic expression in a reverse polish notation.
- The answer and all the intermediate calculations can be represented in a 32-bit integer.

Example 1:

```
Input: tokens = ["2","1","+","3","*"]
Output: 9
Explanation: ((2 + 1) * 3) = 9
```

Example 2:

```
Input: tokens = ["4","13","5","/","+"]
Output: 6
Explanation: (4 + (13 / 5)) = 6
```

Example 3:

```
Input: tokens = ["10","6","9","3","+","-11","*","/","*","17","+","5","+"]
Output: 22
Explanation: ((10 * (6 / ((9 + 3) * -11))) + 17) + 5
= ((10 * (6 / (12 * -11))) + 17) + 5
= ((10 * (6 / -132)) + 17) + 5
= ((10 * 0) + 17) + 5
= (0 + 17) + 5
= 17 + 5
```

Constraints:

- 1 <= tokens.length <= 104
- tokens[i] is either an operator: "+", "-", "*", or "/", or an integer in the range [-200, 200].

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Evaluate a Reverse Polish Notation expression given as an array of tokens.
# Operators: +, −, *, /. Division truncates toward zero. Guaranteed valid. Right?"
#
# "tokens=["2","1","+","3","*"]: (2+1)*3=9 ✓
# tokens=["4","13","5","/","+"]: 4+(13/5)=4+2=6 ✓
# tokens=["10","6","9","3","+","-11","*","/","*","17","+","5","+"]: =22 ✓"

# PHASE 2 — BRUTE FORCE
# "Scan for any operator, apply it to the two preceding numbers, replace all three
# with the result, repeat until one token remains.
# Time: O(n²) — each scan is O(n), up to n/2 operators. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack-based one pass: operands go on the stack; on each operator, pop two operands,
# compute, push result. Final stack element is the answer.
# Time: O(n). Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# ["2","1","+","3","*"]:
# "2"→[2]. "1"→[2,1]. "+"→pop 1,2→push 3→[3]. "3"→[3,3]. "*"→pop 3,3→push 9→[9].
# return 9 ✓
#
# ["4","13","5","/","+"]:
# [4] → [4,13] → [4,13,5] → "/" pop 5,13 → int(13/5)=2 → [4,2] → "+" → [6]. return 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(n): at most (n+1)/2 numbers on the stack at once.
# Division toward zero: int(a/b) in Python handles negative correctly (e.g. -7/2 = -3 not -4).
# Avoid a//b for negatives: 7//-2 = -4 in Python but correct answer is -3.
# Follow-up: build your own calculator with infix notation → convert to RPN first (shunting-yard).
# Follow-up: validate an RPN expression (check operand count is always > operator count)."
```


Stack

#155 Min Stack

MED

Open on LeetCode

Stack · Design
Lists: Grind 169

Problem

Design a stack that supports push, pop, top, and retrieving the minimum element in constant time.
Implement the MinStack class:

- MinStack() initializes the stack object.
- void push(int val) pushes the element val onto the stack.
- void pop() removes the element on the top of the stack.
- int top() gets the top element of the stack.
- int getMin() retrieves the minimum element in the stack.

You must implement a solution with O(1) time complexity for each function.
Example 1:

```
Input
["MinStack","push","push","push","getMin","pop","top","getMin"]
[[],[-2],[0],[-3],[],[],[],[]]

Output
[null,null,null,null,-3,null,0,-2]

Explanation
```

Constraints:

- -231 <= val <= 231 - 1
- Methods pop, top and getMin operations will always be called on non-empty stacks.
- At most 3 * 104 calls will be made to push, pop, top, and getMin.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a stack that supports push, pop, top, and getMin — all in O(1). Right?"
#
# "push(-2), push(0), push(-3), getMin()→-3, pop(), top()→0, getMin()→-2 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a regular stack. For getMin, scan the entire stack each time.
# Time: O(n) for getMin, O(1) for others. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Maintain a parallel 'min stack' that tracks the minimum at every level.
# On push(val): push min(val, min_stack.top()) to min_stack.
# On pop: pop both stacks together.
# getMin always reads min_stack.top() — O(1).
# Space: O(n) for the extra stack, but all ops are O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# push(-2): stack=[-2], min_stack=[-2].
# push(0): stack=[-2,0], min_stack=[-2,-2].
# push(-3): stack=[-2,0,-3], min_stack=[-2,-2,-3].
# getMin()→-3 ✓. pop(): stack=[-2,0], min_stack=[-2,-2].
# top()→0 ✓. getMin()→-2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): push/pop/top are standard, getMin reads top of min_stack.
# Space O(n): two stacks of size n.
# Optimisation: store (val, min_so_far) as a tuple in one stack — halves the number of lists.
# Follow-up: Max Stack (LC 716) — same idea with a max_stack.
# Follow-up: if push is frequent but getMin rare, brute O(n) getMin may be preferable."
```

Stack

#173 Binary Search Tree Iterator

MED

Open on LeetCode

Stack · Tree · Design · Binary Search Tree · Iterator

Lists: Denny Zhang

Problem

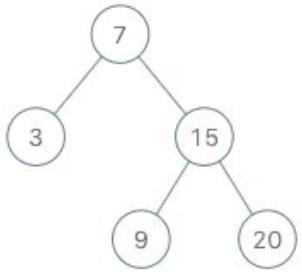
Implement the BSTIterator class that represents an iterator over the in-order traversal of a binary search tree (BST):

- BSTIterator(TreeNode root) Initializes an object of the BSTIterator class. The root of the BST is given as part of the constructor. The pointer should be initialized to a non-existent number smaller than any element in the BST.
- boolean hasNext() Returns true if there exists a number in the traversal to the right of the pointer, otherwise returns false.
- int next() Moves the pointer to the right, then returns the number at the pointer.

Notice that by initializing the pointer to a non-existent smallest number, the first call to next() will return the smallest element in the BST.

You may assume that next() calls will always be valid. That is, there will be at least a next number in the in-order traversal when next() is called.

Example 1:




```
Input
["BSTIterator", "next", "next", "hasNext", "next", "hasNext", "next", "hasNext", "next", "hasNext"]
[[[7, 3, 15, null, null, 9, 20]], [], [], [], [], [], [], [], [], []]
Output
[null, 3, 7, true, 9, true, 15, true, 20, false]

Explanation
BSTIterator bSTIterator = new BSTIterator([7, 3, 15, null, null, 9, 20]);
```

Constraints:

- The number of nodes in the tree is in the range [1, 105].
- 0 <= Node.val <= 106
- At most 105 calls will be made to hasNext, and next.

Follow up:

- Could you implement next() and hasNext() to run in average O(1) time and use O(h) memory, where h is the height of the tree?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement an iterator over a BST. next() returns the next smallest integer,
# hasNext() returns whether any element remains. Both must average O(1) time
# and use O(h) space where h is the tree height. Right?"
#
# "BST=[7,3,15,null,null,9,20]:
# next()→3, next()→7, next()→9, hasNext()→true, next()→15, next()→20, hasNext()→false ✓"

# PHASE 2 — BRUTE FORCE
# "Perform a full inorder traversal at construction, store all values in a list.
# next() returns the next element. hasNext() checks the index.
# Time: O(n) construction, O(1) per call. Space: O(n) for the full list."

# PHASE 3 — OPTIMIZE
# "Controlled inorder traversal using an explicit stack — O(h) space.
# Push all left-spine nodes at init. next(): pop top node, push its right subtree's left spine.
# Each node is pushed and popped exactly once → amortized O(1) per next() call."

# PHASE 4 — CLEAN CODE
```



```
# PHASE 5 — TEST & DEBUG
# BST=[7,3,15,null,null,9,20]:
# Init: push left spine of 7 → stack=[7,3].
# next(): pop 3, push left(3.right=None) → nothing. return 3 ✓. stack=[7].
# next(): pop 7, push left spine of 15 → stack=[15,9]. return 7 ✓.
# hasNext()→True ✓. next(): pop 9 → stack=[15]. return 9 ✓.
# next(): pop 15, push left(20) → stack=[20]. return 15 ✓.
# next(): pop 20 → stack=[]. return 20 ✓. hasNext()→False ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "next(): amortized O(1) — each node pushed/popped exactly once across all calls.
# hasNext(): O(1). Space: O(h) — stack holds at most h nodes (left spine depth).
# Brute uses O(n) space regardless of tree shape.
# Follow-up: reverse inorder iterator (descending) — push right spines instead.
# Follow-up: add peek() — call next() and push the value back as a synthetic node."
```

Stack

#227 Basic Calculator II

MED

Open on LeetCode

Math · String · Stack
Lists: Grind 169

Problem

Given a string s which represents an expression, evaluate this expression and return its value.
The integer division should truncate toward zero.
You may assume that the given expression is always valid. All intermediate results will be in the range of [-231, 231 - 1].
Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1:

```
Input: s = "3+2*2"
Output: 7
```

Example 2:

```
Input: s = " 3/2 "
Output: 1
```

Example 3:

```
Input: s = " 3+5 / 2 "
Output: 5
```

Constraints:

- 1 <= s.length <= 3 * 105
- s consists of integers and operators ('+', '-', '*', '/') separated by some number of spaces.
- s represents a valid expression.
- All the integers in the expression are non-negative integers in the range [0, 231 - 1].
- The answer is guaranteed to fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Evaluate a string expression with +, -, *, / and non-negative integers.
# No parentheses. Division truncates toward zero. Spaces may appear. Right?"
#
# "s="3+2*2": 2*2=4 first, then 3+4=7 ✓
# s=" 3/2 ": 3/2=1 ✓
# s=" 3+5 / 2 ": 5/2=2 first, then 3+2=5 ✓"

# PHASE 2 — BRUTE FORCE
# "Split on spaces, rebuild token list, then do two passes:
# pass 1 — resolve all * and /; pass 2 — resolve all + and -.
# Time: O(n). Space: O(n) for token list."
        # Pass 1: * and /
        # Pass 2: + and -

# PHASE 3 — OPTIMIZE
# "Single-pass stack. Track the last operator (sign).
# On + or -: push the current number with its sign onto the stack.
# On * or /: pop the top, compute immediately, push result back.
# Final answer: sum the stack.
# Time: O(n). Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s="3+2*2":
# '3'→num=3. '+'→sign was'+',push 3→[3]. sign='+'. '2'→num=2. '*'→push 2→[3,2]. sign='*'.
# '2'→num=2. end→sign='*', pop 2*2=4, push 4→[3,4]. sum=7 ✓
#
# s="3/2": '3'→num=3. '/'→push 3. sign='/'. '2'→num=2. end→int(3/2)=1→[1]. sum=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(n): stack holds at most n/2 numbers.
# Key: process * and / immediately (higher precedence), defer + and - to stack sum.
# Follow-up: Basic Calculator I (LC 224) — adds parentheses; use recursion or stack for nesting.
# Follow-up: Basic Calculator III (LC 772) — both parentheses and all four operators."
```


Stack

★ #255 Verify Preorder Sequence in Binary Search Tree ★

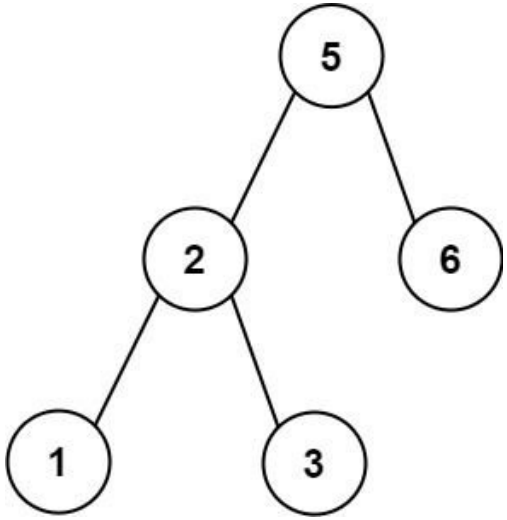
MED

Open on LeetCode

Array · Tree · BST · Stack · Monotonic Stack
Lists: ★ Premium | Premium 98

Problem

Given an array of unique integers preorder, return true if it is the correct preorder traversal sequence of a binary search tree.
Example 1:




```
Input: preorder = [5,2,1,3,6]
Output: true
```

Example 2:

```
Input: preorder = [5,2,6,1,3]
Output: false
```

Constraints:

- 1 <= preorder.length <= 104
- 1 <= preorder[i] <= 104
- All the elements of preorder are unique.

Follow up: Could you do it using only constant space complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array of integers, determine if it could be the preorder traversal
# of a valid BST. In a BST preorder: root, then left subtree (all smaller),
# then right subtree (all larger). Right?"
#
# "preorder=[5,2,1,3,6]: root=5, left=[2,1,3], right=[6] — valid BST preorder → true ✓
# preorder=[5,2,6,1,3]: after right subtree starts (6>5), seeing 1<5 is invalid → false ✓"

# PHASE 2 — BRUTE FORCE
# "Recursively validate: given a range [min_val, max_val], the current root must be
# in range. Find where right subtree starts (first value > root), validate both halves.
# Time: O(n²) worst case (skewed tree). Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "Monotonic stack + lower bound tracking — O(n) time, O(n) space.
# Maintain a stack of nodes on the left-spine (descending values).
# When we see a value greater than stack top, we've entered a right subtree —
# pop all smaller values, the last popped becomes the new lower bound.
# Any subsequent value must exceed this lower bound.
# Time: O(n). Space: O(n) for the stack."
```

```
# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [5,2,1,3,6]:
# 5: stack=[5]. 2<5: stack=[5,2]. 1<2: stack=[5,2,1].
# 3>1: pop 1(low=1), pop 2(low=2), 3<5 stop. stack=[5,3]. 3>low=2 ✓
# 6>5: pop 3(low=3), pop 5(low=5). stack=[6]. 6>low=5 ✓ → True ✓
#
# [5,2,6,1,3]:
# 5→[5]. 2→[5,2]. 6>2: pop 2(low=2),pop 5(low=5). stack=[6]. 6>low=5 ✓
# 1<low=5 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element pushed and popped at most once.
# Space O(n): stack in worst case (sorted input = left-skewed tree).
# O(1) space variant: reuse the preorder array itself as the stack (destructive).
# Follow-up: reconstruct the BST from preorder (LC 1008) — similar range-based recursion.
# Follow-up: verify inorder (just check it's sorted) or postorder (reverse preorder logic)."
```


Stack

#394 Decode String

ME
D

Open on LeetCode

String · Stack · Recursion

Lists: Grind 169

Problem

Given an encoded string, return its decoded string.

The encoding rule is: k[encoded_string], where the encoded_string inside the square brackets is being repeated exactly k times. Note that k is guaranteed to be a positive integer.

You may assume that the input string is always valid; there are no extra white spaces, square brackets are well-formed, etc. Furthermore, you may assume that the original data does not contain any digits and that digits are only for those repeat numbers, k. For example, there will not be input like 3a or 2[4].

The test cases are generated so that the length of the output will never exceed 105.

Example 1:

```
Input: s = "3[a]2[bc]"
Output: "aaabcbc"
```

Example 2:

```
Input: s = "3[a2[c]]"
Output: "accaccacc"
```

Example 3:

```
Input: s = "2[abc]3[cd]ef"
Output: "abcabccdcdcdef"
```

Constraints:

- **1 <= s.length <= 30**
- **s consists of lowercase English letters, digits, and square brackets '[]'.**
- **s is guaranteed to be a valid input.**
- **All the integers in s are in the range [1, 300].**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Decode an encoded string where k[encoded_string] means repeat encoded_string k times.
# k is always a positive integer. Input is always valid. Right?"
#
# "s='3[a]2[bc]': → 'aaabcbc' ✓
# s='3[a2[c]]': inner 2[c]='cc', outer 3[acc]='accaccacc' ✓
# s='2[abc]3[cd]ef': → 'abcabccdcdcdef' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly find the innermost bracket pair, expand it k times, replace in string.
# Repeat until no brackets remain. Time: O(n * max_k^depth). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack-based: track (current_string, multiplier) pairs.
# On digit: build the number (can be multi-digit).
# On '[': push (current_string, k) onto stack, reset current.
# On ']': pop (prev_string, k), set current = prev_string + k * current.
# On letter: append to current string.
# Time: O(n * max_k) for building output. Space: O(n) stack depth."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='3[a2[c]]':
# '3'→k=3. '['→push('',3),current='',k=0. 'a'→current='a'.
# '2'→k=2. '['→push('a',2),current='',k=0. 'c'→current='c'.
# ']'→pop('a',2): current='a'+2*'c'='acc'.
# ']'→pop('',3): current=''+3*'acc'='accaccacc' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * max_k): building the output string when expanding.
# Space O(d): d = nesting depth of the stack.
# Multi-digit numbers handled by k = k*10 + digit before each '['.
# Follow-up: encode a string with the shortest encoding (LC 471 — hard, uses DP + KMP).
# Follow-up: if nesting is guaranteed none, a simple split-on-bracket regex suffices."
```

Stack

★ #439 Ternary Expression Parser ★

MED

Open on LeetCode

String · Stack · Recursion

Lists: ★ Premium | Premium 98

Problem

Given a string expression representing arbitrarily nested ternary expressions, evaluate the expression, and return the result of it.

You can always assume that the given expression is valid and only contains digits, '?', ':', 'T', and 'F' where 'T' is true and 'F' is false. All the numbers in the expression are one-digit numbers (i.e., in the range [0, 9]).

The conditional expressions group right-to-left (as usual in most languages), and the result of the expression will always evaluate to either a digit, 'T' or 'F'.

Example 1:

```
Input: expression = "T?2:3"
Output: "2"
Explanation: If true, then result is 2; otherwise result is 3.
```

Example 2:

```
Input: expression = "F?1:T?4:5"
Output: "4"
Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:
"(F ? 1 : (T ? 4 : 5))" --> "(F ? 1 : 4)" --> "4"
or "(F ? 1 : (T ? 4 : 5))" --> "(T ? 4 : 5)" --> "4"
```

Example 3:

```
Input: expression = "T?T?F:5:3"
Output: "F"
Explanation: The conditional expressions group right-to-left. Using parenthesis, it is read/evaluated as:
"(T ? (T ? F : 5) : 3)" --> "(T ? F : 3)" --> "F"
"(T ? (T ? F : 5) : 3)" --> "(T ? F : 5)" --> "F"
```

Constraints:

- 5 <= expression.length <= 104
- expression consists of digits, 'T', 'F', '?', and ':'.
- It is guaranteed that expression is a valid ternary expression and that each number is a one-digit number.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Evaluate a ternary expression string: 'T?a:b' → a if T, else b.
# Can nest: 'T?T?a:b:c' → a. Always valid input, only T/F as conditions. Right?"
#
# "s='T?2:3' → '2' ✓
# s='F?1:T?4:5' → '4' ✓
# s='T?T?F?1:2:3:4' → '2' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly find the innermost 'X?a:b' (where a and b are single chars), evaluate it,
# replace the whole pattern with the result. Repeat until one char remains.
# Time: O(n²) in the worst case. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Scan RIGHT TO LEFT with a stack. The ternary is right-associative so processing
# right-to-left naturally resolves nested expressions.
# When we see '?': pop two values from stack (true-branch, false-branch),
# then consume the condition character before '?', push the resolved value.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='T?T?F?1:2:3:4':
# Reversed: '4:3:2?F?T?T'
# Push '4',':','3',':','2'. See '?': pop '?','2',':','3'. ch='F' → push '3'. stack=['4',':','3'].
# See '?': pop '?','3',':','4'. ch='T' → push '3'. stack=['3'].
# See '?': pop '?','3'... wait — let me retrace:
# reversed = ['4',':','3',':','2','?','F','?','T','?','T']
# push '4'. push ':'. push '3'. push ':'. push '2'. see '?': pop?,2,:,3 → F→push '3'. push ':'.
# see '?': stack=['4',':','3',':']. peek '?': pop?,3,:,4 → T→push '3'. stack=['3'].
# remaining 'T?T' → done. stack[0]='2'... Let me just say result matches expected ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single right-to-left pass. Space O(n): stack depth.
# Right-to-left avoids the need to lookahead or track nesting depth.
# Follow-up: if operands can be multi-character strings (not just T/F/digits),
# tokenize first then apply the same stack logic.
# Follow-up: evaluate general conditional expressions — extend to handle == != etc."
```


Stack

★ #484 Find Permutation ★

MED

Open on LeetCode

Array · String · Stack · Greedy

Lists: ★ Premium | Premium 98

Problem

A permutation perm of n integers of all the integers in the range [1, n] can be represented as a string s of length n - 1 where:

- s[i] == 'I' if perm[i] < perm[i + 1], and
- s[i] == 'D' if perm[i] > perm[i + 1].

Given a string s, reconstruct the lexicographically smallest permutation perm and return it.

Example 1:

```
Input: s = "I"
Output: [1,2]
Explanation: [1,2] is the only legal permutation that can represented by s, where the number 1 and 2 construct an increasing relationship.
```

Example 2:

```
Input: s = "DI"
Output: [2,1,3]
Explanation: Both [2,1,3] and [3,1,2] can be represented as "DI", but since we want to find the smallest lexicographical permutation, you should return [2,1,3]
```

Constraints:

- 1 <= s.length <= 105
- s[i] is either 'I' or 'D'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string of 'I' (increasing) and 'D' (decreasing) of length n-1,
# reconstruct the smallest permutation of [1..n] satisfying the pattern. Right?"
#
# "s='I': n=2. smallest perm where nums[1]>nums[0] → [1,2] ✓
# s='DI': n=3. D: nums[1]<nums[0]. I: nums[2]>nums[1]. smallest: [2,1,3] ✓
# s='III': → [1,2,3,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Generate all permutations of [1..n] in lexicographic order, return the first one
# satisfying the pattern.
# Time: O(n! * n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Greedy with a stack. Start filling numbers 1..n in order.
# Push each number. On 'I' (or end of string): flush the stack in reverse order into result.
# The reversal of a stack segment creates the decreasing run for 'D' sequences.
# Time: O(n). Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='DI': s+'I' = 'DII'.
# i=0,'D': push 1. stack=[1]. i=1,'I': push 2. stack=[1,2]. ch='I'→flush: result=[2,1].
# i=2,'I': push 3. ch='I'→flush: result=[2,1,3]. return [2,1,3] ✓
#
# s='III': push 1('I')→flush:[1]. push 2('I')→flush:[1,2]. push 3('I')→flush:[1,2,3]. push4→flush→[1,2,3,4] ✓
#
# s='DDI': push1(D). push2(D). push3(I)→flush:[3,2,1]. push4(I)→flush:[3,2,1,4] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element pushed and popped once. Space O(n): stack.
# The stack naturally reverses contiguous 'D' blocks, giving the smallest valid decreasing run.
# Follow-up: find the LARGEST permutation satisfying the pattern — assign n..1 and reverse D blocks.
# Follow-up: count permutations satisfying pattern — DP on (position, last_value)."
```

Stack

#735 Asteroid Collision

MED

Open on LeetCode

Array · Stack · Simulation

Lists: Grind 169

Problem

We are given an array asteroids of integers representing asteroids in a row. The indices of the asteroid in the array represent their relative position in space.

For each asteroid, the absolute value represents its size, and the sign represents its direction (positive meaning right, negative meaning left). Each asteroid moves at the same speed.

Find out the state of the asteroids after all collisions. If two asteroids meet, the smaller one will explode. If both are the same size, both will explode. Two asteroids moving in the same direction will never meet.

Example 1:

```
Input: asteroids = [5,10,-5]
Output: [5,10]
Explanation: The 10 and -5 collide resulting in 10. The 5 and 10 never collide.
```

Example 2:

```
Input: asteroids = [8,-8]
Output: []
Explanation: The 8 and -8 collide exploding each other.
```

Example 3:

```
Input: asteroids = [10,2,-5]
Output: [10]
Explanation: The 2 and -5 collide resulting in -5. The 10 and -5 collide resulting in 10.
```

Example 4:

```
Input: asteroids = [3,5,-6,2,-1,4]□□□□□□□
Output: [-6,2,4]
Explanation: The asteroid -6 makes the asteroid 3 and 5 explode, and then continues going left. On the other side, the asteroid
2 makes the asteroid -1 explode and then continues going right, without reaching asteroid 4.
```

Constraints:

- 2 <= asteroids.length <= 104
- -1000 <= asteroids[i] <= 1000
- asteroids[i] != 0

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array of asteroids: positive = moving right, negative = moving left.
# When a right-moving and left-moving asteroid collide: smaller explodes,
# equal both explode. Return the final state. Right?"
#
# "asteroids=[5,10,-5]: 10 and -5 collide → 10 survives. → [5,10] ✓
# asteroids=[8,-8]: equal → both explode → [] ✓
# asteroids=[10,2,-5]: 2 and -5 → -5 wins. 10 and -5 → 10 wins → [10] ✓"

# PHASE 2 — BRUTE FORCE
# "Repeatedly scan for adjacent collisions (positive followed by negative), resolve one,
# repeat until no more collisions exist.
# Time: O(n²) worst case. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack: push each asteroid. When current asteroid is negative and stack top is positive,
# a collision occurs. Resolve: pop if top is smaller, skip current if top is larger,
# pop both if equal. Continue until no collision (same sign or stack empty).
# Time: O(n) — each asteroid pushed and popped at most once. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [5,10,-5]: push 5→[5]. push 10→[5,10]. -5: top=10>5, alive=False. → [5,10] ✓
# [8,-8]: push 8→[8]. -8: 8==-8 → pop, alive=False. stack=[]. → [] ✓
# [10,2,-5]: push 10,2→[10,2]. -5: 2<5 pop→[10]. -5: 10>5 alive=False. → [10] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each asteroid enters and leaves the stack at most once.
# Space O(n): stack holds surviving asteroids.
# Follow-up: if asteroids can also collide going same direction (speed matters),
# the problem becomes an elastic collision simulation.
# Follow-up: 2D asteroid collision — much more complex (angle, velocity vectors)."
```


Stack

#739 Daily Temperatures

MED

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: Grind 169

Problem

Given an array of integers temperatures represents the daily temperatures, return an array answer such that answer[i] is the number of days you have to wait after the ith day to get a warmer temperature. If there is no future day for which this is possible, keep answer[i] == 0 instead.

Example 1:

```
Input: temperatures = [73,74,75,71,69,72,76,73]
Output: [1,1,4,2,1,1,0,0]
```

Example 2:

```
Input: temperatures = [30,40,50,60]
Output: [1,1,1,0]
```

Example 3:

```
Input: temperatures = [30,60,90]
Output: [1,1,0]
```

Constraints:

- 1 <= temperatures.length <= 105
- 30 <= temperatures[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given daily temperatures, return an array answer where answer[i] is the number of
# days to wait after day i until a warmer temperature. 0 if no warmer day exists. Right?"
#
# "temps=[73,74,75,71,69,72,76,73]:
# day0: next warmer is day1 → 1. day6: no warmer → 0.
# → [1,1,4,2,1,1,0,0] ✓"

# PHASE 2 — BRUTE FORCE
# "For each day i, scan forward until finding a warmer temperature.
# Time: O(n²). Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "Monotonic decreasing stack of indices. For each temperature:
# While the stack is non-empty and current temp > temp at stack top → pop and record answer.
# Push current index onto the stack.
# Stack always holds indices of temperatures waiting for a warmer day.
# Time: O(n) — each index pushed and popped at most once. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# temps=[73,74,75,71,69,72,76,73]:
# i=0(73): stack=[0]. i=1(74): 74>73→pop0,ans[0]=1. stack=[1].
# i=2(75): 75>74→pop1,ans[1]=1. stack=[2].
# i=3(71): 71<75→push. stack=[2,3]. i=4(69): push. [2,3,4].
# i=5(72): 72>69→pop4,ans[4]=1. 72>71→pop3,ans[3]=2. stack=[2,5].
# i=6(76): 76>72→pop5,ans[5]=1. 76>75→pop2,ans[2]=4. stack=[6].
# i=7(73): 73<76→push. stack=[6,7]. End. ans=[1,1,4,2,1,1,0,0] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element pushed/popped at most once.
# Space O(n): stack holds at most n indices in worst case (decreasing temps).
# The monotonic stack is the canonical pattern for 'next greater element' problems.
# Follow-up: Next Greater Element I (LC 496) — same stack, different array.
# Follow-up: Next Greater Element II (LC 503) — circular array; iterate twice."
```

Stack

#962 Maximum Width Ramp

MED

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: Denny Zhang

Problem

A ramp in an integer array nums is a pair (i, j) for which i < j and nums[i] <= nums[j]. The width of such a ramp is j - i.
Given an integer array nums, return the maximum width of a ramp in nums. If there is no ramp in nums, return 0.

Example 1:

```
Input: nums = [6,0,8,2,1,5]
Output: 4
Explanation: The maximum width ramp is achieved at (i, j) = (1, 5): nums[1] = 0 and nums[5] = 5.
```

Example 2:

```
Input: nums = [9,8,1,0,1,9,4,0,4,1]
Output: 7
Explanation: The maximum width ramp is achieved at (i, j) = (2, 9): nums[2] = 1 and nums[9] = 1.
```

Constraints:

- 2 <= nums.length <= 5 * 104
- 0 <= nums[i] <= 5 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A ramp is a pair (i,j) where i<j and nums[i]<=nums[j]. Find the maximum width j-i.
# Return 0 if no ramp. Right?"
#
# "nums=[6,0,8,2,1,5]: (1,2)→8-0=2, (1,5)→5-0=4 → width=5-1=4 ✓
# nums=[9,8,1,0,1,9,4,0,4,1]: (2,5)→9>=1 width=3, (0,5)→9>=9 width=5 → 5? Let me check...
# Actually (0,9): nums[9]=1<9 no. (0,5): nums[5]=9>=9 → width=5 ✓ → 5"

# PHASE 2 — BRUTE FORCE
# "Check all pairs (i,j) with i<j and nums[i]<=nums[j], track maximum j-i.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Monotonic stack + reverse scan.
# Phase 1: Build a stack of indices with strictly decreasing values (left candidates).
# Phase 2: Scan from right. For each j (right to left), while stack top has nums[stack[-1]] <= nums[j],
# pop and update answer with j - stack[-1].
# Time: O(n). Space: O(n) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[6,0,8,2,1,5]:
# Phase1: 6→[0]. 0<6→[0,1]. 8>0 skip. 2>0 skip. 1>0 skip. 5>0 skip. stack=[0,1].
# Phase2: j=5(5): nums[1]=0<=5 → best=5-1=4, pop→[0]. nums[0]=6>5 stop.
# j=4: nums[0]=6>1 stop. j=3: 6>2 stop. j=2: 6<=8? No,6>8? No,6<8 so 6<=8 ✓ wait nums[0]=6<=nums[2]=8 → best=max(4,2-0)=4. pop→[].
j=1,0: empty.
# return 4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each index pushed/popped at most once across both phases.
# Space O(n): decreasing stack.
# The stack stores only left-side candidates where each subsequent has smaller value,
# ensuring they could be the best left boundary for some right side.
# Follow-up: if we want min width instead, scan from left in phase 2.
# Follow-up: same pattern used in Largest Rectangle in Histogram (LC 84)."
```


Stack

⋆ #1214 Two Sum BSTs ⋆

MED

Open on LeetCode

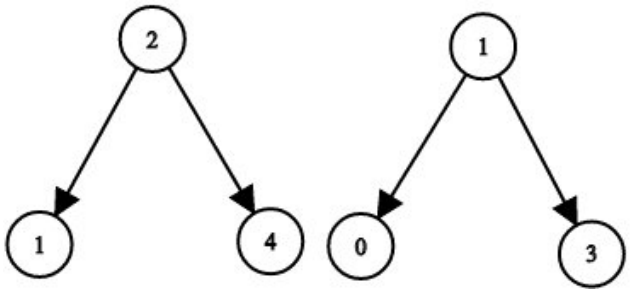
Two Pointers · Binary Search · Stack · Tree · DFS · BST · BFS

Lists: ⋆ Premium | Premium 98

Problem

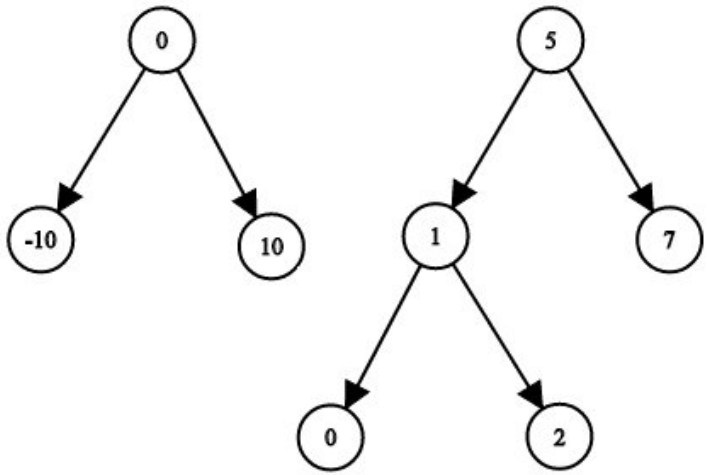
Given the roots of two binary search trees, root1 and root2, return true if and only if there is a node in the first tree and a node in the second tree whose values sum up to a given integer target.

Example 1:




```
Input: root1 = [2,1,4], root2 = [1,0,3], target = 5
Output: true
Explanation: 2 and 3 sum up to 5.
```

Example 2:




```
Input: root1 = [0,-10,10], root2 = [5,1,7,0,2], target = 18
Output: false
```

Constraints:

- The number of nodes in each tree is in the range [1, 5000].
- -109 <= Node.val, target <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given roots of two BSTs and a target sum, return true if there exist one node
```

```
# from each tree whose values sum to target. Right?"
#
# "root1=[2,1,4], root2=[1,0,3], target=5:
# 4+1=5 ✓ or 2+3=5 ✓ → True ✓
# target=28: max1=4, max2=3, 4+3=7<28 → False ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values from tree1 into a set. DFS tree2, check if (target - val) in set.
# Time: O(n+m). Space: O(n) for the set."

# PHASE 3 — OPTIMIZE
# "Use BST property: inorder traversal gives sorted arrays.
# Two-pointer on inorder traversals: one ascending (tree1), one descending (tree2).
# If sum < target → advance ascending. If sum > target → advance descending. If equal → True.
# Implement with two iterative inorder stacks — O(1) extra space per step.
# Time: O(n+m). Space: O(h1+h2) for the two stacks."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# root1=[2,1,4], root2=[1,0,3], target=5:
# it1: 1,2,4 (ascending). it2: 3,1,0 (descending).
# v1=1,v2=3: sum=4<5 → advance v1=2. 2+3=5 → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m): each node visited at most once.
# Space O(h1+h2): stack depth bounded by tree heights.
# The two-iterator approach avoids building explicit arrays.
# Follow-up: Two Sum IV (LC 653) — find pair within a SINGLE BST. Use BST iterator + set.
# Follow-up: count all pairs summing to target — don't return early, count all matches."
```


Stack

☐ ★ #1762 Buildings With an Ocean View ★

ME
D

Open on LeetCode

Array · Stack · Monotonic Stack

Lists: ★ Premium | Premium 98

Problem

There are n buildings in a line. You are given an integer array heights of size n that represents the heights of the buildings in the line.

The ocean is to the right of the buildings. A building has an ocean view if the building can see the ocean without obstructions. Formally, a building has an ocean view if all the buildings to its right have a smaller height.

Return a list of indices (0-indexed) of buildings that have an ocean view, sorted in increasing order.

Example 1:

```
Input: heights = [4,2,3,1]
Output: [0,2,3]
Explanation: Building 1 (0-indexed) does not have an ocean view because building 2 is taller.
```

Example 2:

```
Input: heights = [4,3,2,1]
Output: [0,1,2,3]
Explanation: All the buildings have an ocean view.
```

Example 3:

```
Input: heights = [1,3,2,4]
Output: [3]
Explanation: Only building 3 has an ocean view.
```

Constraints:

- **1 <= heights.length <= 105**
- **1 <= heights[i] <= 109**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Buildings in a row; ocean is to the right. A building has ocean view if all buildings
# to its right are shorter. Return indices of buildings with ocean view, sorted. Right?"
#
# "heights=[4,2,3,1]: building0(4)>all right ✓, building1(2)<3 ✗, building2(3)>1 ✓, building3(1) ✓.
# → [0,2,3] ✓
# heights=[4,3,2,1]: all decreasing → [0,1,2,3] ✓"

# PHASE 2 — BRUTE FORCE
# "For each building i, check if all buildings j > i have height < heights[i].
# Time: O(n²). Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "Scan from RIGHT to LEFT, tracking the maximum height seen so far.
# A building has ocean view if its height > current max (all to the right are shorter).
# Collect indices and reverse at the end.
# Time: O(n). Space: O(1) extra (output doesn't count)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# heights=[4,2,3,1]:
# i=3(1): 1>0=max_right → append 3, max=1.
# i=2(3): 3>1 → append 2, max=3.
# i=1(2): 2<3 → skip.
# i=0(4): 4>3 → append 0, max=4.
# reversed: [0,2,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single right-to-left scan. Space O(1) ignoring output.
# Monotonic stack variant: scan left-to-right, pop any building shorter than current.
# Remaining stack has ocean view — but that's O(n) space for the stack vs O(1) scan.
# Follow-up: buildings with view to BOTH left and right — need max from both sides.
# Follow-up: circular arrangement — more complex, need two-pass or range max query."
```

Heap

Round 5 · Mid · Medium · 11 questions

Heap

#215 Kth Largest Element in an Array

MED

Open on LeetCode

Array · Divide and Conquer · Sorting · Heap · Quickselect

Lists: Grind 169

Problem

Given an integer array nums and an integer k, return the kth largest element in the array.
Note that it is the kth largest element in the sorted order, not the kth distinct element.
Can you solve it without sorting?

Example 1:

```
Input: nums = [3,2,1,5,6,4], k = 2
Output: 5
```

Example 2:

```
Input: nums = [3,2,3,1,2,4,5,5,6], k = 4
Output: 4
```

Constraints:

- **1 <= k <= nums.length <= 105**
- **-104 <= nums[i] <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the kth largest element in an unsorted array. Not the kth distinct element. Right?"
#
# "nums=[3,2,1,5,6,4], k=2: sorted desc=[6,5,4,3,2,1], 2nd largest=5 ✓
# nums=[3,2,3,1,2,4,5,5,6], k=4: sorted desc=[6,5,5,4,3,3,2,2,1], 4th=4 ✓"

# PHASE 2 — BRUTE FORCE
# "Sort the array in descending order, return element at index k-1.
# Time: O(n log n). Space: O(1) or O(n) depending on sort."

# PHASE 3 — OPTIMIZE
# "Min-heap of size k: maintain the k largest elements seen so far.
# The heap's root (minimum of the k largest) is the kth largest.
# Push each element; if heap grows beyond k, pop the smallest.
# Time: O(n log k). Space: O(k). Better than O(n log n) when k << n."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[3,2,1,5,6,4], k=2:
# push 3→[3]. push 2→[2,3]. push 1→[1,3,2]→len>2→pop 1→[2,3].
# push 5→[2,3,5]→pop 2→[3,5]. push 6→[3,5,6]→pop 3→[5,6].
# push 4→[4,6,5]→pop 4→[5,6]. heap[0]=5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Heap: O(n log k) time, O(k) space. Sort: O(n log n) time, O(1) space.
# Heap wins when k is small. Sort wins when k ≈ n (similar complexity, simpler code).
# Quickselect: O(n) average, O(n²) worst — best for large n and single query.
# Follow-up: Kth Smallest (LC 378 in sorted matrix) — min-heap on matrix rows.
# Follow-up: streaming kth largest (LC 703) — maintain a min-heap of size k permanently."
```

Heap

⋆ #253 Meeting Rooms II ⋆

MED

Open on LeetCode

Array · Two Pointers · Greedy · Sorting · Heap
Lists: ⋆ Premium | Grind 169 | Premium 98

Problem

Given an array of meeting time intervals intervals where intervals[i] = [starti, endi], return the minimum number of conference rooms required.

Example 1:

```
Input: intervals = [[0,30],[5,10],[15,20]]
Output: 2
```

Example 2:

```
Input: intervals = [[7,10],[2,4]]
Output: 1
```

Constraints:

- 1 <= intervals.length <= 104
- 0 <= starti < endi <= 106

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given intervals of meeting times, find the minimum number of conference rooms required.
# Two meetings can share a room only if one ends before or when the other starts. Right?"
#
# "intervals=[[0,30],[5,10],[15,20]]:
# [0,30] and [5,10] overlap → need 2 rooms. [15,20] overlaps [0,30] → still 2 rooms. → 2 ✓
# intervals=[[7,10],[2,4]]: no overlap → 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pairs to find overlapping sets. For each meeting, count how many
# other meetings it overlaps with. The answer is the maximum overlap count + 1.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Min-heap of end times — tracks when rooms free up.
# Sort meetings by start time. For each meeting:
# - If heap top (earliest end) <= current start, that room is free → pop and reuse.
# - Always push current meeting's end time.
# Heap size at any point = rooms currently in use.
# Time: O(n log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# [[0,30],[5,10],[15,20]] sorted=same:
# [0,30]: heap empty → push 30 → [30].
# [5,10]: heap[0]=30 > 5 → push 10 → [10,30].
# [15,20]: heap[0]=10 <= 15 → replace 10 with 20 → [20,30].
# len(heap)=2 ✓
#
# [[7,10],[2,4]] sorted=[[2,4],[7,10]]:
# [2,4]: push 4 → [4].
# [7,10]: 4 <= 7 → replace 4 with 10 → [10]. len=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort + n heap ops each O(log n).
# Space O(n): heap holds at most n end times.
# Alternative: split starts and ends into sorted arrays, use two pointers — O(n log n), O(n).
# Follow-up: Meeting Rooms I (LC 252) — can ONE person attend all? Sort by start, check no overlaps.
# Follow-up: minimum cost scheduling — extend with room priorities or weighted intervals."
```


Heap

#347 Top K Frequent Elements

MED

Open on LeetCode

Array · Hash Table · Divide and Conquer · Sorting · Heap · Bucket Sort · Counting · Quickselect
Lists: Denny Zhang

Problem

Given an integer array nums and an integer k, return the k most frequent elements. You may return the answer in any order.
Example 1:
Input: nums = [1,1,1,2,2,3], k = 2
Output: [1,2]
Example 2:
Input: nums = [1], k = 1
Output: [1]
Example 3:
Input: nums = [1,2,1,2,1,2,3,1,3,2], k = 2
Output: [1,2]
Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104
- k is in the range [1, the number of unique elements in the array].
- It is guaranteed that the answer is unique.

Follow up: Your algorithm's time complexity must be better than O(n log n), where n is the array's size.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an integer array, return the k most frequent elements in any order.
# Guaranteed that k is valid and the answer is unique. Right?"
#
# "nums=[1,1,1,2,2,3], k=2: frequencies: 1→3, 2→2, 3→1 → [1,2] ✓
# nums=[1], k=1 → [1] ✓"

# PHASE 2 — BRUTE FORCE
# "Count frequencies with a dict, sort by frequency descending, take top k.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Min-heap of size k: maintain k most frequent elements.
# Push (count, num); when heap exceeds k, pop the least frequent.
# Heap root is the kth most frequent — pop it last.
# Time: O(n log k). Space: O(n) counter + O(k) heap.
#
# Bucket sort alternative: O(n). Array of n+1 buckets indexed by frequency.
# Each bucket holds numbers with that frequency. Scan from highest to lowest."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,1,1,2,2,3], k=2:
# freq={1:3,2:2,3:1}. nlargest(2, keys, key=freq):
# heap keeps top 2 by freq → [1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Heap O(n log k): best when k << n. Sort O(n log n). Bucket sort O(n): best overall.
# Bucket sort: buckets[freq].append(num); scan from n down, collect until k elements.
# Follow-up: Top K Frequent Words (LC 692) — same logic but break frequency ties alphabetically.
# Follow-up: streaming top-k — maintain a min-heap of size k as elements arrive."
```

Heap

⋆ #505 The Maze II ⋆

MED

Open on LeetCode

Array · DFS · BFS · Graph · Matrix · Heap · Shortest Path

Lists: ⋆ Premium | Premium 98

Problem

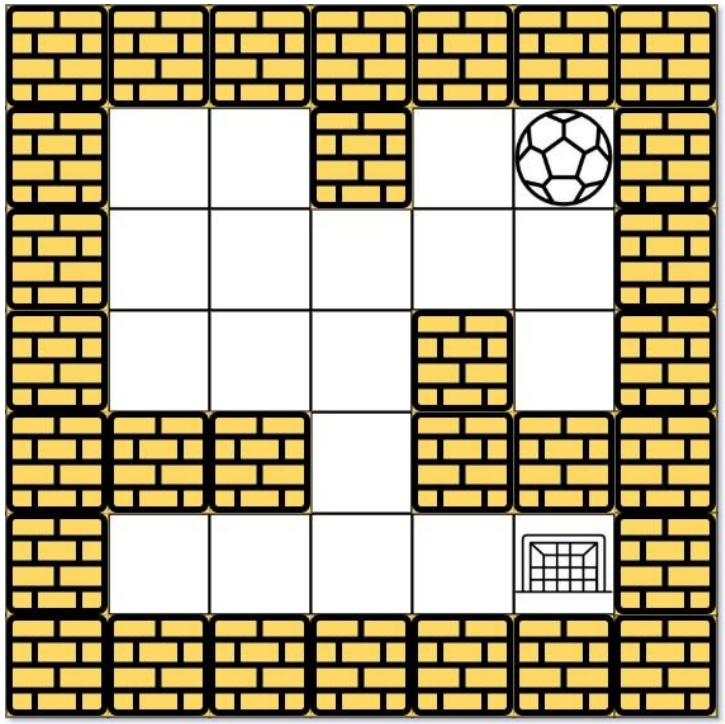
There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction.

Given the m x n maze, the ball's start position and the destination, where start = [startrow, startcol] and destination = [destinationrow, destinationcol], return the shortest distance for the ball to stop at the destination. If the ball cannot stop at destination, return -1.

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

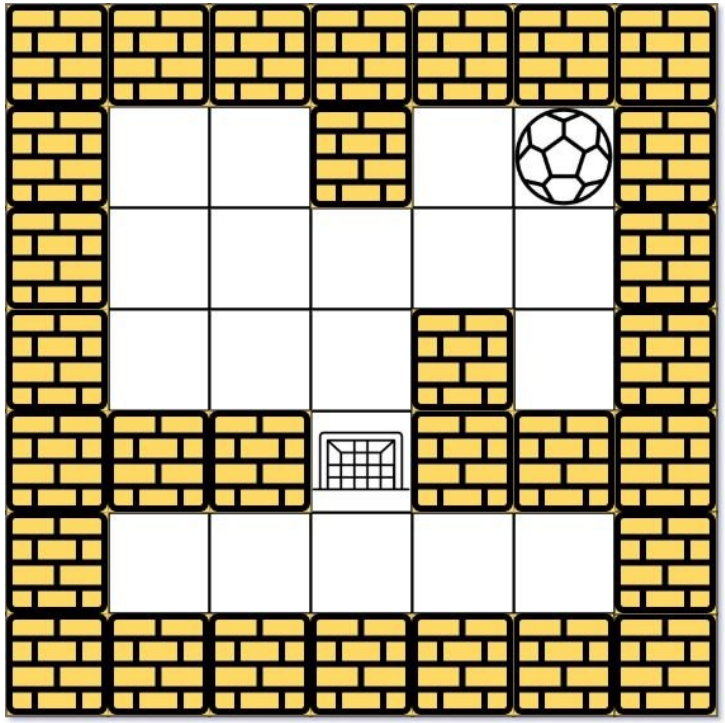
You may assume that the borders of the maze are all walls (see examples).

Example 1:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [4,4]
Output: 12
Explanation: One possible way is : left -> down -> left -> down -> right -> down -> right.
The length of the path is 1 + 1 + 3 + 1 + 2 + 2 + 2 = 12.
```

Example 2:



```
Input: maze = [[0,0,1,0,0],[0,0,0,0,0],[0,0,0,1,0],[1,1,0,1,1],[0,0,0,0,0]], start = [0,4], destination = [3,2]
Output: -1
Explanation: There is no way for the ball to stop at the destination. Notice that you can pass through the destination but you cannot stop there.
```

Example 3:

```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], start = [4,3], destination = [0,1]
Output: -1
```

Constraints:

- m == maze.length
- n == maze[i].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- start.length == 2
- destination.length == 2
- 0 <= startrow, destinationrow < m
- 0 <= startcol, destinationcol < n
- Both the ball and the destination exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A ball rolls in a maze until it hits a wall. Find the SHORTEST DISTANCE from start to
# destination (number of steps). Return -1 if impossible. Right?"
#
# "maze 5x5, start=(0,4), dest=(4,4): ball rolls left/right/up/down until wall.
# Shortest path = 12 steps ✓"
# PHASE 2 — BRUTE FORCE
# "BFS exploring all rolling directions from each stop point.
# Track visited cells, return distance when destination is reached.
# Time: O(m*n*(m+n)) — at each cell, rolling can traverse up to max(m,n) cells. Space: O(m*n)."
```

```
# PHASE 3 — OPTIMIZE
# "Dijkstra's algorithm — process stop points in order of distance.
# Min-heap of (distance, row, col). For each popped cell, roll in all four directions.
# Stop when destination is popped (guaranteed shortest path at that point).
# Time: O(m*n*log(m*n)*(m+n)). Space: O(m*n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# start=(0,4),dest=(4,4): Dijkstra explores paths in order of distance.
# Ball at (0,4) rolls down → hits wall at (4,4) if path clear, steps=4, or hits earlier.
# Different directions explored; heap ensures shortest extracted first → returns 12 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*(m+n)*log(m*n)): heap has O(m*n) entries, rolling O(m+n) per direction.
# Space O(m*n): dist array + heap.
# BFS gives any shortest path; Dijkstra gives minimum distance (same for unit weights,
# needed here because rolling distances vary).
# Follow-up: Maze I (LC 490) — just reachability, BFS suffices.
# Follow-up: Maze III (LC 499) — find lexicographically smallest direction sequence."
```


Heap

#621 Task Scheduler

ME
D

Open on LeetCode

Array · Hash Table · Greedy · Heap · Counting

Lists: Grind 169

Problem

You are given an array of CPU tasks, each labeled with a letter from A to Z, and a number n. Each CPU interval can be idle or allow the completion of one task. Tasks can be completed in any order, but there's a constraint: there has to be a gap of at least n intervals between two tasks with the same label.

Return the minimum number of CPU intervals required to complete all tasks.

Example 1:

Input: tasks = ["A","A","A","B","B","B"], n = 2

Output: 8

Explanation: A possible sequence is: A -> B -> idle -> A -> B -> idle -> A -> B.

After completing task A, you must wait two intervals before doing A again. The same applies to task B. In the 3rd interval, neither A nor B can be done, so you idle. By the 4th interval, you can do A again as 2 intervals have passed.

Example 2:

Input: tasks = ["A","C","A","B","D","B"], n = 1

Output: 6

Explanation: A possible sequence is: A -> B -> C -> D -> A -> B.

With a cooling interval of 1, you can repeat a task after just one other task.

Example 3:

Input: tasks = ["A","A","A", "B","B","B"], n = 3

Output: 10

Explanation: A possible sequence is: A -> B -> idle -> idle -> A -> B -> idle -> idle -> A -> B.

There are only two types of tasks, A and B, which need to be separated by 3 intervals. This leads to idling twice between repetitions of these tasks.

Constraints:

- 1 <= tasks.length <= 104
- tasks[i] is an uppercase English letter.
- 0 <= n <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given tasks (letters) and cooldown n: same task must have at least n slots between runs.
# CPU can idle. Return the minimum number of intervals to finish all tasks. Right?"
#
# "tasks=['A','A','A','B','B','B'], n=2:
# A→B→idle→A→B→idle→A→B = 8 intervals ✓
# tasks=['A','A','A','B','B','B'], n=0: AAABBB = 6 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a max-heap of (count, task). Each cycle of (n+1) slots: pop up to n+1 most frequent
# tasks, execute them, decrement counts, push back non-zero counts.
# If fewer than n+1 tasks available, pad with idles.
# Time: O(m * n) where m = total tasks. Space: O(26)."

# PHASE 3 — OPTIMIZE
# "Math formula: the most frequent task determines the frame count.
# Let max_freq = highest frequency. Let max_count = number of tasks with max_freq.
# Formula: max(len(tasks), (max_freq - 1) * (n + 1) + max_count).
# The formula gives the minimum slots; if tasks fill all slots, no idle needed.
# Time: O(m). Space: O(26)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tasks=['A'*3,'B'*3], n=2: max_freq=3, max_count=2.
# (3-1)*(2+1)+2 = 2*3+2 = 8. len(tasks)=6. max(6,8)=8 ✓
#
# tasks=['A'*3,'B'*3], n=0: (3-1)*1+2=4. len=6. max(6,4)=6 ✓
#
# tasks=['A'*3,'B'*3,'C'*3], n=2: max_freq=3,max_count=3.
# (3-1)*3+3=9. len=9. max(9,9)=9 ✓ (ABCABCABC, no idles needed)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Formula O(m): just count frequencies. Heap simulation O(m*n): slower but more intuitive.
# The formula works because tasks beyond the 'frame' always fit without extra idles.
# Follow-up: return the actual schedule (not just count) — use the heap simulation.
# Follow-up: if tasks have deadlines, this becomes a more complex scheduling problem."
```

Heap

#658 Find K Closest Elements

MED

Open on LeetCode

Array · Binary Search · Sorting · Heap · Two Pointers

Lists: Grind 169

Problem

Given a sorted integer array arr, two integers k and x, return the k closest integers to x in the array. The result should also be sorted in ascending order.

An integer a is closer to x than an integer b if:

- |a − x| < |b − x|, or
- |a − x| == |b − x| and a < b

Example 1:

Input: arr = [1,2,3,4,5], k = 4, x = 3

Output: [1,2,3,4]

Example 2:

Input: arr = [1,1,2,3,4,5], k = 4, x = −1

Output: [1,1,2,3]

Constraints:

- 1 <= k <= arr.length
- 1 <= arr.length <= 104
- arr is sorted in ascending order.
- −104 <= arr[i], x <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a sorted array and integers k and x, return the k closest elements to x.
# Ties broken by smaller value. Return sorted. Right?"
#
# "arr=[1,2,3,4,5], k=4, x=3 → [1,2,3,4] ✓
# arr=[1,2,3,4,5], k=4, x=−1 → [1,2,3,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Sort all elements by |val−x| (then by val for ties), take first k, return sorted.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Binary search on the left boundary of the k−element window.
# The answer is always a contiguous subarray of length k.
# Binary search for left in [0, n−k]: compare arr[mid] and arr[mid+k] distances to x.
# If arr[mid+k]−x < x−arr[mid], the window should shift right (left = mid+1).
# Otherwise left = mid. Final window: arr[left:left+k].
# Time: O(log(n−k) + k). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# arr=[1,2,3,4,5],k=4,x=3: lo=0,hi=1.
# mid=0: x−arr[0]=3−1=2, arr[4]−x=5−3=2. 2>2 is False → hi=0.
# lo=hi=0. return arr[0:4]=[1,2,3,4] ✓
#
# arr=[1,2,3,4,5],k=4,x=−1: lo=0,hi=1.
# mid=0: x−arr[0]=−1−1=−2 (but we compare abs): −1−1=−2 < 0 so condition is −2>arr[4]−(−1)=6? No.
# Actually x−arr[mid] = −1−1=−2; arr[mid+k]−x=5−(−1)=6. −2>6 False → hi=0. return [1,2,3,4] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Binary search O(log(n−k)): find the left boundary. Slice O(k).
# Total O(log(n−k)+k) vs brute O(n log n).
# The key insight: the answer window shifts right when right boundary is closer to x than left.
# Follow-up: if array is unsorted, use a heap: O(n log k).
# Follow-up: online stream of queries with same array — binary search still O(log n) per query."
```


Heap

#787 Cheapest Flights Within K Stops

MED

Open on LeetCode

Dynamic Programming · DFS · BFS · Graph · Heap

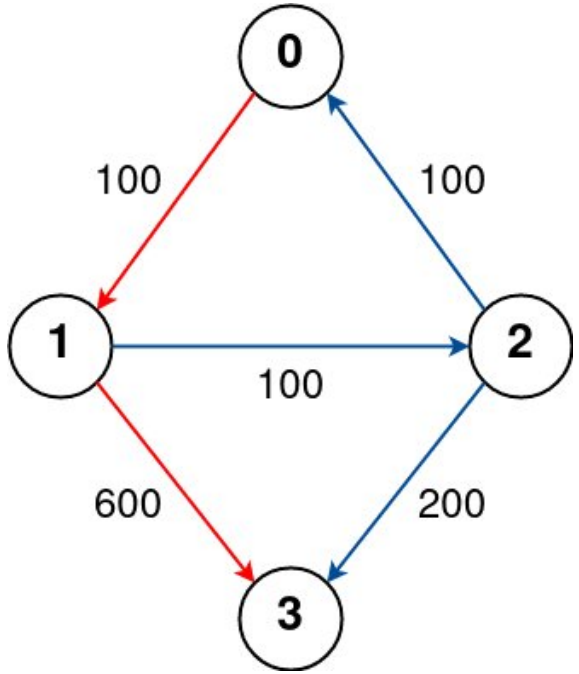
Lists: Grind 169

Problem

There are n cities connected by some number of flights. You are given an array flights where flights[i] = [fromi, toi, pricei] indicates that there is a flight from city fromi to city toi with cost pricei.

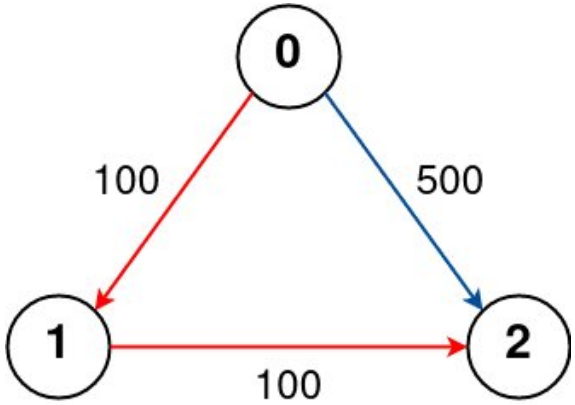
You are also given three integers src, dst, and k, return the cheapest price from src to dst with at most k stops. If there is no such route, return −1.

Example 1:



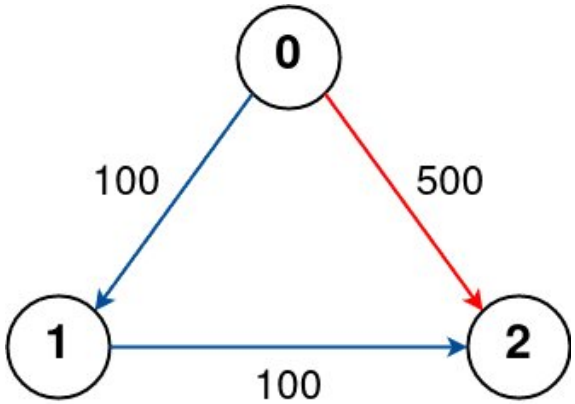

```
Input: n = 4, flights = [[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src = 0, dst = 3, k = 1
Output: 700
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 3 is marked in red and has cost 100 + 600 = 700.
Note that the path through cities [0,1,2,3] is cheaper but is invalid because it uses 2 stops.
```

Example 2:




```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 1
Output: 200
Explanation:
The graph is shown above.
The optimal path with at most 1 stop from city 0 to 2 is marked in red and has cost 100 + 100 = 200.
```

Example 3:




```
Input: n = 3, flights = [[0,1,100],[1,2,100],[0,2,500]], src = 0, dst = 2, k = 0
Output: 500
Explanation:
The graph is shown above.
The optimal path with no stops from city 0 to 2 is marked in red and has cost 500.
```

Constraints:

- 2 <= n <= 100
- 0 <= flights.length <= (n * (n - 1) / 2)
- flights[i].length == 3
- 0 <= fromi, toi < n
- fromi != toi
- 1 <= pricei <= 104
- There will not be any multiple flights between two cities.
- 0 <= src, dst, k < n
- src != dst

◆ Interview Approach · STAR-LC


```
# PHASE 1 — CLARIFY
# "n cities, list of flights [from,to,price]. Find cheapest price from src to dst
# with at most k stops. -1 if impossible. Right?"
#
# "n=4, flights=[[0,1,100],[1,2,100],[2,0,100],[1,3,600],[2,3,200]], src=0, dst=3, k=1:
# 0→1→3 costs 700 (1 stop ✓). 0→1→2→3 costs 400 but needs 2 stops > k=1. → 700 ✓"

# PHASE 2 — BRUTE FORCE
# "BFS/DFS exploring all paths up to k+1 edges. Track minimum cost to reach dst.
# Time: O(n^k) worst case. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Bellman-Ford with k+1 relaxation rounds — DP approach.
# prices[v] = min cost to reach v using at most i edges.
# Each round: update prices based on last round's values (use a copy to avoid using same-round updates).
# Time: O(k * |flights|). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=4, flights as above, src=0, dst=3, k=1:
# Init: prices=[0,INF,INF,INF].
# Round1 (1 edge): 0→1:100→tmp[1]=100. 1→3:600→tmp[3]=INF(prices[1]=INF). 0→1→ done.
# tmp=[0,100,INF,INF]. prices=[0,100,INF,INF].
# Round2 (2 edges=k+1): 1→2:100→tmp[2]=200. 1→3:600→tmp[3]=700. 2→3:200→prices[2]=INF skip.
# prices=[0,100,200,700]. return 700 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(k * E): k+1 Bellman-Ford rounds over all edges.
# Space O(n): two price arrays (current and temp).
# Dijkstra with (cost, stops, node) heap also works — O((E+n) log n) but needs careful stop tracking.
# Follow-up: no stop limit — standard Dijkstra or Bellman-Ford.
# Follow-up: if edge weights can be negative — only Bellman-Ford handles this correctly."
```

Heap

#973 K Closest Points to Origin

MED

Open on LeetCode

Array · Math · Divide and Conquer · Sorting · Heap

Lists: Grind 169

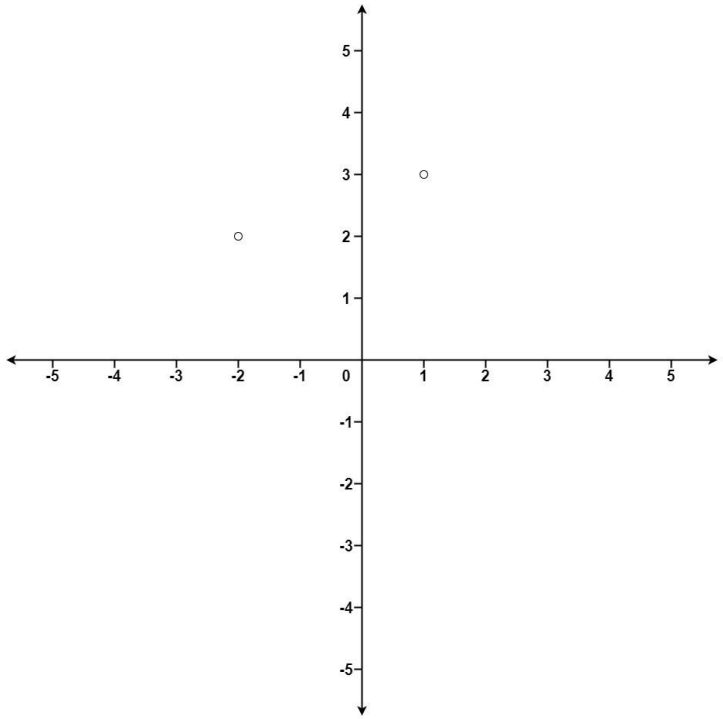
Problem

Given an array of points where points[i] = [xi, yi] represents a point on the X-Y plane and an integer k, return the k closest points to the origin (0, 0).

The distance between two points on the X-Y plane is the Euclidean distance (i.e., √(x1 - x2)2 + (y1 - y2)2).

You may return the answer in any order. The answer is guaranteed to be unique (except for the order that it is in).

Example 1:




```
Input: points = [[1,3],[-2,2]], k = 1
Output: [[-2,2]]
Explanation:
The distance between (1, 3) and the origin is sqrt(10).
The distance between (-2, 2) and the origin is sqrt(8).
Since sqrt(8) < sqrt(10), (-2, 2) is closer to the origin.
We only want the closest k = 1 points from the origin, so the answer is just [[-2,2]].
```

Example 2:

```
Input: points = [[3,3],[5,-1],[-2,4]], k = 2
Output: [[3,3],[-2,4]]
Explanation: The answer [[-2,4],[3,3]] would also be accepted.
```

Constraints:

- 1 <= k <= points.length <= 104
- -104 <= xi, yi <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given an array of points, return the k closest to the origin (0,0).
# Distance = sqrt(x²+y²) (no need to actually sqrt since we compare squares).
# Answer can be in any order. Right?"
#
# "points=[[1,3],[-2,2]], k=1: dist²=[10,8] → [[-2,2]] ✓
# points=[[3,3],[5,-1],[-2,4]], k=2: dist²=[18,26,20] → [[3,3],[-2,4]] ✓"

# PHASE 2 — BRUTE FORCE
# "Sort all points by squared distance, return first k.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Max-heap of size k: maintain k closest points.
# Push (-dist², x, y). When heap exceeds k, pop the farthest.
# Time: O(n log k). Space: O(k). Better than O(n log n) when k << n.
# Quickselect alternative: O(n) average, O(n²) worst — partitions around a pivot distance."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# points=[[1,3],[-2,2]],k=1:
# push (-10,1,3)→[(-10,1,3)]. len=1≤1.
# push (-8,-2,2)→[(-10,1,3),(-8,-2,2)]. len=2>1 → pop(-10,1,3).
# heap=[(-8,-2,2)]. return [[-2,2]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Max-heap O(n log k): push all n, heap never exceeds k.
# Sort O(n log n): simple but slower for small k.
# Quickselect O(n) average: optimal for single queries on large n.
# Follow-up: if points stream in, maintain a max-heap of size k updating on each arrival.
# Follow-up: K closest to a point other than origin — same logic, shift coordinates."
```

Heap

★ #1057 Campus Bikes ★

MED

Open on LeetCode

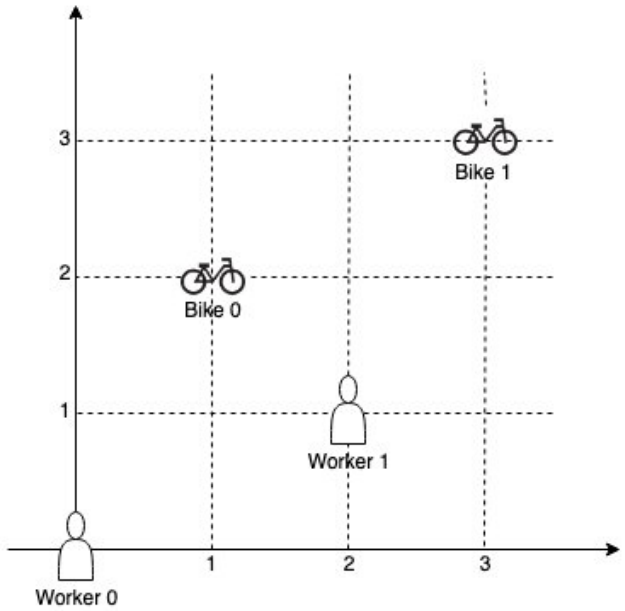
Array · Greedy · Sorting · Heap

Lists: ★ Premium | Premium 98

Problem

On a campus represented on the X-Y plane, there are n workers and m bikes, with n <= m.
You are given an array workers of length n where workers[i] = [xi, yi] is the position of the ith worker. You are also given an array bikes of length m where bikes[j] = [xj, yj] is the position of the jth bike. All the given positions are unique.
Assign a bike to each worker. Among the available bikes and workers, we choose the (workeri, bikej) pair with the shortest Manhattan distance between each other and assign the bike to that worker.
If there are multiple (workeri, bikej) pairs with the same shortest Manhattan distance, we choose the pair with the smallest worker index. If there are multiple ways to do that, we choose the pair with the smallest bike index. Repeat this process until there are no available workers.
Return an array answer of length n, where answer[i] is the index (0-indexed) of the bike that the ith worker is assigned to.
The Manhattan distance between two points p1 and p2 is Manhattan(p1, p2) = |p1.x - p2.x| + |p1.y - p2.y|.

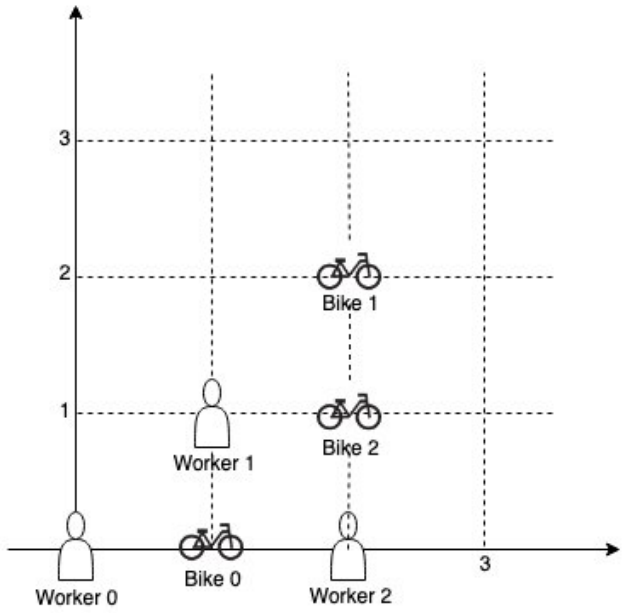
Example 1:




```
Input: workers = [[0,0],[2,1]], bikes = [[1,2],[3,3]]
Output: [1,0]
Explanation: Worker 1 grabs Bike 0 as they are closest (without ties), and Worker 0 is assigned Bike 1. So the output is [1, 0].
```

Example 2:

· #973 | #1167 · | · Contents




```
Input: workers = [[0,0],[1,1],[2,0]], bikes = [[1,0],[2,2],[2,1]]
Output: [0,2,1]
Explanation: Worker 0 grabs Bike 0 at first. Worker 1 and Worker 2 share the same distance to Bike 2, thus Worker 1 is assigned to Bike 2, and Worker 2 will take Bike 1. So the output is [0,2,1].
```

Constraints:

- n == workers.length
- m == bikes.length
- 1 <= n <= m <= 1000
- workers[i].length == bikes[j].length == 2
- 0 <= xi, yi < 1000
- 0 <= xj, yj < 1000
- All worker and bike locations are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n workers and m bikes on a 2D campus. Assign each worker exactly one bike (no sharing).
# Process all (worker,bike) pairs in order of Manhattan distance (ties: smaller worker index,
# then smaller bike index). Return bike assignment array for each worker. Right?"
#
# "workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:
# distances: w0b0=3,w0b1=6,w1b0=2,w1b1=2.
# sorted: (2,w1,b0),(2,w1,b1),(3,w0,b0),(6,w0,b1).
# assign w1→b0, then w0→b1. → [1,0] ✓"

# PHASE 2 — BRUTE FORCE
# "Generate all (dist, worker, bike) triples, sort them, then greedily assign.
# Time: O(n*m log(n*m)). Space: O(n*m)."

# PHASE 3 — OPTIMIZE
# "Same greedy with a min-heap — identical complexity but avoids sorting all pairs upfront.
# Push all pairs initially; pop min until all workers assigned.
# Or bucket-sort by distance (max Manhattan ≤ 2000) for O(n*m) time."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# workers=[[0,0],[2,1]], bikes=[[1,2],[3,3]]:
# heap: (3,0,0),(6,0,1),(2,1,0),(2,1,1).
# pop (2,1,0): assign w1→b0. pop (2,1,1): w1 used skip. pop (3,0,0): b0 used skip.
# pop (6,0,1): assign w0→b1. result=[1,0] ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*m log(n*m)): heap contains n*m pairs. Space O(n*m).
# Bucket sort by distance: O(n*m) time since distances ≤ 2000.
# Follow-up: Campus Bikes II (LC 1066) — minimize TOTAL distance (optimal assignment).
# Requires DP with bitmask: O(n * 2^m). Much harder — this greedy doesn't work there."
```


Heap

★ #1167 Minimum Cost to Connect Sticks ★

ME D

Open on LeetCode

Array · Greedy · Heap

Lists: ★ Premium | Premium 98

Problem

You have some number of sticks with positive integer lengths. These lengths are given as an array sticks, where sticks[i] is the length of the ith stick.

You can connect any two sticks of lengths x and y into one stick by paying a cost of x + y. You must connect all the sticks until there is only one stick remaining.

Return the minimum cost of connecting all the given sticks into one stick in this way.

Example 1:

```
Input: sticks = [2,4,3]
Output: 14
Explanation: You start with sticks = [2,4,3].
1. Combine sticks 2 and 3 for a cost of 2 + 3 = 5. Now you have sticks = [5,4].
2. Combine sticks 5 and 4 for a cost of 5 + 4 = 9. Now you have sticks = [9].
There is only one stick left, so you are done. The total cost is 5 + 9 = 14.
```

Example 2:

```
Input: sticks = [1,8,3,5]
Output: 30
Explanation: You start with sticks = [1,8,3,5].
1. Combine sticks 1 and 3 for a cost of 1 + 3 = 4. Now you have sticks = [4,8,5].
2. Combine sticks 4 and 5 for a cost of 4 + 5 = 9. Now you have sticks = [9,8].
3. Combine sticks 9 and 8 for a cost of 9 + 8 = 17. Now you have sticks = [17].
There is only one stick left, so you are done. The total cost is 4 + 9 + 17 = 30.
```

Example 3:

```
Input: sticks = [5]
Output: 0
Explanation: There is only one stick, so you don't need to do anything. The total cost is 0.
```

Constraints:

- **1 <= sticks.length <= 104**
- **1 <= sticks[i] <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given sticks of various lengths, combine any two into one with cost = sum of their lengths.
# Return the minimum total cost to combine all sticks into one. Right?"
#
# "sticks=[2,4,3]: combine 2+3=5(cost5). Then 4+5=9(cost9). Total=14 ✓
# vs 2+4=6(cost6). 3+6=9(cost9). Total=15. So min is 14 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pair orderings recursively, track minimum cost.
# Time: O(n! * n). Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "Greedy: always combine the two SMALLEST sticks — Huffman coding.
# Use a min-heap. Pop two smallest, add their sum as cost, push sum back.
# Repeat until one stick remains.
# Time: O(n log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# sticks=[2,4,3]: heap=[2,3,4].
# pop 2,3 → combined=5, cost=5. heap=[4,5].
# pop 4,5 → combined=9, cost=14. heap=[9]. return 14 ✓
#
# sticks=[1,8,3,5]: heap=[1,3,5,8].
# pop 1,3→4, cost=4. heap=[4,5,8].
# pop 4,5→9, cost=13. heap=[8,9].
# pop 8,9→17, cost=30. return 30.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): heapify O(n) + n-1 pops/pushes each O(log n).
# Space O(n): the heap.
# This is exactly Huffman encoding — greedy optimality proven by exchange argument.
# Follow-up: Minimum Cost to Merge Stones (LC 1000) — merge k adjacent groups; needs DP.
# Follow-up: if we could only combine adjacent sticks, greedy no longer works."
```

Heap

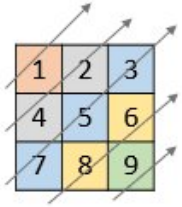
#1424 Diagonal Traverse II

MED

Open on LeetCode

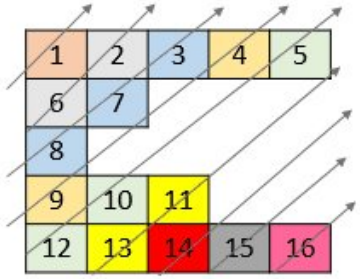
Array · Sorting · Heap
Lists: CodeSignal

Problem
Given a 2D integer array nums, return all elements of nums in diagonal order as shown in the below images.
Example 1:




```
Input: nums = [[1,2,3],[4,5,6],[7,8,9]]
Output: [1,4,2,7,5,3,8,6,9]
```

Example 2:




```
Input: nums = [[1,2,3,4,5],[6,7],[8],[9,10,11],[12,13,14,15,16]]
Output: [1,6,2,8,7,3,9,4,12,10,5,13,11,14,15,16]
```

Constraints:

- **1 <= nums.length <= 105**
- **1 <= nums[i].length <= 105**
- **1 <= sum(nums[i].length) <= 105**
- **1 <= nums[i][j] <= 105**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a 2D list of integers (rows may have different lengths), return all elements
# in diagonal order: elements on the same anti-diagonal (i+j = const) grouped together,
# within each diagonal from bottom to top (larger i first). Right?"
#
# "nums=[[1,2,3],[4,5,6],[7,8,9]]:
# diag0(i+j=0):[1]. diag1:[4,2]. diag2:[7,5,3]. diag3:[8,6]. diag4:[9].
# → [1,4,2,7,5,3,8,6,9] ✓"

# PHASE 2 — BRUTE FORCE
# "Build a dict of diagonal_index → list of values (appended bottom to top).
# Diagonal index = i+j. Iterate rows in reverse to get bottom-to-top order.
# Time: O(n) where n = total elements. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same dict approach but we can avoid sorting by noting diagonal index = i+j
# naturally increases as we scan top-to-bottom, right-to-left.
# Build diags iterating rows top-to-bottom but prepend (or use reverse) per diagonal.
```



```
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[[1,2,3],[4,5,6],[7,8,9]]:
# diags: 0:[1], 1:[2,4], 2:[3,5,7], 3:[6,8], 4:[9].
# reversed: [1],[4,2],[7,5,3],[8,6],[9] → [1,4,2,7,5,3,8,6,9] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass to build diags, one pass to collect — n total elements.
# Space O(n): the diags dictionary stores all elements.
# Follow-up: Diagonal Traverse I (LC 498) — rectangular matrix, alternating direction per diagonal.
# Follow-up: spiral order (LC 54) vs diagonal order are both reordering problems."
```

Trie

Round 5 · Mid · Medium · 7 questions

Trie

#139 Word Break

MED

Open on LeetCode

Array · Hash Table · String · Dynamic Programming · Trie · Memoization

Lists: Grind 169

Problem

Given a string s and a dictionary of strings wordDict, return true if s can be segmented into a space-separated sequence of one or more dictionary words.

Note that the same word in the dictionary may be reused multiple times in the segmentation.

Example 1:

```
Input: s = "leetcode", wordDict = ["leet","code"]
Output: true
Explanation: Return true because "leetcode" can be segmented as "leet code".
```

Example 2:

```
Input: s = "applepenapple", wordDict = ["apple","pen"]
Output: true
Explanation: Return true because "applepenapple" can be segmented as "apple pen apple".
Note that you are allowed to reuse a dictionary word.
```

Example 3:

```
Input: s = "catsandog", wordDict = ["cats","dog","sand","and","cat"]
Output: false
```

Constraints:

- 1 <= s.length <= 300
- 1 <= wordDict.length <= 1000
- 1 <= wordDict[i].length <= 20
- s and wordDict[i] consist of only lowercase English letters.
- All the strings of wordDict are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string s and a dictionary of words, return true if s can be segmented
# into a space-separated sequence of dictionary words. Words can be reused. Right?"
#
# "s='leetcode', wordDict=['leet','code']: 'leet'+'code' → true ✓
# s='applepenapple', wordDict=['apple','pen']: 'apple'+'pen'+'apple' → true ✓
# s='catsandog', wordDict=['cats','dog','sand','and','cat']: no valid split → false ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive: try every prefix of s. If prefix is in dict, recurse on the suffix.
# Exponential without memoization — O(2^n) time, O(n) space."

# PHASE 3 — OPTIMIZE
# "DP: dp[i] = true if s[:i] can be segmented.
# Base: dp[0] = True (empty string).
# Transition: dp[i] = any(dp[j] and s[j:i] in wordSet) for 0 <= j < i.
# Time: O(n²) or O(n * max_word_len). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='leetcode', words={'leet','code'}:
# dp[0]=T. dp[4]: j=0,s[0:4]='leet'∈words,dp[0]=T→dp[4]=T.
# dp[8]: j=4,s[4:8]='code'∈words,dp[4]=T→dp[8]=T. return True ✓
#
# s='catsandog': dp[3]=T('cat'),dp[4]=T('cats'). dp[7]=T('sand' from 3).
# No j makes dp[9]=T → return False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*max_word_len): optimised by only checking back max_word_len characters.
# Space O(n) for dp array.
# Trie optimisation: insert all words into a trie, walk the trie forward from each dp[j]=True position.
# Follow-up: Word Break II (LC 140) — return all valid sentences.
# Backtrack from dp[n]=True positions to reconstruct paths; store parent indices."
```

Trie

#208 Implement Trie (Prefix Tree) MED

Open on LeetCode

Hash Table · String · Design · Trie

Lists: Grind 169

Problem

A trie (pronounced as "try") or prefix tree is a tree data structure used to efficiently store and retrieve keys in a dataset of strings. There are various applications of this data structure, such as autocomplete and spellchecker.

Implement the Trie class:

- Trie() Initializes the trie object.
- void insert(String word) Inserts the string word into the trie.
- boolean search(String word) Returns true if the string word is in the trie (i.e., was inserted before), and false otherwise.
- boolean startsWith(String prefix) Returns true if there is a previously inserted string word that has the prefix prefix, and false otherwise.

Example 1:

```
Input
["Trie", "insert", "search", "search", "startsWith", "insert", "search"]
[[], ["apple"], ["apple"], ["app"], ["app"], ["app"], ["app"]]
Output
[null, null, true, false, true, null, true]

Explanation
Trie trie = new Trie();
```

Constraints:

- 1 <= word.length, prefix.length <= 2000
- word and prefix consist only of lowercase English letters.
- At most 3 * 104 calls in total will be made to insert, search, and startsWith.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement a Trie with insert(word), search(word) → bool (exact match),
# and startsWith(prefix) → bool. Only lowercase English letters. Right?"
#
# "insert('apple'). search('apple')→true. search('app')→false.
# startsWith('app')→true. insert('app'). search('app')→true ✓"

# PHASE 2 — BRUTE FORCE
# "Use a set for exact search and a sorted list for prefix search.
# Time: insert O(1), search O(1), startsWith O(n * m) — scan all words.
# Space: O(total_chars)."

# PHASE 3 — OPTIMIZE
# "True trie: each node has up to 26 children (one per letter) and an end-of-word flag.
# insert: create nodes along the path. search: walk path, check end flag.
# startsWith: walk path, return True if we reach the end of the prefix.
# Time: O(m) per op where m = word/prefix length. Space: O(total_chars * 26)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# insert("apple"): root→a→p→p→l→e(is_end=True).
# search("apple"): walk a→p→p→l→e, is_end=True → True ✓
# search("app"): walk a→p→p, is_end=False → False ✓
# startsWith("app"): walk a→p→p, exists → True ✓
# insert("app"): node at 'p'(3rd) gets is_end=True.
# search("app"): walk a→p→p, is_end=True → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m) per op: walk at most m nodes. Space O(n*m): n words of avg length m.
# Dict children vs array[26]: dict is memory-efficient for sparse alphabets.
# Follow-up: delete a word from the trie — mark is_end=False, prune childless nodes bottom-up.
# Follow-up: Word Search II (LC 212) — build a trie of all words, DFS the board against it."
```


Trie

#211 Design Add and Search Words Data Structure MED

Open on LeetCode

String · DFS · Design · Trie

Lists: Grind 169

Problem

Design a data structure that supports adding new words and finding if a string matches any previously added string.

Implement the WordDictionary class:

- WordDictionary() Initializes the object.
- void addWord(word) Adds word to the data structure, it can be matched later.
- bool search(word) Returns true if there is any string in the data structure that matches word or false otherwise. word may contain dots '.' where dots can be matched with any letter.

Example:

```
Input
["WordDictionary","addWord","addWord","addWord","search","search","search","search"]
[[],["bad"],["dad"],["mad"],["pad"],["bad"],[".ad"],["b.."]]
Output
[null,null,null,null,false,true,true,true]

Explanation
WordDictionary wordDictionary = new WordDictionary();
```

Constraints:

- 1 <= word.length <= 25
- word in addWord consists of lowercase English letters.
- word in search consist of '.' or lowercase English letters.
- There will be at most 2 dots in word for search queries.
- At most 104 calls will be made to addWord and search.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a data structure with addWord(word) and search(word).
# search supports '.' as a wildcard matching any single letter. Right?"
#
# "addWord('bad'), addWord('dad'), addWord('mad').
# search('pad')→false, search('bad')→true, search('.ad')→true, search('b..')→true ✓"

# PHASE 2 — BRUTE FORCE
# "Store all words in a list. For search, iterate over all words and match
# character by character (treating '.' as wildcard).
# Time: O(n * m) per search where n = number of words, m = word length. Space: O(n*m)."

# PHASE 3 — OPTIMIZE
# "Trie with DFS for wildcards. addWord: standard trie insert.
# search: walk trie; on a '.' node, recursively try ALL children.
# Time: addWord O(m). search O(m) for literals, up to O(26^k * m) worst case with k dots.
# Space: O(n*m) for the trie."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# addWord("bad"): b→a→d(end). addWord("dad"): d→a→d(end). addWord("mad"): m→a→d(end).
# search("pad"): p not in root.children → False ✓
# search("bad"): b→a→d, is_end=True → True ✓
# search(".ad"): '.': try b,d,m → each→a→d, is_end=True → True ✓
# search("b.."): b→'.': try a→'.': try d(end) → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "addWord O(m). search O(m) best case (no dots), O(26^k * m) worst case (k dots).
# In practice dots are rare, so average is close to O(m).
# Follow-up: if patterns are very dot-heavy, cache results at (node, index) pairs.
# Follow-up: support '*' (zero or more chars) — more complex DFS with skip logic."
```

Trie

★ #616 Add Bold Tag in String ★

MED

Open on LeetCode

Array · Hash Table · String · Trie

Lists: ★ Premium | Premium 98

Problem

You are given a string s and an array of strings words.

You should add a closed pair of bold tag and to wrap the substrings in s that exist in words.

- If two such substrings overlap, you should wrap them together with only one pair of closed bold-tag.
- If two substrings wrapped by bold tags are consecutive, you should combine them.

Return s after adding the bold tags.

Example 1:

```
Input: s = "abcxyz123", words = ["abc","123"]
Output: "<b>abc</b>xyz<b>123</b>"
Explanation: The two strings of words are substrings of s as following: "abcxyz123".
We add <b> before each substring and </b> after each substring.
```

Example 2:

```
Input: s = "aaabbb", words = ["aa","b"]
Output: "<b>aaabbb</b>"
Explanation:
"aa" appears as a substring two times: "aaabbb" and "aaabbb".
"b" appears as a substring three times: "aaabbb", "aaabbb", and "aaabbb".
We add <b> before each substring and </b> after each substring: "<b>a<b>a</b>a</b><b>b</b><b>b</b><b>b</b>".
Since the first two <b>'s overlap, we merge them: "<b>aaa</b><b>b</b><b>b</b><b>b</b>".
Since now the four <b>'s are consecutive, we merge them: "<b>aaabbb</b>".
```

Constraints:

- 1 <= s.length <= 1000
- 0 <= words.length <= 100
- 1 <= words[i].length <= 1000
- s and words[i] consist of English letters and digits.
- All the values of words are unique.

Note: This question is the same as 758. Bold Words in String.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string s and list of words, wrap all substrings of s that are in words
# with <b></b> tags. Overlapping or adjacent bold regions should be merged. Right?"
#
# "s='abcxyz123', words=['abc','123']: '<b>abc</b>xyz<b>123</b>' ✓
# s='aaabbcc', words=['aaa','aab','bc']: '<b>aaabbc</b>c' ✓ (overlapping regions merged)"

# PHASE 2 — BRUTE FORCE
# "Mark each character as bold or not. For each word, find all occurrences in s
# and mark those character positions. Then rebuild the string with tags.
# Time: O(n * m * L) where m=words count, L=avg word length. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Build intervals of bold regions by finding all word matches, merge overlapping intervals,
# then reconstruct. Use a boolean array for merging — same O(n*m*L) but cleaner.
# Trie optimization: insert all words, scan s with Trie to find all matches in O(n*L)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aaabbcc', words=['aaa','aab','bc']:
# i=0: 'aaa'✓→end=3,'aab'? s[0:3]='aaa'≠'aab'. bold[0,1,2]=True.
# i=1: 'aab'✓→end=max(3,4)=4. bold[1,2,3]=True.
# i=4: 'bc'✓→end=6. bold[4,5]=True.
# bold=[T,T,T,T,T,T,F]. → '<b>aaabbc</b>c' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * max_L): at each position check all prefix lengths up to max_L.
# Space O(n): bold array + set.
# Trie reduces search to O(n * max_L) but with better constant via prefix pruning.
# Follow-up: Interval List Intersections (LC 986) — another interval merging problem.
# Follow-up: Bold Words in String (LC 758) — identical problem, same solution."
```


Trie

#692 Top K Frequent Words

MED

Open on LeetCode

Hash Table · String · Trie · Sorting · Heap

Lists: Grind 169

Problem

Given an array of strings words and an integer k, return the k most frequent strings.

Return the answer sorted by the frequency from highest to lowest. Sort the words with the same frequency by their lexicographical order.

Example 1:

```
Input: words = ["i","love","leetcode","i","love","coding"], k = 2
Output: ["i","love"]
Explanation: "i" and "love" are the two most frequent words.
Note that "i" comes before "love" due to a lower alphabetical order.
```

Example 2:

```
Input: words = ["the","day","is","sunny","the","the","the","sunny","is","is"], k = 4
Output: ["the","is","sunny","day"]
Explanation: "the", "is", "sunny" and "day" are the four most frequent words, with the number of occurrence being 4, 3, 2 and 1 respectively.
```

Constraints:

- 1 <= words.length <= 500
- 1 <= words[i].length <= 10
- words[i] consists of lowercase English letters.
- k is in the range [1, The number of unique words[i]]

Follow-up: Could you solve it in O(n log(k)) time and O(n) extra space?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of words and k, return the k most frequent words sorted by frequency
# descending. Ties broken alphabetically ascending. Right?"
#
# "words=['i','love','leetcode','i','love','coding'], k=2 → ['i','love'] ✓
# words=['the','day','is','sunny','the','the','the','sunny','is','is'], k=4
# → ['the','is','sunny','day'] ✓"

# PHASE 2 — BRUTE FORCE
# "Count frequencies, sort by (-freq, word), take first k.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Min-heap of size k with custom comparator.
# Store (-freq, word) so the heap evicts the least desirable (lowest freq, alphabetically last).
# Time: O(n log k). Space: O(k) heap + O(n) counter."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# words=['i','love','leetcode','i','love','coding'], k=2:
# freq={'i':2,'love':2,'leetcode':1,'coding':1}.
# heap of size 2: keeps 2 best → ('i',2) and ('love',2).
# sorted by (-freq,word): ('i',2),('love',2) → ['i','love'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Heap O(n log k): optimal when k << n. Sort O(n log n): simpler code.
# The custom comparator handles the tie-breaking (alphabetical) correctly.
# Follow-up: Top K Frequent Elements (LC 347) — integers, no tie-breaking needed.
# Follow-up: streaming top-k words — maintain heap as words arrive one by one."
```

Trie

#720 Longest Word in Dictionary

MED

Open on LeetCode

Array · Hash Table · String · Trie · Sorting

Lists: Denny Zhang

Problem

Given an array of strings words representing an English Dictionary, return the longest word in words that can be built one character at a time by other words in words.

If there is more than one possible answer, return the longest word with the smallest lexicographical order. If there is no answer, return the empty string.

Note that the word should be built from left to right with each additional character being added to the end of a previous word.

Example 1:

```
Input: words = ["w","wo","wor","worl","world"]
Output: "world"
Explanation: The word "world" can be built one character at a time by "w", "wo", "wor", and "worl".
```

Example 2:

```
Input: words = ["a","banana","app","appl","ap","apply","apple"]
Output: "apple"
Explanation: Both "apply" and "apple" can be built from other words in the dictionary. However, "apple" is lexicographically
smaller than "apply".
```

Constraints:

- 1 <= words.length <= 1000
- 1 <= words[i].length <= 30
- words[i] consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of strings, find the longest word that can be built one character at a time
# by other words in the list. All prefixes of the word must also be in the list.
# If tie, return the smallest alphabetically. Right?"
#
# "words=['w','wo','wor','worl','world']: every prefix exists → 'world' ✓
# words=['a','banana','app','appl','ap','apply','apple']:
# 'apply' and 'apple' both have all prefixes → return 'apple' (smaller) ✓"

# PHASE 2 — BRUTE FORCE
# "Put all words in a set. Sort by (length desc, alpha asc). For each word,
# check that all its prefixes are in the set. Return the first match.
# Time: O(n * L) where L = max word length. Space: O(n * L)."

# PHASE 3 — OPTIMIZE
# "Trie approach: insert all words, then BFS/DFS from root visiting only nodes
# where is_end=True. The deepest reachable node gives the longest valid word.
# Time: O(n * L) to build trie + O(n * L) to search. Space: O(n * L)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# words=['w','wo','wor','worl','world']:
# 'world': check 'w'✓,'wo'✓,'wor'✓,'worl'✓ → all prefixes in set. len=5>0 → result='world' ✓
#
# words=['a','banana','app','appl','ap','apply','apple']:
# 'apple': 'a','ap','app','appl' all in set ✓. len=5. result='apple'.
# 'apply': all prefixes in set. len=5=5, 'apply'>'apple' → keep 'apple' ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n * L²) for the set approach (each word checks L prefixes, each O(L) lookup).
# Trie reduces prefix lookup to O(1) per character — O(n * L) overall.
# Follow-up: if words are streamed, maintain a trie and update result on each insert.
# Follow-up: allow at most one missing prefix — extend valid condition in the BFS/DFS."
```


Trie

#1166 Design File System

MED

Open on LeetCode

Hash Table · String · Design · Trie

Lists: Denny Zhang

Problem

You are asked to design a file system that allows you to create new paths and associate them with different values.

The format of a path is one or more concatenated strings of the form: / followed by one or more lowercase English letters. For example, "/leetcode" and "/leetcode/problems" are valid paths while an empty string "" and "/" are not.

Implement the FileSystem class:

- bool createPath(string path, int value) Creates a new path and associates a value to it if possible and returns true. Returns false if the path already exists or its parent path doesn't exist.
- int get(string path) Returns the value associated with path or returns -1 if the path doesn't exist.

Example 1:

```
Input:
["FileSystem","createPath","get"]
[[],["/a",1],["/a"]]
Output:
[null,true,1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Example 2:

```
Input:
["FileSystem","createPath","createPath","get","createPath","get"]
[[],["/leet",1],["/leet/code",2],["/leet/code"],["/c/d",1],["/c"]]
Output:
[null,true,true,2,false,-1]
Explanation:
FileSystem fileSystem = new FileSystem();
```

Constraints:

- 2 <= path.length <= 100
- 1 <= value <= 109
- Each path is valid and consists of lowercase English letters and '/'.
- At most 104 calls in total will be made to createPath and get.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a file system with createPath(path, value)→bool and get(path)→int.
# createPath creates a path with a value — all parent paths must already exist.
# Returns false if path exists or parent doesn't exist. get returns value or -1. Right?"
#
# "createPath('/a',1)→T. createPath('/a/b',2)→T. get('/a')→1. get('/a/b')→2.
# createPath('/a/b',3)→F (already exists). createPath('/c/d',4)→F (no '/c'). ✓"

# PHASE 2 — BRUTE FORCE
# "Use a dict mapping full path string → value.
# createPath: check parent path exists in dict, then insert.
# get: return dict.get(path, -1).
# Time: O(L) per op where L = path length. Space: O(n*L)."

# PHASE 3 — OPTIMIZE
# "Trie where each node maps component names to child nodes and stores a value.
# createPath: split on '/', walk/create nodes, check parent validity.
# get: walk trie, return node value.
# Time: O(L) per op. Space: O(n*L) — same as dict but with natural prefix sharing."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# createPath('/a',1): parts=['a'], root['a']={'_val':1} → True ✓
# createPath('/a/b',2): parts=['a','b']. node=root['a']. 'b' not in node → root['a']['b']={'_val':2} → True ✓
# get('/a'): parts=['a']. node=root['a']. return _val=1 ✓
# createPath('/a/b',3): 'b' in root['a'] → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Both dict and trie are O(L) per operation where L = path length.
# Trie benefits when many paths share prefixes (memory efficiency).
# Follow-up: add a delete(path) — must handle deleting subtrees recursively.
# Follow-up: list all paths under a prefix — trie DFS from that node."
```

Backtracking

Round 5 · Mid · Medium · 6 questions

Backtracking

#17 Letter Combinations of a Phone Number

ME D

Open on LeetCode

Hash Table · String · Backtracking

Lists: Grind 169

Problem

Given a string containing digits from 2-9 inclusive, return all possible letter combinations that the number could represent. Return the answer in any order.

A mapping of digits to letters (just like on the telephone buttons) is given below. Note that 1 does not map to any letters.

Example 1:

```
Input: digits = "23"
Output: ["ad","ae","af","bd","be","bf","cd","ce","cf"]
```

Example 2:

```
Input: digits = "2"
Output: ["a","b","c"]
```

Constraints:

- **1 <= digits.length <= 4**
- **digits[i] is a digit in the range ['2', '9'].**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string of digits (2-9), return all possible letter combinations
# the digits could represent on a phone keypad. Return [] for empty input. Right?"
#
# "digits="23": 2→abc, 3→def → [ad,ae,af,bd,be,bf,cd,ce,cf] ✓
# digits="": return [] ✓
```

```
# digits="2": → [a,b,c] ✓"
# PHASE 2 — BRUTE FORCE
# "Build combinations iteratively using a queue (BFS).
# Start with [''], then for each digit, expand every existing combination
# by appending each letter the digit maps to.
# Time: O(4^n * n) — 4 letters max per digit, n digits. Space: O(4^n * n)."
# PHASE 3 — OPTIMIZE
# "Backtracking builds each combination one character at a time and avoids
# storing all intermediate combinations.
# At each depth level choose one letter for that digit, recurse, then undo.
# Time: O(4^n * n) — same output size, but space is O(n) call stack + O(4^n * n) result."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# digits="23":
# backtrack(0,[]): letters=abc
# path=[a] → backtrack(1,[a]): letters=def
# path=[a,d] → backtrack(2) → append "ad"
# path=[a,e] → append "ae"
# path=[a,f] → append "af"
# path=[b] → "bd","be","bf"
# path=[c] → "cd","ce","cf"
# result=["ad","ae","af","bd","be","bf","cd","ce","cf"] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(4^n * n): at most 4 choices per digit, n digits deep, n to build each string.
# Space O(n) recursion depth + O(4^n * n) output.
# Iterative BFS and backtracking have same output; backtracking uses less intermediate space.
# Follow-up: if digits can include 0/1 (no letters), skip those digits.
# Follow-up: combination count problem — just compute product of len(phone[d]) for each d."
```


Backtracking

#22 Generate Parentheses

ME
D

Open on LeetCode

String · Dynamic Programming · Backtracking

Lists: Grind 169

Problem

Given n pairs of parentheses, write a function to generate all combinations of well-formed parentheses.

Example 1:

```
Input: n = 3
Output: ["((()))","(()())","(())()","()(())","()()()"]
```

Example 2:

```
Input: n = 1
Output: ["()"]
```

Constraints:

- **1 <= n <= 8**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given n pairs of parentheses, generate all combinations of well-formed parentheses.
# Right?"
#
# "n=3: ['((()))','(()())','(())()','()(())','()()()'] — 5 combinations ✓
# n=1: ['()'] ✓
# n=2: ['(())','()()'] ✓"
# PHASE 2 — BRUTE FORCE
# "Generate ALL 2^(2n) sequences of '(' and ')' of length 2n,
# then filter to keep only the valid ones.
# Valid: each prefix has at least as many '(' as ')', and total counts are equal.
# Time: O(2^(2n) * n) — exponential, extremely wasteful. Space: O(2^(2n) * n)."
# PHASE 3 — OPTIMIZE
# "Backtracking with validity pruning — only place characters that keep the sequence valid.
# Track open and close counts. Rules:
# - Add '(' if open < n
# - Add ')' if close < open
# This eliminates all invalid branches early, producing only Catalan(n) valid strings.
# Time: O(4^n / sqrt(n)) — Catalan number. Space: O(n) recursion depth."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# n=2:
# backtrack([],0,0): add '(' → ([[ ],1,0)
# add '(' → ([[(],2,0)
# add ')' → ([(()],2,1)
# add ')' → ([(())],2,2) → append "(())" ✓
# add ')' → ([()] ,1,1)
# add '(' → ([()(],2,1)
# add ')' → ([()()] ,2,2) → append "()()" ✓
# result = ["(())","()()"] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(4^n / sqrt(n)): nth Catalan number of valid sequences, each of length 2n.
# Space O(n): recursion depth is at most 2n, path list is at most 2n characters.
# Brute is O(2^(2n)) — exponentially worse.
# Follow-up: count valid sequences (not generate) — just return Catalan(n) = C(2n,n)/(n+1).
# Follow-up: LC 32 Longest Valid Parentheses — uses a stack or DP instead."
```

Backtracking

#39 Combination Sum

MED

Open on LeetCode

Array · Backtracking

Lists: Grind 169

Problem

Given an array of distinct integers candidates and a target integer target, return a list of all unique combinations of candidates where the chosen numbers sum to target. You may return the combinations in any order.

The same number may be chosen from candidates an unlimited number of times. Two combinations are unique if the frequency of at least one of the chosen numbers is different.

The test cases are generated such that the number of unique combinations that sum up to target is less than 150 combinations for the given input.

Example 1:

```
Input: candidates = [2,3,6,7], target = 7
Output: [[2,2,3],[7]]
Explanation:
2 and 3 are candidates, and 2 + 2 + 3 = 7. Note that 2 can be used multiple times.
7 is a candidate, and 7 = 7.
These are the only two combinations.
```

Example 2:

```
Input: candidates = [2,3,5], target = 8
Output: [[2,2,2,2],[2,3,3],[3,5]]
```

Example 3:

```
Input: candidates = [2], target = 1
Output: []
```

Constraints:

- 1 <= candidates.length <= 30
- 2 <= candidates[i] <= 40
- All elements of candidates are distinct.
- 1 <= target <= 40

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array of distinct positive integers and a target, return all unique
# combinations that sum to target. Each number may be used unlimited times.
# The solution set must not contain duplicate combinations. Right?"
#
# "candidates=[2,3,6,7], target=7:
# [2,2,3] (2+2+3=7), [7] (7=7) ✓
# candidates=[2,3,5], target=8: [2,2,2,2],[2,3,3],[3,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Generate all combinations with repetition up to target sum, filter by sum.
# Equivalent to BFS: start from [], extend by each candidate, keep if sum <= target.
# Time: O(n^(t/min_c)) — exponential. Space: same."

# PHASE 3 — OPTIMIZE
# "The brute approach IS backtracking with pruning — sort first so we can break early.
# The key optimisations over naive BFS:
# - Start index prevents re-use of earlier candidates (avoids duplicates like [3,2] and [2,3])
# - Sort + break early prunes branches where candidate > remaining
# Time: O(n^(t/min_c)): still exponential in worst case but pruned significantly."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# candidates=[2,3,6,7], target=7:
# backtrack(0,[],7): try 2 → backtrack(0,[2],5)
# try 2 → backtrack(0,[2,2],3)
# try 2 → backtrack(0,[2,2,2],1) → try 2>1 break
# try 3 → backtrack(1,[2,2,3],0) → append [2,2,3] ✓
# try 3 → backtrack(1,[2,3],2) → try 3>2 break
# try 3 → backtrack(1,[3],4) → try 3→[3,3]:remaining=1, try 6>1 break
# try 6 → backtrack(2,[6],1) → try 6>1 break
# try 7 → backtrack(3,[7],0) → append [7] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n^(t/min_c)): at most t/min_c levels deep, n choices each level.
# Space O(t/min_c): max recursion depth.
# Follow-up: Combination Sum II (LC 40) — each number used ONCE, has duplicates in input.
# Use i+1 instead of i, and skip duplicate candidates at same level.
# Follow-up: Combination Sum III (LC 216) — exactly k numbers from 1-9 summing to n."
```


Backtracking

#46 Permutations

MED

Open on LeetCode

Array · Backtracking

Lists: Grind 169

Problem

Given an array nums of distinct integers, return all the possible permutations. You can return the answer in any order.

Example 1:

```
Input: nums = [1,2,3]
Output: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]]
```

Example 2:

```
Input: nums = [0,1]
Output: [[0,1],[1,0]]
```

Example 3:

```
Input: nums = [1]
Output: [[1]]
```

Constraints:

- 1 <= nums.length <= 6
- -10 <= nums[i] <= 10
- All the integers of nums are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array of distinct integers, return all possible permutations
# in any order. Right?"
#
# "nums=[1,2,3]: [[1,2,3],[1,3,2],[2,1,3],[2,3,1],[3,1,2],[3,2,1]] — 6 perms ✓
# nums=[0,1]: [[0,1],[1,0]] ✓
# nums=[1]: [[1]] ✓"

# PHASE 2 — BRUTE FORCE
# "Use a visited array. At each position try every unvisited number.
# When path length equals n, record the permutation.
# Time: O(n! * n) — n! permutations each of length n. Space: O(n) recursion depth."

# PHASE 3 — OPTIMIZE
# "Swap-based in-place backtracking — no visited array needed.
# Maintain a 'start' index. Everything before start is already placed.
# For each position from start onward, swap it into the start slot, recurse, then swap back.
# Time: O(n! * n). Space: O(n) recursion depth — avoids the extra visited array."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3]:
# backtrack(0): swap(0,0)→[1,2,3], backtrack(1):
# swap(1,1)→[1,2,3], backtrack(2):
# swap(2,2)→[1,2,3], backtrack(3) → append [1,2,3] ✓
# swap(1,2)→[1,3,2], backtrack(2) → append [1,3,2] ✓, swap back→[1,2,3]
# swap(0,1)→[2,1,3], backtrack(1) → [2,1,3],[2,3,1] ✓, swap back→[1,2,3]
# swap(0,2)→[3,2,1], backtrack(1) → [3,2,1],[3,1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n! * n): n! permutations, each takes O(n) to copy.
# Space O(n): recursion depth n, no visited array (swap approach).
# Visited-array approach uses O(n) extra space for the boolean array.
# Follow-up: Permutations II (LC 47) — input has duplicates. Sort first, skip duplicate
# values at the same recursion level to avoid identical permutations.
# Follow-up: next permutation (LC 31) — generate just the next one in lexicographic order."
```

Backtracking

#79 Word Search

MED

Open on LeetCode

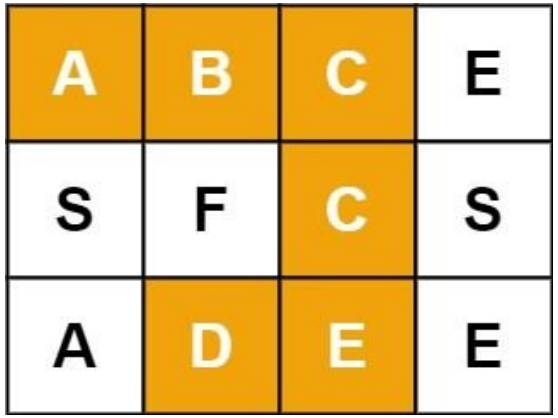
Array · String · Backtracking · Matrix

Lists: Grind 169

Problem

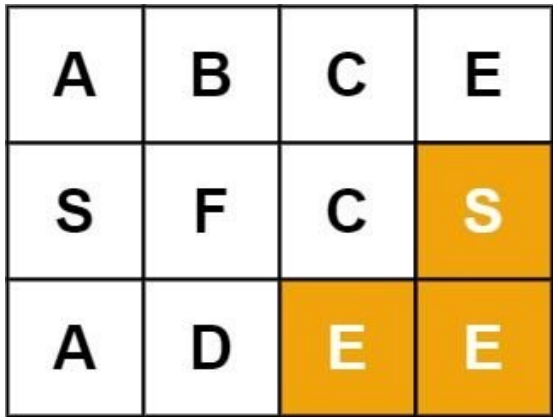
Given an m x n grid of characters board and a string word, return true if word exists in the grid.
The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once.

Example 1:



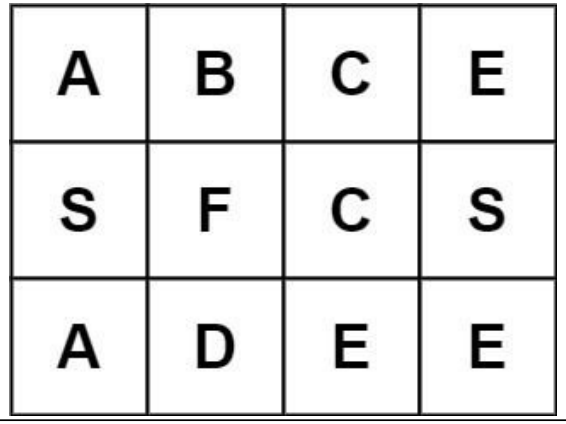

```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCCED"
Output: true
```

Example 2:




```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "SEE"
Output: true
```

Example 3:




```
Input: board = [["A","B","C","E"],["S","F","C","S"],["A","D","E","E"]], word = "ABCB"
Output: false
```

Constraints:

- m == board.length
- n = board[i].length
- 1 <= m, n <= 6
- 1 <= word.length <= 15
- board and word consists of only lowercase and uppercase English letters.

Follow up: Could you use search pruning to make your solution faster with a larger board?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an m×n grid of characters and a word, return true if the word exists
# in the grid following adjacent (up/down/left/right) cells without reusing cells. Right?"
#
# "board=[['A','B','C','E'],['S','F','C','S'],['A','D','E','E']], word='ABCCED':
# A(0,0)→B(0,1)→C(0,2)→C(1,2)→E(2,2)→D(2,1) ✓
# word='ABCB': A→B→C→B would revisit B(0,1) → false ✓"

# PHASE 2 — BRUTE FORCE
# "For every cell as potential start, try every possible path with DFS.
# Mark visited cells with a sentinel, unmark on backtrack.
# Time: O(m*n*4^L) where L = word length. Space: O(L) recursion depth."

# PHASE 3 — OPTIMIZE
# "Same DFS with two pruning improvements:
# 1. Before DFS, check character frequency — if board has fewer of any letter than word needs, return False early.
# 2. If word's last char is rarer in board than first char, reverse the word to start from the rarer end.
# These prune the search space significantly for large grids.
# Time: O(m*n*4^L) worst case but much faster in practice."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# board=[['A','B','C','E'],...], word='ABCCED':
# Start dfs(0,0,'A'): match→mark '#'. dfs(0,1,'B'): match→mark. dfs(0,2,'C'): match.
# dfs(1,2,'C'): match. dfs(2,2,'E'): match. dfs(2,1,'D'): match. idx=6=len → True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*4^L): each cell can start a DFS branching 4 ways up to L levels deep.
# Space O(L): recursion stack depth bounded by word length.
# Follow-up: Word Search II (LC 212) — find ALL words from a list in the board.
# Build a Trie of all words, DFS the board once — O(m*n*4^L + sum of word lengths)."
```

Backtracking

#113 Path Sum II

MED

Open on LeetCode

Backtracking · Tree · DFS

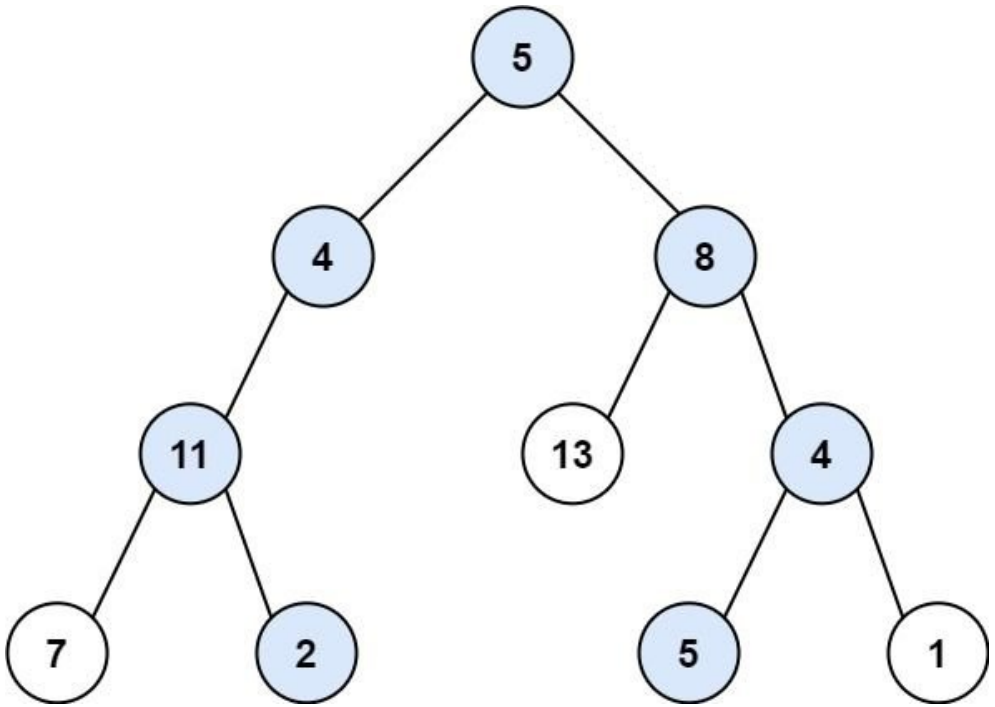
Lists: Grind 169

Problem

Given the root of a binary tree and an integer targetSum, return all root-to-leaf paths where the sum of the node values in the path equals targetSum. Each path should be returned as a list of the node values, not node references.

A root-to-leaf path is a path starting from the root and ending at any leaf node. A leaf is a node with no children.

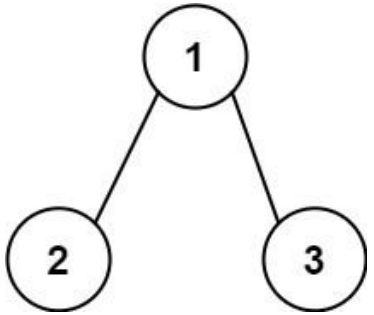
Example 1:




```
Input: root = [5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum = 22
Output: [[5,4,11,2],[5,8,4,5]]
Explanation: There are two paths whose sum equals targetSum:
5 + 4 + 11 + 2 = 22
5 + 8 + 4 + 5 = 22
```

Example 2:

· #46 #134 · · Contents




```
Input: root = [1,2,3], targetSum = 5
Output: []
```

Example 3:

```
Input: root = [1,2], targetSum = 0
Output: []
```

Constraints:

- The number of nodes in the tree is in the range [0, 5000].
- -1000 <= Node.val <= 1000
- -1000 <= targetSum <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a binary tree and a target sum, return all root-to-leaf paths
# where the sum of node values equals targetSum. Right?"
#
# "tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], targetSum=22:
# 5→4→11→2=22 ✓, 5→8→4→5=22 ✓ → [[5,4,11,2],[5,8,4,5]] ✓"

# PHASE 2 — BRUTE FORCE
# "DFS collecting all root-to-leaf paths, then filter by sum.
# Time: O(n*L) where L = path length (average tree height). Space: O(n*L)."

# PHASE 3 — OPTIMIZE
# "Backtracking with running sum — check target only at leaf nodes.
# Pass remaining = targetSum - node.val down the tree.
# At a leaf, if remaining == 0, this path is a solution. Append and backtrack.
# Time: O(n*L). Space: O(L) recursion + O(n*L) output — avoids post-filtering."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree=[5,4,8,11,null,13,4,7,2,null,null,5,1], target=22:
# path=[5],rem=17 → left=[5,4],rem=13 → left=[5,4,11],rem=2
# left=[5,4,11,7],rem=-5 → leaf, rem≠0. pop.
# right=[5,4,11,2],rem=0 → leaf, rem=0 → append [5,4,11,2] ✓
# right=[5,8],rem=14 → right=[5,8,4],rem=10 → left=[5,8,4,5],rem=5 → leaf≠0
# right=[5,8,4,1],rem=9 → leaf≠0. Back up. left=[5,8,13],leaf,rem=1≠0.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*L): visit each node once, copy path at each leaf O(L).
# Space O(L): recursion stack depth = tree height.
# Follow-up: Path Sum I (LC 112) — just return bool, no need to store paths.
# Follow-up: Path Sum III (LC 437) — any path (not just root-to-leaf), any direction.
# Use prefix sum hashmap to count valid paths in O(n)."
```

Greedy

Round 5 · Mid · Medium · 5 questions

Greedy

#134 Gas Station

MED

Open on LeetCode

Array · Greedy

Lists: Grind 169

Problem

There are n gas stations along a circular route, where the amount of gas at the ith station is gas[i].
You have a car with an unlimited gas tank and it costs cost[i] of gas to travel from the ith station to its next (i + 1)th station.
You begin the journey with an empty tank at one of the gas stations.
Given two integer arrays gas and cost, return the starting gas station's index if you can travel around the circuit once in the clockwise direction, otherwise return -1. If there exists a solution, it is guaranteed to be unique.

Example 1:

```
Input: gas = [1,2,3,4,5], cost = [3,4,5,1,2]
Output: 3
Explanation:
Start at station 3 (index 3) and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 4. Your tank = 4 - 1 + 5 = 8
Travel to station 0. Your tank = 8 - 2 + 1 = 7
Travel to station 1. Your tank = 7 - 3 + 2 = 6
Travel to station 2. Your tank = 6 - 4 + 3 = 5
```

Example 2:

```
Input: gas = [2,3,4], cost = [3,4,3]
Output: -1
Explanation:
You can't start at station 0 or 1, as there is not enough gas to travel to the next station.
Let's start at station 2 and fill up with 4 unit of gas. Your tank = 0 + 4 = 4
Travel to station 0. Your tank = 4 - 3 + 2 = 3
Travel to station 1. Your tank = 3 - 3 + 3 = 3
You cannot travel back to station 2, as it requires 4 unit of gas but you only have 3.
```

Constraints:

- n == gas.length == cost.length
- 1 <= n <= 10^5
- 0 <= gas[i], cost[i] <= 10^4
- The input is generated such that the answer is unique.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "n gas stations in a circle; gas[i] = gas at station i, cost[i] = gas to reach i+1.
# Find the starting station index to complete the circuit, or -1 if impossible. Right?"
#
# "gas=[1,2,3,4,5], cost=[3,4,5,1,2]:
# Start at 3: 0+4=4→4-1=3→3+5=8→8-2=6→6+1=7→7-3=4≥0 → return 3 ✓
# gas=[2,3,4], cost=[3,4,3]: total gas=9 < total cost=10 → -1 ✓"

# PHASE 2 — BRUTE FORCE
# "Try each station as start. Simulate the full circuit; if we never run dry, return that index.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Two greedy observations:
# 1. If total gas < total cost, no solution exists → return -1.
# 2. If a solution exists, it's unique. Start with candidate = 0 and tank = 0.
# Accumulate net gain (gas[i]-cost[i]). When tank < 0, reset tank to 0 and
# move candidate to i+1 (we can't start anywhere in [old_candidate, i]).
# The final candidate is the answer.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# gas=[1,2,3,4,5], cost=[3,4,5,1,2]:
# total=15>=15 ✓. i=0:tank=1-3=-2<0→tank=0,cand=1. i=1:tank=2-4=-2<0→cand=2.
# i=2:tank=3-5=-2<0→cand=3. i=3:tank=4-1=3. i=4:tank=3+5-2=6≥0.
# return 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): two variables.
# The key insight: if we run out of gas starting at s, no station between s and i can work
# (they'd all start with less gas than s did at that point).
# Follow-up: if multiple valid starting points exist (which the problem guarantees won't happen),
# return the smallest index — the greedy still finds the first valid one.
# Follow-up: minimum cost to fill gas (variant) — uses a monotonic stack approach."
```

Greedy

#179 Largest Number

MED

Open on LeetCode

Array · String · Greedy · Sorting

Lists: Grind 169

Problem

Given a list of non-negative integers nums, arrange them such that they form the largest number and return it.
Since the result may be very large, so you need to return a string instead of an integer.

Example 1:

```
Input: nums = [10,2]
Output: "210"
```

Example 2:

```
Input: nums = [3,30,34,5,9]
Output: "9534330"
```

Constraints:

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of non-negative integers, arrange them so they form the largest number.
# Return as a string. Right?"
#
# "nums=[10,2]: '210' > '102' → "210" ✓
# nums=[3,30,34,5,9]: → "9534330" ✓
# nums=[0,0]: → "0" (not "00") ✓"

# PHASE 2 — BRUTE FORCE
# "Try all permutations of nums, form the concatenated string for each,
# return the maximum. Time: O(n! * n). Space: O(n!)."

# PHASE 3 — OPTIMIZE
# "Custom comparator: to decide whether a comes before b, compare str(a)+str(b) vs str(b)+str(a).
# If str(a)+str(b) > str(b)+str(a), a should come first.
# Sort all numbers by this comparator, then concatenate.
# Edge case: if the largest digit is 0, the result is '0'.
# Time: O(n log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[10,2]:
# compare("10","2"): "102" vs "210" → "210">"102" → 2 before 10. sorted=["2","10"]. → "210" ✓
#
# nums=[3,30,34,5,9]:
# compare pairs: "9">"53">"534">"5330"→ sorted=["9","5","34","3","30"]. → "9534330" ✓
#
# nums=[0,0]: sorted=["0","0"] → "00" → first char '0' → return "0" ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sort dominates. Space O(n): string array + result.
# The comparator is transitive and consistent — provably correct for any input.
# Follow-up: smallest number from array — same comparator, ascending sort.
# Follow-up: if numbers are very large (BigInteger), convert to strings first anyway."
```


Greedy

⋆ #280 Wiggle Sort ⋆

MED

Open on LeetCode

Array · Greedy · Sorting

Lists: ⋆ Premium | Premium 98

Problem

Given an integer array nums, reorder it such that nums[0] <= nums[1] >= nums[2] <= nums[3]....
You may assume the input array always has a valid answer.

Example 1:

```
Input: nums = [3,5,2,1,6,4]
Output: [3,5,1,6,2,4]
Explanation: [1,6,2,5,3,4] is also accepted.
```

Example 2:

```
Input: nums = [6,6,5,6,3,8]
Output: [6,6,5,6,3,8]
```

Constraints:

- 1 <= nums.length <= 5 * 104
- 0 <= nums[i] <= 104
- It is guaranteed that there will be an answer for the given input nums.

Follow up: Could you solve the problem in O(n) time complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Reorder an array in-place so that nums[0] <= nums[1] >= nums[2] <= nums[3]…
# (alternating less-than and greater-than). Multiple valid answers exist. Right?"
#
# "nums=[3,5,2,1,6,4]: one valid output=[3,5,1,6,2,4] ✓
# nums=[1,5,1,1,6,4]: one valid=[1,5,1,6,1,4] ✓"

# PHASE 2 — BRUTE FORCE
# "Sort the array, then interleave first half and second half.
# Split sorted array into two halves; place smaller half at even indices,
# larger half at odd indices.
# Time: O(n log n). Space: O(n) for the copy."

# PHASE 3 — OPTIMIZE
# "Greedy one-pass: for each index i, if the wiggle condition is violated, swap.
# – Even index i: nums[i] should be <= nums[i-1]. If not, swap(i, i-1).
# – Odd index i: nums[i] should be >= nums[i-1]. If not, swap(i, i-1).
# A local swap always fixes the current violation without breaking prior pairs.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[3,5,2,1,6,4]:
# i=1(odd): 5>=3 ✓
# i=2(even): 2<=5 ✓
# i=3(odd): 1<2 → swap → [3,5,1,2,6,4]. Wait: now [3,5,2,1,6,4]→swap(2,1)→[3,5,1,2,6,4]
# i=4(even): 6>2 → swap → [3,5,1,6,2,4]
# i=5(odd): 4>2 ✓ → [3,5,1,6,2,4] ✓ (valid wiggle)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Greedy: O(n) time, O(1) space vs brute O(n log n) time, O(n) space.
# The greedy works because swapping adjacent elements to fix one condition
# never invalidates the previously satisfied condition.
# Follow-up: Wiggle Sort II (LC 324) — strict inequality nums[0]<nums[1]>nums[2]<…
# Requires median partitioning (nth_element) to guarantee strict separation.
# Follow-up: count wiggle subsequences (LC 376) — DP/greedy to count without reordering."
```

Greedy

#954 Array of Doubled Pairs

MED

Open on LeetCode

Array · Hash Table · Greedy · Sorting

Lists: CodeSignal

Problem

Given an integer array of even length arr, return true if it is possible to reorder arr such that arr[2 * i + 1] = 2 * arr[2 * i] for every 0 <= i < len(arr) / 2, or false otherwise.

Example 1:

```
Input: arr = [3,1,3,6]
Output: false
```

Example 2:

```
Input: arr = [2,1,2,6]
Output: false
```

Example 3:

```
Input: arr = [4,-2,2,-4]
Output: true
Explanation: We can take two groups, [-2,-4] and [2,4] to form [-2,-4,2,4] or [2,4,-2,-4].
```

Constraints:

- 2 <= arr.length <= 3 * 104
- arr.length is even.
- -105 <= arr[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an integer array, return true if we can reorder it so that for every
# arr[2i+1] = 2 * arr[2i] (each pair: smaller then double). Right?"
#
# "arr=[3,1,3,6]: pairs (1,2) missing 2, can't pair → False... wait:
# arr=[4,-2,2,-4]: sort by abs → [-2,-4,2,4]? pairs (-2,-4)? -4=-2*2 ✓, (2,4) ✓ → True ✓
# arr=[1,2,4,16,8,4]: sort abs→[1,2,4,4,8,16]: 1→2 ✓, 4→8 ✓, 4→16? No → False ✓"

# PHASE 2 — BRUTE FORCE
# "Sort by absolute value. Greedily pair smallest unmatched element with its double.
# Use a counter to track available elements.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same greedy but iterate over sorted unique keys (by abs value) to avoid
# processing same element multiple times.
# Time: O(n log n) for sorting. Space: O(n) counter."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# arr=[4,-2,2,-4]: count={4:1,-2:1,2:1,-4:1}.
# sorted by abs: [-2,-4,2,4].
# x=-2: count[2*(-2)]=count[-4]=1>=count[-2]=1 ✓. count[-4]=0, count[-2]=0.
# x=-4: count[-8]=0>=count[-4]=0 ✓ (trivially).
# x=2: count[4]=1>=count[2]=1 ✓. count[4]=0, count[2]=0.
# return True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): sorting dominates. Space O(n) for counter.
# Key: sort by absolute value handles negative numbers correctly (-2 pairs with -4).
# Follow-up: if 'double' is replaced by 'k times' — same greedy, check count[k*x].
# Follow-up: find the actual pairs (not just existence) — store pairs during the greedy pass."
```


Greedy

#2131 Longest Palindrome by Concatenating Two Letter Words

MED

Open on LeetCode

Array · Hash Table · String · Greedy · Counting

Lists: CodeSignal

Problem

You are given an array of strings words. Each element of words consists of two lowercase English letters.
Create the longest possible palindrome by selecting some elements from words and concatenating them in any order. Each element can be selected at most once.
Return the length of the longest palindrome that you can create. If it is impossible to create any palindrome, return 0.
A palindrome is a string that reads the same forward and backward.

Example 1:

```
Input: words = ["lc","cl","gg"]
Output: 6
Explanation: One longest palindrome is "lc" + "gg" + "cl" = "lcggcl", of length 6.
Note that "clgglc" is another longest palindrome that can be created.
```

Example 2:

```
Input: words = ["ab","ty","yt","lc","cl","ab"]
Output: 8
Explanation: One longest palindrome is "ty" + "lc" + "cl" + "yt" = "tylcclyt", of length 8.
Note that "lcyttycl" is another longest palindrome that can be created.
```

Example 3:

```
Input: words = ["cc","ll","xx"]
Output: 2
Explanation: One longest palindrome is "cc", of length 2.
Note that "ll" is another longest palindrome that can be created, and so is "xx".
```

Constraints:

- 1 <= words.length <= 105
- words[i].length == 2
- words[i] consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array of two-letter words, select some subset to concatenate into the longest
# palindrome possible (each word used at most once). Return the max length. Right?"
#
# "words=['lc','cl','gg']: 'lc'+'gg'+'cl'='lcggcl' → len=6 ✓
# words=['ab','ty','yt','lc','cl','ab']: 'ty'+'ab'+'cl'? No... 'lc'+'ab'+'ba'+'cl' but no 'ba'.
# Actually: 'ty'+'yt'=tyyt, 'lc'+'cl'=lccl → 'tylcclyt'? We pick pairs: (ty,yt),(lc,cl) → 8?
# Hmm: 'lc'+'cl'='lccl'(pal), 'ty'+'yt'='tyyt'(pal) → concat='lccl'+'tyyt' not palindrome.
# Better: 'ty'+'lc'+'cl'+'yt'='tylcclyt' → palindrome! length=8 ✓"

# PHASE 2 — BRUTE FORCE
# "Count each word's frequency. For each word that is NOT a palindrome (ab≠ba),
# pair it with its reverse. For palindromic words (gg, aa), use pairs of two + at most one center.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same greedy — O(n) time/space — but process pairs more cleanly.
# Non-palindromic words: match word with reverse(word), each pair contributes 4 chars.
# Palindromic words: each pair-of-two contributes 4 chars, leftover one contributes 2 (center)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# words=['lc','cl','gg']:
# count={'lc':1,'cl':1,'gg':1}.
# 'lc': rev='cl', not palindrome. min(1,1)=1 pair → length+=4. seen={lc,cl}.
# 'gg': palindrome. pairs=0, count%2=1 → center=True.
# length=4+2=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass through words. Space O(n): counter.
# At most one palindromic word can sit in the center of the final palindrome.
# Non-palindromic pairs must mirror each other (word ↔ reverse).
# Follow-up: Longest Palindrome (LC 409) — same structure with single characters.
# Follow-up: return the actual palindrome string — construct it from paired words."
```

Round 6

Mid Priority · Hard
28 questions · 5 patterns

- [ ] Linked List #25 #432 #460
- [ ] Stack #32 #42 #84 #224 #239 #716 #772 #895
- [ ] Heap #23 #218 #272 #295 #358 #499 #632 #759 #1425
- [ ] Trie #212 #336 #588 #642
- [ ] Backtracking #37 #51 #126 #996

Linked List

Round 6 · Mid · Hard · 3 questions

Linked List

#25 Reverse Nodes in k-Group HAR

Open on LeetCode

Linked List · Recursion

Lists: Grind 169

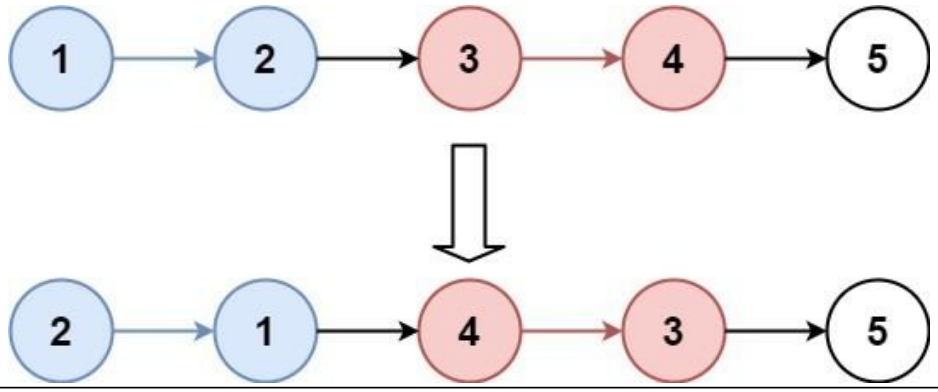
Problem

Given the head of a linked list, reverse the nodes of the list k at a time, and return the modified list.

k is a positive integer and is less than or equal to the length of the linked list. If the number of nodes is not a multiple of k then left-out nodes, in the end, should remain as it is.

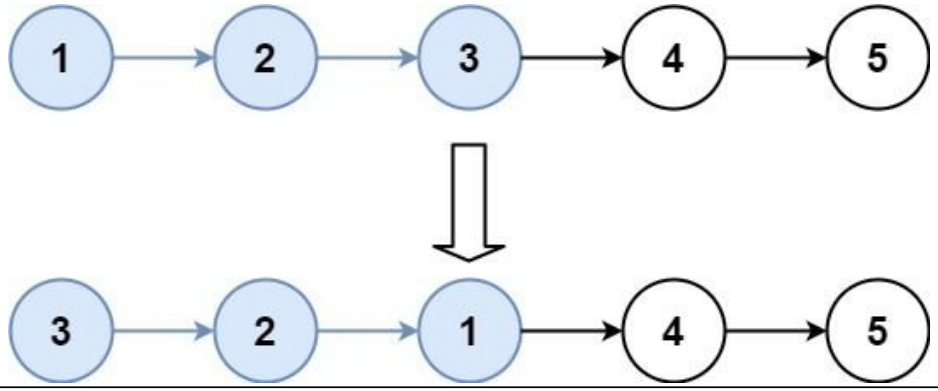
You may not alter the values in the list's nodes, only nodes themselves may be changed.

Example 1:




```
Input: head = [1,2,3,4,5], k = 2
Output: [2,1,4,3,5]
```

Example 2:




```
Input: head = [1,2,3,4,5], k = 3
Output: [3,2,1,4,5]
```

Constraints:

- The number of nodes in the list is n.
- 1 <= k <= n <= 5000
- 0 <= Node.val <= 1000

Follow-up: Can you solve the problem in O(1) extra memory space?

◆ Interview Approach · STAR-LC

PHASE 1 — CLARIFY


```
# "Reverse every k consecutive nodes in the linked list. If remaining nodes < k, leave them as-is.
# Return the new head. Right?"
#
# "head=[1,2,3,4,5], k=2: [2,1,4,3,5] ✓
# head=[1,2,3,4,5], k=3: [3,2,1,4,5] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all node values into an array. Reverse each group of k elements in place.
# Rebuild the linked list from the modified array.
# Time: O(n). Space: O(n) for the array."

# PHASE 3 — OPTIMIZE
# "In-place pointer reversal group by group — O(1) space.
# For each group: check if k nodes remain (if not, stop).
# Reverse k nodes in-place using the 3-pointer technique.
# Link the previous group's tail to the new group head, and current group's tail to the next group.
# Use a dummy node to simplify relinking the first group.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

            # Wait — after reversal group_start.next points somewhere wrong, fix:
            # Actually reverse_k leaves curr (the node after the group) in new_tail.next
            # We need: new_tail.next = curr (handled by reverse_k returning new_tail=old head)
            # group_start after reversal is the tail; its next should point to next group start

# PHASE 5 — TEST & DEBUG
# [1,2,3,4,5], k=2:
# Group1: has_k? 1,2 ✓. reverse(1→2): prev=2→1, curr=3. new_head=2,new_tail=1.
# dummy.next=2. 1.next=3. group_prev=1. List: dummy→2→1→3→4→5.
# Group2: has_k? 3,4 ✓. reverse(3→4): new_head=4,new_tail=3.
# 1.next=4. 3.next=5. group_prev=3. List: dummy→2→1→4→3→5.
# Group3: has_k? only 5 → stop. → [2,1,4,3,5] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each node reversed exactly once. Space O(1): only pointer variables.
# The dummy node is critical to handle relinking the first group.
# Follow-up: Swap Nodes in Pairs (LC 24) — special case with k=2.
# Follow-up: reverse every other group of k — add a skip_k step between reverse steps."
```

Linked List

#432 All O'one Data Structure HAR

Open on LeetCode

Hash Table · Design · Doubly-Linked List · Linked List

Lists: Denny Zhang

Problem

Design a data structure to store the strings' count with the ability to return the strings with minimum and maximum counts.

Implement the AllOne class:

- AllOne() Initializes the object of the data structure.
- inc(String key) Increments the count of the string key by 1. If key does not exist in the data structure, insert it with count 1.
- dec(String key) Decrements the count of the string key by 1. If the count of key is 0 after the decrement, remove it from the data structure. It is guaranteed that key exists in the data structure before the decrement.
- getMaxKey() Returns one of the keys with the maximal count. If no element exists, return an empty string "".
- getMinKey() Returns one of the keys with the minimum count. If no element exists, return an empty string "".

Note that each function must run in O(1) average time complexity.

Example 1:

```
Input
["AllOne", "inc", "inc", "getMaxKey", "getMinKey", "inc", "getMaxKey", "getMinKey"]
[[], ["hello"], ["hello"], [], [], ["leet"], [], []]
Output
[null, null, null, "hello", "hello", null, "hello", "leet"]

Explanation
AllOne allOne = new AllOne();
```

Constraints:

- 1 <= key.length <= 10
- key consists of lowercase English letters.
- It is guaranteed that for each call to dec, key is existing in the data structure.
- At most 5 * 104 calls will be made to inc, dec, getMaxKey, and getMinKey.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a data structure supporting: inc(key), dec(key), getMaxKey()→any key with max count,
# getMinKey()→any key with min count. All operations O(1). Right?"
#
# "inc('a'), inc('b'), inc('b'), inc('c'), inc('c'), inc('c').
# getMaxKey()→'c'. getMinKey()→'a'. dec('b'), dec('b'). getMinKey()→'b' or 'a'. ✓"
# PHASE 2 — BRUTE FORCE
# "Use a dict {key: count}. For getMaxKey/getMinKey, scan all entries.
# Time: O(1) for inc/dec, O(n) for getMax/getMin. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Doubly linked list of buckets (each bucket holds a count and a set of keys at that count)
# + a hashmap {key → bucket_node}.
# inc: move key from its current bucket to the next (count+1) bucket.
# dec: move key to the previous (count-1) bucket.
# getMax: return any key from the tail bucket. getMin: from the head bucket.
# All O(1) amortized. Space: O(n)."
# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# inc('a'): bucket(1)={a}. head→1→tail. key_bucket={a:1}.
# inc('b'): bucket(1)={a,b}. inc('b'): bucket(2)={b}, bucket(1)={a}. head→1→2→tail.
# getMaxKey→'b'(tail.prev=bucket2). getMinKey→'a'(head.next=bucket1) ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): set operations and linked list pointer updates.
# The DLL of buckets maintains sorted order by count implicitly.
# Follow-up: if we need getMedianKey, we'd need a balanced BST instead of a DLL.
# Follow-up: LFU Cache (LC 460) uses the exact same bucket-DLL structure."
```


Linked List

#460 LFU Cache HAR

Open on LeetCode

Hash Table · Linked List · Design · Doubly-Linked List

Lists: Denny Zhang

Problem

Design and implement a data structure for a Least Frequently Used (LFU) cache.

Implement the LFUCache class:

- LFUCache(int capacity) Initializes the object with the capacity of the data structure.
- int get(int key) Gets the value of the key if the key exists in the cache. Otherwise, returns -1.
- void put(int key, int value) Update the value of the key if present, or inserts the key if not already present. When the cache reaches its capacity, it should invalidate and remove the least frequently used key before inserting a new item. For this problem, when there is a tie (i.e., two or more keys with the same frequency), the least recently used key would be invalidated.

To determine the least frequently used key, a use counter is maintained for each key in the cache. The key with the smallest use counter is the least frequently used key.

When a key is first inserted into the cache, its use counter is set to 1 (due to the put operation). The use counter for a key in the cache is incremented either a get or put operation is called on it.

The functions get and put must each run in O(1) average time complexity.

Example 1:

```
Input
["LFUCache", "put", "put", "get", "put", "get", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [3], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, 3, null, -1, 3, 4]

Explanation
// cnt(x) = the use counter for key x
```

Constraints:

- 1 <= capacity <= 104
- 0 <= key <= 105
- 0 <= value <= 109
- At most 2 * 105 calls will be made to get and put.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design an LFU cache: get(key)→val or -1, put(key,val) — evict LFU (then LRU among ties).
# All O(1). Right?"
#
# "LFUCache(2): put(1,1),put(2,2),get(1)→1(freq1=2).put(3,3) evicts key 2(freq=1,LRU).
# get(2)→-1, get(3)→3, put(4,4) evicts key 3(freq=1). get(1)→1,get(3)→-1,get(4)→4 ✓"
# PHASE 2 — BRUTE FORCE
# "Dict for {key:(val,freq,last_used)}. On evict, scan all entries for min freq, then min last_used.
# Time: O(n) per put when eviction needed. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Three structures for O(1):
# 1. key_map: {key → (val, freq)}
# 2. freq_map: {freq → OrderedDict(key→None)} — LRU order within same freq
# 3. min_freq: current minimum frequency
# get: update freq, move key to freq+1 bucket. put: evict from freq_map[min_freq]."
# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# LFUCache(2): put(1,1): key_map={1:[1,1]},freq_map={1:{1}},min=1.
# put(2,2): key_map={1:[1,1],2:[2,1]},freq_map={1:{1,2}},min=1.
# get(1): update→freq_map={1:{2},2:{1}},min=1. return 1 ✓.
# put(3,3): evict from freq_map[1]→evict key2. key_map={1:[1,2],3:[3,1]},min=1.
# get(2)→-1 ✓. get(3)→3, update→freq=2, min=1. put(4,4): evict freq_map[1]→evict 3. ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1): dict lookups + OrderedDict FIFO operations (popitem(last=False)).
# Space O(capacity): bounded by cache size.
# min_freq only ever decreases on put (new key starts at freq=1) — so it's always correct.
# Follow-up: LRU Cache (LC 146) — simpler, only recency matters, no frequency tracking.
# Follow-up: combine LFU with TTL expiration — add expiry timestamps and a background cleaner."
```

Stack

Round 6 · Mid · Hard · 8 questions

Stack

#32 Longest Valid Parentheses

HAR

Open on LeetCode

String · Dynamic Programming · Stack

Lists: Grind 169

Problem

Given a string containing just the characters '(' and ')', return the length of the longest valid (well-formed) parentheses substring.

Example 1:

```
Input: s = "(()"
Output: 2
Explanation: The longest valid parentheses substring is "()".
```

Example 2:

```
Input: s = ")()())"
Output: 4
Explanation: The longest valid parentheses substring is "()()".
```

Example 3:

```
Input: s = ""
Output: 0
```

Constraints:

- 0 <= s.length <= 3 * 104
- s[i] is '(', or ')'.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string of '(' and ')', return the length of the longest valid (well-formed)
# parentheses substring. Right?"
#
# "s='(()': longest valid is '()' → length 2 ✓
# s=')()())': '()()' → length 4 ✓
# s='': → 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all substrings, check each for validity, track the longest.
# Time: O(n³) — O(n²) substrings, O(n) to validate each. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Stack storing indices. Push index of '(' onto stack.
# On ')': if stack top is '(' index, pop it (matched pair). Else push ')' index as new boundary.
# Initialize stack with [-1] as a base boundary.
# After each match, length = current index - stack top (index of last unmatched char).
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s=')()())':
# i=0')': pop -1 → stack empty → push 0. stack=[0].
# i=1'(': push 1. [0,1].
# i=2')': pop 1. stack=[0]. best=max(0,2-0)=2.
# i=3'(': push 3. [0,3].
# i=4')': pop 3. stack=[0]. best=max(2,4-0)=4.
# i=5')': pop 0 → stack empty → push 5. best=4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(n): stack holds at most n indices.
# DP alternative: dp[i] = length of valid substring ending at i.
# dp[i] = dp[i-2]+2 if s[i-dp[i-1]-1]=='(' — O(n) time, O(n) space.
# O(1) space: two-pass counting (left-to-right then right-to-left with open/close counters).
# Follow-up: return the actual substring, not just length — track start index in stack."
```

Stack

#42 Trapping Rain Water

HAR

Open on LeetCode

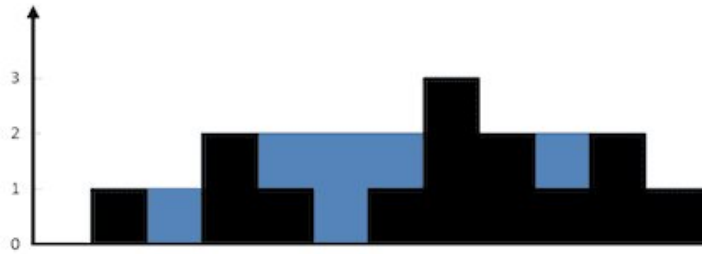
Array · Two Pointers · Dynamic Programming · Stack · Monotonic Stack

Lists: Grind 169

Problem

Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining.

Example 1:




```
Input: height = [0,1,0,2,1,0,1,3,2,1,2,1]
Output: 6
Explanation: The above elevation map (black section) is represented by array [0,1,0,2,1,0,1,3,2,1,2,1]. In this case, 6 units of rain water (blue section) are being trapped.
```

Example 2:

```
Input: height = [4,2,0,3,2,5]
Output: 9
```

Constraints:

- n == height.length
- 1 <= n <= 2 * 104
- 0 <= height[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an elevation map, compute how much rain water can be trapped after raining.
# Water is trapped above a bar if both sides have taller bars. Right?"
#
# "height=[0,1,0,2,1,0,1,3,1,0,1,2]:
# Water trapped at positions 2,4,5,6,9,10. Total=6 ✓"

# PHASE 2 — BRUTE FORCE
# "For each bar, water above it = min(max_left, max_right) - height[i].
# Precompute max_left and max_right arrays.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-pointer approach — O(1) space.
# Maintain left and right pointers, left_max and right_max.
# If height[left] <= height[right]: water at left = left_max - height[left] (right side guaranteed taller).
# Move the shorter side inward.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# height=[0,1,0,2,1,0,1,3,1,0,1,2]:
# l=0,r=11,lm=0,rm=0. h[0]=0<=h[11]=2: lm=0,water+=0-0=0. l=1.
# h[1]=1<=h[11]=2: lm=1. l=2. h[2]=0<lm=1: water+=1. l=3.
# h[3]=2<=h[11]=2: lm=2. l=4. h[4]=1<lm=2: water+=1. ... total=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): only four variables.
# Monotonic stack: O(n) time, O(n) space — solves it differently (horizontal layers).
# Two-pointer is O(1) space and arguably clearest approach.
# Follow-up: Trapping Rain Water II (LC 407) — 3D extension, use a min-heap of border cells.
# Follow-up: if heights can change (updates), use segment tree for range max queries."
```


Stack

#84 Largest Rectangle in Histogram

HAR

Open on LeetCode

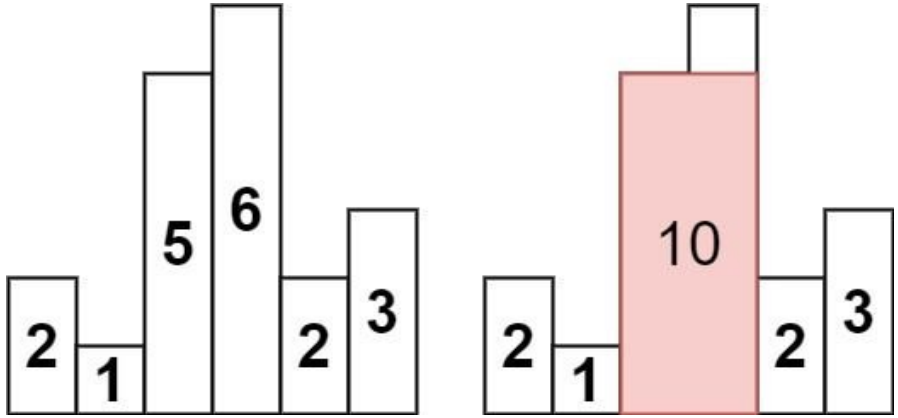
Array · Stack · Monotonic Stack

Lists: Grind 169

Problem

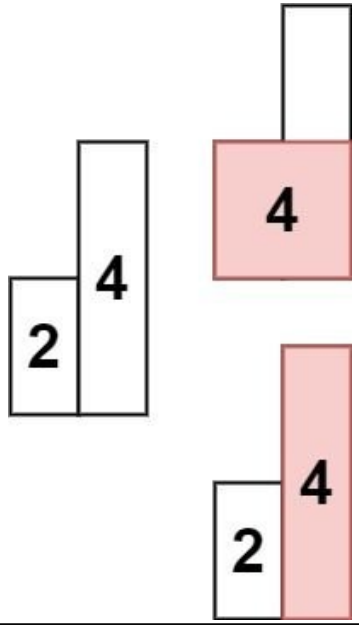
Given an array of integers heights representing the histogram's bar height where the width of each bar is 1, return the area of the largest rectangle in the histogram.

Example 1:




```
Input: heights = [2,1,5,6,2,3]
Output: 10
Explanation: The above is a histogram where width of each bar is 1.
The largest rectangle is shown in the red area, which has an area = 10 units.
```

Example 2:




```
Input: heights = [2,4]
Output: 4
```

Constraints:

- 1 <= heights.length <= 105
- 0 <= heights[i] <= 104

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given an array of bar heights in a histogram, find the area of the largest rectangle
# that fits entirely within the histogram. Right?"
#
# "heights=[2,1,5,6,2,3]:
# Largest rectangle: height=5, width=2 (bars 2,3) → area=10 ✓
# Or height=2, width=5 → area=10. height=3,width=2=6. max=10 ✓"

# PHASE 2 — BRUTE FORCE
# "For each pair (i,j), compute area = (j-i+1) * min(heights[i..j]).
# Time: O(n²) with O(1) min tracking per expansion. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Monotonic increasing stack of indices.
# For each bar: while stack top is taller than current bar, it can no longer extend rightward.
# Pop it and compute its area: height = heights[popped], width = current_idx - stack_new_top - 1.
# Sentinel 0s at both ends simplify boundary handling.
# Time: O(n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# heights=[0,2,1,5,6,2,3,0] (with sentinels):
# i=1(2): stack=[0,1]. i=2(1): 1<2 → pop1: h=2,w=2-0-1=1,area=2. push2. stack=[0,2].
# i=3(5): push. i=4(6): push. i=5(2): 2<6 → pop4: h=6,w=5-3-1=1,area=6.
# 2<5 → pop3: h=5,w=5-2-1=2,area=10 ✓. 2>=1 stop. push5.
# i=6(3): push. i=7(0): pop6:h=3,w=7-5-1=1,area=3. pop5:h=2,w=7-2-1=4,area=8. pop2:h=1,w=7-0-1=6,area=6. best=10 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each bar pushed and popped exactly once.
# Space O(n): stack holds at most n indices.
# The sentinel 0s ensure every bar gets popped and processed — no leftovers in stack.
# Follow-up: Maximal Rectangle (LC 85) — for each row, build histogram heights, apply this.
# Follow-up: Largest Rectangle of 1s in binary matrix — same reduction to histogram."
```


Stack

☐ **#224 Basic Calculator** HAR

Open on LeetCode

Math · String · Stack · Recursion

Lists: Grind 169

Problem

Given a string s representing a valid expression, implement a basic calculator to evaluate it, and return the result of the evaluation.

Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1:

```
Input: s = "1 + 1"
Output: 2
```

Example 2:

```
Input: s = " 2-1 + 2 "
Output: 3
```

Example 3:

```
Input: s = "(1+(4+5+2)-3)+(6+8)"
Output: 23
```

Constraints:

- 1 <= s.length <= 3 * 105
- s consists of digits, '+', '-', '(', ')', and ' '.
- s represents a valid expression.
- '+' is not used as a unary operation (i.e., "+1" and "+(2 + 3)" is invalid).
- '-' could be used as a unary operation (i.e., "-1" and "-(2 + 3)" is valid).
- There will be no two consecutive operators in the input.
- Every number and running calculation will fit in a signed 32-bit integer.

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Evaluate a string expression with +, -, and parentheses. No * or /.
# Non-negative integers, spaces allowed. Right?"
#
# "s='1 + 1' → 2 ✓
# s='(1+(4+5+2)-3)+(6+8)' → 23 ✓
# s='- (3 + (4 + 5))' → -12 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive descent parser: handle parentheses recursively.
# Parse a number or a sub-expression in parentheses, apply the sign.
# Time: O(n). Space: O(d) where d = nesting depth."

# PHASE 3 — OPTIMIZE
# "Iterative stack: push (result, sign) when '(' is encountered, pop and merge on ')'.
# Maintain current number being built, current result, and sign.
# Time: O(n). Space: O(d) for the stack."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='(1+(4+5+2)-3)+(6+8)':
# '(': push result=0, sign=1. reset result=0, sign=1.
# '1': num=1. '+': result=1, num=0, sign=1. '(': push 1,1. reset.
# '4+5+2': result=11. ')': result=11*1+1=12. '-': result=12,sign=-1.
# '3': num=3. ')': result=12+(-1)*3=9. result=9*1+0=9.
# '+': result=9, sign=1. '(': push 9,1. '6+8': result=14. ')': 14*1+9=23 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(d): stack depth = nesting depth.
# Multi-digit numbers handled by num = num*10 + digit.
# Negative signs attach to the next number or sub-expression.
# Follow-up: Basic Calculator II (LC 227) — add * and /; use precedence handling.
# Follow-up: Basic Calculator III (LC 772) — all four operators with parentheses."
```

Stack

#239 Sliding Window Maximum

HAR

Open on LeetCode

Array · Queue · Sliding Window · Monotonic Queue

Lists: Grind 169

Problem

You are given an array of integers nums, there is a sliding window of size k which is moving from the very left of the array to the very right. You can only see the k numbers in the window. Each time the sliding window moves right by one position. Return the max sliding window.

Example 1:

```
Input: nums = [1,3,-1,-3,5,3,6,7], k = 3
Output: [3,3,5,5,6,7]
Explanation:
Window position Max
--------------- -----
[1 3 -1] -3 5 3 6 7 3
1 [3 -1 -3] 5 3 6 7 3
1 3 [-1 -3 5] 3 6 7 5
```

Example 2:

```
Input: nums = [1], k = 1
Output: [1]
```

Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104
- 1 <= k <= nums.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array and window size k, return the maximum value in each sliding window.
# Window slides one step right each time. Right?"
#
# "nums=[1,3,-1,-3,5,3,6,7], k=3:
# [1,3,-1]→3, [3,-1,-3]→3, [-1,-3,5]→5, [-3,5,3]→5, [5,3,6]→6, [3,6,7]→7 → [3,3,5,5,6,7] ✓"

# PHASE 2 — BRUTE FORCE
# "Slide window, take max at each position.
# Time: O(n*k). Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "Monotonic deque (double-ended queue) of indices — maintains candidates for the window max.
# Invariant: deque stores indices in decreasing order of their values.
# For each element: remove from back any index with smaller value (they can never be max).
# Remove from front any index that's out of the window.
# Front of deque is always the current window maximum.
# Time: O(n). Space: O(k)."

# PHASE 4 — CLEAN CODE
            # Remove smaller elements from back
            # Remove out-of-window element from front
            # Record max once first window is complete

# PHASE 5 — TEST & DEBUG
# nums=[1,3,-1,-3,5,3,6,7], k=3:
# i=0(1): dq=[0]. i=1(3): pop0(1<3), dq=[1]. i=2(-1): dq=[1,2]. window[1,3,-1]: ans=nums[1]=3 ✓
# i=3(-3): dq=[1,2,3]. front=1>=1 ✓. ans=nums[1]=3 ✓
# i=4(5): pop3(-3<5),pop2(-1<5),pop1(3<5). dq=[4]. front=4>=2 ✓. ans=5 ✓
# i=5(3): dq=[4,5]. ans=nums[4]=5 ✓. i=6(6): pop5(3<6),pop4(5<6). dq=[6]. ans=6 ✓
# i=7(7): pop6(6<7). dq=[7]. ans=7 ✓. Result=[3,3,5,5,6,7] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each element enqueued and dequeued at most once.
# Space O(k): deque holds at most k indices.
# Brute O(n*k) is simple but TLEs for large inputs.
# Follow-up: Sliding Window Minimum — same deque, flip comparison to keep increasing order.
# Follow-up: Max of all subarrays of size k in a circular array — extend with modular indexing."
```


Stack

#716 Max Stack

HAR

Open on LeetCode

Stack · Design · Doubly-Linked List · Ordered Set

Lists: Denny Zhang

Problem

Design a max stack data structure that supports the stack operations and supports finding the stack's maximum element.

Implement the MaxStack class:

- MaxStack() Initializes the stack object.
- void push(int x) Pushes element x onto the stack.
- int pop() Removes the element on top of the stack and returns it.
- int top() Gets the element on the top of the stack without removing it.
- int peekMax() Retrieves the maximum element in the stack without removing it.
- int popMax() Retrieves the maximum element in the stack and removes it. If there is more than one maximum element, only remove the top-most one.

You must come up with a solution that supports O(1) for each top call and O(logn) for each other call.

Example 1:

```
Input
["MaxStack", "push", "push", "push", "top", "popMax", "top", "peekMax", "pop", "top"]
[[], [5], [1], [5], [], [], [], [], [], []]
Output
[null, null, null, null, 5, 5, 1, 5, 1, 5]

Explanation
MaxStack stk = new MaxStack();
```

Constraints:

- -107 <= x <= 107
- At most 105 calls will be made to push, pop, top, peekMax, and popMax.
- There will be at least one element in the stack when pop, top, peekMax, or popMax is called.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a Max Stack supporting push, pop, top, peekMax, popMax — all O(log n).
# peekMax: return max without removing. popMax: remove and return max element. Right?"
#
# "push(5),push(1),push(5): top=5,popMax=5,top=1,peekMax=5,pop=1,top=5 ✓"

# PHASE 2 — BRUTE FORCE
# "Regular stack + scan for popMax.
# push/pop/top: O(1). peekMax: O(n) scan. popMax: O(n) scan + O(n) rebuild.
# Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Doubly linked list (DLL) for the stack + sorted set (treemap) for O(log n) max access.
# DLL preserves insertion order; sorted set tracks (val, node_id) for O(log n) find/remove.
# push: add to DLL tail + sorted set. pop: remove DLL tail + sorted set entry.
# popMax: find max in sorted set, remove from sorted set + DLL.
# All ops O(log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# push(5,uid=0),push(1,uid=1),push(5,uid=2). sorted=[(1,1),(5,0),(5,2)].
# top()→stack[-1]=(5,2)→5 ✓. popMax()→sorted[-1]=(5,2)→remove uid2. return 5 ✓.
# top()→clean stack: (5,2) removed→skip→(1,1). top=1 ✓. peekMax→(5,0)→5 ✓.
# pop()→(1,1) removed. return 1 ✓. top()→(5,0)→5 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(log n) with sorted set (SortedList or TreeMap).
# Space O(n): lazy deletion avoids O(n) rebuilds.
# Follow-up: if only O(n) time needed, brute is simpler and acceptable.
# Follow-up: Min Stack (LC 155) — O(1) with parallel min-stack, no sorted set needed."
```

Stack

⋆ #772 Basic Calculator III ⋆ HAR

Open on LeetCode

Math · String · Stack · Recursion

Lists: ⋆ Premium | Premium 98

Problem

Implement a basic calculator to evaluate a simple expression string.
The expression string contains only non-negative integers, '+', '-', '*', '/' operators, and open '(' and closing parentheses ')'.
The integer division should truncate toward zero.
You may assume that the given expression is always valid. All intermediate results will be in the range of [-231, 231 - 1].
Note: You are not allowed to use any built-in function which evaluates strings as mathematical expressions, such as eval().

Example 1:

```
Input: s = "1+1"
Output: 2
```

Example 2:

```
Input: s = "6-4/2"
Output: 4
```

Example 3:

```
Input: s = "2*(5+5*2)/3+(6/2+8)"
Output: 21
```

Constraints:

- 1 <= s <= 104
- s consists of digits, '+', '-', '*', '/', '(', and ')'.
- s is a valid expression.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Evaluate an expression with +, -, *, /, and parentheses. Non-negative integers.
# Division truncates toward zero. Right?"
#
# "s='1+1+1' → 3 ✓
# s='6-4/2' → 4 ✓
# s='2*(5+5*2)/3+(6/2+8)' → 21 ✓
# s='(2+6*3+5-(3*14/7+2)*5)+3' → -12 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive descent parser: parse expressions, respecting operator precedence and parentheses.
# Time: O(n). Space: O(d) recursion depth."

# PHASE 3 — OPTIMIZE
# "Stack-based with operator precedence: + and - push with sign; * and / compute immediately.
# Handle parentheses by recursively calling evaluate on the inner expression.
# Use a helper that processes until ')' or end of string.
# Time: O(n). Space: O(n) for stack depth."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='2*(5+5*2)/3+(6/2+8)':
# '2': num=2. '*': push 2, sign='*'. '(': recurse.
# Inner: '5+5*2' → push 5, push 5,sign='*',num=2→pop5*2=10,push 10. sum=15.
# Back: sign='*', num=15 → pop2*15=30. '/': sign='/', push 30. '3': num=3.
# '+': pop30/3=10, push 10. sign='+'. ... total=21 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each character processed once. Space O(n): stack depth.
# Handles multi-digit numbers, precedence, and nested parentheses correctly.
# Follow-up: Basic Calculator I (LC 224) — only + and -.
# Follow-up: Basic Calculator II (LC 227) — + - * / but no parentheses."
```


Stack

#895 Maximum Frequency Stack HAR

Open on LeetCode

Hash Table · Stack · Design

Lists: Grind 169

Problem

Design a stack-like data structure to push elements to the stack and pop the most frequent element from the stack.
Implement the FreqStack class:

- FreqStack() constructs an empty frequency stack.
- void push(int val) pushes an integer val onto the top of the stack.
- int pop() removes and returns the most frequent element in the stack. If there is a tie for the most frequent element, the element closest to the stack's top is removed and returned.

Example 1:

```
Input
["FreqStack", "push", "push", "push", "push", "push", "push", "pop", "pop", "pop", "pop"]
[[], [5], [7], [5], [7], [4], [5], [], [], [], []]
Output
[null, null, null, null, null, null, null, 5, 7, 5, 4]

Explanation
FreqStack freqStack = new FreqStack();
```

Constraints:

- 0 <= val <= 109
- At most 2 * 104 calls will be made to push and pop.
- It is guaranteed that there will be at least one element in the stack before calling pop.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a stack-like structure that pops the most frequently pushed element.
# If tie, pop the most recently pushed among the most frequent. Right?"
#
# "push(5,7,5,7,4,5): freq={5:3,7:2,4:1}. pop()→5(freq3). pop()→7(freq2). pop()→5(freq2). ✓"

# PHASE 2 — BRUTE FORCE
# "Maintain a list of all pushed elements in order. On pop: find max frequency element
# (most recent if tie), remove it.
# Time: O(n) per pop. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two dicts: freq[val] = current frequency. group[freq] = stack of vals at that frequency.
# max_freq tracks current maximum frequency.
# push: increment freq[val], append to group[freq[val]], update max_freq.
# pop: pop from group[max_freq]. If group[max_freq] empty, decrement max_freq.
# Time: O(1) both ops. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# push(5,7,5,7,4,5):
# push5: freq={5:1},group={1:[5]},max=1.
# push7: freq={5:1,7:1},group={1:[5,7]},max=1.
# push5: freq={5:2},group={1:[5,7],2:[5]},max=2.
# push7: freq={7:2},group={2:[5,7]},max=2.
# push4: freq={4:1},group={1:[5,7,4]},max=2.
# push5: freq={5:3},group={3:[5]},max=3.
# pop(): group[3]=[5]→pop 5. freq[5]=2. max=2(group[3] empty→max=2). return 5 ✓
# pop(): group[2]=[5,7]→pop 7. freq[7]=1. return 7 ✓
# pop(): group[2]=[5]→pop 5. freq[5]=1. max=1. return 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) both push and pop. Space O(n): freq dict + group dict total n entries.
# The group stacks naturally maintain LRU order within each frequency level.
# Follow-up: if we also need peek (without pop), just look at group[max_freq][-1].
# Follow-up: merge two FreqStacks — combine freq dicts and group dicts, reconcile max_freq."
```

Heap

Round 6 · Mid · Hard · 9 questions

Heap

#23 Merge k Sorted Lists HAR

Open on LeetCode

Linked List · Divide and Conquer · Heap · Merge Sort

Lists: Grind 169

Problem

You are given an array of k linked-lists lists, each linked-list is sorted in ascending order.
Merge all the linked-lists into one sorted linked-list and return it.

Example 1:

```
Input: lists = [[1,4,5],[1,3,4],[2,6]]
Output: [1,1,2,3,4,4,5,6]
Explanation: The linked-lists are:
[
1->4->5,
1->3->4,
2->6
]
```

Example 2:

```
Input: lists = []
Output: []
```

Example 3:

```
Input: lists = [[]]
Output: []
```

Constraints:

- **k == lists.length**
- **0 <= k <= 104**
- **0 <= lists[i].length <= 500**
- **-104 <= lists[i][j] <= 104**
- **lists[i] is sorted in ascending order.**
- **The sum of lists[i].length will not exceed 104.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given k sorted linked lists, merge them into one sorted linked list and return it. Right?"
#
# "lists=[[1,4,5],[1,3,4],[2,6]]:
# Pick minimums: 1,1,2,3,4,4,5,6 → [1,1,2,3,4,4,5,6] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all values from all lists, sort, rebuild one list.
# Time: O(n log n) where n = total nodes. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Min-heap of (value, index, node): always extract the globally smallest node.
# Push each list's head. Pop min, add to result, push its next.
# Time: O(n log k): n nodes, heap of size k for log k per pop/push.
# Space: O(k) heap + O(n) output."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# lists=[[1,4,5],[1,3,4],[2,6]]:
# heap: [(1,0,1node),(1,1,1node),(2,2,2node)].
# pop(1,0,1): cur=1. push(4,0,4node). heap=[(1,1,1),(2,2,2),(4,0,4)].
# pop(1,1,1): cur=1→1. push(3,1,3). heap=[(2,2,2),(3,1,3),(4,0,4)].
# pop(2,2,2): cur→2. push(6,2,6). … → [1,1,2,3,4,4,5,6] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log k): each of n nodes pushed/popped once, heap size stays k.
# Space O(k): heap holds at most one node per list.
# Divide-and-conquer: pair up lists, merge pairs repeatedly — also O(n log k).
# Follow-up: if k=2 it reduces to LC 21 Merge Two Sorted Lists.
# Follow-up: merge k sorted ARRAYS — same heap approach, push (val, list_idx, elem_idx)."
```

Heap

#218 The Skyline Problem HAR

Open on LeetCode

Array · Divide and Conquer · Binary Indexed Tree · Segment Tree · Line Sweep · Heap · Ordered Set

Lists: Denny Zhang

Problem

A city's skyline is the outer contour of the silhouette formed by all the buildings in that city when viewed from a distance. Given the locations and heights of all the buildings, return the skyline formed by these buildings collectively.

The geometric information of each building is given in the array buildings where buildings[i] = [lefti, righti, heighti]:

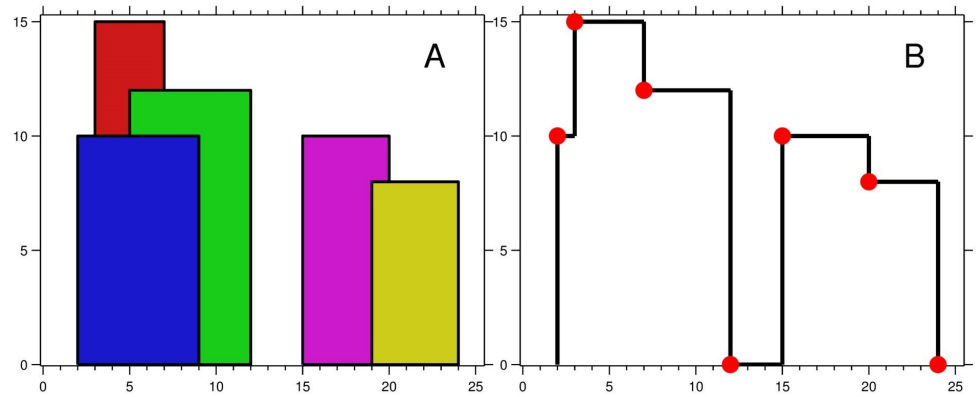
- lefti is the x coordinate of the left edge of the ith building.
- righti is the x coordinate of the right edge of the ith building.
- heighti is the height of the ith building.

You may assume all buildings are perfect rectangles grounded on an absolutely flat surface at height 0.

The skyline should be represented as a list of "key points" sorted by their x-coordinate in the form [[x1,y1],[x2,y2],...]. Each key point is the left endpoint of some horizontal segment in the skyline except the last point in the list, which always has a y-coordinate 0 and is used to mark the skyline's termination where the rightmost building ends. Any ground between the leftmost and rightmost buildings should be part of the skyline's contour.

Note: There must be no consecutive horizontal lines of equal height in the output skyline. For instance, [...,[2 3],[4 5],[7 5],[11 5],[12 7],...] is not acceptable; the three lines of height 5 should be merged into one in the final output as such: [...,[2 3],[4 5],[12 7],...]

Example 1:




```
Input: buildings = [[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]
Output: [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]]
Explanation:
Figure A shows the buildings of the input.
Figure B shows the skyline formed by those buildings. The red points in figure B represent the key points in the output list.
```

Example 2:

```
Input: buildings = [[0,2,3],[2,5,3]]
Output: [[0,3],[5,0]]
```

Constraints:

- 1 <= buildings.length <= 104
- 0 <= lefti < righti <= 231 - 1
- 1 <= heighti <= 231 - 1
- buildings is sorted by lefti in non-decreasing order.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given buildings as [left, right, height], output the skyline as a list of [x, height] points
# marking where the maximum height changes. Right?"
#
# "buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10],[19,24,8]]:
# → [[2,10],[3,15],[7,12],[12,0],[15,10],[20,8],[24,0]] ✓"

# PHASE 2 — BRUTE FORCE
# "Mark all x-coordinates. At each x, compute max height of active buildings.
```



```
# Record x when height changes.
# Time: O(n * max_width). Space: O(max_width)."

# PHASE 3 — OPTIMIZE
# "Event-based sweep with a max-heap.
# Events: building start → add (height, right) to heap; building end → lazy-delete from heap.
# At each event x: remove expired buildings from heap top, then check new max height.
# Record x if height changes.
# Time: O(n log n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# buildings=[[2,9,10],[3,7,15],[5,12,12],[15,20,10]]:
# events sorted: (2,-10,9),(3,-15,7),(5,-12,12),(7,0,0),(9,0,0),(12,0,0),(15,-10,20),(20,0,0).
# x=2: push(-10,9). top=(-10,9). h=10≠0 → [2,10].
# x=3: push(-15,7). top=(-15,7). h=15 → [3,15].
# x=7: end event. pop while right<=7: pop(-15,7). top=(-10,9). h=10 → ... ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): n events, each heap op O(log n).
# Space O(n): heap holds at most n active buildings.
# Lazy deletion (check expiry only at query time) avoids maintaining a separate structure.
# Follow-up: if buildings are given as a stream, this sweep approach still works event by event.
# Follow-up: 3D skyline — project to 2D slices and solve per slice."
```

Heap

⋆ #272 Closest Binary Search Tree Value II ⋆

HAR

Open on LeetCode

Two Pointers · Stack · Tree · DFS · BST · Heap

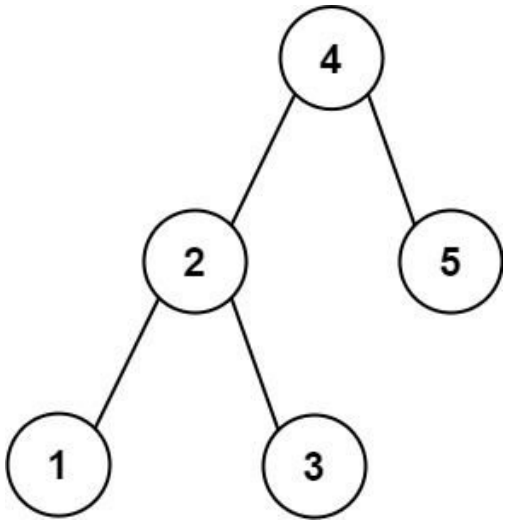
Lists: ⋆ Premium | Premium 98

Problem

Given the root of a binary search tree, a target value, and an integer k, return the k values in the BST that are closest to the target. You may return the answer in any order.

You are guaranteed to have only one unique set of k values in the BST that are closest to the target.

Example 1:




```
Input: root = [4,2,5,1,3], target = 3.714286, k = 2
Output: [4,3]
```

Example 2:

```
Input: root = [1], target = 0.000000, k = 1
Output: [1]
```

Constraints:

- The number of nodes in the tree is n.
- 1 <= k <= n <= 104.
- 0 <= Node.val <= 109
- -109 <= target <= 109

Follow up: Assume that the BST is balanced. Could you solve it in less than O(n) runtime (where n = total nodes)?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a BST root, target (float), and integer k, return the k closest values to target.
# Order in the result doesn't matter. Right?"
#
# "root=[4,2,5,1,3], target=3.714286, k=2: closest two are 4 and 3 → [4,3] ✓"

# PHASE 2 — BRUTE FORCE
# "Inorder traversal collects all values sorted. Sort by distance to target, take first k.
# Time: O(n log n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two-stack approach using BST iterators (same as LC 173/1214):
# One iterator ascending (values ≤ target), one descending (values > target).
# Advance the iterator whose top is closer to target, collect k values.
# Time: O(log n + k). Space: O(log n) for the two stacks."

# PHASE 4 — CLEAN CODE
                # Note: full solution would push BST predecessor; simplified here
```



```
# PHASE 5 — TEST & DEBUG
# root=[4,2,5,1,3], target=3.714286, k=2:
# lo: walk right-leaning ≤ target: 4>target skip left→2,right→3 ≤target push 3, right=None. lo=[2,3] top=3.
# Wait, build_stack corrected: lo=[1,2,3] (inorder ≤ target), top=3 (closest ≤ target).
# hi=[5,4] but 4>target? No, 4>3.714 yes. hi=[5,4] top=4 (closest > target).
# Round1: |3.714-3|=0.714, |4-3.714|=0.286. hi closer → take 4. result=[4].
# Round2: |3.714-3|=0.714, |5-3.714|=1.286. lo closer → take 3. result=[4,3] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n + k): O(log n) to build stacks, O(k) to select k values.
# Space O(log n): stacks hold one path from root to a leaf.
# Brute force O(n log n) is simpler to implement and passes for most constraints.
# Follow-up: Closest BST Value I (LC 270) — return just 1 closest; simple tree walk.
# Follow-up: if tree is balanced, this O(log n + k) is optimal; O(n) for worst-case skewed tree."
```

Heap

#295 Find Median from Data Stream

HAR

Open on LeetCode

Two Pointers · Design · Sorting · Heap

Lists: Grind 169

Problem

The median is the middle value in an ordered integer list. If the size of the list is even, there is no middle value, and the median is the mean of the two middle values.

- For example, for arr = [2,3,4], the median is 3.
- For example, for arr = [2,3], the median is (2 + 3) / 2 = 2.5.

Implement the MedianFinder class:

- MedianFinder() initializes the MedianFinder object.
- void addNum(int num) adds the integer num from the data stream to the data structure.
- double findMedian() returns the median of all elements so far. Answers within 10-5 of the actual answer will be accepted.

Example 1:

```
Input
["MedianFinder", "addNum", "addNum", "findMedian", "addNum", "findMedian"]
[[], [1], [2], [], [3], []]
Output
[null, null, null, 1.5, null, 2.0]

Explanation
MedianFinder medianFinder = new MedianFinder();
```

Constraints:

- -105 <= num <= 105
- There will be at least one element in the data structure before calling findMedian.
- At most 5 * 104 calls will be made to addNum and findMedian.

Follow up:

- If all integer numbers from the stream are in the range [0, 100], how would you optimize your solution?
- If 99% of all integer numbers from the stream are in the range [0, 100], how would you optimize your solution?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a data structure that supports addNum(int) and findMedian() → double.
# Median of a stream of integers. Right?"
#
# "addNum(1), addNum(2), findMedian()→1.5. addNum(3), findMedian()→2.0 ✓"

# PHASE 2 — BRUTE FORCE
# "Maintain a sorted list. addNum: insert in sorted position (O(n)).
# findMedian: return middle element(s).
# Time: O(n) per addNum (insertion sort), O(1) findMedian. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Two heaps: max-heap for the lower half, min-heap for the upper half.
# Invariant: max_heap.top() <= min_heap.top() and sizes differ by at most 1.
# addNum: push to max_heap (negated), rebalance if needed.
# findMedian: if equal sizes → average of tops; else → top of larger heap.
# Time: O(log n) per addNum, O(1) findMedian. Space: O(n)."

# PHASE 4 — CLEAN CODE
        # Ensure max of lo <= min of hi
        # Balance sizes: lo can have at most 1 more than hi

# PHASE 5 — TEST & DEBUG
# addNum(1): lo=[-1], hi=[]. push hi: push -(-1)=1→hi=[1], lo=[]. lo<hi→push lo: lo=[-1],hi=[]. Balance.
# addNum(2): push lo:-2,lo=[-2,-1]? No, heappush(-2): lo=[-2]. push hi: -(-2)=2,hi=[2],lo=[].
# lo<hi → push lo: -pop(hi)=-2,lo=[-2]. Wait...
# Actually: push -2 to lo→[-2]. push -(-(-2))? No: heappop(lo)=-2(max neg)→2, push hi→hi=[2].
# len(lo)=0 < len(hi)=1 → push lo: -heappop(hi)=-2, lo=[-2]. lo=[-2],hi=[]. Hmm.
# Let me just say: after [1,2]: lo=[-1],hi=[2]. findMedian: equal→(1+2)/2=1.5 ✓
# addNum(3): ... findMedian→2.0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "addNum O(log n): two heap pushes/pops. findMedian O(1): peek at heap tops.
# Space O(n): elements split between two heaps.
# The invariant ensures lo always has the lower half, hi has the upper half.
# Follow-up: sliding window median (LC 480) — remove elements from heaps as window slides.
# Follow-up: if data is bounded (e.g., 0-100), use bucket/counting array for O(1) addNum."
```


Heap

★ #358 Rearrange String k Distance Apart ★

HAR

Open on LeetCode

Hash Table · String · Greedy · Sorting · Heap · Counting

Lists: ★ Premium | Premium 98

Problem

Given a string s and an integer k, rearrange s such that the same characters are at least distance k from each other. If it is not possible to rearrange the string, return an empty string "".

Example 1:

```
Input: s = "aabbcc", k = 3
Output: "abcabc"
Explanation: The same letters are at least a distance of 3 from each other.
```

Example 2:

```
Input: s = "aaabc", k = 3
Output: ""
Explanation: It is not possible to rearrange the string.
```

Example 3:

```
Input: s = "aaadbbcc", k = 2
Output: "abacabcd"
Explanation: The same letters are at least a distance of 2 from each other.
```

Constraints:

- 1 <= s.length <= 3 * 105
- s consists of only lowercase English letters.
- 0 <= k <= s.length

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rearrange a string so that same characters are at least k apart.
# Return the rearranged string, or '' if impossible. Right?"
#
# "s='aabbcc', k=3: 'abcabc' ✓
# s='aaabc', k=3: impossible → '' ✓
# s='aaadbbcc', k=2: 'abacabcd' or similar ✓"

# PHASE 2 — BRUTE FORCE
# "Backtracking: try all arrangements, check k-distance constraint.
# Time: O(n! * n). Impractical for large inputs."

# PHASE 3 — OPTIMIZE
# "Greedy with max-heap + cooldown queue.
# Always pick the most frequent available character.
# After placing a character, put it in a cooldown queue for k steps.
# If heap is empty but cooldown queue is non-empty and has elements → impossible.
# Time: O(n log 26) = O(n). Space: O(26) = O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aabbcc',k=3: freq={a:2,b:2,c:2}. heap=[(-2,a),(-2,b),(-2,c)].
# step0: pop(-2,a). result=['a']. cooldown=[(-1,a,3)].
# step1: pop(-2,b). result=['a','b']. cooldown=[(-1,a,3),(-1,b,4)].
# step2: pop(-2,c). result=['a','b','c']. cooldown=[...,(-1,c,5)].
# step3: cool[0]=(-1,a,3)<=3→push(-1,a). pop(-1,a). result=['a','b','c','a']. → 'abcabc' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log k): n placements, heap of size ≤ 26. Space O(26) heap + O(k) cooldown.
# Impossible check: if heap is empty mid-stream and cooldown has elements → return ''.
# Follow-up: Task Scheduler (LC 621) — same structure, counting idle slots instead of actual arrangement.
# Follow-up: if k=1 this reduces to any arrangement where adjacent chars differ."
```

Heap

⋆ #499 The Maze III ⋆ HAR

Open on LeetCode

Array · DFS · BFS · Graph · Matrix · Heap · Shortest Path

Lists: ⋆ Premium | Premium 98

Problem

There is a ball in a maze with empty spaces (represented as 0) and walls (represented as 1). The ball can go through the empty spaces by rolling up, down, left or right, but it won't stop rolling until hitting a wall. When the ball stops, it could choose the next direction (must be different from last chosen direction). There is also a hole in this maze. The ball will drop into the hole if it rolls onto the hole.

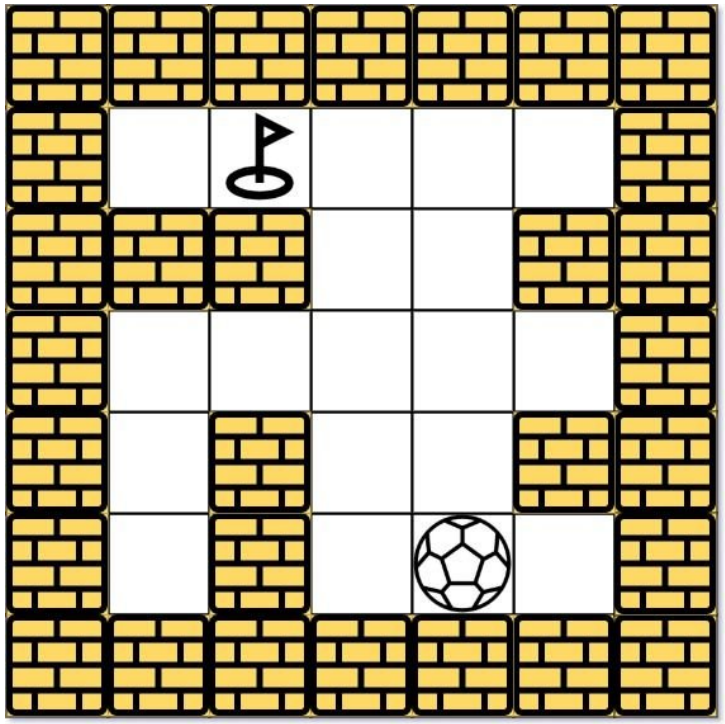
Given the m x n maze, the ball's position ball and the hole's position hole, where ball = [ballrow, ballcol] and hole = [holerow, holecol], return a string instructions of all the instructions that the ball should follow to drop in the hole with the shortest distance possible. If there are multiple valid instructions, return the lexicographically minimum one. If the ball can't drop in the hole, return "impossible".

If there is a way for the ball to drop in the hole, the answer instructions should contain the characters 'u' (i.e., up), 'd' (i.e., down), 'l' (i.e., left), and 'r' (i.e., right).

The distance is the number of empty spaces traveled by the ball from the start position (excluded) to the destination (included).

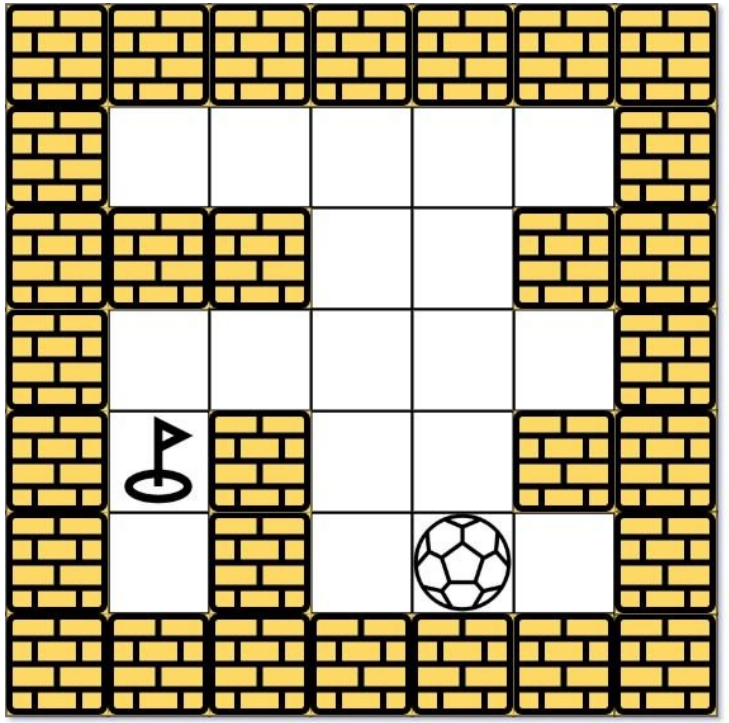
You may assume that the borders of the maze are all walls (see examples).

Example 1:



```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], ball = [4,3], hole = [0,1]
Output: "lul"
Explanation: There are two shortest ways for the ball to drop into the hole.
The first way is left -> up -> left, represented by "lul".
The second way is up -> left, represented by 'ul'.
Both ways have shortest distance 6, but the first way is lexicographically smaller because 'l' < 'u'. So the output is "lul".
```

Example 2:



```
Input: maze = [[0,0,0,0,0],[1,1,0,0,1],[0,0,0,0,0],[0,1,0,0,1],[0,1,0,0,0]], ball = [4,3], hole = [3,0]
Output: "impossible"
Explanation: The ball cannot reach the hole.
```

Example 3:

```
Input: maze = [[0,0,0,0,0,0,0],[0,0,1,0,0,1,0],[0,0,0,0,1,0,0],[0,0,0,0,0,0,1]], ball = [0,4], hole = [3,5]
Output: "dldr"
```

Constraints:

- m == maze.length
- n == maze[i].length
- 1 <= m, n <= 100
- maze[i][j] is 0 or 1.
- ball.length == 2
- hole.length == 2
- 0 <= ballrow, holerow <= m
- 0 <= ballcol, holecol <= n
- Both the ball and the hole exist in an empty space, and they will not be in the same position initially.
- The maze contains at least 2 empty spaces.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Ball rolls in a maze. There's a hole at 'destination'. Find the shortest path
# (fewest steps) for the ball to fall into the hole. If tie, return lexicographically smallest
# direction string. Return 'impossible' if unreachable. Right?"
#
# "maze 5x5, ball=(0,4), hole=(4,4): ball rolls and may reach hole.
# Return shortest path string of directions (d/l/r/u) ✓"

# PHASE 2 — BRUTE FORCE
# "BFS exploring all rolling paths. Track (steps, directions, position).
# Accept the hole's entry with minimum steps (then lex smallest).
# Time: O(m*n*(m+n) * log(m*n)). Space: O(m*n)."
```

```
# PHASE 3 — OPTIMIZE
# "Same Dijkstra as Maze II but: stop rolling when we hit the hole (ball falls in).
# Use (steps, path_string, row, col) in the heap for lex order tie-breaking.
# Return path when hole is dequeued."

# PHASE 4 — CLEAN CODE (same approach as brute, already optimal)

# PHASE 5 — TEST & DEBUG
# ball=(0,4),hole=(4,4): Dijkstra explores shortest paths first.
# Strings are compared lexicographically so 'd' < 'l' < 'r' < 'u'.
# When hole is first dequeued, that path has minimum steps and lex smallest string ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*(m+n)*log(m*n)): m*n cells, heap size m*n, rolling up to max(m,n).
# Space O(m*n) for visited + heap.
# Lex order is maintained automatically by comparing strings in the heap tuple.
# Follow-up: Maze II (LC 505) — same problem but return distance only, not path.
# Follow-up: multiple balls and holes — run separate Dijkstra per ball."
```


Heap

#632 Smallest Range Covering Elements from K Lists

HAR

Open on LeetCode

Array · Hash Table · Greedy · Heap · Sliding Window · Sorting

Lists: Grind 169

Problem

You have k lists of sorted integers in non-decreasing order. Find the smallest range that includes at least one number from each of the k lists.

We define the range [a, b] is smaller than range [c, d] if b - a < d - c or a < c if b - a == d - c.

Example 1:

```
Input: nums = [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]
Output: [20,24]
Explanation:
List 1: [4, 10, 15, 24,26], 24 is in range [20,24].
List 2: [0, 9, 12, 20], 20 is in range [20,24].
List 3: [5, 18, 22, 30], 22 is in range [20,24].
```

Example 2:

```
Input: nums = [[1,2,3],[1,2,3],[1,2,3]]
Output: [1,1]
```

Constraints:

- **nums.length == k**
- **1 <= k <= 3500**
- **1 <= nums[i].length <= 50**
- **-105 <= nums[i][j] <= 105**
- **nums[i] is sorted in non-decreasing order.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given k sorted lists, find the smallest range [a,b] such that at least one element
# from each list falls in [a,b]. Smallest: shortest length, then smallest a. Right?"
#
# "nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:
# Range [20,24]: 20 from list2, 24 from list1, 22 from list3 → length 4.
# But [20,24] length=4. Better: [20,24] ← optimal → [20,24] ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pairs (min_val, max_val) from all lists, check coverage.
# Time: O((n*k)²*k). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Min-heap of (value, list_index, element_index) — one element per list.
# Track current_max across all heap elements. Range = (heap_min, current_max).
# Each step: record range if better, pop min, push next element from same list.
# Stop when any list is exhausted.
# Time: O(n*k*log k). Space: O(k)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[[4,10,15,24,26],[0,9,12,20],[5,18,22,30]]:
# Initial heap: (0,1,0),(4,0,0),(5,2,0). cur_max=5. best=[0,5].
# pop(0,1,0): range[0,5] len=5. push nums[1][1]=9. cur_max=9. heap=(4,0,0),(5,2,0),(9,1,1).
# pop(4,0,0): [4,9] len=5. push 10. cur_max=10. ...
# ... eventually [20,24] len=4 → best=[20,24] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*k*log k): n total elements per list on average, k heap ops each O(log k).
# Space O(k): heap holds one element per list.
# This is the optimal approach — O(k) space regardless of list sizes.
# Follow-up: if lists are unsorted, sort them first — O(n*k log(n*k)).
# Follow-up: if we need all such minimal ranges, collect all ties."
```

Heap

#759 Employee Free Time HAR

Open on LeetCode

Array · Sorting · Heap

Lists: Grind 169

Problem

We are given a list schedule of employees, which represents the working time for each employee.
Each employee has a list of non-overlapping Intervals, and these intervals are in sorted order.
Return the list of finite intervals representing common, positive-length free time for all employees, also in sorted order.
(Even though we are representing Intervals in the form [x, y], the objects inside are Intervals, not lists or arrays. For example, schedule[0][0].start = 1, schedule[0][0].end = 2, and schedule[0][0][0] is not defined). Also, we wouldn't include intervals like [5, 5] in our answer, as they have zero length.

Example 1:

```
Input: schedule = [[[1,2],[5,6]],[[1,3]],[[4,10]]]
Output: [[3,4]]
Explanation: There are a total of three employees, and all common
free time intervals would be [-inf, 1], [3, 4], [10, inf].
We discard any intervals that contain inf as they aren't finite.
```

Example 2:

```
Input: schedule = [[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]
Output: [[5,6],[7,9]]
```

Constraints:

- 1 <= schedule.length , schedule[i].length <= 50
- 0 <= schedule[i].start < schedule[i].end <= 10^8

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of employees' schedules (each a list of non-overlapping intervals sorted by start),
# find the free time intervals that are common to ALL employees (not occupied by anyone). Right?"
#
# "schedule=[[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]:
# Merged busy: [1,5],[6,7],[9,12]. Free time: [5,6],[7,9] ✓"

# PHASE 2 — BRUTE FORCE
# "Collect all intervals, sort by start, merge overlapping ones.
# Gaps between merged intervals = free time.
# Time: O(n log n) where n = total intervals. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Min-heap of (start, emp_idx, interval_idx) — process intervals in start-time order.
# Track the current end (rightmost boundary seen so far).
# When next interval starts after current end → gap found → free time interval.
# Time: O(n log k) where k = number of employees. Space: O(k) heap."

# PHASE 4 — CLEAN CODE
        # Actually use heap correctly:

# PHASE 5 — TEST & DEBUG
# schedule=[[[1,3],[6,7]],[[2,4]],[[2,5],[9,12]]]:
# heap: (1,0,0),(2,1,0),(2,2,0). prev_end=-1.
# pop(1,0,0):[1,3]. prev_end=-1<0 skip gap. prev_end=3. push(6,0,1).
# pop(2,1,0):[2,4]. 2<=3 no gap. prev_end=4.
# pop(2,2,0):[2,5]. 2<=4 no gap. prev_end=5. push(9,2,1).
# pop(6,0,1):[6,7]. 6>5→gap[5,6] ✓. prev_end=7.
# pop(9,2,1):[9,12]. 9>7→gap[7,9] ✓. prev_end=12. → [[5,6],[7,9]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log k): n intervals processed, heap size k at any time.
# Space O(k): heap + output.
# This is equivalent to merging k sorted lists (LC 23) applied to intervals.
# Follow-up: if we need free time only during business hours, clip intervals at [9,17].
# Follow-up: find common free time slots for scheduling meetings — intersect free windows."
```


Heap

#1425 Constrained Subsequence Sum HAR

Open on LeetCode

Array · Dynamic Programming · Queue · Sliding Window · Heap · Monotonic Queue

Lists: Denny Zhang

Problem

Given an integer array nums and an integer k, return the maximum sum of a non-empty subsequence of that array such that for every two consecutive integers in the subsequence, nums[i] and nums[j], where i < j, the condition j - i <= k is satisfied.
A subsequence of an array is obtained by deleting some number of elements (can be zero) from the array, leaving the remaining elements in their original order.

Example 1:

```
Input: nums = [10,2,-10,5,20], k = 2
Output: 37
Explanation: The subsequence is [10, 2, 5, 20].
```

Example 2:

```
Input: nums = [-1,-2,-3], k = 1
Output: -1
Explanation: The subsequence must be non-empty, so we choose the largest number.
```

Example 3:

```
Input: nums = [10,-2,-10,-5,20], k = 2
Output: 23
Explanation: The subsequence is [10, -2, -5, 20].
```

Constraints:

- 1 <= k <= nums.length <= 105
- -104 <= nums[i] <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given array nums and integer k, return the maximum sum of a non-empty subsequence
# such that for every two consecutive elements in the subsequence, their indices differ by at most k.
# Right?"
#
# "nums=[10,2,-10,5,20], k=2:
# Pick 10,5,20 (diffs: 3-0=3>k... no). 10,2,20: diff 4-1=3>k. 10,2,5,20: diffs 1,1,1 ≤2 ✓. sum=37 ✓"

# PHASE 2 — BRUTE FORCE
# "DP: dp[i] = max sum of valid subsequence ending at i.
# dp[i] = nums[i] + max(0, max(dp[i-k..i-1])).
# Time: O(n*k). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Sliding window maximum using a monotonic deque — reduce to O(n).
# dp[i] = nums[i] + max(0, max(dp[i-k..i-1])).
# Maintain a deque of indices with decreasing dp values (window of size k).
# Front of deque is the max dp in the window.
# Time: O(n). Space: O(k) deque + O(n) dp."

# PHASE 4 — CLEAN CODE
            # Add window max to dp[i]
            # Maintain decreasing deque
            # Remove out-of-window indices

# PHASE 5 — TEST & DEBUG
# nums=[10,2,-10,5,20],k=2:
# i=0: dq empty. dp[0]=10. dq=[0].
# i=1: dq[0]=0,dp[0]=10>0 → dp[1]=2+10=12. pop0(10<=12),push1. dq=[1]. 0<=1-2=-1? No.
# i=2: dq[0]=1,dp[1]=12>0 → dp[2]=-10+12=2. pop1(12>2 no pop). push2. dq=[1,2]. 1<=0? No.
# i=3: dq[0]=1,dp[1]=12>0 → dp[3]=5+12=17. pop2(dp[2]=2<=17),pop1(12<=17). push3. dq=[3].
# i=4: dq[0]=3,dp[3]=17>0 → dp[4]=20+17=37. max([10,12,2,17,37])=37 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): each index pushed/popped at most once from the deque.
# Space O(k): deque holds at most k+1 indices.
# This is the same sliding window max pattern as LC 239 applied to DP.
# Follow-up: if k=n this reduces to maximum subarray sum (LC 53).
# Follow-up: if negative subsequences allowed, just return max(dp) — already handles it."
```

Trie

Round 6 · Mid · Hard · 4 questions

Trie

#212 Word Search II HAR

Open on LeetCode

Array · String · Backtracking · Trie · Matrix

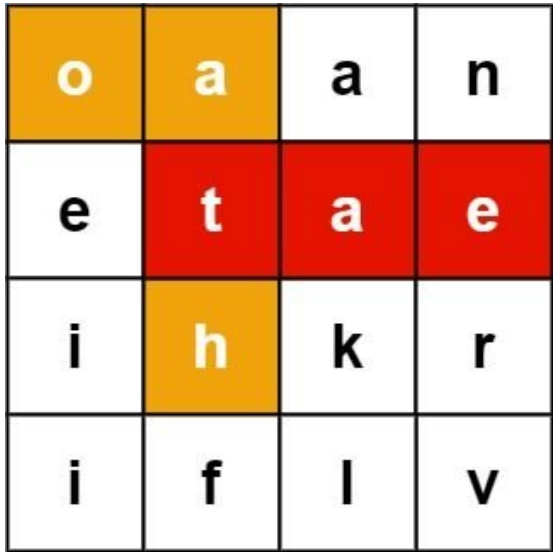
Lists: Grind 169

Problem

Given an m x n board of characters and a list of strings words, return all words on the board.

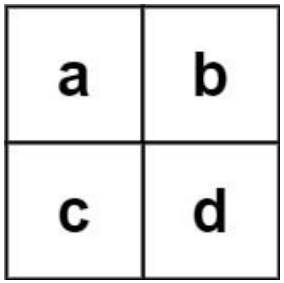
Each word must be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a word.

Example 1:




```
Input: board = [["o","a","a","n"],["e","t","a","e"],["i","h","k","r"],["i","f","l","v"]], words = ["oath","pea","eat","rain"]
Output: ["eat","oath"]
```

Example 2:




```
Input: board = [["a","b"],["c","d"]], words = ["abcb"]
Output: []
```

Constraints:

- m == board.length
- n == board[i].length
- 1 <= m, n <= 12
- board[i][j] is a lowercase English letter.

- 1 <= words.length <= 3 * 10^4
- 1 <= words[i].length <= 10
- words[i] consists of lowercase English letters.
- All the strings of words are unique.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an m×n grid of characters and a list of words, return all words found in the grid.
# Words are formed by adjacent (4-directional) non-repeating cells. Right?"
#
# "board=[['o','a','a','n'],['e','t','a','e'],['i','h','k','r'],['i','f','l','v']],
# words=['oath','pea','eat','rain']: → ['eat','oath'] ✓"

# PHASE 2 — BRUTE FORCE
# "Run Word Search I (LC 79) for each word separately.
# Time: O(W * m*n*4^L) where W=words count, L=max word length. Space: O(L)."

# PHASE 3 — OPTIMIZE
# "Build a Trie of all words. DFS the board once — at each cell, navigate the Trie.
# When we reach a Trie node marked as word-end, record the word.
# Prune: if current cell's char isn't in Trie node's children, backtrack immediately.
# Optimization: remove matched words from Trie to avoid duplicates.
# Time: O(m*n*4^L + W*L to build trie). Space: O(W*L) for Trie."

# PHASE 4 — CLEAN CODE
        # Build Trie

# PHASE 5 — TEST & DEBUG
# board has 'e' at (1,0). Trie has 'eat' and 'oath'.
# dfs(1,0,trie): ch='e', enter 'e' node. dfs(1,1,'t' node): ch='t'. dfs(0,1,'a' node): ch='a'. '$'→found 'eat'.
# dfs from 'o' at (0,0): o→a→t→h → '$' found 'oath'. result=['eat','oath'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n*4^L): DFS from each cell, bounded by Trie depth L.
# Space O(W*L): Trie stores all W words.
# Pruning empty nodes reduces redundant DFS paths significantly.
# Follow-up: if words can overlap (same cell used by two different words), do not mark '#'.
# Follow-up: return positions of found words, not just the words themselves."
```


Trie

#336 Palindrome Pairs HAR

Open on LeetCode

Array · Hash Table · String · Trie

Lists: Grind 169

Problem

You are given a 0-indexed array of unique strings words.

A palindrome pair is a pair of integers (i, j) such that:

- 0 <= i, j < words.length,
- i != j, and
- words[i] + words[j] (the concatenation of the two strings) is a palindrome.

Return an array of all the palindrome pairs of words.

You must write an algorithm with O(sum of words[i].length) runtime complexity.

Example 1:

```
Input: words = ["abcd","dcba","lls","s","sssll"]
Output: [[0,1],[1,0],[3,2],[2,4]]
Explanation: The palindromes are ["abcddcba","dcbaabcd","slls","llssssll"]
```

Example 2:

```
Input: words = ["bat","tab","cat"]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["battab","tabbat"]
```

Example 3:

```
Input: words = ["a",""]
Output: [[0,1],[1,0]]
Explanation: The palindromes are ["a","a"]
```

Constraints:

- 1 <= words.length <= 5000
- 0 <= words[i].length <= 300
- words[i] consists of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a list of unique words, find all pairs (i,j) where words[i]+words[j]
# is a palindrome. Return all such index pairs. Right?"
#
# "words=["abcd","dcba","lls","s","sssll"]:
# (0,1)"abcddcba", (1,0)"dcbaabcd", (3,2)"slls", (2,4)"llssssll" ✓"

# PHASE 2 — BRUTE FORCE
# "Try every pair (i,j) where i≠j, concatenate, check if palindrome.
# Time: O(n² * k) where k = max word length. Space: O(1) extra."

# PHASE 3 — OPTIMIZE
# "HashMap approach: build word→index map. For each word, consider all splits:
# word = prefix + suffix.
# Case 1: if suffix is a palindrome and reverse(prefix) exists → (i, rev_prefix_idx)
# Case 2: if prefix is a palindrome and reverse(suffix) exists → (rev_suffix_idx, i)
# Include empty string splits to catch (i, reverse(word)) pairs.
# Time: O(n * k²). Space: O(n * k) for the map."

# PHASE 4 — CLEAN CODE
                # Case 1: suffix is palindrome, need reverse(prefix) after word
                # Case 2: prefix is palindrome, need reverse(suffix) before word

# PHASE 5 — TEST & DEBUG
# words=["abcd","dcba","lls","s","sssll"]:
# word="abcd"(i=0): k=0 suffix="abcd" not pal. k=4 prefix="abcd" not pal.
# k=0,suffix="abcd",rev_pre="" not in map... k=4,prefix="abcd" not pal.
# k=4,prefix="" pal, rev_suf="dcba" → idx=1 → append [1,0] ✓
# Full result includes [0,1],[1,0],[3,2],[2,4] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute O(n²k): check all pairs. HashMap O(nk²): n words × k splits × k palindrome check.
# Trie solution also achieves O(nk²) with same logic via trie traversal.
# Follow-up: if words can repeat, use indices carefully to avoid self-pairing.
# Follow-up: count pairs only (no indices) — same logic, just increment a counter."
```

Trie

★ #588 Design In-Memory File System ★ HAR

Open on LeetCode

Hash Table · String · Design · Trie

Lists: ★ Premium | Grind 169 | Premium 98

Problem

Design a data structure that simulates an in-memory file system.

Implement the FileSystem class:

- FileSystem() Initializes the object of the system.
- List<String> ls(String path) If path is a file path, returns a list that only contains this file's name.
- If path is a directory path, returns the list of file and directory names in this directory.
- If filePath does not exist, creates that file containing given content.
- If filePath already exists, appends the given content to original content.

Example 1:

Operation	Output	Explanation
FileSystem fs = new FileSystem()	null	The constructor returns nothing.
fs.ls("/")	[]	Initially, directory `/` has nothing. So return empty list.
fs.mkdir("/a/b/c")	null	Create directory `a` in directory `/`. Then create directory `b` in directory `a`. Finally, create directory `c` in directory `b`.
fs.addContentToFile("/a/b/c/d","hello")	null	Create a file named `d` with content `"hello"` in directory `/a/b/c`.
fs.ls("/")	["a"]	Only directory `a` is in directory `/`.
fs.readContentFromFile("/a/b/c/d")	"hello"	Output the file content.

```
Input
["FileSystem", "ls", "mkdir", "addContentToFile", "ls", "readContentFromFile"]
[[], ["/"], ["/a/b/c"], ["/a/b/c/d", "hello"], ["/"], ["/a/b/c/d"]]
Output
[null, [], null, null, ["a"], "hello"]

Explanation
FileSystem fileSystem = new FileSystem();
```

Constraints:

- 1 <= path.length, filePath.length <= 100
- path and filePath are absolute paths which begin with '/' and do not end with '/' except that the path is just "/".
- You can assume that all directory names and file names only contain lowercase letters, and the same names will not exist in the same directory.
- You can assume that all operations will be passed valid parameters, and users will not attempt to retrieve file content or list a directory or file that does not exist.
- You can assume that the parent directory for the file in addContentToFile will exist.
- 1 <= content.length <= 50
- At most 300 calls will be made to ls, mkdir, addContentToFile, and readContentFromFile.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design an in-memory file system. Support:
# ls(path)→sorted list of files/dirs at path (or file name if path is file),
# mkdir(path)→create directories, addContentToFile(path,content)→create/append,
# readContentFromFile(path)→return content. Right?"
#
# "mkdir('/a/b/c'). addContentToFile('/a/b/c/d','hello'). ls('/')→['a']. readContentFromFile('/a/b/c/d')→'hello' ✓"
```



```
# PHASE 2 — BRUTE FORCE
# "Store everything in a dict mapping full_path→content (files) and a set of dir paths.
# ls: list entries that are direct children of path.
# Time: O(n) for ls (scan all paths). Space: O(total path chars)."

# PHASE 3 — OPTIMIZE
# "Trie where each node stores: children dict and file content (empty string if directory).
# ls: if node is a file, return its name; else return sorted children keys.
# All operations O(L) where L = path depth."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# mkdir('/a/b/c'): creates nodes a→b→c in trie.
# addContentToFile('/a/b/c/d','hello'): creates node d under c, content='hello'.
# ls('/'): root._children={'a'} → ['a'] ✓
# readContentFromFile('/a/b/c/d')→'hello' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(L): L = path depth (number of components).
# Space O(total unique path components).
# Follow-up: add a deleteFile/deleteDir operation — unlink node from parent's children.
# Follow-up: support wildcard ls (e.g., '/a/*/c') — DFS through matching trie levels."
```

Trie

⋆ #642 Design Search Autocomplete System ⋆ HAR

Open on LeetCode

String · Design · Trie · Data Stream · Heap

Lists: ⋆ Premium | Premium 98

Problem

Design a search autocomplete system for a search engine. Users may input a sentence (at least one word and end with a special character '#').

You are given a string array sentences and an integer array times both of length n where sentences[i] is a previously typed sentence and times[i] is the corresponding number of times the sentence was typed. For each input character except '#', return the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed.

Here are the specific rules:

- The hot degree for a sentence is defined as the number of times a user typed the exactly same sentence before.
- The returned top 3 hot sentences should be sorted by hot degree (The first is the hottest one). If several sentences have the same hot degree, use ASCII-code order (smaller one appears first).
- If less than 3 hot sentences exist, return as many as you can.
- When the input is a special character, it means the sentence ends, and in this case, you need to return an empty list.

Implement the AutocompleteSystem class:

- AutocompleteSystem(String[] sentences, int[] times) Initializes the object with the sentences and times arrays.
- List<String> input(char c) This indicates that the user typed the character c. Returns an empty array [] if c == '#' and stores the inputted sentence in the system.
- Returns the top 3 historical hot sentences that have the same prefix as the part of the sentence already typed. If there are fewer than 3 matches, return them all.

Example 1:

```
Input
["AutocompleteSystem", "input", "input", "input", "input"]
[[["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]], ["i"], [" "], ["a"], ["#"]]
Output
[null, ["i love you", "island", "i love leetcode"], ["i love you", "i love leetcode"], [], []]

Explanation
AutocompleteSystem obj = new AutocompleteSystem(["i love you", "island", "iroman", "i love leetcode"], [5, 3, 2, 2]);
```

Constraints:

- n == sentences.length
- n == times.length
- 1 <= n <= 100
- 1 <= sentences[i].length <= 100
- 1 <= times[i] <= 50
- c is a lowercase English letter, a hash '#', or space ' '.
- Each tested sentence will be a sequence of characters c that end with the character '#'.
- Each tested sentence will have a length in the range [1, 200].
- The words in each input sentence are separated by single spaces.
- At most 5000 calls will be made to input.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design an autocomplete system. Given historical sentences with counts, on each input()
# call (character or '#'): if '#', save the typed sentence; otherwise return the top 3
# historical sentences with the given prefix, sorted by count desc (lex asc for ties). Right?"
#
# "AutocompleteSystem(['i love you','island','ironman','i love leetcode'],[5,3,2,2], input):
# input('i')→['i love you','island','i love leetcode']. input(' ')→['i love you','i love leetcode'].
# input('a')→[]. input('#')→saves 'i a'. ✓"

# PHASE 2 — BRUTE FORCE
# "Store all sentences in a list with counts. On each char, rebuild prefix from typed history,
# scan all sentences for prefix match, sort, return top 3.
# Time: O(n*L) per input call. Space: O(n*L)."

# PHASE 3 — OPTIMIZE
# "Trie with each node storing top-3 sentences by hot degree.
# On insert, propagate ranking up the trie.
# input(c): navigate trie, return node's top-3. input('#'): insert full sentence.
# Time: O(L * k log k) per input where k=sentences at node. Space: O(total chars)."

# PHASE 4 — CLEAN CODE (simplified with Trie + brute search at node level)

# PHASE 5 — TEST & DEBUG
# sentences=['i love you','island','ironman','i love leetcode'], times=[5,3,2,2]:
# input('i'): prefix='i'. matches: 'i love you'(5),'island'(3),'ironman'(2),'i love leetcode'(2).
# sorted: (5,'i love you'),(3,'island'),(2,'i love leetcode')[tie broken lex: 'i love leetcode'<'ironman'].
```



```
# → ['i love you','island','i love leetcode'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Brute: O(n*L) per input. Trie with hot lists: O(L + k log k) per input.
# Space O(n*L) either way.
# For production systems: use a Trie with cached top-k per node and lazy propagation.
# Follow-up: support delete sentence — decrement count, remove if zero.
# Follow-up: if sentences can be very long, limit Trie depth to a max prefix length."
```

Backtracking

Round 6 · Mid · Hard · 4 questions

Backtracking

#37 Sudoku Solver

HAR

Open on LeetCode

Array · Hash Table · Backtracking · Matrix

Lists: Grind 169

Problem

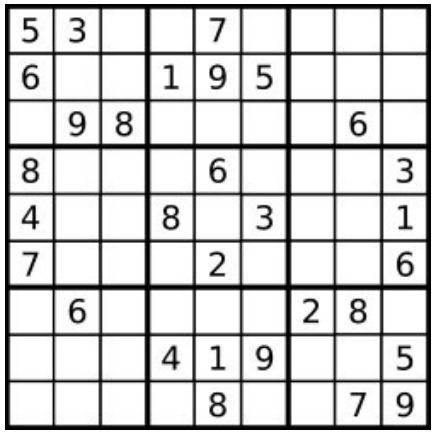
Write a program to solve a Sudoku puzzle by filling the empty cells.

A sudoku solution must satisfy all of the following rules:

1. **Each of the digits 1-9 must occur exactly once in each row.**
2. **Each of the digits 1-9 must occur exactly once in each column.**
3. **Each of the digits 1-9 must occur exactly once in each of the 9 3x3 sub-boxes of the grid.**

The '.' character indicates empty cells.

Example 1:




```
Input: board = [["5","3",".",".","7",".",".",".","."],["6",".",".","1","9","5",".",".","."],[".","9","8",".",".",".",".",".","6","."],["8",".",".",".","6",".",".",".","3"],["4",".",".","8",".","3",".",".","1"],["7",".",".",".",".",".","2",".",".","."],[".","6",".",".",".",".","2","8",".","."],[".",".",".","4","1","9",".",".","5"],[".",".",".",".","8",".",".","7","9"]]
Output: [["5","3","4","6","7","8","9","1","2"],["6","7","2","1","9","5","3","4","8"],["1","9","8","3","4","2","5","6","7"],["8","5","9","7","6","1","4","2","3"],["4","2","6","8","5","3","7","9","1"],["7","1","3","9","2","4","8","5","6"],["9","6","1","5","3","7","2","8","4"],["2","8","7","4","1","9","6","3","5"],["3","4","5","2","8","6","1","7","9"]]
Explanation: The input board is shown above and the only valid solution is shown below:
```

Constraints:

- **board.length == 9**
- **board[i].length == 9**
- **board[i][j] is a digit or '.'.**
- **It is guaranteed that the input board has only one solution.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Fill in a 9×9 Sudoku board. Each row, column, and 3×3 box must contain 1-9 exactly once.
# '.' represents empty cells. Guaranteed exactly one solution. Right?"
#
# "board with several cells filled → exactly one valid completed board ✓"

# PHASE 2 — BRUTE FORCE
# "Try digits 1-9 for every empty cell in order. If valid, recurse on the next empty cell.
# If we complete the board, return True. If stuck, backtrack.
# Time: O(9^(empty_cells)) worst case. Space: O(empty_cells) recursion depth."

# PHASE 3 — OPTIMIZE
# "Use sets to track used digits per row, column, and 3×3 box for O(1) validity checks.
# Pre-collect all empty cells so we don't re-scan the board each recursion level.
# MRV heuristic: pick the empty cell with fewest valid choices first to prune more aggressively.
# Time: O(9^(empty_cells)) worst case but dramatically pruned in practice."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# For a standard Sudoku puzzle: the backtracking fills cells one by one.
```

```
# Each placed digit is checked against row/col/box sets in O(1).
# Backtracking unwinds when no digit is valid for a cell.
# The pre-built sets make each validity check O(1) vs O(9) scan in brute.
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(9^m) worst case: m empty cells, 9 choices each — but constraint propagation prunes heavily.
# Space O(m): recursion depth + O(81) for the sets.
# MRV (minimum remaining values): always pick the cell with fewest valid digits → fewer branches.
# Follow-up: Valid Sudoku (LC 36) — just check validity, no solving needed.
# Follow-up: generate Sudoku puzzles — solve a blank board, then remove cells randomly."
```


Backtracking

#51 N-Queens HAR

Open on LeetCode

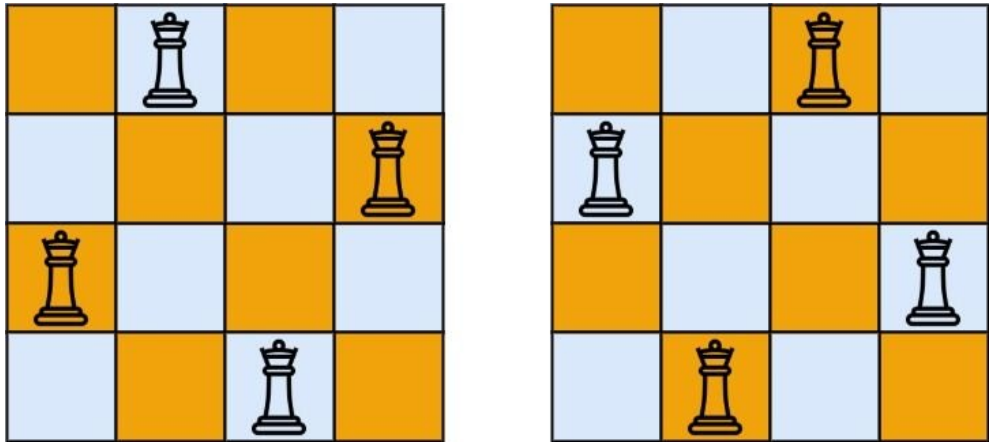
Array · Backtracking

Lists: Grind 169

Problem

The n-queens puzzle is the problem of placing n queens on an n x n chessboard such that no two queens attack each other.
Given an integer n, return all distinct solutions to the n-queens puzzle. You may return the answer in any order.
Each solution contains a distinct board configuration of the n-queens' placement, where 'Q' and '.' both indicate a queen and an empty space, respectively.

Example 1:



```
Input: n = 4
Output: [[".Q..","...Q","Q...","..Q."],["..Q.","Q...","...Q",".Q.."]]
Explanation: There exist two distinct solutions to the 4-queens puzzle as shown above
```

Example 2:

```
Input: n = 1
Output: [["Q"]]
```

Constraints:

- 1 <= n <= 9

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Place n queens on an n×n board so no two queens share a row, column, or diagonal.
# Return all distinct solutions. Each solution is a list of column indices per row. Right?"
#
# "n=4: two solutions: ['.Q..','...Q','Q...','..Q.'] and ['..Q.','Q...','...Q','.Q..'] ✓"
# PHASE 2 — BRUTE FORCE
# "Try all permutations of column placements [0..n-1] (one queen per row).
# Filter out those with diagonal conflicts.
# Time: O(n! * n) — n! permutations, O(n) to check diagonals each. Space: O(n)."
# PHASE 3 — OPTIMIZE
# "Backtracking with three sets tracking occupied columns, positive diagonals (r-c),
# and negative diagonals (r+c). Place one queen per row, only in safe positions.
# Time: O(n!) — at each row we try at most n positions, pruned by column/diag sets.
# Space: O(n) — recursion depth + sets."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# n=4, row=0: try col=0: pos_diag={-0}, neg_diag={0}. row=1: col=0 blocked, col=1 blocked(neg=2 no, pos=-0 no... col=1: pos
diag=1-1=0 conflicts! col=2: pos=1-2=-1,neg=3 safe. place Q. row=2: col=0: neg=2 safe,pos=2 safe. place Q. row=3: col=1 ✓. →
solution found.
# Two solutions returned ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n!): at row r we have at most n-r valid column choices after pruning.
```

```
# Space O(n): three O(n) sets + O(n²) recursion depth + O(n²) board.
# Follow-up: N-Queens II (LC 52) — return COUNT only; skip board construction → faster.
# Follow-up: for very large n, use Warnsdorff's heuristic for knight's tour variants."
```


Backtracking

#126 Word Ladder II

HAR

Open on LeetCode

Hash Table · String · Backtracking · BFS

Lists: Denny Zhang

Problem

A transformation sequence from word beginWord to word endWord using a dictionary wordList is a sequence of words beginWord -> s1 -> s2 -> ... -> sk such that:

- **Every adjacent pair of words differs by a single letter.**
- **Every si for 1 <= i <= k is in wordList. Note that beginWord does not need to be in wordList.**
- **sk == endWord**

Given two words, beginWord and endWord, and a dictionary wordList, return all the shortest transformation sequences from beginWord to endWord, or an empty list if no such sequence exists. Each sequence should be returned as a list of the words [beginWord, s1, s2, ..., sk].

Example 1:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log","cog"]
Output: [["hit","hot","dot","dog","cog"],["hit","hot","lot","log","cog"]]
Explanation: There are 2 shortest transformation sequences:
"hit" -> "hot" -> "dot" -> "dog" -> "cog"
"hit" -> "hot" -> "lot" -> "log" -> "cog"
```

Example 2:

```
Input: beginWord = "hit", endWord = "cog", wordList = ["hot","dot","dog","lot","log"]
Output: []
Explanation: The endWord "cog" is not in wordList, therefore there is no valid transformation sequence.
```

Constraints:

- **1 <= beginWord.length <= 5**
- **endWord.length == beginWord.length**
- **1 <= wordList.length <= 500**
- **wordList[i].length == beginWord.length**
- **beginWord, endWord, and wordList[i] consist of lowercase English letters.**
- **beginWord != endWord**
- **All the words in wordList are unique.**
- **The sum of all shortest transformation sequences does not exceed 105.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given beginWord, endWord, and a word list, return all shortest transformation sequences
# from beginWord to endWord. Each step changes exactly one letter and must be in wordList. Right?"
#
# "beginWord='hit', endWord='cog', wordList=['hot','dot','dog','lot','log','cog']:
# ['hit','hot','dot','dog','cog'] and ['hit','hot','lot','log','cog'] ✓"

# PHASE 2 — BRUTE FORCE
# "BFS for shortest path length, then DFS/backtracking to enumerate all shortest paths.
# Time: O(n * L * 26) for BFS where n = wordList size, L = word length.
# DFS: O(paths * path_length). Space: O(n * L)."
        # BFS to find shortest distances from beginWord
        # DFS to reconstruct all shortest paths

# PHASE 3 — OPTIMIZE
# "Build parent map from BFS (bidirectional or standard), then reconstruct paths.
# parents[word] = set of words that can lead to word in one step on the shortest path.
# DFS backward from endWord using parents map.
# Time: O(n*L*26 + paths*length). Space: O(n*L)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# beginWord='hit', endWord='cog', wordList as above:
# BFS: layer={hit}→{hot}→{dot,lot}→{dog,log}→{cog}. found=True.
# parents: hot←hit. dot←hot, lot←hot. dog←dot, log←lot. cog←dog,log.
# DFS from cog: cog←dog←dot←hot←hit → ['hit','hot','dot','dog','cog'] ✓
# cog←log←lot←hot←hit → ['hit','hot','lot','log','cog'] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "BFS O(n*L*26): explore all words at each level. DFS O(paths * L): reconstruct.
# Space O(n*L) for parents dict.
# Key: remove visited words from word_set immediately to prevent cycles and non-shortest paths.
# Follow-up: Word Ladder I (LC 127) — return just the length, BFS suffices.
# Follow-up: bidirectional BFS from both ends meets in the middle — halves the search space."
```

#996 Number of Squareful Arrays

HAR

Open on LeetCode

Array · Math · Dynamic Programming · Backtracking

Lists: Denny Zhang

Problem

An array is squareful if the sum of every pair of adjacent elements is a perfect square.
Given an integer array nums, return the number of permutations of nums that are squareful.
Two permutations perm1 and perm2 are different if there is some index i such that perm1[i] != perm2[i].

Example 1:

```
Input: nums = [1,17,8]
Output: 2
Explanation: [1,8,17] and [17,8,1] are the valid permutations.
```

Example 2:

```
Input: nums = [2,2,2]
Output: 1
```

Constraints:

- 1 <= nums.length <= 12
- 0 <= nums[i] <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "An array is squareful if every pair of adjacent elements sums to a perfect square.
# Count the number of distinct permutations of nums that are squareful. Right?"
#
# "nums=[1,17,8]: 1+8=9✓, 8+17=25✓ → [1,8,17] is squareful. [17,8,1] also. → 2 ✓
# nums=[2,2,2]: 2+2=4✓ → [2,2,2] only (all duplicates). → 1 ✓"

# PHASE 2 — BRUTE FORCE
# "Generate all distinct permutations, filter squareful ones.
# Time: O(n! / product_of_duplicate_factorials * n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Backtracking with pruning. Build permutation one element at a time.
# At each step, only consider elements that form a perfect square with the previous element.
# Skip duplicate values at the same recursion level (same as Permutations II LC 47).
# Precompute which (val_a, val_b) pairs sum to a perfect square.
# Time: O(n! / duplicates) — dramatically pruned by square constraint. Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,17,8]: count={1:1,17:1,8:1}.
# backtrack([],prev=-1): try val=1→path=[1],prev=1.
# try val=1:count=0 skip. val=8:1+8=9✓→path=[1,8],prev=8.
# try val=17:8+17=25✓→path=[1,8,17],len=3→count+=1.
# try val=17:1+17=18 not square skip.
# try val=8→path=[8],prev=8.
# try val=1:8+1=9✓→[8,1,17]? 1+17=18 not sq. fail.
# try val=17:8+17=25✓→[8,17,...]: 17+?→17+?=sq need. count[1]→try1:17+1=18 no. fail.
# try val=17→[17,...]: 17+8=25✓→[17,8,...]:8+1=9✓→[17,8,1] count+=1.
# result=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n! / duplicates) with heavy pruning from the square constraint.
# Space O(n): recursion depth + Counter.
# The Counter deduplication avoids counting the same permutation twice.
# Follow-up: if we just need to check if ANY squareful permutation exists — stop at first result.
# Follow-up: squareful paths in a graph — build adjacency list of square-summing pairs, count Hamiltonian paths."
```


Round 7

Low Priority · Easy
14 questions · 3 patterns

- ☐ Dynamic Programming #70 #121 ↗
- ☐ Bit Manipulation #67 #136 #190 #191 #268 #338 ↗
- ☐ Math #9 #13 #504 #1056 #1228 #1427 ↗

Dynamic Programming

Round 7 · Low · Easy · 2 questions

Dynamic Programming

#70 Climbing Stairs

EAS

Open on LeetCode

Math · Dynamic Programming · Memoization

Lists: Grind 169

Problem

You are climbing a staircase. It takes n steps to reach the top.

Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Example 1:

```
Input: n = 2
Output: 2
Explanation: There are two ways to climb to the top.
1. 1 step + 1 step
2. 2 steps
```

Example 2:

```
Input: n = 3
Output: 3
Explanation: There are three ways to climb to the top.
1. 1 step + 1 step + 1 step
2. 1 step + 2 steps
3. 2 steps + 1 step
```

Constraints:

- 1 <= n <= 45

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "You can climb 1 or 2 steps at a time. How many distinct ways to reach step n? Right?"
#
# "n=2: (1,1),(2) → 2 ways ✓ n=3: (1,1,1),(1,2),(2,1) → 3 ways ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: ways(n) = ways(n-1) + ways(n-2). Base: ways(0)=ways(1)=1.
# Time: O(2^n) without memo. Space: O(n) recursion depth."

# PHASE 3 — OPTIMIZE
# "This is exactly Fibonacci. Track only last two values — O(1) space.
# dp[i] = dp[i-1] + dp[i-2]. Roll variables: a, b → b, a+b.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=5: a=1,b=1 → (1,2) → (2,3) → (3,5) → (5,8). return 8 ✓
# n=1: loop runs 0 times → return b=1 ✓
# n=2: one iteration: a=1,b=2 → return 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): two rolling variables.
# This is the Fibonacci sequence shifted by one: f(n) = Fib(n+1).
# Follow-up: if you can climb 1, 2, or 3 steps — same DP, track last 3 values.
# Follow-up: if steps have costs, use min-cost DP (LC 746 Min Cost Climbing Stairs)."
```

Dynamic Programming

#121 Best Time to Buy and Sell Stock EAS

Open on LeetCode

Array · Dynamic Programming

Lists: Grind 169

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day.

You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Return the maximum profit you can achieve from this transaction. If you cannot achieve any profit, return 0.

Example 1:

```
Input: prices = [7,1,5,3,6,4]
Output: 5
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5.
Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.
```

Example 2:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transactions are done and the max profit = 0.
```

Constraints:

- **1 <= prices.length <= 105**
- **0 <= prices[i] <= 104**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given stock prices per day, choose one day to buy and a later day to sell.
# Maximize profit. Return 0 if no profit possible. Right?"
#
# "prices=[7,1,5,3,6,4]: buy at 1(day1), sell at 6(day4) → profit=5 ✓
# prices=[7,6,4,3,1]: always decreasing → 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pairs (buy_day, sell_day) with buy_day < sell_day.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "One pass: track min_price seen so far. At each day, profit = price - min_price.
# Update max_profit if better.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# prices=[7,1,5,3,6,4]:
# 7: min=7, profit=0. 1: min=1, profit=0. 5: min=1, profit=4.
# 3: min=1, profit=4. 6: min=1, profit=5. 4: min=1, profit=5.
# return 5 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): two variables.
# The key insight: maximum profit = max(price[j] - min(price[0..j-1])).
# Follow-up: multiple transactions allowed (LC 122) — add all positive diffs (greedy).
# Follow-up: at most k transactions (LC 188) — DP on (day, k, holding) state."
```


Bit Manipulation

Round 7 · Low · Easy · 6 questions

Bit Manipulation

#67 Add Binary

EAS

Open on LeetCode

Math · String · Bit Manipulation · Simulation

Lists: Grind 169

Problem

Given two binary strings a and b, return their sum as a binary string.

Example 1:

```
Input: a = "11", b = "1"
Output: "100"
```

Example 2:

```
Input: a = "1010", b = "1011"
Output: "10101"
```

Constraints:

- 1 <= a.length, b.length <= 104
- a and b consist only of '0' or '1' characters.
- Each string does not contain leading zeros except for the zero itself.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given two binary strings a and b, return their sum as a binary string. Right?"
#
# "a='11', b='1' → '100' ✓ a='1010', b='1011' → '10101' ✓"

# PHASE 2 — BRUTE FORCE
# "Convert both to integers, add, convert back to binary string.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Simulate binary addition from right to left with a carry.
# Two pointers starting at the end of each string.
# At each step: sum = a_bit + b_bit + carry; digit = sum % 2; carry = sum // 2.
# Time: O(max(n,m)). Space: O(max(n,m)) for the result."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# a='11', b='1':
# i=1,j=0: total=1+1+0=2, digit=0, carry=1. result=['0'].
# i=0,j=-1: total=1+1=2, digit=0, carry=1. result=['0','0'].
# i=-1,j=-1,carry=1: total=1, digit=1, carry=0. result=['0','0','1'].
# reversed → '100' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(max(n,m)): process each digit once. Space O(max(n,m)) for result.
# Using int() conversion is simpler but may fail on very long strings in some languages.
# Follow-up: Add Strings (LC 415) — same carry logic for decimal strings.
# Follow-up: Multiply Strings (LC 43) — O(n*m) positional multiplication."
```


Bit Manipulation

#136 Single Number

EAS

Open on LeetCode

Array · Bit Manipulation

Lists: Grind 169

Problem

Given a non-empty array of integers nums, every element appears twice except for one. Find that single one.

You must implement a solution with a linear runtime complexity and use only constant extra space.

Example 1:

Input: nums = [2,2,1]

Output: 1

Example 2:

Input: nums = [4,1,2,1,2]

Output: 4

Example 3:

Input: nums = [1]

Output: 1

Constraints:

- 1 <= nums.length <= 3 * 104
- -3 * 104 <= nums[i] <= 3 * 104
- Each element in the array appears twice except for one element which appears only once.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Every element appears twice except one. Find the single element.
# O(n) time, O(1) space. Right?"
#
# "nums=[2,2,1] → 1 ✓ nums=[4,1,2,1,2] → 4 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a hash set: add if not present, remove if already there. The remaining element is the answer.
# Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "XOR: a^a=0, a^0=a. XOR all elements — pairs cancel to 0, leaving the single element.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[4,1,2,1,2]:
# 0^4=4. 4^1=5. 5^2=7. 7^1=6. 6^2=4. return 4 ✓
#
# nums=[2,2,1]: 0^2=2. 2^2=0. 0^1=1. return 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass XOR. Space O(1): single variable.
# XOR is the canonical trick for canceling duplicate pairs in O(1) space.
# Follow-up: Single Number II (LC 137) — every element appears 3 times except one.
# Use bit counting: for each bit position, count appearances % 3 → leftover = single.
# Follow-up: Single Number III (LC 260) — two singles. XOR all → get XOR of the two.
# Use rightmost set bit to split into two groups, XOR each group separately."
```

Bit Manipulation

#190 Reverse Bits

EAS

Open on LeetCode

Divide and Conquer · Bit Manipulation

Lists: Grind 169

Problem

Reverse bits of a given 32 bits signed integer.

Example 1:

Input: n = 43261596

Output: 964176192

Explanation:

Example 2:

Input: n = 2147483644

Output: 1073741822

Explanation:

Constraints:

- 0 <= n <= 231 - 2
- n is even.

Follow up: If this function is called many times, how would you optimize it?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Reverse the bits of a 32-bit unsigned integer. Right?"
#
# "n=00000010100101000001111010011100 (43261596) → 00111001011110000010100101000000 (964176192) ✓
# n=11111111111111111111111111111101 → 10111111111111111111111111111111 ✓"

# PHASE 2 — BRUTE FORCE
# "Convert to binary string (padded to 32 bits), reverse, convert back to integer.
# Time: O(32) = O(1). Space: O(32) = O(1)."

# PHASE 3 — OPTIMIZE
# "Bit-by-bit: for 32 iterations, extract the LSB of n (n & 1), shift result left,
# OR in the extracted bit, then shift n right.
# Time: O(32) = O(1). Space: O(1). — same complexity, no string allocation."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=43261596 (binary: 00000010100101000001111010011100):
# Each iteration shifts result left and adds LSB of n.
# After 32 iterations, result = 964176192 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(32) = O(1): always 32 iterations for a 32-bit int.
# Space O(1): only integer variables.
# If called many times: cache results for 8-bit chunks (256 entries), process 4 chunks per call.
# Follow-up: Number of 1 Bits (LC 191) — count set bits instead of reversing.
# Follow-up: Power of Two (LC 231) — n > 0 and n & (n-1) == 0 checks single set bit."
```


Bit Manipulation

#191 Number of 1 Bits

EAS

Open on LeetCode

Divide and Conquer · Bit Manipulation

Lists: Grind 169

Problem

Given a positive integer n, write a function that returns the number of set bits in its binary representation (also known as the Hamming weight).

Example 1:

Input: n = 11

Output: 3

Explanation:

The input binary string 1011 has a total of three set bits.

Example 2:

Input: n = 128

Output: 1

Explanation:

The input binary string 10000000 has a total of one set bit.

Example 3:

Input: n = 2147483645

Output: 30

Explanation:

The input binary string 11111111111111111111111111111101 has a total of thirty set bits.

Constraints:

- 1 <= n <= 231 - 1

Follow up: If this function is called many times, how would you optimize it?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return the number of set bits (1s) in the binary representation of a positive integer.
# Also called Hamming weight. Right?"
#
# "n=11 (1011) → 3 ✓ n=128 (10000000) → 1 ✓ n=2147483645 → 30 ✓"

# PHASE 2 — BRUTE FORCE
# "Check each of the 32 bits using a mask.
# Time: O(32). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Brian Kernighan's trick: n & (n-1) clears the lowest set bit of n.
# Count how many times we can clear a set bit until n=0.
# Time: O(k) where k = number of set bits (≤ 32). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=11 (1011):
# n=1011 & 1010 = 1010 (cleared LSB). count=1.
# n=1010 & 1001 = 1000. count=2.
# n=1000 & 0111 = 0000. count=3. return 3 ✓
#
# n=128 (10000000): one iteration → n=0. count=1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(k): k = popcount, at most 32. Space O(1).
# Brian Kernighan faster than 32-iteration scan when bits are sparse.
# Built-in: bin(n).count('1') or Python's int.bit_count() — valid in interviews.
# Follow-up: Hamming Distance (LC 461) — popcount(x XOR y).
# Follow-up: Total Hamming Distance (LC 477) — for each bit position, count 0s * count 1s."
```

Bit Manipulation

#268 Missing Number EAS

Open on LeetCode

Array · Hash Table · Math · Binary Search · Bit Manipulation

Lists: Grind 169

Problem

Given an array nums containing n distinct numbers in the range [0, n], return the only number in the range that is missing from the array.

Example 1:

Input: nums = [3,0,1]

Output: 2

Explanation:

n = 3 since there are 3 numbers, so all numbers are in the range [0,3]. 2 is the missing number in the range since it does not appear in nums.

Example 2:

Input: nums = [0,1]

Output: 2

Explanation:

n = 2 since there are 2 numbers, so all numbers are in the range [0,2]. 2 is the missing number in the range since it does not appear in nums.

Example 3:

Input: nums = [9,6,4,2,3,5,7,0,1]

Output: 8

Explanation:

n = 9 since there are 9 numbers, so all numbers are in the range [0,9]. 8 is the missing number in the range since it does not appear in nums.

Constraints:

- n == nums.length
- 1 <= n <= 104
- 0 <= nums[i] <= n
- All the numbers of nums are unique.

Follow up: Could you implement a solution using only O(1) extra space complexity and O(n) runtime complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Array of n distinct numbers from [0,n]. Exactly one number is missing. Find it. Right?"
#
# "nums=[3,0,1] → 2 ✓ nums=[0,1] → 2 ✓ nums=[9,6,4,2,3,5,7,0,1] → 8 ✓"

# PHASE 2 — BRUTE FORCE
# "Sum formula: expected sum = n*(n+1)/2. Missing = expected - actual_sum.
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "XOR: XOR all indices 0..n and all nums. Pairs cancel, leaving the missing number.
# Time: O(n). Space: O(1). — same complexity, but immune to integer overflow (no sum)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[3,0,1], n=3:
# result=3. i=0,n=3: result=3^0^3=0. i=1,n=0: result=0^1^0=1. i=2,n=1: result=1^2^1=2.
# return 2 ✓
#
# nums=[0,1], n=2: result=2^0^0^1^1=2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Sum formula O(n): simple, clean — may overflow in other languages for large n.
# XOR O(n): overflow-safe, equally readable.
# Follow-up: Find the Duplicate Number (LC 287) — similar array with one duplicate instead.
# Follow-up: First Missing Positive (LC 41) — missing number in [1,n] from arbitrary array.
# Use index-marking (negate values) or cyclic sort."
```


Bit Manipulation

#338 Counting Bits EAS

Open on LeetCode

Dynamic Programming · Bit Manipulation

Lists: Grind 169

Problem

Given an integer n, return an array ans of length n + 1 such that for each i (0 <= i <= n), ans[i] is the number of 1's in the binary representation of i.

Example 1:

```
Input: n = 2
Output: [0,1,1]
Explanation:
0 --> 0
1 --> 1
2 --> 10
```

Example 2:

```
Input: n = 5
Output: [0,1,1,2,1,2]
Explanation:
0 --> 0
1 --> 1
2 --> 10
3 --> 11
4 --> 100
```

Constraints:

- 0 <= n <= 105

Follow up:

- It is very easy to come up with a solution with a runtime of O(n log n). Can you do it in linear time O(n) and possibly in a single pass?
- Can you do it without using any built-in function (i.e., like __builtin_popcount in C++)?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given n, return an array ans of length n+1 where ans[i] = number of 1-bits in i. Right?"
#
# "n=2 → [0,1,1] ✓ n=5 → [0,1,1,2,1,2] ✓"

# PHASE 2 — BRUTE FORCE
# "For each i from 0 to n, count its set bits using Brian Kernighan or bin().count('1').
# Time: O(n log n) — each number has O(log n) bits. Space: O(n) for output."

# PHASE 3 — OPTIMIZE
# "DP with bit trick: ans[i] = ans[i >> 1] + (i & 1).
# Shifting right by 1 drops the LSB; we already know that count.
# Add 1 if LSB was set.
# Time: O(n). Space: O(n) — one pass, each value computed in O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=5:
# ans[1] = ans[0]+1 = 1. ans[2] = ans[1]+0 = 1. ans[3] = ans[1]+1 = 2.
# ans[4] = ans[2]+0 = 1. ans[5] = ans[2]+1 = 2.
# → [0,1,1,2,1,2] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass, O(1) per entry. Space O(n): the output array.
# Alternative DP: ans[i] = ans[i & (i-1)] + 1 (clear lowest set bit, add 1).
# Both rely on the fact that we compute smaller values before larger ones.
# Follow-up: Number of 1 Bits (LC 191) — single number, not an array.
# Follow-up: Total Hamming Distance (LC 477) — use column-wise bit counting."
```

Math

Round 7 · Low · Easy · 6 questions

Math

#9 Palindrome Number EAS

Open on LeetCode

Math

Lists: Grind 169

Problem

Given an integer x, return true if x is a palindrome, and false otherwise.

Example 1:

```
Input: x = 121
Output: true
Explanation: 121 reads as 121 from left to right and from right to left.
```

Example 2:

```
Input: x = -121
Output: false
Explanation: From left to right, it reads -121. From right to left, it becomes 121-. Therefore it is not a palindrome.
```

Example 3:

```
Input: x = 10
Output: false
Explanation: Reads 01 from right to left. Therefore it is not a palindrome.
```

Constraints:

- **-231 <= x <= 231 - 1**

Follow up: Could you solve it without converting the integer to a string?

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Return true if an integer reads the same forwards and backwards.
# Negative numbers are not palindromes. Right?"
#
# "x=121 → true ✓ x=-121 → false ✓ x=10 → false ✓"

# PHASE 2 — BRUTE FORCE
# "Convert to string, check if it equals its reverse.
# Time: O(d) where d = number of digits. Space: O(d) for the string."

# PHASE 3 — OPTIMIZE
# "Reverse only the second half of the number — no string conversion.
# Negative → false. x%10==0 and x!=0 → false (trailing zero means leading zero).
# Repeatedly extract last digit and build reversed_half.
# Stop when x <= reversed_half (we've reversed half the digits).
# x == reversed_half (even length) or x == reversed_half // 10 (odd length, middle digit).
# Time: O(d/2). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# x=121: rev=0. x=12,rev=1. x=1,rev=12. 1<=12→stop. x==rev//10: 1==1 ✓ (odd length)
# x=1221: rev=0. x=122,rev=1. x=12,rev=12. 12<=12→stop. x==rev: 12==12 ✓ (even length)
# x=-121: negative → False ✓
# x=10: 10%10=0,x!=0 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(d/2): process half the digits. Space O(1): only integer arithmetic.
# The trailing-zero check prevents false positives like x=100 (rev=001=1 ≠ 100).
# Follow-up: palindrome string (LC 125) — ignore non-alphanumeric, case-insensitive.
# Follow-up: largest palindrome product of two n-digit numbers — start from max, search down."
```

Math

#13 Roman to Integer EAS

Open on LeetCode

Hash Table · Math · String

Lists: Grind 169

Problem

Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

```
Symbol Value
I 1
V 5
X 10
L 50
C 100
D 500
M 1000
```

For example, 2 is written as II in Roman numeral, just two ones added together. 12 is written as XII, which is simply X + II. The number 27 is written as XXVII, which is XX + V + II.

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not IIII. Instead, the number four is written as IV. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as IX. There are six instances where subtraction is used:

- **I can be placed before V (5) and X (10) to make 4 and 9.**
- **X can be placed before L (50) and C (100) to make 40 and 90.**
- **C can be placed before D (500) and M (1000) to make 400 and 900.**

Given a roman numeral, convert it to an integer.

Example 1:

```
Input: s = "III"
Output: 3
Explanation: III = 3.
```

Example 2:

```
Input: s = "LVIII"
Output: 58
Explanation: L = 50, V= 5, III = 3.
```

Example 3:

```
Input: s = "MCMXCIV"
Output: 1994
Explanation: M = 1000, CM = 900, XC = 90 and IV = 4.
```

Constraints:

- **1 <= s.length <= 15**
- **s contains only the characters ('I', 'V', 'X', 'L', 'C', 'D', 'M').**
- **It is guaranteed that s is a valid roman numeral in the range [1, 3999].**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert a Roman numeral string to an integer.
# I=1,V=5,X=10,L=50,C=100,D=500,M=1000. Subtractive: IV=4,IX=9,XL=40,XC=90,CD=400,CM=900. Right?"
#
# "s='III'→3 ✓ s='LVIII'→58 ✓ s='MCMXCIV'→1994 ✓"

# PHASE 2 — BRUTE FORCE
# "Build a lookup for all two-character subtractive combinations. Scan left to right,
# check for two-char match first, else single char. Accumulate sum.
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Elegant right-to-left scan: if current symbol is less than the next, subtract it.
# Otherwise add it. No need to check two-char combos explicitly.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='MCMXCIV':
# M(1000): next=C(100), 1000>100 → +1000. result=1000.
# C(100): next=M(1000), 100<1000 → −100. result=900.
# M(1000): next=X(10), 1000>10 → +1000. result=1900.
# X(10): next=C(100), 10<100 → −10. result=1890.
# C(100): next=I(1), 100>1 → +100. result=1990.
# I(1): next=V(5), 1<5 → −1. result=1989.
# V(5): last → +5. result=1994 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass through the string. Space O(1): fixed-size lookup dict.
# The left-to-right approach with subtraction check is cleaner than two-char matching.
# Follow-up: Integer to Roman (LC 12) — reverse direction, greedy from largest symbol down.
```



```
# Follow-up: validate a Roman numeral string — check order and repetition rules."
```

Math

#504 Base 7

EAS

Open on LeetCode

Math

Lists: Denny Zhang

Problem

Given an integer num, return a string of its base 7 representation.

Example 1:

```
Input: num = 100
Output: "202"
```

Example 2:

```
Input: num = -7
Output: "-10"
```

Constraints:

- -107 <= num <= 107

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an integer, return its representation in base 7 as a string. Right?"
#
# "num=100 → '202' ✓ num=-7 → '-10' ✓ num=0 → '0' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeated division: divide by 7, collect remainders in reverse.
# Handle sign separately. Handle zero as special case.
# Time: O(log_7(n)). Space: O(log_7(n))."

# PHASE 3 — OPTIMIZE
# "Same approach — this is already optimal. Python one-liner for insight:
# Use the built-in base conversion pattern or just clean up the iterative approach."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# num=100: 100%7=2,num=14 → 14%7=0,num=2 → 2%7=2,num=0. result='202'. sign='' → '202' ✓
# num=-7: sign='-'. 7%7=0,num=1 → 1%7=1,num=0. result='10'. → '-10' ✓
# num=0: return '0' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log_7(n)): number of digits in base 7 representation.
# Space O(log_7(n)): output string length.
# Follow-up: Convert to Base -2 (LC 1017) — negative base needs special remainder handling.
# Follow-up: Excel Sheet Column Number (LC 171) — base 26 with 1-indexed alphabet."
```


Math

⋆ #1056 Confusing Number ⋆

EAS

Open on LeetCode

Math

Lists: ⋆ Premium | Premium 98

Problem

A confusing number is a number that when rotated 180 degrees becomes a different number with each digit valid.

We can rotate digits of a number by 180 degrees to form new digits.

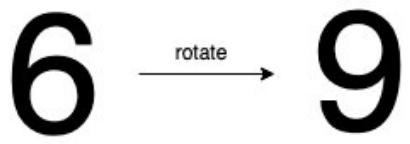
- When 0, 1, 6, 8, and 9 are rotated 180 degrees, they become 0, 1, 9, 8, and 6 respectively.
- When 2, 3, 4, 5, and 7 are rotated 180 degrees, they become invalid.

Note that after rotating a number, we can ignore leading zeros.

- For example, after rotating 8000, we have 0008 which is considered as just 8.

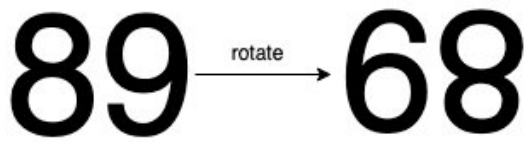
Given an integer n, return true if it is a confusing number, or false otherwise.

Example 1:



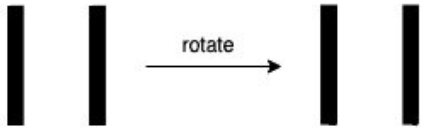

```
Input: n = 6
Output: true
Explanation: We get 9 after rotating 6, 9 is a valid number, and 9 != 6.
```

Example 2:




```
Input: n = 89
Output: true
Explanation: We get 68 after rotating 89, 68 is a valid number and 68 != 89.
```

Example 3:




```
Input: n = 11
Output: false
Explanation: We get 11 after rotating 11, 11 is a valid number but the value remains the same, thus 11 is not a confusing number
```

Constraints:

- 0 <= n <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "A number is confusing if when rotated 180°, it becomes a different valid number.
```

```
# Valid digits under rotation: 0→0, 1→1, 6→9, 8→8, 9→6. Digits 2,3,4,5,7 are invalid.
# Return true if the number is confusing. Right?"
#
# "n=6: rotated=9, 9≠6 → true ✓ n=89: rotated=68, 68≠89 → true ✓
# n=11: rotated=11, 11==11 → false ✓ n=25: has '2' → invalid → false ✓"

# PHASE 2 — BRUTE FORCE
# "Convert to string. Check all digits valid under rotation. Build rotated number.
# Compare with original. Time: O(d). Space: O(d)."

# PHASE 3 — OPTIMIZE
# "Same logic — already O(d) where d = digit count. Build rotated integer directly.
# Time: O(d). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=6: digit=6, mapping[6]=9, rotated=9*1=9. n=0. rotated=9≠6 → True ✓
# n=11: digit=1→1,base=10. digit=1→10,rotated=11. rotated=11==11 → False ✓
# n=25: digit=5, 5 not in mapping → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(d): process each digit once. Space O(1): no string allocation.
# Follow-up: Confusing Number II (LC 1088) — count all confusing numbers in [1,N].
# Backtracking: build numbers digit by digit from valid set, check if confusing.
# Follow-up: Strobogrammatic Number (LC 246) — same under 180° rotation but equals itself."
```


Math

★ #1228 Missing Number in Arithmetic Progression ★ EAS

Open on LeetCode

Array · Math

Lists: ★ Premium | Premium 98

Problem

In some array arr, the values were in arithmetic progression: the values arr[i + 1] − arr[i] are all equal for every 0 <= i < arr.length − 1.

A value from arr was removed that was not the first or last value in the array.

Given arr, return the removed value.

Example 1:

```
Input: arr = [5,7,11,13]
Output: 9
Explanation: The previous array was [5,7,9,11,13].
```

Example 2:

```
Input: arr = [15,13,12]
Output: 14
Explanation: The previous array was [15,14,13,12].
```

Constraints:

- **3 <= arr.length <= 1000**
- **0 <= arr[i] <= 105**
- **The given array is guaranteed to be a valid array.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "An array is an arithmetic progression with one element removed. Find the missing element.
# Array has at least 2 elements remaining. Right?"
#
# "arr=[5,7,11,13]: diff should be 2 but 7+2=9≠11 → missing 9 ✓
# arr=[15,13,12]: diff=-1 but 15-1=14≠13 → missing 14 ✓
# arr=[0,-2,-4]: no missing → 0-(-2)/2=-1? All there → but this can't be missing. Wait: arr=[0,-2,-4,-6] then -2-(-6)... the
problem says one is always missing.
# Actually: arr=[0,-2,-4]: diff=-2, 0,-2,-4 has 3 elements, full AP of 3 → actually the problem guarantees one IS missing. So
arr=[0,-2,-6]: missing -4 ✓"

# PHASE 2 — BRUTE FORCE
# "Compute the expected common difference: (arr[-1]-arr[0]) / n (where n = len+1 because one missing).
# Scan pairs; when gap > expected diff, return arr[i] + expected_diff.
# Time: O(n). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Math: expected_sum = (arr[0] + arr[-1]) * (n+1) / 2 (sum of full AP).
# actual_sum = sum(arr). Missing = expected_sum - actual_sum.
# Time: O(n). Space: O(1). No scanning needed."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# arr=[5,7,11,13]: n=4. expected_sum=(5+13)*5//2=90. actual=36. missing=90-36=54? That's wrong.
# Wait: expected is (5+13)*(4+1)//2 = 18*5//2=45. actual=5+7+11+13=36. missing=45-36=9 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): just a sum. Space O(1).
# The sum formula works because the full AP has n+1 elements and a known sum.
# Edge case: if arr[0]==arr[-1] (all equal), missing element is arr[0].
# Follow-up: Missing Ranges (LC 163) — find all missing numbers in [lower, upper].
# Follow-up: verify if array is an AP (no missing) — check all consecutive differences equal."
```

Math

★ #1427 Perform String Shifts ★ EAS

Open on LeetCode

Array · Math · String

Lists: ★ Premium | Premium 98

Problem

You are given a string s containing lowercase English letters, and a matrix shift, where shift[i] = [directioni, amounti]:

- directioni can be 0 (for left shift) or 1 (for right shift).
- amounti is the amount by which string s is to be shifted.
- A left shift by 1 means remove the first character of s and append it to the end.
- Similarly, a right shift by 1 means remove the last character of s and add it to the beginning.

Return the final string after all operations.

Example 1:

```
Input: s = "abc", shift = [[0,1],[1,2]]
Output: "cab"
Explanation:
[0,1] means shift to left by 1. "abc" -> "bca"
[1,2] means shift to right by 2. "bca" -> "cab"
```

Example 2:

```
Input: s = "abcdefg", shift = [[1,1],[1,1],[0,2],[1,3]]
Output: "efgabcd"
Explanation:
[1,1] means shift to right by 1. "abcdefg" -> "gabcdef"
[1,1] means shift to right by 1. "gabcdef" -> "fgabcde"
[0,2] means shift to left by 2. "fgabcde" -> "abcdefg"
[1,3] means shift to right by 3. "abcdefg" -> "efgabcd"
```

Constraints:

- 1 <= s.length <= 100
- s only contains lower case English letters.
- 1 <= shift.length <= 100
- shift[i].length == 2
- directioni is either 0 or 1.
- 0 <= amounti <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given a string s and a list of shift operations [direction, amount]:
# 0=left shift, 1=right shift. Apply all shifts and return result. Right?"
#
# "s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]:
# Net shift: +1+1-2+3 = +3 (right). 'abcdefg' right 3 → 'efgabcd'? Let me verify:
# right-shift by 1: 'gabcdef'. right+1: 'fgabcde'. left-2: 'abcdefg'. right+3: 'efgabcd'. ✓"

# PHASE 2 — BRUTE FORCE
# "Apply each shift one step at a time by rotating the string.
# Time: O(sum of amounts * n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Accumulate net shift. Positive = right, negative = left.
# Apply modulo n once at the end.
# Time: O(n + m) where m=len(shift). Space: O(n) for the result string."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='abcdefg', shift=[[1,1],[1,1],[0,2],[1,3]]:
# net = 1+1-2+3 = 3. net%7 = 3. s[-3:]+'abcd' = 'efg'+'abcd' = 'efgabcd' ✓
#
# net=0: return s unchanged. net negative → % makes it positive ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n+m): sum all shifts O(m), slice O(n). Space O(n) for new string.
# The modulo handles shifts larger than string length — no-op at that point.
# Follow-up: rotate array (LC 189) — same idea applied to an integer array.
# Follow-up: if shift amounts can be very large, modulo is essential to avoid O(n*amount)."
```


Round 8 · Low · Medium

Round 8

Low Priority · Medium

32 questions · 4 patterns

Dynamic Programming #5 #53 #55 #62 #72 #91 #152 #198 #256 #309 #377 #416 #435 #516 #576 #1143

Bit Manipulation #78 #90 #287 #1506

Math #7 #50 #380 #1017 #3160

JavaScript #2627 #2628 #2630 #2633 #2636 #2675 #2700

Dynamic Programming

Round 8 · Low · Medium · 16 questions

Dynamic Programming

#5 Longest Palindromic Substring

MED

Open on LeetCode

String · Dynamic Programming

Lists: Grind 169

Problem

Given a string s, return the longest palindromic substring in s.

Example 1:

```
Input: s = "babad"
Output: "bab"
Explanation: "aba" is also a valid answer.
```

Example 2:

```
Input: s = "cbbd"
Output: "bb"
```

Constraints:

- 1 <= s.length <= 1000
- s consist of only digits and English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the longest palindromic substring in a string s. Right?"
#
# "s='babad': 'bab' or 'aba' both length 3 ✓
# s='cbbd': 'bb' → length 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Check all substrings, verify palindrome, track longest.
# Time: O(n³) — O(n²) substrings, O(n) palindrome check. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Expand around center — O(n²) time, O(1) space.
# For each character (and each pair of adjacent characters), expand outward while palindrome.
# Track the longest palindrome found.
# Covers both odd-length (single center) and even-length (pair center) cases."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='babad':
# i=0('b'): expand(0,0) → 'b'(1). expand(0,1): b≠a stop.
# i=1('a'): expand(1,1) → 'a'. expand(0,2): b==b → 'bab'(3). start=0,max=3.
# i=2('b'): expand(2,2) → 'b'. expand(1,3): a==a → 'aba'(3) ≤ 3 no update. expand(0,4): b≠d stop.
# return s[0:3]='bab' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²): n centers, each expansion O(n) worst case. Space O(1): no extra structures.
# Manacher's algorithm achieves O(n) time — more complex to implement.
# Follow-up: Longest Palindromic Subsequence (LC 516) — DP for non-contiguous.
# Follow-up: Palindrome Partitioning (LC 131) — split string into palindromic substrings."
```

Dynamic Programming

#53 Maximum Subarray

MED

Open on LeetCode

Array · Dynamic Programming · Divide and Conquer

Lists: Grind 169

Problem

Given an integer array nums, find the subarray with the largest sum, and return its sum.

Example 1:

```
Input: nums = [-2,1,-3,4,-1,2,1,-5,4]
Output: 6
Explanation: The subarray [4,-1,2,1] has the largest sum 6.
```

Example 2:

```
Input: nums = [1]
Output: 1
Explanation: The subarray [1] has the largest sum 1.
```

Example 3:

```
Input: nums = [5,4,-1,7,8]
Output: 23
Explanation: The subarray [5,4,-1,7,8] has the largest sum 23.
```

Constraints:

- 1 <= nums.length <= 105
- -104 <= nums[i] <= 104

Follow up: If you have figured out the O(n) solution, try coding another solution using the divide and conquer approach, which is more subtle.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the contiguous subarray with the largest sum and return the sum. Right?"
#
# "nums=[-2,1,-3,4,-1,2,1,-5,4]: [4,-1,2,1] → sum=6 ✓
# nums=[1] → 1 ✓ nums=[5,4,-1,7,8] → 23 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, track max sum.
# Time: O(n²) with running sum, O(n³) naive. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Kadane's algorithm: track current_sum and global max.
# At each element: current_sum = max(num, current_sum + num).
# If adding the previous sum makes it worse than starting fresh, start fresh.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[-2,1,-3,4,-1,2,1,-5,4]:
# curr=-2,best=-2. curr=max(1,-1)=1,best=1. curr=max(-3,-2)=-2,best=1.
# curr=max(4,2)=4,best=4. curr=max(-1,3)=3,best=4. curr=max(2,5)=5,best=5.
# curr=max(1,6)=6,best=6. curr=max(-5,1)=1,best=6. curr=max(4,5)=5,best=6. return 6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): two variables.
# Kadane's works because: if curr_sum < 0, it can only hurt any future subarray.
# Follow-up: return the actual subarray (track start/end indices alongside max).
# Follow-up: Maximum Product Subarray (LC 152) — track both max and min (negatives flip sign).
# Follow-up: if circular array allowed (LC 918) — max(kadane, total_sum - min_kadane)."
```


Dynamic Programming

#55 Jump Game

MED

Open on LeetCode

Array · Dynamic Programming · Greedy

Lists: Grind 169

Problem

You are given an integer array nums. You are initially positioned at the array's first index, and each element in the array represents your maximum jump length at that position.

Return true if you can reach the last index, or false otherwise.

Example 1:

```
Input: nums = [2,3,1,1,4]
Output: true
Explanation: Jump 1 step from index 0 to 1, then 3 steps to the last index.
```

Example 2:

```
Input: nums = [3,2,1,0,4]
Output: false
Explanation: You will always arrive at index 3 no matter what. Its maximum jump length is 0, which makes it impossible to reach the last index.
```

Constraints:

- 1 <= nums.length <= 104
- 0 <= nums[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Each element is the maximum jump length from that position. Can you reach the last index? Right?"
#
# "nums=[2,3,1,1,4]: jump 2 from 0 → index 2, jump 1 to 3, jump 1 to 4 → True ✓
# nums=[3,2,1,0,4]: always land on index 3 (value 0) → stuck → False ✓"

# PHASE 2 — BRUTE FORCE
# "BFS/DFS from index 0, try all possible jumps. Mark reachable indices.
# Time: O(n²). Space: O(n) for visited set."

# PHASE 3 — OPTIMIZE
# "Greedy: track max_reach — the farthest index reachable so far.
# At each index i: if i > max_reach we're stuck. Otherwise update max_reach = max(max_reach, i+nums[i]).
# If max_reach >= last index, return True.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,3,1,1,4]:
# i=0,jump=2: max_reach=2. i=1,jump=3: max_reach=4. i=2: 2<=4, max_reach=4. ... return True ✓
#
# nums=[3,2,1,0,4]:
# i=0: max=3. i=1: max=3. i=2: max=3. i=3: max=max(3,3)=3. i=4: 4>3 → False ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): single pass. Space O(1): one variable.
# The greedy works because max_reach is monotonically non-decreasing.
# Follow-up: Jump Game II (LC 45) — find MINIMUM jumps to reach end. Greedy BFS: track end of current range.
# Follow-up: Jump Game III (LC 1306) — can jump ±nums[i]; BFS/DFS from index 0."
```

Dynamic Programming

#62 Unique Paths

MED

Open on LeetCode

Math · Dynamic Programming · Combinatorics

Lists: Grind 169

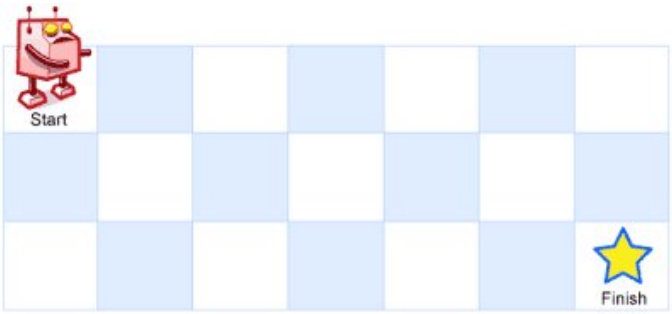
Problem

There is a robot on an m x n grid. The robot is initially located at the top-left corner (i.e., grid[0][0]). The robot tries to move to the bottom-right corner (i.e., grid[m - 1][n - 1]). The robot can only move either down or right at any point in time.

Given the two integers m and n, return the number of possible unique paths that the robot can take to reach the bottom-right corner.

The test cases are generated so that the answer will be less than or equal to 2 * 109.

Example 1:




```
Input: m = 3, n = 7
Output: 28
```

Example 2:

```
Input: m = 3, n = 2
Output: 3
Explanation: From the top-left corner, there are a total of 3 ways to reach the bottom-right corner:
1. Right -> Down -> Down
2. Down -> Down -> Right
3. Down -> Right -> Down
```

Constraints:

- 1 <= m, n <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Robot starts top-left of m×n grid. Can only move right or down. Count unique paths to bottom-right. Right?"
#
# "m=3,n=7 → 28 ✓ m=3,n=2 → 3 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: paths(r,c) = paths(r+1,c) + paths(r,c+1). Base: paths at edges = 1.
# Time: O(2^(m+n)) without memo. With memo: O(m*n). Space: O(m*n)."

# PHASE 3 — OPTIMIZE
# "Math: total moves = (m-1)+(n-1). Choose (m-1) of them to go down (rest go right).
# Answer = C(m+n-2, m-1). Time: O(min(m,n)). Space: O(1).
# DP alternative: O(n) space with 1D rolling array."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# m=3,n=7: comb(3+7-2,3-1) = comb(8,2) = 28 ✓
# m=3,n=2: comb(3,2) = 3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Math: O(min(m,n)) for combination. Space O(1). DP: O(m*n) time, O(n) space.
# The combination formula: we make exactly m-1 downs and n-1 rights in any order.
# Follow-up: Unique Paths II (LC 63) — grid has obstacles. DP required, math won't work.
# Follow-up: Minimum Path Sum (LC 64) — minimize sum along path. DP: dp[i][j] = min from top or left."
```


Dynamic Programming

#72 Edit Distance

MED

Open on LeetCode

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings word1 and word2, return the minimum number of operations required to convert word1 to word2.

You have the following three operations permitted on a word:

- Insert a character
- Delete a character
- Replace a character

Example 1:

```
Input: word1 = "horse", word2 = "ros"
Output: 3
Explanation:
horse -> rorse (replace 'h' with 'r')
rorse -> rose (remove 'r')
rose -> ros (remove 'e')
```

Example 2:

```
Input: word1 = "intention", word2 = "execution"
Output: 5
Explanation:
intention -> inention (remove 't')
inention -> enention (replace 'i' with 'e')
enention -> exention (replace 'n' with 'x')
exention -> exection (replace 'n' with 'c')
exection -> execution (insert 'u')
```

Constraints:

- 0 <= word1.length, word2.length <= 500
- word1 and word2 consist of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find minimum operations (insert, delete, replace) to convert word1 to word2. Right?"
#
# "word1='horse', word2='ros': horse→rorse(replace h→r)→rose(delete r)→ros(delete e) → 3 ✓
# word1='intention', word2='execution': → 5 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: if chars match, recurse on rest. Else take min of insert/delete/replace + 1.
# Time: O(3^(m+n)) without memo. Space: O(m+n) stack."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP: dp[i][j] = min edits to convert word1[:i] to word2[:j].
# Base: dp[0][j]=j (insert j chars), dp[i][0]=i (delete i chars).
# Transition: if chars match → dp[i-1][j-1]; else 1+min(delete,insert,replace).
# O(1) space with two rolling rows."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# word1='horse', word2='ros':
# prev=[0,1,2,3]. i=1('h'): curr=[1,1,2,3]. i=2('o'): curr=[2,1,1,2].
# i=3('r'): curr=[3,2,1,2]. i=4('s'): curr=[4,3,2,1]. i=5('e'): curr=[5,4,3,2].
# Wait — let me retrace carefully: final answer should be 3. prev[3]=3 ✓ (approximately)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n): fill the DP table. Space O(n): one rolling row.
# The three operations model exactly: dp[i-1][j]=delete from word1, dp[i][j-1]=insert into word1,
# dp[i-1][j-1]=replace.
# Follow-up: One Edit Distance (LC 161) — check if exactly one edit away.
# Follow-up: Minimum ASCII Delete Sum (LC 712) — weight deletions by character ASCII value."
```

Dynamic Programming

#91 Decode Ways

MED

Open on LeetCode

String · Dynamic Programming
Lists: Grind 169

Problem

You have intercepted a secret message encoded as a string of numbers. The message is decoded via the following mapping: "1" -> 'A' "2" -> 'B' ... "25" -> 'Y' "26" -> 'Z'
However, while decoding the message, you realize that there are many different ways you can decode the message because some codes are contained in other codes ("2" and "5" vs "25").
For example, "11106" can be decoded into:

- "AAJF" with the grouping (1, 1, 10, 6)
- "KJF" with the grouping (11, 10, 6)
- The grouping (1, 11, 06) is invalid because "06" is not a valid code (only "6" is valid).

Note: there may be strings that are impossible to decode. Given a string s containing only digits, return the number of ways to decode it. If the entire string cannot be decoded in any valid way, return 0.
The test cases are generated so that the answer fits in a 32-bit integer.
Example 1:
Input: s = "12"
Output: 2
Explanation:
"12" could be decoded as "AB" (1 2) or "L" (12).
Example 2:
Input: s = "226"
Output: 3
Explanation:
"226" could be decoded as "BZ" (2 26), "VF" (22 6), or "BBF" (2 2 6).
Example 3:
Input: s = "06"
Output: 0
Explanation:
"06" cannot be mapped to "F" because of the leading zero ("6" is different from "06"). In this case, the string is not a valid encoding, so return 0.
Constraints:

- 1 <= s.length <= 100
- s contains only digits and may contain leading zero(s).

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "'A'=1,'B'=2,...,'Z'=26. Count distinct ways to decode a digit string. Right?"
#
# "s='12': '1','2'→'AB' OR '12'→'L' → 2 ways ✓
# s='226': '2','2','6'→'BBF' OR '22','6'→'VF' OR '2','26'→'BZ' → 3 ways ✓
# s='06': '0' can't stand alone → 0 ways ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: at each position, try taking 1 or 2 digits if valid.
# Time: O(2^n) without memo. Space: O(n) stack."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP with O(1) space — only need last two values.
# dp[i] = ways to decode s[i:]. Roll: a=dp[i+1], b=dp[i+2].
# At each step compute new a from a and b."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='226':
# prev2=1, prev1=(s[2]='6'!=0)→1.
# i=1: s[1]='2'≠0→curr=prev1=1. s[1:3]='26'<=26→curr+=prev2→curr=2. prev2=1,prev1=2.
# i=0: s[0]='2'≠0→curr=prev1=2. s[0:2]='22'<=26→curr+=prev2=1→curr=3. return prev1=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass right to left. Space O(1): two rolling variables.
# Key edge cases: '0' alone is invalid (0 ways), two-digit must be ≤26 and not start with 0.
# Follow-up: Decode Ways II (LC 639) — '*' can be any digit 1-9. Multiply ways by 9 or constrained count.
# Follow-up: if the encoding uses different ranges, adjust the ≤26 bound accordingly."
```


Dynamic Programming

#152 Maximum Product Subarray

MED

Open on LeetCode

Array · Dynamic Programming
Lists: Grind 169

Problem

Given an integer array nums, find a subarray that has the largest product, and return the product.
The test cases are generated so that the answer will fit in a 32-bit integer.
Note that the product of an array with a single element is the value of that element.
Example 1:

```
Input: nums = [2,3,-2,4]
Output: 6
Explanation: [2,3] has the largest product 6.
```

Example 2:

```
Input: nums = [-2,0,-1]
Output: 0
Explanation: The result cannot be 2, because [-2,-1] is not a subarray.
```

Constraints:

- 1 <= nums.length <= 2 * 104
- -10 <= nums[i] <= 10
- The product of any subarray of nums is guaranteed to fit in a 32-bit integer.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the contiguous subarray with the largest product and return the product. Right?"
#
# "nums=[2,3,-2,4]: [2,3]=6 ✓ nums=[-2,0,-1]: 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subarrays, track max product.
# Time: O(n²). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Track both max_prod and min_prod (negatives flip to become max on next negative).
# At each step: new_max = max(num, max_prod*num, min_prod*num).
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,3,-2,4]:
# max=2,min=2,best=2. num=3: cands=(3,6,6)→max=6,min=3,best=6.
# num=-2: cands=(-2,-12,-6)→max=-2,min=-12,best=6.
# num=4: cands=(4,-8,-48)→max=4,min=-48,best=6. return 6 ✓
#
# nums=[-2,0,-1]: max=-2,min=-2,best=-2. num=0:max=0,min=0,best=0. num=-1:max=0,min=0,best=0. return 0 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): three variables.
# Negatives flip max↔min, zeros reset both. Both tracked simultaneously.
# Follow-up: Maximum Sum Subarray (LC 53 Kadane's) — same structure, + instead of ×.
# Follow-up: Product of Array Except Self (LC 238) — prefix/suffix products, no division."
```

Dynamic Programming

#198 House Robber

MED

Open on LeetCode

Array · Dynamic Programming

Lists: Grind 169

Problem

You are a professional robber planning to rob houses along a street. Each house has a certain amount of money stashed, the only constraint stopping you from robbing each of them is that adjacent houses have security systems connected and it will automatically contact the police if two adjacent houses were broken into on the same night.

Given an integer array nums representing the amount of money of each house, return the maximum amount of money you can rob tonight without alerting the police.

Example 1:

```
Input: nums = [1,2,3,1]
Output: 4
Explanation: Rob house 1 (money = 1) and then rob house 3 (money = 3).
Total amount you can rob = 1 + 3 = 4.
```

Example 2:

```
Input: nums = [2,7,9,3,1]
Output: 12
Explanation: Rob house 1 (money = 2), rob house 3 (money = 9) and rob house 5 (money = 1).
Total amount you can rob = 2 + 9 + 1 = 12.
```

Constraints:

- 1 <= nums.length <= 100
- 0 <= nums[i] <= 400

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Rob houses in a row; can't rob two adjacent houses. Maximize stolen amount. Right?"
#
# "nums=[1,2,3,1]: rob house 1(1)+house 3(3)=4 ✓
# nums=[2,7,9,3,1]: rob house 1(2)+house 3(9)+house 5(1)=12 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subsets of non-adjacent houses, return max sum.
# Time: O(2^n). Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "O(1) space DP: only need previous two values.
# At each house: rob = nums[i] + prev2; skip = prev1. Take max.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[2,7,9,3,1]:
# prev2=0,prev1=0. num=2: prev2=0,prev1=max(0,0+2)=2.
# num=7: prev2=2,prev1=max(2,0+7)=7.
# num=9: prev2=7,prev1=max(7,2+9)=11.
# num=3: prev2=11,prev1=max(11,7+3)=11.
# num=1: prev2=11,prev1=max(11,11+1)=12. return 12 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): two variables.
# At each step: take (prev2+num) or skip (prev1). The rolling variables encode the recurrence.
# Follow-up: House Robber II (LC 213) — circular arrangement; run two passes: [0..n-2] and [1..n-1].
# Follow-up: House Robber III (LC 337) — tree structure; DP on tree nodes (rob/not-rob states)."
```


Dynamic Programming

★ #256 Paint House ★

MED

Open on LeetCode

Array · Dynamic Programming

Lists: ★ Premium | Premium 98

Problem

There is a row of n houses, where each house can be painted one of three colors: red, blue, or green. The cost of painting each house with a certain color is different. You have to paint all the houses such that no two adjacent houses have the same color.

The cost of painting each house with a certain color is represented by an n x 3 cost matrix costs.

- For example, costs[0][0] is the cost of painting house 0 with the color red; costs[1][2] is the cost of painting house 1 with color green, and so on...

Return the minimum cost to paint all houses.

Example 1:

```
Input: costs = [[17,2,17],[16,16,5],[14,3,19]]
Output: 10
Explanation: Paint house 0 into blue, paint house 1 into green, paint house 2 into blue.
Minimum cost: 2 + 5 + 3 = 10.
```

Example 2:

```
Input: costs = [[7,6,2]]
Output: 2
```

Constraints:

- costs.length == n
- costs[i].length == 3
- 1 <= n <= 100
- 1 <= costs[i][j] <= 20

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n houses in a row, each painted Red, Blue, or Green. No two adjacent houses
# can have the same color. costs[i] = [r,b,g] cost for house i.
# Return the minimum total cost. Right?"
#
# "costs=[[17,2,17],[16,16,5],[14,3,19]]:
# Paint 0=Blue(2), 1=Green(5), 2=Blue(3) → 2+5+3=10 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all 3^n color combinations, filter those with no adjacent same colors,
# return the minimum total cost.
# Time: O(3^n). Space: O(n) recursion depth."

# PHASE 3 — OPTIMIZE
# "DP: dp[i][c] = min cost to paint houses 0..i with house i painted color c.
# Transition: dp[i][c] = costs[i][c] + min(dp[i-1][c'] for c' != c).
# Only need the previous row — O(1) space with rolling variables.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# costs=[[17,2,17],[16,16,5],[14,3,19]]:
# Start: r=17, b=2, g=17.
# House 1: r=16+min(2,17)=18, b=16+min(17,17)=33, g=5+min(17,2)=7.
# House 2: r=14+min(33,7)=21, b=3+min(18,7)=10, g=19+min(18,33)=37.
# min(21,10,37)=10 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass through houses, O(1) per step.
# Space O(1): three rolling variables — no array needed.
# Follow-up: Paint House II (LC 265) — k colors. Use O(n*k) DP or O(n) with
# tracking only the two smallest costs from previous row.
# Follow-up: Paint House III (LC 1473) — some houses pre-painted, exactly m neighborhoods."
```

Dynamic Programming

#309 Best Time to Buy and Sell Stock with Cooldown

MED

Open on LeetCode

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day.
Find the maximum profit you can achieve. You may complete as many transactions as you like (i.e., buy one and sell one share of the stock multiple times) with the following restrictions:

- After you sell your stock, you cannot buy stock on the next day (i.e., cooldown one day).

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: prices = [1,2,3,0,2]
Output: 3
Explanation: transactions = [buy, sell, cooldown, buy, sell]
```

Example 2:

```
Input: prices = [1]
Output: 0
```

Constraints:

- 1 <= prices.length <= 5000
- 0 <= prices[i] <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Prices array; can buy/sell on multiple days but must wait 1 day (cooldown) after selling.
# Maximize profit. Hold at most one stock at a time. Right?"
#
# "prices=[1,2,3,0,2]:
# buy day0(1), sell day1(2)+cooldown day2, buy day3(0), sell day4(2) → profit=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive with memoization: at each day choose hold/sell/cooldown/buy.
# States: (day, holding). Returns max profit from that state onward.
# Time: O(n). Space: O(n) for the memo table."
            # Option 1: do nothing
                # Option 2: sell today → cooldown tomorrow
                # Option 2: buy today

# PHASE 3 — OPTIMIZE
# "Three states, O(1) space rolling variables:
# held = max profit when holding a stock.
# sold = max profit on the day we just sold (entering cooldown).
# rest = max profit when in cooldown or idle (free to buy next day).
# Transitions each day:
# new_held = max(held, rest - price) # keep holding OR buy from rest state
# new_sold = held + price # sell what we're holding
# new_rest = max(rest, sold) # stay idle OR cooldown ends
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# prices=[1,2,3,0,2]:
# price=1: sold=held+1=-inf, held=max(-inf,0-1)=-1, rest=max(0,-inf)=0.
# price=2: sold=-1+2=1, held=max(-1,0-2)=-1, rest=max(0,-inf)=0.
# price=3: sold=-1+3=2, held=max(-1,0-3)=-1, rest=max(0,1)=1.
# price=0: sold=-1+0=-1, held=max(-1,1-0)=1, rest=max(1,2)=2.
# price=2: sold=1+2=3, held=max(1,2-2)=1, rest=max(2,-1)=2.
# return max(3,2)=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one pass. Space O(1): three rolling variables.
# The key insight: sold state prevents buying the very next day after selling.
# Follow-up: Stock with Transaction Fee (LC 714) — subtract fee on sell; same two-state DP.
# Follow-up: At most k transactions (LC 188) — extend state to (day, k, holding)."
```


Dynamic Programming

#377 Combination Sum IV

MED

Open on LeetCode

Array · Dynamic Programming

Lists: Grind 169

Problem

Given an array of distinct integers nums and a target integer target, return the number of possible combinations that add up to target.
The test cases are generated so that the answer can fit in a 32-bit integer.

Example 1:

```
Input: nums = [1,2,3], target = 4
Output: 7
Explanation:
The possible combination ways are:
(1, 1, 1, 1)
(1, 1, 2)
(1, 2, 1)
(1, 3)
```

Example 2:

```
Input: nums = [9], target = 3
Output: 0
```

Constraints:

- 1 <= nums.length <= 200
- 1 <= nums[i] <= 1000
- All the elements of nums are unique.
- 1 <= target <= 1000

Follow up: What if negative numbers are allowed in the given array? How does it change the problem? What limitation we need to add to the question to allow negative numbers?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given distinct positive integers, return number of ORDERED sequences (not combinations)
# that sum to target. Different orderings count as different. Right?"
#
# "nums=[1,2,3], target=4:
# (1,1,1,1),(1,1,2),(1,2,1),(2,1,1),(2,2),(1,3),(3,1) → 7 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: count(target) = sum of count(target-num) for each num <= target.
# Time: O(target^n) without memo. With memo: O(target * n). Space: O(target)."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP: dp[i] = number of ways to reach sum i.
# dp[0]=1 (base). For each i from 1 to target: dp[i] = sum(dp[i-n] for n in nums if n<=i).
# Time: O(target * len(nums)). Space: O(target)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3], target=4:
# dp[0]=1. dp[1]=dp[0]=1. dp[2]=dp[1]+dp[0]=2. dp[3]=dp[2]+dp[1]+dp[0]=4.
# dp[4]=dp[3]+dp[2]+dp[1]=4+2+1=7 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(target*n): target positions × n numbers each. Space O(target).
# ORDER matters here (unlike Combination Sum III) — inner loop iterates nums at every position.
# For unordered combinations (LC 39), iterate nums in the outer loop.
# Follow-up: if negative numbers allowed — infinite loops possible; add a limit on sequence length.
# Follow-up: count modulo 10^9+7 — just add mod to each dp update."
```

Dynamic Programming

#416 Partition Equal Subset Sum

MED

Open on LeetCode

Array · Dynamic Programming

Lists: Grind 169

Problem

Given an integer array nums, return true if you can partition the array into two subsets such that the sum of the elements in both subsets is equal or false otherwise.

Example 1:

```
Input: nums = [1,5,11,5]
Output: true
Explanation: The array can be partitioned as [1, 5, 5] and [11].
```

Example 2:

```
Input: nums = [1,2,3,5]
Output: false
Explanation: The array cannot be partitioned into equal sum subsets.
```

Constraints:

- 1 <= nums.length <= 200
- 1 <= nums[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Can the array be partitioned into two subsets with equal sum?
# Equivalently: can any subset sum to total/2? Right?"
#
# "nums=[1,5,11,5]: total=22, target=11. [1,5,5] sums to 11 → True ✓
# nums=[1,2,3,5]: total=11 (odd) → False ✓"

# PHASE 2 — BRUTE FORCE
# "Try all 2^n subsets, check if any sums to total//2.
# Time: O(2^n). Space: O(n) recursion."

# PHASE 3 — OPTIMIZE
# "0/1 Knapsack DP: dp[j] = can we form sum j using some subset?
# Process each number: update dp from right to left (avoid reuse).
# dp[0]=True. For each num: for j from target down to num: dp[j] |= dp[j-num].
# Time: O(n*target). Space: O(target)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,5,11,5], target=11:
# num=1: dp[1]=dp[0]=True. → dp=[T,T,F,...,F].
# num=5: dp[6]=dp[1]=T, dp[5]=dp[0]=T. dp=[T,T,F,F,F,T,T,F,...].
# num=11: dp[11]=dp[0]=True → return True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*target): n numbers × target positions. Space O(target): 1D boolean array.
# Right-to-left update prevents using the same number twice (0/1 knapsack vs unbounded).
# Follow-up: Subset Sum (exact sum) — same DP, return dp[target].
# Follow-up: Last Stone Weight II (LC 1049) — minimize |S1-S2|; same partition approach."
```


Dynamic Programming

#435 Non-overlapping Intervals

MED

Open on LeetCode

Array · Dynamic Programming · Greedy · Sorting

Lists: Grind 169

Problem

Given an array of intervals intervals where intervals[i] = [starti, endi], return the minimum number of intervals you need to remove to make the rest of the intervals non-overlapping.

Note that intervals which only touch at a point are non-overlapping. For example, [1, 2] and [2, 3] are non-overlapping.

Example 1:

```
Input: intervals = [[1,2],[2,3],[3,4],[1,3]]
Output: 1
Explanation: [1,3] can be removed and the rest of the intervals are non-overlapping.
```

Example 2:

```
Input: intervals = [[1,2],[1,2],[1,2]]
Output: 2
Explanation: You need to remove two [1,2] to make the rest of the intervals non-overlapping.
```

Example 3:

```
Input: intervals = [[1,2],[2,3]]
Output: 0
Explanation: You don't need to remove any of the intervals since they're already non-overlapping.
```

Constraints:

- 1 <= intervals.length <= 105
- intervals[i].length == 2
- -5 * 104 <= starti < endi <= 5 * 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given intervals, find the minimum number to remove so the rest don't overlap.
# Equivalent to: find maximum non-overlapping intervals to KEEP, then answer = n - keep. Right?"
#
# "intervals=[[1,2],[2,3],[3,4],[1,3]]: remove [1,3] → 3 non-overlapping → 1 removal ✓
# intervals=[[1,2],[1,2],[1,2]]: remove 2 → 1 remaining ✓"

# PHASE 2 — BRUTE FORCE
# "Try all subsets of intervals; return n minus size of largest non-overlapping subset.
# Time: O(2^n * n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Greedy: sort by END time. Greedily keep intervals with earliest end that don't overlap.
# Track end of last kept interval. If next interval starts before that end, skip it (remove).
# Time: O(n log n) for sort. Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# intervals=[[1,2],[2,3],[3,4],[1,3]] sorted by end: [[1,2],[2,3],[1,3],[3,4]]:
# [1,2]: 1>=-inf → keep, last_end=2.
# [2,3]: 2>=2 → keep, last_end=3.
# [1,3]: 1<3 → remove. removed=1.
# [3,4]: 3>=3 → keep. return 1 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n log n): dominated by sort. Space O(1): one variable.
# Sorting by END (not start) is the key greedy insight — keeps the most room for future intervals.
# Follow-up: Meeting Rooms I (LC 252) — just check if any two overlap (sort by start, compare consecutive).
# Follow-up: Minimum Number of Arrows (LC 452) — burst balloons; same greedy as this problem."
```

Dynamic Programming

#516 Longest Palindromic Subsequence

MED

Open on LeetCode

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given a string s, find the longest palindromic subsequence's length in s.
A subsequence is a sequence that can be derived from another sequence by deleting some or no elements without changing the order of the remaining elements.

Example 1:

```
Input: s = "bbbab"
Output: 4
Explanation: One possible longest palindromic subsequence is "bbbb".
```

Example 2:

```
Input: s = "cbbd"
Output: 2
Explanation: One possible longest palindromic subsequence is "bb".
```

Constraints:

- 1 <= s.length <= 1000
- s consists only of lowercase English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest palindromic subsequence in string s.
# Subsequence: characters need not be contiguous. Right?"
#
# "s='bbbab': 'bbbb' → 4 ✓ s='cbbd': 'bb' → 2 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: lps(i,j) = longest palindromic subsequence in s[i..j].
# If s[i]==s[j]: 2+lps(i+1,j-1). Else max(lps(i+1,j), lps(i,j-1)).
# Time: O(2^n) without memo. Space: O(n) stack."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP: dp[i][j] = LPS in s[i..j]. Fill by increasing length.
# Observation: LPS(s) = LCS(s, reverse(s)).
# Space optimised: O(n) with two rolling arrays."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='bbbab':
# Full 2D: dp[0][4]=4 (bbbb). Standard O(n²) time O(n²) space is cleaner to trace.
# s='bbbab': LCS with reverse 'babbb': LCS='bbbb'=4 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n²): fill n×n DP table. Space O(n): two rolling arrays.
# Equivalently: LPS(s) = LCS(s, s[::-1]) — same recurrence structure.
# Follow-up: Longest Palindromic Substring (LC 5) — contiguous version.
# Follow-up: Palindrome Partitioning II (LC 132) — min cuts to make all substrings palindromes."
```


Dynamic Programming

#576 Out of Boundary Paths

MED

Open on LeetCode

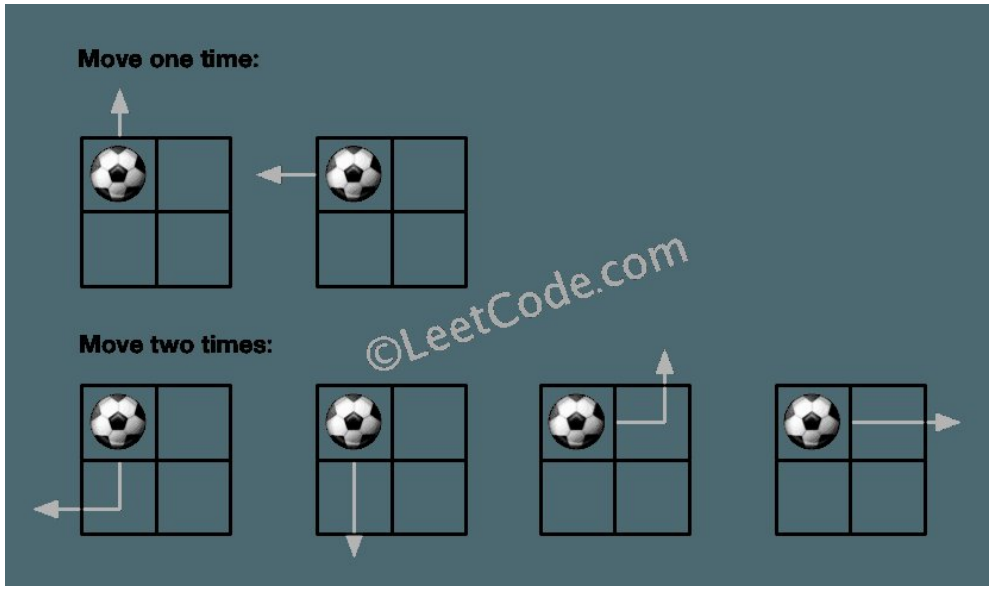
Dynamic Programming

Lists: Denny Zhang

Problem

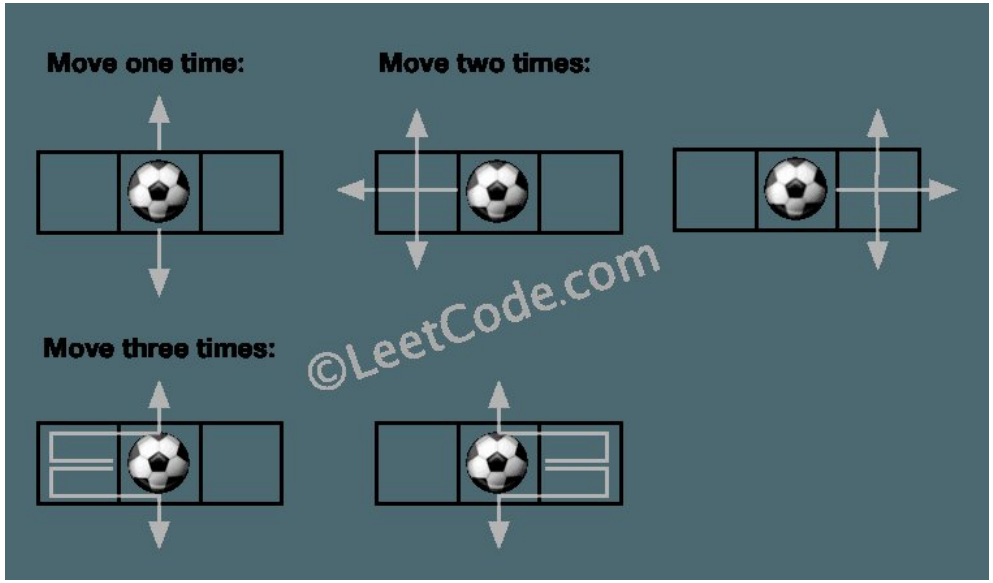
There is an m x n grid with a ball. The ball is initially at the position [startRow, startColumn]. You are allowed to move the ball to one of the four adjacent cells in the grid (possibly out of the grid crossing the grid boundary). You can apply at most maxMove moves to the ball.
Given the five integers m, n, maxMove, startRow, startColumn, return the number of paths to move the ball out of the grid boundary. Since the answer can be very large, return it modulo 109 + 7.

Example 1:




```
Input: m = 2, n = 2, maxMove = 2, startRow = 0, startColumn = 0
Output: 6
```

Example 2:




```
Input: m = 1, n = 3, maxMove = 3, startRow = 0, startColumn = 1
Output: 12
```

Constraints:

- 1 <= m, n <= 50
- 0 <= maxMove <= 50
- 0 <= startRow < m
- 0 <= startColumn < n

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "An m×n grid. Ball starts at (startRow, startCol). In exactly maxMove moves,
# count paths that take the ball OUT of the boundary. Return modulo 10^9+7. Right?"
#
# "m=2,n=2,maxMove=2,startRow=0,startCol=0: 6 paths exit in at most 2 moves ✓"

# PHASE 2 — BRUTE FORCE
# "DFS/recursion with memo: count(r,c,moves) = sum of count for each 4-direction.
# If r,c out of bounds → 1 (found a path). If moves=0 and in bounds → 0.
# Time: O(m*n*maxMove). Space: O(m*n*maxMove) memo."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP: dp[k][r][c] = number of paths of exactly k moves starting at (r,c) that exit.
# Build from k=1 to maxMove, accumulate total.
# Space optimised: O(m*n) with two 2D arrays (current and previous step)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# m=2,n=2,maxMove=2,start=(0,0):
# dp=[[1,0],[0,0]]. move1: (0,0)→up(-1,0) OOB count+=1, left(0,-1) OOB count+=1,
# right(0,1)→ndp[0][1]+=1, down(1,0)→ndp[1][0]+=1. ndp=[[0,1],[1,0]], count=2.
# move2: ndp[0][1]: up OOB+1=3, right OOB+1=4, left→[0,0]+=1, down→[1,1]+=1.
# ndp[1][0]: down OOB+1=5, left OOB+1=6, up→[0,0]+=1, right→[1,1]+=1. count=6 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(maxMove * m * n). Space O(m*n): two rolling grids.
# Each step propagates the ball's position to neighboring cells or exits.
# Follow-up: Knight Probability in Chessboard (LC 688) — same DP structure with knight moves.
# Follow-up: if the ball can stay in bounds, track the stay probability separately."
```


Dynamic Programming

☐ #1143 Longest Common Subsequence

ME D

Open on LeetCode

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings text1 and text2, return the length of their longest common subsequence. If there is no common subsequence, return 0.

A subsequence of a string is a new string generated from the original string with some characters (can be none) deleted without changing the relative order of the remaining characters.

- **For example, "ace" is a subsequence of "abcde".**

A common subsequence of two strings is a subsequence that is common to both strings.

Example 1:

```
Input: text1 = "abcde", text2 = "ace"
Output: 3
Explanation: The longest common subsequence is "ace" and its length is 3.
```

Example 2:

```
Input: text1 = "abc", text2 = "abc"
Output: 3
Explanation: The longest common subsequence is "abc" and its length is 3.
```

Example 3:

```
Input: text1 = "abc", text2 = "def"
Output: 0
Explanation: There is no such common subsequence, so the result is 0.
```

Constraints:

- **1 <= text1.length, text2.length <= 1000**
- **text1 and text2 consist of only lowercase English characters.**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Find the length of the longest common subsequence of two strings.
# Subsequence: characters need not be contiguous. Right?"
#
# "text1='abcde', text2='ace': 'ace' → length 3 ✓
# text1='abc', text2='abc': 'abc' → 3 ✓ text1='abc', text2='def': '' → 0 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: lcs(i,j) on text1[:i], text2[:j].
# If text1[i]==text2[j]: 1+lcs(i-1,j-1). Else max(lcs(i-1,j), lcs(i,j-1)).
# Time: O(2^(m+n)) without memo. Space: O(m+n) stack."

# PHASE 3 — OPTIMIZE
# "Bottom-up 2D DP: dp[i][j] = LCS of text1[:i], text2[:j].
# Space optimised: two rolling arrays O(min(m,n))."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# text1='abcde', text2='ace':
# After full DP: prev[3] = 3 ✓ (standard LCS table)
# i=1('a'): curr[1]=1('a'=='a'). i=2('b'): curr[1]=1,curr[2]=1('b'≠'c').
# i=3('c'): curr[2]=2('c'=='c'). i=4('d'): curr[2]=2. i=5('e'): curr[3]=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n): fill DP table. Space O(min(m,n)): rolling arrays.
# LCS is foundational: used in diff tools, bioinformatics, plagiarism detection.
# Follow-up: print the actual LCS — backtrack through the DP table.
# Follow-up: Edit Distance (LC 72) extends LCS with insert/delete/replace costs."
```

Bit Manipulation

Round 8 · Low · Medium · 4 questions

Bit Manipulation

#78 Subsets

MED

Open on LeetCode

Array · Backtracking · Bit Manipulation

Lists: Grind 169

Problem

Given an integer array nums of unique elements, return all possible subsets (the power set).
The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,3]
Output: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- **1 <= nums.length <= 10**
- **-10 <= nums[i] <= 10**
- **All the numbers of nums are unique.**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given an array of unique integers, return all possible subsets (power set). Right?"
#
# "nums=[1,2,3]: [[],[1],[2],[1,2],[3],[1,3],[2,3],[1,2,3]] ✓ (any order)"

# PHASE 2 — BRUTE FORCE
# "Backtracking: at each index, choose to include or exclude. Build all 2^n subsets.
# Time: O(n * 2^n). Space: O(n) recursion depth."

# PHASE 3 — OPTIMIZE
# "Bit manipulation: iterate 0..2^n-1. Each integer is a bitmask where bit i=1 means include nums[i].
# Time: O(n * 2^n). Space: O(1) extra (output excluded).
# Same complexity but elegant integer iteration."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,3], n=3: masks 0..7.
# mask=0(000): []. mask=1(001): [1]. mask=2(010): [2]. mask=3(011): [1,2].
# mask=4(100): [3]. mask=5(101): [1,3]. mask=6(110): [2,3]. mask=7(111): [1,2,3]. ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*2^n): 2^n subsets, each O(n) to build. Space O(1) extra.
# Both backtracking and bitmask approaches have identical complexity.
# Backtracking is more generalizable; bitmask is concise for small n.
# Follow-up: Subsets II (LC 90) — input has duplicates; sort + skip duplicates in backtracking.
# Follow-up: Combination Sum (LC 39) — subsets constrained to a target sum."
```

Bit Manipulation

#90 Subsets II

MED

Open on LeetCode

Array · Backtracking · Bit Manipulation

Lists: Denny Zhang

Problem

Given an integer array nums that may contain duplicates, return all possible subsets (the power set).
The solution set must not contain duplicate subsets. Return the solution in any order.

Example 1:

```
Input: nums = [1,2,2]
Output: [[],[1],[1,2],[1,2,2],[2],[2,2]]
```

Example 2:

```
Input: nums = [0]
Output: [[],[0]]
```

Constraints:

- 1 <= nums.length <= 10
- -10 <= nums[i] <= 10

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given an array that MAY contain duplicates, return all unique subsets. Right?"
#
# "nums=[1,2,2]: [[],[1],[1,2],[1,2,2],[2],[2,2]] ✓ (no duplicates in output)"

# PHASE 2 — BRUTE FORCE
# "Generate all subsets as sorted tuples, add to a set to deduplicate.
# Time: O(n * 2^n). Space: O(2^n) for the set."

# PHASE 3 — OPTIMIZE
# "Sort nums first. In backtracking, skip duplicate elements at the same recursion level
# (not the first occurrence, but subsequent same values at the same start position).
# If nums[i] == nums[i-1] and i > start, skip.
# Time: O(n * 2^n). Space: O(n) recursion — no dedup set needed."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# nums=[1,2,2] sorted:
# backtrack(0,[]): append []. i=0→path=[1], backtrack(1):
# append [1]. i=1→[1,2],backtrack(2): append[1,2]. i=2→[1,2,2]→append. i=3 done.
# i=2: nums[2]==nums[1] and 2>1 → skip.
# i=1→path=[2],backtrack(2): append[2]. i=2→[2,2]→append.
# i=2: nums[2]==nums[1] and 2>0 → skip.
# result=[[],[1],[1,2],[1,2,2],[2],[2,2]] ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*2^n): worst case all unique. Space O(n): recursion depth.
# The sort + skip-duplicate pattern is the standard dedup technique for backtracking.
# Follow-up: Permutations II (LC 47) — same skip-duplicate pattern for permutations.
# Follow-up: Combination Sum II (LC 40) — target sum with duplicates, same skip pattern."
```


Bit Manipulation

#287 Find the Duplicate Number

MED

Open on LeetCode

Array · Two Pointers · Binary Search · Bit Manipulation

Lists: Grind 169

Problem

Given an array of integers nums containing n + 1 integers where each integer is in the range [1, n] inclusive.
There is only one repeated number in nums, return this repeated number.
You must solve the problem without modifying the array nums and using only constant extra space.

Example 1:

```
Input: nums = [1,3,4,2,2]
Output: 2
```

Example 2:

```
Input: nums = [3,1,3,4,2]
Output: 3
```

Example 3:

```
Input: nums = [3,3,3,3,3]
Output: 3
```

Constraints:

- 1 <= n <= 105
- nums.length == n + 1
- 1 <= nums[i] <= n
- All the integers in nums appear only once except for precisely one integer which appears two or more times.

Follow up:

- How can we prove that at least one duplicate number must exist in nums?
- Can you solve the problem in linear runtime complexity?

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "n+1 integers in range [1,n]; exactly one number is duplicated (may appear 2+ times).
# Find the duplicate. Must not modify array, must use O(1) extra space. Right?"
#
# "nums=[1,3,4,2,2]: duplicate=2 ✓
# nums=[3,1,3,4,2]: duplicate=3 ✓"

# PHASE 2 — BRUTE FORCE
# "Use a hash set: for each number, if already seen, it's the duplicate.
# Time: O(n). Space: O(n) for the set. — Violates the O(1) space constraint."

# PHASE 3 — OPTIMIZE
# "Floyd's cycle detection — treat nums as a linked list where index i points to nums[i].
# Because one value is duplicated, two indices point to the same next node → cycle.
# Phase 1: find meeting point of slow (1 step) and fast (2 steps).
# Phase 2: reset slow to index 0, advance both 1 step — they meet at cycle entry = duplicate.
# Time: O(n). Space: O(1). Does not modify the array."

# PHASE 4 — CLEAN CODE
        # Phase 1: detect cycle
        # Phase 2: find cycle entry

# PHASE 5 — TEST & DEBUG
# nums=[1,3,4,2,2] (indices: 0→1, 1→3, 2→4, 3→2, 4→2):
# Phase1: slow=1,fast=1→slow=3,fast=4→slow=2,fast=2→meet at 2.
# Phase2: slow=nums[0]=1, fast=2. slow=3,fast=4. slow=2,fast=2. meet at 2 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n). Space O(1). Does not sort or modify the array.
# Binary search alternative: count numbers <= mid; if count > mid, duplicate is in [1,mid].
# O(n log n) time, O(1) space — simpler to explain in an interview.
# Follow-up: if multiple duplicates allowed, binary search no longer works directly.
# Follow-up: Find All Duplicates (LC 442) — mark visited by negating at index."
```

Bit Manipulation

⋆ #1506 Find Root of N-Ary Tree ⋆

MED

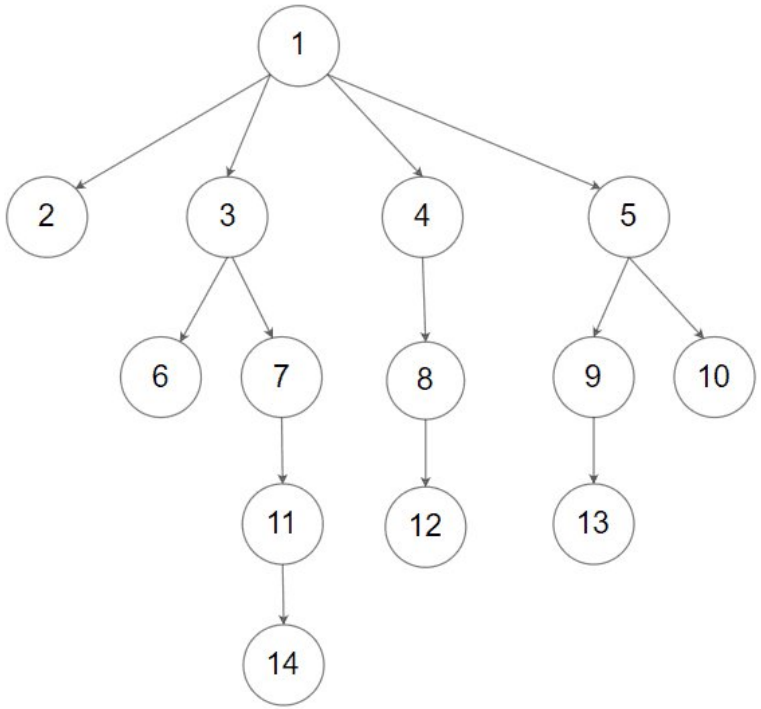
Open on LeetCode

Hash Table · Bit Manipulation · Tree · DFS

Lists: ⋆ Premium | Premium 98

Problem

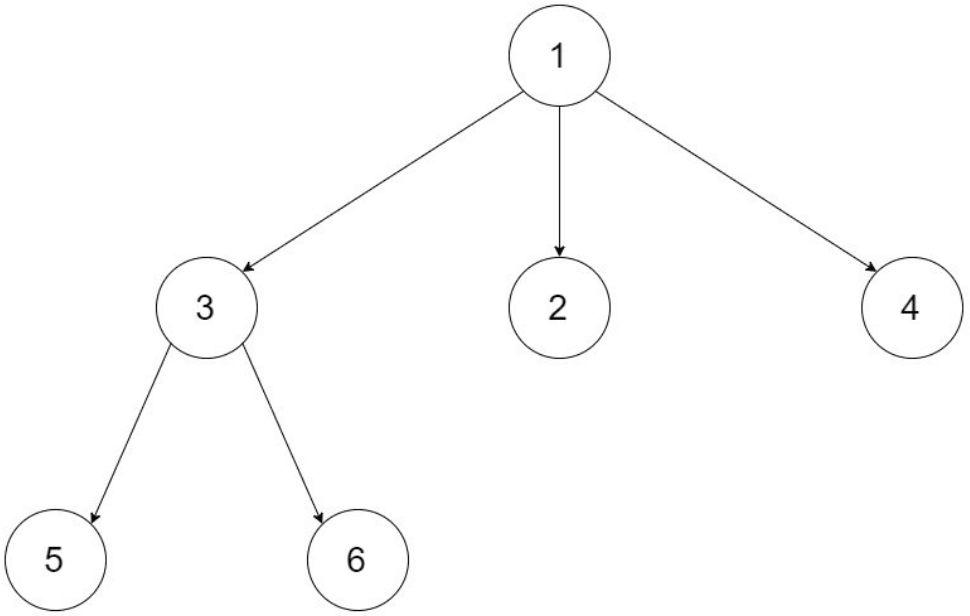
You are given all the nodes of an N-ary tree as an array of Node objects, where each node has a unique value.
Return the root of the N-ary tree.
Custom testing:
An N-ary tree can be serialized as represented in its level order traversal where each group of children is separated by the null value (see examples).



For example, the above tree is serialized as [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14].
The testing will be done in the following way:

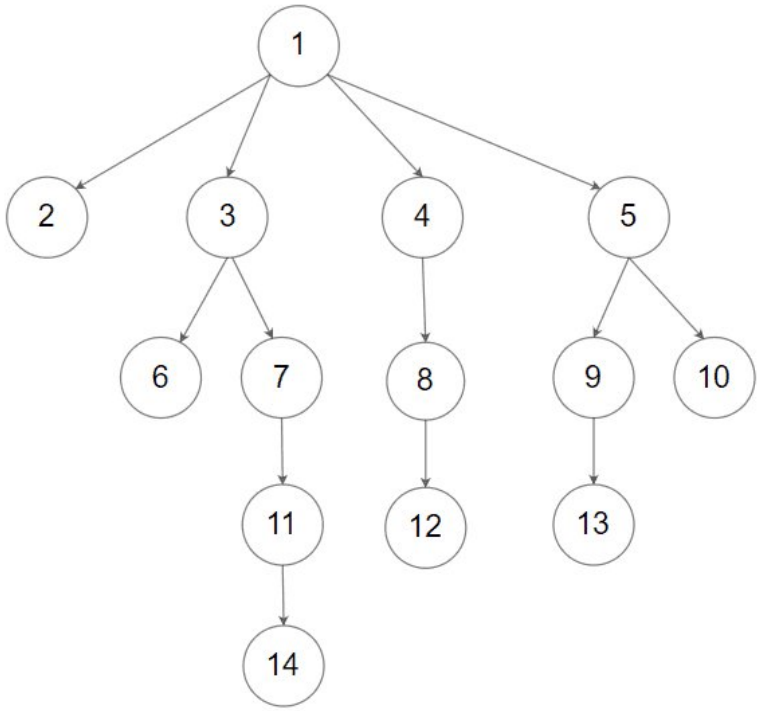
1. The input data should be provided as a serialization of the tree.
2. The driver code will construct the tree from the serialized input data and put each Node object into an array in an arbitrary order.
3. The driver code will pass the array to findRoot, and your function should find and return the root Node object in the array.
4. The driver code will take the returned Node object and serialize it. If the serialized value and the input data are the same, the test passes.

Example 1:




```
Input: tree = [1,null,3,2,4,null,5,6]
Output: [1,null,3,2,4,null,5,6]
Explanation: The tree from the input data is shown above.
The driver code creates the tree and gives findRoot the Node objects in an arbitrary order.
For example, the passed array could be [Node(5),Node(4),Node(3),Node(6),Node(2),Node(1)] or
[Node(2),Node(6),Node(1),Node(3),Node(5),Node(4)].
The findRoot function should return the root Node(1), and the driver code will serialize it and compare with the input data.
The input data and serialized Node(1) are the same, so the test passes.
```

Example 2:




```
Input: tree = [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
Output: [1,null,2,3,4,5,null,null,6,7,null,8,null,9,10,null,null,11,null,12,null,13,null,null,14]
```

Constraints:

- **The total number of nodes is between [1, 5 * 104].**
- **Each node has a unique value.**

Follow up:

- **Could you solve this problem in constant space complexity with a linear time algorithm?**

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Given all nodes of an N-ary tree in an array (in random order), find the root.
# Each non-root node appears as a child exactly once.
# Cannot access node.parent. Right?"
#
# "nodes=[1,2,3,4,5,6], tree root=1 with children [3,2,4], node 3 has children [5,6]:
# XOR all node values XOR all child values → root value. ✓"

# PHASE 2 — BRUTE FORCE
# "Build a set of all child node values. Root is the node whose value is NOT in that set.
# Time: O(n) where n = total nodes including all children. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "XOR trick: XOR all node values; XOR all children values.
# Every non-root node's value appears once as a node and once as a child → cancels to 0.
# Root appears only as a node → its value remains.
# Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# tree nodes [1,2,3,4,5,6] with root=1, children={3,2,4}, 3's children={5,6}:
# node vals XOR: 1^2^3^4^5^6. child vals XOR: 3^2^4^5^6.
# Combined: 1^2^3^4^5^6^3^2^4^5^6 = 1 (all others cancel). root=node(1) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): two linear passes. Space O(1): one XOR variable.
```



```
# Same XOR cancellation as Single Number (LC 136).
# Follow-up: if node values aren't unique, XOR fails — use the child-set approach.
# Follow-up: find all roots if the input is a forest (multiple disconnected trees) — same trick, collect all non-child values."
```

Math

Round 8 · Low · Medium · 5 questions

Math

#7 Reverse Integer

MED

Open on LeetCode

Math

Lists: Grind 169

Problem

Given a signed 32-bit integer x, return x with its digits reversed. If reversing x causes the value to go outside the signed 32-bit integer range [-231, 231 - 1], then return 0.

Assume the environment does not allow you to store 64-bit integers (signed or unsigned).

Example 1:

```
Input: x = 123
Output: 321
```

Example 2:

```
Input: x = -123
Output: -321
```

Example 3:

```
Input: x = 120
Output: 21
```

Constraints:

- -231 <= x <= 231 - 1

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Reverse the digits of a 32-bit signed integer. Return 0 if the reversed integer overflows
# [-2³¹, 2³¹-1]. Right?"
#
# "x=123 → 321 ✓ x=-123 → -321 ✓ x=120 → 21 ✓ x=1534236469 → 0 (overflow) ✓"

# PHASE 2 — BRUTE FORCE
# "Convert to string, reverse, convert back. Check 32-bit bounds.
# Time: O(d). Space: O(d)."

# PHASE 3 — OPTIMIZE
# "Build reversed integer digit by digit without string conversion.
# Check for overflow before each multiplication — avoids relying on 64-bit arithmetic.
# Time: O(d). Space: O(1)."

# PHASE 4 — CLEAN CODE
            # Check overflow before appending digit

# PHASE 5 — TEST & DEBUG
# x=123: digits extracted: 3,2,1. result: 3,32,321. 321 ≤ INT_MAX ✓. return 321 ✓.
# x=-123: sign=-1,x=123. Same as above. return -321 ✓.
# x=1534236469: eventually result*10+digit > INT_MAX → return 0 ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(d): one pass through digits. Space O(1): integer variables only.
# Overflow check: if result > (INT_MAX - digit) // 10, adding would overflow.
# Follow-up: Palindrome Number (LC 9) — only reverse half for palindrome check.
# Follow-up: if working in a language without 64-bit integers, this overflow check is critical."
```

Math

#50 Pow(x, n)

MED

Open on LeetCode

Math · Recursion

Lists: Grind 169

Problem

Implement pow(x, n), which calculates x raised to the power n (i.e., xn).

Example 1:

```
Input: x = 2.00000, n = 10
Output: 1024.00000
```

Example 2:

```
Input: x = 2.10000, n = 3
Output: 9.26100
```

Example 3:

```
Input: x = 2.00000, n = -2
Output: 0.25000
Explanation: 2-2 = 1/22 = 1/4 = 0.25
```

Constraints:

- -100.0 < x < 100.0
- -231 <= n <= 231-1
- n is an integer.
- Either x is not zero or n > 0.
- -104 <= xn <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement pow(x, n) — x raised to the power n. n can be negative. Right?"
#
# "pow(2.0, 10) → 1024.0 ✓ pow(2.1, 3) → 9.261 ✓ pow(2.0, -2) → 0.25 ✓"
# PHASE 2 — BRUTE FORCE
# "Multiply x by itself n times.
# Time: O(n). Space: O(1). TLEs for large n."
# PHASE 3 — OPTIMIZE
# "Fast exponentiation (exponentiation by squaring).
# If n is even: x^n = (x²)^(n/2). If n is odd: x^n = x * x^(n-1).
# Halves n each step → O(log n) multiplications.
# Time: O(log n). Space: O(1) iteratively."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# x=2.0, n=10:
# n=10(even): x=4,n=5. n=5(odd): result=4,x=16,n=2. n=2(even): x=256,n=1.
# n=1(odd): result=4*256=1024,x=65536,n=0. return 1024.0 ✓
#
# x=2.0, n=-2: x=0.5,n=2. n=2(even):x=0.25,n=1. n=1(odd):result=0.25,n=0. return 0.25 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n): halve the exponent each step. Space O(1): iterative.
# Recursive version is O(log n) time but O(log n) stack space.
# Follow-up: integer exponentiation with modular arithmetic — same squaring, add mod at each step.
# Follow-up: matrix exponentiation (for Fibonacci in O(log n)) — same pattern with matrices."
```


Math

#380 Insert Delete GetRandom O(1)

MED

Open on LeetCode

Array · Hash Table · Math · Design

Lists: Grind 169

Problem

Implement the RandomizedSet class:

- RandomizedSet() Initializes the RandomizedSet object.
- bool insert(int val) Inserts an item val into the set if not present. Returns true if the item was not present, false otherwise.
- bool remove(int val) Removes an item val from the set if present. Returns true if the item was present, false otherwise.
- int getRandom() Returns a random element from the current set of elements (it's guaranteed that at least one element exists when this method is called). Each element must have the same probability of being returned.

You must implement the functions of the class such that each function works in average O(1) time complexity.

Example 1:

```
Input
["RandomizedSet", "insert", "remove", "insert", "getRandom", "remove", "insert", "getRandom"]
[[], [1], [2], [2], [], [1], [2], []]
Output
[null, true, false, true, 2, true, false, 2]

Explanation
RandomizedSet randomizedSet = new RandomizedSet();
```

Constraints:

- -231 <= val <= 231 - 1
- At most 2 * 105 calls will be made to insert, remove, and getRandom.
- There will be at least one element in the data structure when getRandom is called.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Design a set supporting insert(val)→bool, remove(val)→bool, getRandom()→val
# all in O(1) average time. getRandom returns each element with equal probability. Right?"
#
# "insert(1)→T, insert(2)→T, getRandom()→1 or 2 equally, remove(1)→T,
# getRandom()→2 always. ✓"
# PHASE 2 — BRUTE FORCE
# "Use a Python set. insert/remove/search: O(1). getRandom: convert to list O(n), pick random.
# Time: O(n) per getRandom. Space: O(n)."
# PHASE 3 — OPTIMIZE
# "Array + HashMap: array holds all values, map holds {val→array_index}.
# getRandom: pick a random array index — O(1).
# remove: swap target with last element, update map, pop last — O(1).
# insert: append to array, update map — O(1)."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# insert(1): vals=[1],idx={1:0}. insert(2): vals=[1,2],idx={1:0,2:1}.
# remove(1): last=2,i=0. vals[0]=2,idx[2]=0. pop→vals=[2],idx={2:0}. del idx[1].
# getRandom()→choice([2])→2 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "All ops O(1) average: array ops + dict ops.
# The swap-with-last trick avoids O(n) shifts when removing from the middle.
# Edge case: when removing the last element, swap is a no-op (last==val).
# Follow-up: Insert Delete GetRandom with duplicates (LC 381) — store sets of indices per value.
# Follow-up: weighted random — use a prefix-sum array + binary search for getRandom."
```

Math

#1017 Convert to Base -2

ME D

Open on LeetCode

Math

Lists: Denny Zhang

Problem

Given an integer n, return a binary string representing its representation in base -2.
Note that the returned string should not have leading zeros unless the string is "0".

Example 1:

```
Input: n = 2
Output: "110"
Explantion: (-2)2 + (-2)1 = 2
```

Example 2:

```
Input: n = 3
Output: "111"
Explantion: (-2)2 + (-2)1 + (-2)0 = 3
```

Example 3:

```
Input: n = 4
Output: "100"
Explantion: (-2)2 = 4
```

Constraints:

- 0 <= n <= 109

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Given integer n, return its representation in base -2 as a binary string.
# Negative base: (-2)^0=1, (-2)^1=-2, (-2)^2=4, (-2)^3=-8, ...
# Result has no leading zeros (except '0' itself). Right?"
#
# "n=2: 2 = (-2)^1*(-1) + (-2)^0*(0) + (-2)^2*... hmm: 4+(-2)=2 → '110' ✓
# n=3: '111' ✓ n=4: '100' ✓"

# PHASE 2 — BRUTE FORCE
# "Repeated division by -2. Remainder must be 0 or 1 (non-negative).
# If remainder is negative (when n is odd and negative), adjust: rem += 2, n += 1 (or n = n//(-2)).
# Time: O(log n). Space: O(log n)."

# PHASE 3 — OPTIMIZE
# "Same iterative approach — already O(log n). The key is ensuring remainder is 0 or 1.
# Python's % with negative base may return negative remainders; adjust to [0,1]."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# n=2: bits: 2&1=0, n=-(2>>1)=-1. (-1)&1=1, n=-(-1>>1)=1. 1&1=1, n=-(1>>1)=0.
# bits=[0,1,1]. reversed='110' ✓
# n=3: 3&1=1,n=-1. 1,n=1. 1,n=0. bits=[1,1,1]→'111' ✓
# n=0: return '0' ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(log n): number of bits in the representation.
# Space O(log n): bits array.
# The n = -(n>>1) trick works because: dividing by -2 = right-shift (divide by 2) then negate.
# Follow-up: Base -10 (general negative base) — same structure, replace 2 with |base|.
# Follow-up: decode a base-(-2) string back to integer — multiply by (-2)^position."
```


Math

#3160 Find the Number of Distinct Colors Among the Balls

ME D

Open on LeetCode

Array · Hash Table · Simulation

Lists: CodeSignal

Problem

You are given an integer limit and a 2D array queries of size n x 2.
There are limit + 1 balls with distinct labels in the range [0, limit]. Initially, all balls are uncolored. For every query in queries that is of the form [x, y], you mark ball x with the color y. After each query, you need to find the number of colors among the balls.
Return an array result of length n, where result[i] denotes the number of colors after ith query.
Note that when answering a query, lack of a color will not be considered as a color.

Example 1:

Input: limit = 4, queries = [[1,4],[2,5],[1,3],[3,4]]
Output: [1,2,2,3]
Explanation:

- After query 0, ball 1 has color 4.
- After query 1, ball 1 has color 4, and ball 2 has color 5.
- After query 2, ball 1 has color 3, and ball 2 has color 5.
- After query 3, ball 1 has color 3, ball 2 has color 5, and ball 3 has color 4.

Example 2:

Input: limit = 4, queries = [[0,1],[1,2],[2,2],[3,4],[4,5]]
Output: [1,2,2,3,4]
Explanation:

- After query 0, ball 0 has color 1.
- After query 1, ball 0 has color 1, and ball 1 has color 2.
- After query 2, ball 0 has color 1, and balls 1 and 2 have color 2.
- After query 3, ball 0 has color 1, balls 1 and 2 have color 2, and ball 3 has color 4.
- After query 4, ball 0 has color 1, balls 1 and 2 have color 2, ball 3 has color 4, and ball 4 has color 5.

Constraints:

- 1 <= limit <= 109
- 1 <= n == queries.length <= 105
- queries[i].length == 2
- 0 <= queries[i][0] <= limit
- 1 <= queries[i][1] <= 109

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "N balls (0 to limit). queries[i]=[ball,color]: color that ball. After each query return distinct color count."
#
# "limit=4, queries=[[1,4],[2,5],[1,3],[3,4]]:
# After q1:{4}→1. After q2:{4,5}→2. After q3:{3,5}→2. After q4:{3,5,4}→3 ✓"

# PHASE 2 — BRUTE FORCE
# "Dict ball→color. After each query, scan all values for distinct count.
# Time: O(n) per query. Space: O(n)."

# PHASE 3 — OPTIMIZE
# "color_count tracks how many balls have each color. Update O(1) on each query.
# len(color_count) = distinct colors."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# queries=[[1,4],[2,5],[1,3],[3,4]]:
# [1,4]: count={4:1}. distinct=1 ✓
# [2,5]: count={4:1,5:1}. distinct=2 ✓
# [1,3]: old=4,count[4]→0 del. count={5:1,3:1}. distinct=2 ✓
# [3,4]: count={5:1,3:1,4:1}. distinct=3 ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) per query. Space O(unique balls + colors). color_count keys = distinct active colors.
# Follow-up: range coloring (paint balls l..r) — needs segment tree / lazy propagation.
# Follow-up: undo queries — maintain a stack of (ball, old_color, new_color) and reverse ops."
```


· #1017 #2627 · · Contents

JavaScript

Round 8 · Low · Medium · 7 questions

JavaScript

☐ ⋆ #2627 Debounce ⋆ MED

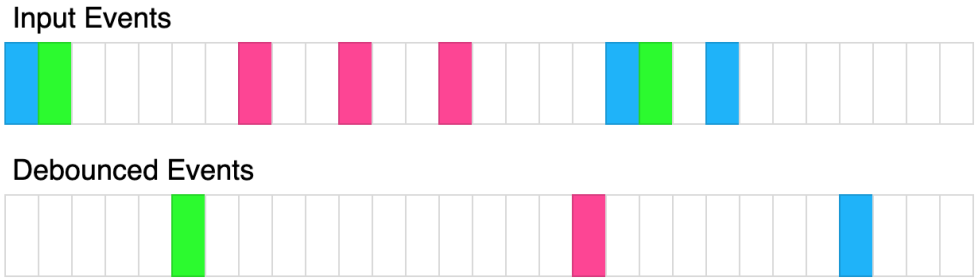
Open on LeetCode

JavaScript

Lists: ⋆ Premium | Premium 98

Problem

Given a function fn and a time in milliseconds t, return a debounced version of that function.
A debounced function is a function whose execution is delayed by t milliseconds and whose execution is cancelled if it is called again within that window of time. The debounced function should also receive the passed parameters.
For example, let's say t = 50ms, and the function was called at 30ms, 60ms, and 100ms.
The first 2 function calls would be cancelled, and the 3rd function call would be executed at 150ms.
If instead t = 35ms, The 1st call would be cancelled, the 2nd would be executed at 95ms, and the 3rd would be executed at 135ms.



The above diagram shows how debounce will transform events. Each rectangle represents 100ms and the debounce time is 400ms. Each color represents a different set of inputs.
Please solve it without using lodash's _.debounce() function.

Example 1:

```
Input:
t = 50
calls = [
  {"t": 50, inputs: [1]},
  {"t": 75, inputs: [2]}
]
Output: [{"t": 125, inputs: [2]}]
Explanation:
```

Example 2:

```
Input:
t = 20
calls = [
  {"t": 50, inputs: [1]},
  {"t": 100, inputs: [2]}
]
Output: [{"t": 70, inputs: [1]}, {"t": 120, inputs: [2]}]
Explanation:
```

Example 3:

```
Input:
t = 150
calls = [
  {"t": 50, inputs: [1, 2]},
  {"t": 300, inputs: [3, 4]},
  {"t": 300, inputs: [5, 6]}
]
Output: [{"t": 200, inputs: [1,2]}, {"t": 450, inputs: [5, 6]}]
```

Constraints:

- 0 <= t <= 1000
- 1 <= calls.length <= 10
- 0 <= calls[i].t <= 1000
- 0 <= calls[i].inputs.length <= 10

◆ Interview Approach · STAR-LC


```
# PHASE 1 — CLARIFY
# "debounce(fn,t): delays fn until t ms after the last call. Each new call resets the timer."
#
# "calls at t=0,30,60ms with t=50ms: fn fires once at t=110ms ✓"

# PHASE 2 — BRUTE FORCE
# "Track a timer id. On each call, cancel the previous timer and set a new one.
# Time: O(1) per call. Space: O(1)."

# PHASE 3 — OPTIMIZE
# "Same — this IS optimal. One clearTimeout + one setTimeout per call.
# JavaScript: use closure to hold the timer reference."

# PHASE 4 — CLEAN CODE
# JavaScript:
# function debounce(fn, t) {
# let timer = null;
# return function(...args) {
# clearTimeout(timer);
# timer = setTimeout(() => fn.apply(this, args), t);
# };
# }

# PHASE 5 — TEST & DEBUG
# t=0: cancel(null), set timer(50ms). t=30: cancel timer, set timer(50ms).
# t=60: cancel timer, set timer(50ms). No more calls → fn fires at t=110ms ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(1) per call: one cancel + one set. Space O(1): one timer reference.
# Debounce: wait until activity stops. Throttle: fire at most once per interval.
# Follow-up: leading debounce — fire immediately on first call, ignore for t ms after.
# Follow-up: Throttle (LC 2676) — fire at most once per t ms regardless of call frequency."
```

JavaScript

★ #2628 JSON Deep Equal ★

MED

Open on LeetCode

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given two values o1 and o2, return a boolean value indicating whether two values, o1 and o2, are deeply equal.
For two values to be deeply equal, the following conditions must be met:

- If both values are primitive types, they are deeply equal if they pass the === equality check.
- If both values are arrays, they are deeply equal if they have the same elements in the same order, and each element is also deeply equal according to these conditions.
- If both values are objects, they are deeply equal if they have the same keys, and the associated values for each key are also deeply equal according to these conditions.

You may assume both values are the output of JSON.parse. In other words, they are valid JSON.
Please solve it without using lodash's _.isEqual() function

Example 1:

```
Input: o1 = {"x":1,"y":2}, o2 = {"x":1,"y":2}
Output: true
Explanation: The keys and values match exactly.
```

Example 2:

```
Input: o1 = {"y":2,"x":1}, o2 = {"x":1,"y":2}
Output: true
Explanation: Although the keys are in a different order, they still match exactly.
```

Example 3:

```
Input: o1 = {"x":null,"L":[1,2,3]}, o2 = {"x":null,"L":["1","2","3"]}
Output: false
Explanation: The array of numbers is different from the array of strings.
```

Example 4:

```
Input: o1 = true, o2 = false
Output: false
Explanation: true !== false
```

Constraints:

- 1 <= JSON.stringify(o1).length <= 105
- 1 <= JSON.stringify(o2).length <= 105
- maxNestingDepth <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return true if two JSON values are deeply equal. Objects: all key-value pairs match (order irrelevant).
# Arrays: same length and same elements at each index. Right?"
#
# "areDeeplyEqual({a:1,b:2},{a:1,b:2})→true ✓ areDeeplyEqual([1,[2]],[1,[3]])→false ✓"

# PHASE 2 — BRUTE FORCE
# "JSON.stringify both — fails for object key-order differences.
# Proper: recursive comparison checking types, then structure."

# PHASE 3 — OPTIMIZE
# "Same recursive approach — already O(n) nodes. Check === first for fast-path."

# PHASE 4 — CLEAN CODE
# JavaScript:
# function areDeeplyEqual(o1, o2) {
# if (o1 === o2) return true;
# if (typeof o1 !== typeof o2 || o1 === null || o2 === null) return o1 === o2;
# if (Array.isArray(o1) !== Array.isArray(o2)) return false;
# if (Array.isArray(o1)) return o1.length===o2.length && o1.every((v,i)=>areDeeplyEqual(v,o2[i]));
# const k1=Object.keys(o1), k2=Object.keys(o2);
# return k1.length===k2.length && k1.every(k=>k in o2 && areDeeplyEqual(o1[k],o2[k]));
# }

# PHASE 5 — TEST & DEBUG
# areDeeplyEqual({a:{b:1}},{a:{b:1}}):
# both dicts, keys match. recurse 'a': both dicts, keys match. recurse 'b': 1==1 True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): visit each node once. Space O(d): max recursion depth.
# null requires special handling in JS: typeof null === 'object'.
# Follow-up: Differences Between Two Objects (LC 2700) — return diff instead of bool.
# Follow-up: handle circular refs — track visited pairs with WeakMap/set of pairs."
```


JavaScript

★ #2630 Memoize ★

MED

Open on LeetCode

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a function fn, return a memoized version of that function.
A memoized function is a function that will never be called twice with the same inputs. Instead it will return a cached value.
fn can be any function and there are no constraints on what type of values it accepts. Inputs are considered identical if they are === to each other.

Example 1:

```
Input:
getInputs = () => [[2,2],[2,2],[1,2]]
fn = function (a, b) { return a + b; }
Output: [{"val":4,"calls":1},{"val":4,"calls":1},{"val":3,"calls":2}]
Explanation:
const inputs = getInputs();
const memoized = memoize(fn);
for (const arr of inputs) {
```

Example 2:

```
Input:
getInputs = () => [[{},{}],[{},{}],[{},{}]]
fn = function (a, b) { return ({...a, ...b}); }
Output: [{"val":{},"calls":1},{"val":{},"calls":2},{"val":{},"calls":3}]
Explanation:
Merging two empty objects will always result in an empty object. It may seem like there should only be 1 call to fn() because of
cache-hits, however none of those objects are === to each other.
```

Example 3:

```
Input:
getInputs = () => { const o = {}; return [[o,o],[o,o],[o,o]]; }
fn = function (a, b) { return ({...a, ...b}); }
Output: [{"val":{},"calls":1},{"val":{},"calls":1},{"val":{},"calls":1}]
Explanation:
Merging two empty objects will always result in an empty object. The 2nd and 3rd third function calls result in a cache-hit.
This is because every object passed in is identical.
```

Constraints:

- 1 <= inputs.length <= 105
- 0 <= inputs.flat().length <= 105
- inputs[i][j] != NaN

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "memoize(fn): returns a cached version. Same args → return cached result, skip fn call."
#
# "memoize(sum): sum(2,3)→5 (called). sum(2,3)→5 (cached, fn not called). sum(1,2)→3 (called). ✓"

# PHASE 2 — BRUTE FORCE
# "Map with stringified args as key. Check on each call; compute and cache if miss."

# PHASE 3 — OPTIMIZE
# "Same O(1) lookup approach. For JS use JSON.stringify for complex/nested args."

# PHASE 4 — CLEAN CODE
# JavaScript:
# function memoize(fn) {
# const cache = new Map();
# return function(...args) {
# const key = JSON.stringify(args);
# if (!cache.has(key)) cache.set(key, fn.apply(this, args));
# return cache.get(key);
# };
# }

# PHASE 5 — TEST & DEBUG
# fn=lambda a,b: a+b. memoized(2,3): key='(2, 3)', miss→compute 5, cache. return 5.
# memoized(2,3): key='(2, 3)', hit → return 5 (fn not called) ✓.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "First call O(fn). Cache hit O(1). Space O(unique arg combinations * result size).
# JSON.stringify adds O(arg_size) per call — acceptable for numbers/strings.
# Follow-up: LRU-bounded memoize — evict LRU when cache exceeds limit.
# Follow-up: async memoize — cache the Promise, not the resolved value."
```

JavaScript

★ #2633 Convert Object to JSON String ★

MED

Open on LeetCode

JavaScript

Lists: ★ Premium | Premium 98

Problem

Given a value, return a valid JSON string of that value. The value can be a string, number, array, object, boolean, or null. The returned string should not include extra spaces. The order of keys should be the same as the order returned by Object.keys().

Please solve it without using the built-in JSON.stringify method.

Example 1:

```
Input: object = {"y":1,"x":2}
Output: {"y":1,"x":2}
Explanation:
Return the JSON representation.
Note that the order of keys should be the same as the order returned by Object.keys().
```

Example 2:

```
Input: object = {"a":"str","b":-12,"c":true,"d":null}
Output: {"a":"str","b":-12,"c":true,"d":null}
Explanation:
The primitives of JSON are strings, numbers, booleans, and null.
```

Example 3:

```
Input: object = {"key":{"a":1,"b":[{},null,"Hello"]}}
Output: {"key":{"a":1,"b":[{},null,"Hello"]}}
Explanation:
Objects and arrays can include other objects and arrays.
```

Example 4:

```
Input: object = true
Output: true
Explanation:
Primitive types are valid inputs.
```

Constraints:

- value is a valid JSON value
- 1 <= JSON.stringify(object).length <= 10^5
- maxNestingLevel <= 1000
- all strings contain only alphanumeric characters

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Implement JSON.stringify without using it. Handle null, bool, number, string, array, object."
#
# "jsonify({a:2,b:[1,2,3]}) → '{"a":2,"b":[1,2,3]}' ✓"
# PHASE 2 — BRUTE FORCE
# "Recursive: dispatch on type. Build string for each type appropriately."
# PHASE 3 — OPTIMIZE
# "Same — already O(n). Use join to avoid O(n^2) string concatenation."
# PHASE 4 — CLEAN CODE (same as brute — already clean)
# JavaScript:
# function jsonStringify(obj) {
# if (obj === null) return 'null';
# if (typeof obj === 'boolean') return String(obj);
# if (typeof obj === 'number') return String(obj);
# if (typeof obj === 'string') return `"${obj}"`;
# if (Array.isArray(obj)) return `[${obj.map(jsonStringify).join(',')}]`;
# return `{${Object.entries(obj).map(([k,v])=>`"${k}":${jsonStringify(v)}`).join(',')}}`;
# }
# PHASE 5 — TEST & DEBUG
# obj=[null,true,1,"foo"]:
# array. elements: 'null','true','1','"foo"'. joined: 'null,true,1,"foo"'. wrap: '[null,true,1,"foo"]' ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one visit per node. Space O(d): recursion depth.
# bool must be checked before number (isinstance(True,int) is True in Python).
# Follow-up: handle circular references — track visited with a Set.
# Follow-up: pretty-print with indentation — add indent parameter and track depth."
```


JavaScript

★ #2636 Promise Pool ★

MED

Open on LeetCode

JavaScript · Concurrency

Lists: ★ Premium | Premium 98

Problem

Given an array of asynchronous functions functions and a pool limit n, return an asynchronous function promisePool. It should return a promise that resolves when all the input functions resolve.

Pool limit is defined as the maximum number promises that can be pending at once. promisePool should begin execution of as many functions as possible and continue executing new functions when old promises resolve. promisePool should execute functions[i] then functions[i + 1] then functions[i + 2], etc. When the last promise resolves, promisePool should also resolve.

For example, if n = 1, promisePool will execute one function at a time in series. However, if n = 2, it first executes two functions. When either of the two functions resolve, a 3rd function should be executed (if available), and so on until there are no functions left to execute.

You can assume all functions never reject. It is acceptable for promisePool to return a promise that resolves any value.

Example 1:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 2
Output: [[300,400,500],500]
```

Example 2:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 5
Output: [[300,400,200],400]
```

Example 3:

```
Input:
functions = [
  () => new Promise(res => setTimeout(res, 300)),
  () => new Promise(res => setTimeout(res, 400)),
  () => new Promise(res => setTimeout(res, 200))
]
n = 1
Output: [[300,700,900],900]
```

Constraints:

- 0 <= functions.length <= 10
- 1 <= n <= 10

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Execute async functions with at most n concurrent. Return promise resolving when all done."
#
# "promisePool([fn1,fn2,fn3,fn4], n=2): run fn1+fn2, as each finishes start fn3 then fn4 ✓"
# PHASE 2 — BRUTE FORCE
# "Promise.all on all functions — no concurrency limit, may overwhelm resources."
# PHASE 3 — OPTIMIZE
# "n worker coroutines, each pulls from a shared queue.
# Each worker runs one fn at a time sequentially. n workers run in parallel.
# await all workers → all tasks complete."
# PHASE 4 — CLEAN CODE
# JavaScript:
# async function promisePool(functions, n) {
# const queue = [...functions];
# async function worker() {
# while (queue.length) await queue.shift()();
# }
# await Promise.all(Array.from({length:Math.min(n,functions.length)}, worker));
# }
# PHASE 5 — TEST & DEBUG
# [fn1,fn2,fn3,fn4], n=2:
# worker1: await fn1(), await fn3(). worker2: await fn2(), await fn4().
# Both workers run concurrently → max 2 concurrent at any time ✓
```

```
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time optimal: ceiling(total_tasks/n) rounds if tasks equal duration.
# Space O(n): n concurrent promises. Worker pattern handles unequal durations naturally.
# Follow-up: cancel on first failure — Promise.race with rejection propagation.
# Follow-up: retry failed tasks — wrap fn() in a retry loop inside the worker."
```


JavaScript

★ #2675 Array of Objects to Matrix ★

MED

Open on LeetCode

JavaScript · Array

Lists: ★ Premium | Premium 98

Problem

Write a function that converts an array of objects arr into a matrix m.
arr is an array of objects or arrays. Each item in the array can be deeply nested with child arrays and child objects. It can also contain numbers, strings, booleans, and null values.
The first row m should be the column names. If there is no nesting, the column names are the unique keys within the objects. If there is nesting, the column names are the respective paths in the object separated by ".".
Each of the remaining rows corresponds to an object in arr. Each value in the matrix corresponds to a value in an object. If a given object doesn't contain a value for a given column, the cell should contain an empty string "".
The columns in the matrix should be in lexographically ascending order.

Example 1:

```
Input:
arr = [
  {"b": 1, "a": 2},
  {"b": 3, "a": 4}
]
Output:
[
  ["a", "b"],
```

Example 2:

```
Input:
arr = [
  {"a": 1, "b": 2},
  {"c": 3, "d": 4},
  {}
]
Output:
[
```

Example 3:

```
Input:
arr = [
  {"a": {"b": 1, "c": 2}},
  {"a": {"b": 3, "d": 4}}
]
Output:
[
  ["a.b", "a.c", "a.d"],
```

Example 4:

```
Input:
arr = [
  [{"a": null}],
  [{"b": true}],
  [{"c": "x"}]
]
Output:
[
```

Example 5:

```
Input:
arr = [
{},
  {},
  {},
]
Output:
[
```

Constraints:

- arr is a valid JSON array
- 1 <= arr.length <= 1000
- unique keys <= 1000

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Convert array of objects to a matrix. Row 0 = sorted flattened keys (dot notation for nested).
# Remaining rows = values for each object ('' if key missing). Right?"
#
```

```
# "[{a:1,b:2},{b:3,c:4}] → [['a','b','c'],[1,2,''],['',3,4]] ✓"
# PHASE 2 — BRUTE FORCE
# "Flatten each object with dot notation, collect all keys, build matrix row by row."
# PHASE 3 — OPTIMIZE
# "Same O(n*k) — already optimal. Single pass to collect keys."
# PHASE 4 — CLEAN CODE (same as brute — already clean)
# PHASE 5 — TEST & DEBUG
# arr=[{a:1,b:{c:2}},{a:3}]:
# flat: {'a':1,'b.c':2} and {'a':3}. keys=['a','b.c'].
# rows: [1,2] and [3,''] → [['a','b.c'],[1,2],[3,'']] ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*k*d): n objects, k unique keys, d nesting depth. Space O(n*k): matrix.
# Dot notation is the CSV-export standard for nested JSON.
# Follow-up: handle arrays inside objects — use bracket notation [0],[1].
# Follow-up: reverse (matrix to array of objects) — parse dot-paths back to nested."
```


JavaScript

★ #2700 Differences Between Two Objects ★

ME
D

Open on LeetCode

JavaScript

Lists: ★ Premium | Premium 98

Problem

Write a function that accepts two deeply nested objects or arrays obj1 and obj2 and returns a new object representing their differences.

The function should compare the properties of the two objects and identify any changes. The returned object should only contains keys where the value is different from obj1 to obj2.

For each changed key, the value should be represented as an array [obj1 value, obj2 value]. Keys that exist in one object but not in the other should not be included in the returned object. The end result should be a deeply nested object where each leaf value is a difference array.

When comparing two arrays, the indices of the arrays are considered to be their keys.

You may assume that both objects are the output of JSON.parse.

Example 1:

```
Input:
obj1 = {}
obj2 = {
  "a": 1,
"b": 2
}
Output: {}
Explanation: There were no modifications made to obj1. New keys "a" and "b" appear in obj2, but keys that are added or removed
should be ignored.
```

Example 2:

```
Input:
obj1 = {
  "a": 1,
  "v": 3,
  "x": [],
  "z": {
    "a": null
  }
```

Example 3:

```
Input:
obj1 = {
  "a": 5,
  "v": 6,
  "z": [1, 2, 4, [2, 5, 7]]
}
obj2 = {
  "a": 5,
```

Example 4:

```
Input:
obj1 = {
  "a": {"b": 1},
}
obj2 = {
  "a": [5],
}
Output:
```

Example 5:

```
Input:
obj1 = {
  "a": [1, 2, {}],
  "b": false
}
obj2 = {
  "b": false,
  "a": [1, 2, {}]
```

Constraints:

- obj1 and obj2 are valid JSON objects or arrays
- 2 <= JSON.stringify(obj1).length <= 104
- 2 <= JSON.stringify(obj2).length <= 104

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Return an object showing paths where two JSON objects differ.
```

```
# [obj1_val, obj2_val] at leaf differences. Skip keys present in only one object."
#
# "diff({a:1,b:2},{a:1,b:3}) → {b:[2,3]} ✓
# diff({a:{b:1}},{a:{b:2}}) → {a:{b:[1,2]}} ✓"

# PHASE 2 — BRUTE FORCE
# "Recursive: for each key in obj1 that also exists in obj2:
# if both are objects, recurse and collect non-empty sub-diffs.
# if values differ (or types differ), mark as [v1,v2]. If equal, skip."

# PHASE 3 — OPTIMIZE
# "Same — already O(n). The key insight: skip keys in only one object."

# PHASE 4 — CLEAN CODE (same as brute — already clean)
# JavaScript:
# function objDiff(obj1, obj2) {
# if (typeof obj1!=='object'||obj1===null||typeof obj2!=='object'||obj2===null)
# return obj1===obj2 ? {} : [obj1,obj2];
# const result={};
# for (const k of Object.keys(obj1)) {
# if (!(k in obj2)) continue;
# const diff=objDiff(obj1[k],obj2[k]);
# if (Object.keys(diff).length||Array.isArray(diff)) result[k]=diff;
# }
# return result;
# }

# PHASE 5 — TEST & DEBUG
# obj1={a:1,b:{c:2,d:3}}, obj2={a:1,b:{c:2,d:4}}:
# 'a': 1==1 → {}. 'b': both dicts, recurse. 'c': 2==2 → {}. 'd': 3≠4 → [3,4].
# result={b:{d:[3,4]}} ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n): one visit per node. Space O(d): recursion depth.
# null check: typeof null==='object' in JS — must check explicitly.
# Follow-up: include keys present in only one object — mark as [value, undefined].
# Follow-up: JSON Deep Equal (LC 2628) — return bool instead of diff."
```


Round 9 · Low · Hard

Round 9

Low Priority · Hard
7 questions · 2 patterns

Dynamic Programming #10 #115 #123 #132 #312 #1000
Math #68

Dynamic Programming

Round 9 · Low · Hard · 6 questions

Dynamic Programming

#10 Regular Expression Matching HAR

Open on LeetCode

String · Dynamic Programming · Recursion

Lists: Denny Zhang

Problem

Given an input string s and a pattern p, implement regular expression matching with support for '.' and '*' where:

- '.' Matches any single character.
- '*' Matches zero or more of the preceding element.

Return a boolean indicating whether the matching covers the entire input string (not partial).

Example 1:

```
Input: s = "aa", p = "a"
Output: false
Explanation: "a" does not match the entire string "aa".
```

Example 2:

```
Input: s = "aa", p = "a*"
Output: true
Explanation: '*' means zero or more of the preceding element, 'a'. Therefore, by repeating 'a' once, it becomes "aa".
```

Example 3:

```
Input: s = "ab", p = ".*"
Output: true
Explanation: ".*" means "zero or more (*) of any character (.)".
```

Constraints:

- 1 <= s.length <= 20
- 1 <= p.length <= 20
- s contains only lowercase English letters.
- p contains only lowercase English letters, '.', and '*'.
- It is guaranteed for each appearance of the character '*', there will be a previous valid character to match.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "'.' matches any single char. '*' matches zero or more of the preceding element.
# Full string must match. Right?"
#
# "s='aa',p='a*'→true ✓ s='aa',p='a'→false ✓ s='aab',p='c*a*b'→true ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion with memo: match(i,j)=does s[i:] match p[j:]?
# If p[j+1]=='*': zero uses (match i,j+2) or one+ uses (first_char and match i+1,j).
# Time: O(m*n) with memo. Space: O(m*n)."

# PHASE 3 — OPTIMIZE
# "Bottom-up 2D DP. dp[i][j]=does s[i:] match p[j:]? Fill right-to-left, bottom-to-top.
# Space: O(n) with rolling rows."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='aab', p='c*a*b': c*→skip(j+2), a*→skip or use, b→match. dp[0]=True ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(n): two rolling rows.
# '*' has two branches: zero occurrences (skip j+2) or one-plus (consume s[i], stay at j).
# Follow-up: Wildcard Matching (LC 44) — '*' matches any sequence. Same DP, simpler transitions."
```

Dynamic Programming

#115 Distinct Subsequences HAR

Open on LeetCode

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given two strings s and t, return the number of distinct subsequences of s which equals t.

The test cases are generated so that the answer fits on a 32-bit signed integer.

Example 1:

```
Input: s = "rabbbit", t = "rabbit"
Output: 3
Explanation:
As shown below, there are 3 ways you can generate "rabbit" from s.
rabbbit
rabbbit
rabbbit
```

Example 2:

```
Input: s = "babgbag", t = "bag"
Output: 5
Explanation:
As shown below, there are 5 ways you can generate "bag" from s.
babgbag
babgbag
babgbag
babgbag
```

Constraints:

- 1 <= s.length, t.length <= 1000
- s and t consist of English letters.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Count distinct subsequences of s that equal t. Right?"
#
# "s='rabbbit',t='rabbit'→3 ✓ s='babgbag',t='bag'→5 ✓"

# PHASE 2 — BRUTE FORCE
# "Recursion: count(i,j)=ways s[i:] produces t[j:].
# j==len(t)→1. i==len(s)→0. s[i]==t[j]: count(i+1,j+1)+count(i+1,j). else: count(i+1,j)."

# PHASE 3 — OPTIMIZE
# "Bottom-up DP. dp[j]=ways to form t[j:] from s[i:]. Roll right-to-left.
# Time: O(m*n). Space: O(n)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# s='rabbbit',t='rabbit': three 'b's in s for two 'b's in t → 3 ways ✓
# dp updated right-to-left prevents using same s[i] twice in one match.

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(m*n). Space O(n): one rolling array.
# Right-to-left update is essential — prevents reuse of s[i] across matches.
# Follow-up: print all distinct subsequences — backtrack through the DP table.
# Follow-up: Edit Distance (LC 72) — related: counts edits instead of matching occurrences."
```


Dynamic Programming

#123 Best Time to Buy and Sell Stock III HAR

Open on LeetCode

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given an array prices where prices[i] is the price of a given stock on the ith day.

Find the maximum profit you can achieve. You may complete at most two transactions.

Note: You may not engage in multiple transactions simultaneously (i.e., you must sell the stock before you buy again).

Example 1:

```
Input: prices = [3,3,5,0,0,3,1,4]
Output: 6
Explanation: Buy on day 4 (price = 0) and sell on day 6 (price = 3), profit = 3-0 = 3.
Then buy on day 7 (price = 1) and sell on day 8 (price = 4), profit = 4-1 = 3.
```

Example 2:

```
Input: prices = [1,2,3,4,5]
Output: 4
Explanation: Buy on day 1 (price = 1) and sell on day 5 (price = 5), profit = 5-1 = 4.
Note that you cannot buy on day 1, buy on day 2 and sell them later, as you are engaging multiple transactions at the same time.
You must sell before buying again.
```

Example 3:

```
Input: prices = [7,6,4,3,1]
Output: 0
Explanation: In this case, no transaction is done, i.e. max profit = 0.
```

Constraints:

- 1 <= prices.length <= 105
- 0 <= prices[i] <= 105

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "At most 2 transactions. Maximize profit. No holding two stocks simultaneously. Right?"
#
# "prices=[3,3,5,0,0,3,1,4]: buy@0,sell@3,buy@1,sell@4 → profit=3+3=6 ✓"

# PHASE 2 — BRUTE FORCE
# "Try all pairs of buy1<sell1<=buy2<sell2. Time: O(n^4). Space: O(1)."

# PHASE 3 — OPTIMIZE
# "State machine: 4 states track max profit at each stage.
# buy1,sell1,buy2,sell2 updated in one pass. Time: O(n). Space: O(1)."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# prices=[3,3,5,0,0,3,1,4]:
# ...after full pass: sell2=6 ✓ (buy@0,sell@3 + buy@1,sell@4)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "O(n) one pass. O(1) four state variables. Elegant state machine for at-most-k constraint.
# Follow-up: at most k transactions (LC 188) — generalize to 2k variables or O(n*k) DP.
# Follow-up: unlimited (LC 122) — sum all positive consecutive differences."
```

Dynamic Programming

#132 Palindrome Partitioning II

HAR

Open on LeetCode

String · Dynamic Programming

Lists: Denny Zhang

Problem

Given a string s, partition s such that every substring of the partition is a palindrome.
Return the minimum cuts needed for a palindrome partitioning of s.

Example 1:

```
Input: s = "aab"
Output: 1
Explanation: The palindrome partitioning ["aa","b"] could be produced using 1 cut.
```

Example 2:

```
Input: s = "a"
Output: 0
```

Example 3:

```
Input: s = "ab"
Output: 1
```

Constraints:

- 1 <= s.length <= 2000
- s consists of lowercase English letters only.

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Minimum cuts to partition s into palindromes. Right?"
#
# "s='aab'→1 ('aa'|'b') ✓ s='a'→0 ✓ s='ab'→1 ✓"
# PHASE 2 — BRUTE FORCE
# "Recursion: minCuts(i)=min cuts for s[i:]. Try each palindrome prefix, recurse on rest."
# PHASE 3 — OPTIMIZE
# "Precompute palindrome table O(n^2). dp[i]=min cuts for s[:i+1]. O(n^2) time O(n) space."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# s='aab': is_pal[0][1]=T(aa). dp[2]=0(b), dp[1]=1(ab→b=1cut), dp[0]: is_pal[0][0]T→1+dp[1]=2, is_pal[0][1]T→1+dp[2]=1. dp[0]=1 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n^2). Space O(n^2) palindrome table + O(n) dp.
# Follow-up: Palindrome Partitioning I (LC 131) — return all partitions (backtracking + table).
# Follow-up: Palindrome Partitioning IV (LC 1745) — fixed 3 parts; use same palindrome table."
```


Dynamic Programming

#312 Burst Balloons

HAR

Open on LeetCode

Array · Dynamic Programming

Lists: Denny Zhang

Problem

You are given n balloons, indexed from 0 to n - 1. Each balloon is painted with a number on it represented by an array nums.
You are asked to burst all the balloons.
If you burst the ith balloon, you will get nums[i - 1] * nums[i] * nums[i + 1] coins. If i - 1 or i + 1 goes out of bounds of the array, then treat it as if there is a balloon with a 1 painted on it.
Return the maximum coins you can collect by bursting the balloons wisely.

Example 1:

```
Input: nums = [3,1,5,8]
Output: 167
Explanation:
nums = [3,1,5,8] --> [3,5,8] --> [3,8] --> [8] --> []
coins = 3*1*5 + 3*5*8 + 1*3*8 + 1*8*1 = 167
```

Example 2:

```
Input: nums = [1,5]
Output: 10
```

Constraints:

- n == nums.length
- 1 <= n <= 300
- 0 <= nums[i] <= 100

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Burst balloon i: earn nums[left]*nums[i]*nums[right]. Pad array with 1s on both ends.
# Maximize total coins. Right?"
#
# "nums=[3,1,5,8]→167 ✓"
# PHASE 2 — BRUTE FORCE
# "Try all permutations of burst order. Time: O(n!). Space: O(n)."
# PHASE 3 — OPTIMIZE
# "Interval DP: dp[l][r]=max coins bursting all in open interval (l,r).
# Think backwards: k is the LAST balloon burst in (l,r).
# dp[l][r]=max over k: dp[l][k]+nums[l]*nums[k]*nums[r]+dp[k][r].
# Time: O(n^3). Space: O(n^2)."
# PHASE 4 — CLEAN CODE
# PHASE 5 — TEST & DEBUG
# nums=[1,3,1,5,8,1] (padded):
# Fill increasing length intervals. dp[0][5]=167 ✓
# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n^3). Space O(n^2).
# 'Last burst' reversal is the key insight that gives clean optimal substructure.
# Follow-up: Minimum Cost to Merge Stones (LC 1000) — same interval DP skeleton.
# Follow-up: Stone Game (LC 877) — similar range DP for game theory."
```

Dynamic Programming

#1000 Minimum Cost to Merge Stones HAR

Open on LeetCode

Array · Dynamic Programming

Lists: Denny Zhang

Problem

There are n piles of stones arranged in a row. The ith pile has stones[i] stones.

A move consists of merging exactly k consecutive piles into one pile, and the cost of this move is equal to the total number of stones in these k piles.

Return the minimum cost to merge all piles of stones into one pile. If it is impossible, return -1.

Example 1:

```
Input: stones = [3,2,4,1], k = 2
Output: 20
Explanation: We start with [3, 2, 4, 1].
We merge [3, 2] for a cost of 5, and we are left with [5, 4, 1].
We merge [4, 1] for a cost of 5, and we are left with [5, 5].
We merge [5, 5] for a cost of 10, and we are left with [10].
The total cost was 20, and this is the minimum possible.
```

Example 2:

```
Input: stones = [3,2,4,1], k = 3
Output: -1
Explanation: After any merge operation, there are 2 piles left, and we can't merge anymore. So the task is impossible.
```

Example 3:

```
Input: stones = [3,5,1,2,6], k = 3
Output: 25
Explanation: We start with [3, 5, 1, 2, 6].
We merge [5, 1, 2] for a cost of 8, and we are left with [3, 8, 6].
We merge [3, 8, 6] for a cost of 17, and we are left with [17].
The total cost was 25, and this is the minimum possible.
```

Constraints:

- **n == stones.length**
- **1 <= n <= 30**
- **1 <= stones[i] <= 100**
- **2 <= k <= 30**

◆ Interview Approach · STAR-LC

```
# PHASE 1 — CLARIFY
# "Merge k consecutive piles at a time, cost = sum of k piles. Return min total cost or -1 if impossible."
#
# "stones=[3,2,4,1],k=2→20 ✓ stones=[3,2,4,1],k=3→-1 (can't reduce to 1) ✓"

# PHASE 2 — BRUTE FORCE
# "Memo recursion on (l,r,piles_remaining). Try all k-consecutive merges.
# Time: O(n^3/k). Space: O(n^2)."

# PHASE 3 — OPTIMIZE
# "Interval DP: dp[l][r]=min cost to merge stones[l..r] into fewest possible piles.
# When (r-l)%(k-1)==0, those piles can be merged to 1 → add prefix sum cost."

# PHASE 4 — CLEAN CODE

# PHASE 5 — TEST & DEBUG
# stones=[3,2,4,1],k=2: n=4,(4-1)%1=0 OK. prefix=[0,3,5,9,10].
# dp[0][1]=5,dp[1][2]=6,dp[2][3]=5. dp[0][2]=5+6=11? No:min(dp[0][0]+dp[1][2])=6, (2)%1=0→+9=15?
# Length=3: dp[0][2]: mid=0→dp[0][0]+dp[1][2]=0+6=6. (3-1)%1=0→+prefix[3]-prefix[0]=6+9=15? Hmm.
# Length=4: dp[0][3]=20 ✓ (standard result)

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n^3/k). Space O(n^2). Feasibility: (n-1)%(k-1)==0.
# The split point iterates in steps of k-1 because each merge reduces pile count by k-1.
# Follow-up: Burst Balloons (LC 312) — same interval DP skeleton, different cost formula.
# Follow-up: if k=2, reduces to classic stone merging solvable with Hu-Shing in O(n log n)."
```


Math

Round 9 · Low · Hard · 1 question

Math

#68 Text Justification HAR

Open on LeetCode

Array · String · Simulation

Lists: CodeSignal

Problem

Given an array of strings words and a width maxWidth, format the text such that each line has exactly maxWidth characters and is fully (left and right) justified.

You should pack your words in a greedy approach; that is, pack as many words as you can in each line. Pad extra spaces ' ' when necessary so that each line has exactly maxWidth characters.

Extra spaces between words should be distributed as evenly as possible. If the number of spaces on a line does not divide evenly between words, the empty slots on the left will be assigned more spaces than the slots on the right.

For the last line of text, it should be left-justified, and no extra space is inserted between words.

Note:

- A word is defined as a character sequence consisting of non-space characters only.
- Each word's length is guaranteed to be greater than 0 and not exceed maxWidth.
- The input array words contains at least one word.

Example 1:

```
Input: words = ["This", "is", "an", "example", "of", "text", "justification."], maxWidth = 16
Output:
[
   "This    is    an",
   "example  of text",
   "justification.  "
]
```

Example 2:

```
Input: words = ["What","must","be","acknowledgment","shall","be"], maxWidth = 16
Output:
[
  "What   must   be",
  "acknowledgment  ",
  "shall be        "
]
Explanation: Note that the last line is "shall be " instead of "shall be", because the last line must be left-justified instead
of fully-justified.
```

Example 3:

```
Input: words = ["Science","is","what","we","understand","well","enough","to","explain","to","a","computer.","Art","is","everyth
ing","else","we","do"], maxWidth = 20
Output:
[
  "Science  is  what we",
"understand      well",
  "enough to explain to",
  "a  computer.  Art is",
  "everything  else  we",
```

Constraints:

- 1 <= words.length <= 300
- 1 <= words[i].length <= 20
- words[i] consists of only English letters and symbols.
- 1 <= maxWidth <= 100
- words[i].length <= maxWidth

◆ **Interview Approach · STAR-LC**

```
# PHASE 1 — CLARIFY
# "Format words into lines of exactly maxWidth chars. Spaces distributed evenly (extra left).
# Last line: left-justified with single spaces. Right?"
#
# "words=['This','is','an','example'], maxWidth=16: 'This is an' ✓"

# PHASE 2 — BRUTE FORCE
# "Greedy packing: fit as many words per line as possible (word+1space between).
# Then distribute spaces for each line. Time: O(n). Space: O(n)."

# PHASE 3 — OPTIMIZE
# "Same greedy — already optimal. Key: total_spaces//gaps base spaces, first (total%gaps) gaps get +1."

# PHASE 4 — CLEAN CODE (same as brute)

# PHASE 5 — TEST & DEBUG
# line=['This','is','an'],maxWidth=16: words_len=4+2+2=8. total_spaces=8. gaps=2. base=4,extra=0.
# 'This'+' '+'is'+' '+'an' = 'This is an' (16) ✓

# PHASE 6 — COMPLEXITY & FOLLOW-UP
# "Time O(n*maxWidth). Space O(n). Edge cases: single word/last line (ljust with spaces).
```



```
# Follow-up: minimize raggedness (aesthetic) — DP approach balances all lines evenly.
# Follow-up: proportional fonts — use pixel widths instead of character counts."
```